1. Playwright Installation – Simple Explanation

1. What is Playwright?

Playwright is an automation testing framework used to test web applications automatically.

Instead of manually opening a website, clicking buttons, entering text, and checking results, Playwright performs these actions through automation scripts.

For example, imagine testing an Amazon website:

1. Open Amazon.
2. Search for a mobile phone.
3. Apply a price filter.
4. Select a product.
5. Add it to the cart.
6. Verify that the product is added successfully.

Playwright can automate this entire process.

Important features of Playwright

Feature	Simple explanation
Test Runner	Executes test cases
Assertions	Verifies expected results
Isolation	Keeps tests independent

Parallel execution	Runs multiple tests simultaneously
Browser support	Supports Chromium, Firefox and WebKit
Headless mode	Runs tests without displaying the browser
Headed mode	Runs tests with a visible browser
HTML Reporter	Generates test execution reports

2. Which browsers does Playwright support?

Google Chrome - YouTube

Chromium

Browser engine used by Chrome and other Chromium-based browsers.

Evolving the Firefox Brand - Mozilla Open Design

Firefox

Browser developed by Mozilla.

U.K. Court Blocks Mass Legal Action Over Google's Alleged Tracking of Safari Users - MacRumors

WebKit

Browser engine used by Safari.

Interview point: Playwright supports Chromium, Firefox and WebKit. It can execute tests across these browser engines.

Note: Chromium is the browser engine, not exactly the same thing as Google Chrome.

3. Installing Playwright

Before installing Playwright, we need:

- Node.js
- npm (comes with Node.js)
- VS Code (recommended code editor)

Step 1: Check Node.js installation

Open your VS Code terminal and execute:

```
node -v
```

This displays your installed Node.js version.

Check npm:

```
npm -v
```

Step 2: Create a Playwright project

Execute:

```
npm init playwright@latest
```

This is the main command used to initialize a new Playwright project.

What happens when you execute this command?

Playwright asks some questions:

Question	What should you select?
TypeScript or JavaScript?	TypeScript (recommended)
Tests folder name?	tests
Add GitHub Actions workflow?	Yes (recommended for CI)
Install Playwright browsers?	Yes

What is TypeScript?

TypeScript is JavaScript with additional features such as static typing.

Example:

JavaScript:

```
let age = 25;
```

TypeScript:

```
let age: number = 25;
```

For Playwright automation, both JavaScript and TypeScript are supported.

4. Understanding the installed project structure

After installation, Playwright creates a project structure.

PlaywrightProject/

playwright.config.ts

Central configuration file for Playwright.

package.json

Contains project details, dependencies and npm scripts.

package-lock.json

Locks exact dependency versions for consistent installation.

tests/

example.spec.ts

Contains the sample automated test case.

Important files explained

1. playwright.config.ts

This is the main configuration file.

We can configure:

- Browser selection.

- Timeout.
- Retries.
- Parallel execution.
- Test directory.
- Reporters.
- Screenshot and video settings.

Example:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30000,
  retries: 1,
  workers: 2,

  use: {
    headless: true,
  },
});
```

2. package.json

This file stores project dependencies and scripts.

Example:

```
{
  "scripts": {
    "test": "npx playwright test"
  },
  "devDependencies": {
    "@playwright/test": "^1.0.0"
  }
}
```

The version above is only an illustrative example.

3. package-lock.json

It stores the exact dependency versions installed in your project.

It helps developers install consistent dependency versions.

4. tests folder

This folder contains our automation test files.

Example:

```
tests/
  login.spec.ts
  search.spec.ts
  cart.spec.ts
```

5. example.spec.ts

This is the sample test file created during installation.

5. How to execute Playwright tests

The most important command:

```
npx playwright test
```

This executes all tests present in the configured test directory.

By default:

- Tests run in headless mode.
- Tests can run in parallel.
- Chromium, Firefox and WebKit projects can execute.
- Results are displayed in the terminal.

Important execution commands

Command	Purpose
`npx playwright test`	Run all tests

`npx playwright test --headed`	Run tests with visible browser
`npx playwright test --project=chromium`	Run tests only in Chromium project
`npx playwright test tests/login.spec.ts`	Run a particular test file
`npx playwright test --ui`	Open Playwright UI Mode

Difference between Headless and Headed mode

[Playwright 101: The Testing Revolution You've Been Waiting For | Stackademic]

Headless mode

Browser runs in the background without displaying its window.

[Playwright Features 2026: What Matters for QA Teams]

Headed mode

Browser window is visible while tests execute.

Example:

```
npx playwright test --headed
```

Real interview example:

Interviewer: How do you execute test cases in Playwright?

Answer: We use `npx playwright test` to execute all test cases. If we want to see browser execution, we use `npx playwright test --headed`. We can also execute a specific test file by providing its path.

6. HTML Test Reports

After test execution, Playwright generates an HTML report.

Execute:

```
npx playwright show-report
```

This opens the HTML report in the browser.

Running and debugging tests | Playwright
playwright.dev

What information does the HTML report contain?

- Total tests executed.
- Passed tests.
- Failed tests.
- Skipped tests.
- Flaky tests.
- Error details.
- Test execution steps.
- Attachments such as screenshots and traces, when configured or generated.

What is a flaky test?

A flaky test is a test that sometimes passes and sometimes fails without any relevant code change.

For example:

A login test passes in the first execution but fails in the second execution because of inconsistent timing or network behavior.

7. Playwright UI Mode

Command:

```
npx playwright test --ui
```

UI Mode provides a graphical interface for executing and debugging tests.

UI Mode | Playwright
playwright.dev

Important features:

1. Watch mode – Automatically detects changes and reruns tests.
2. Live step view – Shows each test execution step.
3. Time travel debugging – Helps inspect what happened during execution.
4. Test filtering – Allows you to select particular tests.
5. Trace integration – Helps investigate failures.

Interview point: UI Mode is mainly useful for interactive test execution and debugging.

8. Updating Playwright

Sometimes we need to update Playwright to the latest version.

Command:

```
npm install -D @playwright/test@latest
```

Here:

- `npm install` – Installs the package.
- `-D` – Saves it as a development dependency.
- `@playwright/test` – Playwright testing package.
- `@latest` – Installs the latest available version.

After updating, install the required browsers and dependencies:

```
npx playwright install --with-deps
```

Check the installed version:

```
npx playwright --version
```

Remember: Updating the Playwright npm package and updating browser binaries are related but separate operations.

9. System requirements

According to the documentation you shared:

Requirement	Supported versions
Node.js	Latest 22.x, 24.x or 26.x
Windows	Windows 11+, Server 2019+
macOS	macOS 14+
Linux	Supported Debian and Ubuntu versions

Node.js is required because Playwright's JavaScript/TypeScript test runner executes using Node.js.

10. Important interview questions and answers

Interview preparation
Beginner to intermediate

These are the questions you should prepare from this installation topic.

Q1. What is Playwright?

Answer:

Playwright is an open-source end-to-end automation testing framework developed by Microsoft. It is used to automate web applications across Chromium, Firefox and WebKit browsers.

It supports JavaScript, TypeScript, Python, Java and .NET.

Q2. How do you install Playwright?

Answer:

First, we install Node.js and npm. Then we open the terminal in VS Code and execute:

`npm init playwright@latest`

It initializes a Playwright project, installs the required dependencies, creates configuration files and allows us to install browsers.

Q3. What is the difference between `npm init playwright@latest` and `npm install -D @playwright/test`?

Answer:

- `npm init playwright@latest`: Initializes a new Playwright project with configuration and sample test files.
- `npm install -D @playwright/test`: Installs Playwright Test as a development dependency in an existing project.

Q4. What is the purpose of playwright.config.ts?

Answer:

It is the central configuration file of Playwright.

We configure test directory, browser projects, timeout, retries, parallel execution, reporters, base URL and other execution settings in this file.

Q5. What is the purpose of package.json?

Answer:

It stores project information, dependencies, development dependencies and npm scripts.

It helps manage the packages and commands required for our automation project.

Q6. What is package-lock.json?

Answer:

It maintains the exact dependency versions installed in the project.

It helps ensure consistent dependency installation across different machines and environments.

Q7. Which browsers does Playwright support?

Answer:

Playwright supports three main browser engines:

1. Chromium
2. Firefox
3. WebKit

We can configure these browsers as separate projects in the Playwright configuration file.

Q8. What is the difference between headed and headless execution?

Answer:

Headed execution runs tests with a visible browser window, whereas headless execution runs tests in the background without displaying the browser window.

Headless mode is commonly used in CI/CD pipelines because it does not require a visible browser interface.

Q9. How do you execute all test cases?

Answer:

```
npx playwright test
```

This command executes all tests matching the configured test file patterns.

Q10. How do you execute a specific test file?

Answer:

```
npx playwright test tests/login.spec.ts
```

This executes only the specified test file.

Q11. How do you execute tests only in Chromium?

Answer:

```
npx playwright test --project=chromium
```

The Chromium project must be configured in `playwright.config.ts`.

Q12. How do you generate and open an HTML report?

Answer:

Playwright's HTML reporter generates a test report after execution.

To open it:

```
npx playwright show-report
```

Q13. What is Playwright UI Mode?

Answer:

UI Mode is a graphical interface used to execute, monitor and debug Playwright tests interactively.

We launch it using:

```
npx playwright test --ui
```

Q14. What is a flaky test?

Answer:

A flaky test is a test that produces inconsistent results. It may pass in one execution and fail in another without any relevant application code change.

Common reasons include timing issues, unstable test data, network problems and improper synchronization.

Q15. How do you update Playwright?

Answer:

We execute:

```
npm install -D @playwright/test@latest
```

Then install the required browser binaries:

```
npx playwright install
```

We can verify the installed version using:

```
npx playwright --version
```

Q16. What is the difference between Playwright and Selenium?

Answer:

Both are browser automation tools, but they differ in architecture and features.

Selenium	Playwright
Supports multiple languages	Supports multiple languages
Uses WebDriver-based automation	Uses browser automation protocols
Supports multiple browsers	Supports Chromium, Firefox, WebKit
Explicit waits are commonly used	Provides auto-waiting
Grid supports distributed execution	Supports built-in parallel execution
Third-party reporting integrations	Built-in HTML reporter

Q17. What is the purpose of npx?

Answer:

npx is a command-line tool that comes with npm. It allows us to execute package binaries without manually specifying their installation path.

For example:

```
npx playwright test
```

It executes the Playwright test command available in the project.

Q18. What is the difference between `npm` and `npx`?

Answer:

- npm is primarily used to install, uninstall and manage packages.
- npx is used to execute package commands.

Example:

```
npm install -D @playwright/test
```

Installs Playwright.

```
npx playwright test
```

Executes Playwright tests.

11. Practical commands – Quick revision

Keep this table for your interview preparation.

Task	Command
Check Node version	`node -v`
Check npm version	`npm -v`
Initialize project	`npm init playwright@latest`
Run all tests	`npx playwright test`
Run headed	`npx playwright test --headed`
Run Chromium	`npx playwright test --project=chromium`

Run specific file	`npx playwright test tests/example.spec.ts`
Open UI Mode	`npx playwright test --ui`
Open HTML report	`npx playwright show-report`
Update Playwright	`npm install -D @playwright/test@latest`
Install browsers	`npx playwright install`
Check version	`npx playwright --version`

Playwright – Writing Tests (Complete Notes)

Important for QA Automation Interviews
Beginner to Intermediate

This is one of the most important Playwright topics. In interviews, you may be asked to write a test from scratch, explain locators, perform browser actions, write assertions, explain test isolation, and use hooks.

Let's understand everything step by step in simple language.

1. Introduction – What does writing tests mean?

In Playwright, writing tests means creating automation scripts that:

1. Open a website.
2. Find elements such as buttons, links and text boxes.

3. Perform actions such as clicking and typing.
4. Verify whether the application behaves as expected.

Example: Login functionality

Imagine we want to test a login page.

「ログインできない」問題を減らすには？ ユーザーの離脱を防ぐ実践的なアプローチ
cs-studio.adish.co.jp

Our test steps:

1. Open the login page.
2. Enter username.
3. Enter password.
4. Click the Login button.
5. Verify that the dashboard is displayed.

This is called an automated test case.

Two important concepts

Actions: What we perform on the application.

Examples: Click, fill, hover, select.

Assertions: What we verify after performing actions.

Examples: Check whether the page title is correct, button is visible, or login is successful.

Remember:

- Action = Perform something.
- Assertion = Verify something.

2. First Playwright Test

Let's understand the code provided in your documentation.

File name: `tests/example.spec.ts`

```
import { test, expect } from '@playwright/test';

test('has title', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  // Verify page title
  await expect(page).toHaveTitle(/Playwright/);
});

test('get started link', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  // Click Get Started link
  await page.getByRole('link', { name: 'Get started' }).click();

  // Verify Installation heading
  await expect(
    page.getByRole('heading', { name: 'Installation' })
  ).toBeVisible();
});
```

Let's understand every line.

Line-by-line explanation

1. Import statement

```
import { test, expect } from '@playwright/test';
```

We import two important functions from the Playwright Test package.

Function	Purpose

`test`	Used to create and execute test cases
`expect`	Used to verify expected results

2. Writing a test

```
test('has title', async ({ page }) => {
```

- `test` – Defines a test case.
- `'has title'` – Test case name.
- `async` – Allows us to use asynchronous operations.
- `{ page }` – Page fixture provided by Playwright.

What is a page?

A page represents a browser tab.

For example, when you open Chrome and visit Amazon, the Amazon tab is a page.

3. Navigate to website

```
await page.goto('https://playwright.dev/');
```

This opens the specified website.

4. Verify the title

```
await expect(page).toHaveTitle(/Playwright/);
```

This checks whether the page title contains the word `Playwright`.

Here:

- `expect()` – Starts the assertion.
- `page` – The browser page.
- `toHaveTitle()` – Checks the page title.
- `/Playwright/` – Regular expression that matches the word Playwright.

5. Click a link

```
await page.getByRole('link', { name: 'Get started' }).click();
```

This finds a link with the accessible name `Get started` and clicks it.

6. Verify heading visibility

```
await expect(
  page.getByRole('heading', { name: 'Installation' })
).toBeVisible();
```

This checks whether the Installation heading is visible on the page.

Execution flow of the first test

Open Playwright website
Verify page title
Find Get Started link
Click the link
Verify Installation heading
Test passes if all assertions succeed

Important: What is `await`?

`await` is used to wait for an asynchronous operation to complete before executing the next statement.

For example:

```
await page.goto('https://playwright.dev/');

await page.getByRole('link', { name: 'Get started' }).click();
```

First, navigation completes. Then the click operation is performed.

Interview point: Most Playwright browser interactions and web-first assertions are asynchronous, so we use `await`.

JavaScript users – @ts-check

If you write JavaScript instead of TypeScript in VS Code, add:

```
// @ts-check
```

at the beginning of the test file.

It enables additional type checking and helps identify certain coding mistakes.

3. Actions in Playwright

Actions are operations that we perform on website elements.

For example:

- Clicking a Login button.
- Entering username and password.
- Selecting a dropdown option.
- Uploading a document.
- Hovering over a menu.

There are three important concepts to understand:

1. Navigation
2. Locators
3. Interactions

3.1 Navigation

Navigation means opening a website or moving to a particular URL.

Example:

```
await page.goto('https://www.amazon.in/');
```

This opens Amazon India.

How does Playwright handle navigation?

Playwright waits for the page to reach its configured load state before continuing.

By default, `page.goto()` waits for the `load` event.

Important navigation methods:

Method	Purpose
`page.goto()`	Open a URL
`page.goBack()`	Navigate to the previous page
`page.goForward()`	Navigate to the next page
`page.reload()`	Refresh the current page
`page.url()`	Get the current page URL

Example:

```
await page.goto('https://www.google.com/');

await page.reload();

console.log(page.url());
```

3.2 What is a Locator?

Very important interview question

A locator is used to find elements on a web page.

For example, suppose a login page contains:

- Username textbox
- Password textbox
- Login button

We need locators to identify these elements before interacting with them.

Playwright provides different locator methods.

Important locator types

Locator	Example
`getByRole()`	Find button, link, heading
`getByText()`	Find text
`getByLabel()`	Find form field using label
`getByPlaceholder()`	Find input using placeholder
`getByTestId()`	Find element using test ID
`locator()`	Find elements using CSS or XPath

Examples:

1. getByRole()

```
await page.getByRole('button', { name: 'Login' }).click();
```

Finds a button named Login.

2. getByText()

```
await page.getByText('Welcome').click();
```

Finds an element containing the text Welcome.

3. getByLabel()

```
await page.getByLabel('Username').fill('Komal');
```

Finds the form field associated with the Username label.

4. getByPlaceholder()

```
await page.getByPlaceholder('Enter email').fill('test@gmail.com');
```

Finds an input field using its placeholder.

5. getByTestId()

```
await page.getByTestId('login-button').click();
```

Finds an element using its test ID.

6. locator()

```
await page.locator('#username').fill('Komal');
```

Finds an element using a CSS ID selector.

Why are locators important?

Without a locator, Playwright does not know which element we want to interact with.

The basic process is:

Web page

Identify element using locator

Perform action

Verify result using assertion

3.3 What is auto-waiting?

This is one of Playwright's most important features.

Suppose you click a Login button.

Sometimes the button takes 2 seconds to load.

In Selenium, we commonly write an explicit wait to wait for the element to become clickable.

In Playwright:

```
await page.getByRole('button', { name: 'Login' }).click();
```

Playwright automatically checks whether the button is ready for clicking.

For a click action, important actionability checks include:

- Element is visible.

- Element is stable.
- Element receives pointer events.
- Element is enabled.

Only after the required checks pass does Playwright perform the action.

What is the benefit?

We don't need to write manual waits for every normal interaction.

Important: Auto-waiting does not mean Playwright waits indefinitely. Actions have timeouts and can fail if the element never becomes actionable.

4. Basic Actions in Playwright

Let's understand each action with practical examples.

4.1 click()

Used to click an element.

```
await page.getByRole('button', { name: 'Login' }).click();
```

Example: Clicking the Login button.

4.2 fill()

Used to enter text into an input field. It replaces the existing value.

```
await page.getByPlaceholder('Enter username').fill('Komal');
```

Example: Entering a username.

4.3 check()

Used to select a checkbox.

```
await page.getByLabel('Remember me').check();
```

Example: Selecting the Remember Me checkbox.

4.4 uncheck()

Used to deselect a checkbox.

```
await page.getByLabel('Remember me').uncheck();
```

4.5 hover()

Used to move the mouse over an element.

```
await page.getByText('Products').hover();
```

Example: Hovering over Products to display a submenu.

4.6 focus()

Used to focus on an element.

```
await page.getByLabel('Username').focus();
```

Example: Placing the cursor inside the username field.

4.7 press()

Used to press a keyboard key.

```
await page.getByLabel('Search').press('Enter');
```

Example: Pressing Enter after entering a search term.

Other examples:

```
await page.keyboard.press('Escape');

await page.keyboard.press('Control+A');
```

4.8 setInputFiles()

Used to upload files.

```
await page.locator('#upload').setInputFiles('files/resume.pdf');
```

Example: Uploading a resume.

4.9 selectOption()

Used to select an option from a dropdown.

```
await page.locator('#country').selectOption('India');
```

Example: Selecting India from the country dropdown.

Note: `selectOption()` is for HTML `select` elements. Custom dropdowns may require clicking and selecting an option separately.

Quick revision of actions

Action	Meaning
`click()`	Click element
`fill()`	Enter or replace text
`check()`	Select checkbox
`uncheck()`	Deselect checkbox
`hover()`	Move mouse over element
`focus()`	Focus element
`press()`	Press keyboard key
`setInputFiles()`	Upload file
`selectOption()`	Select dropdown option

5. Assertions in Playwright

Very important for interviews

5.1 What is an assertion?

An assertion is used to verify whether the actual result matches the expected result.

For example:

You are testing a login page.

Expected result: After clicking Login, the dashboard should appear.

Actual result: Dashboard appears.

If the actual result matches the expected result, the assertion passes.

If it does not match, the assertion fails.

Syntax

```
await expect(locator).toBeVisible();
```

Here:

- `expect()` – Checks the expected condition.
- `locator` – Element we want to verify.
- `toBeVisible()` – Checks whether the element is visible.
- `await` – Waits for the assertion to pass or time out.

5.2 Important Playwright assertions

Let's understand each assertion with examples.

1. toBeVisible()

Checks whether an element is visible.

```
await expect(
  page.getByRole('button', { name: 'Login' })
).toBeVisible();
```

2. toBeEnabled()

Checks whether an element is enabled.

```
await expect(
  page.getByRole('button', { name: 'Submit' })
).toBeEnabled();
```

3. toBeChecked()

Checks whether a checkbox or radio button is selected.

```
await expect(
  page.getByLabel('Remember me')
).toBeChecked();
```

4. toHaveText()

Checks whether an element has the expected text.

```
await expect(
  page.locator('.message')
).toHaveText('Login successful');
```

5. toContainText()

Checks whether an element contains particular text.

```
await expect(
  page.locator('.message')
).toContainText('successful');
```

Difference:

- `toHaveText()` – Checks the expected text.
- `toContainText()` – Checks whether the expected text is present within the element's text.

6. toHaveValue()

Checks the value inside an input field.

```
await expect(
  page.getByLabel('Username')
).toHaveValue('Komal');
```

7. toHaveAttribute()

Checks whether an element has a particular attribute and value.

```
await expect(
  page.locator('#username')
).toHaveAttribute('type', 'text');
```

8. toHaveCount()

Checks the number of matching elements.

```
await expect(
  page.locator('.product')
).toHaveCount(5);
```

This verifies that five product elements are present.

9. toHaveTitle()

Checks the browser page title.

```
await expect(page).toHaveTitle('Amazon.in');
```

10. toHaveURL()

Checks the current page URL.

```
await expect(page).toHaveURL('https://www.amazon.in/');
```

Assertion revision table

Assertion	What does it verify?
`toBeVisible()`	Element is visible
`toBeHidden()`	Element is hidden
`toBeEnabled()`	Element is enabled
`toBeDisabled()`	Element is disabled
`toBeChecked()`	Checkbox is checked

`toHaveText()`	Expected text
`toContainText()`	Text is contained
`toHaveValue()`	Input value
`toHaveAttribute()`	Attribute and its value
`toHaveCount()`	Number of matching elements
`toHaveTitle()`	Page title
`toHaveURL()`	Current URL

5.3 What is auto-retrying assertion?

This is an important Playwright concept.

Suppose you click the Login button.

The dashboard takes 3 seconds to appear.

If you use a normal web-first assertion:

```
await expect(
  page.getByText('Dashboard')
).toBeVisible();
```

Playwright keeps checking whether Dashboard becomes visible until the assertion passes or its timeout expires.

You don't need to add a manual wait for those 3 seconds.

This is called auto-retrying assertion.

Difference between actions and assertions

Actions	Assertions
Perform operations	Verify expected results
`click()`	`toBeVisible()`
`fill()`	`toHaveValue()`
`check()`	`toBeChecked()`
`selectOption()`	`toHaveValue()`

5.4 Generic assertions

Playwright also supports generic assertions.

Example:

```
const success = true;

expect(success).toBeTruthy();
```

Here, `toBeTruthy()` checks whether the value is truthy.

Other examples:

```
expect(10).toBe(10);

expect('Playwright').toContain('Play');

expect([1, 2, 3]).toEqual([1, 2, 3]);
```

Important difference:

Web-first assertions	Generic assertions

Usually used with locators and pages	Used with JavaScript values
Automatically retry	Do not automatically retry
Generally use `await`	Do not use `await`
Example: `toBeVisible()`	Example: `toBeTruthy()`

Remember this difference for interviews.

6. Test Isolation in Playwright

Frequently asked interview question

6.1 What is test isolation?

Test isolation means every test runs independently without depending on another test's execution or result.

Playwright provides a fresh Browser Context for each test by default.

A Browser Context works like a separate browser profile.

For example:

Suppose we have two test cases.

Test 1: Login test

1. Open application.
2. Enter username and password.
3. Login successfully.
4. Verify dashboard.

Test 2: Registration test

1. Open application.

2. Navigate to registration page.
3. Enter registration details.
4. Verify registration.

Test 2 should not depend on Test 1.

Even if Test 1 fails, Test 2 should be able to execute independently.

6.2 What is Browser Context?

A Browser Context is an isolated browser environment.

It maintains browser-related information such as:

- Cookies.
- Local storage.
- Session storage.
- Permissions.
- Browser settings.

Playwright creates a separate context for each test by default.

Browser isolation example

Browser

One browser process can be shared

Test 1

Context 1

Cookies A

Session A

Page 1

Test 2

Context 2

Cookies B

Session B

Page 2

Each test has its own isolated browser context and page.

Example from documentation

```
import { test } from '@playwright/test';

test('example test', async ({ page }) => {
  // This test has its own isolated page and context.
});

test('another test', async ({ page }) => {
  // This test receives another isolated page and context.
});
```

Both tests receive separate page fixtures.

Why is test isolation important?

1. Prevents test dependency.
2. Avoids sharing cookies and session data accidentally.
3. Allows tests to run in parallel safely.
4. Makes debugging easier.
5. Improves test reliability.

Interview example

Interviewer: If one test logs into an application, will the next test automatically remain logged in?

Answer:

No. By default, Playwright creates a new Browser Context for each test. Therefore, the next test starts with a separate browser session and does not automatically inherit the previous test's cookies or login state.

If we want to reuse login state, we can explicitly configure authentication storage state.

7. Test Hooks in Playwright

Important for framework development

7.1 What are test hooks?

Test hooks are special functions that execute at specific stages of test execution.

They help us perform common setup and cleanup operations.

For example:

Suppose we have 10 test cases.

Before each test, we need to open the application.

Instead of writing the same navigation code in all 10 test cases, we can use `beforeEach()`.

Similarly, we can use `afterEach()` to perform cleanup after each test.

7.2 Types of test hooks

Playwright provides four important hooks.

Hook	When does it execute?
`test.beforeEach()`	Before every test
`test.afterEach()`	After every test
`test.beforeAll()`	Once before all tests in a worker

`test.afterAll()`	Once after all tests in a worker

Let's understand each one.

1. beforeEach()

Executes before every test in the group.

Example:

```
import { test, expect } from '@playwright/test';

test.beforeEach(async ({ page }) => {
  await page.goto('https://playwright.dev/');
});

test('test 1', async ({ page }) => {
  await expect(page).toHaveURL('https://playwright.dev/');
});

test('test 2', async ({ page }) => {
  await expect(page).toHaveTitle(/Playwright/);
});
```

Execution:

- Before test 1 → Website opens.
- Test 1 executes.
- Before test 2 → Website opens again.
- Test 2 executes.

2. afterEach()

Executes after every test.

Example:

```
test.afterEach(async ({ page }) => {
  console.log('Test execution completed');
});
```

Useful for:

- Logging test completion.
- Performing cleanup.
- Capturing additional diagnostic information.

3. beforeAll()

Executes once before all tests in a worker.

Example:

```
test.beforeAll(async () => {
  console.log('Starting test execution');
});
```

Useful for common setup that does not need to be repeated before every test.

Important: `beforeAll()` runs once per worker, not necessarily once for the entire test suite if multiple workers are used.

4. afterAll()

Executes once after all tests in a worker.

Example:

```
test.afterAll(async () => {
  console.log('All tests completed');
});
```

Useful for cleanup tasks such as releasing shared resources.

7.3 What is test.describe()?

`test.describe()` is used to group related test cases.

For example, we can group all login-related tests inside one describe block.

```
import { test, expect } from '@playwright/test';

test.describe('Login functionality', () => {
```

```
  test.beforeEach(async ({ page }) => {
    await page.goto('https://example.com/login');
  });

  test('valid login', async ({ page }) => {
    // Login test steps
  });

  test('invalid login', async ({ page }) => {
    // Invalid login test steps
  });

});
```

Here:

- `test.describe()` groups login test cases.
- `beforeEach()` opens the login page before every test.
- Both tests execute independently.

7.4 Understanding hook execution order

For two tests inside one describe block:

```
test.describe('Login Tests', () => {

  test.beforeAll(async () => {
    console.log('Before All');
  });

  test.beforeEach(async () => {
    console.log('Before Each');
  });

  test('Test 1', async ({ page }) => {
    console.log('Executing Test 1');
  });

  test('Test 2', async ({ page }) => {
    console.log('Executing Test 2');
  });
```

```
  test.afterEach(async () => {
    console.log('After Each');
  });

  test.afterAll(async () => {
    console.log('After All');
  });

});
```

Expected execution order in one worker:

1. Before All
2. Before Each
3. Test 1
4. After Each
5. Before Each
6. Test 2
7. After Each
8. After All

Note: This is the basic execution order when both tests are in the same worker and no retries or special execution settings alter the flow.

Difference between beforeEach() and beforeAll()

beforeEach()	**beforeAll()**
Executes before every test	Executes once per worker
Used for repeated setup	Used for shared setup

Receives test fixtures such as page	Cannot use the ordinary per-test page fixture
Example: Navigate to login page	Example: Start a shared test service

Important interview point: The `page` fixture is test-scoped. You cannot directly use it inside `beforeAll()`. If a shared browser page is genuinely needed in `beforeAll()`, create and manage a browser context and page manually, and close them during cleanup.

8. Complete practical example – Login automation

Now let's combine everything you have learned.

Suppose we want to automate login functionality.

Test scenario:

1. Open login page.
2. Enter username.
3. Enter password.
4. Click Login.
5. Verify dashboard is visible.

Example code:

```
import { test, expect } from '@playwright/test';

test.describe('Login functionality', () => {

  test.beforeEach(async ({ page }) => {
    await page.goto('https://example.com/login');
  });

  test('Verify successful login', async ({ page }) => {
```

```
    await page.getByLabel('Username').fill('testuser');

    await page.getByLabel('Password').fill('password123');

    await page.getByRole('button', { name: 'Login' }).click();

    await expect(
      page.getByRole('heading', { name: 'Dashboard' })
    ).toBeVisible();

  });

});
```

Concepts used in this example

Test creation

`test()`

Test grouping

`test.describe()`

Pre-test setup

`beforeEach()`

Navigation

`page.goto()`

Locators

`getByLabel()`, `getByRole()`

Actions

`fill()`, `click()`

Assertion

`toBeVisible()`

This is the basic structure you should understand before moving towards Page Object Model (POM) and framework development.

9. Important Playwright Interview Questions and Answers

These questions are especially useful for QA Automation interviews requiring 1–2 years of experience.

A. Basic test-writing questions

Q1. How do you write a test case in Playwright?

Answer:

We use the `test()` function provided by `@playwright/test` to write a test case.

It accepts a test name and an asynchronous callback function.

Example:

```
import { test, expect } from '@playwright/test';

test('Verify page title', async ({ page }) => {
  await page.goto('https://playwright.dev/');
  await expect(page).toHaveTitle(/Playwright/);
});
```

Q2. What is the difference between `test` and `expect`?

Answer:

- `test` is used to define and execute test cases.
- `expect` is used to verify actual results against expected results.

Example:

```
test('Verify login', async ({ page }) => {
  await page.goto('https://example.com');
```

```
  await expect(page).toHaveTitle(/Example/);
});
```

Q3. What is a page fixture in Playwright?

Answer:

A page fixture provides a browser page (tab) to the test.

Playwright automatically creates the required browser context and page for the test and manages their lifecycle.

Example:

```
test('My test', async ({ page }) => {
  await page.goto('https://google.com');
});
```

Q4. Why do we use async and await in Playwright?

Answer:

Browser operations are asynchronous. We use `async` to define asynchronous functions and `await` to wait for an asynchronous operation to complete before moving to the next statement.

Q5. What is a `.spec.ts` file?

Answer:

It is a TypeScript test specification file where we write automated test cases.

For example:

- `login.spec.ts`
- `registration.spec.ts`
- `search.spec.ts`

Playwright's default test matching pattern includes files ending with `.spec.ts` and `.test.ts`.

B. Locator and action questions

Q6. What is a locator in Playwright?

Answer:

A locator is used to identify web elements such as buttons, text boxes, links and dropdowns.

Playwright provides methods such as `getByRole()`, `getByText()`, `getByLabel()`, `getByPlaceholder()`, `getByTestId()` and `locator()`.

Q7. What is the difference between `getByRole()` and `locator()`?

Answer:

`getByRole()` identifies elements based on their accessible role and name.

`locator()` identifies elements using CSS selectors, XPath or other supported locator selectors.

Example:

```
await page.getByRole('button', { name: 'Login' }).click();

await page.locator('#login').click();
```

I generally prefer user-facing locators such as `getByRole()` when they are available and reliable.

Q8. What is auto-waiting in Playwright?

Answer:

Auto-waiting means Playwright automatically checks whether an element is ready before performing an action.

For example, before clicking a button, Playwright checks whether it is visible, stable, enabled and receiving pointer events.

This reduces the need for manual waits and helps prevent flaky tests.

Q9. What is the difference between `fill()` and `press()`?

Answer:

- `fill()` enters or replaces text inside an input field.
- `press()` simulates pressing a keyboard key.

Example:

```
await page.getByLabel('Search').fill('Laptop');

await page.getByLabel('Search').press('Enter');
```

Q10. What is the difference between `check()` and `click()`?

Answer:

`check()` is specifically used to select a checkbox or radio button.

`click()` performs a click action on an element.

The advantage of `check()` is that it ensures the checkbox is checked. If it is already checked, it does not unnecessarily toggle it off.

Q11. How do you upload a file in Playwright?

Answer:

We use `setInputFiles()`.

```
await page.locator('#upload').setInputFiles('resume.pdf');
```

It is used to upload files through file input elements.

Q12. How do you handle dropdowns in Playwright?

Answer:

For standard HTML select dropdowns, we use `selectOption()`.

```
await page.locator('#country').selectOption('India');
```

For custom dropdowns, we locate the dropdown, click it and select the required option.

C. Assertion questions

Q13. What is an assertion in Playwright?

Answer:

Assertions verify whether the actual application behavior matches the expected result.

For example:

```
await expect(
  page.getByText('Login successful')
).toBeVisible();
```

This verifies whether the success message is visible.

Q14. What are web-first assertions?

Answer:

Web-first assertions automatically retry until the expected condition is satisfied or the timeout is reached.

For example:

```
await expect(page.getByText('Welcome')).toBeVisible();
```

Playwright keeps checking the element's visibility instead of checking only once.

Q15. What is the difference between `toHaveText()` and `toContainText()`?

Answer:

`toHaveText()` checks whether the element's text matches the expected text.

`toContainText()` checks whether the element contains the specified text.

Example:

```
await expect(page.locator('.message'))
  .toHaveText('Login successful');

await expect(page.locator('.message'))
  .toContainText('successful');
```

Q16. What is the difference between `toBeTruthy()` and `toBeVisible()`?

Answer:

`toBeTruthy()` is a generic assertion used to check whether a JavaScript value is truthy.

`toBeVisible()` is a web-first assertion used to check whether a web element is visible.

Example:

```
expect(true).toBeTruthy();

await expect(page.getByText('Welcome')).toBeVisible();
```

D. Test isolation and hooks questions

Q17. What is test isolation in Playwright?

Answer:

Test isolation means each test executes in an independent browser environment.

Playwright creates a separate Browser Context and Page for every test by default.

This prevents one test's cookies, session and other browser state from affecting another test.

Q18. What is a Browser Context?

Answer:

A Browser Context is an isolated browser session similar to a separate browser profile.

It maintains its own cookies, local storage, session storage and other browser-related information.

Multiple browser contexts can run inside the same browser instance.

Q19. What are hooks in Playwright?

Answer:

Hooks are special functions used to execute setup and cleanup operations at different stages of test execution.

Playwright provides:

- `beforeEach()`
- `afterEach()`
- `beforeAll()`
- `afterAll()`

Q20. What is the difference between beforeEach and beforeAll?

Answer:

`beforeEach()` executes before every test case.

`beforeAll()` executes once before all tests in a worker.

For example, if there are five tests:

- `beforeEach()` executes five times.
- `beforeAll()` executes once per worker.

Q21. What is test.describe()?

Answer:

`test.describe()` is used to group related test cases.

For example, we can group login-related tests inside a Login functionality block.

It also allows us to define hooks for that group.

Q22. Can we use the page fixture inside beforeAll()?

Answer:

No, the normal `page` fixture is test-scoped, whereas `beforeAll()` is a worker-scoped hook.

If we need a page in `beforeAll()`, we must create a browser context and page manually using the browser fixture and close them during cleanup.

Q23. Why should we avoid dependent test cases?

Answer:

Dependent test cases can cause execution failures when one test fails.

For example, if the registration test depends on the login test, failure in login may affect registration testing.

Independent tests are easier to execute in parallel, debug and maintain.

Q24. How does Playwright reduce flaky tests?

Answer:

Playwright reduces flaky tests through:

1. Automatic actionability checks.
2. Auto-retrying web-first assertions.
3. Browser context isolation.
4. Automatic waiting for supported actions.
5. Reliable locator strategies.

However, flaky tests can still occur due to unstable test data, application issues, external services and poor test design.

10. Practical interview scenarios

These are questions where interviewers expect you to write code rather than simply explain definitions.

Scenario 1: Open Google and verify the title.

```
test('Verify Google title', async ({ page }) => {

  await page.goto('https://www.google.com/');

  await expect(page).toHaveTitle(/Google/);

});
```

Scenario 2: Enter text into a search box and press Enter.

```
test('Google search', async ({ page }) => {

  await page.goto('https://www.google.com/');

  await page.getByRole('combobox', { name: /search/i })
    .fill('Playwright automation');

  await page.keyboard.press('Enter');

  await expect(page).toHaveURL(/search/);
```

```
});
```

Scenario 3: Verify that a checkbox is checked.

```
await page.getByLabel('Remember me').check();

await expect(
  page.getByLabel('Remember me')
).toBeChecked();
```

Scenario 4: Verify that five products are displayed.

```
await expect(
  page.locator('.product')
).toHaveCount(5);
```

Scenario 5: Verify successful login.

```
test('Login test', async ({ page }) => {

  await page.goto('https://example.com/login');

  await page.getByLabel('Username').fill('admin');

  await page.getByLabel('Password').fill('admin123');

  await page.getByRole('button', { name: 'Login' }).click();

  await expect(
    page.getByText('Welcome')
  ).toBeVisible();

});
```

These are illustrative examples. Actual locators and expected results depend on the application's HTML and behavior.

11. Quick revision notes

Before an interview, revise these concepts:

Topic	Remember
`test()`	Creates a test case
`expect()`	Performs assertions
`async/await`	Handles asynchronous operations
`page`	Browser tab
`page.goto()`	Navigates to URL
Locator	Identifies web elements
Auto-waiting	Waits for actionability
Web-first assertion	Automatically retries
`test.describe()`	Groups test cases
`beforeEach()`	Runs before every test
`afterEach()`	Runs after every test
`beforeAll()`	Runs once per worker before tests
`afterAll()`	Runs once per worker after tests
Browser Context	Isolated browser session
Test isolation	Tests execute independently

Playwright – Generating Tests (Codegen)

Important for QA Automation Interviews
Beginner to Intermediate

This topic is about automatically generating Playwright test scripts by performing actions on a website instead of writing all the code manually.

It is especially useful when you are learning Playwright, creating test scripts quickly, or identifying locators for web elements.

1. What is Codegen in Playwright?

Codegen stands for Code Generator.

It is a built-in Playwright tool that automatically generates automation code based on the actions you perform in a browser.

For example, suppose you want to automate a login page.

Normally, you would write code manually:

```
await page.goto('https://example.com/login');

await page.getByLabel('Username').fill('Komal');

await page.getByLabel('Password').fill('password123');

await page.getByRole('button', { name: 'Login' }).click();
```

But using Codegen:

1. Open Codegen.
2. Enter the website URL.
3. Browser opens automatically.
4. Enter username and password.
5. Click Login.
6. Playwright automatically generates the corresponding code.

You don't have to write each action manually.

[Generating tests | Playwright Python
playwright.dev]

Important components of Codegen

Codegen mainly provides two windows:

Component	Purpose
Browser window	Where you perform actions on the website
Playwright Inspector	Records actions, generates code and helps select locators

Remember: Browser is where you interact with the application. Inspector is where you view and manage generated code.

2. How to run Codegen

The command used to start the Playwright test generator is:

```
npx playwright codegen demo.playwright.dev/todomvc
```

Let's understand this command.

Command	Meaning
`npx`	Executes the Playwright command
`playwright`	Playwright CLI
`codegen`	Starts the test generator
`demo.playwright.dev/todomvc`	Website URL to open

What happens after executing this command?

1. Playwright opens a browser window.
2. Playwright Inspector opens.
3. You can interact with the website.
4. Playwright records your actions.
5. Generated code appears in the Inspector.

Can we run Codegen without a URL?

Yes!

You can execute:

```
npx playwright codegen
```

The browser opens without a predefined website URL.

You can enter the website address directly into the browser.

Interview point: The URL in the Codegen command is optional.

3. Recording a test

Very important

Recording means performing actions in the browser while Playwright automatically generates the corresponding automation code.

Let's understand this with a practical example.

Example: Automating a Todo application

Test generator | Playwright
playwright.dev

Suppose you want to test a Todo application.

Test scenario:

1. Open the Todo application.

2. Enter a task: Learn Playwright.
3. Press Enter.
4. Verify that the task appears.

Step 1: Start Codegen

Open the VS Code terminal:

```
npx playwright codegen demo.playwright.dev/todomvc
```

Step 2: Interact with the browser

Perform these actions:

- Click the Todo input field.
- Enter `Learn Playwright`.
- Press Enter.

Step 3: Check the generated code

Playwright automatically generates code similar to:

```
import { test, expect } from '@playwright/test';

test('test', async ({ page }) => {

  await page.goto('https://demo.playwright.dev/todomvc/');

  await page.getByPlaceholder('What needs to be done?')
    .fill('Learn Playwright');

  await page.getByPlaceholder('What needs to be done?')
    .press('Enter');

});
```

The exact generated code can differ depending on the website and the actions performed.

Step 4: Add an assertion

You can use the Inspector's assertion recording feature or manually add:

```
await expect(
  page.getByText('Learn Playwright')
).toBeVisible();
```

Now the test verifies whether the task is displayed.

Step 5: Copy the generated code

The Inspector provides these important controls:

Button	Purpose
Record	Start or resume recording
Copy	Copy generated code
Clear	Clear generated code
Pick Locator	Select and generate locators

Once you finish recording, use Copy to copy the generated code into your VS Code test file.

You can then edit, save and execute it.

4. What actions can Codegen record?

Codegen automatically generates code for browser interactions.

Common examples:

Browser action	Generated Playwright method

Click a button	`click()`
Enter text	`fill()`
Select checkbox	`check()`
Hover over element	`hover()`
Press Enter	`press()`
Select dropdown	`selectOption()`

For example, if you enter your username:

```
await page.getByLabel('Username').fill('Komal');
```

If you click Login:

```
await page.getByRole('button', { name: 'Login' }).click();
```

If you press Enter:

```
await page.keyboard.press('Enter');
```

Recording assertions

Codegen can also generate assertions.

An assertion means verifying whether something is correct after performing an action.

The Inspector provides three important assertion options.

1. Assert visibility

Checks whether an element is visible.

Example:

```
await expect(
  page.getByText('Login successful')
).toBeVisible();
```

2. Assert text

Checks whether an element contains specific text.

Example:

```
await expect(
  page.locator('.message')
).toContainText('Login successful');
```

3. Assert value

Checks whether an input field has a particular value.

Example:

```
await expect(
  page.getByLabel('Username')
).toHaveValue('Komal');
```

Important difference

Assertion option	What it verifies
Assert visibility	Element is visible
Assert text	Element contains expected text
Assert value	Input has expected value

Interview point: Codegen can generate both actions and assertions. Assertions can be recorded using the assertion toolbar in Playwright Inspector.

5. Generating Locators

Very important for interviews

This is another major feature of Codegen.

5.1 What is a locator?

A locator identifies an element on a web page.

For example:

- Login button.
- Username textbox.
- Password textbox.
- Search box.
- Submit button.

Codegen helps us identify these elements and automatically suggests locator code.

How to generate a locator?

Follow these steps:

Step 1: Run Codegen.

```
npx playwright codegen https://example.com
```

Step 2: Browser and Inspector open.

Step 3: Stop recording by clicking the Record button.

Step 4: Click the Pick Locator button.

Step 5: Move your mouse over the required web element.

The locator is highlighted in the browser.

Step 6: Click the required element.

The generated locator appears in the locator playground.

Step 7: Copy the locator and use it in your automation script.

Test generator | Playwright
playwright.dev

5.2 What is Pick Locator?

Pick Locator is a feature inside Playwright Inspector that helps identify and generate locators for web elements.

For example, suppose you want to identify the Login button.

Codegen may generate:

```
page.getByRole('button', { name: 'Login' })
```

You can copy this locator and use it:

```
await page.getByRole('button', { name: 'Login' }).click();
```

5.3 What is Locator Playground?

Locator Playground is the area in Playwright Inspector where you can view and modify the generated locator.

For example, Codegen generates:

```
page.getByText('Login')
```

You can edit the locator and see which element it matches in the browser.

This helps you check whether your locator uniquely identifies the correct element.

Why is Locator Playground useful?

1. Helps generate locators quickly.
2. Helps identify incorrect locators.
3. Allows us to modify and test locators.
4. Highlights matching elements.
5. Helps reduce locator-related failures.

6. How does Codegen choose the best locator?

This is an important interview concept.

Playwright Codegen analyzes the rendered webpage and recommends locators in the following priority:

1. Role locator (`getByRole()`)
2. Text locator (`getByText()`)
3. Test ID locator (`getByTestId()`)

It generally prefers locators based on how users interact with the application rather than relying on complex CSS or XPath selectors.

Example

Suppose a webpage contains a Login button.

Codegen may generate:

```
page.getByRole('button', { name: 'Login' })
```

Why is this useful?

Because it identifies the button using its role and accessible name.

What happens if multiple elements match?

Suppose a webpage has two buttons named Submit.

A locator like this:

```
page.getByRole('button', { name: 'Submit' })
```

may match two elements.

Codegen tries to refine the locator to uniquely identify the intended element.

For example, it might generate a locator using additional information or a more specific locator combination.

Interview point: Playwright Codegen prioritizes role, text and test ID locators and attempts to make generated locators uniquely identify the target element.

Important: Always review generated locators. Codegen helps create locators, but it cannot guarantee that every generated locator is the best choice for a maintainable automation framework.

7. Emulation in Playwright Codegen

Important concept

What is emulation?

Emulation means testing how an application behaves under different device and browser settings without needing to use a physical device for every test.

Playwright Codegen allows us to generate tests using different emulation settings.

You can emulate:

Emulation	Meaning
Viewport	Different screen sizes
Device	Mobile or tablet device settings
Color scheme	Light or dark mode
Geolocation	Location coordinates
Language	Browser language
Timezone	Different time zones
Authentication	Preserve logged-in state

7.1 Viewport emulation

Viewport means the visible area of the browser.

For example:

- Desktop: 1920 × 1080
- Tablet: 768 × 1024
- Mobile: 390 × 844

This helps test responsive web applications.

7.2 Device emulation

You can generate tests using device profiles such as mobile phones and tablets.

For example, testing whether a website works properly on an iPhone-sized viewport.

7.3 Color scheme emulation

You can test applications in:

- Light mode
- Dark mode

This helps verify whether the website's appearance changes correctly.

7.4 Geolocation emulation

You can test applications that display location-specific information.

Example: A shopping application displaying products based on the user's location.

7.5 Language emulation

You can test whether the application supports different languages.

For example:

- English
- Hindi

- French

7.6 Timezone emulation

You can test how an application behaves in different time zones.

For example, testing date and time displays for users in India and the USA.

7.7 Authentication state

Codegen can preserve an authenticated state.

This means you can generate tests while maintaining an existing login session rather than having to log in again during recording.

For example, when using the Codegen authentication state option:

```
npx playwright codegen --save-storage=auth.json https://example.com
```

This saves browser storage state after you log in.

You can later use it in a browser context:

```
const context = await browser.newContext({
  storageState: 'auth.json'
});
```

Note: Authentication storage files can contain sensitive session information. Do not commit them to a public Git repository.

8. Advantages and limitations of Codegen

Advantages

1. Reduces manual coding effort.
2. Helps beginners learn Playwright syntax.
3. Generates locators automatically.
4. Records browser actions.
5. Helps create assertions.

6. Supports different emulation settings.
7. Speeds up initial test creation.

Limitations

1. Generated code may contain unnecessary steps.
2. Locators may need manual improvement.
3. It does not automatically create a complete professional automation framework.
4. It does not replace manual test-case design.
5. Complex scenarios may require additional coding.
6. Generated scripts need review, debugging and maintenance.

Real-world QA example:

In an automation project, you can use Codegen to quickly generate login, search and navigation actions.

After generating the code, you can organize it into Page Object Model (POM), add reusable methods, improve locators, implement test data handling and integrate reporting.

This is how Codegen can be used effectively in real projects.

9. Complete practical example

Let's create a test for a Todo application using Codegen.

Scenario: Add a task and verify it.

Playwright End-to-End Testing: A Step-by-Step Guide | Better Stack Community
betterstack.com

First, execute:

```
npx playwright codegen demo.playwright.dev/todomvc
```

Perform the following actions in the browser:

1. Click the input box.
2. Enter `Learn Playwright`.
3. Press Enter.
4. Select the task checkbox.
5. Verify the task is displayed.

The generated and reviewed test could look like this:

```
import { test, expect } from '@playwright/test';

test('Add and complete a todo task', async ({ page }) => {

  // Navigate to application
  await page.goto('https://demo.playwright.dev/todomvc/');

  // Enter task
  const todoInput = page.getByPlaceholder('What needs to be done?');

  await todoInput.fill('Learn Playwright');

  // Press Enter
  await todoInput.press('Enter');

  // Verify task is visible
  const todoItem = page.getByText('Learn Playwright');

  await expect(todoItem).toBeVisible();

  // Mark task as completed
  await page.getByRole('checkbox').check();

  // Verify checkbox is checked
  await expect(page.getByRole('checkbox')).toBeChecked();

});
```

This example demonstrates:

- Navigation

- Locator creation
- Text entry
- Keyboard interaction
- Assertions
- Checkbox interaction

You can execute this test using:

```
npx playwright test
```

10. Important Interview Questions and Answers

Interview preparation
1–2 Years Experience

These are the questions you should prepare from the Codegen topic.

A. Basic Codegen questions

Q1. What is Codegen in Playwright?

Answer:

Codegen stands for Code Generator. It is a built-in Playwright tool that automatically generates test scripts based on user interactions with a website.

We can perform actions such as clicking, filling text fields and selecting options. Playwright records these interactions and generates corresponding automation code.

Q2. How do you start Codegen?

Answer:

We use the following command:

```
npx playwright codegen https://example.com
```

It opens a browser window and Playwright Inspector, where we can record interactions and generate test scripts.

Q3. Can we run Codegen without providing a URL?

Answer:

Yes. The URL is optional.

```
npx playwright codegen
```

The browser opens, and we can enter the website URL manually.

Q4. What is Playwright Inspector?

Answer:

Playwright Inspector is a graphical tool used to record browser interactions, generate automation code, inspect locators and manage recorded tests.

It also provides features such as Pick Locator and Locator Playground.

Q5. What is the difference between Codegen and Playwright Test Runner?

Answer:

Codegen is used to generate test scripts automatically by recording browser interactions.

Playwright Test Runner is used to execute test cases, manage fixtures, run tests in parallel, perform assertions and generate reports.

Codegen	Test Runner
Generates test code	Executes test code
Records browser actions	Runs written test cases
Helps identify locators	Evaluates assertions
Used for initial test creation	Used for actual test execution

B. Recording questions

Q6. What actions can Codegen record?

Answer:

Codegen can record browser interactions such as:

- Clicking elements.
- Filling input fields.
- Selecting options.
- Checking checkboxes.
- Hovering over elements.
- Pressing keyboard keys.

It can also generate assertions through the Inspector.

Q7. How do you record assertions using Codegen?

Answer:

We click the assertion toolbar in Playwright Inspector and select an element in the browser.

We can choose:

1. Assert visibility.
2. Assert text.
3. Assert value.

Playwright generates the corresponding assertion code.

Q8. How do you stop recording and copy generated code?

Answer:

We use the Record button in Playwright Inspector to stop or pause recording.

Then we click Copy to copy the generated code into our test file.

We can also use Clear to remove generated code and start again.

Q9. Does Codegen generate assertions automatically?

Answer:

Yes. We can generate assertions using the assertion toolbar in Playwright Inspector.

It supports visibility, text and value assertions.

Q10. Can we modify the code generated by Codegen?

Answer:

Yes. Generated code is normal Playwright code. We can copy it into our editor and modify locators, assertions, test names and test structure according to our requirements.

C. Locator questions

Q11. What is Pick Locator?

Answer:

Pick Locator is a feature in Playwright Inspector that helps us identify web elements and generate their corresponding locator code.

We click Pick Locator, hover over the required element and select it. Playwright generates the locator.

Q12. What is Locator Playground?

Answer:

Locator Playground is a feature in Playwright Inspector where we can view and edit generated locators.

It highlights the matching element in the browser, helping us verify whether the locator identifies the correct element.

Q13. How does Codegen select locators?

Answer:

Codegen analyzes the rendered webpage and prioritizes:

1. Role locators.
2. Text locators.
3. Test ID locators.

It also attempts to make locators unique when multiple elements match.

Q14. What happens if multiple elements match the same locator?

Answer:

Codegen attempts to refine the locator to uniquely identify the intended element.

If the locator still matches multiple elements, we should manually improve it using a more specific locator.

For example:

```
page.getByRole('button', { name: 'Submit' }).nth(1);
```

However, using a more meaningful locator is preferable when possible.

Q15. Does Codegen always generate the best locator?

Answer:

Not necessarily. Codegen recommends locators based on the page structure and accessibility information.

We should review generated locators and make improvements if necessary to ensure uniqueness, readability and maintainability.

D. Emulation questions

Q16. What is emulation in Playwright?

Answer:

Emulation allows us to test web applications under different device and browser settings without requiring physical devices for every test.

We can emulate viewport sizes, mobile devices, geolocation, timezone, language and color schemes.

Q17. Can Playwright test mobile applications using emulation?

Answer:

Playwright supports mobile browser emulation, including device configurations for mobile viewports, touch and other device characteristics.

However, this is browser-based mobile web testing, not native Android or iOS application automation.

Q18. Can we test dark mode using Playwright?

Answer:

Yes. Playwright supports color scheme emulation.

We can test both light and dark modes using the `colorScheme` setting.

Example:

```
const context = await browser.newContext({
  colorScheme: 'dark'
});
```

Q19. What is geolocation emulation?

Answer:

Geolocation emulation allows us to test applications using specific geographical coordinates.

For example, we can test whether a location-based application displays the correct information for users in different cities.

Q20. Can Codegen preserve authentication state?

Answer:

Yes. Codegen can save browser storage state after authentication using the `--save-storage` option.

We can reuse that state in subsequent tests to avoid repeating the login process.

Example:

```
npx playwright codegen --save-storage=auth.json https://example.com
```

E. Real-world scenario-based questions

Q21. Suppose you need to automate an application but don't know its locators. How will you proceed?

Answer:

I will use Playwright Codegen to open the application and interact with its elements.

I will use Pick Locator to identify buttons, text fields and other elements.

After generating the locators, I will review them, copy the required code into my test file and add appropriate assertions.

Q22. Can we use Codegen for an existing automation project?

Answer:

Yes. We can use Codegen to generate additional test scenarios or identify locators for an existing application.

We can copy the generated code into the existing project and integrate it with the current framework.

Q23. Does Codegen create a complete Page Object Model framework automatically?

Answer:

No. Codegen generates test scripts and locators, but it does not automatically create a complete maintainable POM framework.

We need to organize the generated code into page classes, reusable methods, test files, configuration and test data management.

Q24. What is the difference between manually writing tests and generating tests using Codegen?

Answer:

In manual test creation, we write locators, actions and assertions ourselves.

In Codegen, Playwright records browser interactions and automatically generates the corresponding code.

Both approaches produce Playwright test scripts. Generated code may still require manual review and improvement.

11. Quick Revision – Codegen

Topic	Important point
Codegen	Automatically generates test code
Command	`npx playwright codegen URL`
URL	Optional
Browser	Used to interact with website
Inspector	Records and displays generated code
Record	Starts or resumes recording
Copy	Copies generated code
Clear	Clears generated code
Pick Locator	Identifies web elements
Locator Playground	Edits and tests locators
Assert visibility	Checks element visibility
Assert text	Checks expected text
Assert value	Checks input value

Emulation	Simulates different device settings
Authentication state	Preserves browser storage state

13. What to prepare practically before your interview

Try this exercise in VS Code:

1. Open a terminal.
2. Run `npx playwright codegen` https://demo.playwright.dev/todomvc.
3. Add a Todo item using the browser.
4. Generate a locator using Pick Locator.
5. Record an assertion.
6. Copy the generated code into `tests/todo.spec.ts`.
7. Run `npx playwright test`.

You will understand Codegen much better by doing this once yourself.

Playwright – Running and Debugging Tests

This is an important topic for QA Automation interviews because interviewers often ask:

- How do you execute Playwright tests?
- How do you run tests in Chrome, Firefox, and WebKit?
- What is the difference between headed mode and headless mode?
- How do you debug a failed test?
- What is Playwright Inspector?
- How do you generate an HTML report?

Let's understand everything step by step with practical examples.

1. Introduction: Running Playwright tests

Playwright allows you to execute:

1. A single test.
2. A specific test file.
3. Multiple test files.
4. All test cases.
5. Tests on different browsers.
6. Only previously failed tests.

By default, Playwright runs tests:

- In headless mode – browser window is not visible.
- In parallel mode – multiple tests can execute simultaneously, depending on configuration.
- On configured browsers – such as Chromium, Firefox, and WebKit.

What is headless mode?

Headless means the browser runs in the background without displaying its window.

For example, suppose you have an automation test:

```
import { test, expect } from '@playwright/test';

test('verify login page', async ({ page }) => {
  await page.goto('https://example.com');

  await expect(page).toHaveTitle(/Example/);
});
```

When you execute:

```
npx playwright test
```

Playwright opens the browser in the background, executes the test, and displays the result in the terminal.

[Playwright 101: The Testing Revolution You've Been Waiting For | Stackademic](stackademic.com)

Real-world example: In a CI/CD pipeline, such as Jenkins or GitHub Actions, tests commonly run in headless mode because nobody needs to watch the browser.

2. Running tests from the command line

A. Run all tests

Command:

```
npx playwright test
```

Meaning:

Part	Explanation
`npx`	Executes a locally installed npm package
`playwright`	Playwright package
`test`	Runs the test runner

This command executes all test files matching the configured test file pattern, generally files ending in `.spec.ts` or `.test.ts`.

Example folder:

```
PlaywrightProject/
│
├── tests/
│   ├── login.spec.ts
│   ├── signup.spec.ts
│   └── checkout.spec.ts
│
├── playwright.config.ts
└── package.json
```

Running `npx playwright test` executes all matching test files.

B. Run tests in UI Mode

Command:

`npx playwright test --ui`
Recommended for learning and debugging

UI Mode provides a graphical interface where you can:

- See all test cases.
- Execute individual tests.
- See each test step.
- View errors and logs.
- Inspect network requests.
- Inspect DOM snapshots.
- Use the locator picker.
- Watch tests while editing your code.

UI Mode | Playwright
UI Mode | Playwright
Running and Debugging Tests With Playwright UI Mode
6

Practical example:

Suppose your login test is failing because the username textbox locator is incorrect.

In UI Mode, you can:

1. Open the failed test.
2. Select the failed action.
3. Inspect the page and DOM snapshot.
4. Use Pick Locator to select the username textbox.

5. See the locator generated by Playwright.
6. Copy the corrected locator into your test.

This makes debugging easier than repeatedly checking the terminal.

C. Run tests in Headed Mode

Command:

```
npx playwright test --headed
```

Headed means the browser window is visible while the test executes.

Running and debugging tests | Playwright
playwright.dev

Example:

```
import { test, expect } from '@playwright/test';

test('verify Google page', async ({ page }) => {
  await page.goto('https://www.google.com');

  await expect(page).toHaveTitle(/Google/);
});
```

Execute:

```
npx playwright test --headed
```

You can visually observe the browser opening and navigating to Google.

Difference between Headless, Headed and UI Mode

Feature	Headless	Headed	UI Mode
Browser visible	No	Yes	Yes, through UI
Test execution	Yes	Yes	Yes

Debugging features	Limited by default	Limited by default	Rich debugging tools
Best use	CI/CD execution	Visual observation	Development and debugging
Command	`npx playwright test`	`--headed`	`--ui`

Interview tip: Headed mode and UI Mode are not the same. Headed mode simply displays the browser, while UI Mode provides a dedicated interface for inspecting and debugging tests.

3. Run tests on different browsers

Playwright supports three major browser engines:

WizWrite - Voice Productivity for Mac, iPhone & iPad

Chromium

Chrome and Edge use Chromium
Mozilla Firefox 91 brings support for Windows single sign-on | Windows Central

Firefox

Firefox browser engine
U.K. Court Blocks Mass Legal Action Over Google's Alleged Tracking of Safari Users - MacRumors

WebKit

Engine used by Safari

Playwright uses the `--project` flag to select configured browser projects.

Run tests on WebKit

```
npx playwright test --project=webkit
```

Only the configured project named `webkit` will execute.

Run tests on Firefox

```
npx playwright test --project=firefox
```

Run tests on multiple browsers

```
npx playwright test --project=webkit --project=firefox
```

This runs the tests on both configured projects.

Example: playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',

  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
  ],
});
```

Now execute:

```
npx playwright test
```

All three projects will run.

Real-world QA example:

Suppose you are testing an e-commerce application.

You want to verify whether:

- Login works in Chrome.
- Product search works in Firefox.
- Checkout works in Safari.

Cross-browser testing helps identify browser-specific issues.

Important: The project names passed to `--project` must match the names defined in `playwright.config.ts`.

4. Run specific tests

This is very useful when your project contains 100+ test cases and you don't want to execute everything every time.

A. Run a single test file

```
npx playwright test landing-page.spec.ts
```

Only the specified test file runs.

Example:

```
tests/
├── login.spec.ts
├── landing-page.spec.ts
└── checkout.spec.ts
```

Command:

```
npx playwright test tests/login.spec.ts
```

Only login tests will execute.

B. Run multiple directories

```
npx playwright test tests/todo-page/ tests/landing-page/
```

This runs matching test files inside both directories.

C. Run files based on their names

```
npx playwright test landing login
```

This matches test file paths containing `landing` or `login`.

D. Run a test using its title

Use the `-g` flag.

```
npx playwright test -g "add a todo item"
```

Here, `-g` means grep. It filters tests using their title.

Example:

```
import { test, expect } from '@playwright/test';

test('add a todo item', async ({ page }) => {
  await page.goto('https://demo.playwright.dev/todomvc/');

  await page.getByPlaceholder('What needs to be done?').fill('Learn
Playwright');

  await page.keyboard.press('Enter');

  await expect(page.getByText('Learn Playwright')).toBeVisible();
});

test('delete a todo item', async ({ page }) => {
  // Delete todo test
});
```

Command:

```
npx playwright test -g "add a todo item"
```

Only the test with the matching title will be selected.

E. Run only the last failed tests

Command:

```
npx playwright test --last-failed
```

Suppose you execute 10 tests:

Test	Result
Login	Passed
Signup	Passed
Search	Failed
Add to cart	Passed
Checkout	Failed
Payment	Passed
Logout	Passed
Profile	Failed
Product details	Passed
Order history	Passed

Three tests failed.

When you run:

```
npx playwright test --last-failed
```

Playwright reruns only:

- Search
- Checkout
- Profile

It uses the previous run's failure information stored in `.last-run.json` inside the configured output directory.

You can also specify a custom file:

```
npx playwright test --last-failed --last-failed-file=.cache/last-run-shard-1.json
```

Important: This is useful for rerunning failures from the previous run. It does not mean Playwright automatically fixes the failed tests.

5. Run tests in VS Code

You can execute Playwright tests directly from Visual Studio Code.

First, install the official Playwright Test for VS Code extension.

Getting started - VS Code | Playwright
cuketest.com

After installation:

1. Open your Playwright project in VS Code.
2. Open the Testing sidebar.
3. Playwright discovers your test cases.
4. Click the green triangle beside a test to execute it.
5. You can also run all tests from the Testing sidebar.

Advantages:

- No need to type commands every time.
- Run an individual test easily.
- Debug tests using breakpoints.
- View test results inside VS Code.

6. Debugging Playwright tests

Now let's understand debugging.

What is debugging?

Debugging means finding and fixing errors in your automation code.

Suppose your login test fails:

```
test('login test', async ({ page }) => {
  await page.goto('https://example.com');

  await page.locator('#username').fill('komal');

  await page.locator('#password').fill('password123');

  await page.locator('#login-btn').click();

  await expect(page.getByText('Welcome')).toBeVisible();
});
```

Possible reasons for failure:

- Incorrect locator.
- Element is not visible.
- Application is taking too long to load.
- Incorrect test data.
- Application bug.
- Login credentials are invalid.

Playwright provides several debugging methods.

7. Debug using UI Mode

Command:

```
npx playwright test --ui
```

This is one of the easiest ways to debug Playwright tests.

[UI Mode | Playwright
playwright.dev]

You can inspect:

Feature	Purpose
Test list	View all test cases
Timeline	See the order of test actions
Logs	View execution information
Errors	Understand why a test failed
Network	Inspect network requests
DOM snapshot	Inspect the page structure at a particular step
Pick Locator	Identify elements and generate locators
Watch mode	Rerun tests when relevant files change

What is Pick Locator?

Pick Locator helps you identify an element on the webpage and generate its Playwright locator.

Example: You want to identify the Login button.

You click Pick Locator and select the Login button.

Playwright may generate:

```
page.getByRole('button', { name: 'Login' });
```

You can test and edit the locator in the locator playground and see the matching element highlighted.

Then click Copy Locator and paste it into your test code.

Interview point: Pick Locator helps create and inspect locators. It does not automatically guarantee that the locator is the best choice for every situation; review it before using it in your framework.

8. Debug using Playwright Inspector

Playwright Inspector is a debugging tool that lets you execute your test step by step.

Command:

```
npx playwright test --debug
```

Running and debugging tests | Playwright
playwright.dev

When you run this command:

1. A browser window opens.
2. Playwright Inspector opens.
3. You can inspect your test.
4. Execute the test step by step.
5. Inspect locators and debug logs.

What is Step Over?

Step Over allows you to execute the next line of code and then pause again at the following step.

Example:

```
test('login test', async ({ page }) => {

  await page.goto('https://example.com');

  await page.locator('#username').fill('komal');
```

```
  await page.locator('#password').fill('password123');

  await page.locator('#login-btn').click();

});
```

Using Step Over, you can execute each line one at a time and observe what happens in the browser.

Play button

The Play button continues execution of the test until completion or another pause point.

Once the test finishes, the browser window closes.

Debug a specific test file

```
npx playwright test example.spec.ts --debug
```

Only tests from the specified file are selected for debugging.

Debug a specific test using line number

```
npx playwright test example.spec.ts:10 --debug
```

This selects the test defined at or associated with line 10 in that file.

Important: The line number should point to the test definition, such as the line containing `test(...)`.

9. Difference between UI Mode and Playwright Inspector

This is a common interview question.

Feature	UI Mode	Playwright Inspector
Command	`npx playwright test --ui`	`npx playwright test --debug`

Interface	Full test-runner UI	Dedicated debugging inspector
Test execution	Run individual or multiple tests	Step-by-step debugging
Timeline	Yes	Debugging controls and logs
Locator picker	Yes	Yes
Network inspection	Yes	Not the same full UI Mode experience
Best use	Test analysis and debugging	Stepping through code

Remember:

- UI Mode = Visual test execution and analysis.
- Inspector = Step-by-step debugging.

10. Other ways to debug tests

Playwright runs on Node.js, so you can also use normal JavaScript/TypeScript debugging techniques.

A. Use console.log()

```
test('login test', async ({ page }) => {

  await page.goto('https://example.com');

  const title = await page.title();

  console.log('Page title:', title);

});
```

Output appears in the terminal or test output.

Useful for checking:

- Variable values.
- Page title.
- API response data.
- Test data.
- Execution progress.

B. Use page.pause()

You can insert:

```
await page.pause();
```

Example:

```
test('login test', async ({ page }) => {

  await page.goto('https://example.com');

  await page.pause();

  await page.locator('#username').fill('komal');

});
```

When execution reaches `page.pause()`, Playwright pauses and opens debugging tools.

To use this effectively, run the test in headed mode or use the Inspector:

```
npx playwright test --debug
```

C. Use VS Code breakpoints

You can set a breakpoint beside a line of code in VS Code.

When the test reaches that line, execution pauses.

You can inspect:

- Variable values.
- Call stack.
- Current execution line.
- Errors.

This is useful when debugging complex automation frameworks.

11. Test Reports – HTML Reporter

After test execution, how do you know which tests passed and failed?

Playwright provides an HTML Reporter.

Command:

```
npx playwright show-report
```

This opens the HTML test report in your browser.

Running and debugging tests | Playwright

What information does the HTML report contain?

Feature	Explanation
Passed tests	Tests that executed successfully
Failed tests	Tests that encountered failures
Skipped tests	Tests intentionally not executed
Flaky tests	Tests that failed initially but passed on retry
Browser filter	Filter results by browser

Search	Search for test cases
Error details	Understand test failure
Test steps	Inspect individual execution steps

Example

Suppose your automation suite contains 100 tests.

Sample Playwright execution report

85

Passed

10

Failed

5

Skipped
Illustrative results: 100 total tests.

You can click a failed test to view its error details and inspect the steps.

Configure HTML Reporter

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  reporter: 'html',
});
```

Now execute:

```
npx playwright test
```

Then open the report:

```
npx playwright show-report
```

By default, the HTML report is configured to open automatically after a run when tests fail. You can configure this behavior using the reporter options.

For example:

```
reporter: [
  ['html', { open: 'never' }]
],
```

This prevents automatic opening. You can still manually open it with `npx playwright show-report`.

12. Important commands – Quick revision

Requirement	Command
Run all tests	`npx playwright test`
Run in UI Mode	`npx playwright test --ui`
Run in headed mode	`npx playwright test --headed`
Run on WebKit	`npx playwright test --project=webkit`
Run on Firefox	`npx playwright test --project=firefox`
Run on multiple browsers	`npx playwright test --project=webkit --project=firefox`

Run a single file	`npx playwright test login.spec.ts`
Run multiple directories	`npx playwright test tests/login tests/home`
Run by test title	`npx playwright test -g "login test"`
Rerun failed tests	`npx playwright test --last-failed`
Debug all tests	`npx playwright test --debug`
Debug a specific file	`npx playwright test login.spec.ts --debug`
Debug a test by line	`npx playwright test login.spec.ts:10 --debug`
Open HTML report	`npx playwright show-report`

13. Interview Questions and Answers

Basic questions

Q1. How do you run all Playwright test cases?

Answer:

```
npx playwright test
```

It executes all matching test files based on the configuration.

Q2. What is the default execution mode in Playwright?

Answer: Playwright runs tests in headless mode by default, meaning the browser window is not displayed. Tests also run in parallel by default, subject to configuration.

Q3. What is headless mode?

Answer: Headless mode executes browser automation without displaying the browser window. It is commonly used in CI/CD pipelines.

Q4. How can you execute tests in headed mode?

Answer:

```
npx playwright test --headed
```

It displays the browser window while tests execute.

Q5. What is UI Mode in Playwright?

Answer: UI Mode is a graphical test-running and debugging interface. It allows testers to execute tests, inspect test steps, view errors, inspect DOM snapshots, and use the locator picker.

Q6. How do you run a specific test file?

Answer:

```
npx playwright test login.spec.ts
```

Q7. How do you run a specific test by its title?

Answer:

```
npx playwright test -g "login test"
```

The `-g` flag filters tests using their title.

Q8. How do you run tests in Firefox?

Answer:

```
npx playwright test --project=firefox
```

The Firefox project must be configured in `playwright.config.ts`.

Browser-related questions

Q9. How do you execute tests on multiple browsers?

Answer:

```
npx playwright test --project=chromium --project=firefox
```

It executes tests on both configured browser projects.

Q10. What is cross-browser testing?

Answer: Cross-browser testing verifies whether an application works correctly across different browsers, such as Chrome, Firefox, and Safari.

Q11. What is the purpose of the `--project` flag?

Answer: It selects one or more browser/device projects configured in `playwright.config.ts`.

Debugging questions

Q12. What is debugging?

Answer: Debugging is the process of identifying, analyzing, and fixing errors in automation scripts.

Q13. How do you debug all Playwright tests?

Answer:

```
npx playwright test --debug
```

It opens the browser and Playwright Inspector.

Q14. What is Playwright Inspector?

Answer: Playwright Inspector is a debugging tool that allows testers to execute tests step by step, inspect API calls, examine logs, and explore locators.

Q15. What is the purpose of the Step Over button?

Answer: It executes the current line of code and pauses at the next step, allowing you to inspect the execution.

Q16. What is the difference between UI Mode and Inspector?

Answer:

- UI Mode provides a complete test-running interface with timelines, logs, network requests, and DOM snapshots.
- Inspector provides focused step-by-step debugging controls.

Q17. How do you pause test execution?

Answer: Use:

```
await page.pause();
```

It pauses execution and opens debugging tools when run in a suitable headed/debugging session.

Q18. How can you inspect the locator of an element?

Answer: Use the Pick Locator feature in UI Mode or Playwright Inspector. Select the element to generate a locator.

Q19. Can we debug Playwright using VS Code?

Answer: Yes. Install the Playwright Test extension, use breakpoints, and debug tests directly from VS Code.

Reports and execution questions

Q20. How do you generate or open an HTML report?

Answer:

```
npx playwright show-report
```

The command opens the HTML report generated by the previous test run.

Q21. What information does an HTML report provide?

Answer: It provides passed, failed, skipped, and flaky test results, along with errors, test steps, and filtering options.

Q22. What is a flaky test?

Answer: A flaky test is a test that produces inconsistent results. For example, it fails on one execution but passes on another without any relevant code change.

Q23. How do you rerun only failed tests?

Answer:

```
npx playwright test --last-failed
```

It reruns tests that failed in the previous run, using the stored last-run information.

Q24. What would you do if a Playwright test failed?

Answer:

1. Check the terminal error.
2. Open the HTML report.
3. Run the test in UI Mode.
4. Inspect the failed step.
5. Verify the locator and test data.
6. Check whether the failure is caused by the application or automation script.
7. Fix the issue and rerun the test.

14. Practical interview scenario

Question: Suppose you have 100 test cases in your Playwright project. Out of them, 10 are failing. How will you execute only those 10 failed test cases?

Answer:

First, I execute all test cases:

```
npx playwright test
```

After identifying failures, I execute:

```
npx playwright test --last-failed
```

This reruns only the tests that failed in the previous execution.

If I want to investigate a failure, I can use:

```
npx playwright test --ui
```

or:

```
npx playwright test --debug
```

Then I inspect the failed step, locator, logs, and error details.

Question: You have to verify your application on Chrome, Firefox, and Safari. How will you execute it?

Answer:

I configure Chromium, Firefox, and WebKit projects in `playwright.config.ts`, then execute:

```
npx playwright test
```

This runs tests on all configured projects.

15. Test your knowledge

Try these questions without looking at the notes.

Playwright quiz

0/6 answered

1. Which command runs all Playwright tests?

npx playwright run
npx playwright test
npx test playwright
playwright start

2. Which flag runs tests with a visible browser window?

--ui
--debug
--headed
--visible

3. Which command runs tests only on Firefox?

npx playwright test --browser=firefox
npx playwright test --project=firefox
npx playwright firefox
npx playwright test firefox

4. Which command reruns only previously failed tests?

--retry
--failed
--last-failed
--rerun

5. Which tool helps you step through Playwright API calls?

HTML Reporter
Playwright Inspector
package.json
npm

6. Which command opens the HTML report?

npx playwright report
npx playwright show-report
npx playwright html
npx show report
Submit answers

What you should remember for interviews

The most important commands from this section are:

1. `npx playwright test` – Run all tests.
2. `npx playwright test --headed` – Run with a visible browser.
3. `npx playwright test --ui` – Open UI Mode.
4. `npx playwright test --project=firefox` – Run on Firefox.
5. `npx playwright test -g "test title"` – Run by test title.
6. `npx playwright test --last-failed` – Rerun failed tests.
7. `npx playwright test --debug` – Debug using Inspector.
8. `npx playwright show-report` – Open HTML report.

Playwright – Trace Viewer

Trace Viewer is one of the most important Playwright debugging features. It is especially useful when your automation test fails in Jenkins, GitHub Actions, or another CI/CD pipeline.

Let's understand it step by step with practical examples and interview questions.

1. What is Playwright Trace Viewer?

Playwright Trace Viewer is a GUI (Graphical User Interface) tool used to investigate and debug Playwright test executions.

It records what happened during a test and allows you to go back and inspect each action.

For example, suppose you are automating an e-commerce application:

1. Open the application.
2. Login.
3. Search for a mobile phone.
4. Select a product.
5. Add it to the cart.
6. Verify the cart.

Suppose the test fails at step 5 because the Add to Cart button was not clickable.

Using Trace Viewer, you can investigate:

- What happened before clicking the button?
- Was the button visible?
- Was it enabled?
- What did the page look like?
- Was there an error?
- What network requests were made?
- Which locator was used?

Best Practices | Playwright
playwright.dev

Simple definition for interviews:

Trace Viewer is a debugging tool in Playwright that records test execution details and allows us to inspect each action, DOM snapshot, logs, network requests, and errors.

2. What is a Trace?

A trace is a recorded history of a Playwright test execution.

Playwright stores this recording in a file called:

`trace.zip`

This file contains information about the test execution, such as:

- Actions performed.
- Screenshots.
- DOM snapshots.
- Test source code.
- Console logs.
- Network activity.
- Errors and execution details.

Think of it like a video recording of your automation test, but with additional technical information that helps you investigate failures.

Example:

Your test performs these actions:

```
test('add product to cart', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByRole('button', { name: 'Login' }).click();

  await page.getByPlaceholder('Search products').fill('iPhone');

  await page.getByRole('button', { name: 'Search' }).click();

  await page.getByRole('button', { name: 'Add to Cart' }).click();
```

```
});
```

A trace lets you inspect the execution of these actions one by one.

3. How to record a Trace?

Playwright provides different ways to record traces.

A. Configure Trace in playwright.config.ts

By default, the recommended configuration in the documentation is:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  retries: process.env.CI ? 2 : 0,

  use: {
    trace: 'on-first-retry',
  },

});
```

Let's understand this configuration.

What is retries?

Retries means how many times Playwright will retry a failed test.

```
retries: process.env.CI ? 2 : 0
```

This is a JavaScript conditional expression.

Condition	Retries
Running locally	0
Running in CI	2

Here, `process.env.CI` checks whether the CI environment variable is set.

For example:

- Local execution: If a test fails, it is not retried.
- CI execution: If a test fails, Playwright can retry it up to two times.

What is trace: 'on-first-retry'?

`trace: 'on-first-retry'`

It means Playwright records the trace when a failed test is retried for the first time.

Example:

Suppose a test fails during its first execution.

Example: Test retry and trace recording

1. First execution

Failed

Test fails. No trace is recorded for this initial attempt under this setting.

2. First retry

Trace recorded

Playwright records the trace during this retry.

3. Second retry

No new trace

No trace is recorded for this attempt under on-first-retry.

The trace from the first retry can then be inspected to understand the failure.

Important: `on-first-retry` does not record a trace during the initial execution. It records one on the first retry.

B. Record Trace locally using the command line

If you want to record traces locally without waiting for a retry, use:

```
npx playwright test --trace on
```

This forces tracing to be enabled for the test run.

For example:

```
npx playwright test login.spec.ts --trace on
```

This runs the login test file with tracing enabled.

Difference between the two approaches:

Configuration/command	When trace is recorded
`trace: 'on-first-retry'`	On the first retry
`--trace on`	During every test execution in that run
`trace: 'on'`	During every test execution
`trace: 'off'`	No trace recording

Other useful trace options include:

- `retain-on-failure` – retain traces for failed tests.
- `on-all-retries` – record traces on retries.
- `retain-on-first-failure` – retain the first failed attempt's trace.

For everyday debugging, `--trace on` is useful. For CI pipelines, `on-first-retry` is a common choice because it balances debugging information and storage usage.

4. What is CI/CD, and why are traces useful there?

CI/CD stands for Continuous Integration and Continuous Delivery/Deployment.

Examples of CI/CD tools:

- Jenkins

- GitHub Actions
- GitLab CI
- Azure DevOps

Suppose you have written 100 Playwright automation tests.

You push your code to GitHub. GitHub Actions automatically executes your tests.

But nobody is watching the browser.

Suppose 10 tests fail.

How will you understand what happened?

This is where Trace Viewer helps.

[Debugging Playwright Failures in GitHub Actions | SentinelQA
sentinelqa.com]

With traces, you can investigate the failed execution after the pipeline has finished.

Interview example:

"If my Playwright test fails in Jenkins, I check the test report and open the trace attached to the failed test. I inspect the action timeline, DOM snapshot, logs, and network requests to identify the root cause."

5. How to open the HTML Report?

Before opening a trace through the HTML report, first generate the test results.

Execute:

```
npx playwright test
```

Then open the HTML report:

```
npx playwright show-report
```

The HTML report displays:

- Passed tests.
- Failed tests.

- Skipped tests.
- Flaky tests.
- Browser information.
- Execution time.
- Error details.
- Test steps.
- Available traces.

Trace viewer | Playwright
playwright.dev

How to open a trace from HTML Report?

Follow these steps:

1. Execute your Playwright tests.
2. Run `npx playwright show-report`.
3. Open the failed test.
4. Look for the trace icon next to the test or open the detailed test view.
5. Go to the Traces tab.
6. Click the trace screenshot to open Trace Viewer.

Embedded Playwright Trace Viewer | TestDino
testdino.com

Important: A trace will only be available if tracing was enabled and the trace was retained for that execution.

6. How to open Trace Viewer directly?

You can also open a trace ZIP file directly.

Command:

```
npx playwright show-trace trace.zip
```

Here:

- `npx` executes the package.
- `playwright` is the Playwright package.
- `show-trace` opens the Trace Viewer.
- `trace.zip` is the recorded trace file.

This is useful when you have downloaded a trace artifact from Jenkins or GitHub Actions.

You can also open Trace Viewer online through the official Playwright Trace Viewer page:

Playwright Trace Viewer – Official website

You can load a trace ZIP file there to inspect it.

Security tip: Trace files can contain page content, screenshots, URLs, and other potentially sensitive information. Don't upload traces containing confidential application data to public tools unless your organization's policies permit it.

7. Understanding Trace Viewer interface

This is the most important part of the topic.

When you open a trace, you will see different sections that help you understand test execution.

Trace viewer | Playwright
playwright.dev

Let's understand the important components.

A. Actions / Timeline

The timeline shows the sequence of actions performed during test execution.

Example:

```
1. Navigate to application
2. Fill username
3. Fill password
4. Click Login
5. Verify dashboard
```

You can click each action to inspect what happened.

You can also hover over the timeline to inspect different moments of execution.

Why is it useful?

Suppose your test fails after clicking Login.

You can select the Login action and investigate what happened immediately before and after it.

B. Screenshots

Trace Viewer provides screenshots associated with actions.

You can inspect how the page looked during the execution.

For example:

- Was the Login button visible?
- Was a popup displayed?
- Was the page still loading?
- Was the wrong page opened?

This helps identify UI-related issues.

C. DOM Snapshot

DOM stands for Document Object Model.

The DOM represents the structure of an HTML webpage.

Example:

```
<html>
  <body>
    <h1>Login Page</h1>
```

```
    <input id="username" />

    <input id="password" />

    <button>Login</button>

  </body>
</html>
```

Trace Viewer captures DOM snapshots that allow you to inspect the page state at different points in the test.

Example:

Your locator is:

```
page.locator('#username')
```

But the test fails.

You can inspect the DOM snapshot and check whether the element actually has the ID `username`.

This helps identify incorrect locators or changes in the webpage structure.

D. Logs

Logs show information about actions performed during test execution.

For example:

```
navigating to "https://example.com"
waiting for locator("#username")
filling "komal"
clicking "Login"
```

They help you understand what Playwright was doing before the failure.

E. Source

The Source section shows the test source code associated with the selected action.

Example:

```
await page.locator('#username').fill('komal');

await page.locator('#password').fill('password123');

await page.getByRole('button', { name: 'Login' }).click();
```

You can identify which line of code corresponds to the selected action.

F. Network

The Network section shows network requests made during the test.

For example:

```
GET /login
POST /api/authenticate
GET /api/user-profile
```

You can investigate:

- Request URL.
- HTTP method.
- Response status.
- Request and response details, where available.
- Failed network requests.

Practical example:

Your login test fails because the application does not navigate to the dashboard.

You inspect the Network section and notice:

```
POST /api/authenticate → 401 Unauthorized
```

This indicates that the authentication request was rejected. You can investigate the credentials, request payload, or application behavior.

G. Console

The Console section displays browser console messages, including errors and warnings captured during the trace.

Example:

```
Uncaught TypeError: Cannot read properties of undefined
```

This can help identify JavaScript errors in the application.

H. Errors

The Errors section helps you identify errors associated with the test execution.

For example:

```
Error: Timed out waiting for expect(locator).toBeVisible()
```

You can use this information to identify the failed assertion or action.

8. Practical example – Debug an e-commerce test

Let's create a small example similar to a real QA automation project.

Scenario: Search for a product and add it to the cart.

```
import { test, expect } from '@playwright/test';

test('search product and add to cart', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByPlaceholder('Search products')
    .fill('Laptop');

  await page.getByRole('button', { name: 'Search' })
    .click();

  await page.getByRole('button', { name: 'Add to Cart' })
    .click();

  await expect(page.getByText('Added to cart'))
    .toBeVisible();

});
```

Run the test with tracing:

```
npx playwright test --trace on
```

Suppose the test fails at:

```
await page.getByRole('button', { name: 'Add to Cart' }).click();
```

How will you debug it?

1. Open the HTML report.
2. Select the failed test.
3. Open its trace.
4. Select the Add to Cart action.
5. Inspect the screenshot and DOM snapshot.
6. Check whether the button exists.
7. Inspect the logs and network requests.
8. Determine whether the problem is an incorrect locator, application issue, or failed API request.

This is how you use Trace Viewer in real automation projects.

9. Difference between UI Mode, Inspector and Trace Viewer

This is a very important interview question because these three tools are often confused.

Feature	UI Mode	Inspector	Trace Viewer
Main purpose	Run and analyze tests	Step-by-step debugging	Analyze recorded execution
Execution	Can run tests	Can debug tests	Views recorded traces

Timeline	Yes	Debugging controls	Yes
DOM snapshots	Yes	Locator inspection	Yes
Network details	Yes	Not its main purpose	Yes
Historical execution	Through available test results	Not its main purpose	Yes, through recorded trace
Typical command	`--ui`	`--debug`	`show-trace trace.zip`

Easy way to remember:

- UI Mode: Run and visually analyze tests.
- Inspector: Debug code step by step.
- Trace Viewer: Investigate what happened during a recorded test execution.

10. Interview Questions and Answers

Basic interview questions

Q1. What is Playwright Trace Viewer?

Answer: Trace Viewer is a GUI debugging tool in Playwright that allows us to inspect recorded test executions, including actions, screenshots, DOM snapshots, logs, network requests, and errors.

Q2. What is a trace file?

Answer: A trace file is a ZIP archive, generally named `trace.zip`, containing information recorded during Playwright test execution.

Q3. What is the purpose of Trace Viewer?

Answer: It helps identify the root cause of failed tests by allowing us to inspect the execution step by step.

Q4. How do you enable tracing in Playwright?

Answer: We can configure tracing in `playwright.config.ts`:

```
use: {
  trace: 'on-first-retry',
}
```

Or enable it from the command line:

```
npx playwright test --trace on
```

Q5. What is `on-first-retry`?

Answer: It records a trace when a failed test is executed for the first time as a retry.

Q6. What is the difference between `on-first-retry` and on?

Answer:

- `on-first-retry`: Records a trace on the first retry.
- on: Records a trace for every test execution.

Q7. What is the purpose of retries in Playwright?

Answer: Retries allow Playwright to execute a failed test again. They help identify intermittent failures and collect additional debugging information.

Practical interview questions

Q8. How do you open an HTML report?

Answer:

```
npx playwright show-report
```

Q9. How do you open a trace ZIP file directly?

Answer:

```
npx playwright show-trace trace.zip
```

Q10. Can we open Trace Viewer without running tests locally?

Answer: Yes. If a trace ZIP file is available, we can open it directly using the Trace Viewer. This is particularly useful for investigating traces downloaded from CI/CD pipelines.

Q11. What is a DOM snapshot?

Answer: A DOM snapshot is a recorded representation of the webpage's DOM structure at a particular point in the test execution. It helps inspect elements and investigate locator-related failures.

Q12. What information can we inspect in Trace Viewer?

Answer:

- Actions and timeline.
- Screenshots.
- DOM snapshots.
- Logs.
- Source code.
- Network requests.
- Console messages.
- Errors.

Q13. What is the Network section used for?

Answer: It helps inspect network requests and responses made during the test, which is useful when investigating API or application loading failures.

Q14. What is the Console section used for?

Answer: It displays browser console messages, warnings, and errors captured during the trace.

Q15. What is the Source section?

Answer: It shows the test source code associated with the selected action, helping us identify which line of code was executed.

Scenario-based interview questions

Q16. Your test passes locally but fails in Jenkins. How will you debug it?

Answer:

1. Open the Jenkins execution report.
2. Download the failed test's trace artifact.
3. Open the ZIP using Trace Viewer.
4. Inspect the timeline and failed action.
5. Check the DOM snapshot, logs, and network requests.
6. Identify whether the failure is due to application behavior, timing, test data, or locator issues.

Q17. Your test fails while clicking the Add to Cart button. How will you investigate?

Answer: I will open Trace Viewer, select the failed click action, inspect the screenshot and DOM snapshot, verify the locator, and check the logs and network requests to understand why the click failed.

Q18. Why is Trace Viewer useful in CI/CD?

Answer: In CI/CD, tests usually run without a person watching the browser. Trace Viewer provides recorded execution details, so developers and testers can investigate failures after execution.

Q19. Does Trace Viewer execute tests?

Answer: No. Trace Viewer is primarily used to inspect recorded test execution. We run tests using the Playwright test runner and then open their traces.

Q20. What is the difference between an HTML Report and Trace Viewer?

Answer:

- HTML Report provides an overview of test execution, including passed, failed, skipped, and flaky tests.
- Trace Viewer provides detailed debugging information about an individual recorded execution.

Q21. Why should we not always record traces for every test execution?

Answer: Trace recording can increase execution overhead and consume storage. Recording traces on the first retry is a practical way to collect debugging information mainly for failures.

11. Quick revision notes

Term	Remember
Trace Viewer	GUI tool for debugging recorded tests
Trace file	`trace.zip`
`on-first-retry`	Record on first retry
`--trace on`	Force tracing on
`show-report`	Open HTML report
`show-trace`	Open trace ZIP
DOM Snapshot	Inspect webpage structure
Network	Inspect requests and responses
Console	Inspect browser messages and errors
Source	Inspect related test code
Timeline	Navigate through test actions

12. Test your knowledge

Trace Viewer quiz

0/6 answered

1. What is the main purpose of Trace Viewer?

Write test cases
Inspect recorded test execution
Install browsers
Generate locators only

2. Which file contains the recorded trace?

report.html
package.json
trace.zip
config.json

3. What does trace: 'on-first-retry' mean?

Record on every run
Record only when tests pass
Record on the first retry
Never record

4. Which command forces tracing on during a test run?

npx playwright test --trace on
npx playwright show-report
npx playwright --debug
npx playwright trace

5. Which section helps inspect API/network requests?

Source
Network
Timeline
DOM

6. Which command opens an existing trace ZIP?

npx playwright show-report

npx playwright test --debug
npx playwright show-trace trace.zip
npx playwright trace.zip
Submit answers

My interview preparation tip: Be particularly confident explaining `trace: 'on-first-retry'`, the difference between Trace Viewer and HTML Reporter, and how you would debug a test f

Playwright – Trace Viewer

Trace Viewer is one of the most important Playwright debugging features. It is especially useful when your automation test fails in Jenkins, GitHub Actions, or another CI/CD pipeline.

Let's understand it step by step with practical examples and interview questions.

1. What is Playwright Trace Viewer?

Playwright Trace Viewer is a GUI (Graphical User Interface) tool used to investigate and debug Playwright test executions.

It records what happened during a test and allows you to go back and inspect each action.

For example, suppose you are automating an e-commerce application:

1. Open the application.
2. Login.
3. Search for a mobile phone.
4. Select a product.
5. Add it to the cart.
6. Verify the cart.

Suppose the test fails at step 5 because the Add to Cart button was not clickable.

Using Trace Viewer, you can investigate:

- What happened before clicking the button?

- Was the button visible?
- Was it enabled?
- What did the page look like?
- Was there an error?
- What network requests were made?
- Which locator was used?

Best Practices | Playwright
playwright.dev

Simple definition for interviews:

Trace Viewer is a debugging tool in Playwright that records test execution details and allows us to inspect each action, DOM snapshot, logs, network requests, and errors.

2. What is a Trace?

A trace is a recorded history of a Playwright test execution.

Playwright stores this recording in a file called:

`trace.zip`

This file contains information about the test execution, such as:

- Actions performed.
- Screenshots.
- DOM snapshots.
- Test source code.
- Console logs.
- Network activity.
- Errors and execution details.

Think of it like a video recording of your automation test, but with additional technical information that helps you investigate failures.

Example:

Your test performs these actions:

```
test('add product to cart', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByRole('button', { name: 'Login' }).click();

  await page.getByPlaceholder('Search products').fill('iPhone');

  await page.getByRole('button', { name: 'Search' }).click();

  await page.getByRole('button', { name: 'Add to Cart' }).click();

});
```

A trace lets you inspect the execution of these actions one by one.

3. How to record a Trace?

Playwright provides different ways to record traces.

A. Configure Trace in playwright.config.ts

By default, the recommended configuration in the documentation is:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  retries: process.env.CI ? 2 : 0,

  use: {
    trace: 'on-first-retry',
  },

});
```

Let's understand this configuration.

What is retries?

Retries means how many times Playwright will retry a failed test.

```
retries: process.env.CI ? 2 : 0
```

This is a JavaScript conditional expression.

Condition	Retries
Running locally	0
Running in CI	2

Here, `process.env.CI` checks whether the CI environment variable is set.

For example:

- Local execution: If a test fails, it is not retried.
- CI execution: If a test fails, Playwright can retry it up to two times.

What is trace: 'on-first-retry'?

```
trace: 'on-first-retry'
```

It means Playwright records the trace when a failed test is retried for the first time.

Example:

Suppose a test fails during its first execution.

Example: Test retry and trace recording

1. First execution

Failed

Test fails. No trace is recorded for this initial attempt under this setting.

2. First retry

Trace recorded

Playwright records the trace during this retry.

3. Second retry

No new trace

No trace is recorded for this attempt under on-first-retry.

The trace from the first retry can then be inspected to understand the failure.

Important: on`-first-retry` does not record a trace during the initial execution. It records one on the first retry.

B. Record Trace locally using the command line

If you want to record traces locally without waiting for a retry, use:

```
npx playwright test --trace on
```

This forces tracing to be enabled for the test run.

For example:

```
npx playwright test login.spec.ts --trace on
```

This runs the login test file with tracing enabled.

Difference between the two approaches:

Configuration/command	When trace is recorded
`trace: 'on-first-retry'`	On the first retry
`--trace on`	During every test execution in that run
`trace: 'on'`	During every test execution
`trace: 'off'`	No trace recording

Other useful trace options include:

- `retain-on-failure` – retain traces for failed tests.
- `on-all-retries` – record traces on retries.
- `retain-on-first-failure` – retain the first failed attempt's trace.

For everyday debugging, `--trace on` is useful. For CI pipelines, `on-first-retry` is a common choice because it balances debugging information and storage usage.

4. What is CI/CD, and why are traces useful there?

CI/CD stands for Continuous Integration and Continuous Delivery/Deployment.

Examples of CI/CD tools:

- Jenkins
- GitHub Actions
- GitLab CI
- Azure DevOps

Suppose you have written 100 Playwright automation tests.

You push your code to GitHub. GitHub Actions automatically executes your tests.

But nobody is watching the browser.

Suppose 10 tests fail.

How will you understand what happened?

This is where Trace Viewer helps.

Debugging Playwright Failures in GitHub Actions | SentinelQA
sentinelqa.com

With traces, you can investigate the failed execution after the pipeline has finished.

Interview example:

"If my Playwright test fails in Jenkins, I check the test report and open the trace attached to the failed test. I inspect the action timeline, DOM snapshot, logs, and network requests to identify the root cause."

5. How to open the HTML Report?

Before opening a trace through the HTML report, first generate the test results.

Execute:

```
npx playwright test
```

Then open the HTML report:

```
npx playwright show-report
```

The HTML report displays:

- Passed tests.
- Failed tests.
- Skipped tests.
- Flaky tests.
- Browser information.
- Execution time.
- Error details.
- Test steps.
- Available traces.

Trace viewer | Playwright
playwright.dev

How to open a trace from HTML Report?

Follow these steps:

1. Execute your Playwright tests.

2. Run `npx playwright show-report`.
3. Open the failed test.
4. Look for the trace icon next to the test or open the detailed test view.
5. Go to the Traces tab.
6. Click the trace screenshot to open Trace Viewer.

Embedded Playwright Trace Viewer | TestDino
testdino.com

Important: A trace will only be available if tracing was enabled and the trace was retained for that execution.

6. How to open Trace Viewer directly?

You can also open a trace ZIP file directly.

Command:

```
npx playwright show-trace trace.zip
```

Here:

- `npx` executes the package.
- `playwright` is the Playwright package.
- `show-trace` opens the Trace Viewer.
- `trace.zip` is the recorded trace file.

This is useful when you have downloaded a trace artifact from Jenkins or GitHub Actions.

You can also open Trace Viewer online through the official Playwright Trace Viewer page:

Playwright Trace Viewer – Official website

You can load a trace ZIP file there to inspect it.

Security tip: Trace files can contain page content, screenshots, URLs, and other potentially sensitive information. Don't upload traces containing confidential application data to public tools unless your organization's policies permit it.

7. Understanding Trace Viewer interface

This is the most important part of the topic.

When you open a trace, you will see different sections that help you understand test execution.

Trace viewer | Playwright
playwright.dev

Let's understand the important components.

A. Actions / Timeline

The timeline shows the sequence of actions performed during test execution.

Example:

```
1. Navigate to application
2. Fill username
3. Fill password
4. Click Login
5. Verify dashboard
```

You can click each action to inspect what happened.

You can also hover over the timeline to inspect different moments of execution.

Why is it useful?

Suppose your test fails after clicking Login.

You can select the Login action and investigate what happened immediately before and after it.

B. Screenshots

Trace Viewer provides screenshots associated with actions.

You can inspect how the page looked during the execution.

For example:

- Was the Login button visible?
- Was a popup displayed?
- Was the page still loading?
- Was the wrong page opened?

This helps identify UI-related issues.

C. DOM Snapshot

DOM stands for Document Object Model.

The DOM represents the structure of an HTML webpage.

Example:

```
<html>
  <body>
    <h1>Login Page</h1>

    <input id="username" />

    <input id="password" />

    <button>Login</button>

  </body>
</html>
```

Trace Viewer captures DOM snapshots that allow you to inspect the page state at different points in the test.

Example:

Your locator is:

```
page.locator('#username')
```

But the test fails.

You can inspect the DOM snapshot and check whether the element actually has the ID `username`.

This helps identify incorrect locators or changes in the webpage structure.

D. Logs

Logs show information about actions performed during test execution.

For example:

```
navigating to "https://example.com"
waiting for locator("#username")
filling "komal"
clicking "Login"
```

They help you understand what Playwright was doing before the failure.

E. Source

The Source section shows the test source code associated with the selected action.

Example:

```
await page.locator('#username').fill('komal');

await page.locator('#password').fill('password123');

await page.getByRole('button', { name: 'Login' }).click();
```

You can identify which line of code corresponds to the selected action.

F. Network

The Network section shows network requests made during the test.

For example:

```
GET /login
POST /api/authenticate
GET /api/user-profile
```

You can investigate:

- Request URL.

- HTTP method.
- Response status.
- Request and response details, where available.
- Failed network requests.

Practical example:

Your login test fails because the application does not navigate to the dashboard.

You inspect the Network section and notice:

```
POST /api/authenticate → 401 Unauthorized
```

This indicates that the authentication request was rejected. You can investigate the credentials, request payload, or application behavior.

G. Console

The Console section displays browser console messages, including errors and warnings captured during the trace.

Example:

```
Uncaught TypeError: Cannot read properties of undefined
```

This can help identify JavaScript errors in the application.

H. Errors

The Errors section helps you identify errors associated with the test execution.

For example:

```
Error: Timed out waiting for expect(locator).toBeVisible()
```

You can use this information to identify the failed assertion or action.

8. Practical example – Debug an e-commerce test

Let's create a small example similar to a real QA automation project.

Scenario: Search for a product and add it to the cart.

```
import { test, expect } from '@playwright/test';

test('search product and add to cart', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByPlaceholder('Search products')
    .fill('Laptop');

  await page.getByRole('button', { name: 'Search' })
    .click();

  await page.getByRole('button', { name: 'Add to Cart' })
    .click();

  await expect(page.getByText('Added to cart'))
    .toBeVisible();

});
```

Run the test with tracing:

```
npx playwright test --trace on
```

Suppose the test fails at:

```
await page.getByRole('button', { name: 'Add to Cart' }).click();
```

How will you debug it?

1. Open the HTML report.
2. Select the failed test.
3. Open its trace.
4. Select the Add to Cart action.
5. Inspect the screenshot and DOM snapshot.
6. Check whether the button exists.
7. Inspect the logs and network requests.

8. Determine whether the problem is an incorrect locator, application issue, or failed API request.

This is how you use Trace Viewer in real automation projects.

9. Difference between UI Mode, Inspector and Trace Viewer

This is a very important interview question because these three tools are often confused.

Feature	UI Mode	Inspector	Trace Viewer
Main purpose	Run and analyze tests	Step-by-step debugging	Analyze recorded execution
Execution	Can run tests	Can debug tests	Views recorded traces
Timeline	Yes	Debugging controls	Yes
DOM snapshots	Yes	Locator inspection	Yes
Network details	Yes	Not its main purpose	Yes
Historical execution	Through available test results	Not its main purpose	Yes, through recorded trace
Typical command	`--ui`	`--debug`	`show-trace trace.zip`

Easy way to remember:

- UI Mode: Run and visually analyze tests.
- Inspector: Debug code step by step.

- Trace Viewer: Investigate what happened during a recorded test execution.

10. Interview Questions and Answers

Basic interview questions

Q1. What is Playwright Trace Viewer?

Answer: Trace Viewer is a GUI debugging tool in Playwright that allows us to inspect recorded test executions, including actions, screenshots, DOM snapshots, logs, network requests, and errors.

Q2. What is a trace file?

Answer: A trace file is a ZIP archive, generally named `trace.zip`, containing information recorded during Playwright test execution.

Q3. What is the purpose of Trace Viewer?

Answer: It helps identify the root cause of failed tests by allowing us to inspect the execution step by step.

Q4. How do you enable tracing in Playwright?

Answer: We can configure tracing in `playwright.config.ts`:

```
use: {
  trace: 'on-first-retry',
}
```

Or enable it from the command line:

```
npx playwright test --trace on
```

Q5. What is `on-first-retry`?

Answer: It records a trace when a failed test is executed for the first time as a retry.

Q6. What is the difference between `on-first-retry` and `on`?

Answer:

- `on-first-retry`: Records a trace on the first retry.

- on: Records a trace for every test execution.

Q7. What is the purpose of retries in Playwright?

Answer: Retries allow Playwright to execute a failed test again. They help identify intermittent failures and collect additional debugging information.

Practical interview questions

Q8. How do you open an HTML report?

Answer:

```
npx playwright show-report
```

Q9. How do you open a trace ZIP file directly?

Answer:

```
npx playwright show-trace trace.zip
```

Q10. Can we open Trace Viewer without running tests locally?

Answer: Yes. If a trace ZIP file is available, we can open it directly using the Trace Viewer. This is particularly useful for investigating traces downloaded from CI/CD pipelines.

Q11. What is a DOM snapshot?

Answer: A DOM snapshot is a recorded representation of the webpage's DOM structure at a particular point in the test execution. It helps inspect elements and investigate locator-related failures.

Q12. What information can we inspect in Trace Viewer?

Answer:

- Actions and timeline.
- Screenshots.
- DOM snapshots.
- Logs.
- Source code.

- Network requests.
- Console messages.
- Errors.

Q13. What is the Network section used for?

Answer: It helps inspect network requests and responses made during the test, which is useful when investigating API or application loading failures.

Q14. What is the Console section used for?

Answer: It displays browser console messages, warnings, and errors captured during the trace.

Q15. What is the Source section?

Answer: It shows the test source code associated with the selected action, helping us identify which line of code was executed.

Scenario-based interview questions

Q16. Your test passes locally but fails in Jenkins. How will you debug it?

Answer:

1. Open the Jenkins execution report.
2. Download the failed test's trace artifact.
3. Open the ZIP using Trace Viewer.
4. Inspect the timeline and failed action.
5. Check the DOM snapshot, logs, and network requests.
6. Identify whether the failure is due to application behavior, timing, test data, or locator issues.

Q17. Your test fails while clicking the Add to Cart button. How will you investigate?

Answer: I will open Trace Viewer, select the failed click action, inspect the screenshot and DOM snapshot, verify the locator, and check the logs and network requests to understand why the click failed.

Q18. Why is Trace Viewer useful in CI/CD?

Answer: In CI/CD, tests usually run without a person watching the browser. Trace Viewer provides recorded execution details, so developers and testers can investigate failures after execution.

Q19. Does Trace Viewer execute tests?

Answer: No. Trace Viewer is primarily used to inspect recorded test execution. We run tests using the Playwright test runner and then open their traces.

Q20. What is the difference between an HTML Report and Trace Viewer?

Answer:

- HTML Report provides an overview of test execution, including passed, failed, skipped, and flaky tests.
- Trace Viewer provides detailed debugging information about an individual recorded execution.

Q21. Why should we not always record traces for every test execution?

Answer: Trace recording can increase execution overhead and consume storage. Recording traces on the first retry is a practical way to collect debugging information mainly for failures.

11. Quick revision notes

Term	Remember
Trace Viewer	GUI tool for debugging recorded tests
Trace file	`trace.zip`
`on-first-retry`	Record on first retry
`--trace on`	Force tracing on

`show-report`	Open HTML report
`show-trace`	Open trace ZIP
DOM Snapshot	Inspect webpage structure
Network	Inspect requests and responses
Console	Inspect browser messages and errors
Source	Inspect related test code
Timeline	Navigate through test actions

12. Test your knowledge

Trace Viewer quiz

6/6 answered

1. What is the main purpose of Trace Viewer?

Write test cases
Inspect recorded test execution
Install browsers
Generate locators only

Correct

Trace Viewer helps investigate recorded test executions.

2. Which file contains the recorded trace?

report.html
package.json
trace.zip
config.json

Correct

Playwright stores trace data in a ZIP archive.

3. What does trace: 'on-first-retry' mean?

Record on every run
Record only when tests pass
Record on the first retry
Never record

Correct

The trace is recorded during the first retry of a failed test.

4. Which command forces tracing on during a test run?

npx playwright test --trace on
npx playwright show-report
npx playwright --debug
npx playwright trace

Correct

Use `--trace on` to enable tracing for the run.

5. Which section helps inspect API/network requests?

Source
Network
Timeline
DOM

Correct

The Network section displays recorded network activity.

6. Which command opens an existing trace ZIP?

npx playwright show-report
npx playwright test --debug
npx playwright show-trace trace.zip
npx playwright trace.zip

Correct

`show-trace` opens the recorded trace in Trace Viewer.

Score: 6/6

Try again

My interview preparation tip: Be particularly confident explaining `trace: 'on-first-retry'`, the difference between Trace Viewer and HTML Reporter, and how you would debug a test failure in Jenkins. These are useful practical topics for a QA Automation Engineer working with Playwright.

Next topic in your Playwright learning path: Running Playwright tests on CI with GitHub Actions.

ailure in Jenkins. These are useful practical topics for a QA Automation Engineer working with Playwright.

Next topic in your Playwright learning path: Running Playwright tests on CI with GitHub Actions.

Playwright – VS Code Extension

This section explains how to use Playwright directly inside Visual Studio Code (VS Code) to write, execute, debug, and generate automation tests.

For QA Automation Engineers, this is particularly useful because you don't need to execute every command manually in the terminal. You can manage most Playwright activities directly from the VS Code interface.

Let's understand each feature with practical examples and interview questions.

1. What is the Playwright VS Code Extension?

The Playwright VS Code Extension is an extension developed by Microsoft that integrates Playwright Test into Visual Studio Code.

It allows you to:

- Run individual test cases.
- Run all test cases.
- Debug tests.

- Set breakpoints.
- View test execution results.
- Generate tests using CodeGen.
- Pick locators.
- View Trace Viewer.
- Execute tests on multiple browsers.

VS Code | Playwright
playwright.dev

Interview definition:

The Playwright VS Code Extension integrates Playwright Test into VS Code and provides a graphical interface to run, debug, and generate automation tests.

2. Prerequisites

Before installing the extension, you need:

1. Node.js – LTS version recommended.
2. Visual Studio Code.
3. A Playwright project or the ability to create one.

Step 1: Install VS Code

Download:

Download Visual Studio Code

Step 2: Install Node.js

Download:

Download Node.js

Verify installation using:

```
node -v
npm -v
```

These commands display the installed Node.js and npm versions.

3. Install Playwright Extension

Follow these steps:

Run Playwright Tests in VS Code - AutomationTestingHub
automationtestinghub.com

1. Open VS Code.
2. Open the Extensions view.
3. Search for `Playwright`.
4. Find the official Playwright extension by Microsoft.
5. Click Install.

Shortcut:

```
Ctrl + Shift + X
```

This opens the Extensions panel.

You can also install it from:

Playwright Test for VS Code – Microsoft

4. Install Playwright using Command Palette

After installing the extension, you need to install Playwright in your project.

What is Command Palette?

Command Palette is a VS Code feature that allows you to search and execute commands.

Shortcut:

```
Ctrl + Shift + P
```

Steps:

1. Open VS Code.
2. Open your project folder.
3. Press `Ctrl + Shift + P`.
4. Search for `Test: Install Playwright`.
5. Select the command.
6. Choose the browsers you want to install.
7. Select whether you want to add a GitHub Actions workflow.

Playwright Test for VSCode - Visual Studio Marketplace
Open VSX Registry

You can select:

- Chromium
- Firefox
- WebKit

The extension sets up the Playwright project and its configuration.

Important: The VS Code extension and Playwright package are different things.

VS Code Extension	Playwright Package
Integrates Playwright into VS Code	Provides the test automation framework
Provides Testing sidebar	Provides test runner and browser automation APIs
Helps run and debug tests visually	Executes actual test code
Provides CodeGen integration	Provides Playwright functionality

5. Opening the Testing Sidebar

After installation, open the Testing icon in the VS Code Activity Bar.

VS Code | Playwright
playwright.dev

The Testing sidebar displays:

- Test cases.
- Test files.
- Playwright projects.
- Browser configurations.
- Test execution options.
- Debugging options.

For example, your project may look like this:

```
PlaywrightProject/
│
├── tests/
│   ├── login.spec.ts
│   ├── search.spec.ts
│   └── checkout.spec.ts
│
├── pages/
│   ├── LoginPage.ts
│   └── HomePage.ts
│
├── playwright.config.ts
└── package.json
```

The extension discovers tests from your Playwright configuration and displays them in the Testing sidebar.

6. Core Feature – Running Tests

There are different ways to execute tests directly from VS Code.

A. Run a Single Test

Suppose your file contains:

```
import { test, expect } from '@playwright/test';

test('verify login page', async ({ page }) => {

  await page.goto('https://example.com');

  await expect(page).toHaveTitle(/Example/);

});
```

You will see a green Play icon beside the test.

Getting started - VS Code | Playwright
cuketest.com

Click the green Play icon.

The extension:

1. Executes that particular test.
2. Displays a green checkmark if it passes.
3. Displays a red X if it fails.
4. Shows the execution duration.
5. Displays the Test Results panel.

Example result:

```
✓ verify login page
Duration: 2.5s
```

B. Run All Tests

You can run tests at different levels.

Action	How to execute

Run one test	Click Play beside the test
Run all tests in a file	Click Play beside the file
Run the complete project	Click Play at the top of Test Explorer

VS Code | Playwright
playwright.dev

Real-world example:

Suppose your project has 100 test cases.

Instead of executing all 100 cases every time, you can execute only the login test while developing it.

Once development is complete, run the complete test suite.

C. Run Tests on Multiple Browsers

Playwright allows you to execute tests on multiple browser projects.

In the Playwright sidebar, select the browser projects using the checkboxes.

For example:

Open VSX Registry
open-vsx.org

Suppose you select:

- Chromium
- Firefox
- WebKit

When you execute your tests, they run against the selected projects.

What is a Project in Playwright?

A project is a configuration that allows you to run the same tests with different settings.

For example:

```
projects: [
  {
    name: 'chromium',
    use: { browserName: 'chromium' },
  },
  {
    name: 'firefox',
    use: { browserName: 'firefox' },
  },
  {
    name: 'webkit',
    use: { browserName: 'webkit' },
  },
],
```

Each project represents a browser configuration.

A project can also have different viewport sizes, device emulation, or other settings.

D. Show Browser

By default, tests may execute in headless mode.

If you want to see the browser while tests execute, enable Show Browsers in the Playwright sidebar.

Enabling debugging functionalities in Playwright tests for VSCode. – TechNotes noraweisser.com

Show Browser	Execution
Enabled	Browser window is visible
Disabled	Browser runs in headless mode

This is useful when you want to visually observe how your automation script interacts with the application.

7. Debugging Tests in VS Code

Debugging means identifying and fixing errors in automation scripts.

The VS Code extension provides several debugging features.

A. Using Breakpoints

What is a breakpoint?

A breakpoint is a point in your code where test execution pauses temporarily.

Suppose you have this test:

```
test('login test', async ({ page }) => {

  await page.goto('https://example.com');

  await page.locator('#username').fill('komal');

  await page.locator('#password').fill('password123');

  await page.locator('#login-btn').click();

});
```

Suppose you want to investigate the username field.

Follow these steps:

1. Open the test file.
2. Click in the left gutter beside the line number where you want execution to pause.
3. A red dot appears.
4. Right-click the test.
5. Select Debug Test.
6. Execution pauses at the breakpoint.

[Debugging Tests | Playwright](playwright.dev)

Now you can inspect:

- Variable values.

- Current execution line.
- Call stack.
- Errors.
- Test execution flow.

Interview definition:

A breakpoint is used to pause test execution at a particular line so that we can inspect variables and debug the code.

B. Live Debugging

The extension also supports live debugging.

When Show Browsers is enabled, you can click a locator in your code and Playwright highlights the corresponding element in the browser.

Example:

```
await page.getByRole('button', { name: 'Login' }).click();
```

When you inspect this locator, the matching Login button can be highlighted in the browser.

Best Practices | Playwright
playwright.dev

Why is this useful?

Suppose your locator is:

```
page.locator('#login-btn')
```

You want to check whether it points to the correct button.

Live debugging helps you visually verify the element.

C. Viewing Error Messages

When a test fails, the extension displays detailed error information inside VS Code.

For example:

```
Error: expect(locator).toBeVisible()

Expected: visible
Received: hidden

Timeout: 5000ms
```

You can also inspect the call log to understand what Playwright was doing before the failure.

Debugging Tests | Playwright
playwright.dev

Example:

Your test expects a success message:

```
await expect(page.getByText('Login successful')).toBeVisible();
```

But the message is not displayed.

The error information helps you understand why the assertion failed.

D. Fix with AI

According to the supplied documentation, the extension also provides an AI-powered fix option through GitHub Copilot.

When a test fails:

1. Open the error details.
2. Click the sparkle icon beside the error.
3. Copilot analyzes the error.
4. It suggests a possible code change.

VS Code | Playwright
playwright.dev

Important: AI suggestions should always be reviewed. A suggested fix may not address the actual root cause.

This feature depends on the relevant Copilot functionality being available and configured.

8. Debugging with Trace Viewer in VS Code

The VS Code extension also integrates Trace Viewer.

You can enable Show Trace Viewer in the Playwright sidebar.

When your test finishes, the trace can automatically open if trace data is available.

[Advanced Playwright Dashboard Testing | Metaplay Docs](docs.metaplay.io)

Trace Viewer helps you inspect:

Feature	Purpose
Step-by-step analysis	Navigate through test actions
DOM inspection	Inspect webpage structure
Network monitoring	Inspect network requests and responses
Console logs	View console messages and errors
Source mapping	Navigate to the source code associated with actions
Visual debugging	Inspect screenshots from execution

Practical example:

Suppose your checkout test fails.

You can open the trace and inspect:

1. Product selection.
2. Add to Cart.

3. Cart page navigation.
4. Checkout button click.
5. Failed checkout assertion.

You can investigate exactly which step caused the failure.

For more details, refer to the previous Trace Viewer section.

9. Generating Tests with CodeGen in VS Code

What is CodeGen?

CodeGen is Playwright's test generation tool.

It records your browser interactions and automatically generates Playwright code.

For example, instead of manually writing every action, you can:

1. Open your application.
2. Click Login.
3. Enter username and password.
4. Click Submit.
5. Navigate through the application.

Playwright generates corresponding test code.

A. Record a New Test

In the Playwright sidebar, click Record new.

Test generator | Playwright
playwright.dev

A browser window opens.

Interact with your application, and Playwright generates code based on your actions.

Example generated code:

```
import { test, expect } from '@playwright/test';

test('test', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByRole('button', { name: 'Login' }).click();

  await page.getByPlaceholder('Username').fill('komal');

  await page.getByPlaceholder('Password').fill('password123');

  await page.getByRole('button', { name: 'Submit' }).click();

});
```

You can also generate assertions using the recording toolbar.

B. Record at Cursor

This is a useful feature when you already have a test case and want to add more actions.

Steps:

1. Open an existing test.
2. Place the cursor at the required position inside the test.
3. Click Record at cursor.
4. Perform browser actions.
5. Generated actions are added at the cursor position.

Playwright Codegen Test Recorder: Full Guide | Bondar Academy
bondaracademy.com

Example:

You already have:

```
test('checkout test', async ({ page }) => {

  await page.goto('https://example.com');

  // Add new browser actions here

});
```

You can use Record at Cursor to add the next steps without manually writing every locator and action.

C. Pick a Locator

Pick Locator allows you to select an element from the browser and generate a Playwright locator.

テストジェネレーター | Playwright
playwright.dokyumento.jp

Example:

You select the Login button.

Playwright may generate:

```
page.getByRole('button', { name: 'Login' });
```

You can copy and use the locator in your automation script.

Difference between Record New and Record at Cursor:

Record New	Record at Cursor
Creates a new recorded test	Adds actions to an existing test
Starts a new recording	Starts recording at the selected code position
Useful for creating tests from scratch	Useful for extending existing tests

10. Advanced Features

A. Project Dependencies

Project dependencies allow you to define setup tests that execute before other tests.

For example, suppose you have 20 tests that require login.

Without a shared authentication setup, every test might repeat the login process.

You can create a setup project that logs in once and saves the authenticated browser state.

Other projects can depend on that setup.

Example configuration:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  projects: [
    {
      name: 'setup',
      testMatch: /auth.setup.ts/,
    },

    {
      name: 'chromium',
      use: { browserName: 'chromium' },
      dependencies: ['setup'],
    },
  ],

});
```

The setup test can save authentication state, and the dependent tests can reuse it.

Why is it useful?

- Avoids repeating login steps.
- Keeps authentication setup separate.

- Makes test execution more organized.

Important: Project dependencies run setup tests before dependent projects. They do not automatically share a logged-in browser session. To reuse authentication, you must save and configure the storage state, for example using `storageState`.

B. Global Setup

Sometimes you need to execute an operation only once before the complete test suite.

Examples:

- Seed a test database.
- Create test data.
- Prepare an environment.
- Initialize shared resources.

For these situations, Playwright supports Global Setup.

In the VS Code sidebar, you can trigger Global Setup and teardown manually.

Difference between Project Dependencies and Global Setup:

Project Dependencies	Global Setup
Setup is defined as a test project	Setup is defined as a global setup function
Can be represented in Test Explorer	Can be triggered from the sidebar
Can use test-runner fixtures and assertions	Uses the global setup configuration mechanism
Can be used to prepare dependent projects	Used for suite-wide initialization

C. Multiple Configurations

Sometimes a project has multiple configuration files.

For example:

```
playwright.config.ts
playwright.staging.config.ts
playwright.production.config.ts
```

Each configuration can represent a different test environment or test suite.

In VS Code, use the gear icon in the Playwright sidebar to switch between configuration files.

VS Code | Playwright
playwright.dev

Practical example:

Configuration	Environment
`playwright.config.ts`	Development
`playwright.staging.config.ts`	Staging
`playwright.production.config.ts`	Production

This allows you to work with different test environments.

11. Quick Revision – VS Code Extension

Requirement	How to do it
Install extension	Extensions → Search Playwright
Install Playwright	Command Palette → Test: Install Playwright

Open Testing sidebar	Click Testing icon
Run one test	Click green Play icon
Run all tests	Click Play at top of Test Explorer
Run multiple browsers	Select browser projects
Show browser	Enable Show Browsers
Debug test	Set breakpoint → Debug Test
Verify locator	Live Debugging
View errors	Test Results panel
Generate test	Record new
Add actions to existing test	Record at cursor
Generate locator	Pick a Locator
View trace	Enable Show Trace Viewer

12. Interview Questions and Answers

Basic questions

Q1. What is the Playwright VS Code Extension?

Answer: It is an extension that integrates Playwright Test into VS Code and allows us to run, debug, and generate automation tests directly from the editor.

Q2. Who provides the official Playwright VS Code extension?

Answer: Microsoft.

Q3. What are the prerequisites for installing the extension?

Answer: Node.js and Visual Studio Code.

Q4. How do you install Playwright through VS Code?

Answer:

1. Install the Playwright extension.
2. Open Command Palette.
3. Execute `Test: Install Playwright`.
4. Select browsers.
5. Complete the setup.

Q5. What is the Testing sidebar?

Answer: It is a VS Code panel that displays discovered tests, projects, and test execution options.

Running tests

Q6. How do you execute a single test from VS Code?

Answer: Click the green Play icon next to the test case.

Q7. How do you execute all tests?

Answer: Click the Play icon at the top of the Test Explorer.

Q8. How do you run tests on multiple browsers?

Answer: Select the required browser projects using the checkboxes in the Playwright sidebar and execute the tests.

Q9. What is the Show Browsers option?

Answer: It displays the browser window during test execution. When disabled, tests can execute in headless mode.

Debugging

Q10. What is a breakpoint?

Answer: A breakpoint pauses test execution at a particular line so that we can inspect variables and execution flow.

Q11. How do you debug a test using VS Code?

Answer:

1. Open the test file.
2. Set a breakpoint.
3. Right-click the test.
4. Select Debug Test.
5. Inspect variables and execution.

Q12. What is live debugging?

Answer: Live debugging allows us to inspect locators in the code and highlight their corresponding elements in the visible browser.

Q13. How does the extension help when a test fails?

Answer: It displays detailed error messages, expected and received values, and call logs directly in the editor.

Q14. What is Fix with AI?

Answer: It uses GitHub Copilot to analyze test failures and suggest possible code changes.

Q15. How can you inspect a failed test using Trace Viewer in VS Code?

Answer: Enable Show Trace Viewer in the Playwright sidebar and inspect the recorded execution timeline, DOM snapshots, network requests, logs, and screenshots when a trace is available.

CodeGen

Q16. What is CodeGen?

Answer: CodeGen is a Playwright tool that records browser interactions and automatically generates test code.

Q17. What is Record New?

Answer: It starts a new browser recording and generates a new test based on user interactions.

Q18. What is Record at Cursor?

Answer: It records browser interactions and inserts the generated actions at the selected cursor position in an existing test.

Q19. What is Pick Locator?

Answer: It allows us to select an element in the browser and generate a corresponding Playwright locator.

Advanced questions

Q20. What are project dependencies?

Answer: Project dependencies allow one project, such as an authentication setup project, to execute before another project.

Q21. What is Global Setup?

Answer: Global Setup is used for initialization tasks that need to execute once before the test suite, such as preparing test data or seeding a database.

Q22. What is the purpose of multiple configurations?

Answer: They allow us to execute tests with different environments or settings, such as development, staging, and production.

13. Practical Interview Scenario

Question: You have 50 test cases in your Playwright project. You want to run only one test, debug it, and verify its locator. How will you do it in VS Code?

Answer:

1. Open the Playwright Testing sidebar.

2. Find the required test.
3. Click the green Play icon to execute it.
4. If it fails, set a breakpoint and select Debug Test.
5. Enable Show Browsers.
6. Inspect the locator using live debugging or Pick Locator.
7. Check the error details and trace to identify the root cause.
8. Fix the issue and execute the test again.

This workflow helps debug tests without repeatedly using terminal commands.

14. Test Your Knowledge

VS Code Extension Quiz

0/8 answered

1. Which shortcut opens the VS Code Extensions panel?

Ctrl + Shift + P
Ctrl + Shift + X
Ctrl + Alt + X
Ctrl + P

2. Which command installs Playwright through VS Code?

Test: Install Playwright
Playwright: Start
Install: Browser
Test: Run Playwright

3. How do you run one test from Test Explorer?

Click the green Play icon beside it
Restart VS Code
Open package.json
Use only the terminal

4. What is a breakpoint?

A browser setting
A point where execution pauses
A test report
A locator

5. Which feature generates code by recording browser actions?

Global Setup
Trace Viewer
CodeGen
Project Dependencies

6. What does Record at Cursor do?

Creates a new project
Adds recorded actions to an existing test at the cursor
Deletes a test
Opens the HTML report

7. What is the purpose of Project Dependencies?

Install browsers
Run setup projects before dependent projects
Generate reports
Change editor themes

8. Which option makes the browser visible during execution?

Show Trace Viewer
Show Browsers
Pick Locator
Global Setup
Submit answers

15. Most important points for your QA Automation interview

Focus especially on these topics:

1. How to install and configure the Playwright VS Code extension.
2. How to run individual tests and the complete test suite.

3. How to execute tests on multiple browser projects.
4. How to debug using breakpoints.
5. Difference between Record New and Record at Cursor.
6. How Pick Locator works.
7. Project Dependencies versus Global Setup.
8. How to inspect failed tests using Trace Viewer.

Practical suggestion: Try installing the extension and running one of your existing Playwright login or e-commerce tests from the Testing sidebar. Practising the workflow will make these interview answers much easier to explain.

Playwright Canary Releases

1. What is a Canary Release?

A Canary Release is an early, unreleased version of Playwright that allows developers and QA engineers to test newly implemented features before they become part of an official stable release.

Playwright publishes these releases daily under the `next` NPM tag.

Simple example:

Suppose Playwright is currently on stable version 1.x. The Playwright team develops a new feature for Trace Viewer.

Instead of waiting for the next official stable release, you can install the canary version and try that feature immediately.

You can then report bugs or provide feedback to the Playwright maintainers.

Why use Canary Releases?

1. Test new features early: Try features before their stable release.
2. Provide feedback: Report issues to Playwright maintainers.

3. Early compatibility testing: Check whether new features work with your automation framework.
4. Help improve Playwright: Your feedback helps maintainers identify problems.

2. Are Canary Releases Safe?

According to the documentation, using a canary release in production might seem risky, but in practice, it is not necessarily risky.

Why?

- Canary releases pass all automated tests.
- They are also used to test Playwright tools, such as:
    - HTML Reporter
    - Trace Viewer
    - Playwright Inspector
- These tools are tested using end-to-end tests.

Important: Passing automated tests does not guarantee that a canary release is completely bug-free. For critical projects, stable releases are generally the safer choice.

3. Next NPM Dist Tag

Playwright publishes different versions using NPM distribution tags (dist tags).

The documentation describes three important tags:

NPM tag	Meaning	Release frequency
`latest`	Stable releases	Official stable releases
`next`	Canary releases	Daily

`beta`	Releases after a release branch is created	Usually about a week before stable release

Understand with an example

Imagine Playwright is preparing a new stable release.

1. Latest

Stable version available for normal project use.

2. Next

Daily canary builds published for new code-related commits on the `main` branch.

3. Beta

Builds published after a release branch is created, usually around one week before the stable release.

Remember:

- `latest` = Stable
- `next` = Daily canary
- `beta` = Pre-release builds from the release branch

4. How to Install a Canary Release

The command given in the documentation is:

```
npm install -D @playwright/test@next
```

Let's understand each part:

Command	Meaning
`npm install`	Installs an NPM package
`-D`	Saves it as a development dependency

`@playwright/test`	Playwright Test package
`@next`	Installs the version published under the `next` tag

Example

Suppose you have an existing Playwright automation project.

Your current package installation uses:

```
npm install -D @playwright/test
```

To install the canary version, use:

```
npm install -D @playwright/test@next
```

You can then execute your tests:

```
npx playwright test
```

Practical QA recommendation: Try canary versions in a separate experimental branch or test project rather than immediately changing the dependency version in your main automation framework. Keep your stable version locked in your main project.

5. Canary Documentation

Playwright provides two documentation versions:

- Stable documentation: For officially released features.
- Next documentation: For upcoming features and unreleased changes.

According to the documentation you shared, to view the `next` documentation:

1. Open Playwright documentation.
2. Press the `Shift` key five times.
3. You can access the next documentation.

This helps you learn about upcoming features before they are officially released.

6. Real-Time QA Project Scenario

Imagine you are working on an e-commerce automation framework using Playwright and TypeScript.

Your framework uses:

- Playwright Test
- Page Object Model
- TypeScript
- Allure Reports
- Screenshots and Trace Viewer

The Playwright team introduces a new feature related to Trace Viewer.

As a QA automation engineer, you can:

1. Install the canary version using `@next`.
2. Try the new Trace Viewer feature.
3. Execute your existing automation test cases.
4. Check whether the feature works correctly.
5. Report any bugs or unexpected behavior to the Playwright team.

This allows you to explore upcoming functionality before the stable release.

7. Interview Questions and Answers

Q1. What is a Canary Release in Playwright?

A canary release is an unreleased version of Playwright that allows users to test new features before they are included in a stable release.

Q2. How frequently are Playwright canary releases published?

They are published daily under the `next` NPM tag.

Q3. What is the NPM `next` tag?

It identifies the latest canary release of Playwright.

Q4. How do you install a Playwright canary release?

```
npm install -D @playwright/test@next
```

Q5. What is the difference between `latest` and `next`?

- `latest` represents stable releases.
- `next` represents daily canary releases.

Q6. What is the `beta` tag?

It identifies pre-release builds published after a release branch is created, usually about a week before a stable release.

Q7. Why should QA engineers use canary releases?

To test upcoming features, identify bugs early, verify compatibility, and provide feedback to maintainers.

Q8. Are canary releases tested?

Yes. According to the documentation, they pass automated tests and are used to test Playwright tools through end-to-end tests.

Q9. What is the difference between a stable release and a canary release?

A stable release is an official release intended for normal use. A canary release provides early access to unreleased changes.

Q10. Where can you find the next version of Playwright documentation?

At playwright.dev, using the documentation's `next` version switch described in the source.

8. Quick Revision Notes

Topic	Remember
Canary release	Early, unreleased version

NPM tag	`next`
Release frequency	Daily
Stable tag	`latest`
Beta tag	`beta`
Install command	`npm install -D @playwright/test@next`
Purpose	Test new features and provide feedback
Tools tested	HTML Reporter, Trace Viewer, Inspector

9. Quick Interview Quiz

Test your understanding

0/4 answered

1. Which NPM tag is used for Playwright canary releases?

latest
next
beta
stable

2. Which command installs the canary version?

npm install @playwright/test
npm install -D @playwright/test@next
npx playwright install@latest
npm update playwright

3. How frequently are canary releases published?

Daily
Weekly

Monthly
Yearly

4. What does the latest tag represent?

Canary release
Beta release
Stable release
Development branch
Submit answers

Most important interview point: Remember the three NPM tags — `latest`, `next`, and `beta` — and the canary installation command. These are the key facts from this section.

Playwright Test Agents

1. What are Playwright Test Agents?

Playwright provides three built-in AI-powered Test Agents:

1. 🎭 Planner – Explores the application and creates a test plan.
2. 🎭 Generator – Converts the test plan into executable Playwright test files.
3. 🎭 Healer – Executes failing tests and attempts to repair them automatically.

The main purpose of these agents is to reduce manual effort in creating, maintaining, and debugging automation test cases.

Simple example:

Suppose you are testing an e-commerce application. You want to automate the guest checkout functionality.

Traditionally, you would:

- Understand the application.
- Identify test scenarios.
- Write Playwright test scripts.

- Execute the tests.
- Debug failures and update locators.

With Playwright Test Agents, these activities can be assisted by three agents.

How the three agents work

Planner

Explore application and create test plan

Output: Markdown test plan

Generator

Convert test plan into Playwright scripts

Output: Executable test files

Healer

Execute tests and attempt to fix failures

Output: Passing test or skipped test

These agents can be used independently, sequentially, or chained together in an agentic loop.

2. Getting Started – Initialize Playwright Agents

Before using the agents, you need to add their definitions to your Playwright project.

The documentation provides this command:

```
npx playwright init-agents --loop=vscode
```

Understanding the command

Part	Meaning
`npx`	Executes a package command
`playwright`	Playwright CLI
`init-agents`	Initializes agent definitions
`--loop=vscode`	Configures the agent workflow for VS Code

The documentation also lists support for these AI coding environments:

- VS Code
- Claude Code
- Codex
- OpenCode

Important: The generated agent definitions should be regenerated whenever you update Playwright so that they include the latest tools and instructions.

VS Code requirement

The documentation specifies VS Code version 1.105, released October 9, 2025, for the agentic experience to function properly in VS Code.

After initialization, you can use your AI tool to instruct the agents to create Playwright tests.

3. 🎭 Planner Agent

What is Planner?

The Planner agent explores your application and creates a Markdown test plan for one or more scenarios or user flows.

Think of Planner as a QA engineer who explores an application and prepares test scenarios before automation begins.

Input required by Planner

The documentation mentions three inputs:

1. Clear request

You need to tell the Planner what you want to test.

Example:

```
Generate a test plan for guest checkout.
```

2. Seed test

A seed test prepares the environment required to interact with your application.

3. Product Requirement Document (PRD) – Optional

A PRD provides information about the product requirements and expected functionality.

For example, a PRD might describe:

- Users should be able to search for products.
- Users should be able to add products to the cart.
- Guest users should be able to checkout.
- Users should receive an order confirmation.

What is a seed test?

A seed test is a test file that initializes the environment and provides a ready-to-use page context for the Planner.

Example from your documentation:

```
import { test, expect } from './fixtures';

test('seed', async ({ page }) => {
  // This test uses custom fixtures from ./fixtures
});
```

Let's understand it:

Code	Explanation
`import { test, expect }`	Imports Playwright test functions
`./fixtures`	Imports custom fixtures
`test('seed')`	Defines the seed test
`{ page }`	Receives the Playwright page fixture

What does Planner do with the seed test?

According to the documentation:

1. Planner includes `seed.spec.ts` in its context.
2. Planner runs the seed test.
3. It executes the necessary initialization.
4. This includes global setup, project dependencies, fixtures and hooks.
5. It uses the seed test as an example for generated tests.

You can also mention the seed file name directly in your prompt.

Output of Planner

Planner generates a Markdown test plan, for example:

```
specs/
   basic-operations.md
```

Example of what a test plan might contain:

```
# Guest Checkout Test Plan

## Scenario 1: Add product to cart

1. Open the application.
2. Search for a product.
3. Select a product.
4. Add it to cart.
5. Verify the cart contains the product.

## Scenario 2: Guest checkout

1. Open the cart.
2. Proceed to checkout.
3. Enter delivery details.
4. Select payment method.
5. Verify order confirmation.
```

Illustrative example of a test plan, not a literal generated output from the documentation.

Remember: Planner creates the plan; it does not produce the final executable test suite.

4. 🎭 Generator Agent

What is Generator?

The Generator agent converts the Markdown test plan into executable Playwright test files.

Think of Generator as an automation tester who converts manual test scenarios into automation scripts.

Input required

Generator takes the Markdown test plan from the `specs/` directory.

For example:

`specs/basic-operations.md`

You can instruct it:

`Generate Playwright tests using specs/basic-operations.md.`

What does Generator do?

1. Reads the Markdown test plan.
2. Understands the test scenarios and steps.
3. Interacts with the application.
4. Verifies selectors and assertions while performing scenarios.
5. Generates Playwright test files.

The documentation also mentions generation hints and a catalog of assertions to support efficient structural and behavioral validation.

Output of Generator

It creates test files under the `tests/` directory.

Example:

```
tests/
   create/
      add-valid-todo.spec.ts
```

An illustrative generated test might look like this:

```
import { test, expect } from '@playwright/test';

test('add product to cart', async ({ page }) => {

  await page.goto('https://example.com');
```

```
  await page.getByRole('textbox', {
    name: 'Search'
  }).fill('Laptop');

  await page.getByRole('button', {
    name: 'Search'
  }).click();

  await page.getByRole('button', {
    name: 'Add to cart'
  }).first().click();

  await expect(
    page.getByText('Added to cart')
  ).toBeVisible();

});
```

Illustrative example. Actual generated code and locators depend on the application.

Important: Generated tests may initially contain errors. The Healer agent can attempt to repair them.

5. 🎭 Healer Agent

What is Healer?

The Healer agent executes the generated test suite and attempts to repair failing tests automatically.

Think of Healer as an automation maintenance engineer.

Input required

The Healer agent takes the name of a failing test.

Example:

```
Fix the failing test: add-valid-todo.spec.ts
```

What does Healer do?

According to the documentation:

1. Replays failing steps: Repeats the actions that caused the failure.
2. Inspects the current UI: Checks the current application interface.
3. Finds equivalent elements or flows: Attempts to identify updated locators or alternative interactions.
4. Suggests a patch: For example, a locator update, wait adjustment or test-data fix.
5. Re-runs the test: Checks whether the proposed repair works.

Example: Locator failure

Suppose your original test contains:

```
await page.locator('#loginBtn').click();
```

But the application changes and the button ID becomes `#submitLogin`.

The test fails because the old locator no longer matches.

The Healer may inspect the UI, identify the updated button and suggest a replacement locator.

For example:

```
await page.getByRole('button', {
  name: 'Login'
}).click();
```

It then reruns the test.

Output of Healer

There are two possible outcomes:

1. Passing test

The proposed repair works and the test passes.

2. Skipped test

The Healer believes the functionality is broken and skips the test.

Important interview point: Healer does not guarantee that every failure can be fixed. It operates within guardrails and may stop if it cannot safely repair the test.

Also, a passing test after healing does not automatically prove that the application is correct. A QA engineer should review the suggested changes and verify that the original business requirement is still being tested.

6. Artifacts and Folder Structure

After initializing and using the agents, your project follows a structure similar to this:

```
repo/
│
├── .github/
│   └── agent definitions
│
├── specs/
│   └── basic-operations.md
│
├── tests/
│   ├── seed.spec.ts
│   │
│   └── tests/
│       └── create/
│           └── add-valid-todo.spec.ts
│
└── playwright.config.ts
```

Let's understand each folder.

Folder/File	Purpose
`.github/`	Stores agent definitions
`specs/`	Stores human-readable test plans

`basic-operations.md`	Contains test scenarios and expected results
`tests/`	Stores generated Playwright tests
`seed.spec.ts`	Initializes the environment
`playwright.config.ts`	Contains Playwright configuration

Agent Definitions

Agent definitions are collections of:

- Instructions
- MCP tools

They are provided by Playwright and should be regenerated whenever Playwright is updated.

Specs Directory

The `specs/` directory contains structured test plans.

These plans include:

- Scenarios
- Steps
- Expected outcomes
- Test data

They can be created from scratch or extended from a seed test.

Tests Directory

The `tests/` directory contains generated executable Playwright tests.

The documentation says tests should align one-to-one with specifications wherever feasible.

Seed Test

The `seed.spec.ts` file provides a ready-to-use page context to bootstrap execution.

It helps initialize the application and provides a pattern for generated tests.

7. Real-World QA Automation Project Example

Let's connect this topic with an e-commerce automation project.

Suppose you are automating an e-commerce application with login, product search, cart and checkout functionality.

Scenario: Automate product purchase

1. Planner

Creates scenarios for product search, adding to cart and checkout.

Output: `specs/ecommerce-checkout.md`

2. Generator

Converts those scenarios into Playwright TypeScript test scripts.

Output: `tests/checkout.spec.ts`

3. Healer

Executes the tests and attempts to repair failures, such as outdated locators.

Output: Repaired passing test or skipped test

What is the benefit for a QA engineer?

- Faster test creation.
- Less repetitive manual scripting.

- Assistance with locator maintenance.
- Faster investigation of certain test failures.
- Better starting point for automation coverage.

However, the QA engineer remains responsible for test quality, business validation, test-data correctness and reviewing AI-generated changes.

8. Important Interview Questions and Answers

Basic questions

Q1. What are Playwright Test Agents?

Playwright Test Agents are built-in agents that help explore applications, generate Playwright tests and repair failing tests.

Q2. How many built-in Playwright Test Agents are available?

Three: Planner, Generator and Healer.

Q3. What is the role of Planner?

Planner explores the application and creates a human-readable Markdown test plan.

Q4. What is the role of Generator?

Generator converts the Markdown test plan into executable Playwright test files.

Q5. What is the role of Healer?

Healer executes failing tests, investigates the UI and attempts to repair the failures.

Q6. Can these agents work independently?

Yes. They can be used independently, sequentially or chained together in an agentic loop.

Practical questions

Q7. How do you initialize Playwright Test Agents?

```
npx playwright init-agents --loop=vscode
```

Q8. What is a seed test?

A seed test initializes the environment required for the agents to interact with the application. It also acts as an example for generated tests.

Q9. What is the purpose of a PRD?

A Product Requirement Document provides additional product and business context to help Planner prepare relevant test scenarios.

Q10. What is the output of Planner?

A Markdown test plan saved under the `specs/` directory.

Q11. What is the output of Generator?

Executable Playwright test files saved under the `tests/` directory.

Q12. What happens if Generator creates a failing test?

The Healer agent can investigate the failure and attempt to repair the test.

Q13. What happens if Healer cannot repair a test?

It may stop because of its guardrails. The documentation also describes a skipped test as an output when the agent believes the functionality is broken.

Q14. What is stored inside `.github/`?

Static agent definitions containing instructions and MCP tools.

Q15. Why should agent definitions be regenerated after updating Playwright?

To pick up the latest tools and instructions provided by Playwright.

Scenario-based questions

Q16. Your application changes a button ID and your test fails. Which agent can help?

Healer. It can inspect the UI and attempt to identify an equivalent element or updated locator.

Q17. You have manual test scenarios and want to convert them into automation scripts. Which agent should you use?

Generator, after preparing a Markdown test plan.

Q18. You want to explore the application and prepare test scenarios automatically. Which agent should you use?

Planner.

Q19. Can Healer fix an actual application defect?

No. It is intended to help repair failing tests. If the application functionality is genuinely broken, the issue needs to be investigated and fixed by the development team.

Q20. Do Playwright Test Agents replace QA engineers?

No. They assist with test planning, test generation and test maintenance. QA engineers still need to validate business requirements, review generated tests and investigate actual application defects.

9. Quick Revision Notes

Agent	Main responsibility	Input	Output
Planner	Explore application	Request, seed test, optional PRD	Markdown plan
Generator	Create automation tests	Markdown plan	Playwright test files
Healer	Repair failing tests	Failing test name	Passing or skipped test

Important paths and commands

Item	Remember
Initialize agents	`npx playwright init-agents --loop=vscode`

Agent definitions	`.github/`
Test plans	`specs/`
Generated tests	`tests/`
Seed test	`seed.spec.ts`
Configuration	`playwright.config.ts`
VS Code requirement	Version 1.105

10. Test Your Knowledge

Playwright Test Agents Quiz

0/6 answered

1. Which agent explores the application and creates a Markdown test plan?

Generator
Healer
Planner
Reporter

2. Which agent converts Markdown plans into executable Playwright tests?

Planner
Generator
Healer
Inspector

3. What is the purpose of a seed test?

Generate HTML reports
Initialize the environment for agent execution
Repair failed locators
Install browsers

4. Where are human-readable test plans stored?

.github/
specs/
node_modules/
playwright-report/

5. A test fails because a locator has changed. Which agent can attempt to repair it?

Planner
Generator
Healer
Seed

6. Which command initializes agents for the VS Code loop?

npx playwright test --agents
npx playwright init-agents --loop=vscode
npm install -D @playwright/agents
npx playwright codegen --loop
Submit answers

Interview tip: Remember this sequence:

Planner → Generator → Healer

- Planner = Plan
- Generator = Generate
- Healer = Heal

This is the most important concept to understand from this documentation section.

Playwright Annotations

1. What are Annotations in Playwright?

Playwright supports tags and annotations to add additional information to test cases.

They help QA engineers:

- Skip tests that are not applicable.

- Mark tests that are expected to fail.
- Identify tests that need fixing.
- Increase the timeout for slow tests.
- Categorize tests.
- Attach issue links and additional information.
- Filter and execute specific tests.

Annotations and tags are displayed in the test report.

Simple example:

Suppose you have an e-commerce automation framework with 100 test cases.

You want to:

- Skip mobile tests that are not ready.
- Mark a known failing test.
- Identify slow checkout tests.
- Run only smoke tests.

Playwright provides built-in annotations and tags to handle these situations.

2. Built-in Annotations

Playwright provides four important built-in annotations:

1. `test.skip()`

Skip

Marks a test as irrelevant. Playwright does not execute it.

Use when a test is not applicable in a particular configuration.

```
test.skip('mobile checkout', async ({ page }) => {
  // This test will not run
});
```

2. `test.fail()`

Expected failure

Marks a test as expected to fail. Playwright still executes it and checks whether it actually fails.

```
test.fail('known issue test', async ({ page }) => {
  expect(1).toBe(2);
});
```

If the test fails as expected, Playwright treats it as an expected failure. If it passes unexpectedly, Playwright reports that as a problem.

3. `test.fixme()`

Playwright Command Line (CLI)

1. What is Playwright CLI?

CLI stands for Command Line Interface.

Playwright provides a powerful command-line interface that allows QA automation engineers to:

- Run Playwright tests.
- Execute specific test files or test cases.
- Run tests in different browsers.
- Debug failing tests.
- Generate automation scripts.
- Open HTML reports and Trace Viewer.
- Install browsers.
- Merge test reports.
- Manage Playwright cache.

You can execute these commands from the VS Code terminal or any supported command-line terminal.

Important command:

```
npx playwright --help
```

This displays the latest available Playwright CLI commands and arguments.

Pasted markdown(2).md

2. Essential Commands

A. Run Tests

The most commonly used Playwright command is:

```
npx playwright test
```

It executes all tests in your Playwright project.

Different ways to execute tests

1. Run all tests

```
npx playwright test
```

Executes all tests discovered by Playwright.

2. Run a single test file

```
npx playwright test tests/todo-page.spec.ts
```

Executes only the specified test file.

3. Run multiple test folders

```
npx playwright test tests/todo-page/ tests/landing-page/
```

Executes tests from the specified folders.

4. Run tests at a specific line

```
npx playwright test my-spec.ts:42
```

Executes the test located at or associated with line 42.

5. Run tests by title

```
npx playwright test -g "add a todo item"
```

The `-g` option filters tests by their title using a regular expression.

6. Run tests in headed mode

```
npx playwright test --headed
```

Opens the browser UI while executing tests.

7. Run tests in a specific browser project

```
npx playwright test --project=chromium
```

Executes tests only in the Chromium project.

8. Get help for test commands

```
npx playwright test --help
```

Displays available test-running options.

Pasted markdown(2).md

More useful execution commands

Command	Purpose
`npx playwright test`	Run all tests
`npx playwright test --workers=1`	Run using one worker
`npx playwright test --debug`	Debug using Playwright Inspector
`npx playwright test --ui`	Open interactive UI Mode

`npx playwright test --headed`	Run with visible browser
`npx playwright test --last-failed`	Run only previously failing tests
`npx playwright test --list`	List tests without executing them
`npx playwright test -x`	Stop after the first failure
`npx playwright test --retries=2`	Retry failed tests up to two times

Pasted markdown(2).md
Pasted markdown(2).md

B. Important CLI Options

Let's understand the options that are particularly useful in QA automation interviews.

Option	Explanation
`--headed`	Runs tests with visible browsers
`--debug`	Opens Playwright Inspector for debugging
`--ui`	Opens interactive UI Mode
`--grep`	Runs tests matching a title pattern
`--grep-invert`	Runs tests that do not match a pattern
`--project`	Runs a specific browser project
`--workers`	Controls parallel workers

`--retries`	Sets the number of retries
`--last-failed`	Runs only previously failed tests
`--max-failures`	Stops after a specified number of failures
`--timeout`	Sets the test timeout in milliseconds
`--trace`	Controls trace recording
`--reporter`	Selects the test reporter
`--forbid-only`	Fails if `test.only()` is used
`--fully-parallel`	Runs all tests in parallel

Pasted markdown(2).md

Understand some important options with examples

1. Workers

```
npx playwright test --workers=1
```

Runs tests using one worker instead of parallel workers.

Why is this useful?

Suppose two tests are modifying the same user account or test data. Running them simultaneously might create conflicts. Running with one worker can help investigate such issues.

2. Retries

```
npx playwright test --retries=2
```

If a test fails, Playwright can retry it up to two times.

3. Stop after first failure

```
npx playwright test -x
```

Useful when you want to stop execution immediately after the first failure.

4. Run only failed tests

```
npx playwright test --last-failed
```

Useful when a large test suite has multiple failures and you want to rerun only those tests.

5. Run tests in a specific project

```
npx playwright test --project=chromium
```

If your configuration contains Chromium, Firefox and WebKit projects, this command selects Chromium only.

6. Filter tests using title

```
npx playwright test --grep "login"
```

Runs tests whose titles match `login`.

3. Test List

Playwright also supports `--test-list` and `--test-list-invert`.

These options allow you to control which tests are executed using a test list file.

A. `--test-list`

Runs only tests included in the specified list.

```
npx playwright test --test-list=tests-to-run.txt
```

Example contents of `tests-to-run.txt`:

```
# Run all tests in a file
tests/login.spec.ts

# Run a particular test
tests/cart.spec.ts › add product to cart
```

B. `--test-list-invert`

Runs tests that are not included in the specified test list.

```
npx playwright test --test-list-invert=tests-to-skip.txt
```

Where is this useful?

Imagine you have 100 automation test cases, but you want to execute only 10 selected cases.

You can maintain those test names in a list file and execute that list.

The documentation supports entries for entire files, test groups, nested groups and fully qualified tests.

Pasted markdown(2).md

4. Show HTML Report

After executing tests, you may want to view the test execution results.

Use:

```
npx playwright show-report
```

This opens the HTML report from the previous test run.

Other examples

Open a specific report directory:

```
npx playwright show-report playwright-report/
```

Open the report on a custom port:

```
npx playwright show-report --port 8080
```

The default port is `9323`.

You can also specify the host:

```
npx playwright show-report --host localhost
```

Pasted markdown(2).md

Practical QA scenario

Suppose you execute 100 test cases:

- 85 passed.
- 10 failed.
- 5 skipped.

You can open the HTML report to inspect the test results and investigate failures.

Remember:

- `npx playwright test` → Executes tests.
- `npx playwright show-report` → Displays the HTML report from the previous run.

5. Install Browsers

Playwright supports three major browser engines:

- Chromium
- Firefox
- WebKit

You can install the browsers required by your project using the CLI.

A. Install all browsers

```
npx playwright install
```

B. Install only Chromium

```
npx playwright install chromium
```

C. Install specific browsers

```
npx playwright install chromium webkit
```

D. Install browsers with system dependencies

```
npx playwright install --with-deps
```

This also installs the browser system dependencies needed by Playwright.

Other installation commands

Command	Purpose
`npx playwright install`	Install all supported browsers
`npx playwright install chromium`	Install Chromium
`npx playwright install firefox`	Install Firefox
`npx playwright install webkit`	Install WebKit
`npx playwright install --with-deps`	Install browsers and system dependencies
`npx playwright install --force`	Force reinstall browsers
`npx playwright install --list`	List browsers from Playwright installations
`npx playwright uninstall`	Uninstall Playwright browsers

Pasted markdown(2).md

Interview point: Playwright uses browser binaries compatible with its installed version. After installing or updating Playwright, you may need to install the corresponding browsers.

6. Generation and Debugging Tools

A. Code Generation

Playwright Codegen records your browser interactions and generates automation code.

Command:

```
npx playwright codegen
```

It opens an interactive browser and recording interface.

Generate code for a specific website

```
npx playwright codegen https://playwright.dev
```

Generate Python code

```
npx playwright codegen --target=python
```

Generate code into a file

```
npx playwright codegen -o tests/generated.spec.ts
```

Important Codegen options

Option	Purpose
`-b` or `--browser`	Select browser
`-o` or `--output`	Specify output file
`--target`	Select programming language
`--test-id-attribute`	Specify test ID attribute

Example:

```
npx playwright codegen --browser=firefox --target=playwright-test
```

This launches Codegen using Firefox and generates Playwright Test code.

Pasted markdown(2).md

Real QA use case: You want to automate a login page but are unsure about the correct locators. Use Codegen to record the login flow and generate an initial script. Review and improve the generated code before adding it to your framework.

B. Trace Viewer

Trace Viewer helps investigate test execution.

Command:

```
npx playwright show-trace trace.zip
```

It opens the Trace Viewer with the specified trace file.

Other commands:

```
# Open Trace Viewer without a specific trace
npx playwright show-trace

# Open trace from a directory
npx playwright show-trace trace/
```

You can also specify the browser, host and port.

Pasted markdown(2).md

Difference between HTML Report and Trace Viewer

HTML Reporter	Trace Viewer
Shows overall test results	Investigates individual test execution
Displays passed and failed tests	Displays recorded execution details
Helps identify failed test cases	Helps understand why a test failed

Open using `show-report` Open using `show-trace`

Example:

```
npx playwright show-report
npx playwright show-trace trace.zip
```

7. Specialized Commands

A. Merge Reports

Playwright supports merging Blob reports.

This is useful when tests run across multiple workers or CI shards and you want to combine their results.

Command:

```
npx playwright merge-reports ./reports
```

Here, `./reports` is the directory containing the Blob reports.

You can also specify a reporter:

```
npx playwright merge-reports ./reports --reporter=html
```

The available reporter choices include `list`, `line`, `dot`, `json`, `junit`, `null`, `github`, `html`, `blob` and `perfetto`.

Pasted markdown(2).md

Practical example

Imagine your CI pipeline runs tests in three shards:

- Shard 1 executes login tests.
- Shard 2 executes cart tests.
- Shard 3 executes checkout tests.

Each shard generates a Blob report. You can merge the reports to create a combined report.

B. Clear Cache

Command:

```
npx playwright clear-cache
```

This clears Playwright caches.

Pasted markdown(2).md

8. Most Important Commands for QA Interviews

Here is a quick reference table to revise before your interview.

Requirement	Command
Run all tests	`npx playwright test`
Run one file	`npx playwright test tests/login.spec.ts`
Run by title	`npx playwright test -g "login"`
Run headed	`npx playwright test --headed`
Run in Chromium	`npx playwright test --project=chromium`
Run in UI mode	`npx playwright test --ui`
Debug tests	`npx playwright test --debug`

Run with one worker	`npx playwright test --workers=1`
Retry failed tests	`npx playwright test --retries=2`
Run previous failures	`npx playwright test --last-failed`
Stop after first failure	`npx playwright test -x`
Show HTML report	`npx playwright show-report`
Install browsers	`npx playwright install`
Generate code	`npx playwright codegen`
Open trace	`npx playwright show-trace trace.zip`
Merge reports	`npx playwright merge-reports ./reports`
Clear cache	`npx playwright clear-cache`
See CLI help	`npx playwright --help`

9. Interview Questions and Answers

Q1. What is Playwright CLI?

Playwright CLI is a command-line interface used to execute tests, generate scripts, debug tests, install browsers and manage reports.

Q2. How do you run all Playwright tests?

```
npx playwright test
```

Q3. How do you run a specific test file?

```
npx playwright test tests/login.spec.ts
```

Q4. How do you run a test using its title?

```
npx playwright test -g "login"
```

Q5. What is the difference between `--headed` and `--ui`?

- `--headed` runs tests in a visible browser.
- `--ui` opens Playwright's interactive test UI for running and debugging tests.

Q6. How do you run tests in Chromium only?

```
npx playwright test --project=chromium
```

Q7. How do you execute tests using a single worker?

```
npx playwright test --workers=1
```

Q8. How do you rerun only failed tests?

```
npx playwright test --last-failed
```

Q9. How do you stop test execution after the first failure?

```
npx playwright test -x
```

Q10. How do you configure retries from the CLI?

```
npx playwright test --retries=2
```

Q11. How do you open the HTML report?

```
npx playwright show-report
```

Q12. How do you generate automation scripts using Codegen?

```
npx playwright codegen
```

Q13. How do you open Trace Viewer?

```
npx playwright show-trace trace.zip
```

Q14. How do you install only Firefox?

```
npx playwright install firefox
```

Q15. What is the use of `--grep-invert`?

It excludes tests matching the specified regular expression.

```
npx playwright test --grep-invert "@slow"
```

Q16. What is the use of `--forbid-only`?

It makes the test run fail if `test.only()` is present. This is especially useful in CI to prevent accidentally committing focused tests.

Q17. What is the use of `--list`?

It collects and displays discovered tests without executing them.

```
npx playwright test --list
```

Q18. What is the use of `--shard`?

It allows you to execute a portion of the test suite.

```
npx playwright test --shard=1/3
```

This selects the first shard out of three.

Q19. What is the use of `merge-reports`?

It combines Blob reports generated by separate test runs or shards.

Q20. How can you find all available Playwright CLI commands?

```
npx playwright --help
```

10. Quick Practice Quiz

Playwright CLI Quiz

8/8 answered

1. Which command executes all Playwright tests?

npx playwright run
npx playwright test
npx playwright execute

npm playwright start

Correct! `npx playwright test` runs the test suite.

2. Which option runs tests with a visible browser?

--ui
--debug
--headed
--visible

Correct! --headed runs tests in headed browsers.

3. Which command opens the HTML report?

npx playwright report
npx playwright show-report
npx playwright html
npx playwright test --report

Correct! show-report displays the HTML report from a previous run.

4. Which option reruns only previously failed tests?

--retry
--last-failed
--failed-only
--rerun

Incorrect. --last-failed selects previously failing tests.

5. Which command generates test code by recording browser interactions?

npx playwright codegen
npx playwright generate
npx playwright record
npx playwright create

Correct! Codegen records interactions and generates scripts.

6. Which command opens a trace file?

npx playwright show-report trace.zip
npx playwright trace trace.zip
npx playwright show-trace trace.zip

npx playwright debug trace.zip

Correct! show-trace opens the Trace Viewer.

7. Which option stops execution after the first failure?

--workers=1
--retries=0
-x
--last-failed

Incorrect. -x is shorthand for --max-failures=1.

8. Which command merges Blob reports?

npx playwright merge-reports ./reports
npx playwright combine ./reports
npx playwright join-reports
npx playwright report-merge

Correct! merge-reports combines Blob reports from the specified directory.

Your score: 6/8

Try again

My recommendation for your QA Automation preparation: Practise these commands in your own Playwright TypeScript project, especially `test`, `--grep`, `--project`, `--headed`, `--debug`, `--last-failed`, `show-report` and `show-trace`. These are useful both in day-to-day automation work and in technical interviews.

Playwright Configuration – Complete Beginner-Friendly Explanation

The `playwright.config.ts` file is the central configuration file of a Playwright project.

Think of it as the control centre of your automation framework. It tells Playwright:

- Where your test files are located.

- Which browsers to use.
- How many tests can run simultaneously.
- How many times failed tests should be retried.
- Which reports to generate.
- How long tests should wait.
- How to start your application before executing tests.

Let's understand each section with practical QA examples.

1. Basic configuration

A typical `playwright.config.ts` looks like this:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: 'tests',
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
  globalTimeout: 60 * 60 * 1000,
  reporter: 'html',

  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
  },

  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
  ],

  webServer: {
    command: 'npm run start',
```

```
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },
});
```

Important rule: Test runner options such as `testDir`, `retries`, `workers`, and `reporter` belong at the top level. They should not be placed inside `use`.

Understanding each option

1. `testDir`

Specifies the folder where Playwright searches for test files.

```
testDir: 'tests',
```

Example folder structure:

```
my-project/
│
├── tests/
│   ├── login.spec.ts
│   ├── cart.spec.ts
│   └── checkout.spec.ts
│
└── playwright.config.ts
```

Playwright looks inside the `tests` folder for test files.

2. `fullyParallel`

Controls whether tests can run in parallel, including tests within the same file.

```
fullyParallel: true,
```

Suppose you have:

- Login test
- Search product test
- Add to cart test
- Checkout test

With full parallel execution enabled, Playwright can run independent tests simultaneously.

Without parallel execution

Login

Search

Cart

Checkout

Tests execute one after another.

With parallel execution

Login

Parallel

Search

Parallel

Cart

Parallel

Checkout

Parallel

Independent tests can execute at the same time.

Practical benefit: Reduces total execution time.

Remember: Tests should not depend on each other's data or execution order when running in parallel.

3. `forbidOnly`

Prevents accidental execution of a test suite containing `test.only()`.

```
forbidOnly: !!process.env.CI,
```

Example:

```
test.only('Login test', async ({ page }) => {
  await page.goto('/login');
});
```

`test.only()` runs only the focused test, which is useful during debugging.

But imagine you accidentally push this code to GitHub. Your CI pipeline might execute only the login test instead of the entire suite.

`forbidOnly: true` makes the run fail if it finds `test.only()`.

4. `retries`

Specifies how many times Playwright should retry a failed test.

```
retries: process.env.CI ? 2 : 0,
```

Meaning:

Environment	Retries
Local machine	0
CI pipeline	2

Example: A test fails on its first attempt because of a temporary network issue. Playwright can retry it up to two times.

Important: A test passing on retry may be classified as flaky. Retries do not fix the underlying problem.

5. `workers`

Specifies how many worker processes can execute tests concurrently.

```
workers: process.env.CI ? 1 : undefined,
```

- `1`: One worker process.
- `undefined`: Playwright chooses the worker count automatically.
- You can also specify a percentage such as `'50%'`.

Workers vs fullyParallel

Option	Meaning
`workers`	Controls how many worker processes run concurrently
`fullyParallel`	Controls whether tests within the same file can also run in parallel

6. `globalTimeout`

Sets the maximum duration for the entire test run.

```
globalTimeout: 60 * 60 * 1000,
```

This is 60 minutes because the value is in milliseconds.

If the complete test execution exceeds this limit, Playwright stops the run and reporters still produce a report.

7. `reporter`

Specifies the report format.

```
reporter: 'html',
```

After execution:

```
npx playwright show-report
```

You can view the HTML report with passed, failed and skipped test details.

8. `use`

The `use` section contains settings shared by tests, such as browser and page configuration.

```
use: {
  baseURL: 'http://localhost:3000',
  trace: 'on-first-retry',
},
```

`baseURL` avoids writing the full URL in every test.

Without `baseURL`:

```
await page.goto('http://localhost:3000/login');
```

With baseURL:

```
await page.goto('/login');
```

`trace: 'on-first-retry'` collects a trace when a test is retried for the first time.

9. `projects`

Projects allow you to execute tests with different configurations, such as different browsers.

```
projects: [
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'] },
  },
  {
    name: 'firefox',
    use: { ...devices['Desktop Firefox'] },
  },
  {
    name: 'webkit',
    use: { ...devices['Desktop Safari'] },
  },
],
```

Now your tests can run against Chromium, Firefox and WebKit.

For example:

```
npx playwright test --project=chromium
```

Runs only the Chromium project.

10. `webServer`

Starts your application before executing tests.

```
webServer: {
  command: 'npm run start',
  url: 'http://localhost:3000',
  reuseExistingServer: !process.env.CI,
},
```

Explanation:

Property	Meaning
`command`	Command used to start the application
`url`	URL Playwright waits for before running tests
`reuseExistingServer`	Reuses an existing server locally when possible

Example: Your e-commerce application must be running before Playwright can test login, search and checkout.

2. Filtering tests

Playwright provides two important options:

- `testMatch`
- `testIgnore`

`testMatch`

Specifies which test files Playwright should execute.

```
testMatch: '*todo-tests/*.spec.ts',
```

Example:

```
tests/
├── todo-tests/
│   ├── add.spec.ts
│   └── delete.spec.ts
│
└── login.spec.ts
```

With the above pattern, Playwright selects matching `.spec.ts` files under `todo-tests`.

testIgnore

Specifies which files or folders Playwright should ignore.

```
testIgnore: '*test-assets',
```

Example:

```
tests/
├── login.spec.ts
├── cart.spec.ts
└── test-assets/
    └── sample-data.spec.ts
```

Playwright ignores files matching the specified pattern.

Difference:

testMatch	testIgnore
Selects test files	Excludes test files
Defines what to run	Defines what not to run

3. Advanced configuration

1. outputDir

Specifies where test execution artifacts are stored.

```
outputDir: 'test-results',
```

Artifacts can include screenshots, videos and traces.

Example:

```
test-results/
├── login-test/
│   ├── trace.zip
│   └── screenshot.png
```

2. globalSetup

Runs a setup function before the test suite.

```
globalSetup: require.resolve('./global-setup'),
```

Example use cases:

- Prepare test data.
- Authenticate a test user.
- Set up the environment.

Example file:

```
// global-setup.ts
import { chromium, FullConfig } from '@playwright/test';

async function globalSetup(config: FullConfig) {
  const browser = await chromium.launch();
  const page = await browser.newPage();

  await page.goto('http://localhost:3000');

  // Perform setup tasks here

  await browser.close();
}

export default globalSetup;
```

3. globalTeardown

Runs after the test suite finishes.

```
globalTeardown: require.resolve('./global-teardown'),
```

Possible use cases:

- Clean up test data.
- Close external resources.
- Remove temporary setup files.

4. `timeout`

Specifies the maximum duration allowed for an individual test.

```
timeout: 30000,
```

30,000 milliseconds = 30 seconds.

If the test takes longer than 30 seconds, it times out.

Remember the difference:

Option	Applies to
`timeout`	Individual test
`globalTimeout`	Entire test run
`expect.timeout`	Web-first assertion

4. Expect options

You can configure how long assertions wait and how screenshot comparisons work.

Example:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  expect: {
    timeout: 5000,

    toHaveScreenshot: {
      maxDiffPixels: 10,
    },

    toMatchSnapshot: {
      maxDiffPixelRatio: 0.1,
    },
```

```
  },
});
```

1. expect.timeout

```
expect: {
  timeout: 5000,
},
```

Web-first assertions wait up to 5 seconds for the expected condition.

Example:

```
await expect(
  page.getByText('Login successful')
).toBeVisible();
```

If the text does not become visible within the timeout, the assertion fails.

2. toHaveScreenshot

```
toHaveScreenshot: {
  maxDiffPixels: 10,
},
```

Used for visual comparison.

Suppose you have a baseline screenshot of the login page. Playwright compares the new screenshot with the baseline.

`maxDiffPixels: 10` allows up to 10 differing pixels.

3. toMatchSnapshot

```
toMatchSnapshot: {
  maxDiffPixelRatio: 0.1,
},
```

Specifies the acceptable ratio of differing pixels between screenshots.

`0.1` means 10%.

5. Practical configuration for a QA automation project

Imagine you are creating an e-commerce automation framework with Playwright and TypeScript.

You want to:

- Run tests in Chromium, Firefox and WebKit.
- Use a common base URL.
- Generate HTML reports.
- Retry failures in CI.
- Save traces when tests fail and are retried.
- Keep execution time under control.

A possible configuration:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',

  fullyParallel: true,

  forbidOnly: !!process.env.CI,

  retries: process.env.CI ? 2 : 0,

  workers: process.env.CI ? 1 : undefined,

  reporter: 'html',

  timeout: 30000,

  expect: {
    timeout: 5000,
  },

  outputDir: 'test-results',
```

```
  use: {
    baseURL: 'https://example.com',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
  },

  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
  ],
});
```

This is an illustrative configuration. Replace the base URL with your application's actual URL.

6. Important interview questions and answers

Q1. What is `playwright.config.ts`?

It is the central configuration file used to control test execution, browsers, retries, timeouts, reports and other test settings.

Q2. What is the difference between `use` and top-level configuration?

Top-level configuration contains test runner options such as `testDir`, `workers` and `retries`. The `use` section contains shared browser and page options such as `baseURL`, `headless`, `trace` and `screenshot`.

Q3. What is `testDir`?

It specifies the directory where Playwright searches for test files.

Q4. What is `fullyParallel`?

It enables tests within the same file as well as across files to run in parallel.

Q5. What is the difference between `workers` and `fullyParallel`?

`workers` controls the number of concurrent worker processes. `fullyParallel` controls whether tests within the same file can also execute independently in parallel.

Q6. What is `forbidOnly`?

It prevents accidental use of `test.only()` in the test suite, which is especially useful in CI.

Q7. What is the purpose of `retries`?

It specifies how many times Playwright retries failed tests.

Q8. What is the difference between `timeout` and `globalTimeout`?

`timeout` limits an individual test's duration, whereas `globalTimeout` limits the duration of the complete test run.

Q9. What is `baseURL`?

It defines the application's base URL so tests can navigate using relative paths.

Q10. What is `trace: 'on-first-retry'`?

It collects a trace when a test is retried for the first time, helping investigate failures.

Q11. What are Playwright projects?

Projects allow tests to run under different configurations, such as Chromium, Firefox, WebKit or different device settings.

Q12. What is `webServer`?

It starts the application before tests execute and waits for the configured URL to become available.

Q13. What is `testMatch`?

It defines patterns for test files that Playwright should include.

Q14. What is `testIgnore`?

It defines patterns for test files that Playwright should exclude.

Q15. What is `outputDir`?

It specifies where Playwright stores test artifacts such as screenshots, videos and traces.

Q16. What is `globalSetup`?

It runs a setup function before the test suite, for example to prepare test data or authenticate a user.

Q17. What is `globalTeardown`?

It runs after the test suite to clean up resources or temporary data.

Q18. What is `expect.timeout`?

It defines the maximum time a web-first assertion waits for its expected condition.

Q19. What is `maxDiffPixels`?

It specifies the maximum number of differing pixels allowed in a screenshot comparison.

Q20. What is `maxDiffPixelRatio`?

It specifies the maximum acceptable proportion of differing pixels in a snapshot comparison.

7. Quick revision sheet

Configuration	Purpose
`testDir`	Test folder
`fullyParallel`	Parallel test execution
`workers`	Concurrent worker processes
`forbidOnly`	Prevent accidental `test.only()`

`retries`	Retry failed tests
`globalTimeout`	Overall execution time limit
`timeout`	Individual test time limit
`reporter`	Report format
`baseURL`	Base application URL
`projects`	Multiple browser/configuration setups
`webServer`	Start application before tests
`testMatch`	Select test files
`testIgnore`	Exclude test files
`outputDir`	Store test artifacts
`globalSetup`	Setup before test suite
`globalTeardown`	Cleanup after test suite
`expect.timeout`	Assertion waiting time
`maxDiffPixels`	Allowed screenshot pixel differences

8. Test your understanding

Configuration quiz

6/6 answered

1. Which option specifies the folder containing test files?

testMatch
testDir
outputDir
testIgnore

Correct — testDir tells Playwright where to search for test files.

2. Which option controls concurrent worker processes?

fullyParallel
workers
retries
timeout

Incorrect — workers controls the number of worker processes.

3. Which setting prevents accidental test.only() in CI?

forbidOnly
testIgnore
globalSetup
reporter

Incorrect — forbidOnly makes the run fail if test.only() is present.

4. Which timeout controls the complete test run?

timeout
expect.timeout
globalTimeout
actionTimeout

Incorrect — globalTimeout limits the complete test run.

5. Which option starts the application before tests?

projects
webServer
globalTeardown
outputDir

Correct — webServer starts the application and waits for its URL.

6. Which option configures assertion waiting time?

workers
retries
expect.timeout
testDir

Correct — expect.timeout controls how long web-first assertions wait.

3/6 correct

Try again

Interview tip: Be especially confident explaining `workers` vs `fullyParallel`, `timeout` vs `globalTimeout` vs `expect.timeout`, and `testMatch` vs `testIgnore`. These comparisons help demonstrate practical understanding of Playwright configuration.

Playwright Configuration (use) – Complete Guide for QA Automation

In the previous section, we learned about the main Playwright configuration options, such as `testDir`, `workers`, `retries` and `reporter`.

Now we are learning about the `use: {}` section.

The `use` section configures how the browser and browser context behave while executing tests.

For example, you can configure:

1. Base URL and authentication.
2. Mobile and tablet emulation.
3. Locale, timezone and geolocation.
4. Network settings.
5. Screenshots, videos and traces.
6. Browser execution options.

7. Configuration at global, project and test levels.

Let's understand these with practical examples.

1. Basic options

Two important options are:

- `baseURL`
- `storageState`

A. `baseURL`

It defines the base URL of your application.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    baseURL: 'http://localhost:3000',
  },
});
```

Without `baseURL`, you need to write the complete URL:

```
await page.goto('http://localhost:3000/login');
await page.goto('http://localhost:3000/cart');
```

With `baseURL`:

```
await page.goto('/login');
await page.goto('/cart');
```

Playwright automatically combines the relative path with the configured base URL.

Real-time QA example: In an e-commerce application, you can configure the application URL once and reuse it across login, search, cart and checkout tests.

B. `storageState`

This option loads previously saved browser storage information into the browser context.

```
use: {
  storageState: 'state.json',
},
```

What does this mean?

Imagine you have a login test:

1. Open the application.
2. Enter username and password.
3. Click Login.
4. Application authenticates the user.
5. Browser stores authentication information, such as cookies or local storage.

You can save this authentication state and reuse it in other tests.

Example: Save authentication state

```
import { test } from '@playwright/test';

test('save login state', async ({ page }) => {
  await page.goto('/login');

  await page.getByLabel('Email').fill('test@example.com');
  await page.getByLabel('Password').fill('Password123');

  await page.getByRole('button', { name: 'Login' }).click();

  await page.context().storageState({
    path: 'state.json',
  });
});
```

Example: Reuse authentication state

```
export default defineConfig({
  use: {
    baseURL: 'http://localhost:3000',
    storageState: 'state.json',
  },
});
```

Now tests using this configuration start with the saved authentication state, if it is still valid.

How storageState works

Login test

Enter credentials and authenticate

Save state.json

Cookies and local storage

Other tests

Reuse authentication without repeating login

Important: Authentication state files can contain sensitive session information. Do not commit them to Git or share them publicly.

2. Emulation options

Emulation means making the browser behave like a particular device or user environment.

For example, you can test your application as if it were being accessed from:

- A mobile phone.
- A tablet.
- A desktop.
- A user with a different language.
- A different timezone.
- A user with dark mode enabled.

Example:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
```

```
    colorScheme: 'dark',

    geolocation: {
      longitude: 12.492507,
      latitude: 41.889938,
    },

    locale: 'en-GB',

    permissions: ['geolocation'],

    timezoneId: 'Europe/Paris',

    viewport: {
      width: 1280,
      height: 720,
    },
  },
});
```

Let's understand each option.

A. `colorScheme`

```
colorScheme: 'dark',
```

Emulates the user's preferred color scheme.

Available values:

- `light`
- `dark`

Example: Your application supports dark mode and light mode. You can test whether the correct theme is displayed.

B. `geolocation`

```
geolocation: {
  longitude: 12.492507,
  latitude: 41.889938,
},
```

Sets the browser context's geographical location.

Example: Suppose an e-commerce application displays nearby stores based on the user's location.

You can provide test coordinates and verify whether the expected nearby stores appear.

C. `locale`

```
locale: 'en-GB',
```

Changes the browser's locale.

Examples:

Locale	Language/region
`en-US`	English – United States
`en-GB`	English – United Kingdom
`de-DE`	German – Germany
`fr-FR`	French – France
`hi-IN`	Hindi – India

Practical example: Your application displays dates and numbers differently depending on the user's locale.

You can verify that the correct format appears.

D. `permissions`

```
permissions: ['geolocation'],
```

Grants permissions to the browser context.

For example:

- Geolocation permission.
- Notifications permission.

Example:

```
use: {
  geolocation: {
    latitude: 18.5204,
    longitude: 73.8567,
  },
  permissions: ['geolocation'],
},
```

This allows the application to access the configured location through the browser's geolocation API.

E. timezoneId

```
timezoneId: 'Europe/Paris',
```

Changes the timezone of the browser context.

Example: A booking application displays available appointment slots based on the user's timezone.

You can test whether the correct local time is displayed.

F. viewport

```
viewport: {
  width: 1280,
  height: 720,
},
```

Sets the browser page's viewport size.

Example:

Device type	Example viewport
Desktop	1280 × 720

Tablet	768 × 1024
Mobile-sized viewport	375 × 812

These are example viewport dimensions, not fixed requirements for every device.

Important interview point: `viewport` changes the page's visible area. For more complete device emulation, use Playwright's device descriptors.

Practical mobile emulation example

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    {
      name: 'mobile-chrome',
      use: {
        ...devices['Pixel 7'],
      },
    },
  ],
});
```

This uses the device descriptor to configure mobile-related browser settings.

3. Network options

Network options help you test how your application behaves under different network conditions.

For example:

- File downloads.
- HTTP headers.
- HTTP authentication.
- HTTPS certificate errors.
- Offline mode.

- Proxy configuration.

A. `acceptDownloads`

```
acceptDownloads: false,
```

Controls whether downloads are accepted automatically.

The documentation states that the default is `true`.

Example:

```
test('download invoice', async ({ page }) => {
  await page.goto('/invoice');

  const downloadPromise = page.waitForEvent('download');

  await page.getByRole('button', {
    name: 'Download Invoice',
  }).click();

  const download = await downloadPromise;

  console.log(download.suggestedFilename());
});
```

This example waits for a download and reads its suggested filename.

B. `extraHTTPHeaders`

```
extraHTTPHeaders: {
  'X-My-Header': 'value',
},
```

Adds additional HTTP headers to requests.

Example:

```
use: {
  extraHTTPHeaders: {
    'X-Environment': 'QA',
    'X-Client': 'Playwright',
  },
},
```

Practical use: Some applications or APIs require specific headers to identify the client or environment.

C. `httpCredentials`

```
httpCredentials: {
  username: 'user',
  password: 'pass',
},
```

Provides credentials for HTTP authentication.

This is useful when an application is protected by HTTP Basic Authentication.

Do not hardcode real credentials in your configuration. Use environment variables or a secure secret store.

D. `ignoreHTTPSErrors`

```
ignoreHTTPSErrors: true,
```

Allows navigation to continue despite HTTPS certificate errors.

Example: You may be testing a QA environment with a self-signed certificate.

Use carefully. It should not be enabled blindly in production-security validation tests.

E. `offline`

```
offline: true,
```

Emulates an offline network connection.

Practical QA scenario:

Suppose a shopping website is open and the network connection is lost.

You can verify:

- Whether an appropriate offline message appears.
- Whether the application handles failed requests.
- Whether previously loaded content remains available.

F. proxy

```
proxy: {
  server: 'http://myproxy.com:3128',
  bypass: 'localhost',
},
```

Configures a proxy server for browser traffic.

A proxy acts as an intermediary between the browser and the destination server.

Possible uses include corporate network routing and testing through a particular proxy.

Network mocking – important note

The documentation mentions that you do not need to configure a special `use` option to mock network requests.

You can use Playwright's `page.route()` or `browserContext.route()`.

Example:

```
test('mock product API', async ({ page }) => {

  await page.route('**/api/products', async route => {

    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify([
        {
          id: 1,
          name: 'Laptop',
          price: 50000,
        },
      ]),
    });

  });

  await page.goto('/products');

  await expect(
    page.getByText('Laptop')
```

```
  ).toBeVisible();
});
```

Here, Playwright intercepts the API request and returns mock product data instead of relying on the actual API response.

4. Recording options

Playwright can record three important types of test evidence:

1. Screenshots.
2. Videos.
3. Traces.

These are particularly useful when debugging failed automation tests.

Example configuration:

```
export default defineConfig({
  use: {
    screenshot: 'only-on-failure',
    trace: 'on-first-retry',
    video: 'on-first-retry',
  },
});
```

A. Screenshot

```
screenshot: 'only-on-failure',
```

Controls when Playwright captures screenshots.

Option	Meaning
`off`	No screenshots
on	Capture screenshots for every test

`only-on-failure`	Capture screenshots when tests fail

Example: Your checkout test fails. You can inspect the screenshot to see what was displayed on the page.

B. Video

```
video: 'on-first-retry',
```

Records browser activity as a video.

Video modes:

- `off`
- `on`
- `retain-on-failure`
- `retain-on-first-failure`
- `retain-on-failure-and-retries`
- `on-first-retry`
- `on-all-retries`

Videos help you understand the sequence of actions that led to a failure.

C. Trace

```
trace: 'on-first-retry',
```

Records detailed execution information that you can inspect using Trace Viewer.

Trace information can include:

- Actions performed.
- Screenshots.
- DOM snapshots.

- Network activity.
- Console logs.
- Errors.

To open a trace:

```
npx playwright show-trace trace.zip
```

Understanding trace modes

This is an important interview topic.

Assume you configured:

```
retries: 2,
```

That means a test can have three attempts:

- First run.
- First retry.
- Second retry.

Let's compare the trace modes.

Mode	When recorded	When retained
`off`	Never	Never
on	Every run	Always
`retain-on-failure`	Every run	Failed runs
`retain-on-first-failure`	First run only	If first run fails
`retain-on-failure-and-retries`	Every run	Failed runs and retries

`on-first-retry`	First retry only	Always
`on-all-retries`	Every retry	Always

Example: `on-first-retry`

Suppose the first attempt fails and the first retry passes.

- First attempt: No trace.
- First retry: Trace recorded.
- Test passes.

Suppose both retries also fail:

- First attempt: No trace.
- First retry: Trace recorded.
- Second retry: No trace.

Only the first retry is recorded.

Trace vs Screenshot vs Video

Feature	What it helps you understand
Screenshot	What the page looked like
Video	What happened during execution
Trace	Detailed execution history, actions, DOM and network information

Example: Checkout test failure

Refero — UI/UX Design Inspiration for Your Next Project

Screenshot

Shows the page when the test failed.

Timeline Debugging with Test Reporting & Analytics | BrowserStack Docs

Video

Shows the sequence of browser actions.

Trace viewer | Playwright

Trace Viewer

Shows detailed test execution information.

5. Other options

A. `actionTimeout`

```
actionTimeout: 0,
```

Controls the maximum time an individual Playwright action can take.

The documentation specifies that the default is `0`, meaning no action-specific timeout limit.

Example:

```
use: {
  actionTimeout: 10000,
},
```

Now an action such as `click()` can wait up to 10 seconds.

```
await page.getByRole('button', {
  name: 'Login',
}).click();
```

If the action cannot complete within the configured timeout, it fails.

Remember:

- `actionTimeout`: Browser actions such as click and fill.
- `expect.timeout`: Web-first assertions.
- `timeout`: Individual test.

B. `browserName`

```
browserName: 'chromium',
```

Specifies which browser engine to use.

Available options:

- `chromium`
- `firefox`
- `webkit`

C. `bypassCSP`

```
bypassCSP: true,
```

Controls whether Playwright bypasses the website's Content Security Policy.

CSP is a browser security mechanism that restricts which resources and scripts a page can load or execute.

This option is mainly useful for specialized testing scenarios. Avoid enabling it in normal production-security testing because it changes the security behavior being tested.

D. `channel`

```
channel: 'chrome',
```

Specifies the browser channel.

Examples:

- `chrome`
- `chrome-beta`
- `msedge`
- `msedge-beta`

Difference between `browserName` and `channel`:

`browserName`	`channel`
Selects browser engine	Selects a particular browser distribution/channel
Example: `chromium`	Example: `chrome`

E. `headless`

```
headless: false,
```

Controls whether the browser window is visible.

Value	Meaning
`true`	Headless; browser window is not shown
`false`	Headed; browser window is shown

Example:

```
use: {
  headless: false,
},
```

Useful when you want to visually observe test execution.

F. `testIdAttribute`

By default, Playwright's `getByTestId()` looks for the `data-testid` attribute.

You can change this:

```
testIdAttribute: 'pw-test-id',
```

Now you can use:

```
<button pw-test-id="login-button">Login</button>
```

And locate it using:

```
await page.getByTestId('login-button').click();
```

This is useful when your application uses a custom test ID attribute.

6. More browser and context options

Playwright allows additional browser options through:

- `launchOptions`
- `contextOptions`
- `connectOptions`

Example:

```
export default defineConfig({
  use: {
    launchOptions: {
      slowMo: 50,
    },
  },
});
```

`slowMo: 50` adds a 50-millisecond delay between browser operations.

This is useful when observing execution during debugging.

Most commonly used settings, such as `headless` and `viewport`, can be directly specified inside `use`.

7. Explicit context creation and option inheritance

This is a slightly advanced concept.

Suppose you configure:

```
use: {
  userAgent: 'some custom ua',
  viewport: {
    width: 100,
    height: 100,
  },
},
```

Now you create a browser context manually inside a test:

```
test('check context options', async ({ browser }) => {

  const context = await browser.newContext();

  const page = await context.newPage();

  console.log(
    await page.evaluate(() => navigator.userAgent)
  );

  console.log(
    await page.evaluate(() => window.innerWidth)
  );

  await context.close();
});
```

According to the documentation, contexts created through the same Playwright instance used by the test runner inherit the configured context options.

In this example:

- The custom user agent is inherited.
- The viewport width is 100.

Important: If you explicitly provide context options while creating a context, those explicit options take precedence over inherited values.

For example:

```
const context = await browser.newContext({
  viewport: {
    width: 1280,
    height: 720,
  },
});
```

Here, the explicitly provided viewport takes precedence.

8. Configuration scopes

This is one of the most important concepts in this section.

Playwright allows configuration at three levels:

1. Global configuration.
2. Project-level configuration.
3. Test-level configuration.

Think of it like this:

Global configuration

Applies to all tests by default

locale: en-GB

Project-level configuration

Overrides the global value for a project

locale: de-DE

Test-level configuration

Overrides the value for a test file or describe group

locale: fr-FR

A. Global configuration

In `playwright.config.ts`:

```
export default defineConfig({
  use: {
    locale: 'en-GB',
  },
});
```

All tests use the English UK locale by default.

B. Project-level configuration

You can override the locale for a particular project.

```
export default defineConfig({
  projects: [
    {
      name: 'chromium',
      use: {
        ...devices['Desktop Chrome'],
        locale: 'de-DE',
      },
    },
  ],
});
```

Now Chromium tests use German locale.

C. Test-level configuration

You can override the locale for a specific test file using `test.use()`.

```
import { test, expect } from '@playwright/test';

test.use({
  locale: 'fr-FR',
});

test('French language test', async ({ page }) => {
  await page.goto('/');
});
```

Now tests in this file use French locale.

You can also use `test.use()` inside a `test.describe()` block:

```
test.describe('French language tests', () => {

  test.use({
    locale: 'fr-FR',
  });

  test('verify French page', async ({ page }) => {
    await page.goto('/');
```

```
  });

});
```

When would you use this in a real project?

Suppose your e-commerce application supports three languages:

- English.
- German.
- French.

You can use:

- Global locale: English.
- German project: German.
- French test group: French.

This avoids creating separate configuration files for every small variation.

9. Reset an option

Sometimes you configure an option globally or for a test file but want to reset it.

Example global configuration:

```
export default defineConfig({
  use: {
    baseURL: 'https://playwright.dev',
  },
});
```

Now, in a test file:

```
import { test } from '@playwright/test';

test.use({
  baseURL: 'https://playwright.dev/docs/intro',
});
```

The file now uses the documentation intro page as its base URL.

But suppose you want a particular `describe` group to use the original global base URL.

You can reset it:

```
test.describe('Home page tests', () => {

  test.use({
    baseURL: undefined,
  });

  test('navigate to home', async ({ page }) => {
    await page.goto('/');
  });

});
```

Here, `undefined` resets the option to the value defined in the configuration file.

Completely unset an option

The documentation also shows a long-form fixture notation:

```
test.use({
  baseURL: [
    async ({}, use) => use(undefined),
    { scope: 'test' },
  ],
});
```

This completely unsets the base URL rather than reverting to the configured global value.

Difference:

Code	Result
`test.use({ baseURL: undefined })`	Resets to config-defined value
Long-form fixture notation	Completely sets the option to `undefined`

10. Interview questions and answers

Q1. What is the purpose of the `use` section in Playwright?

The `use` section configures browser and browser-context settings such as base URL, storage state, viewport, locale, network conditions, screenshots, videos and traces.

Q2. What is `baseURL`?

It defines the application's base URL so we can navigate using relative paths instead of writing the complete URL every time.

Q3. What is `storageState`?

It loads previously saved browser storage information, such as cookies and local storage, into a new browser context. It is commonly used to reuse authentication.

Q4. What is browser emulation?

It allows us to simulate device and user environments, such as mobile viewports, locale, timezone, geolocation and color scheme.

Q5. What is the difference between `locale` and `timezoneId`?

`locale` controls language and regional formatting. `timezoneId` controls the browser context's timezone.

Q6. What is `geolocation`?

It configures the geographical coordinates available to the browser context.

Q7. What is the purpose of `permissions`?

It grants specified permissions, such as geolocation, to pages in the browser context.

Q8. What is `viewport`?

It sets the visible dimensions of the browser page.

Q9. What is `offline`?

It emulates an offline network connection, allowing us to test how an application handles network loss.

Q10. What is `extraHTTPHeaders`?

It configures additional HTTP headers that are sent with requests.

Q11. What is `httpCredentials`?

It provides credentials for HTTP authentication.

Q12. What is `ignoreHTTPSErrors`?

It allows navigation to continue despite HTTPS certificate errors.

Q13. What is the difference between screenshot, video and trace?

A screenshot captures a page image, video records browser activity, and a trace provides detailed execution information such as actions, snapshots, logs and network activity.

Q14. What is `actionTimeout`?

It sets the maximum duration for an individual Playwright action such as `click()` or `fill()`.

Q15. What is the difference between `browserName` and `channel`?

`browserName` selects the browser engine, such as Chromium, Firefox or WebKit. `channel` selects a particular browser distribution, such as Chrome or Edge.

Q16. What is `headless`?

It controls whether the browser window is displayed during test execution.

Q17. What is `testIdAttribute`?

It changes the default attribute used by Playwright's `getByTestId()` locator.

Q18. What is `launchOptions`?

It allows us to configure additional browser launch options, such as `slowMo`.

Q19. What are configuration scopes in Playwright?

Configuration can be defined globally, per project or per test file/describe group. More specific settings can override broader settings.

Q20. What is the difference between `test.use()` and global `use`?

Global `use` configures browser-context options for the whole project by default. `test.use()` overrides those options for a particular test file or describe group.

Q21. What is the difference between `on-first-retry` and `retain-on-failure` for trace?

`on-first-retry` records only on the first retry. `retain-on-failure` records every run but keeps the trace only for failed runs.

Q22. How can you mock an API response in Playwright?

Using `page.route()` or `browserContext.route()` to intercept a network request and return a mock response using `route.fulfill()`.

11. Quick revision sheet

Category	Important options
Basic	`baseURL`, `storageState`
Emulation	`colorScheme`, `geolocation`, `locale`, `permissions`, `timezoneId`, `viewport`
Network	`acceptDownloads`, `extraHTTPHeaders`, `httpCredentials`, `ignoreHTTPSErrors`, `offline`, `proxy`
Recording	`screenshot`, `video`, `trace`
Other	`actionTimeout`, `browserName`, `bypassCSP`, `channel`, `headless`, `testIdAttribute`
Advanced	`launchOptions`, `contextOptions`, `connectOptions`
Scopes	Global, project, test file or describe group
Reset	`test.use({ baseURL: undefined })`

12. Practice quiz

Playwright `use` quiz

8/8 answered

1. Which option is used to reuse a previously saved login session?

baseURL
storageState
viewport
locale

Correct — storageState loads saved browser storage, commonly used for authentication.

2. Which option changes the browser's timezone?

locale
colorScheme
timezoneId
geolocation

Correct — timezoneId sets the browser context's timezone.

3. Which option emulates an offline network?

proxy
offline
ignoreHTTPSErrors
acceptDownloads

Correct — offline: true emulates a disconnected network.

4. Which option records browser activity as a video?

screenshot
trace
video
outputDir

Correct — video configures browser video recording.

5. What does actionTimeout control?

Entire test run
Assertion wait
Individual browser action
Browser startup only

Correct — actionTimeout limits actions such as click and fill.

6. Which method intercepts a network request?

page.route()
page.goto()
page.pause()
page.reload()

Correct — page.route() intercepts requests and can provide mock responses.

7. Which method overrides use options for a test file?

test.describe()
test.use()
test.only()
test.skip()

Correct — test.use() sets options for a test file or describe group.

8. Which trace mode records only on the first retry?

on
retain-on-failure
on-first-retry
on-all-retries

Correct — on-first-retry records the first retry only.

8/8 correct

Try again

My recommendation for your QA Automation interviews: Practise explaining `storageState`, device emulation, network mocking, trace modes and configuration scopes with real examples. These topics help show that you understand how to build and maintain a practical Playwright framework, not just write test scripts.

Playwright Emulation – Complete Guide for QA Automation

In this section, we will learn how to test a web application under different device and user conditions using Playwright.

Emulation means making the browser behave like a particular device or user environment without needing to own that physical device.

For example, using Playwright, you can test:

- How your website looks on an iPhone or Android phone.
- Whether the mobile menu works correctly.
- Whether the application supports different languages.
- Whether dates and times display correctly in different timezones.
- Whether notifications work.
- Whether dark mode works.
- How the application behaves when the internet is disconnected.
- Whether the website works without JavaScript.

Let's understand each concept with practical examples.

1. Devices

Playwright provides a predefined collection of device configurations through `devices`.

You can use these configurations to emulate devices such as:

- Desktop Chrome
- Desktop Firefox
- iPhone
- Pixel

- iPad

A device configuration can include:

Property	Meaning
`userAgent`	Identifies the browser and device to the website
`screenSize`	Screen dimensions
`viewport`	Visible browser page dimensions
`hasTouch`	Whether touch input is enabled
`isMobile`	Enables mobile-specific browser behavior

Example: Emulate Desktop Chrome and iPhone 13

In `playwright.config.ts`:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    {
      name: 'chromium',
      use: {
        ...devices['Desktop Chrome'],
      },
    },

    {
      name: 'Mobile Safari',
      use: {
        ...devices['iPhone 13'],
      },
    },
```

```
  ],
});
```

Explanation:

- `devices` imports Playwright's predefined device configurations.
- `devices['Desktop Chrome']` provides desktop browser settings.
- `devices['iPhone 13']` provides iPhone-related emulation settings.
- `...` is the spread operator. It copies the device settings into the project's `use` configuration.

Now you can execute tests against both projects.

```
npx playwright test
```

To execute only mobile Safari:

```
npx playwright test --project="Mobile Safari"
```

[The playwright window maximizes and views responsively. | by GuRu-DGS | Devmap | Jan, 2026 | Medium]

Desktop Chrome

Desktop browser emulation

[iOS 15: How to put the Safari address bar back to the top on your iPhone » nextpit]

iPhone 13

Mobile Safari emulation

Important interview point: Predefined devices assume a specific platform. For example, `Desktop Chrome` provides a Windows-specific user agent string.

If you want to use the user agent of the actual platform running your tests, you can unset it:

```
const context = await browser.newContext({
  ...devices['Desktop Chrome'],
  userAgent: undefined,
});
```

2. Viewport

A viewport is the visible area of a webpage inside the browser.

For example:

- Desktop: 1280 × 720
- Mobile-sized viewport: 375 × 812
- Tablet: 768 × 1024

You can configure a viewport in the Playwright configuration file.

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    {
      name: 'chromium',
      use: {
        ...devices['Desktop Chrome'],

        viewport: {
          width: 1280,
          height: 720,
        },
      },
    },
  ],
});
```

Important: Define `viewport` after spreading the device configuration. Otherwise, the device's viewport setting can overwrite your custom value.

Override viewport inside a test file

```
import { test, expect } from '@playwright/test';

test.use({
  viewport: {
    width: 1600,
    height: 1200,
```

```
  },
});

test('my test', async ({ page }) => {
  await page.goto('/');
});
```

Override viewport inside a describe block

```
test.describe('Large viewport tests', () => {

  test.use({
    viewport: {
      width: 1600,
      height: 1200,
    },
  });

  test('verify page layout', async ({ page }) => {
    await page.goto('/');
  });

});
```

Change viewport dynamically

You can also change the viewport while a test is running:

```
test('change viewport', async ({ page }) => {

  await page.goto('/');

  await page.setViewportSize({
    width: 1280,
    height: 720,
  });

  await page.setViewportSize({
    width: 375,
    height: 812,
  });

});
```

Real QA scenario: Your e-commerce website should display a full navigation menu on desktop but a hamburger menu on mobile. You can test both layouts using different viewport sizes.

3. `isMobile`

This option controls mobile-specific browser behavior.

```
isMobile: true,
```

According to the documentation, it determines whether:

- The meta viewport tag is taken into account.
- Touch events are enabled.

Example:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    {
      name: 'chromium',
      use: {
        ...devices['Desktop Chrome'],
        isMobile: false,
      },
    },
  ],
});
```

Important: Define `isMobile` after the spread operator if you want to override the device's predefined value.

Remember that `isMobile` is not supported by every browser engine. Check the supported device/browser combinations when configuring projects.

Difference between viewport and isMobile

Viewport	isMobile

Controls visible page dimensions	Controls mobile-specific behavior
Example: 375 × 812	Example: `true`
Can change page layout through responsive CSS	Enables mobile emulation behavior

4. Locale and Timezone

These settings help test applications for users in different countries.

A. Locale

Locale controls the browser's language and regional preferences.

```
locale: 'en-GB',
```

Examples:

Locale	Language/region
`en-US`	English – United States
`en-GB`	English – United Kingdom
`de-DE`	German – Germany
`fr-FR`	French – France
`hi-IN`	Hindi – India

Example configuration:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    locale: 'en-GB',
```

```
  },
});
```

B. Timezone

Timezone controls the browser context's timezone.

```
timezoneId: 'Europe/Paris',
```

Example:

```
export default defineConfig({
  use: {
    locale: 'en-GB',
    timezoneId: 'Europe/Paris',
  },
});
```

Override locale and timezone for a specific test file

```
import { test, expect } from '@playwright/test';

test.use({
  locale: 'de-DE',
  timezoneId: 'Europe/Berlin',
});

test('German language test', async ({ page }) => {

  await page.goto('https://www.bing.com');

});
```

Now the browser uses German locale and Berlin timezone for this test file.

Practical QA scenario:

Imagine you are testing an international shopping application.

- A user in Germany should see German language.
- A user in France should see French language.
- A user in the UK should see UK regional formatting.

You can verify the correct language, date formatting and timezone-dependent behavior.

Important interview point

The documentation makes an important distinction:

`locale` and `timezoneId` affect the browser, not the Playwright test runner's own timezone.

To change the test runner timezone, use the TZ environment variable.

5. Permissions

Permissions control whether a browser application can access certain device capabilities.

Examples:

- Notifications.
- Geolocation.

A. Grant notification permission globally

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    permissions: ['notifications'],
  },
});
```

Now pages in the configured browser context are granted notification permission.

B. Grant permission for a specific domain

```
import { test } from '@playwright/test';

test.beforeEach(async ({ context }) => {

  await context.grantPermissions(
    ['notifications'],
```

```
    { origin: 'https://skype.com' }
  );

});

test('notification test', async ({ page }) => {

  await page.goto('https://skype.com');

});
```

Here, notification permission is granted for the specified origin.

C. Revoke all permissions

You can revoke permissions using:

```
await context.clearPermissions();
```

Difference:

Method	Purpose
`permissions`	Grant permissions through configuration
`context.grantPermissions()`	Grant permissions programmatically
`context.clearPermissions()`	Clear granted permissions

Real QA scenario: A communication application asks users for notification permission. You can test whether the application behaves correctly when notification permission is granted or denied.

6. Geolocation

Geolocation allows you to simulate a user's geographical location.

For example, suppose your application displays nearby restaurants or stores based on the user's location.

You can configure a location in Playwright.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    geolocation: {
      longitude: 12.492507,
      latitude: 41.889938,
    },

    permissions: ['geolocation'],
  },
});
```

Important: Coordinates alone are not enough. You also need to grant the geolocation permission.

Set geolocation for a specific test

```
import { test, expect } from '@playwright/test';

test.use({
  geolocation: {
    longitude: 41.890221,
    latitude: 12.492348,
  },

  permissions: ['geolocation'],
});

test('test with geolocation', async ({ page }) => {

  await page.goto('https://www.bing.com');

});
```

Change location during test execution

You can also change the location dynamically:

```
test('change location', async ({ page, context }) => {
```

```
  await page.goto('/');

  // Initial location

  await context.setGeolocation({
    longitude: 41.890221,
    latitude: 12.492348,
  });

  // Change to another location

  await context.setGeolocation({
    longitude: 48.858455,
    latitude: 2.294474,
  });

});
```

`context.setGeolocation()` changes the geolocation for all pages in that browser context.

Practical QA scenario:

Suppose a food delivery application displays restaurants based on location.

You can:

1. Set the location to one city.
2. Verify nearby restaurants.
3. Change the location.
4. Verify that the displayed restaurants change accordingly.

7. Color Scheme and Media

Modern applications support light mode and dark mode.

Playwright allows you to emulate the user's preferred color scheme.

Supported values:

- `light`

- dark
- no-preference

Configure dark mode globally

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    colorScheme: 'dark',
  },
});
```

Configure dark mode for a specific test file

```
import { test, expect } from '@playwright/test';

test.use({
  colorScheme: 'dark',
});

test('verify dark mode', async ({ page }) => {

  await page.goto('/');

});
```

Verify the color scheme

You can check the browser's preferred color scheme using `matchMedia()`.

```
test('check dark mode', async ({ page }) => {

  await page.goto('/');

  const isDark = await page.evaluate(() =>
    window.matchMedia('(prefers-color-scheme: dark)').matches
  );

  expect(isDark).toBe(true);

});
```

What is `page.emulateMedia()`?

It allows you to emulate media-related browser settings dynamically.

For example:

```
await page.emulateMedia({
  colorScheme: 'dark',
});
```

You can also emulate media types such as `screen` and `print`:

```
await page.emulateMedia({
  media: 'print',
});
```

Real QA scenario: Verify whether the application's text, buttons, background and navigation remain readable in dark mode.

8. User Agent

A User Agent is a browser identification string sent to websites.

It can provide information about the browser, operating system and device.

Example:

```
import { test } from '@playwright/test';

test.use({
  userAgent: 'My user agent',
});

test('my user agent test', async ({ page }) => {

  await page.goto('/');

  const ua = await page.evaluate(() => navigator.userAgent);

  console.log(ua);

});
```

Why do QA engineers use it?

Suppose an application provides different content depending on whether a visitor uses a desktop or mobile browser.

You can override the user agent and test how the website responds.

Important: Changing only the user agent does not completely emulate a mobile device. For proper mobile testing, use a device descriptor that configures other properties too.

9. Offline

Offline emulation allows you to test an application without a regular browser network connection.

Configuration:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    offline: true,
  },
});
```

Example test:

```
test('test offline behavior', async ({ page }) => {

  await page.goto('/');

  await expect(
    page.getByText('You are offline')
  ).toBeVisible();

});
```

This is an illustrative test; the application must actually display that message for the assertion to pass.

Real QA scenario:

Suppose a user is shopping online and the internet suddenly disconnects.

You should verify:

- Does the application show an offline message?
- Does it handle failed API requests?
- Does it avoid displaying incorrect product information?
- Does it recover when the connection returns?

You can also change the network state during a test:

```
test('switch between online and offline', async ({ context, page }) => {

  await page.goto('/');

  await context.setOffline(true);

  // Test offline behavior

  await context.setOffline(false);

  // Test recovery behavior

});
```

Important limitation

According to the documentation, offline emulation affects requests using the browser's regular network stack, including:

- Page navigation.
- `fetch()`.
- `XMLHttpRequest`.
- WebSockets.

It does not interrupt WebRTC traffic. Existing WebRTC connections can continue sending and receiving media over UDP.

To test WebRTC connection loss, you need to interrupt the connection outside the browser, for example through a TURN server or OS-level firewall.

10. JavaScript Enabled

You can emulate a user scenario in which JavaScript is disabled.

Configuration:

```
import { test, expect } from '@playwright/test';

test.use({
  javaScriptEnabled: false,
});

test('test without JavaScript', async ({ page }) => {

  await page.goto('/');

});
```

What happens?

Normally, websites use JavaScript for:

- Form validation.
- Dropdown menus.
- Dynamic content.
- API requests.
- Interactive buttons.

When JavaScript is disabled, these features may not work.

Practical QA scenario: Verify whether important page content remains accessible when JavaScript is disabled.

For example, check whether the application displays a useful fallback message or whether essential content is still available.

11. Practical project example – E-commerce mobile testing

Let's combine multiple emulation options in one realistic project.

Requirement: Test an e-commerce website on an iPhone-like browser environment using dark mode and an Indian locale.

Configuration:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    {
      name: 'iPhone Mobile Testing',

      use: {
        ...devices['iPhone 13'],

        locale: 'en-IN',

        timezoneId: 'Asia/Kolkata',

        colorScheme: 'dark',

        screenshot: 'only-on-failure',
      },
    },
  ],
});
```

Example test:

```
import { test, expect } from '@playwright/test';

test('mobile shopping experience', async ({ page }) => {

  await page.goto('/');

  // Verify product search
  await page.getByPlaceholder('Search products')
```

```
    .fill('Laptop');

  await page.keyboard.press('Enter');

  // Verify product is displayed
  await expect(
    page.getByText('Laptop')
  ).toBeVisible();

});
```

This project tests the shopping flow under a mobile device configuration with Indian regional preferences and dark mode.

12. Important interview questions and answers

Question	Short answer
What is emulation?	Simulating a device or browser environment for testing.
What is `devices`?	Playwright's predefined device configuration registry.
What does `...devices['iPhone 13']` do?	Copies the iPhone 13 device settings into the project.
What is viewport?	The visible dimensions of a webpage.
What is `isMobile`?	Enables mobile-specific behavior, including viewport meta handling and touch events.
What is locale?	Browser language and regional preferences.
What is timezoneId?	Browser context timezone.
What are permissions?	Browser capabilities granted to the application.

What is geolocation?	Simulating a geographical location using coordinates.
What is colorScheme?	Emulating light or dark appearance preferences.
What is User Agent?	A string identifying the browser and platform to websites.
What is offline mode?	Simulates a disconnected browser network.
What is `javaScriptEnabled`?	Controls whether JavaScript is enabled in the browser context.
What is `setViewportSize()`?	Changes the viewport size during a test.
What is `setGeolocation()`?	Changes the simulated location for all pages in a context.
What is `clearPermissions()`?	Clears granted browser permissions.

Important differences for interviews

1. `viewport` vs `isMobile`

- `viewport` changes the visible page dimensions.
- `isMobile` enables mobile-specific browser behavior.

2. `locale` vs `timezoneId`

- `locale` controls language and regional preferences.
- `timezoneId` controls the browser's timezone.

3. `userAgent` vs device emulation

- `userAgent` changes the browser identification string.
- Device emulation configures multiple device properties, not just the user agent.

4. `permissions` vs `geolocation`

- `permissions` grants access to browser capabilities.
- `geolocation` supplies the simulated geographical coordinates.

5. `offline` vs network mocking

- `offline` simulates a disconnected browser network.
- Network mocking intercepts requests and returns controlled responses.

13. Quick revision commands and methods

Purpose	Playwright option/method
Emulate iPhone	`devices['iPhone 13']`
Set viewport	`page.setViewportSize()`
Enable mobile behavior	`isMobile: true`
Set language	`locale: 'en-IN'`
Set timezone	`timezoneId: 'Asia/Kolkata'`
Grant permissions	`context.grantPermissions()`
Clear permissions	`context.clearPermissions()`
Set location	`geolocation`
Change location	`context.setGeolocation()`
Set dark mode	`colorScheme: 'dark'`

Change media settings	`page.emulateMedia()`
Change User Agent	`userAgent`
Simulate offline	`offline: true`
Disable JavaScript	`javaScriptEnabled: false`

14. Practice quiz

Try these interview-style questions.

Emulation quiz

8/8 answered

1. Which Playwright property provides predefined device configurations?

browserName
devices
viewport
userAgent

Correct — The devices registry provides predefined device settings.

2. Which property controls the visible dimensions of a webpage?

isMobile
screenSize
viewport
hasTouch

Correct — viewport defines the visible page dimensions.

3. Which option enables mobile-specific browser behavior?

isMobile
headless
locale
offline

Correct — isMobile controls mobile-specific behavior such as viewport meta handling and touch events.

4. Which property changes the browser language and regional preferences?

timezoneId
locale
geolocation
colorScheme

Correct — locale controls browser language and regional preferences.

5. Which method changes geolocation during test execution?

page.setViewportSize()
context.setGeolocation()
context.clearPermissions()
page.emulateMedia()

Correct — context.setGeolocation() changes location for all pages in that context.

6. Which method clears granted browser permissions?

context.clearPermissions()
context.setOffline()
page.route()
page.close()

Correct — clearPermissions() clears granted permissions.

7. Which property simulates a disconnected network?

ignoreHTTPSErrors
offline
proxy
acceptDownloads

Incorrect — offline: true emulates an offline browser network.

8. Which option disables JavaScript in the browser context?

javaScriptEnabled: false
headless: false
bypassCSP: true
isMobile: true

Correct — javaScriptEnabled: false disables JavaScript.

7/8 correct

Try again

My interview preparation advice: Focus especially on device emulation, viewport vs `isMobile`, locale vs timezone, permissions, geolocation and offline testing. Practise writing the configuration yourself and explaining one real e-commerce testing scenario for each.

Playwright Fixtures – Complete Guide for QA Automation

1. What are fixtures in Playwright?

A fixture is a reusable setup that prepares everything a test needs before it runs and cleans it up after the test finishes.

Think of a fixture as a ready-made testing environment.

For example, imagine you are testing an e-commerce application. Before testing the Add to Cart functionality, you may need:

1. Open the browser.
2. Open the application.
3. Log in.
4. Navigate to the product page.
5. Add some initial test data.

Instead of repeating this setup in every test, you can create a fixture that performs it automatically.

Simple definition for interviews:

A Playwright fixture provides a test with the required environment, data, or objects. Fixtures support setup and teardown, are reusable, and are isolated between tests.

Your first fixture: page

```
import { test, expect } from '@playwright/test';

test('basic test', async ({ page }) => {
  await page.goto('https://playwright.dev/');

  await expect(page).toHaveTitle(/Playwright/);
});
```

What is happening here?

- `test()` defines the test case.
- `{ page }` requests the built-in page fixture.
- Playwright creates an isolated page for the test.
- `page.goto()` navigates to the website.
- `expect()` verifies the page title.

You do not need to manually create a browser or page. Playwright manages them through fixtures.

2. Built-in fixtures

Playwright provides several fixtures out of the box.

Fixture	Type	Purpose
`page`	`Page`	Isolated browser page for the test
`context`	`BrowserContext`	Isolated browser context
`browser`	`Browser`	Browser instance shared across tests in a worker
`browserName`	`string`	Current browser: Chromium, Firefox or WebKit

request	APIRequestContext	Used for API requests

Understanding each fixture

1. page

Used to interact with a web page.

```
test('login test', async ({ page }) => {
  await page.goto('https://example.com/login');
  await page.getByLabel('Email').fill('test@gmail.com');
});
```

2. context

Represents an isolated browser session. It can contain multiple pages and has its own cookies, local storage and session-related browser state.

```
test('context example', async ({ context }) => {
  const page1 = await context.newPage();
  const page2 = await context.newPage();

  await page1.goto('https://example.com');
  await page2.goto('https://playwright.dev');
});
```

3. browser

Represents the browser instance.

```
test('browser example', async ({ browser }) => {
  const context = await browser.newContext();
  const page = await context.newPage();

  await page.goto('https://example.com');

  await context.close();
});
```

4. browserName

Useful when you want different behavior depending on the browser.

```
test('browser check', async ({ page, browserName }) => {
  console.log('Running on:', browserName);

  await page.goto('https://example.com');
});
```

5. request

Used to test APIs without using the browser UI.

```
test('API test', async ({ request }) => {
  const response = await request.get(
    'https://jsonplaceholder.typicode.com/users'
  );

  expect(response.status()).toBe(200);
});
```

3. Fixtures vs beforeEach and afterEach

This is an important interview topic.

Traditionally, we use hooks to prepare and clean up test data.

Traditional approach using hooks

```
import { test } from '@playwright/test';
import { TodoPage } from './todo-page';

test.describe('Todo tests', () => {
  let todoPage;

  test.beforeEach(async ({ page }) => {
    todoPage = new TodoPage(page);

    await todoPage.goto();
    await todoPage.addToDo('item1');
    await todoPage.addToDo('item2');
  });

  test.afterEach(async () => {
    await todoPage.removeAll();
  });
```

```
  test('should add an item', async () => {
    await todoPage.addToDo('my item');
  });

  test('should remove an item', async () => {
    await todoPage.remove('item1');
  });
});
```

Here:

- `beforeEach()` prepares the Todo page before every test.
- `afterEach()` removes the test data after every test.
- `todoPage` is stored in a shared variable inside the `describe` block.

Fixture-based approach

```
import { test as base } from '@playwright/test';
import { TodoPage } from './todo-page';

export const test = base.extend<{ todoPage: TodoPage }>({
  todoPage: async ({ page }, use) => {

    // Setup
    const todoPage = new TodoPage(page);
    await todoPage.goto();
    await todoPage.addToDo('item1');
    await todoPage.addToDo('item2');

    // Test uses this fixture
    await use(todoPage);

    // Teardown
    await todoPage.removeAll();
  },
});
```

Use it in the test:

```
import { test } from './my-test';

test('should add an item', async ({ todoPage }) => {
  await todoPage.addToDo('my item');
});
```

The most important part: `use()`

In a custom fixture, `use()` separates setup, test execution and teardown.

```
todoPage: async ({ page }, use) => {

  // 1. Setup
  const todoPage = new TodoPage(page);

  // 2. Provide fixture to the test
  await use(todoPage);

  // 3. Teardown
  await todoPage.removeAll();
}
```

The test runs while execution is paused at `await use(todoPage)`. Once the test finishes, Playwright continues with the code after `use()`.

Advantages of fixtures

Advantage	Meaning
Encapsulation	Setup and cleanup are in one place
Reusability	Same fixture can be used in multiple test files
On-demand	Only required fixtures are initialized
Composability	One fixture can depend on another
Flexibility	Tests can request different combinations of fixtures

Cleaner grouping	Tests can be grouped by functionality rather than setup requirements

Interview point: Fixtures do not replace every hook. Hooks are still useful for actions that logically belong to a test suite. Fixtures are especially useful when setup and teardown form a reusable environment or resource.

4. Creating custom fixtures using `test.extend()`

`test.extend()` allows you to create your own fixtures.

For example, suppose your framework has two Page Object Model classes:

- `TodoPage`
- `SettingsPage`

You can make both available as fixtures.

```
import { test as base } from '@playwright/test';

import { TodoPage } from './todo-page';
import { SettingsPage } from './settings-page';

// Define fixture types
type MyFixtures = {
  todoPage: TodoPage;
  settingsPage: SettingsPage;
};

// Extend the base test
export const test = base.extend<MyFixtures>({

  todoPage: async ({ page }, use) => {

    const todoPage = new TodoPage(page);

    await todoPage.goto();
    await todoPage.addToDo('item1');

    await use(todoPage);
```

```
    await todoPage.removeAll();
  },

  settingsPage: async ({ page }, use) => {

    const settingsPage = new SettingsPage(page);

    await use(settingsPage);
  },

});

export { expect } from '@playwright/test';
```

How to use these fixtures

```
import { test, expect } from './my-test';

test.beforeEach(async ({ settingsPage }) => {
  await settingsPage.switchToDarkMode();
});

test('add todo item', async ({ todoPage, page }) => {

  await todoPage.addToDo('something nice');

  await expect(
    page.getByTestId('todo-title')
  ).toContainText(['something nice']);

});
```

Notice that you simply mention the fixture names in the test arguments.

Playwright initializes them and supplies the values automatically.

TypeScript benefit: `type MyFixtures` tells TypeScript what each fixture provides. This gives type checking and editor suggestions.

Naming rule from the documentation: custom fixture names must start with a letter or underscore and can contain only letters, numbers and underscores.

5. Overriding built-in fixtures

You can also modify an existing Playwright fixture to suit your framework.

For example, suppose every test should automatically open your application's home page.

Instead of writing `page.goto()` in every test, override the built-in `page` fixture.

```
import { test as base } from '@playwright/test';

export const test = base.extend({
  page: async ({ page, baseURL }, use) => {

    // Setup: open application
    await page.goto(baseURL!);

    // Provide the page to the test
    await use(page);

  },
});
```

Configure the base URL:

```
// playwright.config.ts

import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    baseURL: 'https://playwright.dev',
  },
});
```

Now tests can use the page directly:

```
import { test, expect } from './my-test';

test('verify home page', async ({ page }) => {
  await expect(page).toHaveURL('https://playwright.dev/');
});
```

Completely replacing a fixture

You can also replace the default behavior of a fixture.

For example, override `storageState` to provide a custom authentication cookie:

```
import { test as base } from '@playwright/test';

export const test = base.extend({
  storageState: async ({}, use) => {

    const cookie = await getAuthCookie();

    await use({
      cookies: [cookie],
    });

  },
});
```

Here, the default `storageState` behavior is replaced by the custom value.

Interview distinction:

- Overriding `page` can add behavior to the existing page fixture.
- Overriding `storageState` can supply a completely custom authentication state.

6. Worker-scoped fixtures

This is one of the most important concepts in fixtures.

Playwright executes tests using worker processes. A worker can execute multiple test files.

There are two important fixture scopes:

Scope	Lifetime	Example

`test`	Created and cleaned up for each test	page, test-specific test data
`worker`	Created once per worker process and reused by its tests	Shared test account, server

Example: Create one account per worker

Imagine you have 20 test cases. Creating a new user account before every test can take extra time.

Instead, create one account per worker and reuse it for tests running in that worker.

```
import { test as base, expect } from '@playwright/test';

type Account = {
  username: string;
  password: string;
};

export const test = base.extend<{}, { account: Account }>({

  account: [async ({ browser }, use, workerInfo) => {

    // Generate a unique account for this worker
    const username = 'user' + workerInfo.workerIndex;
    const password = 'verysecure';

    // Create account
    const page = await browser.newPage();

    await page.goto('/signup');

    await page.getByLabel('User Name').fill(username);
    await page.getByLabel('Password').fill(password);

    await page.getByText('Sign up').click();

    await expect(page.getByTestId('result'))
      .toHaveText('Success');
```

```
    await page.close();

    // Share account with tests in this worker
    await use({ username, password });

  }, { scope: 'worker' }],

  page: async ({ page, account }, use) => {

    // Log in before each test
    await page.goto('/signin');

    await page.getByLabel('User Name').fill(account.username);
    await page.getByLabel('Password').fill(account.password);

    await page.getByText('Sign in').click();

    await expect(page.getByTestId('userinfo'))
      .toHaveText(account.username);

    await use(page);
  },

});

export { expect } from '@playwright/test';
```

Understand the flow

Worker fixture lifecycle

Worker starts

Create one unique account

Test 1

- Log in using worker account
- Execute test

- Close test page/context

Test 2

- Log in using the same worker account
- Execute test
- Close test page/context

Worker stops

Worker fixture teardown

Important details:

- `workerInfo.workerIndex` identifies the worker.
- Each worker can create its own unique account.
- `scope: 'worker'` tells Playwright to initialize the fixture once per worker.
- The `page` fixture is still test-scoped, so each test gets an isolated page/context.
- The account is reused, but the browser session is not automatically reused.

Real-world QA example: You can create a test customer once per worker and use that customer for several checkout, order-history and profile tests.

One caution: if multiple workers share the same external account or modify the same data, tests can interfere with one another. Worker-specific accounts help avoid that problem.

7. Automatic fixtures

Normally, Playwright initializes a fixture only when a test or another fixture requests it.

But sometimes you want a fixture to run for every test automatically, even when the test does not mention its name.

For this, use:

```
{ auto: true }
```

Example: Automatically save debug logs when a test fails

```
import debug from 'debug';
import fs from 'fs';

import { test as base } from '@playwright/test';

export const test = base.extend<{ saveLogs: void }>({

  saveLogs: [async ({}, use, testInfo) => {

    const logs: string[] = [];

    debug.log = (...args) =>
      logs.push(args.map(String).join(''));

    debug.enable('myserver');

    // Test executes here
    await use();

    // Check test result
    if (testInfo.status !== testInfo.expectedStatus) {

      const logFile = testInfo.outputPath('logs.txt');

      await fs.promises.writeFile(
        logFile,
        logs.join('\n'),
        'utf8'
      );

      testInfo.attachments.push({
        name: 'logs',
        contentType: 'text/plain',
        path: logFile,
      });
    }

  }, { auto: true }],

});
```

Even if the test does not request `saveLogs`, this fixture runs automatically.

```
test('login test', async ({ page }) => {
  await page.goto('/login');
});
```

What is `testInfo`?

`testInfo` provides information about the currently running test.

In the example:

- `testInfo.status` gives the actual test status.
- `testInfo.expectedStatus` gives the expected status.
- `testInfo.outputPath()` creates a unique output path.
- `testInfo.attachments` allows you to attach files to the test report.

Practical use cases:

- Save debug logs when a test fails.
- Attach diagnostic information to reports.
- Collect screenshots or other debugging artifacts.
- Run common setup or monitoring logic automatically.

8. Fixture timeout

A fixture is considered part of the test lifecycle. Its setup and teardown time normally contributes to the test timeout.

For example, suppose a fixture takes 45 seconds to create test data, but your test timeout is 30 seconds. The test can time out during fixture setup.

You can assign a separate timeout to a fixture.

```
import { test as base } from '@playwright/test';

const test = base.extend<{ slowFixture: string }>({

  slowFixture: [async ({}, use) => {
```

```
    // Perform a slow operation

    await use('hello');

  }, { timeout: 60000 }],

});

test('example test', async ({ slowFixture }) => {
  console.log(slowFixture);
});
```

Here:

- `60000` means 60,000 milliseconds.
- That is 60 seconds.
- The fixture gets its own timeout.

Worker-scoped fixtures also have their own timeout, equal to the default test timeout unless you configure a different timeout.

Interview question: Can a slow fixture cause a test timeout?

Yes. Fixture setup and teardown are part of the test lifecycle, so a slow fixture can cause a timeout. You can configure a separate fixture timeout.

9. Fixture options

An option fixture allows you to make test configuration reusable, configurable and type-safe.

For example, imagine your Todo application needs a default item.

You want different projects to use different default items:

- Shopping project: `Buy milk`
- Wellbeing project: `Exercise!`

Instead of hardcoding the item inside the fixture, define an option.

```
import { test as base } from '@playwright/test';
import { TodoPage } from './todo-page';

export type MyOptions = {
  defaultItem: string;
};

type MyFixtures = {
  todoPage: TodoPage;
};

export const test = base.extend<MyOptions & MyFixtures>({

  // Option fixture
  defaultItem: ['Something nice', { option: true }],

  // Custom fixture uses the option
  todoPage: async ({ page, defaultItem }, use) => {

    const todoPage = new TodoPage(page);

    await todoPage.goto();
    await todoPage.addToDo(defaultItem);

    await use(todoPage);

    await todoPage.removeAll();
  },

});

export { expect } from '@playwright/test';
```

Now configure different values in `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';
import type { MyOptions } from './my-test';

export default defineConfig<MyOptions>({

  projects: [
    {
```

```
      name: 'shopping',
      use: {
        defaultItem: 'Buy milk',
      },
    },

    {
      name: 'wellbeing',
      use: {
        defaultItem: 'Exercise!',
      },
    },
  ],

});
```

Understand the difference

Normal fixture	Option fixture
Provides a resource or setup	Provides a configurable value
Example: `todoPage`	Example: `defaultItem`
Uses `await use(value)`	Often uses `{ option: true }`
Can perform setup and cleanup	Can be configured from the config file

Array as an option value

If the option itself is an array, wrap it in an additional array when defining its value with the fixture tuple syntax.

```
type Person = {
  name: string;
};

const test = base.extend<{ persons: Person[] }>({
```

```
  persons: [[], { option: true }],

});

const actualPersons = [
  { name: 'Alice' },
  { name: 'Bob' },
];

test.use({
  persons: [actualPersons, { scope: 'test' }],
});
```

Why the extra array?

Because Playwright uses the outer array for fixture configuration and the inner value is the actual array of people.

10. Resetting an option

You can reset an option to the value defined in the configuration file by setting it to `undefined`.

Example config:

```
export default defineConfig({
  use: {
    baseURL: 'https://playwright.dev',
  },
});
```

In a test file:

```
import { test } from '@playwright/test';

// Set a different URL for this file
test.use({
  baseURL: 'https://playwright.dev/docs/intro',
});
```

For a particular describe block, reset it:

```
test.describe(() => {
```

```
  test.use({
    baseURL: undefined,
  });

  test('open home page', async ({ page }) => {
    await page.goto('/');
  });

});
```

Here, `undefined` resets the value to the configuration-defined value.

Completely unset an option

If you want to remove the value completely instead of falling back to the config value, use the long-form fixture notation:

```
test.use({
  baseURL: [
    async ({}, use) => use(undefined),
    { scope: 'test' },
  ],
});
```

Remember:

- `test.use({ baseURL: undefined })` → reset to the config-defined value.
- Long-form fixture returning `undefined` → completely unset the option.

11. Fixture execution order

This is a frequently asked Playwright interview topic.

Every fixture has two phases:

1. Setup: Code before `await use()`.
2. Teardown: Code after `await use()`.

Playwright follows dependency rules to decide when fixtures run.

Important execution rules

Rule 1: Dependencies run first.

If fixture A depends on fixture B:

- B is initialized first.
- A is initialized second.
- A is cleaned up first.
- B is cleaned up second.

Rule 2: Non-automatic fixtures are lazy.

A fixture is initialized only if a test, hook or another fixture needs it.

Rule 3: Test-scoped fixtures run for every test.

They are cleaned up after each test.

Rule 4: Worker-scoped fixtures run once per worker.

They are cleaned up when the worker process finishes.

Rule 5: Automatic fixtures run without being requested directly.

They are initialized automatically according to their scope.

Example

```
import { test as base } from '@playwright/test';

const test = base.extend<{
  testFixture: string;
  autoTestFixture: string;
  unusedFixture: string;
}, {
  workerFixture: string;
  autoWorkerFixture: string;
}>({

  workerFixture: [async ({}, use) => {
    console.log('Worker setup');
```

```
    await use('worker');

    console.log('Worker teardown');
  }, { scope: 'worker' }],

  autoWorkerFixture: [async ({}, use) => {
    console.log('Automatic worker setup');

    await use('auto worker');

    console.log('Automatic worker teardown');
  }, { scope: 'worker', auto: true }],

  testFixture: [async ({ workerFixture }, use) => {
    console.log('Test fixture setup');

    await use('test');

    console.log('Test fixture teardown');
  }, { scope: 'test' }],

  autoTestFixture: [async ({}, use) => {
    console.log('Automatic test setup');

    await use('auto test');

    console.log('Automatic test teardown');
  }, { scope: 'test', auto: true }],

  unusedFixture: [async ({}, use) => {
    console.log('Unused fixture setup');

    await use('unused');

    console.log('Unused fixture teardown');
  }, { scope: 'test' }],

});

test('first test', async ({ page }) => {
  console.log('First test');
```

```
});

test('second test', async ({ testFixture }) => {
  console.log('Second test');
});
```

What happens?

Fixture	First test	Second test
`autoWorkerFixture`	Runs	Reused
`autoTestFixture`	Runs	Runs again
`workerFixture`	Not needed	Runs when requested
`testFixture`	Not needed	Runs
`unusedFixture`	Never runs	Never runs
`page`	Runs if requested	Not requested in this test

The worker fixtures are torn down when the worker stops. Test fixtures are torn down after their respective tests.

Important dependency example

`testFixture` depends on `workerFixture`:

```
testFixture: async ({ workerFixture }, use) => {
  await use('test');
}
```

Therefore, when the second test requests `testFixture`, Playwright first initializes `workerFixture`.

This is called fixture dependency management.

Dependency setup and teardown

Setup: workerFixture

Setup: testFixture

Test executes

Teardown: testFixture

Teardown: workerFixture when worker stops

One nuance: `beforeAll` and `afterAll` hooks run once per worker for their applicable test file or describe scope. They are not automatically global across the entire test run.

12. Combine fixtures from multiple files

Suppose your framework has separate fixture modules:

- Database fixtures
- Accessibility fixtures
- Browser fixtures

You can combine them using `mergeTests()`.

```
import { mergeTests } from '@playwright/test';

import { test as dbTest } from 'database-test-utils';
import { test as a11yTest } from 'a11y-test-utils';

export const test = mergeTests(dbTest, a11yTest);
```

Now use the merged test object:

```
import { test } from './fixtures';

test('accessibility and database test', async ({
  database,
  page,
  a11y
}) => {

  // Use database fixture
```

```
  // Use page fixture
  // Use accessibility fixture

});
```

Why is this useful?

In a large automation framework, different teams or modules may provide their own fixtures. `mergeTests()` combines them so tests can use them through a single imported `test` object.

13. Box fixtures

Custom fixtures normally appear as separate steps in:

- Playwright UI Mode
- Trace Viewer
- Test reports
- Error messages

Sometimes a fixture performs common background setup that creates unnecessary noise in reports.

You can hide its fixture step using `box: true`.

```
import { test as base } from '@playwright/test';

export const test = base.extend({

  helperFixture: [async ({}, use) => {

    console.log('Common setup');

    await use();

    console.log('Common cleanup');

  }, { box: true }],

});
```

`box: true` vs `box: 'self'`

Setting	Meaning
`box: true`	Hides the fixture step and its internal steps from the report
`box: 'self'`	Hides only the fixture step; internal steps remain visible

Use boxing for frequently used helper fixtures when their internal operations do not need to appear in every test report.

14. Custom fixture title

You can give a fixture a custom title that appears in reports and error messages.

```
import { test as base } from '@playwright/test';

export const test = base.extend({

  innerFixture: [async ({}, use) => {

    console.log('Fixture setup');

    await use();

    console.log('Fixture teardown');

  }, { title: 'my fixture' }],

});
```

Here:

- Actual fixture name: `innerFixture`
- Display title: `my fixture`

The custom title changes how the fixture is displayed, not the name used to request it in tests.

15. Global beforeEach and afterEach using automatic fixtures

Normally:

- `test.beforeEach()` runs before each test in its applicable file or describe scope.
- `test.afterEach()` runs after each test in its applicable file or describe scope.

But what if you want common before/after behavior across multiple test files?

You can create an automatic test-scoped fixture.

Example

`fixtures.ts`

```
import { test as base } from '@playwright/test';

export const test = base.extend<{ forEachTest: void }>({

  forEachTest: [async ({ page }, use) => {

    // Before every test
    await page.goto('http://localhost:8000');

    await use();

    // After every test
    console.log('Last URL:', page.url());

  }, { auto: true }],

});
```

Use this fixture in your test files:

```
import { test } from './fixtures';
import { expect } from '@playwright/test';
```

```
test('basic test', async ({ page }) => {

  await expect(page).toHaveURL('http://localhost:8000');

  await page.goto('https://playwright.dev');

});
```

Because the fixture is automatic, you do not have to include `forEachTest` in the test arguments.

Practical example: You could use this pattern for common per-test logging, diagnostics or navigation setup. Be careful not to put expensive or unnecessary work in an automatic fixture because it runs for every test using that extended `test` object.

16. Global beforeAll and afterAll using worker fixtures

You can also create common worker-level setup and teardown.

Use:

```
{ scope: 'worker', auto: true }
```

Example:

`fixtures.ts`

```
import { test as base } from '@playwright/test';

export const test = base.extend<{}, { forEachWorker: void }>({

  forEachWorker: [async ({}, use) => {

    // Before all tests in this worker
    console.log('Starting worker');

    await use();

    // After all tests in this worker
    console.log('Stopping worker');

  }, { scope: 'worker', auto: true }],
```

```
});
```

Import it in your test file:

```
import { test } from './fixtures';

test('basic test', async ({ page }) => {

  // Test code

});
```

You do not need to request `forEachWorker` in the test arguments.

Important: It runs once per worker process, not once globally for the complete test run. If Playwright starts four workers, this worker fixture can run four times.

17. Real-world example: Fixtures in your QA automation framework

Imagine you are automating an e-commerce application with Login, Search, Cart and Checkout modules.

You can organize your framework like this:

```
playwright-project/
│
├── fixtures/
│   └── test-fixtures.ts
│
├── pages/
│   ├── LoginPage.ts
│   ├── HomePage.ts
│   ├── CartPage.ts
│   └── CheckoutPage.ts
│
├── tests/
│   ├── login.spec.ts
│   ├── search.spec.ts
│   ├── cart.spec.ts
│   └── checkout.spec.ts
```

```
│
└── playwright.config.ts
```

Your custom fixture file could provide reusable page objects:

```
// fixtures/test-fixtures.ts

import { test as base, expect } from '@playwright/test';

import { LoginPage } from '../pages/LoginPage';
import { HomePage } from '../pages/HomePage';
import { CartPage } from '../pages/CartPage';

type MyFixtures = {
  loginPage: LoginPage;
  homePage: HomePage;
  cartPage: CartPage;
};

export const test = base.extend<MyFixtures>({

  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },

  homePage: async ({ page }, use) => {
    await use(new HomePage(page));
  },

  cartPage: async ({ page }, use) => {
    await use(new CartPage(page));
  },

});

export { expect };
```

Now your test can use the fixtures:

```
import { test, expect } from '../fixtures/test-fixtures';

test('Add product to cart', async ({
  loginPage,
```

```
  homePage,
  cartPage
}) => {

  await loginPage.goto();
  await loginPage.login('test@gmail.com', 'password');

  await homePage.searchProduct('Laptop');

  await homePage.addFirstProductToCart();

  await expect(cartPage.cartTitle).toBeVisible();

});
```

This makes your tests cleaner because you don't need to create page objects manually in every test.

18. Important interview questions and answers

Basic questions

Q1. What are fixtures in Playwright?

Fixtures provide the environment, resources or objects required by a test. They support setup, teardown, reusability and isolation.

Q2. What are the built-in fixtures in Playwright?

Common built-in fixtures are `page`, `context`, `browser`, `browserName` and `request`.

Q3. What is the purpose of the page fixture?

It provides an isolated browser page for a test, allowing us to interact with web elements and verify application behavior.

Q4. What is `test.extend()`?

It is used to create custom fixtures or override existing fixtures by extending Playwright's base test object.

Q5. What is the purpose of `use()` in a fixture?

`use()` passes the fixture value to the test. Code before `use()` is setup, and code after `use()` is teardown.

Q6. What is the difference between fixtures and hooks?

Hooks such as `beforeEach` and `afterEach` organize setup and teardown within a test suite. Fixtures encapsulate reusable resources and their lifecycle, and can be shared across test files.

Q7. Are fixtures isolated between tests?

Test-scoped fixtures are isolated between tests. For example, each test gets its own page and browser context.

Intermediate questions

Q8. What is the difference between test-scoped and worker-scoped fixtures?

- Test-scoped fixtures are created for each test.
- Worker-scoped fixtures are created once per worker and reused by tests in that worker.

Q9. How do you create a worker-scoped fixture?

Use tuple syntax with `{ scope: 'worker' }`.

```
account: [
  async ({}, use) => {
    await use('test-account');
  },
  { scope: 'worker' }
]
```

Q10. What is an automatic fixture?

An automatic fixture runs without being explicitly requested by the test.

```
{ auto: true }
```

Q11. Can one fixture depend on another fixture?

Yes. For example:

```
todoPage: async ({ page }, use) => {
  // page is a dependency
```

```
  await use(page);
}
```

Playwright initializes the dependency first and tears it down after the dependent fixture.

Q12. What is `workerInfo.workerIndex`?

It provides the worker index. It can be used to create unique test accounts or test data for different workers.

Q13. Can we override built-in fixtures?

Yes. We can override fixtures such as `page` and `storageState` to provide custom behavior or values.

Q14. What is an option fixture?

An option fixture provides a configurable, type-safe value that can be set through the Playwright configuration file or test configuration.

Q15. What happens if a fixture is not used?

A non-automatic fixture is not initialized if no test, hook or other fixture requests it.

Advanced questions

Q16. What is fixture dependency execution order?

If fixture A depends on fixture B, B is set up first. During teardown, A is cleaned up first and then B.

Q17. Can a fixture cause a test timeout?

Yes. Fixture setup and teardown time normally count toward the test timeout. A separate fixture timeout can be configured.

Q18. What is `mergeTests()`?

It combines fixtures from multiple test modules into one test object.

Q19. What is fixture boxing?

Boxing hides fixture steps from the test report to reduce unnecessary noise. It uses `box: true` or `box: 'self'`.

Q20. What is the difference between `auto: true` and `scope: 'worker'`?

- `auto: true` controls whether a fixture runs automatically.
- `scope: 'worker'` controls how long the fixture lives.

They can be combined.

Q21. Can worker-scoped fixtures use test-scoped fixtures?

No. A worker-scoped fixture cannot depend on a test-scoped fixture because the worker fixture lives longer than an individual test.

Q22. How would you use fixtures in a POM framework?

I would create custom fixtures for page objects such as LoginPage, HomePage and CartPage. Tests can request these objects directly, making test code reusable and easier to maintain.

19. Quick revision notes

Concept	Remember
Built-in fixture	`page`, `context`, `browser`, `request`
Custom fixture	`test.extend()`
Setup	Code before `await use()`
Teardown	Code after `await use()`
Test scope	One instance per test
Worker scope	One instance per worker
Automatic fixture	`{ auto: true }`
Worker fixture	`{ scope: 'worker' }`

Fixture option	`{ option: true }`
Override fixture	Customize existing fixture
Fixture timeout	`{ timeout: 60000 }`
Merge fixtures	`mergeTests()`
Hide fixture steps	`{ box: true }`
Custom report title	`{ title: 'my fixture' }`
Test metadata	`testInfo`
Worker identification	`workerInfo.workerIndex`

Most important syntax to remember:

```
const test = base.extend<MyFixtures>({

  myFixture: async ({ page }, use) => {

    // Setup
    const value = 'Hello';

    // Provide to test
    await use(value);

    // Teardown
    console.log('Cleanup');

  },

});
```

20. Test your understanding

Fixtures quiz

8/8 answered

1. Which method is used to create custom fixtures?

test.describe()
test.extend()
test.beforeEach()
test.use()

Correct

test.extend() creates a new test object with custom fixtures.

2. What does await use(value) do?

Closes the browser
Passes the fixture value to the test
Runs another test
Creates a worker

Correct

The test receives the value passed to use(). Teardown continues after the test finishes.

3. Which scope creates a fixture once per worker?

test
worker
auto
global

Correct

scope: 'worker' creates one fixture instance per worker process.

4. What does auto: true mean?

Run only when requested
Run automatically without being requested
Run only after failure
Run once globally

Correct

Automatic fixtures run even if the test doesn't list them in its arguments.

5. What happens if fixture A depends on fixture B?

A initializes before B
Both initialize together
B initializes before A
B never initializes

Incorrect – B initializes before A

Dependencies are initialized before the fixtures that depend on them.

6. What is used to combine fixtures from multiple modules?

mergeTests()
mergeFixtures()
test.combine()
test.extendAll()

Incorrect – mergeTests()

mergeTests() combines test objects and their fixtures.

7. What does { box: true } do?

Creates a browser context
Hides fixture steps from reports
Increases timeout
Runs fixture once per worker

Incorrect – Hides fixture steps from reports

Boxing hides the fixture and its internal steps from reports.

8. Which object provides the current worker index?

testInfo
page
workerInfo
browserName

Correct

workerInfo.workerIndex identifies the worker.

5 / 8 correct

Try again

My recommendation for interview preparation: Focus especially on `test.extend()`, `use()`, test vs worker scope, automatic fixtures, fixture dependency order and fixture overriding. These are the concepts most likely to be tested in practical Playwright discussions.

Playwright Global Setup and Teardown – Complete Guide for QA Automation

1. What are Global Setup and Teardown?

In Playwright, Global Setup means performing common initialization steps once before the test projects run.

Global Teardown means performing cleanup activities once after the dependent tests have completed.

Imagine you are automating an e-commerce application. Before executing your tests, you might need to:

- Create test data in the database.
- Create a test user.
- Authenticate a user.
- Prepare the application environment.

After all tests finish, you might need to:

- Delete test data.
- Remove temporary accounts.
- Clean up database records.

- Stop services.

Playwright provides two approaches:

1. Project Dependencies – Recommended approach.
2. `globalSetup` and `globalTeardown` configuration options – Traditional approach.

2. Project Dependencies (Recommended)

Project dependencies allow you to create a separate project that runs before the actual test projects.

For example:

- `setup db` – Creates the database and prepares test data.
- `chromium with db` – Executes browser tests using the prepared database.
- `cleanup db` – Deletes the test data after dependent projects finish.

Step 1: Configure the setup project

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  testDir: './tests',

  projects: [
    {
      name: 'setup db',
      testMatch: /global\.setup\.ts/,
    },

    {
      name: 'chromium with db',
      use: {
        browserName: 'chromium',
      },
```

```
      dependencies: ['setup db'],
    },
  ],

});
```

Understand the configuration:

Property	Meaning
`name`	Name of the project
`testMatch`	Identifies which test file belongs to the project
`dependencies`	Specifies which project must run first
`use`	Configures browser and other test options

Here, `chromium with db` depends on `setup db`.

Therefore, the setup project must finish successfully before the dependent browser tests start.

Step 2: Create the setup test

Create `tests/global.setup.ts`:

```
import { test as setup } from '@playwright/test';

setup('create new database', async ({}) => {

  console.log('Creating new database...');

  // Initialize the database
  // Create required test data

});
```

Notice that setup code is written as a normal Playwright test using `test()`.

Step 3: Create your actual test

Create `tests/menu.spec.ts`:

```
import { test, expect } from '@playwright/test';

test('menu test', async ({ page }) => {

  await page.goto('/');

  // Test that uses the prepared database

});
```

Execution flow

Project dependency execution

1. Setup project

Create database and test data

2. Dependent test project

Run Chromium tests using prepared data

3. Teardown project

Delete database and test data

3. Project Dependencies – Teardown

After setup and test execution, you can configure a teardown project to clean up resources.

In the setup project, use the `teardown` property to specify the cleanup project.

Step 1: Update `playwright.config.ts`

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
```

```
  testDir: './tests',

  projects: [

    {
      name: 'setup db',
      testMatch: /global\.setup\.ts/,
      teardown: 'cleanup db',
    },

    {
      name: 'cleanup db',
      testMatch: /global\.teardown\.ts/,
    },

    {
      name: 'chromium',
      use: {
        ...devices['Desktop Chrome'],
      },
      dependencies: ['setup db'],
    },

  ],

});
```

Understand these two properties:

- `dependencies: ['setup db']` means Chromium tests must wait for the setup project.
- `teardown: 'cleanup db'` means the cleanup project runs after the dependent projects finish.

Step 2: Create the teardown file

Create `tests/global.teardown.ts`:

```
import { test as teardown } from '@playwright/test';

teardown('delete database', async ({}) => {
```

```
  console.log('Deleting test database...');

  // Delete test data
  // Remove temporary accounts

});
```

Complete execution sequence

1. Run `setup db`.
2. Create the database.
3. Run the Chromium project.
4. Execute all dependent tests.
5. Run `cleanup db`.
6. Delete the database and test data.

Important: Project dependencies are useful when setup and teardown need to be visible as actual Playwright test projects.

4. Test Filtering with Project Dependencies

Playwright supports test filtering using options such as:

- `--grep`
- `--grep-invert`
- `--shard`
- Selecting a particular test file or test location.
- `test.only()`

Suppose you have selected only one test from the Chromium project.

If Chromium depends on the `setup db` project, Playwright also runs the dependency project.

This ensures that the selected test has its required setup.

What is `--no-deps`?

You can skip project dependencies and teardowns using:

```
npx playwright test --no-deps
```

This means only the directly selected projects run; their dependencies and teardowns are ignored.

Example:

```
npx playwright test tests/menu.spec.ts --no-deps
```

This runs the selected test without running its setup dependency or configured teardown.

Use this carefully: the test may fail if the required database or test data has not already been prepared.

5. Option 2: Configure `globalSetup` and `globalTeardown`

The second approach is to use configuration options directly.

- `globalSetup` runs once before all tests.
- `globalTeardown` runs once after all tests.

Step 1: Configure the files

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  globalSetup: require.resolve('./global-setup'),

  globalTeardown: require.resolve('./global-teardown'),

});
```

Here:

- `global-setup.ts` contains initialization logic.
- `global-teardown.ts` contains cleanup logic.

The global setup file must export a single function that accepts the Playwright configuration object.

Step 2: Create `global-setup.ts`

```
import type { FullConfig } from '@playwright/test';

async function globalSetup(config: FullConfig) {

  console.log('Global setup started');

  // Initialize database
  // Create test data
  // Prepare environment

}

export default globalSetup;
```

Step 3: Create `global-teardown.ts`

```
async function globalTeardown() {

  console.log('Global teardown started');

  // Delete test data
  // Clean up resources

}

export default globalTeardown;
```

This approach runs the global setup function before the test run and the teardown function after the test run.

You can also return a function from `globalSetup` and Playwright will use that function as the global teardown.

6. Authentication using Global Setup

This is a very important practical example.

Imagine you have 50 test cases. If every test logs in separately, it can increase execution time.

You can authenticate once during global setup, save the authentication state, and reuse it in your tests.

Playwright uses the `storageState` option for this.

Step 1: Create `global-setup.ts`

```
import {
  chromium,
  type FullConfig
} from '@playwright/test';

async function globalSetup(config: FullConfig) {

  const { baseURL, storageState } =
    config.projects[0].use;

  const browser = await chromium.launch();

  const page = await browser.newPage();

  await page.goto(baseURL!);

  await page.getByLabel('User Name').fill('user');

  await page.getByLabel('Password').fill('password');

  await page.getByText('Sign in').click();

  // Save authentication state
  await page.context().storageState({
    path: storageState as string,
  });

  await browser.close();
```

```
}

export default globalSetup;
```

Understand the code

Code	Purpose
`chromium.launch()`	Launches Chromium manually
`browser.newPage()`	Creates a page
`page.goto()`	Opens the application
`fill()`	Enters login credentials
`click()`	Performs login
`storageState()`	Saves cookies and local storage state
`browser.close()`	Closes the browser

Step 2: Configure `playwright.config.ts`

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  globalSetup: require.resolve('./global-setup'),

  use: {
    baseURL: 'http://localhost:3000/',
    storageState: 'state.json',
  },

});
```

Here:

- `baseURL` defines the application URL.
- `storageState` defines where the authentication state is saved and where tests load it from.

Step 3: Use the authenticated state in tests

```
import { test, expect } from '@playwright/test';

test('verify logged-in user', async ({ page }) => {

  await page.goto('/');

  // User is already authenticated

  await expect(
    page.getByText('Welcome')
  ).toBeVisible();

});
```

Authentication flow

Global Setup

Log in once

Save `state.json`

Store authentication state

Test execution

Tests load the saved authentication state

Important: This example assumes authentication is stored in cookies or local storage supported by Playwright's storage state. It does not save every kind of browser state, such as session storage.

Also, protect `state.json`: it can contain sensitive authentication information. Do not commit it to Git.

7. Passing data from Global Setup to Tests

Sometimes you want to share information from global setup with your tests.

For example:

- Authentication tokens.
- Port numbers.
- Environment details.
- Test configuration values.

You can use environment variables through `process.env`.

Global setup

```
import type { FullConfig } from '@playwright/test';

async function globalSetup(config: FullConfig) {

  process.env.FOO = 'some data';

  process.env.BAR = JSON.stringify({
    some: 'data'
  });

}

export default globalSetup;
```

Access values inside tests

```
import { test, expect } from '@playwright/test';

test('read global setup data', async ({ page }) => {

  const { FOO, BAR } = process.env;

  expect(FOO).toBe('some data');

  const complexData = JSON.parse(BAR!);
```

```
  expect(complexData).toEqual({
    some: 'data'
  });

});
```

Remember: Environment variables set during global setup are available to the test processes launched by that Playwright run. They are not a way to share arbitrary JavaScript objects directly; serialize complex values using JSON.

8. Capturing traces when Global Setup fails

Sometimes the global setup itself fails.

For example:

- Login button is not found.
- Application is unavailable.
- Authentication fails.
- A page navigation times out.

Normally, debugging this can be difficult because global setup is not part of the regular test report in the same way as a dependency project.

The documentation shows how to capture a trace manually using browser context tracing.

Example

```
import {
  chromium,
  type FullConfig
} from '@playwright/test';

async function globalSetup(config: FullConfig) {

  const { baseURL, storageState } =
    config.projects[0].use;
```

```
const browser = await chromium.launch();

const context = await browser.newContext();

const page = await context.newPage();

try {

  // Start tracing
  await context.tracing.start({
    screenshots: true,
    snapshots: true,
  });

  await page.goto(baseURL!);

  await page.getByLabel('User Name').fill('user');

  await page.getByLabel('Password').fill('password');

  await page.getByText('Sign in').click();

  await context.storageState({
    path: storageState as string,
  });

  // Save successful setup trace
  await context.tracing.stop({
    path: './test-results/setup-trace.zip',
  });

  await browser.close();

} catch (error) {

  // Save trace when setup fails
  await context.tracing.stop({
    path: './test-results/failed-setup-trace.zip',
  });

  await browser.close();
```

```
    throw error;
  }

}

export default globalSetup;
```

Understand the tracing methods

Method	Purpose
`context.tracing.start()`	Starts recording the trace
`screenshots: true`	Records screenshots
`snapshots: true`	Records DOM snapshots
`context.tracing.stop()`	Stops recording and saves the trace
`path`	Location of the trace ZIP file

You can inspect a saved trace using:

```
npx playwright show-trace ./test-results/failed-setup-trace.zip
```

Interview point: With the `globalSetup` configuration approach, trace recording is not automatically integrated into the Playwright test report. You can manually start and stop tracing as shown above.

9. Project Dependencies vs globalSetup

This is the most important comparison from this documentation.

Feature	Project Dependencies	`globalSetup`
Runs before tests	Yes	Yes

HTML report visibility	Shown as a separate project	Not shown
Trace recording	Fully supported through test runner	Not automatically supported
Playwright fixtures	Fully supported	Not supported
Browser management	Uses browser fixture	Manual browser launch
Parallelism and retries	Supported through standard configuration	Not applicable to setup function
Config options	Automatically applied	Not automatically applied in the same way

Recommended
Project Dependencies

Why does Playwright recommend project dependencies?

1. Setup appears in the HTML report.
2. You can use built-in and custom fixtures.
3. Browser management is handled by Playwright.
4. Traces and other test artifacts can be recorded.
5. It works with the normal project execution and dependency model.

When should you use each approach?

Scenario	Recommended
Create database before tests	Project Dependencies

Authenticate using browser fixtures	Project Dependencies
Reusable setup with custom fixtures	Project Dependencies
Setup needs to appear in HTML report	Project Dependencies
Simple one-time external initialization	`globalSetup` can be suitable
Legacy project already using global setup	`globalSetup`

10. Real-world QA framework example

Suppose you are automating an e-commerce application.

Your test execution requires:

- Creating a test customer.
- Preparing products.
- Logging in.
- Running checkout tests.
- Deleting test data.

A good structure could be:

```
playwright-project/
│
├── tests/
│   ├── global.setup.ts
│   ├── global.teardown.ts
│   ├── login.spec.ts
│   ├── cart.spec.ts
│   └── checkout.spec.ts
│
├── pages/
│   ├── LoginPage.ts
```

```
│   ├── HomePage.ts
│   └── CartPage.ts
│
├── playwright.config.ts
│
└── test-results/
```

Execution:

1. Setup project creates the customer and prepares products.
2. Browser project executes login, cart and checkout tests.
3. Teardown project deletes test data.

This avoids repeating expensive setup operations unnecessarily.

11. Interview questions and answers

Q1. What is Global Setup in Playwright?

Global setup is a process that runs once before the test run to initialize the testing environment, such as creating test data, preparing a database or authenticating a user.

Q2. What is Global Teardown?

Global teardown runs after the tests to clean up resources, such as deleting test data or closing external services.

Q3. What are the two ways to implement global setup and teardown?

1. Project Dependencies.
2. `globalSetup` and `globalTeardown` configuration options.

Q4. Which approach is recommended by Playwright?

Project Dependencies, because they integrate better with the Playwright test runner and support fixtures, reports and traces.

Q5. What are project dependencies?

Project dependencies allow one project to run before another project. The dependent project starts only after its dependency has completed successfully.

Q6. How do you configure project dependencies?

Using the `dependencies` property:

```
{
  name: 'chromium',
  dependencies: ['setup db'],
}
```

Q7. How do you configure project teardown?

Using the `teardown` property:

```
{
  name: 'setup db',
  teardown: 'cleanup db',
}
```

Q8. What is the purpose of `testMatch`?

It identifies which test files should be included in a project.

Q9. Can we use Playwright fixtures in project dependency setup?

Yes. Project dependency setup is implemented as a regular Playwright test, so built-in and custom fixtures can be used.

Q10. Can we use Playwright fixtures in `globalSetup`?

No. The `globalSetup` function does not support Playwright test fixtures. Browser management must be done manually using browser APIs.

Q11. What is `--no-deps`?

It is a command-line option that skips project dependencies and teardowns.

Q12. How can you reuse authentication across tests?

Log in during setup, save the browser context's storage state to a file and configure `use.storageState` to load that file for tests.

Q13. What is `storageState`?

It stores browser authentication-related state, including cookies and local storage, so it can be reused in a new browser context.

Q14. How can you pass data from global setup to tests?

Using environment variables through `process.env`.

Q15. How can you debug a failure during global setup?

Start tracing using `context.tracing.start()`, stop it using `context.tracing.stop()` and save the trace ZIP file. Use `try...catch` to save the trace when setup fails.

Q16. What is the difference between global setup and `beforeAll()`?

- `globalSetup` runs once before the test run.
- `beforeAll()` runs once per applicable test file or describe scope in each worker process.

Q17. Does global setup appear in the HTML report?

With project dependencies, setup appears as a separate project. A function configured through `globalSetup` does not appear as a regular test in the HTML report.

Q18. Can setup and teardown projects use retries?

They are regular Playwright test projects, so standard test-runner configuration such as retries applies to them.

12. Quick revision notes

Topic	Syntax / Key point
Project dependency	`dependencies: ['setup db']`
Setup project	`testMatch: /global\.setup\.ts/`
Teardown project	`teardown: 'cleanup db'`
Teardown file	`global.teardown.ts`

Global setup config	`globalSetup`
Global teardown config	`globalTeardown`
Skip dependencies	`--no-deps`
Save authentication	`context.storageState()`
Reuse authentication	`use: { storageState: 'state.json' }`
Share data	`process.env`
Start tracing	`context.tracing.start()`
Stop tracing	`context.tracing.stop()`

Most important interview answer to remember:

Project Dependencies are recommended over `globalSetup` because they integrate with the Playwright test runner, support fixtures, appear in HTML reports and provide trace recording.

13. Test your understanding

Global Setup & Teardown Quiz

8/8 answered

1. Which approach does Playwright recommend for global setup?

globalSetup only
Project Dependencies
beforeEach
afterEach

Correct answer: Project Dependencies

Project Dependencies integrate better with the Playwright test runner.

2. Which property specifies the setup project?

dependencies
teardown
testMatch
globalTeardown

Correct answer: testMatch

testMatch selects the setup test file for the setup project.

3. Which property specifies that another project must run first?

dependencies
testMatch
storageState
globalSetup

Correct answer: dependencies

dependencies defines the prerequisite projects.

4. Which property configures the cleanup project?

cleanup
teardown
afterAll
globalTeardown

Correct

The teardown property names the project that runs after dependent projects.

5. Which CLI option skips dependencies and teardowns?

--skip-setup
--no-deps
--no-global
--ignore-dependencies

Correct answer: --no-deps

--no-deps skips project dependencies and teardowns.

6. How can authentication state be saved?

page.saveLogin()
browser.login()
context.storageState()
test.saveState()

Correct

storageState() saves cookies and local storage state.

7. How can global setup pass data to tests?

page fixtures
process.env
test.describe
browserName

Correct answer: process.env

Environment variables can pass values from global setup to test processes.

8. Can globalSetup directly use the page fixture?

Yes
No
Only in Chromium
Only in headed mode

Correct

globalSetup is not a test function and does not support Playwright test fixtures.

3/8 correct

Try again

Interview preparation tip: Be ready to explain a real scenario: "I would use project dependencies to create test data before executing my e-commerce tests, and a teardown project to clean up that data afterward. For authentication, I can save the storage state and reuse it in the test projects."

Playwright Parallelism – Complete Guide for QA Automation

1. What is parallelism in Playwright?

Parallelism means running multiple tests at the same time instead of executing them one by one.

Playwright uses multiple worker processes to execute tests simultaneously. This helps reduce the total execution time of a test suite.

For example, suppose you have 100 test cases:

- Sequential execution: tests run one after another.
- Parallel execution: multiple workers execute tests simultaneously.

Sequential execution – 1 worker

T1

T2

T3

T4

T5

T6

T1 → T2 → T3 → T4 → T5 → T6

Parallel execution – 3 workers

Worker 1

T1

T2

Worker 2

T3

T4

Worker 3

T5

T6

Three workers execute tests at the same time.

Important default behaviour:

- Different test files run in parallel by default.
- Tests inside one file run in declaration order by default.
- Tests in the same file normally run in the same worker process.

2. What are worker processes?

A worker is an independent operating-system process that Playwright Test uses to run tests.

Each worker:

- Has its own browser instance.
- Has an independent environment.
- Runs tests independently of other workers.
- Cannot communicate directly with other workers.

Example:

Suppose you have three test files:

```
tests/
  login.spec.ts
  search.spec.ts
  cart.spec.ts
```

Playwright may distribute these files across different workers.

Worker distribution example

3 workers

Worker 0 – login.spec.ts

Own browser

Worker 1 – search.spec.ts

Own browser

Worker 2 – cart.spec.ts

Own browser

Playwright tries to reuse a worker for multiple files to improve performance.

Important: If a test fails, Playwright shuts down that worker. It starts a replacement worker for subsequent tests to provide a clean environment.

3. Limit the number of workers

You can control how many worker processes execute tests simultaneously.

Option 1: Command line

```
npx playwright test --workers 4
```

Meaning: Playwright can use a maximum of four workers.

Option 2: Configuration file

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  workers: process.env.CI ? 2 : undefined,
});
```

Explanation:

- `process.env.CI` checks whether the tests are running in a CI environment.
- If CI is true, Playwright uses two workers.

- Otherwise, `undefined` lets Playwright use its default worker setting.

Real-time QA scenario: On your local machine, you may have enough resources to run multiple workers. In a CI pipeline, you might limit workers to avoid overloading the build agent.

4. Disable parallelism

Sometimes you want to run tests one by one.

Set the worker count to 1.

Command:

```
npx playwright test --workers=1
```

Or configuration:

```
export default defineConfig({
  workers: 1,
});
```

When is this useful?

- Debugging tests.
- Investigating flaky tests.
- Testing applications with shared resources.
- Running tests that must execute sequentially.

Remember: one worker disables parallel execution, but it does not automatically fix dependencies between tests.

5. Parallelize tests inside a single file

By default, tests inside one file execute in order.

You can change this using:

```
test.describe.configure({ mode: 'parallel' });
```

Example:

```
import { test, expect } from '@playwright/test';

test.describe.configure({ mode: 'parallel' });

test('Login test', async ({ page }) => {
  await page.goto('/login');
  await expect(page).toHaveTitle(/Login/);
});

test('Search test', async ({ page }) => {
  await page.goto('/products');
  await expect(page.getByRole('heading')).toBeVisible();
});
```

Now these tests can run in separate worker processes.

Important points:

- Parallel tests cannot share global variables or state.
- Each test executes its relevant hooks independently, including `beforeAll` and `afterAll`.
- Tests should be independent of each other.

Fully parallel mode

You can enable parallel execution for all tests across all files using:

```
export default defineConfig({
  fullyParallel: true,
});
```

You can also enable it for a particular project:

```
projects: [
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'] },
    fullyParallel: true,
  },
]
```

This is useful when you want Chromium tests to run fully parallel but want other projects to use their normal execution mode.

6. Default mode vs parallel mode vs serial mode

This is an important interview topic.

Mode	Behaviour
`default`	Files run in parallel; tests within a file run in order
`parallel`	Tests within the group can run in parallel
`serial`	Tests in the group run in order and depend on each other

Serial mode

Serial mode is used when tests depend on previous tests.

Example:

```
import { test, type Page } from '@playwright/test';

test.describe.configure({ mode: 'serial' });

let page: Page;

test.beforeAll(async ({ browser }) => {
  page = await browser.newPage();
});

test.afterAll(async () => {
  await page.close();
});

test('First test - Open website', async () => {
  await page.goto('https://playwright.dev/');
});
```

```
test('Second test - Click Get Started', async () => {
  await page.getByText('Get Started').click();
});
```

Here, the second test uses the page opened by the first test.

What happens if the first test fails?

- Subsequent tests in the serial group are skipped.
- The serial group is retried together.

Best practice
Playwright documentation does not recommend serial mode. It is better to create independent tests that can run separately.

7. Test locks

Imagine two tests modify the same user settings:

- Test 1 updates the user's profile settings.
- Test 2 renames the same user.

If both run simultaneously, they may interfere with each other.

Test locks prevent tests that use the same named lock from running at the same time. Other tests can still run in parallel.

Example

File 1: `settings.spec.ts`

```
import { test } from '@playwright/test';

test('Update user settings',
  { lock: 'user-settings' },
  async ({ page }) => {
    // Update user settings
  }
);
```

File 2: `profile.spec.ts`

```
import { test } from '@playwright/test';

test('Rename user',
  { lock: 'user-settings' },
  async ({ page }) => {
    // Rename user
  }
);
```

Both tests use the same lock name: `user-settings`.

Therefore, Playwright will not execute them simultaneously, even if they are in different files, workers, or projects.

Multiple locks

A test can request more than one lock:

```
test('Reset database', {
  lock: ['database', 'external-api']
}, async () => {
  // Reset database
});
```

The test must wait until both locks are available.

You can also apply locks to an entire `describe` group:

```
test.describe.configure({
  lock: 'user-settings'
});
```

Important: Playwright acquires the locks before a test starts and releases them when it finishes. In default and serial modes, tests in one file run together in order, so a lock declared on a test is held for the duration of the whole file.

8. Opt out of fully parallel mode

Suppose your configuration has:

```
export default defineConfig({
  fullyParallel: true,
});
```

But one group of tests should run in the default mode.

You can override it:

```
test.describe('Order tests', () => {

  test.describe.configure({ mode: 'default' });

  test('Create order', async ({ page }) => {
    // Test 1
  });

  test('Cancel order', async ({ page }) => {
    // Test 2
  });

});
```

Here, tests inside this group run in order rather than in parallel.

9. Avoid shared state in parallel tests

This is one of the most important concepts in parallel execution.

Playwright already isolates browser state:

- Each test gets its own `BrowserContext`.
- Cookies are isolated.
- Local storage is isolated.
- In-memory browser state is isolated.

However, problems can happen when tests share external data, such as:

- The same database record.
- The same user account.
- The same downloaded or exported file path.
- The same external service settings.

Let's understand two common solutions.

A. Give each test unique backend data

Suppose two tests create orders using the same order ID.

Both may try to modify the same record, causing a conflict.

Use `testInfo.testId` to create a unique ID for each test.

```
import { test, expect } from '@playwright/test';

test('Create an order', async ({ page }, testInfo) => {

  const orderId = `order-${testInfo.testId}`;

  await page.goto(`/orders/new?id=${orderId}`);

  await expect(
    page.getByText(orderId)
  ).toBeVisible();

});
```

Now each test has its own order ID.

Practical example: If 10 tests create orders simultaneously, each can create a separate order instead of competing for one shared order record.

B. Write to a unique file path

Suppose multiple tests export a CSV file named `export.csv` into the same location.

One test might overwrite another test's file.

Use `testInfo.outputPath()`:

```
import { test } from '@playwright/test';
import fs from 'fs';

test('Export CSV', async ({ page }, testInfo) => {

  const file = testInfo.outputPath('export.csv');

  await fs.promises.writeFile(
    file,
```

```
    'a,b,c',
    'utf8'
  );

});
```

Playwright generates a test-specific output path, preventing tests from overwriting one another's files.

C. Keep tests independent

This is the main best practice.

Avoid this:

```
let orderId: string;

test('Create order', async ({ page }) => {
  orderId = 'ORD123';
  // Create order
});

test('Verify order', async ({ page }) => {
  // Depends on the previous test's orderId
});
```

The second test depends on the first test. This can break when tests run in parallel or in a different order.

Better approach: each test should create its own data or use a fixture to prepare the required data.

10. Shard tests between multiple machines

Sharding means distributing a test suite across multiple machines or CI jobs.

Suppose you have a large test suite and want to run it across three machines.

Command:

```
npx playwright test --shard=2/3
```

Meaning:

- 2 = run shard number 2.
- 3 = total number of shards.

For example:

Example: 3 CI machines

Shard 1

Machine 1

1/3

Shard 2

Machine 2

2/3

Shard 3

Machine 3

3/3

Each machine runs its assigned portion of the suite.

This is especially useful in CI/CD pipelines when a large regression suite takes too long on one machine.

11. Limit failures and fail fast

Suppose your test suite has 500 test cases, and the application is currently broken.

You may not want Playwright to continue running all 500 tests after many failures.

Use `--max-failures`.

Command:

```
npx playwright test --max-failures=10
```

Meaning: stop the test run after reaching 10 failed tests and skip tests that have not yet executed.

You can also configure it:

```
export default defineConfig({
  maxFailures: process.env.CI ? 10 : undefined,
});
```

Real-world use: This can save CI execution time and resources when a major issue causes many tests to fail.

12. Worker index and parallel index

Every worker gets two identifiers:

1. `workerIndex`
2. `parallelIndex`

Let's understand the difference.

Property	Meaning
`workerIndex`	Unique worker process ID, starting from 0
`parallelIndex`	Identifies the worker's parallel slot, from 0 to workers - 1

You can access them through `testInfo`:

```
import { test } from '@playwright/test';

test('Check worker information', async ({ page }, testInfo) => {

  console.log('Worker Index:', testInfo.workerIndex);

  console.log('Parallel Index:', testInfo.parallelIndex);

});
```

You can also access them through environment variables:

```
process.env.TEST_WORKER_INDEX
process.env.TEST_PARALLEL_INDEX
```

Important interview concept

Suppose Worker 1 fails and Playwright restarts it.

- The replacement worker gets a new `workerIndex`.
- It retains the same `parallelIndex`.

Why?

The `parallelIndex` represents the worker slot, whereas `workerIndex` identifies the actual worker process.

13. Isolate test data between parallel workers

This is a very practical concept for real QA automation projects.

Imagine you have four workers running tests simultaneously. All four tests use the same database user account.

One worker changes the password while another is trying to log in. This can cause failures.

Solution: create a separate database user for each worker using `workerIndex`.

Step 1: Create a custom fixture

Create:

`playwright/fixtures.ts`

```
import { test as baseTest, expect } from '@playwright/test';

import {
  createUserInTestDatabase,
  deleteUserFromTestDatabase
} from './my-db-utils';

export * from '@playwright/test';

export const test = baseTest.extend<{}, {
  dbUserName: string
}>({
```

```
  dbUserName: [async ({}, use) => {

    const userName = `user-${test.info().workerIndex}`;

    // Create database user
    await createUserInTestDatabase(userName);

    // Make user available to tests
    await use(userName);

    // Delete database user after worker finishes
    await deleteUserFromTestDatabase(userName);

  }, { scope: 'worker' }],

});
```

Step 2: Import the custom fixture in your test

tests/example.spec.ts

```
import { test, expect } from '../playwright/fixtures';

test('Login with worker-specific user', async ({ dbUserName }) => {

  console.log('Database User:', dbUserName);

});
```

Example worker users:

Worker	Database user
Worker 0	user-0
Worker 1	user-1
Worker 2	user-2

```
Worker 3   user-3
```

Each worker reuses its own database user for the tests it runs.

Important: The fixture is worker-scoped, so setup and cleanup happen for the worker's lifecycle, not after every individual test. If a worker is restarted, a replacement worker may create a new user.

Also, tests using the same worker-specific account should avoid conflicting changes to that account.

14. Control test order

By default:

- Tests in a single file run in declaration order.
- Different files have no guaranteed execution order because files run in parallel.

Option A: Sort test files alphabetically

If you disable parallelism using:

```
export default defineConfig({
  workers: 1,
});
```

Playwright runs test files alphabetically.

You can name files like this:

```
tests/
  001-user-signin-flow.spec.ts
  002-create-document.spec.ts
  003-delete-document.spec.ts
```

This gives you a way to control file execution order when using one worker.

Option B: Use a test list file

The documentation also describes a test list file for explicitly controlling test order.

For example:

feature-a.spec.ts

```
import { test } from '@playwright/test';

export default function createTests() {

  test('Feature A test', async ({ page }) => {
    // Test code
  });

}
```

feature-b.spec.ts

```
import { test } from '@playwright/test';

export default function createTests() {

  test.use({ viewport: { width: 500, height: 500 } });

  test('Feature B test', async ({ page }) => {
    // Test code
  });

}
```

Create `test.list.ts`:

```
import { test } from '@playwright/test';

import featureBTests from './feature-b.spec.ts';
import featureATests from './feature-a.spec.ts';

test.describe(featureBTests);
test.describe(featureATests);
```

Configure Playwright:

```
export default defineConfig({
  workers: 1,
  testMatch: 'test.list.ts',
});
```

This runs Feature B tests before Feature A tests.

Documentation warning
Test lists are discouraged and supported on a best-effort basis. Some features, including the VS Code extension and tracing, may not work properly with them.

Also, do not define tests directly in helper files. Wrap them in functions and explicitly call those functions from the test list file, as shown above.

15. Interview questions and answers

These are useful for Playwright QA Automation interviews.

Q1. What is parallelism in Playwright?

Parallelism means executing multiple tests simultaneously using multiple worker processes to reduce overall execution time.

Q2. How does Playwright execute tests by default?

By default, different test files run in parallel, while tests within the same file run in order.

Q3. What is a worker in Playwright?

A worker is an independent operating-system process that executes tests. Each worker starts its own browser.

Q4. How do you limit the number of workers?

Using the command:

```
npx playwright test --workers=4
```

Or in configuration:

```
workers: 4
```

Q5. How do you disable parallel execution?

Set the worker count to one:

```
npx playwright test --workers=1
```

Q6. How can you run tests in a single file in parallel?

Use:

```
test.describe.configure({ mode: 'parallel' });
```

Q7. What is `fullyParallel`?

It enables fully parallel execution across test files and within files.

```
export default defineConfig({
  fullyParallel: true,
});
```

Q8. What is serial mode?

Serial mode executes interdependent tests in order. If one test fails, subsequent tests in the serial group are skipped. Playwright documentation generally recommends independent tests instead.

Q9. What is the difference between default, parallel and serial modes?

- Default: files run in parallel, tests within a file run in order.
- Parallel: tests in the configured group can run simultaneously.
- Serial: tests in the group run in order and depend on one another.

Q10. What are test locks?

Test locks prevent tests using the same lock name from executing simultaneously. They help protect shared resources.

Q11. Can parallel tests share global variables?

No. Parallel tests execute in separate worker processes and cannot share in-memory global variables.

Q12. How do you prevent data conflicts in parallel execution?

Use unique test data, such as `testInfo.testId`, and separate backend records or worker-specific accounts.

Q13. What is sharding in Playwright?

Sharding distributes a test suite across multiple machines or CI jobs.

Example:

```
npx playwright test --shard=2/3
```

Q14. What is the difference between `workerIndex` and `parallelIndex`?

`workerIndex` identifies a worker process uniquely. `parallelIndex` identifies its parallel worker slot. If a worker restarts, its `workerIndex` changes, but its `parallelIndex` remains the same.

Q15. What is `testInfo.outputPath()`?

It generates a file path scoped to the current test, helping prevent parallel tests from overwriting each other's files.

Q16. What is `maxFailures`?

It limits the number of failed tests before Playwright stops executing the remaining tests.

Q17. What happens when a worker fails?

Playwright shuts down the worker and starts a replacement worker to provide a clean environment for subsequent tests.

Q18. How do you isolate database users between workers?

Use `testInfo.workerIndex` to generate a unique user for each worker and create a worker-scoped fixture for setup and cleanup.

Q19. Can tests in different files execute in a specific order by default?

No. Different files run in parallel by default, so there is no guaranteed execution order across files.

Q20. What is the recommended approach: serial tests or independent tests?

Independent tests. Each test should create or prepare its own data and should not depend on another test's execution.

16. Quick revision notes

Concept	Important syntax
Run tests	`npx playwright test`
Four workers	`--workers=4`

Disable parallelism	`--workers=1`
Parallel tests in a file	`mode: 'parallel'`
Serial tests	`mode: 'serial'`
Fully parallel	`fullyParallel: true`
Test lock	`{ lock: 'user-settings' }`
Sharding	`--shard=2/3`
Stop after failures	`--max-failures=10`
Worker ID	`testInfo.workerIndex`
Parallel slot ID	`testInfo.parallelIndex`
Unique test file path	`testInfo.outputPath()`
Unique test ID	`testInfo.testId`

Most important concepts to remember

1. Different files run in parallel by default.
2. Tests inside one file run in order by default.
3. Every worker has its own browser.
4. Parallel tests should be independent.
5. Use unique test data to avoid database conflicts.
6. Use test locks for shared resources that cannot be accessed concurrently.

7. Use sharding to distribute tests across multiple machines.
8. Use worker-scoped fixtures to isolate data between workers.

17. Test your knowledge

Try these interview-style questions. Select your answers and submit to check your score.

Parallelism quiz

8/8 answered

1. What is the default Playwright execution behaviour?

All tests run sequentially
Files run in parallel; tests within a file run in order
All tests run in parallel

Correct

By default, test files run in parallel, while tests within a file run in order.

2. Which command disables parallelism?

npx playwright test --workers=0
npx playwright test --workers=1
npx playwright test --parallel=false

Incorrect — npx playwright test --workers=1

One worker means only one worker process executes tests at a time.

3. Which option enables fully parallel execution?

fullyParallel: true
parallel: true
workers: true

Correct

fullyParallel: true enables parallel execution across files and within files.

4. Which method configures tests in one file to run in parallel?

test.parallel()
test.describe.configure({ mode: 'parallel' })
test.configure({ parallel: true })

Correct

Use test.describe.configure with mode set to parallel.

5. What is the purpose of a test lock?

To stop all tests
To prevent tests sharing a lock from running simultaneously
To retry failed tests

Correct

Tests with the same lock name cannot run at the same time.

6. Which property helps create unique data for each test?

testInfo.testId
testInfo.titlePath
testInfo.timeout

Correct

testInfo.testId can be used to generate unique backend data.

7. What does --shard=2/3 mean?

Two workers out of three
Run the second shard out of three
Run each test twice

Correct

It selects shard 2 from a total of 3 shards.

8. A worker restarts after a failure. What happens to its indexes?

Both indexes remain the same
Both indexes change
workerIndex changes; parallelIndex remains the same

Incorrect — workerIndex changes; parallelIndex remains the same

The replacement gets a new workerIndex but retains the same parallelIndex.

6 / 8 correct

Try again

Next topic: Once you are comfortable with Parallelism, the next Playwright documentation topic can be explained in the same format, with practical examples and interview preparation.

Playwright – Parameterize Tests

I'll explain this topic in simple language, with practical examples and detailed interview questions and answers. I'll also highlight important points that you should remember for QA Automation / SDET interviews.

1. What is Parameterized Testing?

Simple meaning:

Parameterized testing means executing the same test case multiple times with different sets of test data.

Instead of writing separate test cases for every input, we write one test logic and pass different data to it.

Real-time example: Login functionality

Suppose you want to test login with three different users:

Username	Password	Expected result
Alice	Alice123	Login successful
Bob	Bob123	Login successful
Charlie	Wrong123	Login failed

Without parameterization, you might write three separate test cases.

With parameterization, you write one test and execute it with three different sets of data.

Benefits:

1. Reduces code duplication.
2. Improves code reusability.
3. Makes test maintenance easier.
4. Allows testing with multiple input combinations.
5. Improves test coverage.

2. Types of Parameterization in Playwright

Playwright supports parameterization at different levels:

1. Test-level parameterization.
2. Project-level parameterization.
3. Environment variable-based parameterization.
4. CSV-based test data parameterization.

Let's understand each one.

2.1 Test-Level Parameterization

This means running the same test with different data.

Look at the example:

```
[
  { name: 'Alice', expected: 'Hello, Alice!' },
  { name: 'Bob', expected: 'Hello, Bob!' },
  { name: 'Charlie', expected: 'Hello, Charlie!' },
].forEach(({ name, expected }) => {
```

```
  test(`testing with ${name}`, async ({ page }) => {

    await page.goto(`https://example.com/greet?name=${name}`);

    await expect(page.getByRole('heading'))
      .toHaveText(expected);

  });

});
```

Step-by-step explanation

Step 1: Create test data

```
[
  { name: 'Alice', expected: 'Hello, Alice!' },
  { name: 'Bob', expected: 'Hello, Bob!' },
  { name: 'Charlie', expected: 'Hello, Charlie!' },
]
```

Here, we have an array containing three objects.

Each object contains:

- `name`: Input data.
- `expected`: Expected output.

Step 2: Use forEach()

```
.forEach(({ name, expected }) => {
```

`forEach()` iterates through every object in the array.

It executes the test once for each object.

Step 3: Create a unique test name

```
test(`testing with ${name}`, async ({ page }) => {
```

The test name changes depending on the input.

For example:

- testing with Alice
- testing with Bob
- testing with Charlie

Step 4: Open the URL

```
await page.goto(`https://example.com/greet?name=${name}`);
```

It opens the URL using the current name.

Step 5: Validate the expected result

```
await expect(page.getByRole('heading'))
  .toHaveText(expected);
```

It checks whether the heading contains the expected greeting.

Execution flow

Test data array

Alice, Bob, Charlie

forEach()

Test 1

Alice

Test 2

Bob

Test 3

Charlie

Execute and validate each test independently

Important interview point: Each iteration creates a separate Playwright test. It is not one test executing three assertions.

2.2 Before and After Hooks with Parameterized Tests

First, understand the hooks:

Hook	Purpose
`beforeEach()`	Executes before every test
`afterEach()`	Executes after every test
`beforeAll()`	Executes once before tests in a scope
`afterAll()`	Executes once after tests in a scope

Example 1: Hooks outside forEach()

```
test.beforeEach(async ({ page }) => {
  console.log('Before test');
});

test.afterEach(async ({ page }) => {
  console.log('After test');
});

[
  { name: 'Alice', expected: 'Hello, Alice!' },
  { name: 'Bob', expected: 'Hello, Bob!' },
  { name: 'Charlie', expected: 'Hello, Charlie!' },
].forEach(({ name, expected }) => {

  test(`testing with ${name}`, async ({ page }) => {
    await page.goto(`https://example.com/greet?name=${name}`);

    await expect(page.getByRole('heading'))
      .toHaveText(expected);
  });

});
```

Here, hooks are declared outside `forEach()`.

They apply to the tests in the enclosing scope.

- `beforeEach()` runs before each test.
- `afterEach()` runs after each test.

Example 2: Hooks inside test.describe()

```
[
  { name: 'Alice', expected: 'Hello, Alice!' },
  { name: 'Bob', expected: 'Hello, Bob!' },
  { name: 'Charlie', expected: 'Hello, Charlie!' },
].forEach(({ name, expected }) => {

  test.describe(() => {

    test.beforeEach(async ({ page }) => {
      await page.goto(
        `https://example.com/greet?name=${name}`
      );
    });

    test(`testing with ${expected}`, async ({ page }) => {

      await expect(page.getByRole('heading'))
        .toHaveText(expected);

    });

  });

});
```

Here, each iteration creates its own `describe` scope.

The `beforeEach()` hook belongs to that scope and runs before its corresponding test.

Interview point: If you want hooks specific to each parameterized group, place them inside `test.describe()`.

3. Parameterized Projects

Simple meaning:

Project-level parameterization means running the same test suite with different configurations.

For example, you want to test an application with different users:

- Project 1: Alice
- Project 2: Bob

Instead of creating separate test files, you can configure multiple projects in Playwright.

Step 1: Create a custom test fixture

File: `my-test.ts`

```
import { test as base } from '@playwright/test';

export type TestOptions = {
  person: string;
};

export const test = base.extend<TestOptions>({

  person: ['John', { option: true }],

});
```

Explanation:

- `base.extend()` is used to extend Playwright's existing fixtures.
- `person` is a custom option.
- `string` means the person value should be a string.
- `John` is the default value.
- `{ option: true }` identifies it as a configurable option.

Step 2: Use the option in the test

File: `example.spec.ts`

```
import { test } from './my-test';

test('test 1', async ({ page, person }) => {

  await page.goto('/index.html');

  await expect(page.locator('#node'))
    .toContainText(person);

});
```

Here, `person` receives the value configured for the current project.

Step 3: Configure multiple projects

File: `playwright.config.ts`

```
import { defineConfig } from '@playwright/test';

import type { TestOptions } from './my-test';

export default defineConfig<TestOptions>({

  projects: [

    {
      name: 'alice',
      use: { person: 'Alice' },
    },

    {
      name: 'bob',
      use: { person: 'Bob' },
    },

  ],

});
```

Explanation:

We created two projects:

- Alice project passes `Alice`.
- Bob project passes Bob.

The same test runs with both configurations.

Execution flow

Single test file

example.spec.ts

Playwright projects

Project Alice

person = Alice

Test execution

Project Bob

person = Bob

Test execution

Real-time example: You have an e-commerce application. You want to run the same checkout tests with different user accounts, roles, or configurations.

Project-level parameterization helps you do this.

Important difference

Test-level	Project-level
Uses different test data	Uses different configurations
Commonly uses arrays and `forEach()`	Uses `projects` in config

Creates separate tests for each data set	Runs tests under each project
Example: Alice, Bob, Charlie	Example: Chrome, Firefox, different users

4. Passing Environment Variables

Simple meaning:

Environment variables allow us to pass configuration values from outside the test code.

Examples:

- Username
- Password
- Application URL
- Environment name
- API keys

Why do we use them?

We should not hardcode sensitive information such as usernames and passwords directly into test files.

Step 1: Read environment variables

```
test('Login test', async ({ page }) => {

  await page.getByLabel('User Name')
    .fill(process.env.USER_NAME!);

  await page.getByLabel('Password')
    .fill(process.env.PASSWORD!);

});
```

Explanation:

- `process.env` accesses environment variables in Node.js.
- `USER_NAME` stores the username.
- `PASSWORD` stores the password.
- `!` is TypeScript's non-null assertion operator. It tells TypeScript to treat the value as non-null, but it does not verify that the variable actually exists.

Step 2: Pass variables from the command line

Bash:

```
USER_NAME=me PASSWORD=secret npx playwright test
```

PowerShell:

```
$env:USER_NAME="me"
$env:PASSWORD="secret"
npx playwright test
```

Now Playwright reads those values through `process.env`.

Environment-based URL configuration

Suppose you have two environments:

- Staging
- Production

You can configure the URL like this:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  use: {

    baseURL: process.env.STAGING === '1'
      ? 'http://staging.example.test/'
      : 'http://example.test/'

  }
```

```
});
```

Run staging:

```
STAGING=1 npx playwright test
```

Run production/default:

```
npx playwright test
```

Important: The value of an environment variable is generally a string, so we compare it with `'1'`, not the number `1`.

5. .env Files

Simple meaning:

A `.env` file stores environment variables in one separate file.

Instead of passing username, password and URL every time through the command line, we can store them in `.env`.

Step 1: Install dotenv

```
npm install dotenv
```

Step 2: Create a .env file

```
STAGING=0
USER_NAME=me
PASSWORD=secret
```

Step 3: Read the .env file

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

import dotenv from 'dotenv';
import path from 'path';

dotenv.config({
```

```
  path: path.resolve(__dirname, '.env')
});

export default defineConfig({

  use: {

    baseURL: process.env.STAGING === '1'
      ? 'http://staging.example.test/'
      : 'http://example.test/'

  }

});
```

Step 4: Access the values

In your test:

```
test('Login test', async ({ page }) => {

  await page.goto('/login');

  await page.getByLabel('User Name')
    .fill(process.env.USER_NAME!);

  await page.getByLabel('Password')
    .fill(process.env.PASSWORD!);

});
```

Run normally:

```
npx playwright test
```

Environment variables vs .env files

Environment variables	.env file
Values passed through the system or command line	Values stored in a file

Accessed using `process.env`	Also accessed using `process.env` after loading
Useful in CI/CD pipelines	Useful for local development
No dotenv required if already supplied	dotenv can load the file

Interview point: Never commit real credentials or secrets to GitHub. Add `.env` to `.gitignore` and use secure secret storage in CI/CD.

6. Create Tests Using a CSV File

Simple meaning:

CSV stands for Comma-Separated Values.

It stores test data in tabular format, similar to Excel.

Instead of writing test data directly inside the code, we can store it in a CSV file and read it dynamically.

Step 1: Create input.csv

```
test_case,some_value,some_other_value
value 1,value 11,foobar1
value 2,value 22,foobar21
value 3,value 33,foobar321
value 4,value 44,foobar4321
```

Here:

- `test_case`
- `some_value`
- `some_other_value`

are column names.

The remaining rows contain test data.

Step 2: Install CSV parser

```
npm install csv-parse
```

Step 3: Read CSV file in Playwright

File: `test.spec.ts`

```
import fs from 'fs';
import path from 'path';

import { test } from '@playwright/test';

import { parse } from 'csv-parse/sync';

const records = parse(
  fs.readFileSync(path.join(__dirname, 'input.csv')),
  {
    columns: true,
    skip_empty_lines: true
  }
);

for (const record of records) {

  test(`foo: ${record.test_case}`, async ({ page }) => {

    console.log(
      record.test_case,
      record.some_value,
      record.some_other_value
    );

  });

}
```

Explanation of important methods

Method	Explanation

`fs`	Node.js file system module
`readFileSync()`	Reads the CSV file
`path.join()`	Creates a file path
`__dirname`	Current directory of the executing module
`parse()`	Converts CSV data into JavaScript records
`columns: true`	Uses the first row as column names
`skip_empty_lines: true`	Ignores empty lines
`for...of`	Iterates through each CSV record

Execution flow

CSV file

input.csv – 4 data rows

Read file using fs

Parse using csv-parse

CSV converted into JavaScript records

for...of loop

Test 1

CSV row 1

Test 2

CSV row 2

Test 3

CSV row 3

Test 4

CSV row 4

Real-time example: Suppose you need to test 100 different customer records. Instead of manually creating 100 test cases, you can maintain customer information in a CSV file and generate tests automatically.

7. Important Playwright Interview Questions and Answers

These are the questions I recommend preparing from this topic.

Basic interview questions

Q1. What is parameterized testing in Playwright?

Answer:

Parameterized testing is a technique in which we execute the same test with different sets of input data.

In Playwright, we can use arrays with `forEach()` to generate multiple tests.

For example, we can test login functionality with multiple usernames and passwords without writing separate test cases for each user.

Q2. What are the advantages of parameterized testing?

Answer:

1. Reduces duplicate code.
2. Improves code reusability.
3. Makes test maintenance easier.
4. Supports testing multiple input combinations.
5. Improves test coverage.

Q3. How do you parameterize tests in Playwright?

Answer:

We can use an array of objects and `forEach()`.

Example:

```
const users = [
  { username: 'Alice', password: '123' },
  { username: 'Bob', password: '456' }
];

users.forEach(({ username, password }) => {

  test(`Login with ${username}`, async ({ page }) => {

    await page.goto('/login');

    await page.getByLabel('Username').fill(username);
    await page.getByLabel('Password').fill(password);

  });

});
```

Each iteration creates an independent test.

Q4. What is the difference between test-level and project-level parameterization?

Answer:

Test-level parameterization is used when we want to execute the same test with different test data.

Project-level parameterization is used when we want to execute tests with different configurations using Playwright projects.

For example:

- Test-level: Different usernames and passwords.
- Project-level: Different browsers, environments or user configurations.

Q5. What is `forEach()` in JavaScript?

Answer:

`forEach()` is an array method used to iterate over every element of an array.

In Playwright, we use it to generate multiple test cases from different data sets.

Q6. What is the purpose of `test.describe()`?

Answer:

`test.describe()` is used to group related test cases.

It also allows us to define hooks such as `beforeEach()` and `afterEach()` for a particular group of tests.

Q7. What is the difference between `beforeAll()` and `beforeEach()`?

Answer:

- `beforeAll()` executes once before all tests in its scope.
- `beforeEach()` executes before every individual test.

For example, if we have five tests:

- `beforeAll()` executes once.
- `beforeEach()` executes five times.

Q8. What are environment variables in Playwright?

Answer:

Environment variables are external configuration values used to manage test data and application settings.

We access them using `process.env`.

Example:

```
process.env.USER_NAME
process.env.PASSWORD
```

They help avoid hardcoding credentials and URLs in test scripts.

Q9. What is a `.env` file?

Answer:

A `.env` file is used to store environment-specific configuration values such as usernames, passwords and application URLs.

We can use the `dotenv` package to load these values into `process.env`.

Q10. What is the purpose of `dotenv`?

Answer:

`dotenv` is an npm package that loads environment variables from a `.env` file into `process.env`.

Q11. How can you execute Playwright tests using environment variables?

Answer:

In Bash:

```
USER_NAME=admin PASSWORD=admin123 npx playwright test
```

In the test file:

```
process.env.USER_NAME
process.env.PASSWORD
```

Q12. What is project-level parameterization?

Answer:

Project-level parameterization allows us to execute the same test suite with different configurations using Playwright projects.

We define custom options using fixtures and provide different values in `playwright.config.ts`.

Q13. What is `base.extend()`?

Answer:

`base.extend()` is used to create custom fixtures or extend existing Playwright fixtures.

For example, we can create a custom fixture for a username, user role or application configuration.

Q14. How do you read a CSV file in Playwright?

Answer:

We can use Node.js `fs` module to read the CSV file and `csv-parse` to convert the CSV data into JavaScript objects.

Then we can iterate through the records using `for...of` or `forEach()` and generate tests.

Q15. What is the difference between `forEach()` and `for...of`?

Answer:

- `forEach()` is an array method that executes a callback for each array element.
- `for...of` is a loop used to iterate over iterable values such as arrays.

Both can be used to generate parameterized tests.

Scenario-based interview questions

Q16. Suppose you have 50 usernames and passwords. How will you test login functionality?

Answer:

I would use parameterized testing.

I would store the test data in an array or CSV file and iterate through each record.

For every record, I would:

1. Open the login page.
2. Enter the username.
3. Enter the password.
4. Click the login button.
5. Verify the expected result.

This avoids writing 50 separate test cases.

Q17. Suppose you want to run the same test on Chrome, Firefox and WebKit. How will you do that?

Answer:

I would configure multiple projects in `playwright.config.ts`.

```
projects: [
  { name: 'chromium', use: { browserName: 'chromium' } },
  { name: 'firefox', use: { browserName: 'firefox' } },
  { name: 'webkit', use: { browserName: 'webkit' } }
]
```

This allows the same test suite to run across multiple browsers.

Q18. Why should we avoid hardcoding usernames and passwords?

Answer:

Hardcoding credentials creates security risks because sensitive information may be exposed in source code or Git repositories.

Instead, I would use environment variables, `.env` files for local development and secure CI/CD secrets for pipelines.

Q19. What happens if one parameterized test fails?

Answer:

Each iteration creates an independent test.

If one test fails, Playwright reports that particular test as failed, while the other tests can still execute depending on the configured execution and failure settings.

Q20. Can we use hooks inside `forEach()`?

Answer:

Yes. We can put hooks inside `test.describe()` within each iteration if we need hooks specific to each parameterized group.

If hooks are declared outside the loop, they apply to the tests in their enclosing scope rather than being recreated for every iteration.

8. Quick Revision Notes

Topic	Remember this
Parameterized testing	Same test, different data
`forEach()`	Iterates over an array
`test.describe()`	Groups tests
`beforeEach()`	Before every test
`afterEach()`	After every test
Project parameterization	Different test configurations
`base.extend()`	Creates custom fixtures/options
`process.env`	Accesses environment variables
`.env`	Stores configuration values
`dotenv`	Loads `.env` values
`fs`	Reads files
`csv-parse`	Parses CSV data
`for...of`	Iterates through records

9. Test Your Understanding

Try answering these interview-style MCQs.

Playwright parameterization quiz

5/5 answered

1. Which method is commonly used for test-level parameterization in Playwright?

map()
forEach()
filter()
reduce()

Correct

forEach() iterates over test data and creates a separate test for each item.

2. Which object is used to access environment variables in Node.js?

process.env
browser.env
test.env
page.env

Correct

process.env provides access to environment variables.

3. Which package is commonly used to load .env files?

csv-parse
dotenv
fs
path

Correct

dotenv loads values from a .env file into process.env.

4. Which Playwright method is used to create custom fixtures?

test.create()
test.describe()

base.extend()
test.config()

Correct

base.extend() extends Playwright's fixtures and allows custom options.

5. Which option treats the first CSV row as column names?

skip_empty_lines: true
columns: true
header: false
first_row: true

Correct

columns: true uses the first CSV row as object property names.

Score: 5/5

Try again

Interview preparation tip: Focus especially on `forEach()`, `test.describe()`, hooks, project-level parameterization, `process.env`, `.env`, and CSV-based testing. Be ready to explain one practical example rather than only giving definitions.

Send me the next Playwright documentation section, and I'll explain it in the same format with examples and interview Q&A.

Playwright – Projects

Let's understand Projects in Playwright from basics to interview level, with simple explanations, practical examples, execution flow, and detailed interview questions and answers.

1. What is a Project in Playwright?

Simple meaning:

A project is a logical group of tests that runs with the same configuration.

We use projects when we want to execute the same test cases with different configurations.

For example, suppose you have an e-commerce application. You want to test the login functionality on:

- Chrome
- Firefox
- Safari
- Mobile Chrome
- Mobile Safari

Instead of creating separate test files for every browser, you can configure multiple projects in `playwright.config.ts`.

The same test cases will run across all configured projects.

Why do we use projects?

1. Cross-browser testing.
2. Mobile and tablet testing.
3. Testing on different environments (staging and production).
4. Running tests with different timeouts and retries.
5. Separating Smoke and Regression test suites.
6. Running tests with different configurations, such as logged-in and logged-out users.

Important interview definition:

A Playwright project is a logical group of tests that share the same configuration. It allows us to execute tests across different browsers, devices, environments and configurations.

2. Configure Projects for Multiple Browsers

Playwright supports three main browser engines:

Browser	Browser engine
Chromium	Chromium
Firefox	Firefox
WebKit	WebKit

We can also test branded browsers such as Google Chrome and Microsoft Edge.

Example: playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({

  projects: [

    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },

    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },

    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
```

```
    // Mobile browsers

    {
      name: 'Mobile Chrome',
      use: { ...devices['Pixel 5'] },
    },

    {
      name: 'Mobile Safari',
      use: { ...devices['iPhone 12'] },
    },

    // Branded browsers

    {
      name: 'Microsoft Edge',
      use: {
        ...devices['Desktop Edge'],
        channel: 'msedge'
      },
    },

    {
      name: 'Google Chrome',
      use: {
        ...devices['Desktop Chrome'],
        channel: 'chrome'
      },
    },

  ],

});
```

Understand the code step by step

1. Import defineConfig and devices

```
import { defineConfig, devices } from '@playwright/test';
```

- `defineConfig`: Used to define Playwright configuration.

- `devices`: Provides predefined device configurations.

2. Create projects

```
projects: [
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'] },
  }
]
```

Here:

- `name` identifies the project.
- `use` defines the browser and device configuration.
- `Desktop Chrome` provides predefined desktop browser settings.

3. What is the spread operator (`...`)?

```
use: { ...devices['Desktop Chrome'] }
```

The spread operator copies the properties from the predefined device configuration into the `use` object.

These configurations can include viewport size, browser settings, device scale factor and other emulation settings.

4. What is `channel`?

```
channel: 'chrome'
```

It tells Playwright to launch the installed branded Google Chrome browser instead of its bundled Chromium browser.

Similarly:

```
channel: 'msedge'
```

Launches Microsoft Edge.

Important difference:

- `chromium` means Playwright's Chromium browser.

- `channel: 'chrome'` means branded Google Chrome.
- `channel: 'msedge'` means branded Microsoft Edge.

3. How to Run Projects

By default, Playwright runs all configured projects.

Run all projects

```
npx playwright test
```

Suppose you have seven projects. Playwright runs the tests in all seven configurations.

Example output:

```
Running 7 tests using 5 workers

✓ [chromium] basic test
✓ [firefox] basic test
✓ [webkit] basic test
✓ [Mobile Chrome] basic test
✓ [Mobile Safari] basic test
✓ [Microsoft Edge] basic test
✓ [Google Chrome] basic test
```

Here, the same test executes under different project configurations.

Run only one project

```
npx playwright test --project=firefox
```

This command executes tests only in the Firefox project.

Other examples:

```
npx playwright test --project=chromium
npx playwright test --project="Mobile Chrome"
```

Interview point: We use `--project` to execute tests for a specific project.

4. Run Projects Using VS Code

You can also run Playwright projects directly from VS Code without using the terminal.

Steps:

1. Open VS Code.
2. Open the Testing sidebar.
3. Find the Playwright test runner.
4. Click the dropdown arrow next to the Play button.
5. Select the required browser profile.
6. Select multiple profiles or all profiles if needed.

For example, you can choose Chrome, Firefox or WebKit.

Remember: The default profile may use Chrome. You can change the default profile from the Testing sidebar.

5. Configure Projects for Multiple Environments

Simple meaning:

We can use projects to run the same tests against different application environments.

Common environments:

- Development
- Testing
- Staging
- Production

Real-time example

Suppose your company has two application environments:

Environment	URL	Retries
Staging	staging.example.com	2
Production	production.example.com	0

You want to execute the same tests in both environments but with different retry settings.

Code: playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  timeout: 60000,

  projects: [

    {
      name: 'staging',

      use: {
        baseURL: 'https://staging.example.com',
      },

      retries: 2,
    },

    {
      name: 'production',

      use: {
        baseURL: 'https://production.example.com',
      },
```

```
      retries: 0,
    },

  ],

});
```

Explanation

1. Common timeout

```
timeout: 60000
```

The default test timeout is set to 60 seconds for all projects unless overridden at a more specific level.

2. Staging project

```
{
  name: 'staging',

  use: {
    baseURL: 'https://staging.example.com',
  },

  retries: 2,
}
```

- Project name is staging.
- Application URL points to staging.
- If a test fails, Playwright can retry it up to two times.

3. Production project

```
{
  name: 'production',

  use: {
    baseURL: 'https://production.example.com',
  },
```

```
  retries: 0,
}
```

- Project name is production.
- Application URL points to production.
- No retries are configured.

How do you use baseURL in a test?

```
test('Login test', async ({ page }) => {

  await page.goto('/login');

});
```

Playwright automatically combines `/login` with the configured `baseURL`.

For example:

- Staging: https://staging.example.com/login
- Production: https://production.example.com/login

Interview point: Project-level configuration allows us to run the same test suite against multiple environments without changing the test code.

6. Splitting Tests into Projects

Simple meaning:

We can divide test cases into different groups using projects.

For example, in a real-time project, we have:

1. Smoke tests.
2. Regression tests.

We may want to execute Smoke tests quickly without retries and Regression tests with two retries.

We can create separate projects for these groups.

Code: playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  timeout: 60000,

  projects: [

    {
      name: 'Smoke',

      testMatch: /.*smoke.spec.ts/,

      retries: 0,
    },

    {
      name: 'Default',

      testIgnore: /.*smoke.spec.ts/,

      retries: 2,
    },

  ],

});
```

Understand important properties

1. testMatch

```
testMatch: /.*smoke.spec.ts/
```

It tells Playwright to include test files matching the specified pattern.

For example:

```
login.smoke.spec.ts
checkout.smoke.spec.ts
```

Both match the pattern.

2. testIgnore

```
testIgnore: /.*smoke.spec.ts/
```

It tells Playwright to exclude files matching the specified pattern.

For example, Smoke test files will not be included in the Default project.

3. retries

```
retries: 0
```

No retries.

```
retries: 2
```

Playwright retries a failed test up to two additional times.

Example folder structure

```
tests/
│
├── login.smoke.spec.ts
├── checkout.smoke.spec.ts
├── login.regression.spec.ts
├── payment.regression.spec.ts
└── profile.regression.spec.ts
```

Execution

All test files

5 test files

Smoke project

0 retries

login.smoke.spec.ts

checkout.smoke.spec.ts

Default project

2 retries

login.regression.spec.ts

payment.regression.spec.ts

profile.regression.spec.ts

Why is this useful?

Suppose developers deploy a new build. We want to execute Smoke tests first to verify that the critical functionality works.

After that, we can execute the complete Regression suite.

Interview point: `testMatch` includes matching test files, whereas `testIgnore` excludes matching test files.

7. Project Dependencies

This is an important topic for interviews.

Simple meaning:

Project dependencies mean one project must execute successfully before another dependent project starts.

In other words, one project depends on another project.

Real-time example

Suppose you are testing an e-commerce application.

Before executing browser tests, you need to:

1. Open the application.
2. Log in.
3. Save the authentication state.
4. Run tests using that saved login session.

Instead of logging in before every test, you can create a separate setup project.

Then your browser projects depend on that setup project.

Code: playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({

  projects: [

    {
      name: 'setup',

      testMatch: '**/*.setup.ts',
    },

    {
      name: 'chromium',

      use: { ...devices['Desktop Chrome'] },

      dependencies: ['setup'],
    },

    {
      name: 'firefox',

      use: { ...devices['Desktop Firefox'] },

      dependencies: ['setup'],
    },

    {
      name: 'webkit',

      use: { ...devices['Desktop Safari'] },

      dependencies: ['setup'],
    },

  ],
```

```
});
```

Explanation

Step 1: Setup project

```
{
  name: 'setup',
  testMatch: '**/*.setup.ts',
}
```

This creates a project named `setup`.

`testMatch` tells Playwright to execute test files ending with `.setup.ts`.

For example:

`auth.setup.ts`

Step 2: Chromium project

```
{
  name: 'chromium',

  use: { ...devices['Desktop Chrome'] },

  dependencies: ['setup'],
}
```

Here:

- Project name is chromium.
- It runs using Desktop Chrome configuration.
- `dependencies: ['setup']` means setup must complete successfully before Chromium tests start.

The same dependency is configured for Firefox and WebKit.

Example: Authentication setup file

File: `auth.setup.ts`

```
import { test as setup, expect } from '@playwright/test';

setup('authenticate', async ({ page }) => {

  await page.goto('/login');

  await page.getByLabel('Username').fill('admin');

  await page.getByLabel('Password').fill('admin123');

  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page).toHaveURL(/dashboard/);

  await page.context().storageState({
    path: 'playwright/.auth/user.json',
  });

});
```

This example logs in and saves the authentication state.

The dependent browser projects can reuse that state by configuring `storageState` in their `use` settings.

For example:

```
{
  name: 'chromium',

  use: {
    ...devices['Desktop Chrome'],
    storageState: 'playwright/.auth/user.json',
  },

  dependencies: ['setup'],
}
```

Important: A dependency alone does not automatically share login sessions or browser data. You must save and configure the authentication state separately.

Execution sequence

Setup project

Login and save authentication state

Setup must pass first

Chromium

Run

Firefox

Run

WebKit

Run

Dependent projects can run in parallel, subject to worker limits.

What happens if the setup project fails?

Suppose you have three projects:

- Setup
- Chromium
- Firefox

If Setup fails, Chromium and Firefox will not run because they depend on Setup.

Multiple dependencies

Suppose the E2E project depends on two projects:

- Browser Login
- Database

Both dependency projects run first.

Project	Result
Browser Login	Passed
Database	Failed
E2E Tests	Not run

Even though Browser Login passed, E2E tests will not run because Database failed.

Interview point: All project dependencies must pass before the dependent project can execute.

8. Project Teardown

Simple meaning:

Teardown is used to clean up resources after dependent projects have finished execution.

For example:

- Setup: Create a test user.
- Execution: Run browser tests.
- Teardown: Delete the test user.

Example

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  projects: [

    {
      name: 'setup',

      testMatch: '**/*.setup.ts',
```

```
      teardown: 'cleanup',
    },

    {
      name: 'chromium',

      dependencies: ['setup'],
    },

    {
      name: 'cleanup',

      testMatch: '**/*.teardown.ts',
    },

  ],

});
```

Here:

- `setup` executes first.
- Chromium tests execute after setup passes.
- `cleanup` executes after the dependent projects finish.

The cleanup project is linked to setup through the `teardown` property.

Difference between setup and teardown:

Setup	Teardown
Executes before dependent projects	Executes after dependent projects
Prepares test data	Cleans test data
Example: Create user	Example: Delete user

9. Test Filtering with Project Dependencies

Simple meaning:

Test filtering means selecting only the tests or projects you want to execute instead of running everything.

Playwright provides different filtering options.

Command / option	Purpose
`--grep`	Runs tests matching a title pattern
`--grep-invert`	Excludes tests matching a title pattern
`--shard`	Divides test execution into smaller groups
`test.only()`	Runs only the selected test
Test location	Runs tests from a specified file or location

Example 1: Run tests matching a title

```
npx playwright test --grep "Login"
```

Runs tests whose titles match Login.

Example 2: Exclude tests matching a title

```
npx playwright test --grep-invert "Login"
```

Runs tests whose titles do not match Login.

Example 3: Run a particular project

```
npx playwright test --project=chromium
```

What happens to dependencies when we filter tests?

Important concept:

If you select tests belonging to a project that has dependencies, Playwright also runs the dependency projects.

For example:

`Setup → Chromium → Login test`

If you select a Login test from Chromium, Setup also runs.

What is `--no-deps`?

```
npx playwright test --project=chromium --no-deps
```

This tells Playwright to ignore project dependencies and teardowns.

Only the directly selected project runs.

Interview point: `--no-deps` skips dependency and teardown projects. Use it when you intentionally want to run the selected project without its dependency workflow.

10. Important Playwright Interview Questions and Answers

These questions cover basic, intermediate and scenario-based interview questions.

Basic interview questions

Q1. What is a project in Playwright?

Answer:

A project is a logical group of tests that runs with the same configuration.

We use projects to execute the same test cases across different browsers, devices, environments and configurations.

For example, we can create separate projects for Chromium, Firefox and WebKit in `playwright.config.ts`.

Q2. Why do we use projects in Playwright?

Answer:

We use projects for:

1. Cross-browser testing.
2. Mobile and tablet testing.
3. Testing different environments.
4. Running tests with different timeouts and retries.
5. Separating Smoke and Regression tests.
6. Running tests with different user configurations.

Q3. Where do we configure projects in Playwright?

Answer:

We configure projects inside the `projects` property of `playwright.config.ts`.

Q4. Which browsers does Playwright support?

Answer:

Playwright supports three main browser engines:

- Chromium
- Firefox
- WebKit

It also supports branded browsers such as Google Chrome and Microsoft Edge.

Q5. How do you run all projects in Playwright?

Answer:

```
npx playwright test
```

By default, Playwright runs all configured projects.

Q6. How do you execute a specific project?

Answer:

We use the `--project` command-line option.

```
npx playwright test --project=firefox
```

This executes only the Firefox project.

Q7. What is the purpose of `devices` in Playwright?

Answer:

The `devices` object provides predefined browser and device configurations.

For example:

```
use: { ...devices['iPhone 12'] }
```

It allows us to emulate mobile devices with predefined settings such as viewport size and device scale factor.

Q8. What is the difference between Chromium and Chrome?

Answer:

Chromium is an open-source browser project and is the browser engine used by Playwright's bundled Chromium browser.

Google Chrome is a branded browser based on Chromium.

We can launch branded Chrome using:

```
use: {
  browserName: 'chromium',
  channel: 'chrome'
}
```

Q9. What is the difference between `testMatch` and `testIgnore`?

Answer:

- `testMatch` specifies which test files should be included.
- `testIgnore` specifies which test files should be excluded.

For example, we can use `testMatch` to execute only Smoke test files.

Q10. What is the purpose of retries in Playwright?

Answer:

Retries allow Playwright to execute a failed test again.

For example:

```
retries: 2
```

It means Playwright can retry a failed test up to two additional times.

Q11. What is a project dependency?

Answer:

A project dependency means one project must complete successfully before another project starts.

For example, browser tests can depend on a setup project that performs login and saves authentication state.

Q12. What is teardown in Playwright?

Answer:

Teardown is used to perform cleanup activities after dependent projects have finished.

For example, deleting test users, cleaning test data or closing external resources.

Q13. What is `--no-deps`?

Answer:

`--no-deps` is a command-line option used to skip project dependencies and teardown projects.

```
npx playwright test --project=chromium --no-deps
```

Q14. Can multiple Playwright projects run in parallel?

Answer:

Yes. Playwright can execute multiple independent projects in parallel, subject to the configured worker limit.

However, if a project has dependencies, those dependencies must finish successfully before the dependent project starts.

Q15. What happens if a dependency project fails?

Answer:

If a dependency project fails, the projects that depend on it will not execute.

For example, if the setup project fails, Chromium, Firefox and WebKit projects that depend on it will not run.

Scenario-based interview questions

Q16. Suppose you want to execute the same test on Chrome, Firefox and Safari. How will you achieve this?

Answer:

I would configure three separate projects in `playwright.config.ts`.

```
projects: [
  {
    name: 'chromium',
    use: { browserName: 'chromium' }
  },
  {
    name: 'firefox',
    use: { browserName: 'firefox' }
  },
  {
    name: 'webkit',
    use: { browserName: 'webkit' }
  }
]
```

Then I would execute:

```
npx playwright test
```

Playwright will execute the tests across all three browser projects.

Q17. Suppose your company has Staging and Production environments. How will you execute tests in both?

Answer:

I would create two projects in the Playwright configuration.

The Staging project will have the staging URL, and the Production project will have the production URL.

I can also configure different retry settings for each project.

This allows us to execute the same test scripts against both environments without modifying the test code.

Q18. Suppose you want to execute Smoke tests without retries and Regression tests with two retries. What will you do?

Answer:

I would create two projects:

- Smoke project with `testMatch` and `retries: 0`.
- Regression project with `testIgnore` and `retries: 2`.

This separates test execution and allows us to configure different retry strategies.

Q19. Suppose you need to log in before executing 100 test cases. How will you avoid logging in before every test?

Answer:

I would use Playwright project dependencies and authentication state.

First, I would create a setup project to perform login and save the authentication state using `storageState()`.

Then, I would configure the browser projects to depend on the setup project and reuse the saved authentication state.

This avoids repeating the login process for every test.

Q20. Suppose your setup project fails. Will Chromium and Firefox tests execute?

Answer:

No. If Chromium and Firefox depend on the setup project, they will not execute when setup fails.

Q21. What is the difference between projects and test.describe()?

Answer:

`test.describe()` is used to group related test cases inside a test file.

Projects are used to group tests with different configurations.

For example:

- `test.describe()`: Groups Login and Logout tests.
- Projects: Runs tests on Chrome, Firefox or different environments.

Q22. Can we configure different timeouts for different projects?

Answer:

Yes. We can configure timeout settings globally or at the project level.

For example:

```
projects: [
  {
    name: 'staging',
    timeout: 60000
  },
  {
    name: 'production',
    timeout: 30000
  }
]
```

This allows different timeout settings for different projects.

Q23. What is the difference between `--grep` and `--grep-invert`?

Answer:

- `--grep` executes tests matching a particular title.
- `--grep-invert` excludes tests matching a particular title.

For example:

```
npx playwright test --grep "Login"
```

Runs matching Login tests.

```
npx playwright test --grep-invert "Login"
```

Excludes matching Login tests.

Q24. What is the difference between project dependencies and teardown?

Answer:

Dependencies define which projects must execute before another project.

Teardown defines which cleanup project should execute after the dependent projects have finished.

For example, setup creates test data, browser projects use it, and teardown deletes it.

11. Quick Revision Notes

Keep this table for last-minute interview preparation.

Concept	Important point
Project	Logical group of tests
`projects`	Configure projects
`devices`	Predefined device configurations
`channel`	Launch branded Chrome or Edge
`--project`	Run a specific project
`baseURL`	Base application URL
`retries`	Retry failed tests

`testMatch`	Include matching test files
`testIgnore`	Exclude matching test files
`dependencies`	Execute another project first
`teardown`	Execute cleanup project afterward
`--grep`	Filter tests by title
`--grep-invert`	Exclude tests by title
`--no-deps`	Skip dependencies and teardowns
`storageState()`	Save authentication state

12. Practical Interview Preparation

These are the most important topics I recommend practising in your Playwright framework:

Practice checklist

Your practice progress

0/8 completed
Create Chromium, Firefox and WebKit projects
Execute only the Firefox project using CLI
Configure Staging and Production projects
Create separate Smoke and Regression projects
Configure different retry settings
Create an authentication setup project
Reuse saved storageState in browser projects
Configure a teardown project

My interview advice: Give extra attention to these five questions:

1. What is a project in Playwright?
2. How do you execute tests in multiple browsers?
3. How do you configure different environments?
4. What are project dependencies?
5. How do you implement authentication setup and teardown?

These are especially useful for QA Automation and SDET interviews because they test whether you understand how to organize and execute a real-world automation framework.

Send the next Playwright documentation section whenever you're ready. I'll continue with the same simple explanation, code breakdown, practical examples and detailed interview Q&A.

Playwright – Reporters

Let's study Reporters in Playwright in the same format: simple explanation, code breakdown, real-time examples and detailed interview questions and answers.

1. What is a Reporter in Playwright?

Simple meaning:

A reporter is a tool that displays and records the results of test execution.

After running automation test cases, we need to know:

- How many tests passed?
- How many tests failed?
- How many tests were skipped?
- Which test failed?
- What was the failure reason?
- How much time did test execution take?

Playwright reporters provide this information.

Real-time example:

Suppose you have 100 automation test cases.

After execution:

- 85 tests passed.
- 10 tests failed.
- 5 tests were skipped.

The reporter displays these results in a readable format.

Interview definition:

A reporter in Playwright is used to display and generate test execution results. It provides information about passed, failed and skipped tests, along with failure details and execution status.

2. Types of Reporters in Playwright

Playwright provides several built-in reporters.

Reporter	Purpose
List	Displays each test result in the terminal
Line	Displays the latest test progress in one line
Dot	Displays a symbol for each test
HTML	Generates an interactive HTML report
JSON	Generates test results in JSON format
JUnit	Generates XML test results

Blob	Stores complete test-run details, useful for merging sharded reports
Perfetto	Displays test execution as a timeline
GitHub	Creates failure annotations in GitHub Actions

Let's understand each reporter.

3. How to Configure a Reporter

There are two ways to configure reporters.

Method 1: Using the command line

```
npx playwright test --reporter=line
```

Here, `--reporter=line` tells Playwright to use the Line reporter.

Other examples:

```
npx playwright test --reporter=list
npx playwright test --reporter=html
npx playwright test --reporter=dot
```

Method 2: Using playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: 'line',

});
```

Here, the reporter is configured in the Playwright configuration file.

Interview point: We can configure reporters either through the command line or through `playwright.config.ts`.

4. Multiple Reporters

Simple meaning:

We can use multiple reporters in the same Playwright project.

For example, we may want:

- List reporter to display test results in the terminal.
- JSON reporter to save test execution results in a JSON file.

Code

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: [

    ['list'],

    ['json', {
      outputFile: 'test-results.json'
    }],

  ],

});
```

Explanation

```
reporter: [
  ['list'],
  ['json', { outputFile: 'test-results.json' }]
]
```

Here:

- List reporter displays test results in the terminal.
- JSON reporter generates a file named `test-results.json`.

Real-time example: In a company, testers may need readable terminal output while CI/CD tools need structured JSON test results.

Multiple reporters allow both requirements to be handled together.

5. Reporters on CI/CD

Simple meaning:

We can configure different reporters for local execution and CI/CD execution.

For example:

- Local execution: List reporter.
- CI/CD execution: Dot reporter.

Code

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: process.env.CI ? 'dot' : 'list',

});
```

Explanation

`process.env.CI` checks whether the CI environment variable is set.

- If CI is enabled, Playwright uses the Dot reporter.
- Otherwise, it uses the List reporter.

Why use Dot reporter in CI?

Because CI pipelines may execute hundreds or thousands of tests. Dot reporter provides concise output without printing every successful test individually.

6. Built-in Reporters Explained

6.1 List Reporter

Important

Simple meaning:

List reporter displays every test case separately in the terminal.

It shows:

- Test name.
- Test execution status.
- Execution time.
- Failure details.

It is the default reporter for normal local execution, except when CI defaults apply.

Command

```
npx playwright test --reporter=list
```

Configuration

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: 'list',

});
```

Example output

```
Running 5 tests using 2 workers

1 ✓ Login test (438ms)
2 ✓ Search test (515ms)
```

```
3 x Checkout test (691ms)
4 ✓ Logout test (932ms)
5 ✓ Payment test (500ms)
```

Here:

- ✓ means passed.
- x means failed.

Additional List reporter options

1. printSteps

```
reporter: [
  ['list', { printSteps: true }]
]
```

It prints individual test steps.

For example:

- Navigate to login page.
- Enter username.
- Enter password.
- Click login.
- Verify dashboard.

2. printFailuresInline

```
reporter: [
  ['list', { printFailuresInline: true }]
]
```

It displays failure details immediately after a failed test instead of waiting until the end.

3. omitTags

```
reporter: [
  ['list', { omitTags: true }]
]
```

It hides tags automatically appended to test titles.

Interview point: List reporter is useful when we want detailed test-by-test output in the terminal.

6.2 Line Reporter

Simple meaning:

Line reporter displays the latest completed test on a single line.

It is more concise than List reporter.

It does not continuously print every successful test as a separate line.

Command

```
npx playwright test --reporter=line
```

Configuration

```
reporter: 'line'
```

Example output

```
Running 124 tests using 6 workers

[23/124] gitignore.spec.ts
```

If a test fails, the failure details are displayed inline.

When do we use it?

When we have a large test suite and want to monitor execution progress without filling the terminal with too much output.

Interview point: Line reporter shows the latest test progress and displays failure information when it occurs.

6.3 Dot Reporter

Important

Simple meaning:

Dot reporter displays one symbol for each executed test.

It is very concise and is the default on CI.

Command

```
npx playwright test --reporter=dot
```

Configuration

```
reporter: 'dot'
```

Example output

```
Running 124 tests using 6 workers

......F.................................
```

Meaning of symbols

Symbol	Meaning
·	Passed
F	Failed
×	Failed or timed out and will be retried
±	Passed on retry (flaky)
T	Timed out
°	Skipped

Real-time example:

Suppose your automation suite contains 500 test cases.

Instead of printing 500 individual test names, Dot reporter displays compact symbols indicating their status.

Interview point: Dot reporter is useful for large test suites and CI/CD pipelines because it produces concise output.

6.4 HTML Reporter

Very Important

Simple meaning:

HTML reporter generates a visual, interactive report that we can open in a browser.

It provides detailed information about test execution.

It is useful for analysing test results.

Command

```
npx playwright test --reporter=html
```

After execution, Playwright generates an HTML report in:

```
playwright-report/
```

Open the report

```
npx playwright show-report
```

This opens the latest HTML report in your browser.

Running and debugging tests | Playwright
playwright.dev

Configure HTML reporter

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: [
    ['html', {
      open: 'never',
```

```
      outputFolder: 'my-report'
    }]
  ],

});
```

Understand important options

1. open

Controls when the report opens automatically.

Value	Meaning
`always`	Always open the report
`never`	Never open automatically
`on-failure`	Open when tests fail (default)

2. outputFolder

```
outputFolder: 'my-report'
```

Specifies where the HTML report will be saved.

3. host and port

These configure the hostname and port used to serve the HTML report.

Important commands:

```
npx playwright show-report
```

Open the default report.

```
npx playwright show-report my-report
```

Open a report from a custom folder.

Interview point: HTML reporter generates an interactive browser-based report containing detailed test results and failure information.

6.5 JSON Reporter

Simple meaning:

JSON reporter generates test execution results in JSON format.

JSON stands for JavaScript Object Notation.

It stores structured test information that can be consumed by other tools or scripts.

Command

```
npx playwright test --reporter=json
```

Configure JSON output file

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: [
    ['json', {
      outputFile: 'results.json'
    }]
  ],

});
```

Here, Playwright generates:

```
results.json
```

When do we use it?

When we want to integrate test execution results with other systems or process results programmatically.

Interview point: JSON reporter generates structured test execution results in JSON format.

6.6 JUnit Reporter

Simple meaning:

JUnit reporter generates test results in XML format following the JUnit-style reporting format.

It is useful for integrating test results with CI/CD tools.

Command

```
npx playwright test --reporter=junit
```

Configuration

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: [
    ['junit', {
      outputFile: 'results.xml'
    }]
  ],

});
```

It generates:

```
results.xml
```

When do we use it?

JUnit XML reports are commonly consumed by CI systems and test-result dashboards.

Interview point: JUnit reporter generates XML test reports for integration with CI/CD tools.

6.7 Blob Reporter

Simple meaning:

Blob reporter stores detailed information about a test run in a ZIP report.

Its primary purpose is to combine test reports from multiple shards.

First, understand sharding.

Sharding means dividing a large test suite into smaller groups so that they can run separately, often on different machines.

For example:

- Shard 1: Tests 1–50.
- Shard 2: Tests 51–100.
- Shard 3: Tests 101–150.

Each shard can generate a Blob report. These reports can later be merged into one final report.

Command

```
npx playwright test --reporter=blob
```

Default output folder:

```
blob-report/
```

The report is stored as a ZIP file.

When do we use it?

When tests are distributed across multiple machines or CI/CD jobs and we want to merge their results.

Interview point: Blob reporter generates ZIP reports containing detailed test execution information and is mainly used to merge reports from sharded test runs.

6.8 Perfetto Reporter

Simple meaning:

Perfetto reporter generates a JSON trace file that displays test execution as a timeline.

It helps analyse:

- Test execution duration.

- Worker activity.
- Before and after hooks.
- Fixtures.
- Individual test steps.

Command

```
npx playwright test --reporter=perfetto
```

Default output:

```
test-results/perfetto.json
```

You can open the generated trace file using Perfetto UI or Chrome tracing.

Interview point: Perfetto reporter is used to visualize test execution as a timeline and analyse worker activity and test steps.

6.9 GitHub Reporter

Simple meaning:

GitHub reporter creates automatic failure annotations when tests run in GitHub Actions.

It helps developers identify test failures directly in GitHub Actions.

Configuration

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: process.env.CI
    ? 'github'
    : 'list',

});
```

Here:

- In GitHub Actions CI, the GitHub reporter is used.
- During local execution, the List reporter is used.

Interview point: GitHub reporter is useful for showing test failure annotations directly in GitHub Actions.

7. Custom Reporters

Simple meaning:

Apart from built-in reporters, Playwright allows us to create our own custom reporter.

We can customize how test results are displayed or processed.

For example, we may want to:

- Print custom test execution messages.
- Log test start and end times.
- Send test results to an external system.
- Generate customized reports.

Example: my-awesome-reporter.ts

```
import type {
  FullConfig,
  FullResult,
  Reporter,
  Suite,
  TestCase,
  TestResult
} from '@playwright/test/reporter';

class MyReporter implements Reporter {

  onBegin(config: FullConfig, suite: Suite) {

    console.log(
      `Starting the run with ${suite.allTests().length} tests`
    );
```

```
  }

  onTestBegin(test: TestCase, result: TestResult) {

    console.log(`Starting test ${test.title}`);

  }

  onTestEnd(test: TestCase, result: TestResult) {

    console.log(
      `Finished test ${test.title}: ${result.status}`
    );

  }

  onEnd(result: FullResult) {

    console.log(
      `Finished the run: ${result.status}`
    );

  }

}

export default MyReporter;
```

Understand the important methods

Method	Purpose
`onBegin()`	Called when the test run starts
`onTestBegin()`	Called when an individual test starts
`onTestEnd()`	Called when an individual test finishes

`onEnd()`	Called when the entire test run finishes

Configure custom reporter

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  reporter: './my-awesome-reporter.ts',

});
```

Or run it through the command line:

```
npx playwright test --reporter="./my-awesome-reporter.ts"
```

Interview point: We create custom reporters by implementing the Playwright `Reporter` interface and its lifecycle methods.

8. Difference Between Important Reporters

This table is particularly useful for interviews.

Reporter	Output	Main purpose
List	Terminal	Displays every test
Line	Terminal	Shows latest test progress
Dot	Symbols	Compact execution status
HTML	HTML	Interactive visual report

JSON	JSON	Structured test results
JUnit	XML	CI/CD integration
Blob	ZIP	Merge sharded reports
Perfetto	JSON timeline	Performance and execution analysis
GitHub	Annotations	GitHub Actions failure details
Custom	User-defined	Customized reporting

9. Important Interview Questions and Answers

Basic questions

Q1. What is a reporter in Playwright?

Answer:

A reporter is used to display and generate test execution results.

It provides information about passed, failed and skipped tests, execution time and failure details.

Q2. What are the different built-in reporters in Playwright?

Answer:

Playwright provides List, Line, Dot, HTML, JSON, JUnit, Blob, Perfetto and GitHub reporters.

Q3. What is the default reporter in Playwright?

Answer:

List reporter is the default for local execution, while Dot reporter is the default on CI.

Q4. How can you configure a reporter in Playwright?

Answer:

We can configure reporters through the command line or `playwright.config.ts`.

Example:

```
npx playwright test --reporter=html
```

Or:

```
reporter: 'html'
```

Q5. Can we use multiple reporters in Playwright?

Answer:

Yes. We can configure multiple reporters in the `reporter` array.

For example, List reporter displays results in the terminal, while JSON reporter generates a JSON file.

Q6. What is the difference between List and Line reporters?

Answer:

List reporter displays every test case on a separate line.

Line reporter displays the latest completed test progress in a single line and reports failures inline.

Q7. What is the difference between Line and Dot reporters?

Answer:

Line reporter displays the latest test progress and failure information.

Dot reporter displays one character for every test, making it more concise.

Q8. Which reporter is useful for CI/CD pipelines?

Answer:

Dot reporter is useful for concise terminal output in CI.

JUnit reporter is useful when the CI system needs XML test results, and HTML reporter is useful for visual test-result analysis.

Q9. How do you generate an HTML report?

Answer:

We can execute:

```
npx playwright test --reporter=html
```

After execution, we can open the report using:

```
npx playwright show-report
```

Q10. Where is the HTML report stored by default?

Answer:

It is stored in the `playwright-report` folder.

Q11. What is the purpose of `outputFile`?

Answer:

`outputFile` specifies the location and name of the generated report file.

For example:

```
reporter: [
  ['json', { outputFile: 'results.json' }]
]
```

Q12. What is the purpose of the `open` option in HTML reporter?

Answer:

It controls when the HTML report should open automatically.

The available values are `always`, `never` and `on-failure`.

Q13. What is a JSON reporter?

Answer:

JSON reporter generates structured test execution results in JSON format, which can be consumed by other tools and scripts.

Q14. What is a JUnit reporter?

Answer:

JUnit reporter generates test execution results in XML format. It is useful for integration with CI/CD tools.

Q15. What is a Blob reporter?

Answer:

Blob reporter generates ZIP reports containing detailed test-run information. It is mainly used to merge reports from different shards.

Intermediate and scenario-based questions

Q16. What is a custom reporter?

Answer:

A custom reporter is a user-defined reporter that allows us to customize how Playwright test execution results are handled or displayed.

We create a class that implements the Playwright `Reporter` interface.

Q17. What are the important lifecycle methods in a custom reporter?

Answer:

- `onBegin()`: Executes when the test run starts.
- `onTestBegin()`: Executes when an individual test starts.
- `onTestEnd()`: Executes when an individual test finishes.
- `onEnd()`: Executes when the complete test run ends.

Q18. What is the difference between JSON and JUnit reporters?

Answer:

JSON reporter generates structured test results in JSON format.

JUnit reporter generates XML-format test results, commonly used by CI/CD systems.

Q19. Suppose you have 1,000 test cases. Which reporter would you prefer for terminal execution?

Answer:

I would prefer Dot reporter because it generates concise output using symbols for test statuses. It avoids displaying every test name individually.

Q20. Suppose your manager wants an interactive report after test execution. Which reporter would you use?

Answer:

I would use the HTML reporter because it provides a browser-based report with detailed test results and failure information.

Q21. Suppose your CI/CD tool requires XML test results. Which reporter would you use?

Answer:

I would use the JUnit reporter because it generates XML-format test execution results.

Q22. Suppose you want to see test results in the terminal and also generate a JSON file. What will you do?

Answer:

I would configure multiple reporters.

```
reporter: [
  ['list'],
  ['json', { outputFile: 'results.json' }]
]
```

Q23. What is the difference between built-in and custom reporters?

Answer:

Built-in reporters are provided by Playwright, such as HTML, List, JSON and JUnit.

Custom reporters are developed by implementing the Reporter interface to meet specific project requirements.

Q24. What is the purpose of the GitHub reporter?

Answer:

It generates automatic test failure annotations in GitHub Actions, making it easier to identify failures directly in the CI workflow.

Q25. What is Perfetto reporter?

Answer:

Perfetto reporter generates a trace-event JSON file that displays test execution as a timeline, including workers, hooks, fixtures and test steps.

10. Quick Revision Notes

Topic	Remember
Reporter	Displays test execution results
List	Every test on a separate line
Line	Latest test progress
Dot	One symbol per test
HTML	Interactive browser report
JSON	Structured JSON output
JUnit	XML output
Blob	ZIP report for merging shards
Perfetto	Execution timeline
GitHub	Failure annotations

Custom reporter	User-defined reporting
`onBegin()`	Test run starts
`onTestBegin()`	Individual test starts
`onTestEnd()`	Individual test ends
`onEnd()`	Entire test run ends
`show-report`	Opens HTML report

11. Test Your Understanding – Interview MCQs

Reporters – MCQ practice

7/7 answered

1. Which reporter generates an interactive browser-based report?

Dot
Line
HTML
JSON

Correct

HTML reporter creates a visual report that can be opened in a browser.

2. Which reporter displays one symbol for each test?

List
Dot
JUnit
HTML

Correct

Dot reporter uses symbols such as ·, F, T and °.

3. Which reporter generates XML output?

JSON
Blob
JUnit
Perfetto

Correct

JUnit reporter generates JUnit-style XML reports.

4. Which reporter is mainly used to merge reports from sharded test runs?

List
HTML
Blob
Line

Correct

Blob reports store detailed test-run information and facilitate merging shard reports.

5. Which method is called when an individual test finishes in a custom reporter?

onBegin()
onEnd()
onTestBegin()
onTestEnd()

Correct

onTestEnd() is called when an individual test finishes.

6. Which command opens the last HTML report?

npx playwright test
npx playwright show-report
npx playwright open-html
npx playwright report

Correct

Use npx playwright show-report to serve and open the latest HTML report.

7. Which reporter creates failure annotations in GitHub Actions?

GitHub
Perfetto
Blob
List

Correct

The GitHub reporter generates automatic failure annotations in GitHub Actions.

Score: 7/7

Try again

My recommendation for your QA Automation / SDET interviews: Practise HTML, List, Dot, JSON, JUnit and custom reporters particularly well. Understand when to use each one and be able to explain how you configured reporting in your own Playwright framework.

Send the next Playwright section whenever you're ready. I'll continue with the same format.

Playwright – Retries

This is an important Playwright topic for QA Automation and SDET interviews. Let's understand it from basics with practical examples.

1. What are retries in Playwright?

Retries mean automatically executing a failed test again.

Sometimes a test fails because of temporary or unpredictable issues, such as:

- Network delays.
- Slow application response.
- Temporary server issues.
- Element loading inconsistently.

- Timing-related issues.

These tests are sometimes called flaky tests.

Example:

Suppose you execute 3 tests:

1. Login test – Passed.
2. Add to Cart test – Failed.
3. Checkout test – Passed.

If retries are enabled, Playwright automatically runs the failed Add to Cart test again.

If it passes on retry, Playwright marks it as flaky, not simply passed.

Important: Retries help identify intermittent failures, but they do not fix the root cause of flaky tests.

2. How does Playwright handle worker processes?

Before understanding retries, understand workers.

A worker is an independent operating system process in which Playwright executes tests.

- Each worker has its own browser.
- Workers have identical environments.
- Tests can execute in separate workers.
- Playwright Test manages these workers.

Scenario 1: All tests pass

Consider this code:

```
import { test } from '@playwright/test';

test.describe('suite', () => {
  test.beforeAll(async () => {
    console.log('Before All');
```

```
  });

  test('first good', async ({ page }) => {
    console.log('First test');
  });

  test('second flaky', async ({ page }) => {
    console.log('Second test');
  });

  test('third good', async ({ page }) => {
    console.log('Third test');
  });

  test.afterAll(async () => {
    console.log('After All');
  });
});
```

Execution flow:

Worker 1

1. Worker process starts.
2. `beforeAll()` executes.
3. First test passes.
4. Second test passes.
5. Third test passes.
6. `afterAll()` executes.

All tests passed

All tests run in the same worker in this example.

Scenario 2: A test fails without retries

Suppose the second test fails.

What happens?

Playwright discards the entire worker process and its browser. It starts a new worker to continue with the next test.

Worker 1

1. `beforeAll()` executes.
2. First test passes.
3. Second test fails
4. Worker is discarded.

Worker 2

1. `beforeAll()` executes again.
2. Third test executes and passes.
3. `afterAll()` executes.

Important interview point: When a worker is discarded, its browser is also discarded. The `beforeAll()` hook runs again in the replacement worker.

Scenario 3: A test fails with retries enabled

Now suppose we configure one retry.

Worker 1

- `beforeAll()` executes.
- First test passes.
- Second test fails.
- Worker is discarded.

Worker 2

- `beforeAll()` executes again.
- Second test is retried and passes.
- Third test passes.
- `afterAll()` executes.

Second test passed on retry

This is how Playwright isolates failures from healthy tests.

3. How to configure retries

By default, Playwright does not retry failed tests.

You can enable retries in two ways.

Method 1: Using the command line

```
npx playwright test --retries=3
```

This means a failed test can get up to 3 additional attempts.

For example:

- First execution: fails.
- Retry 1: fails.
- Retry 2: fails.
- Retry 3: passes.

The test passes after 3 retries and is classified as flaky.

Method 2: Using playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  retries: 3,
});
```

Now all tests in this configuration can be retried up to 3 times.

4. Three test statuses in Playwright

Playwright categorizes test results into three types.

Status	Meaning
Passed	Test passed on the first execution.

Flaky	Test failed initially but passed on retry.
Failed	Test failed initially and on all retries.

Example:

Suppose you have 3 tests:

- Login – Passed first time.
- Add to Cart – Failed first time, passed on retry.
- Checkout – Failed on first execution and all retries.

Result:

- 1 Passed.
- 1 Flaky.
- 1 Failed.

Interview point: A flaky test is not a completely reliable success. It indicates that the test needs investigation.

5. Detect retries using testInfo.retry

Playwright provides `testInfo.retry` to identify the current retry attempt.

Example:

```
import { test, expect } from '@playwright/test';

test('my test', async ({ page }, testInfo) => {

  if (testInfo.retry) {
    await cleanSomeCachesOnTheServer();
  }

  // Test steps
});
```

Explanation:

- `testInfo` provides information about the currently running test.
- `testInfo.retry` tells you which retry attempt is running.
- Its value is `0` on the first execution.
- Its value is `1` on the first retry, `2` on the second retry, and so on.
- If it is greater than `0`, the test is being retried.

Real-world example:

Suppose your e-commerce application has temporary server-side cart data.

If the Add to Cart test fails, you might need to clear the old cart state before retrying.

```
if (testInfo.retry) {
  await clearCartData();
}
```

This is useful when retrying requires cleanup.

6. Configure retries for a specific test group

You don't always need to enable retries globally.

You can configure retries for a particular `describe` group using `test.describe.configure()`.

```
import { test, expect } from '@playwright/test';

test.describe(() => {

  test.describe.configure({ retries: 2 });

  test('test 1', async ({ page }) => {
    // Test steps
  });

  test('test 2', async ({ page }) => {
    // Test steps
  });
```

```
});
```

Here:

- Only tests inside this group get the configured retry setting.
- Each failed test can be retried up to 2 times.
- Other tests outside this group use their applicable configuration.

Interview question: Can we configure retries for a specific group of tests?

Answer: Yes. We can use `test.describe.configure({ retries: 2 })` inside a `describe` block.

7. What is serial mode?

Normally, Playwright encourages tests to be independent.

But sometimes tests depend on each other.

For example:

1. Create a user.
2. Update the user's profile.
3. Delete the user.

The second test depends on the first test, and the third depends on the previous tests.

For such cases, Playwright provides serial mode.

You can configure it using:

```
test.describe.configure({ mode: 'serial' });
```

Or use `test.describe.serial()`.

How serial mode works

Consider:

```
import { test } from '@playwright/test';
```

```
test.describe.configure({ mode: 'serial' });

test.beforeAll(async () => {
  // Setup
});

test('first good', async ({ page }) => {
  // Test 1
});

test('second flaky', async ({ page }) => {
  // Test 2
});

test('third good', async ({ page }) => {
  // Test 3
});
```

Without retries:

If the second test fails:

- First test passes.
- Second test fails.
- Third test is skipped.

Serial execution

Failure
1. Passed

First test
2. Failed

Second test
3. Skipped

Third test

With retries:

If a test fails, Playwright retries the entire serial group together.

For example:

Worker 1 – First execution

- First test passes.
- Second test fails.
- Third test is skipped.

Worker 2 – Retry

- First test runs again and passes.
- Second test runs again and passes.
- Third test runs and passes.

Serial group passed on retry

Important: Serial mode is generally not recommended unless tests genuinely depend on one another. Independent tests are easier to maintain, parallelize and retry.

8. Reuse a single Page between tests

By default, Playwright Test creates a separate isolated Page object for every test.

This means:

- Test 1 gets its own page.
- Test 2 gets another page.
- Test 3 gets another page.

However, if you want to reuse one page across multiple tests, you can create it in `beforeAll()` and close it in `afterAll()`.

Example:

```
import { test, type Page } from '@playwright/test';

test.describe.configure({ mode: 'serial' });

let page: Page;

test.beforeAll(async ({ browser }) => {
```

```
  page = await browser.newPage();
});

test.afterAll(async () => {
  await page.close();
});

test('runs first', async () => {
  await page.goto('https://playwright.dev/');
});

test('runs second', async () => {
  await page.getByText('Get Started').click();
});
```

Code explanation

Code	Meaning
`let page: Page`	Declares a Page variable.
`beforeAll()`	Creates the page before the tests start.
`browser.newPage()`	Creates a new page.
`afterAll()`	Runs after the tests finish.
`page.close()`	Closes the page.
`mode: 'serial'`	Runs the tests in order as a serial group.

Why serial mode here?

Because both tests use the same page variable. The second test expects the first test's navigation to have happened.

Important interview point: Reusing a page removes test-level page isolation. It can make tests dependent on each other's state, so use it only when necessary.

9. Important interview questions and answers

Q1. What are retries in Playwright?

Answer: Retries are a feature that automatically re-executes failed tests. They help identify flaky tests caused by intermittent issues such as network delays or temporary application problems.

Q2. What is the default retry count in Playwright?

Answer: By default, retries are disabled. The retry count is zero.

Q3. How do you enable retries in Playwright?

Answer: We can configure retries in `playwright.config.ts` using `retries: 3`, or use the CLI command `npx playwright test --retries=3`.

Q4. What is a flaky test?

Answer: A flaky test is a test that does not produce consistent results. It may fail in one execution and pass in another execution without any relevant test-code change.

Q5. What is the difference between passed, flaky and failed?

Answer:

- Passed: Passed on the first attempt.
- Flaky: Failed initially but passed on retry.
- Failed: Failed on the initial attempt and all retries.

Q6. What happens when a test fails in a worker?

Answer: Playwright discards the worker process and its browser, then starts a new worker to continue test execution. The `beforeAll()` hook runs again in the new worker when applicable.

Q7. What is `testInfo.retry`?

Answer: It is a property that tells us the current retry attempt number. It is `0` for the initial run and greater than `0` for retry attempts.

Q8. Can we configure retries for a specific test group?

Answer: Yes. We can use:

```
test.describe.configure({ retries: 2 });
```

Q9. What is serial mode?

Answer: Serial mode executes tests in a group in order. If one test fails, subsequent tests are skipped. When retries are enabled, the serial group is retried together.

Q10. What is the difference between parallel and serial execution?

Parallel	Serial
Tests can run concurrently.	Tests run in sequence.
Tests should be independent.	Tests may depend on one another.
Better execution speed.	Can be slower.
Failures are handled independently.	A failure skips subsequent tests in the group.

Q11. Should we use retries to fix flaky tests?

Answer: No. Retries help detect flaky tests, but they do not solve the root cause. We should investigate synchronization, application stability, test data, network issues and locator reliability.

Q12. Why does Playwright recommend independent tests?

Answer: Independent tests can run in parallel and can be retried individually. They are easier to debug and maintain. One test failure does not affect the execution of unrelated tests.

Q13. Can we reuse one page across multiple tests?

Answer: Yes. We can create a page using `browser.newPage()` inside `beforeAll()` and close it in `afterAll()`. However, this reduces test isolation and usually requires serial execution.

Q14. What is the difference between page fixture and `browser.newPage()`?

Answer:

- page fixture provides an isolated page for each test.
- `browser.newPage()` manually creates a page, which can be stored and reused when needed.

Q15. A test passes locally but fails intermittently in CI. What would you do?

Answer:

1. Check the Playwright test report and failure screenshot.
2. Check traces and logs.
3. Investigate timing and synchronization issues.
4. Verify test data and environment stability.
5. Check network or API failures.
6. Use retries temporarily to identify flaky behavior.
7. Fix the root cause instead of depending permanently on retries.

10. Quick revision notes

Topic	Remember
Retries	Re-run failed tests

Default retries	0
CLI	`--retries=3`
Config	`retries: 3`
Passed	Passed first time
Flaky	Failed first, passed on retry
Failed	Failed all attempts
`testInfo.retry`	Current retry number
Group retries	`test.describe.configure()`
Serial mode	Tests execute in order
Serial failure	Subsequent tests skipped
Page fixture	Isolated page per test
`browser.newPage()`	Manually creates page

11. Practice quiz

Test your understanding

5/5 answered

1. What is the default retry count in Playwright?

1
2
0

3

Correct — Retries are disabled by default.

2. Which property tells us the current retry attempt?

testInfo.status
testInfo.retry
testInfo.timeout
testInfo.worker

Correct — testInfo.retry is 0 on the first run and increases for each retry.

3. A test fails initially but passes on retry. What is its status?

Passed
Failed
Skipped
Flaky

Correct — A test that fails initially and passes on retry is classified as flaky.

4. What happens to subsequent tests in a serial group when one test fails?

They always execute
They are skipped
They run in parallel
They are deleted

Incorrect — In serial mode, subsequent tests are skipped after a failure.

5. Which method creates a new page manually?

browser.newPage()
page.newBrowser()
test.newPage()
browser.createTest()

Correct — browser.newPage() creates a new Page object.

Score: 4/5

Try again

Interview tip: Remember these three concepts especially well: `retries`, `testInfo.retry`, and `serial mode`. They are common areas for scenario-based Playwright questions.

Send the next Playwright documentation section, and I'll explain it in the same format.

4. Merging reports from multiple shards

Suppose you execute your tests using 4 shards.

Each shard generates its own test report:

- Shard 1 → Report 1.
- Shard 2 → Report 2.
- Shard 3 → Report 3.
- Shard 4 → Report 4.

The problem is that you have four separate reports. It is difficult to check the complete test execution in one place.

Solution: Merge all shard reports into one combined report.

Playwright provides a special reporter called the Blob Reporter.

What is a Blob Reporter?

A Blob Reporter stores test execution results and attachments, including:

- Test results.
- Traces.
- Screenshots.
- Screenshot differences.

These reports can later be merged into one report.

Pasted markdown(6).md

Step 1: Configure Blob Reporter

Open `playwright.config.ts`.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',

  reporter: process.env.CI ? 'blob' : 'html',
});
```

Explanation:

Code	Meaning
`testDir: './tests'`	Location of test files.
`process.env.CI`	Checks whether tests are running in a CI environment.
`'blob'`	Generates Blob reports in CI.
`'html'`	Generates HTML reports locally.

This means:

- When running in CI, Playwright uses the Blob Reporter.
- When running locally, Playwright uses the HTML Reporter.

Step 2: Collect Blob reports

Each shard generates a Blob report in the `blob-report` directory.

You need to collect the Blob reports from all shards into one directory.

For example:

```
all-blob-reports/
    blob-report-shard-1.zip
```

```
blob-report-shard-2.zip
blob-report-shard-3.zip
blob-report-shard-4.zip
```

The actual Blob report filenames are generated by Playwright.

Step 3: Merge reports

Run this command:

```
npx playwright merge-reports --reporter html ./all-blob-reports
```

Explanation:

- `merge-reports` → Merges multiple Blob reports.
- `--reporter html` → Generates an HTML report.
- `./all-blob-reports` → Folder containing the Blob reports.

Report merging flow

Shard 1

Blob report

Shard 2

Blob report

Shard 3

Blob report

Shard 4

Blob report

Merge reports

One combined HTML report

playwright-report/

The merged HTML report is generated in the `playwright-report` directory by default.

Pasted markdown(6).md

Interview point: Blob Reporter is useful when tests are distributed across multiple shards and you want one combined report.

5. Sharding with GitHub Actions

Now let's understand how sharding works in a real CI/CD pipeline.

What is GitHub Actions?

GitHub Actions is a CI/CD platform that allows you to automatically build, test and deploy your application.

For example, whenever a developer pushes code to GitHub:

1. GitHub Actions starts a workflow.
2. It installs project dependencies.
3. It installs Playwright browsers.
4. It executes automated tests.
5. It generates test reports.

Why use sharding with GitHub Actions?

Suppose your project has 400 test cases.

Instead of running all tests in one GitHub Actions job, you can create four jobs.

Each job runs one shard.

The GitHub Actions `matrix` strategy helps create multiple jobs using different values.

Step 1: Configure the matrix

Example:

```
strategy:
  fail-fast: false
  matrix:
    shardIndex: [1, 2, 3, 4]
    shardTotal: [4]
```

Explanation:

Property	Meaning
`strategy`	Defines how jobs are executed.
`matrix`	Creates multiple jobs using combinations of values.
`shardIndex`	Identifies the current shard.
`shardTotal`	Total number of shards.
`fail-fast: false`	Allows other matrix jobs to continue if one job fails.

This creates four jobs:

- Job 1 → Shard 1/4.
- Job 2 → Shard 2/4.
- Job 3 → Shard 3/4.
- Job 4 → Shard 4/4.

Step 2: Execute tests

```
- name: Run Playwright tests
  run: npx playwright test --
shard=${{ matrix.shardIndex }}/${{ matrix.shardTotal }}
```

Here:

- `matrix.shardIndex` gives the current shard number.
- `matrix.shardTotal` gives the total number of shards.
- Each job executes its own portion of the test suite.

Step 3: Upload Blob reports

```
- name: Upload blob report
  if: ${{ !cancelled() }}
  uses: actions/upload-artifact@v4
  with:
    name: blob-report-${{ matrix.shardIndex }}
    path: blob-report
    retention-days: 1
```

Explanation:

- `upload-artifact` uploads the Blob report.
- `name` gives each artifact a unique name.
- `path` specifies the report folder.
- `retention-days` specifies how long the artifact is retained.
- `if: ${{ !cancelled() }}` allows the upload step to run unless the job has been cancelled.

Step 4: Merge reports after all shards finish

A separate job is created to merge all reports.

```
merge-reports:
  if: ${{ !cancelled() }}
  needs: [playwright-tests]
  runs-on: ubuntu-latest
```

Important:

`needs: [playwright-tests]` means the merge job depends on the sharded test job.

It waits for the sharded jobs to finish before starting.

Then download the Blob reports:

```
- name: Download blob reports
  uses: actions/download-artifact@v5
  with:
    path: all-blob-reports
```

```
    pattern: blob-report-*
    merge-multiple: true
```

Finally, merge them:

```
- name: Merge into HTML Report
  run: npx playwright merge-reports --reporter html ./all-blob-
reports
```

Upload the final report:

```
- name: Upload HTML report
  uses: actions/upload-artifact@v4
  with:
    name: html-report
    path: playwright-report
    retention-days: 14
```

Complete workflow in simple words:

1. Developer pushes code to GitHub.
2. GitHub Actions starts the workflow.
3. Four parallel jobs execute four shards.
4. Each job uploads its Blob report.
5. Merge job downloads all Blob reports.
6. Playwright combines the reports into one HTML report.
7. Final HTML report is uploaded as an artifact.

This is the workflow described in the documentation.

Pasted markdown(6).md
Pasted markdown(6).md

6. Merging reports from multiple environments

This is different from sharding.

Sharding: Splits tests across multiple machines or jobs.

Multiple environments: Runs the same tests in different environments.

For example, you may execute the same tests against:

- Development environment.

- QA environment.
- Staging environment.

You need to identify which test results belong to which environment.

Playwright provides the `tag` configuration property for this.

Example:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  reporter: process.env.CI ? 'blob' : 'html',

  tag: process.env.CI_ENVIRONMENT_NAME,
});
```

Suppose:

```
CI_ENVIRONMENT_NAME = @APIv2
```

Then the tests are tagged with `@APIv2`.

The Blob Reporter and merge tool can use these tags to identify the environment associated with the results.

Pasted markdown(6).md

Interview example:

Your company runs the same automation suite in QA and staging.

You can tag the runs with different environment names so that the merged report can distinguish their results.

7. Merge-reports CLI options

Playwright provides additional options with `merge-reports`.

Option 1: `--reporter`

Specifies which report format you want to generate.

Example:

```
npx playwright merge-reports --reporter=html,github ./blob-reports
```

This generates HTML and GitHub reports.

Option 2: `--config`

Specifies a configuration file for the merge operation.

Example:

```
npx playwright merge-reports --config=merge.config.ts ./blob-reports
```

Example `merge.config.ts`:

```
export default {
  testDir: 'e2e',
  reporter: [['html', { open: 'never' }]],
};
```

Here:

- `testDir` specifies the test root directory.
- `reporter` specifies the output reporter.
- `open: 'never'` prevents automatically opening the HTML report.

When merging reports generated on different operating systems, the documentation also notes that an explicit merge configuration may be needed to identify the test root directory correctly.

Pasted markdown(6).md

8. Important interview questions and answers

Q1. What is sharding in Playwright?

Answer: Sharding is a technique of splitting a large test suite into smaller groups called shards and running them simultaneously on multiple machines or CI jobs to reduce execution time.

Q2. Why do we use sharding?

Answer: We use sharding to distribute tests across multiple machines or CI jobs. It helps reduce the overall execution time of large test suites.

Q3. How do you execute sharding in Playwright?

Answer: We use the `--shard` command-line option.

```
npx playwright test --shard=1/4
```

This executes the first shard out of four.

Q4. What is the difference between parallel execution and sharding?

Answer:

- Parallel execution runs tests simultaneously using multiple workers.
- Sharding divides the test suite into smaller groups and distributes them across multiple jobs or machines.

Sharding can use parallel execution within each machine as well.

Q5. What is `fullyParallel: true`?

Answer: It allows individual tests to run in parallel and enables test-level distribution across shards, helping balance the workload.

Q6. What happens if `fullyParallel` is not enabled?

Answer: By default, Playwright distributes complete test files across shards. If files have different numbers of tests or execution times, some shards may take longer than others.

Q7. What is Blob Reporter?

Answer: Blob Reporter generates reports containing test execution results and attachments such as traces and screenshots. These reports can be merged into a single report.

Q8. How do you merge reports from multiple shards?

Answer: First, collect the Blob reports into one directory. Then execute:

```
npx playwright merge-reports --reporter html ./all-blob-reports
```

This generates a combined HTML report.

Q9. What is the purpose of the GitHub Actions matrix?

Answer: The matrix strategy creates multiple jobs using different combinations of values. We use it to create separate jobs for different shards.

Q10. What is the purpose of needs in GitHub Actions?

Answer: `needs` defines job dependencies. In our example, the merge job waits for the sharded test jobs to finish.

Q11. What is the difference between sharding and multiple environments?

Answer:

- Sharding divides the test suite across multiple machines.
- Multiple environments execute the same tests against different environments, such as QA and staging.

Q12. What is the purpose of `testConfig.tag`?

Answer: It is used to tag tests with an environment name. The Blob Reporter and merge tool can use these tags to identify the environment associated with the test results.

Q13. Does sharding guarantee that tests execute four times faster?

Answer: No. It can significantly reduce execution time, but the actual speed depends on test distribution, machine resources, setup overhead and execution time.

Q14. What is the difference between HTML Reporter and Blob Reporter?

Answer:

HTML Reporter	Blob Reporter
Generates an HTML test report.	Generates a Blob report.
Useful for viewing results.	Useful for collecting and merging shard results.
Commonly used for local execution.	Commonly used in CI sharding workflows.

9. Quick revision notes

Concept	Important point
Sharding	Splitting tests into smaller groups
Purpose	Reduce execution time
CLI command	`--shard=1/4`
`fullyParallel: true`	Test-level distribution
Without fullyParallel	File-level distribution
Blob Reporter	Stores results and attachments
Merge command	`npx playwright merge-reports`
GitHub Actions matrix	Creates multiple jobs
`needs`	Defines job dependency

`tag`	Identifies environment
`--reporter`	Selects report format
`--config`	Specifies merge configuration

10. Practice quiz

Sharding knowledge test

6/6 answered

1. What is the main purpose of sharding?

Reduce test execution time
Create test cases
Fix locators
Generate screenshots

Correct — Sharding distributes tests across multiple jobs or machines.

2. Which command runs the first shard out of four?

--shard=4/1
--shard=1/4
--parallel=1/4
--workers=4

Correct — The syntax is --shard=currentShard/totalShards.

3. What does fullyParallel: true enable for sharding?

File-level distribution only
Test-level distribution
Disable parallel execution
Disable retries

Incorrect — Individual tests can be distributed across shards.

4. Which reporter is useful for merging reports from shards?

List

Dot
Blob
Line

Correct — Blob reports can be merged into a combined report.

5. Which command merges Blob reports into an HTML report?

npx playwright test --html
npx playwright merge-reports --reporter html ./all-blob-reports
npx playwright show-report
npm merge html

Correct — merge-reports reads Blob reports and produces the selected reporter output.

6. What does needs: [playwright-tests] do in GitHub Actions?

Starts tests again
Installs browsers
Makes merge job depend on test jobs
Creates four shards

Correct — The merge job waits for the sharded test job to finish.

Score: 5/6

Retry quiz

Most important for interviews: Focus on the difference between sharding and parallel execution, `fullyParallel`, Blob Reporter, and merging reports. These concepts are especially useful when explaining how you would speed up a large Playwright automation suite in a CI/CD pipeline.

Playwright – Timeouts

Timeouts are one of the most important topics in Playwright interviews.

Let's understand each timeout from basics, with simple examples, practical QA scenarios, code explanations and interview questions.

1. What are timeouts in Playwright?

Timeout means the maximum amount of time Playwright waits for an operation to complete.

For example:

Suppose you click the Login button. After clicking, your application takes time to process the request.

- If the operation completes within the allowed time, the test continues.
- If it takes longer than the allowed time, Playwright throws a timeout error.

Real-world example

Suppose you are automating an e-commerce application.

You perform these steps:

1. Open the application.
2. Enter username and password.
3. Click Login.
4. Wait for the home page.
5. Verify the welcome message.

If the application takes too long to load, your test may fail because of a timeout.

Remember: Timeout does not necessarily mean your application is broken. It means the operation did not complete within the configured time.

2. Types of timeouts in Playwright

Playwright provides different timeouts for different purposes.

Timeout	Default	Purpose

Test timeout	30 seconds	Maximum time for a test
Expect timeout	5 seconds	Maximum time for an auto-retrying assertion
Global timeout	No default	Maximum time for the entire test run
Action timeout	No timeout	Maximum time for an action
Navigation timeout	No timeout	Maximum time for navigation
beforeAll/afterAll timeout	30 seconds	Maximum time for each hook
Fixture timeout	No timeout	Maximum time for an individual fixture when separately configured

Let's understand them one by one.

3. Test timeout

Test timeout defines the maximum time allowed for an individual test.

Default value: 30,000 milliseconds (30 seconds).

This timeout includes:

- Test function execution.
- Fixture setup.
- `beforeEach()` hook execution.

For example:

```
import { test, expect } from '@playwright/test';

test('login test', async ({ page }) => {

  await page.goto('https://example.com');

  await page.fill('#username', 'admin');

  await page.fill('#password', 'admin123');

  await page.click('#login');

});
```

If the test takes more than 30 seconds, Playwright throws a timeout error.

Example error:

```
Timeout of 30000ms exceeded.
```

Method 1: Set test timeout globally in config

`playwright.config.ts`

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  timeout: 120_000,
});
```

Here:

- `120_000` means 120,000 milliseconds.
- 120,000 milliseconds = 120 seconds = 2 minutes.
- This timeout applies to each test by default.

Method 2: Set timeout for a single test

```
import { test, expect } from '@playwright/test';

test('very slow test', async ({ page }) => {
```

```
  test.setTimeout(120_000);

  await page.goto('https://example.com');

});
```

Here, only this test gets a timeout of 120 seconds.

Method 3: Use test.slow()

```
test('slow test', async ({ page }) => {

  test.slow();

  await page.goto('https://example.com');

});
```

`test.slow()` triples the default test timeout.

For example:

- Default timeout = 30 seconds.
- After `test.slow()` = 90 seconds.

Interview question: What is the difference between `test.setTimeout()` and `test.slow()`?

Answer:

- `test.setTimeout()` allows us to specify an exact timeout.
- `test.slow()` triples the default timeout.

4. Change timeout using beforeEach hook

Sometimes all tests need extra time because they share a slow setup operation.

For this, we can use `testInfo.setTimeout()`.

Example:

```
import { test, expect } from '@playwright/test';

test.beforeEach(async ({ page }, testInfo) => {

  testInfo.setTimeout(testInfo.timeout + 30_000);

});
```

Explanation:

- `testInfo` provides information about the currently running test.
- `testInfo.timeout` gives the current test timeout.
- `+ 30_000` adds another 30 seconds.
- `testInfo.setTimeout()` updates the timeout.

For example:

Original timeout = 30 seconds.

Additional time = 30 seconds.

New timeout = 60 seconds.

This applies to tests running the relevant `beforeEach()` hook.

5. beforeAll and afterAll timeout

Remember these two hooks:

- `beforeAll()` executes before all tests in its scope.
- `afterAll()` executes after all tests in its scope.

These hooks have their own timeout.

By default, each hook has a timeout equal to the test timeout.

You can change it separately.

Example:

```
import { test, expect } from '@playwright/test';

test.beforeAll(async () => {

  test.setTimeout(60_000);

  // Setup operation

});

test.afterAll(async () => {

  test.setTimeout(60_000);

  // Cleanup operation

});
```

Here, each hook gets a 60-second timeout.

Important: The time spent in `beforeAll()` and `afterAll()` does not count against an individual test's timeout.

Also, fixture teardown and `afterEach()` hooks have a separate timeout budget after the test function finishes, with the same timeout value by default.

6. Expect timeout

This is another very important timeout.

Expect timeout defines how long Playwright waits for an assertion to pass.

Default value: 5 seconds.

Example:

```
import { test, expect } from '@playwright/test';

test('login test', async ({ page }) => {

  await page.goto('https://example.com');

  await expect(page.locator('#welcome'))
```

```
    .toHaveText('Welcome Admin');

});
```

Suppose the welcome message takes 3 seconds to appear.

Playwright keeps checking the assertion until it passes or the 5-second timeout is reached.

If the expected text does not appear within 5 seconds, the assertion fails.

Method 1: Configure expect timeout globally

playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  expect: {
    timeout: 10_000,
  },

});
```

Now all applicable auto-retrying assertions have a 10-second timeout by default.

Method 2: Set timeout for a single assertion

```
await expect(page.locator('#welcome'))
  .toHaveText('Welcome Admin', { timeout: 10_000 });
```

Here, only this assertion gets 10 seconds.

Difference between test timeout and expect timeout

Test timeout	Expect timeout
Default 30 seconds	Default 5 seconds

Controls the overall test execution budget	Controls an individual auto-retrying assertion
Configured using `timeout`	Configured using `expect.timeout`
Can use `test.setTimeout()`	Can pass `{ timeout: ... }` to the assertion

Important interview point: Expect timeout is separate from test timeout, but the assertion still runs within the overall test execution budget.

7. Global timeout

Global timeout defines the maximum time allowed for the entire test execution run.

Default: No timeout.

Suppose your project has 1,000 test cases.

You want the complete test suite to finish within one hour.

You can configure:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  globalTimeout: 3_600_000,

});
```

Explanation:

- `3_600_000` milliseconds.
- 3,600 seconds.
- 60 minutes.
- 1 hour.

If the entire test execution exceeds one hour, Playwright stops the run.

Difference:

- Test timeout → Maximum time for one test.
- Global timeout → Maximum time for the complete test run.

8. Action timeout

Action timeout defines the maximum time allowed for an individual browser action.

Examples:

- Click.
- Fill.
- Check.
- Hover.

Default: No timeout configured at the action-timeout level.

Method 1: Configure action timeout globally

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  use: {
    actionTimeout: 10_000,
  },

});
```

Now actions have a default timeout of 10 seconds.

Method 2: Set timeout for one action

```
await page.getByText('Login').click({
  timeout: 10_000,
});
```

Here, Playwright waits up to 10 seconds for the click action to complete.

Real-world example:

Suppose the Add to Cart button is temporarily disabled while the page loads.

You can allow extra time for the click action.

```
await page.getByRole('button', {
  name: 'Add to Cart'
}).click({ timeout: 15_000 });
```

Remember that Playwright also performs actionability checks before interacting with elements.

9. Navigation timeout

Navigation timeout defines the maximum time allowed for navigation operations.

For example:

- `page.goto()`
- `page.reload()`
- `page.goBack()`

Default: No timeout configured at the navigation-timeout level.

Method 1: Configure navigation timeout globally

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  use: {
    navigationTimeout: 30_000,
  },

});
```

Method 2: Set timeout for one navigation

```
await page.goto('https://playwright.dev', {
  timeout: 30_000,
});
```

Here, Playwright waits up to 30 seconds for the navigation operation.

Interview point: Action timeout is for browser actions, whereas navigation timeout is for navigation operations.

10. Fixture timeout

First, understand what a fixture is.

A fixture is a reusable setup mechanism in Playwright that provides required resources to tests.

Examples:

- Browser.
- Page.
- Authentication data.
- Database connection.
- Test data.

Sometimes fixture setup takes longer than the test itself.

In such cases, we can provide a separate fixture timeout.

Example:

```
import { test as base, expect } from '@playwright/test';

const test = base.extend<{ slowFixture: string }>({

  slowFixture: [
    async ({}, use) => {

      // Perform a slow operation

      await use('hello');

    },
    {
      timeout: 60_000,
```

```
    }
  ]

});

test('example test', async ({ slowFixture }) => {

  console.log(slowFixture);

});
```

Explanation

Code	Meaning
`base.extend()`	Creates a custom fixture.
`slowFixture`	Name of the custom fixture.
`use('hello')`	Provides the fixture value to the test.
`timeout: 60_000`	Gives the fixture a separate 60-second timeout.

By default, fixtures share the test timeout. A separate fixture timeout is useful for slow fixtures.

11. Complete timeout comparison

This table is especially important for interviews.

Timeout	Default	Configuration
Test timeout	30 sec	`timeout: 30_000`
Expect timeout	5 sec	`expect: { timeout: 5_000 }`

Global timeout	No default	`globalTimeout: 3_600_000`
Action timeout	No timeout	`use: { actionTimeout: 10_000 }`
Navigation timeout	No timeout	`use: { navigationTimeout: 30_000 }`
beforeAll/afterAll	30 sec	`test.setTimeout(60_000)` inside hook
Fixture timeout	No separate default	`{ timeout: 60_000 }` in fixture options

12. Complete practical configuration example

You can configure multiple timeout types together in `playwright.config.ts`.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  // Timeout for each test
  timeout: 60_000,

  // Timeout for assertions
  expect: {
    timeout: 10_000,
  },

  // Timeout for entire test run
  globalTimeout: 3_600_000,

  use: {

    // Timeout for browser actions
    actionTimeout: 10_000,
```

```
    // Timeout for navigation
    navigationTimeout: 30_000,

  },

});
```

This configuration means:

- Each test gets 60 seconds.
- Each auto-retrying assertion gets 10 seconds.
- The entire test run gets a maximum of one hour.
- Each browser action gets 10 seconds.
- Each navigation operation gets 30 seconds.

13. Important interview questions and answers

Q1. What are timeouts in Playwright?

Answer: Timeouts define the maximum time Playwright waits for a test, assertion, action or navigation to complete.

Q2. What is the default test timeout?

Answer: 30 seconds or 30,000 milliseconds.

Q3. What is the default expect timeout?

Answer: 5 seconds or 5,000 milliseconds.

Q4. What is the difference between test timeout and expect timeout?

Answer: Test timeout controls the overall test execution budget, whereas expect timeout controls how long an auto-retrying assertion waits for the expected condition.

Q5. How can you increase the timeout for one test?

Answer:

```
test('slow test', async ({ page }) => {
  test.setTimeout(120_000);
});
```

Q6. What is `test.slow()`?

Answer: It marks a test as slow and triples its default timeout.

Q7. How can you set a global timeout?

Answer:

```
globalTimeout: 3_600_000
```

This sets a one-hour limit for the complete test run.

Q8. What is action timeout?

Answer: Action timeout specifies how long Playwright waits for a browser action, such as clicking or filling an element.

Q9. What is navigation timeout?

Answer: Navigation timeout specifies how long Playwright waits for a navigation operation such as `page.goto()`.

Q10. What is fixture timeout?

Answer: Fixture timeout defines the maximum time allowed for an individual fixture. It is useful when fixture setup requires more time than the normal test timeout budget.

Q11. How can you set a timeout for one assertion?

Answer:

```
await expect(page.locator('#message'))
  .toHaveText('Success', { timeout: 10_000 });
```

Q12. How do you increase timeout using beforeEach?

Answer:

```
test.beforeEach(async ({ page }, testInfo) => {
  testInfo.setTimeout(testInfo.timeout + 30_000);
});
```

Q13. What happens if a test exceeds its timeout?

Answer: Playwright marks the test as timed out and reports a timeout error.

Q14. Does expect timeout replace test timeout?

Answer: No. They are separate timeout settings. The assertion has its own timeout, but it still executes within the test's overall timeout budget.

Q15. Your test fails with a timeout error. What will you do?

Answer:

1. Check the exact timeout error.
2. Check whether the locator is correct.
3. Verify whether the element is visible and enabled.
4. Check application loading and network response.
5. Check whether the assertion is expecting the correct value.
6. Review screenshots, traces and logs.
7. Increase timeout only if the operation genuinely needs more time.

14. Quick revision notes

Remember	Value
Default test timeout	30 seconds
Default expect timeout	5 seconds
`test.slow()`	Triple timeout

`test.setTimeout()`	Set test timeout
`testInfo.setTimeout()`	Change timeout through test information
`globalTimeout`	Whole test run
`actionTimeout`	Click, fill, hover, etc.
`navigationTimeout`	`goto`, reload, etc.
Fixture timeout	Individual fixture

15. Practice quiz

Test your knowledge

7/7 answered

1. What is the default test timeout in Playwright?

5 seconds
10 seconds
30 seconds
60 seconds

Correct — The default test timeout is 30,000 milliseconds.

2. What is the default expect timeout?

5 seconds
30 seconds
60 seconds
No timeout

Correct — Auto-retrying assertions have a default timeout of 5 seconds.

3. Which method triples the default test timeout?

test.timeout()

test.slow()
test.retry()
test.wait()

Correct — test.slow() triples the default test timeout.

4. Which timeout controls the entire test execution run?

Action timeout
Expect timeout
Global timeout
Navigation timeout

Correct — globalTimeout controls the overall test run.

5. Which timeout is used for page.goto() by default at the navigation-timeout level?

5 seconds
30 seconds
No timeout
60 seconds

Incorrect — Navigation timeout has no default timeout configured at this level.

6. Which setting controls the timeout for a click action?

navigationTimeout
actionTimeout
globalTimeout
expect.timeout

Correct — actionTimeout controls browser actions.

7. Which method changes timeout using testInfo?

testInfo.setTimeout()
testInfo.retry()
testInfo.change()
testInfo.wait()

Correct — testInfo.setTimeout() updates the timeout.

Score: 6/7

Try again

My interview preparation tip: Give special attention to the difference between Test Timeout, Expect Timeout, Action Timeout and Navigation Timeout. Interviewers often give a scenario and ask which timeout you would configure.

For your Playwright TypeScript preparation, the next useful exercise would be a small login and checkout test where you configure all four timeout types.

Playwright TypeScript – Complete Explanation

This topic is important for QA Automation and SDET interviews because it explains how Playwright handles TypeScript, how to configure TypeScript, and how to manage test compilation.

Let's go section by section with simple explanations and practical examples.

1. Introduction – TypeScript support in Playwright

Playwright supports TypeScript out of the box.

This means you can directly create and execute TypeScript test files (`.ts`) without manually converting them into JavaScript.

Execution flow:

TypeScript test file

login.spec.ts

Playwright transforms TypeScript into JavaScript

Playwright executes the test

Example:

```
import { test, expect } from '@playwright/test';
```

```
test('Login test', async ({ page }) => {
  await page.goto('https://example.com/login');

  await page.getByLabel('Username').fill('komal');
  await page.getByLabel('Password').fill('password123');
  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page).toHaveURL(/dashboard/);
});
```

Run the test:

```
npx playwright test
```

Important: Does Playwright check TypeScript types?

No. Playwright transforms TypeScript into JavaScript, but it does not perform full TypeScript type checking when running tests.

Example:

```
let age: number = 25;

age = "twenty-five";
```

Here, TypeScript identifies a type error because age is declared as a number, but a string is assigned to it.

Playwright may still run the test because it does not automatically type-check your code.

How to check TypeScript errors?

Use the TypeScript compiler (`tsc`).

```
npx tsc -p tsconfig.json --noEmit
```

Command	Meaning
`npx`	Executes a package command
`tsc`	TypeScript compiler

`-p`	Specifies the project/configuration
`tsconfig.json`	TypeScript configuration file
`--noEmit`	Checks types without generating JavaScript files

Watch mode:

```
npx tsc -p tsconfig.json --noEmit -w
```

`-w` means watch mode. It continuously checks your files and reports errors when you make changes.

Type checking in GitHub Actions

```
- name: Run type checks
  run: npx tsc -p tsconfig.json --noEmit

- name: Run Playwright tests
  run: npx playwright test
```

Why are these two steps separate?

- First step checks TypeScript errors.
- Second step executes Playwright tests.
- Both help maintain code quality.

Interview point: Playwright supports TypeScript out of the box, but type checking should be performed separately using `tsc`.

2. tsconfig.json

`tsconfig.json` is the configuration file for a TypeScript project.

It tells TypeScript how to handle TypeScript files and which compiler options to use.

Example project structure:

```
project/
|
```

```
├── src/
│   └── source.ts
│
├── tests/
│   ├── tsconfig.json
│   └── example.spec.ts
│
├── tsconfig.json
│
└── playwright.config.ts
```

Here, there are two `tsconfig.json` files:

File	Purpose
Root `tsconfig.json`	General TypeScript configuration
`tests/tsconfig.json`	Configuration specifically for test files

Why create a separate tsconfig inside tests?

Suppose your application source code and Playwright tests need different TypeScript settings.

You can configure them separately without changing the general project configuration.

According to the documentation, Playwright supports these `tsconfig` options:

- `allowJs`
- `baseUrl`
- `paths`
- `jsx`
- `jsxFactory`
- `jsxFragmentFactory`
- `jsxImportSource`
- `references`

- `extends`

Remember: Playwright supports only the listed options for its own TypeScript loading and transformation.

3. tsconfig path mapping

Path mapping allows you to import files using shorter, custom paths instead of long relative paths.

Imagine your project has this structure:

```
project/
│
├── packages/
│   └── myhelper/
│       └── credentials.ts
│
└── tests/
    └── example.spec.ts
```

Without path mapping, you might need a relative import such as:

```
import { username, password }
from '../packages/myhelper/credentials';
```

With path mapping, you can use:

tsconfig.json

```
{
  "compilerOptions": {
    "paths": {
      "@myhelper/*": ["packages/myhelper/*"]
    }
  }
}
```

Now you can import like this:

```
import { test, expect } from '@playwright/test';

import {
  username,
```

```
  password
} from '@myhelper/credentials';

test('example', async ({ page }) => {
  await page.getByLabel('User Name').fill(username);
  await page.getByLabel('Password').fill(password);
});
```

Explanation

- `@myhelper/*` is a custom import alias.
- `packages/myhelper/*` represents the actual folder path.
- `*` is a wildcard that matches different file or folder names.
- The mapping path is relative to the `tsconfig.json` file.

Practical QA use case: You can keep reusable test data, credentials, utilities and helper functions in separate folders and import them using shorter paths.

4. tsconfig JSX options

First, understand JSX.

JSX is a syntax extension commonly used with React and similar libraries to describe UI elements.

Playwright supports JSX compilation in:

- `.tsx` files
- `.jsx` files

It uses these configuration options:

Option	Purpose
`jsx`	Defines how JSX is transformed
`jsxFactory`	Defines the function used to create JSX elements

`jsxFragmentFactory`	Defines the function used for JSX fragments
`jsxImportSource`	Specifies the package providing the automatic JSX runtime

Example:

```
{
  "compilerOptions": {
    "jsx": "react-jsx",
    "jsxImportSource": "preact"
  }
}
```

Explanation:

- `jsx: "react-jsx"` enables the automatic JSX runtime format.
- `jsxImportSource: "preact"` specifies Preact as the JSX runtime source.

One important detail from the documentation:

When `jsx` is not specified, or is set to `preserve` or `react-native`, Playwright uses the automatic runtime from `react/jsx-runtime`.

Interview point: JSX options control how Playwright transforms JSX inside `.tsx` and `.jsx` files.

5. tsconfig resolution

This section explains how Playwright finds the appropriate TypeScript configuration file.

Default behaviour

Playwright automatically searches for the closest `tsconfig.json` or `jsconfig.json` for each imported file.

It searches by moving up the directory structure.

Example:

```
project/
|
```

```
├── tsconfig.json
│
├── src/
│   └── source.ts
│
└── tests/
    ├── tsconfig.json
    └── login.spec.ts
```

If Playwright loads `login.spec.ts`, it can use `tests/tsconfig.json` because it is the closest configuration file.

Run normally:

```
npx playwright test
```

Playwright automatically resolves the configuration.

Option 1: Specify tsconfig through CLI

```
npx playwright test --tsconfig=tsconfig.test.json
```

This tells Playwright to use one specified TypeScript configuration for all imported files, rather than resolving the closest config separately.

Option 2: Specify tsconfig in playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  tsconfig: './tsconfig.test.json',
});
```

Now Playwright uses the specified configuration when loading test files, reporters, and other relevant files.

Important difference: This configuration does not apply while loading `playwright.config.ts` itself or files imported from it.

Comparison

Method	How it works

Automatic resolution	Finds closest tsconfig for each imported file
CLI `--tsconfig`	Uses one specified tsconfig for all imported files
Config `tsconfig`	Sets the tsconfig in Playwright configuration for loading test-related files

6. Manually compile tests with TypeScript

Normally, Playwright transforms TypeScript automatically.

But sometimes your TypeScript code uses experimental or very recent TypeScript features that Playwright cannot transform correctly.

In that situation, you can compile the tests yourself before running them.

Step 1: Create tests/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "commonjs",
    "moduleResolution": "Node",
    "sourceMap": true,
    "outDir": "../tests-out"
  }
}
```

Explanation:

Option	Meaning
`target`	JavaScript language version to generate
ESNext	Uses the latest ECMAScript target

`module`	Specifies the module system
`commonjs`	Uses CommonJS modules
`moduleResolution`	Specifies how modules are resolved
`sourceMap`	Generates source map files
`outDir`	Specifies the output folder

After compilation, generated JavaScript files are placed in `tests-out`.

Step 2: Configure package.json

```
{
  "scripts": {
    "pretest": "tsc --incremental -p tests/tsconfig.json",
    "test": "playwright test -c tests-out"
  }
}
```

Let's understand both scripts.

First script:

```
tsc --incremental -p tests/tsconfig.json
```

- `tsc`: Runs the TypeScript compiler.
- `--incremental`: Enables incremental compilation.
- `-p`: Specifies the TypeScript project configuration.

It compiles TypeScript test files into JavaScript.

Second script:

```
playwright test -c tests-out
```

- `playwright test`: Runs Playwright tests.

- `-c`: Specifies the Playwright configuration directory or file.
- `tests-out`: Tells Playwright to use the compiled test directory as its configuration/test location in this example.

Step 3: Execute

`npm run test`

The `pretest` script runs automatically before the `test` script when using `npm run test`.

npm run test

pretest

TypeScript compilation

JavaScript generated

tests-out directory

Playwright executes compiled tests

When is manual compilation useful?

When Playwright's built-in TypeScript transformation cannot handle the TypeScript features used by your project.

7. Quick revision – Important commands

Command	Purpose
`npx playwright test`	Execute Playwright tests
`npx tsc -p tsconfig.json --noEmit`	Type-check without generating JS
`npx tsc -p tsconfig.json --noEmit -w`	Continuously type-check

`npx playwright test --tsconfig=tsconfig.test.json`	Use a specific tsconfig
`tsc --incremental -p tests/tsconfig.json`	Compile tests incrementally
`npm run test`	Run pretest compilation followed by tests

8. SDET interview questions and answers

Q1. Does Playwright support TypeScript?

Yes. Playwright supports TypeScript out of the box. It transforms TypeScript into JavaScript and executes the tests.

Q2. Does Playwright perform TypeScript type checking automatically?

No. Playwright transforms TypeScript but does not perform full type checking. We use the TypeScript compiler (`tsc`) separately.

Q3. What is tsconfig.json?

It is the TypeScript configuration file. It defines compiler options and project-specific settings.

Q4. What is the purpose of `--noEmit`?

It checks TypeScript errors without generating JavaScript output files.

Q5. What is TypeScript watch mode?

Watch mode continuously monitors TypeScript files and checks them whenever changes are made.

Q6. What is path mapping?

Path mapping allows us to create custom import aliases in `tsconfig.json`, avoiding long relative import paths.

Q7. How does Playwright resolve tsconfig.json?

By default, Playwright searches for the closest `tsconfig.json` or `jsconfig.json` for each imported file.

Q8. How can you specify a custom tsconfig file?

Using the CLI:

```
npx playwright test --tsconfig=tsconfig.test.json
```

Or in Playwright configuration:

```
export default defineConfig({
  tsconfig: './tsconfig.test.json',
});
```

Q9. When do you manually compile TypeScript tests?

When Playwright cannot correctly transform certain experimental or very recent TypeScript features.

Q10. What is the purpose of `outDir`?

It specifies the directory where compiled JavaScript files are generated.

Q11. What is the difference between TypeScript compilation and type checking?

Compilation transforms TypeScript into JavaScript. Type checking verifies whether the code follows the declared types. In this documentation's normal Playwright workflow, these responsibilities are handled separately.

9. Practice MCQ quiz

Test your understanding before moving to the next topic.

TypeScript knowledge check

5/5 answered

1. Does Playwright automatically perform full TypeScript type checking?

Yes
No
Only in headed mode
Only in Chromium

Incorrect – No

Playwright transforms TypeScript but recommends running tsc separately for type checking.

2. Which command checks TypeScript without generating JavaScript?

npx playwright test
npx tsc --noEmit
npm run test
npx tsc --watch

Incorrect – npx tsc --noEmit

--noEmit performs type checking without producing output files.

3. What is the purpose of paths in tsconfig.json?

Set browser timeout
Create import aliases
Generate reports
Configure retries

Correct

Path mapping lets you use shorter custom import paths.

4. Which CLI option specifies a TypeScript configuration file?

--config-ts
--tsconfig
--typescript
--project-config

Correct

Use --tsconfig followed by the configuration file path.

5. What does outDir specify?

Test timeout
Source file extension
Output directory for compiled files
Browser name

Incorrect – Output directory for compiled files

outDir tells the compiler where to put generated files.

Your score: 2/5

Try again

Most important interview takeaway: Understand the difference between Playwright's TypeScript transformation and TypeScript type checking. Also remember `tsconfig.json`, path mapping, automatic config resolution, and manual compilation.

Playwright – UI Mode (Complete Explanation)

UI Mode is an interactive interface provided by Playwright that helps you explore, run, monitor and debug automated tests visually.

Instead of executing tests only through the terminal, you can use UI Mode to see your test files, individual test cases, execution steps, errors, browser snapshots, console logs and network requests.

It is especially useful for QA Automation Engineers and SDETs when debugging failed test cases.

1. What is UI Mode?

UI Mode provides:

- Visual execution of Playwright tests.
- Time-travel debugging.
- Watch mode to rerun tests when code changes.
- Filtering by test name, project, tag and execution status.
- DOM snapshots for individual actions.
- Console and network logs.
- Locator selection and inspection.

Example – E-commerce application

Suppose you have automated these test cases:

1. Login
2. Search product
3. Add product to cart
4. Checkout
5. Place order

Using UI Mode, you can execute an individual test, inspect every action and identify exactly where a test failed.

UI Mode | Playwright
playwright.dev

2. Opening UI Mode

To open UI Mode, run:

```
npx playwright test --ui
```

Explanation:

Command	Meaning
`npx`	Executes a package command
`playwright test`	Runs the Playwright test runner
`--ui`	Opens interactive UI Mode

After executing the command, the Playwright UI opens with your test files displayed in the sidebar.

UI Mode | Playwright
playwright.dev

3. Running your tests

Once UI Mode opens, you can see all your test files.

You can run tests at different levels:

Level	What it does
All tests	Runs the complete test suite
Test file	Runs all tests inside a particular file
Describe block	Runs tests inside a particular test group
Single test	Runs only one selected test case

Example:

Suppose your test file contains:

```
import { test, expect } from '@playwright/test';

test.describe('Login functionality', () => {

  test('Valid login', async ({ page }) => {
    await page.goto('/login');
  });

  test('Invalid login', async ({ page }) => {
    await page.goto('/login');
  });

});
```

In UI Mode:

- Click the triangle beside the file to run the complete file.
- Click the triangle beside `Login functionality` to run that describe block.

- Click the triangle beside `Valid login` to run only that test.

This is useful when you want to debug one failed test without running the entire regression suite.

4. Filtering tests

UI Mode allows you to filter tests using different criteria.

You can filter by:

1. Test name: Search for a particular test.
2. Projects: Filter by browser or project configured in `playwright.config.ts`.
3. @tag: Filter tests using tags.
4. Execution status: Passed, failed or skipped.

Example:

```
import { test, expect } from '@playwright/test';

test('Login functionality @smoke', async ({ page }) => {
  await page.goto('/login');
});

test('Checkout functionality @regression', async ({ page }) => {
  await page.goto('/checkout');
});
```

You can filter using:

- `@smoke`
- `@regression`
- Login
- Passed
- Failed
- Skipped

Important: Project dependencies

If your tests use project dependencies, such as a setup project that performs authentication, UI Mode does not automatically account for the setup tests when you run dependent tests.

You must manually run the setup tests first.

For example:

```
Setup project → Login and save authentication state
                          ↓
Main project → Run authenticated tests
```

5. Timeline view

UI Mode | Playwright
playwright.dev

The Timeline view shows the sequence of actions performed during a test.

You can:

- Hover over different actions to see snapshots.
- Navigate backward and forward through the test.
- Double-click an action to inspect its time range.
- Use the timeline slider to select multiple actions.

When you select a range of actions, the Actions tab and console/network logs are filtered to that selected range.

Practical example:

Suppose your test performs:

```
await page.goto('/login');
await page.getByLabel('Username').fill('komal');
await page.getByLabel('Password').fill('password123');
await page.getByRole('button', { name: 'Login' }).click();
await expect(page).toHaveURL(/dashboard/);
```

The timeline helps you inspect what happened during navigation, filling the fields, clicking Login and verifying the dashboard URL.

Interview point: Timeline view helps debug tests by showing the sequence and timing of actions with visual snapshots.

6. Actions tab

UI Mode | Playwright
playwright.dev

The Actions tab shows details about every action performed in your test.

It displays:

- Locator used for the action.
- Duration of the action.
- DOM snapshot changes.
- Before and After snapshots.

What are Before and After tabs?

They help you compare the DOM before and after an action.

Example:

```
await page.getByRole('button', { name: 'Add to cart' }).click();
```

- Before: Product is not yet added to the cart.
- After: Cart count or cart contents may change.

This makes it easier to understand what changed after a particular action.

7. Pop out and inspect the DOM

UI Mode | Playwright
playwright.dev

DOM means Document Object Model.

It represents the structure of a webpage using HTML elements.

In UI Mode, you can pop out a DOM snapshot into a separate window.

Steps:

1. Open a test execution.
2. Select an action.
3. Find the DOM snapshot.
4. Click the pop-out icon.
5. Inspect the snapshot in the separate window.
6. Open browser DevTools to inspect HTML, CSS and Console.

Why is this useful?

Suppose your locator is not working:

```
page.locator('#login-button');
```

You can inspect the DOM snapshot to check whether the element actually has the expected ID.

You can also compare DOM snapshots from different actions.

8. Pick locator

Running and Debugging Tests With Playwright UI Mode
dzone.com

This is one of the most useful features for automation testers.

The Pick Locator feature helps you identify and generate locators visually.

Steps:

1. Open UI Mode.
2. Click the Pick Locator button.
3. Hover over an element in the DOM snapshot.
4. Playwright highlights the element.

5. Click the element.
6. The locator appears in the locator playground.
7. Modify and validate the locator.
8. Copy the locator into your test code.

Example:

You want to identify the Login button.

Playwright may generate:

```
page.getByRole('button', { name: 'Login' });
```

You can then use it in your test:

```
await page.getByRole('button', { name: 'Login' }).click();
```

Why is this useful?

- Helps identify elements.
- Reduces manual locator-writing effort.
- Allows you to check whether a locator matches an element.
- Helps debug incorrect locators.

Interview point: Pick Locator is used to visually identify elements and generate or validate Playwright locators.

9. Source tab

Debugging Playwright Tests in VS Code - DEV Community
dev.to

The Source panel displays the test code associated with the selected action.

When you hover over an action, the corresponding line of code is highlighted.

There is also an Open in VSCode button.

When clicked, it opens the test file in VS Code at the corresponding line.

Example:

```
await page.getByRole('button', { name: 'Login' }).click();
```

If the Login action fails, you can select it in UI Mode and see the corresponding source-code line highlighted.

This makes debugging faster.

10. Call tab

The Call tab displays information about a particular action.

It includes:

- Time taken by the action.
- Locator used.
- Whether strict mode was involved.
- Key used, where applicable.

Example:

```
await page.getByRole('button', { name: 'Login' }).click();
```

The Call tab helps you inspect the details of this click operation.

Interview point: The Call tab provides detailed information about an individual Playwright action.

11. Log tab

UI Mode | Playwright
playwright.dev

The Log tab shows what Playwright is doing internally while executing an action.

For example, when you execute:

```
await page.getByRole('button', { name: 'Login' }).click();
```

Playwright may perform steps such as:

1. Find the locator.
2. Scroll the element into view.
3. Wait for the element to be visible.
4. Check whether it is enabled.
5. Check whether it is stable.
6. Perform the click.

The Log tab helps you understand these operations.

Practical scenario:

Your test is failing because the Login button is not clickable.

In the Log tab, you may discover that Playwright is waiting for the button to become enabled.

This helps identify the reason behind the failure.

12. Errors tab

Trace viewer | Playwright
playwright.dev

If a test fails, the Errors tab displays its error messages.

It also helps you:

- Identify the failed test.
- See the error message.
- Locate the failing source-code line.
- Identify the failure position on the timeline.

Example:

```
await expect(page).toHaveURL(/dashboard/);
```

Suppose the actual URL remains `/login`.

The Errors tab displays the assertion failure, and the timeline highlights where the failure occurred.

Interview point: The Errors tab helps identify the cause and location of failed tests.

13. Console tab

Running and Debugging Tests With Playwright UI Mode
dzone.com

The Console tab displays console logs from two sources:

1. Browser console.
2. Playwright test file.

Example browser console:

```
console.log('Login page loaded');
```

Example test-file console:

```
console.log('Executing login test');
```

Different icons identify whether the message came from the browser or the test file.

Why is it useful?

Suppose the login page shows an error after clicking Login.

You can inspect browser console messages to identify JavaScript errors and inspect test logs to understand the test execution.

14. Network tab

Running and Debugging Tests With Playwright UI Mode
dzone.com

The Network tab displays all network requests made during test execution.

You can sort requests using:

- Request type.
- Status code.

- HTTP method.
- Request name.
- Content type.
- Duration.
- Size.

When you click a request, you can inspect:

Information	Meaning
Request headers	Information sent with the request
Response headers	Information returned by the server
Request body	Data sent to the server
Response body	Data returned by the server

Practical QA example:

Suppose you click the Login button.

The application sends a login API request:

```
POST /api/login
```

The Network tab helps you inspect the request and response.

For example, you might see:

```
Request Method: POST
Status Code: 200
Content-Type: application/json
```

This helps investigate whether the problem is related to the frontend, backend or network request.

15. Attachments tab

[My Transition to Playwright for Frontend End-to-End Testing | Blog | Nyef.nl
nyef.nl]

The Attachments tab allows you to inspect files attached to a test.

One important use case is visual regression testing.

It allows you to compare:

- Actual screenshot.
- Expected screenshot.
- Image difference (diff).

Example:

Suppose your expected checkout page screenshot has a blue Submit Order button, but the actual screenshot has a red button.

The image diff helps identify the visual difference.

You can also use a slider to compare expected and actual screenshots.

Interview point: The Attachments tab is useful for inspecting test artifacts, especially screenshots and visual regression differences.

16. Metadata tab

The Metadata tab displays additional information about a test execution.

It includes:

- Browser.
- Viewport size.
- Test duration.
- Other test execution details.

Example:

Metadata	Example
Browser	Chromium
Viewport	1280 × 720
Test duration	5 seconds

Why is it useful?

Suppose a test passes in Chromium but fails in Firefox.

Metadata helps you identify which browser and viewport were used for that execution.

17. Watch mode

Playwright の UI Mode で E2E テストに入門する｜jig.jp engineers note.com

Watch mode automatically reruns tests when you make changes to the test code.

In the UI Mode sidebar, you will see an eye icon next to each test.

How to use it:

1. Open UI Mode.
2. Find the test you want to monitor.
3. Click the eye icon beside that test.
4. Make changes to the test code.
5. Playwright reruns the watched test.

You can watch:

- One test.
- Multiple tests.
- All tests using the eye icon at the top of the sidebar.

Example:

You are debugging a login test and changing the locator repeatedly.

Without watch mode, you need to manually execute the test after each change.

With watch mode, Playwright automatically reruns the selected test when you make changes.

Interview point: Watch mode improves development productivity by automatically rerunning selected tests when code changes.

18. Docker and GitHub Codespaces

Usually, UI Mode runs locally.

But when you work inside Docker or GitHub Codespaces, you may want to access UI Mode through a browser outside the container.

For this, Playwright provides the `--ui-host` option.

Command 1: Specify host

```
npx playwright test --ui-host=0.0.0.0
```

`0.0.0.0` allows the UI server to listen on all network interfaces.

This makes the endpoint accessible from outside the container, subject to network and firewall configuration.

Command 2: Specify port

```
npx playwright test --ui-port=8080 --ui-host=0.0.0.0
```

Option	Meaning
`--ui-host`	Specifies the host interface
`0.0.0.0`	Listens on all network interfaces

`--ui-port`	Specifies the port
`8080`	Example port number

In GitHub Codespaces, the port can be forwarded automatically, allowing you to open UI Mode through the browser.

Security warning

When you use `--ui-host=0.0.0.0`, the UI Mode endpoint and its traces may expose sensitive information, including passwords and secrets, to other machines on the network.

Use it only in an appropriately secured environment.

19. Important UI Mode features – Quick revision

Feature	Main purpose
UI Mode	Explore, run and debug tests visually
Timeline	Inspect test actions over time
Actions	View locators and action details
Before/After	Compare DOM snapshots
Pop out DOM	Inspect DOM in a separate window
Pick Locator	Generate and validate locators
Source	View corresponding test code
Call	Inspect individual action details

Log	Understand Playwright's internal execution steps
Errors	Investigate failed tests
Console	View browser and test logs
Network	Inspect network requests and responses
Attachments	Inspect screenshots and image diffs
Metadata	View browser and execution details
Watch mode	Automatically rerun tests after changes

20. UI Mode vs Debug Mode

This is a useful interview comparison.

UI Mode	Debug Mode
Opens an interactive test interface	Runs tests in debugging mode
Displays test files and test cases	Helps debug test execution
Supports timeline and snapshots	Supports step-by-step debugging
Has built-in watch mode	Focuses on debugging
Includes locator picker and network panel	Provides debugging tools such as Playwright Inspector

Example commands:

```
# UI Mode
npx playwright test --ui

# Debug Mode
npx playwright test --debug
```

21. SDET interview questions and answers

Q1. What is Playwright UI Mode?

UI Mode is an interactive interface that allows us to explore, execute, filter and debug Playwright tests visually. It provides features such as timeline, DOM snapshots, locator picker, console and network logs.

Q2. How do you open UI Mode?

```
npx playwright test --ui
```

Q3. What is time-travel debugging?

It allows us to move backward and forward through test actions and inspect the DOM snapshot at different execution moments.

Q4. What is Pick Locator?

It helps us visually select an element, generate its locator and validate that locator in the locator playground.

Q5. What is the purpose of the Network tab?

It displays network requests made during test execution and provides request and response details.

Q6. What is the purpose of the Log tab?

It shows Playwright's execution steps, including scrolling, waiting for visibility, checking whether elements are enabled and performing actions.

Q7. What is Watch mode?

Watch mode automatically reruns selected tests when changes are made to the test code.

Q8. What is the difference between Actions and Call tabs?

The Actions tab provides an overview of test actions and their DOM snapshots. The Call tab gives detailed information about an individual action, such as duration, locator and strict-mode information.

Q9. What is the purpose of the Attachments tab?

It allows us to inspect test attachments, including expected and actual screenshots and image differences during visual regression testing.

Q10. How do you access UI Mode in Docker?

```
npx playwright test --ui-host=0.0.0.0
```

Q11. What is the purpose of `--ui-port`?

It specifies the port on which UI Mode is hosted.

Q12. Does UI Mode support filtering?

Yes. Tests can be filtered by name, project, tag and execution status such as passed, failed and skipped.

22. Practice MCQ quiz

UI Mode MCQ test

7/7 answered

1. Which command opens Playwright UI Mode?

npx playwright test --debug
npx playwright test --ui
npx playwright test --watch
npx playwright ui

Correct

The --ui flag opens the interactive UI Mode.

2. Which feature helps visually generate and validate locators?

Timeline
Metadata
Pick Locator
Attachments

Correct

Pick Locator selects an element and provides a locator playground.

3. Which tab displays request headers and response bodies?

Console
Network
Source
Call

Correct

The Network tab shows requests and their details.

4. Which feature automatically reruns selected tests after code changes?

Watch mode
Timeline
Metadata
Pop out DOM

Correct

Watch mode reruns tests when changes are made.

5. Which tab shows Playwright's internal steps such as scrolling and waiting?

Errors
Attachments
Log
Source

Correct answer: Log

The Log tab shows the steps Playwright performs behind the scenes.

6. Which tab helps compare expected and actual screenshots?

Call
Attachments
Console
Metadata

Correct

Attachments provides access to screenshot comparisons and image diffs.

7. What does --ui-host=0.0.0.0 do?

Runs tests in Firefox
Enables watch mode
Listens on all network interfaces
Creates a screenshot

Correct

It makes the UI endpoint listen on all network interfaces, subject to network access controls.

Score: 6/7

Retake quiz

Most important for interviews: Focus on UI Mode, Timeline, Actions, Pick Locator, Log, Network, Attachments and Watch Mode. These are practical features you can explain using real debugging examples from your automation project.

Sure! From now on, whenever you share a Playwright section, I'll explain it in simple English with practical examples, keeping QA Automation interviews in mind.

For every section, I'll follow this structure:

1. Introduction: What is it and why do we use it?
2. Simple explanation: Explain each concept in easy language.
3. Syntax and code explanation: Explain important code line by line.
4. Important properties/options: Explain each property with its purpose.
5. Real-time example: How we use it in automation projects.
6. Important points: What you should remember for interviews.
7. Interview questions and answers: Basic, intermediate and scenario-based questions with detailed answers.
8. Practical interview questions: Questions where you may need to explain or write code.

Section 1: Web Server in Playwright

1. What is a Web Server?

A web server is a system that runs your web application and makes it accessible through a URL.

For example:

Suppose you are developing an e-commerce application. You want to automate its login, search, cart and checkout functionalities.

Before executing your Playwright test cases, your application must be running.

You can start it manually, or Playwright can start it automatically using the `webServer` configuration.

In simple words:

Playwright's `webServer` option automatically starts your application before executing test cases and makes sure the application is ready.

2. Why do we use Web Server?

Imagine this situation:

Without webServer:

1. Open your project.
2. Start your application manually.
3. Wait until the application starts.
4. Open another terminal.
5. Execute Playwright test cases.

With webServer:

1. Execute Playwright test cases.
2. Playwright automatically starts the application.

3. It waits until the application is ready.
4. It executes test cases.

Main advantage: We don't need to start the application manually every time.

3. Configuring a Web Server

Example:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  webServer: {
    command: 'npm run start',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
    stdout: 'ignore',
    stderr: 'pipe',
  },
});
```

Let's understand each property.

Line-by-line explanation

1. `command`

```
command: 'npm run start'
```

- Specifies the command used to start your application.
- Playwright executes this command automatically.
- Here, `npm run start` starts the development server.

2. `url`

```
url: 'http://localhost:3000'
```

- Specifies the URL where your application runs.
- Playwright continuously checks this URL.

- Once the URL responds with an accepted status code, Playwright considers the server ready.

3. `reuseExistingServer`

```
reuseExistingServer: !process.env.CI
```

- Checks whether an application is already running.
- If it is running, Playwright can reuse that server.
- If it is not running, Playwright starts a new server.

Here:

- `!process.env.CI` means true when the CI environment variable is not set.
- Locally, it generally reuses an existing server.
- In CI, it generally starts a fresh server.

4. `stdout`

```
stdout: 'ignore'
```

- Controls the standard output logs of the server.
- `ignore` means don't display normal server logs in the terminal.
- `pipe` means display those logs.

5. `stderr`

```
stderr: 'pipe'
```

- Controls error messages generated by the server.
- `pipe` means display error messages in the terminal.
- `ignore` means don't display them.

4. Important Web Server Properties

Property	Simple meaning

`command`	Command to start the application
`url`	Application URL
`cwd`	Directory from which the command runs
`env`	Environment variables
`timeout`	Maximum time to wait for server startup
`reuseExistingServer`	Reuse an already running server
`stdout`	Controls normal output logs
`stderr`	Controls error logs
`name`	Custom name for the server
`ignoreHTTPSErrors`	Ignore HTTPS certificate errors when checking the URL
`gracefulShutdown`	Controls how the server is stopped
`wait`	Wait for specific output before considering the process started

Important: `port` is deprecated. Use `url` instead.

5. Adding a Server Timeout

Sometimes your application takes longer to start.

For example:

- Your application normally starts in 10 seconds.

- Sometimes it takes 90 seconds because of database initialization or other processes.
- The default timeout is 60 seconds.

You can increase it:

```
webServer: {
  command: 'npm run start',
  url: 'http://localhost:3000',
  reuseExistingServer: !process.env.CI,
  timeout: 120 * 1000,
}
```

Here:

```
120 * 1000 = 120000 milliseconds = 120 seconds
```

So Playwright waits up to 2 minutes for the server to become ready.

6. Adding a baseURL

What is baseURL?

baseURL is the common starting URL of your application.

Without baseURL:

```
await page.goto('http://localhost:3000/login');
await page.goto('http://localhost:3000/home');
await page.goto('http://localhost:3000/cart');
```

You need to write the complete URL every time.

With baseURL:

```
use: {
  baseURL: 'http://localhost:3000',
}
```

Now you can write:

```
await page.goto('/login');
await page.goto('/home');
await page.goto('/cart');
```

Playwright automatically combines the baseURL with the relative path.

For example:

Code	Actual URL
`page.goto('/login')`	http://localhost:3000/login
`page.goto('/home')`	http://localhost:3000/home
`page.goto('/cart')`	http://localhost:3000/cart

Complete configuration

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  webServer: {
    command: 'npm run start',
    url: 'http://localhost:3000',
    reuseExistingServer: !process.env.CI,
  },

  use: {
    baseURL: 'http://localhost:3000',
  },
});
```

Test file:

```
import { test } from '@playwright/test';

test('Login test', async ({ page }) => {
  await page.goto('/login');
});
```

Remember the difference:

- `webServer` starts your application.
- `baseURL` avoids writing the complete URL repeatedly.

They have different purposes.

7. Multiple Web Servers

Sometimes an application has two parts:

- Frontend: React application running on port 3000.
- Backend: Java/Node.js API running on port 3333.

You can start both servers using an array.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  webServer: [
    {
      command: 'npm run start',
      url: 'http://localhost:3000',
      name: 'Frontend',
    },
    {
      command: 'npm run backend',
      url: 'http://localhost:3333',
      name: 'Backend',
    },
  ],

  use: {
    baseURL: 'http://localhost:3000',
  },
});
```

Explanation:

- First server starts the frontend.
- Second server starts the backend.
- Playwright waits for both servers to become ready.
- Tests navigate to the frontend using baseURL.

8. Important Interview Questions and Answers

Here are the questions you should prepare for a Playwright QA Automation interview.

Q1. What is webServer in Playwright?

Answer:

WebServer is an option available in the Playwright configuration file that allows us to start a local development server automatically before executing test cases.

It executes a specified command, such as `npm run start`, and waits until the application becomes accessible through the configured URL.

This eliminates the need to start the application manually before running tests.

Q2. Why do we use webServer in Playwright?

Answer:

We use webServer to automate application startup before test execution.

For example, if our application runs on localhost:3000, we configure the webServer option with the startup command and URL.

Whenever we execute our Playwright tests, it starts the application, waits until it is ready and then executes the test cases.

Q3. What is the difference between webServer and baseURL?

Answer:

Both have different purposes.

- `webServer` is responsible for starting the application before test execution.
- `baseURL` defines the common base address of the application.

For example, if baseURL is `http://localhost:3000`, we can navigate using `page.goto('/login')` instead of writing the complete URL.

Q4. What is reuseExistingServer?

Answer:

`reuseExistingServer` checks whether the application is already running.

If it is set to true and the server is already available, Playwright reuses the existing server instead of starting another one.

If the server is not running, Playwright executes the configured startup command.

It is commonly configured as:

```
reuseExistingServer: !process.env.CI
```

This allows reuse locally but generally starts a fresh server in CI.

Q5. What happens if the web server takes more than 60 seconds to start?

Answer:

By default, Playwright waits for 60 seconds for the web server to become ready.

If the server does not become available within that time, the startup process fails with a timeout error.

We can increase the timeout using the `timeout` property.

```
timeout: 120 * 1000
```

This increases the waiting time to 120 seconds.

Q6. Can we configure multiple web servers in Playwright?

Answer:

Yes. Playwright supports multiple web servers using an array of webServer configurations.

For example, we can configure one server for the frontend and another for the backend.

Playwright starts both servers and waits until they become ready before executing tests.

Q7. What is the difference between stdout and stderr?

Answer:

- `stdout` stands for Standard Output. It contains normal messages generated by the application.
- `stderr` stands for Standard Error. It contains error messages generated by the application.

In Playwright, we can configure both using `pipe` or `ignore`.

Q8. What is the default timeout of webServer?

Answer:

The default webServer startup timeout is 60,000 milliseconds, which is equal to 60 seconds.

We can customize it using the `timeout` property.

Q9. Can we use page.goto('/login') without specifying the complete URL?

Answer:

Yes. We need to configure `baseURL` in the Playwright configuration file under the `use` section.

For example:

```
use: {
  baseURL: 'http://localhost:3000',
}
```

Then:

```
await page.goto('/login');
```

Playwright resolves the relative URL using the configured baseURL.

Q10. What happens if the web server is already running and reuseExistingServer is false?

Answer:

Playwright throws an error if another process is already listening on the configured URL or port. It does not reuse that existing server.

9. Scenario-Based Interview Questions

Q11. Your Playwright tests are failing because the application is not running. How will you solve this?

Answer:

I will configure the webServer option in `playwright.config.ts`.

I will provide the application startup command and URL. Playwright will automatically start the application and wait until it becomes ready before executing test cases.

Q12. Your application takes 90 seconds to start, but Playwright is failing after 60 seconds. What will you do?

Answer:

I will increase the webServer timeout.

```
timeout: 120 * 1000
```

This allows Playwright to wait up to 120 seconds for the application to become ready.

Q13. Your frontend and backend applications need to run before automation. How will you configure them?

Answer:

I will configure multiple web servers using an array in the Playwright configuration file.

I will provide separate commands and URLs for the frontend and backend.

Q14. You are repeatedly writing the complete URL in every test case. How will you avoid this?

Answer:

I will configure `baseURL` inside the `use` section of `playwright.config.ts`.

Then I can use relative URLs in `page.goto()`.

10. Quick Revision Notes

Topic	Remember
WebServer	Starts application automatically
command	Starts the application
url	Checks application readiness
baseURL	Avoids writing complete URLs
reuseExistingServer	Reuses an existing server
timeout	Default 60 seconds
stdout	Normal logs
stderr	Error logs
Multiple servers	Frontend + Backend
port	Deprecated; use url

11. Test Your Knowledge

Playwright Web Server Quiz

5/5 answered

Q1. What is the main purpose of webServer in Playwright?

To generate reports
To start the application before tests
To take screenshots
To create test cases

Incorrect — To start the application before tests

webServer starts the local application and waits until it is ready before test execution.

Q2. What is the default webServer timeout?

10 seconds
30 seconds
60 seconds
120 seconds

Incorrect — 60 seconds

The default startup timeout is 60,000 milliseconds.

Q3. Which property avoids writing the full URL repeatedly?

command
timeout
baseURL
reuseExistingServer

Incorrect — baseURL

baseURL lets you use relative URLs such as /login.

Q4. What does reuseExistingServer: !process.env.CI generally do?

Always stops the server
Reuses an existing server locally but not in CI
Disables all tests
Starts two servers

Incorrect — Reuses an existing server locally but not in CI

The negation is true when CI is not set, allowing local reuse.

Q5. How do you configure frontend and backend servers?

Using two baseURLs

Using an array of webServer configurations
Using stdout
Using two test files

Correct

Playwright accepts an array of webServer configurations.

Submit answers

Score: 1/5

Try again

My recommendation: Focus especially on `webServer`, `baseURL`, `reuseExistingServer`, `timeout`, and multiple web servers. These are important concepts to explain confidently in a Playwright Automation interview.

Send me the next Playwright documentation section, and we'll continue with the same detailed format.

Playwright – Library

This section is very important for interviews because interviewers may ask:

- What is Playwright Library?
- What is the difference between Playwright Library and Playwright Test?
- Why do we use `@playwright/test`?
- What is the purpose of Browser, Context and Page?
- What is headless mode?
- What is Codegen?

Let's understand everything in simple language with practical examples.

1. Introduction – What is Playwright Library?

Playwright provides two ways to automate browsers:

1. Playwright Library
2. Playwright Test

Playwright Library

Playwright Library provides APIs that allow us to:

- Launch browsers.
- Open web pages.
- Click buttons.
- Enter text.
- Navigate between pages.
- Take screenshots.
- Interact with web elements.

In simple words, Playwright Library provides the tools to control browsers through code.

Playwright Test

Playwright Test provides browser automation capabilities along with a complete testing framework.

It includes:

- Test runner.
- Assertions.
- Fixtures.
- Parallel execution.

- Reporting.
- Retries.
- Configuration.
- Automatic browser and page management.

Important interview point:

For normal end-to-end automation testing projects, we generally use `@playwright/test` instead of using the Playwright Library directly.

Why? Because Playwright Test provides additional testing features and handles many tasks automatically.

2. Difference between Playwright Library and Playwright Test

First, understand these two packages:

Playwright Library	Playwright Test
`playwright`	`@playwright/test`
Browser automation APIs	Browser automation + test framework
Manual browser initialization	Automatic fixtures
No built-in Web-First assertions	Built-in Web-First assertions
Manual cleanup	Automatic cleanup
Run using Node.js	Run using Playwright Test Runner
Manual test execution management	Test execution management

No built-in test reporting	Built-in reporting support
Manual parallel execution setup	Built-in parallel execution

Installation difference

Playwright Library:

```
npm install -D playwright
```

Playwright Test:

```
npm init playwright@latest
```

Remember:

- `npm install` installs the Playwright Library package.
- `npm init playwright@latest` initializes a Playwright Test project and guides you through the setup.

3. Library Example – Understanding the Code

Let's understand the example from your documentation.

```
import { chromium, devices } from 'playwright';
import assert from 'node:assert';

(async () => {

  // Setup
  const browser = await chromium.launch();

  const context = await browser.newContext(
    devices['iPhone 11']
  );

  const page = await context.newPage();

  // Actual test
  await context.route('**.jpg', route => route.abort());
```

```
  await page.goto('https://example.com/');

  assert(await page.title() === 'Example Domain');

  // Cleanup
  await context.close();
  await browser.close();

})();
```

Let's understand line by line.

Step 1: Import browser

```
import { chromium, devices } from 'playwright';
```

- `chromium`: Used to launch the Chromium browser.
- `devices`: Provides predefined device configurations.

For example, we can use iPhone 11 settings to emulate a mobile device.

Step 2: Import assert

```
import assert from 'node:assert';
```

`assert` is a Node.js built-in module used to verify conditions.

For example:

```
assert(10 > 5);
```

If the condition is true, the test continues.

If the condition is false, an assertion error occurs.

Step 3: Launch browser

```
const browser = await chromium.launch();
```

This launches the Chromium browser.

Step 4: Create browser context

```
const context = await browser.newContext(
  devices['iPhone 11']
);
```

A Browser Context is an isolated browser session.

Here, we create a context with iPhone 11 device settings.

It helps us test how our application behaves on a mobile device.

Step 5: Create a page

```
const page = await context.newPage();
```

This opens a new browser tab inside the context.

Step 6: Intercept network requests

```
await context.route(
  '**.jpg',
  route => route.abort()
);
```

This is an important concept.

- `context.route()` intercepts network requests.
- `**.jpg` matches requests for JPG files.
- `route.abort()` blocks those requests.

For example, if the application tries to load an image called `product.jpg`, Playwright blocks that request.

This is useful when testing application behavior without loading certain resources.

Step 7: Navigate to website

```
await page.goto('https://example.com/');
```

This opens the specified website in the browser page.

Step 8: Verify page title

```
assert(await page.title() === 'Example Domain');
```

This checks whether the page title is equal to `Example Domain`.

If it matches, the assertion passes.

If it does not match, the assertion fails.

Interview important
This is a normal Node.js assertion, not a Playwright Web-First assertion. It does not automatically retry until the expected condition is satisfied.

Step 9: Close context and browser

```
await context.close();
await browser.close();
```

We manually close the browser context and browser to release resources.

This is called cleanup or teardown.

4. Test Example – Same functionality using Playwright Test

Now let's see how we perform similar automation using `@playwright/test`.

```
import { expect, test, devices } from '@playwright/test';

test.use(devices['iPhone 11']);

test('should be titled', async ({ page, context }) => {

  await context.route('**.jpg', route => route.abort());

  await page.goto('https://example.com/');

  await expect(page).toHaveTitle('Example');

});
```

Let's understand the differences.

Step 1: Import

```
import { expect, test, devices } from '@playwright/test';
```

- `expect`: Used for assertions.
- `test`: Used to write and execute test cases.
- `devices`: Provides predefined device configurations.

Step 2: Configure device

```
test.use(devices['iPhone 11']);
```

This configures the test to use iPhone 11 device settings.

We don't need to manually create a browser context.

Step 3: Write test case

```
test('should be titled', async ({ page, context }) => {
```

Here:

- `test()` defines a test case.
- `page` is a built-in fixture.
- `context` is another built-in fixture.

Playwright Test automatically provides these fixtures when required.

Step 4: Block JPG images

```
await context.route('**.jpg', route => route.abort());
```

Same functionality as the Library example.

Step 5: Open website

```
await page.goto('https://example.com/');
```

Navigates to the website.

Step 6: Verify title

```
await expect(page).toHaveTitle('Example');
```

This is called a Web-First Assertion.

What is special about it?

Suppose the page title takes some time to appear.

- Normal assertion checks the condition immediately.
- Web-First assertion automatically waits and retries until the condition passes or the assertion timeout is reached.

This makes Playwright Test assertions more reliable for dynamic web applications.

Note: The documentation's example uses `Example` as the expected title. In your own tests, use the exact expected title or a suitable regular expression for your application.

5. What are Browser, Browser Context and Page?

This is one of the most frequently asked Playwright interview topics.

Modern Web Testing with Playwright | Automation Panda
automationpanda.com

Understand the hierarchy:

Browser → Browser Context → Page

Browser

Browser represents the actual browser instance.

Example: Chromium, Firefox or WebKit.

```
const browser = await chromium.launch();
```

Browser Context

A Browser Context represents an isolated browser session.

It maintains separate:

- Cookies
- Local storage
- Session storage
- Session-related browser state

```
const context = await browser.newContext();
```

For example, we can create two different contexts:

- Context 1: User A logs in.
- Context 2: User B logs in.

Both users have separate sessions.

Page

A Page represents an individual browser tab.

```
const page = await context.newPage();
```

For example:

- Page 1: Login page.
- Page 2: Dashboard.
- Page 3: Profile page.

Multiple pages can exist inside one context.

Interview example:

If an interviewer asks how you test two users simultaneously without sharing login sessions, you can explain that you create separate browser contexts for each user.

6. Usage – How to install Playwright Library

If you want to use Playwright Library directly, install it using:

```
npm i -D playwright
```

Here:

- `npm`: Node Package Manager.
- `i`: Short form of install.
- `-D`: Installs the package as a development dependency.
- `playwright`: Package name.

Install browsers

After installing the package, we also need browser binaries.

```
npx playwright install chromium firefox webkit
```

This downloads the three supported browser engines.

Browser	Engine
Chrome	Chromium
Microsoft Edge	Chromium
Firefox	Firefox
Safari	WebKit

Important: Playwright uses Chromium, Firefox and WebKit browser engines. It does not directly automate Safari using the Safari application itself.

Alternative installation

```
npm i -D @playwright/browser-chromium
npm i -D @playwright/browser-firefox
npm i -D @playwright/browser-webkit
```

These are browser-specific packages that can download the corresponding browsers during package installation.

7. First Script – Launch Browser and Take Screenshot

Let's understand the first automation script.

```
const { webkit } = require('playwright');

(async () => {

  const browser = await webkit.launch();

  const page = await browser.newPage();

  await page.goto('https://playwright.dev/');

  await page.screenshot({ path: 'example.png' });

  await browser.close();

})();
```

Explanation

1. Import WebKit

```
const { webkit } = require('playwright');
```

Imports WebKit from the Playwright package.

2. Launch browser

```
const browser = await webkit.launch();
```

Launches the WebKit browser.

3. Create a page

```
const page = await browser.newPage();
```

Creates a new page and its associated context.

4. Navigate to website

```
await page.goto('https://playwright.dev/');
```

Opens the Playwright website.

5. Take screenshot

```
await page.screenshot({ path: 'example.png' });
```

Captures a screenshot and saves it as `example.png`.

6. Close browser

```
await browser.close();
```

Closes the browser after execution.

What is headless mode?

By default, Playwright launches browsers in headless mode.

Headless means the browser runs in the background without displaying its graphical user interface.

Example:

```
const browser = await chromium.launch();
```

To see the browser on your screen:

```
const browser = await chromium.launch({
  headless: false
});
```

What is slowMo?

`slowMo` slows down Playwright operations by the specified number of milliseconds.

```
const browser = await chromium.launch({
  headless: false,
  slowMo: 50
});
```

Here:

- `headless: false` displays the browser.
- `slowMo: 50` adds a 50-millisecond delay to Playwright operations.

Why use slowMo?

It helps us visually observe actions during debugging.

8. Record Scripts – Codegen

What is Codegen?

Codegen is a Playwright tool that automatically generates automation code based on our manual browser interactions.

Suppose you want to automate a login flow.

Instead of writing every locator and action manually, you can:

1. Open Codegen.
2. Enter the website URL.
3. Perform login manually.
4. Click buttons and navigate through pages.
5. Playwright generates the corresponding automation code.

Command:

```
npx playwright codegen wikipedia.org
```

This opens a browser and the Playwright Inspector.

Test generator | Playwright
playwright.dev

Advantages:

- Automatically generates locators.
- Generates code for user interactions.
- Helps beginners learn Playwright syntax.
- Saves time when creating initial test scripts.

Interview point: Codegen helps generate automation scripts, but we should review and improve the generated code before using it in a real project.

9. Browser Downloads

Command:

```
npx playwright install
```

Downloads the supported browsers required by Playwright.

To install only Chromium:

```
npx playwright install chromium
```

To install only Firefox:

```
npx playwright install firefox
```

To install only WebKit:

```
npx playwright install webkit
```

Download behind a firewall or proxy

Sometimes company networks restrict direct downloads.

In that case, we can use the HTTPS_PROXY environment variable.

Example:

```
HTTPS_PROXY=https://192.0.2.1 npx playwright install
```

This tells Playwright to download browsers through the specified proxy.

Download from an artifact repository

Companies sometimes maintain their own internal artifact repositories.

We can use:

```
PLAYWRIGHT_DOWNLOAD_HOST=192.0.2.1 npx playwright install
```

This specifies an alternative browser download host.

Skip browser download

Sometimes browser binaries are already managed separately.

We can skip automatic browser downloads using:

```
PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD=1 npm install
```

Remember these three variables:

Variable	Purpose
`HTTPS_PROXY`	Download through a proxy
`PLAYWRIGHT_DOWNLOAD_HOST`	Download from an alternative host
`PLAYWRIGHT_SKIP_BROWSER_DOWNLOAD`	Skip browser downloads

The examples above show the Bash/Linux syntax. Windows PowerShell and Command Prompt use different environment-variable syntax.

10. TypeScript Support

Playwright supports both JavaScript and TypeScript.

What is TypeScript?

TypeScript is a programming language built on JavaScript that adds static type checking.

For example:

JavaScript:

```
let age = 25;
```

TypeScript:

```
let age: number = 25;
```

Here, `number` specifies the variable's data type.

TypeScript in Playwright

Example:

```
let page: import('playwright').Page;
```

Here:

- `page` is a variable.
- `import('playwright').Page` specifies its type.
- It tells TypeScript that this variable represents a Playwright Page object.

JavaScript type checking

You can add:

```
// @ts-check
```

At the top of your JavaScript file.

This enables TypeScript checking in supported editors such as VS Code.

You can also use JSDoc:

```
/** @type {import('playwright').Page} */
let page;
```

This tells the editor that `page` should be treated as a Playwright Page object.

11. What is async/await and IIFE?

Your documentation uses this syntax:

```
(async () => {

  // Your code

})();
```

Let's understand it.

What is async?

`async` is used to declare an asynchronous function.

```
async function test() {
  console.log('Hello');
}
```

What is await?

`await` waits for a Promise to settle before continuing to the next statement.

Example:

```
await page.goto('https://playwright.dev/');
```

Playwright waits for the navigation operation to complete according to the navigation's completion criteria.

What is IIFE?

IIFE stands for Immediately Invoked Function Expression.

It is a function that executes immediately after it is defined.

Example:

```
(function () {
  console.log('Hello');
})();
```

In Playwright:

```
(async () => {
  const browser = await chromium.launch();
  await browser.close();
})();
```

This is an asynchronous IIFE.

Why do we use it?

It allows us to use `await` inside the function and execute the script immediately without explicitly calling a separately declared function.

12. Important Interview Questions and Answers

These questions are particularly important for QA Automation / Playwright interviews.

Q1. What is Playwright Library?

Answer:

Playwright Library is a Node.js library that provides APIs to automate browsers like Chromium, Firefox and WebKit.

It allows us to perform browser operations such as launching browsers, navigating to websites, clicking elements, filling forms, handling browser contexts and taking screenshots.

It provides browser automation capabilities, but it does not provide a complete built-in test runner like Playwright Test.

Q2. What is the difference between Playwright Library and Playwright Test?

Answer:

Playwright Library provides browser automation APIs, whereas Playwright Test provides browser automation along with a complete testing framework.

In Playwright Library, we need to manually launch the browser, create browser contexts, create pages and close browser resources.

In Playwright Test, these operations are managed using built-in fixtures.

Playwright Test also provides:

- Web-First assertions.
- Parallel execution.
- Test reporting.
- Retries.

- Configuration.
- Projects.
- Tracing.

For end-to-end automation testing, I would generally prefer Playwright Test because it provides all the required testing features.

Q3. What is the difference between `playwright` and `@playwright/test`?

Answer:

`playwright` is the Playwright Library package, which provides browser automation APIs.

`@playwright/test` is the Playwright Test package, which includes browser automation capabilities along with a test runner, assertions, fixtures and reporting.

We import from:

```
import { chromium } from 'playwright';
```

For Playwright Test:

```
import { test, expect } from '@playwright/test';
```

Q4. What are the three browsers supported by Playwright?

Answer:

Playwright supports three major browser engines:

1. Chromium – Used by Chrome and Edge.
2. Firefox – Mozilla Firefox engine.
3. WebKit – Used by Safari.

We can execute the same automation scripts across these browser engines to perform cross-browser testing.

Q5. What is Browser Context?

Answer:

A Browser Context is an isolated browser session.

It maintains its own cookies, local storage and session-related state.

For example, if I want to test two different user accounts simultaneously, I can create two separate browser contexts.

One context can be used for User A and another for User B. Their browser sessions remain isolated.

Q6. What is the difference between Browser, Browser Context and Page?

Answer:

- Browser represents the actual browser instance.
- Browser Context represents an isolated browser session.
- Page represents an individual browser tab.

For example, one browser can have multiple contexts, and each context can contain multiple pages.

Q7. What is the difference between headless and headed mode?

Answer:

Headless mode means the browser executes automation in the background without displaying the browser window.

Headed mode means the browser window is visible on the screen.

Example:

```
const browser = await chromium.launch({
  headless: false
});
```

We generally use headless mode in CI/CD pipelines and headed mode when debugging or observing test execution.

Q8. What is slowMo in Playwright?

Answer:

`slowMo` is an option that slows down Playwright operations by a specified number of milliseconds.

For example:

```
const browser = await chromium.launch({
  headless: false,
  slowMo: 100
});
```

This adds a 100-millisecond delay to operations and helps us observe browser interactions during debugging.

Q9. What is Codegen in Playwright?

Answer:

Codegen is a Playwright command-line tool that records our manual browser interactions and generates corresponding automation code.

For example, if we open a website and perform login, search and add-to-cart operations, Codegen generates the Playwright code for those interactions.

Command:

`npx playwright codegen https://example.com`

It is useful for generating locators and creating initial automation scripts.

Q10. What is the difference between assert and expect?

Answer:

`assert` is a Node.js assertion module, whereas `expect` is Playwright Test's assertion API.

Example of Node.js assert:

```
import assert from 'node:assert';

assert(await page.title() === 'Example Domain');
```

Example of Playwright expect:

```
await expect(page).toHaveTitle('Example Domain');
```

The important difference is that Playwright Web-First assertions automatically wait and retry until the condition is satisfied or the timeout is reached.

Q11. Does Playwright Library support async/await?

Answer:

Yes. Playwright APIs are asynchronous and return Promise objects.

We generally use async/await to handle these operations.

For example:

```
await page.goto('https://example.com');
await page.locator('#username').fill('Komal');
await page.locator('#password').fill('12345');
```

This makes the automation code easier to read and maintain.

Q12. Why do we need to close the browser after execution?

Answer:

We close the browser to release resources such as memory and browser processes.

When using Playwright Library directly, we need to manually close the context and browser.

```
await context.close();
await browser.close();
```

In Playwright Test, built-in browser fixtures are automatically managed by the Test Runner.

Q13. How can you take a screenshot using Playwright?

Answer:

We use the `screenshot()` method available on the Page object.

```
await page.screenshot({
  path: 'example.png'
});
```

It captures the current page and saves the screenshot at the specified path.

Q14. How can you execute Playwright Library code?

Answer:

We can execute a JavaScript file using Node.js.

```
node my-script.js
```

For Playwright Test, we use:

```
npx playwright test
```

Q15. What are Web-First Assertions?

Answer:

Web-First assertions are Playwright Test assertions that automatically wait and retry until the expected condition is met or the assertion timeout is reached.

Example:

```
await expect(page.locator('#message'))
  .toHaveText('Login Successful');
```

This is useful when elements or their values take some time to appear or update.

Q16. What are Projects in Playwright Test?

Answer:

Projects allow us to execute the same test suite with different configurations.

For example, we can configure projects for:

- Chromium
- Firefox
- WebKit

- Desktop Chrome
- Mobile Chrome

This helps us perform cross-browser and cross-device testing without modifying the test scripts.

Q17. Can we use Playwright Library without Playwright Test?

Answer:

Yes. We can directly install the `playwright` package and write Node.js scripts to automate browsers.

However, we need to manage browser initialization, assertions, test execution and cleanup ourselves.

13. Scenario-Based Interview Questions

Q18. You need to test the same application in Chrome, Firefox and Safari. How will you approach this?

Answer:

I will use Playwright Test projects to configure different browser engines.

I can define Chromium, Firefox and WebKit projects in the Playwright configuration file.

Then I can execute the same test cases across all three browser engines without changing the test code.

Q19. Your test passes in headed mode but fails in headless mode. What will you check?

Answer:

I will check:

1. Whether the application behaves differently in headless mode.
2. Whether there are timing or synchronization issues.
3. Whether elements are properly visible and accessible.

4. Whether the test depends on screen size or device settings.
5. Whether there are browser-specific issues.

I will also check Playwright execution logs, screenshots and traces to identify the issue.

Q20. You want to test an application on a mobile device. How will you do it?

Answer:

I will use Playwright's predefined device configurations.

For example:

```
import { test, devices } from '@playwright/test';

test.use(devices['iPhone 11']);
```

This applies the device's predefined settings, such as viewport, user agent and mobile emulation options.

Q21. You want to block images while testing a website. How will you do it?

Answer:

I will use Playwright's network interception functionality.

```
await context.route('**.jpg', route => route.abort());
```

This intercepts matching JPG requests and aborts them.

Q22. Your company network blocks Playwright browser downloads. What will you do?

Answer:

I will configure the `HTTPS_PROXY` environment variable if the company provides a proxy.

Alternatively, if the company maintains an internal artifact repository, I can configure `PLAYWRIGHT_DOWNLOAD_HOST` to download the browsers from that repository.

14. Most Important Differences to Remember

Concept	Playwright Library	Playwright Test
Package	playwright	@playwright/test
Purpose	Browser automation	End-to-end testing
Browser launch	Manual	Managed through fixtures
Context creation	Manual	Automatic
Page creation	Manual	Automatic
Assertions	External assertion library or custom checks	Built-in expect
Auto-waiting assertions	Not built in	Yes
Cleanup	Manual	Automatic for built-in fixtures
Test runner	Not included	Included
Reports	Need to manage separately	Built-in reporting support
Parallel execution	Need to implement/manage	Supported
Retries	Need to implement/manage	Supported

15. Quick Revision – One-Line Answers

1. Playwright Library: Provides APIs to automate browsers.
2. Playwright Test: Browser automation plus test runner.
3. Browser: Actual browser instance.
4. Browser Context: Isolated browser session.
5. Page: Individual browser tab.
6. Headless: Browser runs without displaying UI.
7. Headed: Browser runs with visible UI.
8. slowMo: Slows down browser operations.
9. Codegen: Records interactions and generates code.
10. assert: Node.js assertion module.
11. expect: Playwright Test assertion API.
12. IIFE: Immediately Invoked Function Expression.
13. async: Declares an asynchronous function.
14. await: Waits for a Promise to settle.
15. Screenshot: Captures the current browser page.

16. Interview Preparation – Priority Topics

High priority

Make sure you can confidently explain these without looking at notes:

- Difference between Playwright Library and Playwright Test.
- Browser vs Browser Context vs Page.
- `async` and `await`.
- Headless vs Headed mode.

- `slowMo`.
- Codegen.
- `assert` vs `expect`.
- Browser installation.
- Mobile device emulation.
- Manual vs automatic cleanup.

My suggestion: Since you are preparing for Selenium + Playwright QA Automation interviews, understand the Library vs Test Runner difference particularly well. Interviewers often check whether you understand how Playwright manages browser sessions, test execution and assertions.

You can continue sending the next documentation section, and I'll maintain this same format with explanations, code examples and detailed interview questions.

Continuing with the complete Playwright Accessibility Testing explanation, including all the code examples from your documentation and interview questions.

1. Introduction – What is Accessibility Testing?

Accessibility testing means checking whether a website or application can be used by everyone, including people with disabilities.

For example:

- A visually impaired user may use a screen reader.
- A user with low vision may need readable text and sufficient color contrast.
- A user who cannot use a mouse may navigate using a keyboard.
- A user with hearing impairment may need captions for videos.

Common accessibility issues

Issue	Example
Poor color contrast	Light grey text on a white background
Missing labels	A username input without an accessible label
Duplicate IDs	Two HTML elements with the same ID
Missing accessible names	A button that a screen reader cannot identify

Why is it important for QA?

It helps us find accessibility defects, improve usability, and check whether the application follows accessibility standards.

2. What is axe-core?

Your documentation uses the `@axe-core/playwright` package.

- axe-core: An accessibility testing engine.
- @axe-core/playwright: Integrates axe-core into Playwright tests.
- Accessibility scan: Checks a webpage for automatically detectable accessibility violations.

Install it:

```
npm install -D @axe-core/playwright
```

Import it:

```
import AxeBuilder from '@axe-core/playwright';
```

One important point: automated accessibility testing cannot detect every accessibility issue. We should combine automated scans with manual testing and inclusive user testing.

3. Scanning an Entire Page

This is the first and most important example.

Requirement: Open a webpage and check whether it has automatically detectable accessibility violations.

Code

```
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test.describe('homepage', () => {

  test('should not have any automatically detectable accessibility
issues', async ({ page }) => {

    await page.goto('https://your-site.com/');

    const accessibilityScanResults =
      await new AxeBuilder({ page }).analyze();

    expect(accessibilityScanResults.violations).toEqual([]);

  });
});
```

Explanation line by line

1. Import Playwright functions

```
import { test, expect } from '@playwright/test';
```

- `test` – Creates and executes a test case.
- `expect` – Checks whether the actual result matches the expected result.

2. Import AxeBuilder

```
import AxeBuilder from '@axe-core/playwright';
```

Imports the axe accessibility testing tool.

3. Create a test group

```
test.describe('homepage', () => {
```

`test.describe()` groups related test cases.

Here, the test belongs to the homepage test group.

4. Navigate to the website

```
await page.goto('https://your-site.com/');
```

Opens the application homepage.

5. Run the accessibility scan

```
const accessibilityScanResults =
  await new AxeBuilder({ page }).analyze();
```

This is the most important line.

- `new AxeBuilder({ page })` – Creates an axe scanner for the current Playwright page.
- `.analyze()` – Runs the accessibility scan.
- `await` – Waits until the scan finishes.
- `accessibilityScanResults` – Stores the scan results.

6. Verify the violations

```
expect(accessibilityScanResults.violations).toEqual([]);
```

Checks whether the violations array is empty.

Result	Meaning
[ ]	No automatically detected violations
Contains violation objects	Accessibility issues were detected

Important: This does not prove that the entire website is accessible. It only means that this scan found no violations under the configured rules.

4. Scanning a Specific Part of a Page

Sometimes we don't want to scan the entire webpage. We only want to scan a particular component.

For example:

- Navigation menu
- Login form
- Registration form
- Shopping cart
- Footer

For this, we use `AxeBuilder.include()`.

Code

```
test('navigation menu accessibility test', async ({ page }) => {

  await page.goto('https://your-site.com/');

  await page.getByRole('button', { name: 'Navigation
Menu' }).click();

  await page.locator('#navigation-menu-flyout').waitFor();

  const accessibilityScanResults = await new AxeBuilder({ page })
    .include('#navigation-menu-flyout')
    .analyze();

  expect(accessibilityScanResults.violations).toEqual([]);

});
```

Explanation

Step 1: Open the website

```
await page.goto('https://your-site.com/');
```

Step 2: Click the navigation button

```
await page.getByRole('button', { name: 'Navigation Menu' }).click();
```

Finds the button by its accessible role and name, then clicks it.

Step 3: Wait for the menu

```
await page.locator('#navigation-menu-flyout').waitFor();
```

Waits until the navigation menu element is available in the DOM.

Why is this important?

The navigation menu may appear only after clicking the button. We must wait until the page reaches the required state before scanning.

Step 4: Scan only the navigation menu

```
.include('#navigation-menu-flyout')
```

`include()` tells axe to scan only the selected element and its relevant content.

Step 5: Run the scan

```
.analyze();
```

Runs the accessibility scan on that selected section.

Important method: `include()`

Method	Purpose
`include()`	Scan only a specific part of a page
`analyze()`	Execute the accessibility scan

Interview point: If an element is hidden initially and appears after a user interaction, first perform the interaction and wait for the element to reach the desired state. Then call `analyze()`.

5. Scanning for WCAG Violations

What is WCAG?

WCAG stands for Web Content Accessibility Guidelines.

It provides guidelines for making web content accessible to people with disabilities.

WCAG has three conformance levels:

Level	Meaning
A	Basic accessibility requirements
AA	More comprehensive requirements; commonly targeted by organizations
AAA	Highest and most demanding level

For example, WCAG includes guidance about:

- Color contrast.
- Keyboard accessibility.
- Text alternatives for images.
- Form labels.
- Readable and understandable content.

What is `withTags()`?

By default, axe checks many accessibility rules, including some best-practice rules.

If we want to scan rules associated with particular WCAG criteria, we can use `withTags()`.

Code

```
test('should not have WCAG A or AA violations', async ({ page }) =>
{

  await page.goto('https://your-site.com/');

  const accessibilityScanResults = await new AxeBuilder({ page })
    .withTags([
      'wcag2a',
      'wcag2aa',
      'wcag21a',
      'wcag21aa'
    ])
    .analyze();

  expect(accessibilityScanResults.violations).toEqual([]);

});
```

Explanation

```
.withTags([
  'wcag2a',
  'wcag2aa',
  'wcag21a',
  'wcag21aa'
])
```

This selects axe rules tagged for:

- `wcag2a` – WCAG 2.0 Level A.
- `wcag2aa` – WCAG 2.0 Level AA.
- `wcag21a` – WCAG 2.1 Level A.
- `wcag21aa` – WCAG 2.1 Level AA.

Why do we use it?

To restrict the accessibility scan to rules associated with the selected WCAG criteria.

Remember: Automated scans cannot detect every WCAG violation, even when all these tags are selected.

6. Handling Known Accessibility Issues

Suppose we run an accessibility scan and find 10 violations.

Some are existing issues that the development team already knows about and has not fixed yet.

What should we do?

The documentation explains three approaches:

1. `exclude()` – Exclude specific elements.
2. `disableRules()` – Disable specific accessibility rules.
3. Snapshots – Track known violations using fingerprints.

A. Excluding individual elements using `exclude()`

Suppose a particular footer component has a known accessibility issue.

We want to scan the rest of the page but temporarily skip that component.

Code

```
test('accessibility test excluding known issue', async ({ page }) =>
{

  await page.goto('https://your-site.com/page-with-known-issues');

  const accessibilityScanResults = await new AxeBuilder({ page })
    .exclude('#element-with-known-issue')
    .analyze();

  expect(accessibilityScanResults.violations).toEqual([]);

});
```

Explanation

`.exclude('#element-with-known-issue')`

Excludes the selected element from the scan.

Important disadvantages:

- It excludes the element and all its descendants.
- All accessibility rules are skipped for that element, not just the known issue.

For this reason, avoid excluding large components containing many child elements.

B. Disabling individual rules using `disableRules()`

Suppose many pages have the same known duplicate-ID issue.

Instead of excluding multiple elements, we can temporarily disable the specific rule.

Code

```
test('accessibility test disabling known rule', async ({ page }) =>
{

  await page.goto('https://your-site.com/page-with-known-issues');

  const accessibilityScanResults = await new AxeBuilder({ page })
    .disableRules(['duplicate-id'])
    .analyze();

  expect(accessibilityScanResults.violations).toEqual([]);

});
```

Explanation

`.disableRules(['duplicate-id'])`

Temporarily disables the axe rule identified by `duplicate-id`.

The rule ID can be found in the `id` property of a violation returned by axe.

Difference between exclude and disableRules:

`exclude()`	`disableRules()`
Excludes specific elements	Disables specific rules
Skips all rules on those elements	Skips the selected rule across the scan
Useful for a known problematic component	Useful when one rule causes many known violations

Interview example:

If one footer has an accessibility problem, use `exclude()` when temporarily necessary.

If duplicate IDs are a known issue across many pages, `disableRules(['duplicate-id'])` can temporarily suppress that rule.

Both are temporary workarounds. The actual accessibility defects should still be tracked and fixed.

7. Using Snapshots to Allow Specific Known Issues

Sometimes we don't want to exclude an entire element or disable a complete accessibility rule.

Instead, we want to allow a specific set of existing violations while ensuring that no new violations are introduced.

For this, we can use Playwright snapshots with violation fingerprints.

What is a snapshot?

A snapshot stores an expected result. During later test runs, Playwright compares the current result with the saved snapshot.

If the result changes, the snapshot test can fail.

Why shouldn't we snapshot the entire violations array?

The complete violations array contains implementation details, including rendered HTML snippets.

Even if the actual accessibility issue hasn't changed, an unrelated HTML change could make the snapshot fail.

The documentation recommends creating a smaller representation called a fingerprint.

Avoid this approach

```
expect(accessibilityScanResults.violations).toMatchSnapshot();
```

This can be fragile because it stores too much implementation detail.

Recommended approach

```
expect(
  violationFingerprints(accessibilityScanResults)
).toMatchSnapshot();
```

Create a helper function:

```
function violationFingerprints(accessibilityScanResults) {

  const violationFingerprints =
    accessibilityScanResults.violations.map(violation => ({
      rule: violation.id,

      targets: violation.nodes.map(node => node.target),
    }));

  return JSON.stringify(violationFingerprints, null, 2);
}
```

Explanation

1. Map through all violations

```
accessibilityScanResults.violations.map(violation => ({
```

Loops through each accessibility violation.

2. Store the rule ID

`rule: violation.id,`

Stores the identifier of the accessibility rule that failed.

Example: `duplicate-id`.

3. Store the affected elements

`targets: violation.nodes.map(node => node.target),`

Stores the CSS selectors identifying elements affected by that violation.

4. Convert the result into JSON

`JSON.stringify(violationFingerprints, null, 2);`

Converts the fingerprint data into a formatted JSON string.

5. Compare it with the snapshot

`.toMatchSnapshot();`

Checks whether the current fingerprint matches the stored snapshot.

Example of a fingerprint

```
[
  {
    "rule": "duplicate-id",
    "targets": [
      ["#login-button"],
      ["#submit-button"]
    ]
  }
]
```

This is an illustrative example of the fingerprint structure.

Remember: Snapshots are useful when we want to track known violations without storing the entire scan result. They do not fix the accessibility problems.

8. Exporting Scan Results as a Test Attachment

When we run an accessibility scan, axe returns more information than just violations.

The results can contain:

- `violations` – Accessibility issues detected.
- `passes` – Rules that passed.
- `incomplete` – Rules for which axe could not determine a definite result and which may need manual review.
- `inapplicable` – Rules that did not apply to the scanned page.

Sometimes, we need the complete report for debugging.

For this, Playwright provides:

`testInfo.attach()`

Code

```
test('example with attachment', async ({ page }, testInfo) => {

  await page.goto('https://your-site.com/');

  const accessibilityScanResults =
    await new AxeBuilder({ page }).analyze();

  await testInfo.attach('accessibility-scan-results', {
    body: JSON.stringify(accessibilityScanResults, null, 2),
    contentType: 'application/json'
  });

  expect(accessibilityScanResults.violations).toEqual([]);

});
```

Explanation

1. Accept `testInfo`

```
async ({ page }, testInfo) => {
```

- `page` – Playwright page fixture.
- `testInfo` – Provides information and APIs related to the currently running test.

2. Run the accessibility scan

```
const accessibilityScanResults =
  await new AxeBuilder({ page }).analyze();
```

Stores the complete axe scan results.

3. Attach the report

```
await testInfo.attach('accessibility-scan-results', {
```

Attaches the report to the test result.

4. Convert results into JSON

```
body: JSON.stringify(accessibilityScanResults, null, 2),
```

Converts the complete scan results into readable JSON.

5. Specify the content type

```
contentType: 'application/json'
```

Tells Playwright that the attachment contains JSON data.

6. Verify accessibility violations

```
expect(accessibilityScanResults.violations).toEqual([]);
```

Fails the test if violations are found.

Why do we use attachments?

Suppose an accessibility test fails in Jenkins or CI.

The attached JSON report helps the QA team investigate:

- Which rules failed?
- Which elements were affected?
- Which rules passed?
- Which results need further investigation?

Interview point: `testInfo.attach()` is used to attach additional files or data to a Playwright test result.

9. Using a Test Fixture for Common axe Configuration

Imagine we have 20 test files.

Each test file needs the same accessibility configuration:

- Same WCAG tags.
- Same excluded element.
- Same axe setup.

Should we write the same configuration in all 20 files?

No. We can create a reusable custom fixture.

What is a fixture?

A fixture is a reusable setup mechanism in Playwright.

It provides the required resources or configuration to a test.

Examples of built-in fixtures:

- `page`
- `browser`
- `context`

We can also create custom fixtures.

A. Creating a custom accessibility fixture

Create a file named:

`axe-test.ts`

Code

```
import { test as base } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

type AxeFixture = {
  makeAxeBuilder: () => AxeBuilder;
};

export const test = base.extend<AxeFixture>({

  makeAxeBuilder: async ({ page }, use) => {

    const makeAxeBuilder = () => new AxeBuilder({ page })
      .withTags([
        'wcag2a',
        'wcag2aa',
        'wcag21a',
        'wcag21aa'
      ])
      .exclude('#commonly-reused-element-with-known-issue');

    await use(makeAxeBuilder);
  }

});

export { expect } from '@playwright/test';
```

Explanation

1. Import the base test

```
import { test as base } from '@playwright/test';
```

Imports Playwright's original test function and renames it to `base`.

We will extend this base test with our custom fixture.

2. Import AxeBuilder

```
import AxeBuilder from '@axe-core/playwright';
```

Imports the accessibility scanner.

3. Define the fixture type

```
type AxeFixture = {
  makeAxeBuilder: () => AxeBuilder;
};
```

Defines a TypeScript type for the custom fixture.

It tells TypeScript that `makeAxeBuilder` is a function that returns an `AxeBuilder` instance.

4. Extend the base test

```
export const test = base.extend<AxeFixture>({
```

Creates a new test function with the additional fixture named `makeAxeBuilder`.

5. Create the fixture

```
makeAxeBuilder: async ({ page }, use) => {
```

Receives the Playwright `page` fixture and provides the custom fixture through `use`.

6. Configure AxeBuilder

```
const makeAxeBuilder = () => new AxeBuilder({ page })
  .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
  .exclude('#commonly-reused-element-with-known-issue');
```

Creates an axe builder with common configuration.

Every test using this fixture gets:

- The selected WCAG rules.
- The commonly excluded element.

7. Provide the fixture

```
await use(makeAxeBuilder);
```

Makes the configured builder available to the test.

8. Export expect

```
export { expect } from '@playwright/test';
```

Allows other test files to import `expect` from the same fixture file.

B. Using the custom fixture

Now create another test file.

```
import { test, expect } from './axe-test';

test('example using custom fixture', async ({
  page,
  makeAxeBuilder
}) => {

  await page.goto('https://your-site.com/');

  const accessibilityScanResults = await makeAxeBuilder()
    .include('#specific-element-under-test')
    .analyze();

  expect(accessibilityScanResults.violations).toEqual([]);

});
```

Explanation

```
import { test, expect } from './axe-test';
```

Imports the custom test and assertion function from our fixture file.

```
async ({ page, makeAxeBuilder }) => {
```

The test receives:

- `page` – To interact with the webpage.
- `makeAxeBuilder` – Our custom accessibility fixture.

`await makeAxeBuilder()`

Creates an AxeBuilder using the shared configuration.

`.include('#specific-element-under-test')`

Adds an extra configuration for this particular test.

`.analyze();`

Executes the scan.

Why use fixtures?

Without fixture	With fixture
Repeat configuration in every test	Share common configuration
More duplicate code	Less duplicate code
Changes needed in multiple files	Update common configuration once
Difficult to maintain	Easier to maintain

Important: The fixture is a factory function here. Calling `makeAxeBuilder()` creates a configured builder for the current page. The test can add its own options before calling `analyze()`.

10. Important AxeBuilder Methods – Quick Revision

These are the main methods from your documentation.

Method	Purpose	Example
`analyze()`	Runs accessibility scan	`.analyze()`

`include()`	Scans selected element(s)	`.include('#login')`
`exclude()`	Excludes selected element(s)	`.exclude('#footer')`
`withTags()`	Selects accessibility rule tags	`.withTags(['wcag2a'])`
`disableRules()`	Disables selected axe rules	`.disableRules(['duplicate-id'])`

Other important Playwright methods used in these examples:

Method	Purpose
`page.goto()`	Navigates to a webpage
`getByRole()`	Finds an element by accessible role
`click()`	Clicks an element
`locator()`	Locates an element
`waitFor()`	Waits for an element to reach the requested state
`expect()`	Verifies expected results
`toEqual([])`	Checks that an array is empty
`toMatchSnapshot()`	Compares actual output with a stored snapshot
`testInfo.attach()`	Attaches a report to test results

11. Real-time QA Automation Example

Suppose you are working on an e-commerce application.

You need to test the accessibility of the login page.

The login page contains:

- Email field.
- Password field.
- Login button.
- Forgot password link.

You can automate the accessibility scan like this:

```
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test('Login page accessibility test', async ({ page }) => {

  await page.goto('https://your-site.com/login');

  const results = await new AxeBuilder({ page })
    .withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
    .analyze();

  console.log('Accessibility violations:', results.violations);

  expect(results.violations).toEqual([]);

});
```

Expected behaviour:

- If no automatically detectable violations are found, the assertion passes.
- If violations are found, the assertion fails.
- QA can investigate the violation details and report defects to developers.

Note that this example scans the whole login page. It does not replace manual keyboard or screen-reader testing.

12. Important Playwright Accessibility Interview Questions and Answers

Basic interview questions

Q1. What is accessibility testing?

Answer: Accessibility testing is the process of checking whether a website or application can be used by people with disabilities, including visual, hearing, motor, or cognitive impairments.

It checks things like:

- Color contrast.
- Accessible labels.
- Keyboard navigation.
- Screen reader compatibility.
- Alternative text for images.

Q2. What is axe-core?

Answer: axe-core is an open-source accessibility testing engine that identifies common accessibility violations in web applications.

We can integrate it with Playwright using the `@axe-core/playwright` package.

Q3. How do you perform accessibility testing in Playwright?

Answer: We install `@axe-core/playwright`, import `AxeBuilder`, create an instance using the Playwright page, and call `analyze()`.

Example:

```
const results = await new AxeBuilder({ page }).analyze();

expect(results.violations).toEqual([]);
```

This checks that the scan returned no automatically detectable violations.

Q4. What is WCAG?

Answer: WCAG stands for Web Content Accessibility Guidelines. These guidelines explain how to make web content accessible to people with disabilities.

It has three conformance levels:

- A – Basic requirements.
- AA – More comprehensive requirements.
- AAA – Highest and most demanding requirements.

Q5. Can automated accessibility testing detect all accessibility issues?

Answer: No. Automated testing detects common issues such as missing labels, invalid attributes, and certain color contrast problems.

However, some issues require manual testing, such as whether keyboard navigation is logical or whether screen reader instructions make sense.

We should combine automated, manual, and inclusive user testing.

Intermediate interview questions

Q6. What is the purpose of `AxeBuilder.analyze()`?

Answer: It executes the accessibility scan on the current state of the webpage and returns the scan results, including violations, passes, incomplete checks, and inapplicable rules.

Q7. What is the difference between `include()` and `exclude()`?

Answer:

- `include()` restricts the scan to the selected part of the page.
- `exclude()` skips the selected element and its descendants from the scan.

For example, we can use `include('#login-form')` to scan only the login form.

Q8. What is the difference between `exclude()` and `disableRules()`?

Answer:

- `exclude()` skips all accessibility rules for selected elements.
- `disableRules()` skips selected accessibility rules across the scan.

For example, `disableRules(['duplicate-id'])` disables that specific rule.

Q9. What is `withTags()`?

Answer: `withTags()` is used to select which accessibility rule tags axe should check.

For example:

```
.withTags(['wcag2a', 'wcag2aa', 'wcag21a', 'wcag21aa'])
```

This selects rules tagged for WCAG 2.0 and WCAG 2.1 Level A and AA.

Q10. What is `testInfo.attach()`?

Answer: `testInfo.attach()` attaches additional files or data to Playwright test results.

In accessibility testing, we can attach the complete axe scan report in JSON format for debugging and reporting.

Q11. Why do we use fixtures in accessibility testing?

Answer: Fixtures help us reuse common axe configuration across multiple test files.

For example, we can define common WCAG tags and exclude a known problematic component in one fixture, instead of repeating the same configuration in every test.

Q12. What is a violation fingerprint?

Answer: A violation fingerprint is a simplified representation of an accessibility issue. It can contain the rule ID and CSS selectors of the affected elements.

We use fingerprints with snapshots to track known issues without snapshotting the entire axe violations array.

Scenario-based interview questions

Q13. You have a webpage with 10 accessibility violations. What will you do?

Answer:

1. Run the axe accessibility scan.
2. Check the `violations` array.
3. Identify the affected elements and rule IDs.
4. Report the issues to developers.
5. Retest after fixes.
6. If some issues are already known and cannot be fixed immediately, use a controlled temporary suppression approach, such as `exclude()` or `disableRules()`.
7. Track suppressed issues so they are not forgotten.

Q14. A navigation menu is hidden initially and appears after clicking a button. How will you test its accessibility?

Answer: First, navigate to the page and click the navigation button. Then wait for the menu to appear. Finally, use `include()` to scan the navigation menu.

```
await page.getByRole('button', { name: 'Navigation Menu' }).click();

await page.locator('#navigation-menu-flyout').waitFor();

const results = await new AxeBuilder({ page })
  .include('#navigation-menu-flyout')
  .analyze();

expect(results.violations).toEqual([]);
```

Q15. Your application has the same accessibility issue on multiple pages. How will you manage it?

Answer: I would create a reusable Playwright fixture with common AxeBuilder configuration. This avoids duplicating the same setup in multiple test files.

Q16. How will you debug an accessibility test failure in CI?

Answer: I would inspect the violation details, identify the failed rule and affected elements, and attach the full axe scan report using `testInfo.attach()`. I would then investigate the issue and share the findings with developers.

Q17. What happens if `accessibilityScanResults.violations` is not empty?

Answer: The assertion below fails because the actual array does not match the expected empty array.

```
expect(accessibilityScanResults.violations).toEqual([]);
```

The test report will show the failed assertion. We can inspect the violation details to identify the accessibility problem.

13. Practical Interview Coding Questions

Try to understand these examples because interviewers may ask you to write or explain them.

1. Write a test to scan a complete webpage.

```
const results = await new AxeBuilder({ page }).analyze();

expect(results.violations).toEqual([]);
```

2. Write a test to scan only the login form.

```
const results = await new AxeBuilder({ page })
  .include('#login-form')
  .analyze();

expect(results.violations).toEqual([]);
```

3. Write a test to exclude the footer.

```
const results = await new AxeBuilder({ page })
  .exclude('#footer')
  .analyze();
```

4. Disable the duplicate-ID rule.

```
const results = await new AxeBuilder({ page })
  .disableRules(['duplicate-id'])
  .analyze();
```

5. Scan for WCAG A and AA rules.

```
const results = await new AxeBuilder({ page })
  .withTags([
    'wcag2a',
    'wcag2aa',
    'wcag21a',
    'wcag21aa'
  ])
  .analyze();
```

14. Quick Revision – Must Remember

Topic	Remember this
Accessibility testing	Checks whether everyone can use the application
axe-core	Accessibility testing engine
`@axe-core/playwright`	Integrates axe with Playwright
`AxeBuilder`	Creates an accessibility scanner
`analyze()`	Runs the scan
`violations`	Detected accessibility issues
`include()`	Scans a selected section
`exclude()`	Skips selected elements
`withTags()`	Selects accessibility rule tags

`disableRules()`	Disables selected rules
Snapshot	Compares current results with stored results
Fingerprint	Simplified identification of known violations
`testInfo.attach()`	Attaches the scan report
Fixture	Reuses common configuration

Interview priority
Focus especially on `analyze()`, `include()`, `exclude()`, `withTags()`, `disableRules()`, WCAG, and the difference between automated and manual accessibility testing.

Next step: I can prepare a set of Playwright interview questions from this accessibility topic, with answers and coding practice.

Playwright – Actions

This is one of the most important Playwright topics for QA Automation interviews.

Actions are operations that Playwright performs on webpage elements, such as entering text, clicking buttons, selecting checkboxes, uploading files, pressing keyboard keys, dragging elements, and scrolling.

For example, in an e-commerce application, a QA automation script may:

1. Enter a username and password.
2. Click the Login button.
3. Select a product size.
4. Add the product to the cart.
5. Upload a profile picture.
6. Scroll down to view product details.

Playwright provides built-in locator methods to perform these actions.

1. Text Input – `fill()`

`locator.fill()` is used to enter text into input fields, text areas, and content-editable elements.

It is generally the easiest and recommended method for filling form fields.

Syntax

```
await locator.fill('text');
```

Example 1: Normal text input

```
await page.getByRole('textbox').fill('Peter');
```

Explanation:

- `getByRole('textbox')` locates a textbox.
- `fill('Peter')` enters the text Peter.
- `await` waits for the operation to complete.

Example 2: Date input

```
await page.getByLabel('Birth date').fill('2020-02-02');
```

Enters the date into the Birth date field.

Example 3: Time input

```
await page.getByLabel('Appointment time').fill('13:15');
```

Enters 1:15 PM in the time field.

Example 4: Local datetime input

```
await page.getByLabel('Local time').fill('2020-03-02T05:15');
```

Enters a local date and time.

Important points

Method	Purpose
`fill()`	Enters text into a field
`getByRole()`	Locates elements using their accessible role
`getByLabel()`	Locates form controls using their associated label

Interview question: What is the difference between `fill()` and `pressSequentially()`?

- `fill()` enters the complete text directly and triggers an input event.
- `pressSequentially()` types the text character by character and emits keyboard events.

Use `fill()` for normal form filling. Use `pressSequentially()` when the application has special keyboard handling.

2. Checkboxes and Radio Buttons

Playwright provides methods to select and deselect checkboxes and radio buttons.

Important methods:

- `check()`
- `uncheck()`
- `setChecked()`
- `isChecked()`
- `toBeChecked()`

A. Check a checkbox

```
await page.getByLabel('I agree to the terms above').check();
```

This selects the checkbox.

B. Verify checkbox state

```
await expect(
  page.getByLabel('Subscribe to newsletter')
).toBeChecked();
```

Checks whether the checkbox is selected.

C. Select a radio button

```
await page.getByLabel('XL').check();
```

Selects the XL radio button.

Difference between checkbox and radio button

Checkbox

Multiple options can be selected independently.

Playwright – Assertions

Assertions are one of the most important topics in Playwright interviews, especially for QA Automation and SDET roles.

1. What are Assertions?

Assertions are used to verify whether the actual result matches the expected result.

In simple words, after performing an action, we check whether the application is behaving correctly.

Real-time example: Login page

Suppose you are testing a login page.

1. Enter username.
2. Enter password.
3. Click Login.

4. Verify that the dashboard is displayed.

The first three steps are actions. The fourth step is an assertion.

```
import { test, expect } from '@playwright/test';

test('Login functionality', async ({ page }) => {
  await page.goto('https://example.com/login');

  await page.getByLabel('Username').fill('komal');
  await page.getByLabel('Password').fill('password123');

  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page.getByText('Dashboard')).toBeVisible();
});
```

Here:

- `expect()` starts an assertion.
- `toBeVisible()` checks whether the Dashboard is visible.
- If the Dashboard appears, the assertion passes.
- If it does not appear within the timeout, the assertion fails.

2. Types of Assertions in Playwright

Playwright assertions can be divided into three main categories:

1. Auto-retrying assertions

Recommended for web elements

Automatically checks the condition repeatedly until it passes or times out.

2. Non-retrying assertions

No automatic retry

Checks the condition once and immediately passes or fails.

3. Soft assertions

Continue execution

Records assertion failures but allows the remaining test steps to execute.

Let's understand each one.

3. Generic Assertions (Non-retrying)

Generic assertions are used to verify JavaScript values, strings, numbers, arrays, and objects.

Syntax

```
expect(actualValue).matcher(expectedValue);
```

Example

```
const success = true;

expect(success).toBeTruthy();
```

Explanation:

- `success` is the actual value.
- `toBeTruthy()` checks whether the value is truthy.
- Since `success` is `true`, the assertion passes.

Important generic matchers

Matcher	Purpose	Example
`toBe()`	Checks exact value/reference equality	`expect(10).toBe(10)`
`toEqual()`	Checks deep equality	`expect({a:1}).toEqual({a:1})`

`toStrictEqual()`	Checks deep equality including undefined properties and types	`expect({a:1}).toStrictEqual({a:1})`
`toBeTruthy()`	Checks truthy value	`expect(true).toBeTruthy()`
`toBeFalsy()`	Checks falsy value	`expect(false).toBeFalsy()`
`toBeNull()`	Checks null	`expect(null).toBeNull()`
`toBeUndefined()`	Checks undefined	`expect(undefined).toBeUndefined()`
`toContain()`	Checks whether a string/array contains a value	`expect('Playwright').toContain('Play')`
`toHaveLength()`	Checks length	`expect([1,2,3]).toHaveLength(3)`
`toBeGreaterThan()`	Checks greater value	`expect(10).toBeGreaterThan(5)`
`toMatch()`	Checks regular expression	`expect('Komal').toMatch(/Kom/)`
`toHaveProperty()`	Checks object property	`expect(user).toHaveProperty('name')`

Important interview question: Difference between `toBe()` and `toEqual()`

`toBe()` checks whether two values are identical using JavaScript's `Object.is()` comparison.

`toEqual()` checks whether objects or arrays have the same structure and values (deep equality).

```
expect(10).toBe(10);
```

```
expect({ name: 'Komal' }).toEqual({ name: 'Komal' });
```

For objects with the same properties and values, `toEqual()` is usually the appropriate choice.

4. Auto-retrying Assertions

Very important for interviews

Auto-retrying assertions are Playwright's web-specific assertions.

They repeatedly check the condition until:

- The assertion passes, or
- The assertion timeout is reached.

The default assertion timeout is 5 seconds.

Example

```
await expect(page.getByTestId('status'))
  .toHaveText('Submitted');
```

Suppose the application takes 3 seconds to display the Submitted status.

Playwright does not immediately fail. It keeps checking until the expected text appears or the timeout is reached.

How auto-retrying works

Start assertion

```
expect(locator).toHaveText('Submitted')
```

Check actual text

Matches? Yes

Assertion passes

Matches? No

Retry until timeout

Important auto-retrying matchers

Matcher	What it verifies
`toBeVisible()`	Element is visible
`toBeHidden()`	Element is hidden or not visible
`toBeAttached()`	Element is attached to DOM
`toBeEnabled()`	Element is enabled
`toBeDisabled()`	Element is disabled
`toBeChecked()`	Checkbox/radio is checked
`toBeEditable()`	Element is editable
`toBeFocused()`	Element has focus
`toHaveText()`	Element text matches
`toContainText()`	Element contains expected text

`toHaveValue()`	Input has expected value
`toHaveAttribute()`	Element has expected attribute
`toHaveClass()`	Element has expected class
`toHaveCount()`	Locator matches expected number of elements
`toHaveURL()`	Page has expected URL
`toHaveTitle()`	Page has expected title
`toHaveCSS()`	Element has expected CSS property
`toBeInViewport()`	Element intersects the viewport
`toBeOK()`	API response has an OK status

Practical examples

```
// Verify button is visible
await expect(page.getByRole('button', { name: 'Login' }))
  .toBeVisible();

// Verify checkbox is selected
await expect(page.getByLabel('Accept Terms'))
  .toBeChecked();

// Verify input value
await expect(page.getByLabel('Username'))
  .toHaveValue('Komal');

// Verify page title
await expect(page).toHaveTitle('Dashboard');

// Verify URL
```

```
await expect(page).toHaveURL(/dashboard/);

// Verify number of products
await expect(page.locator('.product'))
  .toHaveCount(5);
```

Important: Always use `await` with Playwright's asynchronous locator and page assertions.

5. Non-retrying Assertions vs Auto-retrying Assertions

This is a frequently asked interview question.

Auto-retrying	Non-retrying
Checks condition repeatedly	Checks condition once
Waits until success or timeout	Does not wait for the condition
Mainly used for webpage elements	Mainly used for JavaScript values
Must use `await`	Usually does not require `await`
Example: `toBeVisible()`	Example: `toEqual()`

Example: Non-retrying

```
const count = 5;

expect(count).toBe(5);
```

It checks the value immediately.

Example: Auto-retrying

```
await expect(page.getByText('Welcome'))
  .toBeVisible();
```

It keeps checking whether the Welcome text becomes visible.

Interview question: Why should we prefer auto-retrying assertions?

Answer: Web applications are dynamic. Elements and data may load asynchronously. Auto-retrying assertions wait for the expected condition, reducing flaky test failures caused by timing issues.

6. Asymmetric Matchers

Asymmetric matchers are used when we do not want to compare an entire value exactly. Instead, we want to check whether it satisfies a particular condition.

Important asymmetric matchers

Matcher	Meaning
`expect.any()`	Matches any value of a specified type/class
`expect.anything()`	Matches any value except `null` and `undefined`
`expect.arrayContaining()`	Checks whether an array contains specified elements
`expect.arrayOf()`	Checks whether array elements match a specified matcher
`expect.objectContaining()`	Checks whether an object contains specified properties
`expect.stringContaining()`	Checks whether a string contains specific text
`expect.stringMatching()`	Checks whether a string matches a pattern
`expect.closeTo()`	Matches an approximately equal number

Example 1: `objectContaining()`

```
const user = {
  name: 'Komal',
  age: 25,
  city: 'Pune'
};

expect(user).toEqual(
  expect.objectContaining({
    name: 'Komal'
  })
);
```

It checks that the object contains the `name` property with value `Komal`. Other properties are allowed.

Example 2: `arrayContaining()`

```
const colors = ['red', 'green', 'blue'];

expect(colors).toEqual(
  expect.arrayContaining(['red', 'blue'])
);
```

It verifies that the array contains both red and blue.

7. Negating Matchers – `.not`

Sometimes we need to verify that something is NOT true.

Playwright provides `.not` before a matcher to check the opposite condition.

Example 1: Verify value is not zero

```
expect(10).not.toEqual(0);
```

This passes because 10 is not equal to 0.

Example 2: Verify text is not present

```
await expect(page.getByText('Error'))
  .not.toBeVisible();
```

This checks that the Error text is not visible.

Example 3: Verify button is not disabled

```
await expect(page.getByRole('button', { name: 'Submit' }))
  .not.toBeDisabled();
```

Interview question: What is the use of `.not` in Playwright?

`.not` is used to verify the opposite of the expected condition. For example, `toBeVisible()` checks visibility, whereas `not.toBeVisible()` checks that the element is not visible.

8. Soft Assertions – `expect.soft()`

Important interview topic

By default, when an assertion fails, Playwright stops executing the remaining steps in that test.

But sometimes we want to verify multiple things even if one assertion fails.

For this, Playwright provides soft assertions.

Normal assertion example

```
await expect(page.getByTestId('status'))
  .toHaveText('Success');

await expect(page.getByTestId('eta'))
  .toHaveText('1 day');

await page.getByRole('link', { name: 'Next page' }).click();
```

Suppose the first assertion fails.

The test stops at the first failure, so the second assertion and click will not execute.

Soft assertion example

```
await expect.soft(page.getByTestId('status'))
  .toHaveText('Success');
```

```
await expect.soft(page.getByTestId('eta'))
  .toHaveText('1 day');

await page.getByRole('link', { name: 'Next page' }).click();
```

Here:

- First assertion fails → failure is recorded.
- Second assertion is still executed.
- Next page link is clicked.
- Test continues, but its final status will be failed because a soft assertion failed.

Normal vs Soft assertions

Normal assertion	Soft assertion
Uses `expect()`	Uses `expect.soft()`
Stops test at failure	Continues test after failure
Useful for critical validations	Useful for checking multiple independent conditions
Failure is reported	Failure is also reported

Checking soft assertion failures

```
await expect.soft(page.getByTestId('status'))
  .toHaveText('Success');

await expect.soft(page.getByTestId('eta'))
  .toHaveText('1 day');

expect(test.info().errors).toHaveLength(0);
```

`test.info().errors` contains the errors recorded during the test.

`toHaveLength(0)` checks that there are no recorded errors.

If there are soft assertion failures, this final assertion also fails.

Important: Soft assertions are supported by the Playwright Test runner. They are not a way to make a failing test pass; they allow additional checks before the test is reported as failed.

Real-time QA example

Suppose you are testing an order confirmation page. You want to validate:

- Order status.
- Delivery date.
- Order number.
- Total amount.

You can use soft assertions to check all four fields in one test and see all the failures together.

9. Custom Expect Message

You can provide a meaningful message with an assertion to make test reports easier to understand.

Syntax

```
await expect(locator, 'custom message').toBeVisible();
```

Example

```
await expect(
  page.getByText('Name'),
  'Username should be displayed after login'
).toBeVisible();
```

If the assertion fails, the reporter displays the custom message along with the assertion details.

Why is it useful?

Imagine you have 50 assertions in your automation suite. A meaningful message helps you understand what the test was expecting without having to inspect every line of code.

You can also use a custom message with soft assertions:

```
await expect.soft(
  page.getByText('Success'),
  'Success message should be displayed'
).toBeVisible();
```

10. `expect.configure()`

`expect.configure()` allows you to create a pre-configured expect instance with your own default settings.

You can configure:

- Assertion timeout.
- Soft assertion behavior.

Example 1: Custom timeout

```
const slowExpect = expect.configure({
  timeout: 10000
});

await slowExpect(locator).toHaveText('Submit');
```

Explanation:

- `expect.configure()` creates a configured expect instance.
- `timeout: 10000` sets the assertion timeout to 10 seconds.
- `slowExpect()` uses this configuration for its assertions.

The default assertion timeout is 5 seconds. Here, we increased it to 10 seconds.

Example 2: Always use soft assertions

```
const softExpect = expect.configure({
  soft: true
});

await softExpect(locator).toHaveText('Submit');
```

This configured instance behaves like `expect.soft()`.

Difference between `expect.soft()` and `expect.configure()`

`expect.soft()`	`expect.configure()`
Makes a particular assertion soft	Creates a configured expect instance
Used directly for an assertion	Can be reused for multiple assertions
Example: `expect.soft(locator)`	Example: `expect.configure({soft:true})`

11. `expect.poll()`

Important for API testing

`expect.poll()` is used when we want to repeatedly check the result of a function until the expected condition is met.

It is especially useful for:

- API status verification.
- Background processing.
- Eventual consistency.
- Checking whether an order status changes.
- Waiting for asynchronous backend operations.

Example: Check API response status

```
await expect.poll(async () => {
  const response = await page.request.get(
    'https://api.example.com'
  );

  return response.status();

}, {
  message: 'Make sure API eventually succeeds',
  timeout: 10000
}).toBe(200);
```

Line-by-line explanation

1. expect.poll()

```
await expect.poll(async () => {
```

Starts polling. Playwright repeatedly executes the callback.

2. Send API request

```
const response = await page.request.get(
  'https://api.example.com'
);
```

Sends a GET request to the API.

3. Return response status

```
return response.status();
```

Returns the HTTP status code.

For example:

- 200 – OK.
- 404 – Not Found.
- 500 – Internal Server Error.

4. Configure timeout

```
timeout: 10000
```

Playwright keeps checking for up to 10 seconds.

5. Expected result

```
.toBe(200);
```

The polling assertion passes when the returned status becomes 200.

Custom polling intervals

```
await expect.poll(async () => {
  const response = await page.request.get(
    'https://api.example.com'
  );

  return response.status();

}, {
  intervals: [1000, 2000, 10000],
  timeout: 60000
}).toBe(200);
```

Here:

- `intervals` specifies the delay between polling attempts.
- `timeout` sets the maximum time to poll.

The intervals are used progressively. After the configured intervals are exhausted, the last interval is reused for further attempts.

`expect.poll()` with soft assertions

```
await expect.soft.poll(async () => {
  const response = await page.request.get(
    'https://api.example.com'
  );

  return response.status();

}).toBe(200);
```

This performs polling but records the failure as a soft assertion if the expected status is not reached.

12. `expect.toPass()`

`expect.toPass()` is used to retry an entire block of code until all assertions inside that block pass.

Example

```
await expect(async () => {
  const response = await page.request.get(
    'https://api.example.com'
  );

  expect(response.status()).toBe(200);

}).toPass();
```

How does it work?

1. Executes the callback.
2. Sends the API request.
3. Checks whether the response status is 200.
4. If the assertion fails, retries the entire callback.
5. Continues until it passes or the configured timeout is reached.

Custom timeout and intervals

```
await expect(async () => {
  const response = await page.request.get(
    'https://api.example.com'
  );

  expect(response.status()).toBe(200);

}).toPass({
  intervals: [1000, 2000, 10000],
```

```
  timeout: 60000
});
```

Important: According to the documentation, `toPass()` has a default timeout of `0` and does not use the normal configured expect timeout. Set its timeout explicitly when you need retries over a period of time.

Difference between `expect.poll()` and `expect.toPass()`

`expect.poll()`	`expect.toPass()`
Repeats a function and checks its returned value	Repeats an entire block of code
Used for polling a value	Used for retrying multiple operations/assertions
Matcher is applied to the returned value	Assertions are written inside the callback
Example: API status becomes 200	Example: API request and multiple validations succeed

Easy way to remember:

- `poll()` → Keep checking a value.
- `toPass()` → Keep retrying a block of code.

13. Custom Matchers – `expect.extend()`

Sometimes Playwright's built-in matchers are not sufficient for a particular business requirement.

In that case, we can create our own assertion using `expect.extend()`.

A custom matcher allows us to define our own validation logic and use it like a built-in matcher.

Real-time example

Suppose an e-commerce application has a cart with a custom HTML attribute:

```
<div class="cart" data-amount="4">
```

We want to verify that the cart contains 4 items using a custom matcher:

```
await expect(page.locator('.cart')).toHaveAmount(4);
```

Here, `toHaveAmount()` is not a built-in Playwright matcher. We create it ourselves.

Step 1: Create `fixtures.ts`

```
import { expect as baseExpect } from '@playwright/test';
import type { Locator } from '@playwright/test';

export { test } from '@playwright/test';

export const expect = baseExpect.extend({
  async toHaveAmount(
    locator: Locator,
    expected: number,
    options?: { timeout?: number }
  ) {
    const assertionName = 'toHaveAmount';

    let pass: boolean;
    let matcherResult: any;

    try {
      const expectation = this.isNot
        ? baseExpect(locator).not
        : baseExpect(locator);

      await expectation.toHaveAttribute(
        'data-amount',
        String(expected),
        options
      );

      pass = true;
    } catch (e: any) {
```

```
      matcherResult = e.matcherResult;
      pass = false;
    }

    if (this.isNot) {
      pass = !pass;
    }

    const message = () =>
      `Expected cart amount: ${expected}`;

    return {
      message,
      pass,
      name: assertionName,
      expected,
      actual: matcherResult?.actual,
    };
  },
});
```

Understand the important parts

Code	Explanation
`baseExpect.extend()`	Registers a custom matcher
`toHaveAmount()`	Name of our custom assertion
`locator`	Element on which assertion is performed
`expected`	Expected cart amount
`options`	Optional timeout configuration
`this.isNot`	Checks whether `.not` was used

`toHaveAttribute()`	Verifies the `data-amount` attribute
`pass`	Indicates whether assertion passed
`message`	Provides assertion failure information

The custom matcher returns an object containing the result and diagnostic information.

Note: The full documentation example includes more detailed formatting of the error message. The example above illustrates the core custom matcher structure.

Step 2: Use the custom matcher

Create `example.spec.ts`:

```
import { test, expect } from './fixtures';

test('verify cart amount', async ({ page }) => {
  await page.goto('https://example.com/cart');

  await expect(page.locator('.cart'))
    .toHaveAmount(4);
});
```

Now `toHaveAmount()` can be used like a normal Playwright assertion.

Interview question: Why do we use `expect.extend()`?

We use `expect.extend()` to create reusable custom assertions when built-in matchers do not cover our application's specific business requirements.

14. Playwright Expect vs Other Expect Libraries

Playwright provides its own `expect` function through:

```
import { expect } from '@playwright/test';
```

Do not confuse it with standalone assertion libraries that also export a function named `expect`.

Why should we use Playwright's own expect?

Playwright's expect is integrated with the Playwright Test runner and provides:

- Auto-retrying web assertions.
- Soft assertions.
- Assertion timeout configuration.
- Custom matchers.
- Test reporter integration.

For locator assertions, prefer Playwright's own `expect`.

15. Combining Custom Matchers – `mergeExpects()`

In larger automation frameworks, we may create custom matchers in different files or modules.

For example:

- Database-related assertions.
- Accessibility-related assertions.
- Application-specific assertions.

Instead of maintaining separate expect objects, Playwright provides `mergeExpects()`.

Example

```
import {
  mergeTests,
  mergeExpects
} from '@playwright/test';

import {
  test as dbTest,
  expect as dbExpect
} from 'database-test-utils';

import {
```

```
  test as a11yTest,
  expect as a11yExpect
} from 'a11y-test-utils';

export const expect = mergeExpects(
  dbExpect,
  a11yExpect
);

export const test = mergeTests(
  dbTest,
  a11yTest
);
```

Explanation

Method	Purpose
`mergeExpects()`	Combines custom assertion libraries
`mergeTests()`	Combines test objects and their fixtures

For example, you can now use a database custom matcher:

```
import { test, expect } from './fixtures';

test('verify database user', async ({ database }) => {
  await expect(database).toHaveDatabaseUser('admin');
});
```

Here, `toHaveDatabaseUser()` comes from the database assertion module.

When is this useful?

When working on a large automation framework where different teams maintain reusable test fixtures and custom assertions.

16. Important Assertions for Real-Time QA Automation

Suppose you are automating an e-commerce application.

Test scenario	Assertion
Verify login button appears	`toBeVisible()`
Verify login button is enabled	`toBeEnabled()`
Verify username is entered correctly	`toHaveValue()`
Verify welcome message	`toHaveText()`
Verify product list contains 5 items	`toHaveCount()`
Verify selected checkbox	`toBeChecked()`
Verify product has expected class	`toHaveClass()`
Verify product has expected ID	`toHaveId()`
Verify navigation URL	`toHaveURL()`
Verify page title	`toHaveTitle()`
Verify API response is successful	`toBeOK()`

Complete practical example

```
import { test, expect } from '@playwright/test';

test('E-commerce product validation', async ({ page }) => {

  await page.goto('https://example.com');

  // Verify page title
  await expect(page).toHaveTitle(/Example/);
```

```
  // Verify search box is visible
  await expect(page.getByRole('textbox'))
    .toBeVisible();

  // Search for a product
  await page.getByRole('textbox').fill('Laptop');

  // Verify entered value
  await expect(page.getByRole('textbox'))
    .toHaveValue('Laptop');

  // Click search button
  await page.getByRole('button', { name: 'Search' }).click();

  // Verify product appears
  await expect(page.getByText('Laptop'))
    .toBeVisible();

});
```

This example demonstrates how actions and assertions work together in a test.

17. Important Playwright Interview Questions and Answers

Basic interview questions

Q1. What is an assertion in Playwright?

An assertion is used to verify whether the actual application result matches the expected result. For example, after clicking Login, we can verify whether the Dashboard is visible using `toBeVisible()`.

Q2. What is the difference between actions and assertions?

- Actions perform operations on webpage elements.
- Assertions verify whether those operations produced the expected result.

Example:

```
await page.getByRole('button', { name: 'Login' }).click();

await expect(page.getByText('Dashboard')).toBeVisible();
```

Clicking Login is an action. Checking Dashboard visibility is an assertion.

Q3. What are auto-retrying assertions?

These assertions repeatedly check a condition until it passes or the assertion timeout is reached. They are useful for dynamic web applications.

Example:

```
await expect(page.locator('.success'))
  .toBeVisible();
```

Q4. What is the default assertion timeout in Playwright?

The default assertion timeout is 5 seconds. It can be changed using `expect.configure()` or the `expect` setting in Playwright configuration.

Q5. Do we need to use `await` with assertions?

For asynchronous Playwright web assertions, yes.

```
await expect(page.locator('#message')).toBeVisible();
```

Generic JavaScript assertions such as `expect(10).toBe(10)` are synchronous and do not need `await`.

Q6. What is the difference between `toHaveText()` and `toContainText()`?

- `toHaveText()` checks whether the element's text matches the expected text.
- `toContainText()` checks whether the element contains the expected text.

```
await expect(locator).toHaveText('Login Successful');

await expect(locator).toContainText('Login');
```

Q7. What is the difference between `toBeVisible()` and `toBeAttached()`?

- `toBeVisible()` checks whether the element is visible to the user.
- `toBeAttached()` checks whether the element is attached to the DOM.

An element can be attached to the DOM but hidden from the user.

Intermediate interview questions

Q8. What is a soft assertion?

A soft assertion records a failure but allows the test to continue executing. It is useful when we want to validate multiple independent elements and collect all failures in one test.

```
await expect.soft(locator1).toBeVisible();
await expect.soft(locator2).toBeVisible();
```

Q9. What is `expect.poll()`?

`expect.poll()` repeatedly executes a function and checks its returned value until the expected condition is met or the timeout expires. It is useful for API status checks and asynchronous backend operations.

Q10. What is `expect.toPass()`?

It retries an entire block of code until all assertions inside that block pass or the configured timeout is reached.

Q11. What is `expect.configure()`?

It creates a reusable expect instance with custom default settings, such as timeout and soft assertion behavior.

Q12. What is `expect.extend()`?

It allows us to create custom assertion matchers for application-specific validation.

Q13. What is `expect.arrayContaining()`?

It checks whether an array contains the specified elements, even if the array has additional elements.

Q14. What is the difference between `expect.soft()` and `expect.poll()`?

`expect.soft()` changes how an assertion failure affects test execution. `expect.poll()` repeatedly checks a function's returned value. They can also be combined.

Scenario-based interview questions

Q15. Your application takes 8 seconds to display a success message. How will you handle it?

I would increase the assertion timeout to 10 seconds.

```
await expect(page.locator('.success'))
  .toBeVisible({ timeout: 10000 });
```

This allows Playwright to keep checking for the success message for up to 10 seconds.

Q16. You need to validate 10 fields on a registration page. If one field fails, you still want to validate the remaining fields. What will you use?

I will use soft assertions.

```
await expect.soft(page.locator('#name'))
  .toHaveValue('Komal');

await expect.soft(page.locator('#email'))
  .toHaveValue('test@gmail.com');

await expect.soft(page.locator('#city'))
  .toHaveValue('Pune');
```

Q17. An API initially returns 202, but eventually returns 200. How will you validate it?

I will use `expect.poll()` to repeatedly check the API status.

```
await expect.poll(async () => {
  const response = await page.request.get(
    'https://example.com/api/status'
  );

  return response.status();
}, {
  timeout: 30000
}).toBe(200);
```

Q18. You need to check whether a product has a specific price attribute. How can you create a reusable assertion?

I can create a custom matcher using `expect.extend()` and use it across multiple test files.

Q19. Your test is failing because the webpage loads data asynchronously. What would you do?

I would prefer Playwright's auto-retrying assertions instead of immediate JavaScript checks. For example:

```
await expect(page.locator('.product-name'))
  .toBeVisible();
```

Q20. What happens when a normal assertion fails?

The assertion throws an error and the current test stops executing further steps. The Playwright Test runner records the failure and reports the test as failed.

18. Quick Revision – Most Important Methods

Method	Remember it as
`expect()`	Start an assertion
`toBe()`	Exact value comparison
`toEqual()`	Deep equality
`toBeTruthy()`	Value is truthy
`toBeVisible()`	Element is visible
`toBeHidden()`	Element is hidden
`toHaveText()`	Verify text
`toContainText()`	Verify partial text
`toHaveValue()`	Verify input value

`toHaveURL()`	Verify URL
`toHaveTitle()`	Verify page title
`expect.soft()`	Continue after assertion failure
`.not`	Check opposite condition
`expect.configure()`	Configure expect defaults
`expect.poll()`	Retry checking a returned value
`expect.toPass()`	Retry a block of code
`expect.extend()`	Create custom matchers
`mergeExpects()`	Combine custom matchers

19. Test Your Understanding

Playwright Assertions Quiz

5/5 answered

1. Which assertion automatically retries until the element becomes visible?

toBe()
toBeVisible()
toEqual()
toBeTruthy()

Correct

toBeVisible() is a Playwright web assertion that automatically retries until the element becomes visible or the timeout is reached.

2. Which assertion allows the test to continue after a failure?

expect()
expect.poll()
expect.soft()
expect.toPass()

Correct

expect.soft() records the failure but allows subsequent test steps to execute.

3. What is the default Playwright assertion timeout?

1 second
3 seconds
5 seconds
30 seconds

Correct

The default assertion timeout is 5,000 milliseconds.

4. Which method is used to repeatedly check an API response value?

expect.extend()
expect.poll()
expect.configure()
toHaveText()

Correct

expect.poll() repeatedly runs a function and checks its returned value.

5. Which method is used to create custom assertion matchers?

expect.extend()
expect.soft()
expect.toPass()
mergeTests()

Correct

expect.extend() registers custom matchers on the expect object.

5 / 5

Try again

My recommendation: Give extra attention to auto-retrying assertions, normal vs soft assertions, `expect.poll()`, and `expect.toPass()`. These are particularly useful to understand for practical Playwright QA interviews.

Playwright – API Testing

API Testing is an important topic for QA Automation and SDET interviews. Since you're learning Playwright with TypeScript, this topic will help you understand how to test APIs without always opening a browser.

1. What is API Testing in Playwright?

Playwright supports API testing using `APIRequestContext`.

It allows us to send HTTP requests directly to the application server without opening a webpage or interacting with its UI.

For example, suppose you are testing an e-commerce application.

You want to verify:

- Whether a user can log in.
- Whether a product can be added.
- Whether an order is created successfully.
- Whether the server returns the correct response.

You can perform these checks directly through APIs.

What is APIRequestContext?

`APIRequestContext` is a Playwright class that allows us to send HTTP requests such as:

- GET – Retrieve data.

- POST – Create data.
- PUT – Update or replace data.
- PATCH – Partially update data.
- DELETE – Delete data.

Why do we use API testing in Playwright?

There are three major use cases.

1. Test APIs directly

Verify API responses, status codes, response bodies, and server behavior without opening a browser.

2. Prepare test data

Create users, products, or orders through APIs before starting UI tests.

3. Validate server-side results

Perform an action through the browser and verify through an API that the data was saved correctly.

Example: UI testing vs API testing

UI testing	API testing
Opens a browser	Does not require a browser
Clicks Login button	Sends a POST login request
Enters username and password	Sends credentials in request body
Verifies dashboard	Verifies response and token
Usually slower	Usually faster

2. Writing API Tests

Playwright provides an API request fixture called `request`.

You can use it directly inside a Playwright test.

Basic syntax

```
const response = await request.get('https://example.com/api/users');
```

Other HTTP methods:

```
await request.get('/users');

await request.post('/users', {
  data: {
    name: 'Komal'
  }
});

await request.put('/users/1', {
  data: {
    name: 'Komal Updated'
  }
});

await request.patch('/users/1', {
  data: {
    name: 'Komal'
  }
});

await request.delete('/users/1');
```

Important response methods

Method	Purpose
`response.ok()`	Returns true if HTTP status is in the 200–299 range

`response.status()`	Returns HTTP status code
`response.statusText()`	Returns status text
`response.json()`	Reads response body as JSON
`response.text()`	Reads response body as text
`response.headers()`	Gets response headers
`response.body()`	Gets response body as a buffer

Important: `response.json()` is asynchronous, so use `await`.

Example: GET request

```
import { test, expect } from '@playwright/test';

test('Get user details', async ({ request }) => {

  const response = await request.get(
    'https://jsonplaceholder.typicode.com/users/1'
  );

  expect(response.status()).toBe(200);

  const responseBody = await response.json();

  expect(responseBody.name).toBe('Leanne Graham');

});
```

Explanation:

1. `request.get()` sends a GET request.
2. `response.status()` verifies the HTTP status.
3. `response.json()` extracts the response body.

4. `toBe()` verifies the user's name.

3. Configuration – baseURL and extraHTTPHeaders

In real-time projects, we should avoid repeating the API URL and authorization headers in every test.

We can configure them once in `playwright.config.ts`.

Example

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    baseURL: 'https://api.github.com',

    extraHTTPHeaders: {
      'Accept': 'application/vnd.github.v3+json',

      'Authorization': `token ${process.env.API_TOKEN}`,
    },
  },
});
```

Understand the properties

Property	Meaning
`baseURL`	Common base URL for API requests
`extraHTTPHeaders`	Headers automatically added to requests
`Accept`	Specifies the response format expected
`Authorization`	Sends authentication credentials
`process.env.API_TOKEN`	Reads the API token from an environment variable

What is baseURL?

Suppose your base URL is:

`baseURL: 'https://api.github.com'`

Instead of writing the complete URL:

```
await request.get('https://api.github.com/users');
```

You can write:

```
await request.get('/users');
```

Playwright combines the relative path with the configured base URL.

What is extraHTTPHeaders?

HTTP headers provide additional information to the server.

For example:

```
extraHTTPHeaders: {
  'Content-Type': 'application/json',
  'Authorization': 'Bearer your-token'
}
```

- `Content-Type` tells the server the format of the request body.
- `Authorization` provides authentication information.

The exact authentication format depends on the API. GitHub's documentation example uses its token format.

Interview question: Why do we configure baseURL?

To avoid repeating the complete API URL in every test. It makes test scripts shorter, cleaner, and easier to maintain.

4. Proxy Configuration

Sometimes company networks use a proxy server to access the internet or internal APIs.

Playwright allows us to configure a proxy in `playwright.config.ts`.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    proxy: {
      server: 'http://my-proxy:8080',
      username: 'user',
      password: 'secret'
    }
  }
});
```

Property	Meaning
`server`	Proxy server address
`username`	Proxy authentication username
`password`	Proxy authentication password

The built-in `request` fixture uses the configured proxy.

Interview question: Why do we configure a proxy in Playwright?

To allow requests to pass through an organization's proxy server when direct internet access is restricted or network traffic must be routed through a proxy.

5. Creating and Validating API Data

The documentation demonstrates testing issue creation using the GitHub API.

The test performs these steps:

1. Send a POST request to create an issue.
2. Verify the response is successful.
3. Send a GET request to retrieve issues.
4. Verify that the newly created issue exists.

Example: Create an issue

```
const REPO = 'test-repo-1';
const USER = 'github-username';

test('should create a bug report', async ({ request }) => {

  const newIssue = await request.post(
    `/repos/${USER}/${REPO}/issues`,
    {
      data: {
        title: '[Bug] report 1',
        body: 'Bug description'
      }
    }
  );

  expect(newIssue.ok()).toBeTruthy();

  const issues = await request.get(
    `/repos/${USER}/${REPO}/issues`
  );

  expect(issues.ok()).toBeTruthy();

  expect(await issues.json()).toContainEqual(
    expect.objectContaining({
      title: '[Bug] report 1',
      body: 'Bug description'
    })
  );
});
```

Line-by-line explanation

1. Repository and username

```
const REPO = 'test-repo-1';
const USER = 'github-username';
```

These variables store the repository name and GitHub username.

2. Send POST request

```
const newIssue = await request.post(
  `/repos/${USER}/${REPO}/issues`,
  {
    data: {
      title: '[Bug] report 1',
      body: 'Bug description'
    }
  }
);
```

- `request.post()` sends a POST request.
- The endpoint identifies the repository's issues resource.
- `data` contains the request body.
- The server creates a new issue.

3. Verify the POST response

```
expect(newIssue.ok()).toBeTruthy();
```

Checks whether the HTTP response status is in the successful 200–299 range.

4. Send GET request

```
const issues = await request.get(
  `/repos/${USER}/${REPO}/issues`
);
```

Retrieves the issues from the repository.

5. Verify GET response

```
expect(issues.ok()).toBeTruthy();
```

Checks whether the GET request was successful.

6. Verify the created issue

```
expect(await issues.json()).toContainEqual(
  expect.objectContaining({
    title: '[Bug] report 1',
    body: 'Bug description'
  })
);
```

- `issues.json()` extracts the response data.
- `toContainEqual()` checks whether the returned array contains a matching object.
- `objectContaining()` checks the title and body without requiring every other property to match.

Important HTTP methods in this example

Method	Operation	Example
GET	Read issues	`request.get()`
POST	Create an issue	`request.post()`
PUT	Replace/update a resource	`request.put()`
PATCH	Partially update a resource	`request.patch()`
DELETE	Delete a resource	`request.delete()`

Interview question: How do you validate an API response body in Playwright?

I use `response.json()` to extract the response body and Playwright assertions such as `toEqual()`, `toContainEqual()`, or `toHaveProperty()` to validate the expected data.

6. Setup and Teardown – `beforeAll()` and `afterAll()`

In real-time automation, tests often need test data before execution.

For example, before testing GitHub issue creation, the repository must already exist.

We can use Playwright hooks:

- `beforeAll()` – Executes once before all tests in the test file.
- `afterAll()` – Executes once after all tests in the test file.

Example

```
test.beforeAll(async ({ playwright }) => {

  const apiContext = await playwright.request.newContext({
    baseURL: 'https://api.github.com',
    extraHTTPHeaders: {
      'Accept': 'application/vnd.github.v3+json',
      'Authorization': `token ${process.env.API_TOKEN}`
    }
  });

  const response = await apiContext.post('/user/repos', {
    data: {
      name: REPO
    }
  });

  expect(response.ok()).toBeTruthy();

  await apiContext.dispose();
});
```

The example above demonstrates the setup operation. In an actual test suite, if you need the same request context for setup, tests, and teardown, keep it in a shared variable and dispose it in `afterAll()`.

Complete setup and teardown pattern

```
import { test, expect } from '@playwright/test';

const REPO = 'test-repo-1';

let apiContext;

test.beforeAll(async ({ playwright }) => {

  apiContext = await playwright.request.newContext({
    baseURL: 'https://api.github.com',
    extraHTTPHeaders: {
      'Accept': 'application/vnd.github.v3+json',
      'Authorization': `token ${process.env.API_TOKEN}`
```

```
    }
  });

  // Create repository
  const response = await apiContext.post('/user/repos', {
    data: {
      name: REPO
    }
  });

  expect(response.ok()).toBeTruthy();
});

test('create an issue', async () => {

  const response = await apiContext.post(
    `/repos/${process.env.GITHUB_USERNAME}/${REPO}/issues`,
    {
      data: {
        title: 'Bug report',
        body: 'Login is not working'
      }
    }
  );

  expect(response.ok()).toBeTruthy();
});

test.afterAll(async () => {

  // Delete repository
  const response = await apiContext.delete(
    `/repos/${process.env.GITHUB_USERNAME}/${REPO}`
  );

  expect(response.ok()).toBeTruthy();

  // Dispose request context
  await apiContext.dispose();
});
```

This is an illustrative pattern; use a `try/finally` cleanup strategy in production so the request context is disposed even if deletion fails. Also ensure the repository is unique to the test run and that your GitHub token has the necessary permissions.

Why do we use setup and teardown?

Hook	Purpose	Example
`beforeAll()`	Prepare test data	Create repository
`beforeEach()`	Prepare data before each test	Create a test user
`afterEach()`	Clean up after each test	Delete test order
`afterAll()`	Clean up after all tests	Delete repository

Interview question: What is the difference between `beforeAll()` and `beforeEach()`?

`beforeAll()` runs once before all tests in a test file, whereas `beforeEach()` runs before every individual test.

For example, if there are 5 tests:

- `beforeAll()` runs once.
- `beforeEach()` runs 5 times.

7. Using Request Context – `APIRequestContext`

Until now, we used the built-in `request` fixture.

Playwright also allows us to create an API request context manually using:

`playwright.request.newContext()`

This gives us more control over API requests.

Example

```
import { request } from '@playwright/test';

(async () => {

  const context = await request.newContext({
    baseURL: 'https://api.github.com'
  });

  // Create repository
  await context.post('/user/repos', {
    headers: {
      'Accept': 'application/vnd.github.v3+json',
      'Authorization': `token ${process.env.API_TOKEN}`
    },
    data: {
      name: 'test-repo-1'
    }
  });

  // Delete repository
  await context.delete('/repos/github-username/test-repo-1', {
    headers: {
      'Accept': 'application/vnd.github.v3+json',
      'Authorization': `token ${process.env.API_TOKEN}`
    }
  });

  await context.dispose();

})();
```

Understand the code

Code	Meaning
`request.newContext()`	Creates a new API request context
`baseURL`	Sets the common API endpoint

`context.post()`	Sends POST request
`context.delete()`	Sends DELETE request
`context.dispose()`	Releases the request context and its resources
`(async () => {})()`	Async IIFE, used to run asynchronous code in a standalone script

Built-in request fixture vs manually created context

Built-in `request` fixture	Manual request context
Provided by Playwright Test	Created using `request.newContext()` or `playwright.request.newContext()`
Convenient for individual tests	Gives more control over context lifecycle
Automatically managed by test runner	You must dispose it yourself
Uses configuration such as `baseURL` and headers	You specify the context options

Interview question: When would you create an API request context manually?

When I need a separate API session, custom headers, independent cookies, or a request context that I want to reuse across multiple operations.

8. Sending API Requests from UI Tests

This is one of the most useful features in real-world QA automation.

Sometimes we need to combine UI testing and API testing in the same test.

There are two important approaches:

1. Establishing preconditions using APIs.
2. Validating postconditions using APIs.

Let's understand both.

A. Establishing Preconditions

Meaning: Prepare the required test data through an API before testing the application UI.

Example: You want to verify whether a newly created GitHub issue appears at the top of the Issues page.

Instead of creating the issue manually through the UI, create it using an API and then verify it through the browser.

Example

```
import { test, expect } from '@playwright/test';

const REPO = 'test-repo-1';
const USER = 'github-username';

let apiContext;

test.beforeAll(async ({ playwright }) => {

  apiContext = await playwright.request.newContext({
    baseURL: 'https://api.github.com',

    extraHTTPHeaders: {
      'Accept': 'application/vnd.github.v3+json',
      'Authorization': `token ${process.env.API_TOKEN}`
    }
  });

});

test.afterAll(async () => {
  await apiContext.dispose();
});
```

```
test('last created issue should be first in the list', async
({ page }) => {

  // Create issue using API
  const newIssue = await apiContext.post(
    `/repos/${USER}/${REPO}/issues`,
    {
      data: {
        title: '[Feature] request 1'
      }
    }
  );

  expect(newIssue.ok()).toBeTruthy();

  // Open issues page in browser
  await page.goto(`https://github.com/${USER}/${REPO}/issues`);

  // Locate first issue
  const firstIssue = page
    .locator(`a[data-hovercard-type='issue']`)
    .first();

  // Verify issue title
  await expect(firstIssue).toHaveText('[Feature] request 1');

});
```

Execution flow

Step 1: API request

Create a new issue using POST.

Step 2: Open browser

Navigate to the GitHub Issues page.

Step 3: UI verification

Find the first issue in the list.

Step 4: Assertion

Verify that the new issue appears at the top.

Why is this useful?

Creating test data through APIs is generally faster than performing all setup steps manually through the UI. It also allows us to focus the UI test on the functionality we actually want to verify.

B. Validating Postconditions

Meaning: Perform an action through the UI and then verify through an API that the server saved the correct result.

Example:

1. Open GitHub Issues page.
2. Click New Issue.
3. Enter issue title and description.
4. Submit the issue.
5. Verify through API that the issue exists on the server.

Example

```
test('verify issue created through UI', async ({ page }) => {

  await page.goto(`https://github.com/${USER}/${REPO}/issues`);

  // Create issue through UI
  await page.getByText('New Issue').click();

  await page.getByRole('textbox', { name: 'Title' })
    .fill('Bug report 1');

  await page.getByRole('textbox', { name: 'Comment body' })
    .fill('Bug description');

  await page.getByText('Submit new issue').click();

  // Extract issue ID from URL
  const issueId = new URL(page.url()).pathname.split('/').pop();
```

```
  // Verify issue through API
  const newIssue = await apiContext.get(
    `https://api.github.com/repos/${USER}/${REPO}/issues/${issueId}`
  );

  expect(newIssue.ok()).toBeTruthy();

  expect(await newIssue.json()).toEqual(
    expect.objectContaining({
      title: 'Bug report 1'
    })
  );

});
```

What is happening here?

Step	Operation	Type
1	Open GitHub Issues page	UI
2	Click New Issue	UI
3	Enter issue details	UI
4	Submit issue	UI
5	Extract issue ID	Browser URL
6	Retrieve issue using GET	API
7	Verify issue title	API assertion

Interview question: What is the difference between preconditions and postconditions?

- Preconditions are the conditions or test data prepared before executing the actual test. We can use API requests to create the required data.

- Postconditions are the results we verify after performing the test. We can use API requests to confirm that the application saved the expected data.

Real-time e-commerce example

Preconditions using API	UI test	Postconditions using API
Create test user	Login	Verify login session
Create product	Open product page	Verify product details
Prepare cart data	Place order	Verify order in database/API
Create address	Complete checkout	Verify saved delivery address

9. Reusing Authentication State – `storageState()`

Important interview topic

Web applications commonly use cookies or tokens to maintain authenticated sessions.

Playwright allows us to save and reuse authentication state using `storageState()`.

What is storage state?

Storage state represents the browser/API authentication-related state that Playwright can export and reuse.

It can contain:

- Cookies.
- Local storage data for supported origins.
- IndexedDB data when requested through the relevant option.

This helps us avoid logging in repeatedly.

Example: Save authentication state

```
const requestContext = await request.newContext({
  httpCredentials: {
    username: 'user',
    password: 'passwd'
  }
});

// Make login request
await requestContext.get('https://api.example.com/login');

// Save storage state
await requestContext.storageState({
  path: 'state.json'
});
```

Explanation

1. Create request context

```
const requestContext = await request.newContext({
  httpCredentials: {
    username: 'user',
    password: 'passwd'
  }
});
```

Creates an API request context with HTTP authentication credentials.

Note: `httpCredentials` handles HTTP authentication; it is not automatically the same as submitting an application's username and password through a login endpoint.

2. Make login request

```
await requestContext.get('https://api.example.com/login');
```

Sends a request to the login endpoint. The actual authentication mechanism depends on the application.

3. Save state

```
await requestContext.storageState({
  path: 'state.json'
});
```

Exports the request context's storage state into a JSON file.

Reuse the saved state in browser

```
const context = await browser.newContext({
  storageState: 'state.json'
});
```

Creates a browser context using the saved state.

This is useful when an API login establishes cookies that the browser application can use.

Authentication flow

API login

Authenticate using an API request.

Save storage state

Export cookies and supported storage data to `state.json`.

Create browser context

Load the saved authentication state.

Access authenticated application

Avoid repeating the login process.

Important: API and browser storage state can be reused when the application's authentication mechanism is compatible. A bearer token stored only in an API request header, for example, does not automatically become a browser cookie.

10. Context Request vs Global Request

This is a very important concept in Playwright API testing.

Playwright provides two types of `APIRequestContext`:

1. API request associated with a `BrowserContext`.
2. Independent API request context created using `newContext()`.

A. Context-associated request

You can access the API request context through:

`context.request`

or:

`page.request`

This request context shares cookies with its associated browser context.

Example

```
test('context request shares cookies', async ({
  page,
  context
}) => {

  const response = await context.request.get(
    'https://example.com'
  );

  const cookies = await context.cookies();

  console.log(cookies);

});
```

What happens?

- API request uses cookies from the associated browser context.
- If the API response contains `Set-Cookie`, those cookies are automatically updated in the browser context.
- The browser and API can share the same cookie-based authentication session.

B. Independent API request context

You can create a separate request context using:

```
const request = await playwright.request.newContext();
```

This creates an independent cookie store.

Example

```
test('independent API request context', async ({
  browser,
  playwright
}) => {

  const request = await playwright.request.newContext();

  // Send API request
  const response = await request.get('https://example.com');

  // Save API cookies
  const storageState = await request.storageState();

  // Create browser context with API storage state
  const browserContext = await browser.newContext({
    storageState
  });

  const cookies = await browserContext.cookies();

  console.log(cookies);

  // Cleanup
  await browserContext.close();
  await request.dispose();

});
```

Here:

- API context maintains its own cookies.
- The browser context does not automatically receive those cookies.

- We explicitly export the API storage state and load it into a new browser context.

Comparison table

Feature	Context-associated request	Independent request context
Created through	`page.request` or `context.request`	`playwright.request.newContext()`
Cookie storage	Shared with browser context	Separate cookie storage
API response `Set-Cookie`	Updates browser cookies	Does not update browser cookies
Authentication	Can share browser session	Independent session
Best use	API calls using browser login	Separate API testing session

Interview question: What is the difference between `page.request` and `playwright.request.newContext()`?

`page.request` is associated with the browser context and shares its cookie storage. `playwright.request.newContext()` creates an independent API request context with separate cookie storage.

11. Important API Testing Methods – Quick Reference

Method	Purpose
`request.get()`	Send GET request

`request.post()`	Send POST request
`request.put()`	Send PUT request
`request.patch()`	Send PATCH request
`request.delete()`	Send DELETE request
`request.fetch()`	Send a request using a provided request object or options
`request.newContext()`	Create an independent API request context
`response.status()`	Get HTTP status code
`response.ok()`	Check whether response is successful
`response.json()`	Read response as JSON
`response.text()`	Read response as text
`response.headers()`	Get response headers
`response.body()`	Read response body as a buffer
`request.storageState()`	Export authentication storage state
`request.dispose()`	Release API request context resources

Important HTTP status codes

Status	Meaning

200	OK
201	Created
202	Accepted
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error
503	Service Unavailable

Remember: `response.ok()` checks whether the status is between 200 and 299. It does not check whether the response body contains the expected business data.

12. Important Playwright API Testing Interview Questions

Basic questions

Q1. What is API testing in Playwright?

Playwright provides API testing capabilities through `APIRequestContext`. We can send HTTP requests directly to the server and validate status codes, response headers, and response bodies without opening a browser.

Q2. What is APIRequestContext?

`APIRequestContext` is a Playwright API that allows us to send HTTP requests such as GET, POST, PUT, PATCH, and DELETE.

Q3. What is the request fixture in Playwright?

`request` is a built-in Playwright Test fixture that provides an API request context. We can use it directly in test functions.

```
test('GET users', async ({ request }) => {
  const response = await request.get('/users');

  expect(response.status()).toBe(200);
});
```

Q4. What is the difference between GET and POST?

- GET retrieves data from the server.
- POST sends data to the server, commonly to create a resource.

Q5. How do you validate an API status code?

```
const response = await request.get('/users');

expect(response.status()).toBe(200);
```

Q6. How do you validate an API response body?

```
const response = await request.get('/users/1');

const body = await response.json();

expect(body.name).toBe('Leanne Graham');
```

Q7. What is the purpose of `response.ok()`?

It returns `true` if the HTTP status code is between 200 and 299. Otherwise, it returns `false`.

Q8. What is `baseURL`?

It is the common base URL configured for requests. It avoids repeating the full API URL in every test.

Intermediate questions

Q9. How do you pass request body data in a POST request?

Using the `data` option.

```
await request.post('/users', {
  data: {
    name: 'Komal',
    email: 'komal@example.com'
  }
});
```

Q10. How do you pass authentication headers?

Using `extraHTTPHeaders` in the configuration or `headers` in an individual request.

```
await request.get('/users', {
  headers: {
    Authorization: `Bearer ${process.env.API_TOKEN}`
  }
});
```

Q11. What is the purpose of `beforeAll()` and `afterAll()` in API testing?

- `beforeAll()` prepares data before the tests execute.
- `afterAll()` cleans up test data after all tests finish.

Q12. What is `storageState()`?

It exports authentication-related storage state, such as cookies and supported local storage data. We can reuse it to authenticate another browser or API context when the authentication mechanism is compatible.

Q13. What is the difference between API testing and UI testing?

API testing verifies server endpoints directly through HTTP requests. UI testing interacts with the application through a browser and verifies what the user sees.

Q14. Can we perform API testing and UI testing in the same Playwright test?

Yes. We can create test data through APIs, perform UI actions, and verify server-side results through APIs in the same test.

Q15. Why do we use `dispose()`?

It releases the resources associated with a manually created API request context, including stored response resources. We should dispose of manually created contexts when finished.

Scenario-based interview questions

Q16. You need to create a user before executing a UI login test. How will you do it?

I will use an API POST request in the test setup to create the user. Then I will open the browser, navigate to the login page, and test the login functionality using that user.

Q17. You place an order through the UI. How will you verify that the order was saved successfully on the server?

After placing the order through the UI, I will extract the order ID and send a GET request to the order API. Then I will verify the response status, order ID, and order details.

Q18. How do you avoid repeating authentication headers in every API test?

I will configure `baseURL` and `extraHTTPHeaders` in `playwright.config.ts` or create a reusable API request context.

Q19. How do you handle API authentication in Playwright?

I can pass a bearer token or other required credentials through request headers or use the authentication mechanism supported by the application. I would store sensitive tokens in environment variables rather than hardcoding them.

Q20. How do you verify that an API eventually returns a successful response?

I can use `expect.poll()` to repeatedly send the request and check its status until it returns the expected value or the timeout expires.

```
await expect.poll(async () => {
  const response = await request.get('/api/status');
  return response.status();
}).toBe(200);
```

13. Practical Interview Coding Questions

Try understanding these examples because interviewers may ask you to write API test scripts.

Question 1: Write a GET API test

```
test('Get user details', async ({ request }) => {

  const response = await request.get(
    'https://jsonplaceholder.typicode.com/users/1'
  );

  expect(response.status()).toBe(200);

  const body = await response.json();

  expect(body.id).toBe(1);
  expect(body.name).toBe('Leanne Graham');

});
```

Question 2: Write a POST API test

```
test('Create user', async ({ request }) => {

  const response = await request.post(
    'https://jsonplaceholder.typicode.com/users',
    {
      data: {
        name: 'Komal',
        email: 'komal@example.com'
      }
    }
  );

  expect(response.status()).toBe(201);

  const body = await response.json();

  expect(body.name).toBe('Komal');

});
```

Question 3: Write a DELETE API test

```
test('Delete user', async ({ request }) => {

  const response = await request.delete(
    'https://jsonplaceholder.typicode.com/users/1'
  );

  expect(response.status()).toBe(200);

});
```

Note: JSONPlaceholder is a practice API. Its operations are simulated and do not permanently modify a real user database.

14. Final Revision – What You Must Remember

API Testing Cheat Sheet

Most important topics for interviews

1. `APIRequestContext` – Send API requests.
2. `request` fixture – Built-in API fixture.
3. GET, POST, PUT, PATCH, DELETE – HTTP methods.
4. `baseURL` – Common API URL.
5. `extraHTTPHeaders` – Common request headers.
6. `response.status()` – Verify status code.
7. `response.json()` – Read response body.
8. `beforeAll()` / `afterAll()` – Test setup and cleanup.
9. `request.newContext()` – Create independent API context.
10. `storageState()` – Reuse authentication state.
11. Preconditions – Prepare test data using APIs.
12. Postconditions – Verify server data after UI actions.

13. Context request vs independent request – Understand cookie sharing.

My recommendation for your Selenium-to-Playwright QA interview preparation: Focus particularly on API requests, response validation, authentication, API + UI integration, and preconditions/postconditions. These are practical topics that help demonstrate both API and UI automation skills.

Playwright Authentication – Complete Explanation

Authentication is one of the most important topics in Playwright interviews, especially for QA Automation and SDET roles.

Let's understand it step by step with simple examples, practical scenarios, and interview questions.

1. What is Authentication in Playwright?

Authentication means verifying the identity of a user before allowing access to an application.

For example, when you log in to an e-commerce application:

1. Open the login page.
2. Enter username and password.
3. Click Login.
4. The application verifies your credentials.
5. If credentials are correct, you are redirected to the home page.
6. The application stores authentication information, usually in cookies or browser storage.

Normally, if you run 10 tests, each test might log in separately.

This takes extra execution time.

Playwright provides a better approach: authenticate once, save the authenticated browser state, and reuse it in multiple tests.

Example

Suppose you have three tests:

- Test 1: Verify the home page.
- Test 2: Verify the profile page.
- Test 3: Verify the orders page.

Without authentication reuse:

```
Test 1 → Login → Home page
Test 2 → Login → Profile page
Test 3 → Login → Orders page
```

With authentication reuse:

```
Setup → Login once → Save authentication state

Test 1 → Reuse saved state → Home page
Test 2 → Reuse saved state → Profile page
Test 3 → Reuse saved state → Orders page
```

Main benefit
Faster test execution and less duplicate login code.

2. Important concepts

Before understanding the code, learn these concepts.

Concept	Meaning
Authentication	Verifying the user's identity
Browser Context	An isolated browser session
Cookies	Small pieces of data used to maintain sessions

Local Storage	Browser storage that can persist across page reloads
Session Storage	Browser storage limited to a tab's session
Storage State	Saved authentication information
Setup Project	Project that performs authentication before test projects
Worker	A Playwright process that executes tests
`storageState`	Option used to load saved authentication information

What is storage state?

Storage state is a snapshot of browser authentication-related information.

It can contain:

- Cookies
- Local storage
- IndexedDB (when configured to include it)
- Passkey/WebAuthn-related state

Playwright can save this information in a JSON file.

Example:

```
playwright/
   .auth/
      user.json
```

When a test starts, Playwright loads this file and creates an authenticated browser context.

Important: The JSON file may contain sensitive session information. Anyone who obtains it may be able to access your test account.

Therefore, never commit authentication files to GitHub.

Add this to `.gitignore`:

```
playwright/.auth
```

3. Basic approach – One shared account for all tests

Recommended for independent tests

This is the simplest and most commonly used approach.

You authenticate once in a setup project and reuse the saved state in all testing projects.

When should you use it?

Use one shared account when:

- Tests do not modify shared server-side data.
- Tests can run simultaneously without affecting each other.
- All tests can use the same account.

Example:

- Test A checks the profile page.
- Test B checks the dashboard.
- Test C checks the contact page.

These tests can safely use the same account if they do not interfere with one another.

Step 1: Create the authentication directory

```
mkdir -p playwright/.auth
```

Add the directory to `.gitignore`:

```
playwright/.auth
```

Step 2: Create the authentication setup file

Create:

`tests/auth.setup.ts`

```
import { test as setup, expect } from '@playwright/test';
import path from 'path';

const authFile = path.join(
  __dirname,
  '../playwright/.auth/user.json'
);

setup('authenticate', async ({ page }) => {

  await page.goto('https://github.com/login');

  await page.getByLabel('Username or email address')
    .fill('username');

  await page.getByLabel('Password')
    .fill('password');

  await page.getByRole('button', { name: 'Sign in' })
    .click();

  await page.waitForURL('https://github.com/');

  await expect(
    page.getByRole('button', { name: 'View profile and more' })
  ).toBeVisible();

  await page.context().storageState({ path: authFile });

});
```

Explanation line by line

1. Import statements

```
import { test as setup, expect } from '@playwright/test';
import path from 'path';
```

- `test as setup`: Imports Playwright's test function with the name `setup`.
- `expect`: Used to verify successful login.
- `path`: Node.js module used to create file paths.

2. Authentication file path

```
const authFile = path.join(
  __dirname,
  '../playwright/.auth/user.json'
);
```

This defines where the authentication state will be saved.

- `__dirname`: Current directory of the setup file.
- `path.join()`: Combines directory paths.
- `user.json`: File containing saved authentication state.

3. Setup test

```
setup('authenticate', async ({ page }) => {
```

This creates a setup test called `authenticate`.

The `page` fixture provides a browser page.

4. Open login page

```
await page.goto('https://github.com/login');
```

Navigates to the login page.

5. Enter username and password

```
await page.getByLabel('Username or email address')
  .fill('username');

await page.getByLabel('Password')
  .fill('password');
```

- `getByLabel()` locates an input using its accessible label.
- `fill()` enters the given text.

6. Click Sign in

```
await page.getByRole('button', { name: 'Sign in' }).click();
```

Finds the button by its role and accessible name, then clicks it.

7. Wait for successful login

```
await page.waitForURL('https://github.com/');
```

Waits until the browser navigates to the expected URL.

Why is this important?

Sometimes login involves multiple redirects. Waiting for the final URL helps ensure the authentication process has completed.

8. Verify login

```
await expect(
  page.getByRole('button', { name: 'View profile and more' })
).toBeVisible();
```

Checks whether the profile button is visible.

This confirms that the user is logged in.

9. Save authentication state

```
await page.context().storageState({ path: authFile });
```

This is the most important line.

It saves the current browser context's authentication state into `user.json`.

Step 3: Configure Playwright projects

Open:

```
playwright.config.ts
```

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({

  projects: [
```

```
    {
      name: 'setup',
      testMatch: /.*\.setup\.ts/,
    },

    {
      name: 'chromium',
      use: {
        ...devices['Desktop Chrome'],
        storageState: 'playwright/.auth/user.json',
      },
      dependencies: ['setup'],
    },

    {
      name: 'firefox',
      use: {
        ...devices['Desktop Firefox'],
        storageState: 'playwright/.auth/user.json',
      },
      dependencies: ['setup'],
    },

  ],

});
```

Important properties

Property	Meaning
`projects`	Defines different test projects
`name`	Project name
`testMatch`	Identifies setup files
`use`	Configures browser and test options

`storageState`	Loads saved authentication
`dependencies`	Runs another project before this project

What does `dependencies: ['setup']` mean?

It tells Playwright that the setup project must complete successfully before the dependent project starts.

Execution flow:

Setup Project

Login and save `user.json`

Chromium

Reuse user.json

Firefox

Reuse user.json

Step 4: Write your test

`tests/example.spec.ts`

```
import { test } from '@playwright/test';

test('verify authenticated user', async ({ page }) => {

  await page.goto('https://github.com/');

  // User is already authenticated.

});
```

Notice that we are not entering the username or password again.

Playwright automatically loads the saved authentication state through the configuration.

4. Moderate approach – One account per parallel worker

For tests that modify server data

Now consider a different situation.

Suppose you have two tests:

- Test A changes the user's profile settings.
- Test B verifies the user's profile settings.

Both tests run in parallel using the same account.

This can cause conflicts.

For example:

```
Test A → Changes username to Komal123
Test B → Expects username to be Komal
```

The tests may interfere with each other.

Solution: One account per worker

Each parallel worker gets its own account and authentication state.

Example:

```
Worker 0 → Account 0 → 0.json
Worker 1 → Account 1 → 1.json
Worker 2 → Account 2 → 2.json
```

What is a worker?

A worker is a separate process used by Playwright Test to execute tests.

Playwright can execute tests in parallel using multiple workers.

Step 1: Create fixtures.ts

`playwright/fixtures.ts`

```
import { test as baseTest, expect } from '@playwright/test';
import fs from 'fs';
import path from 'path';

export * from '@playwright/test';

export const test = baseTest.extend<{}, {
  workerStorageState: string
}>({

  storageState: ({ workerStorageState }, use) =>
    use(workerStorageState),

  workerStorageState: [async ({ browser }, use) => {

    const id = test.info().parallelIndex;

    const fileName = path.resolve(
      test.info().project.outputDir,
      `.auth/${id}.json`
    );

    if (fs.existsSync(fileName)) {
      await use(fileName);
      return;
    }

    const page = await browser.newPage({
      storageState: undefined
    });

    const account = await acquireAccount(id);

    await page.goto('https://github.com/login');

    await page.getByLabel('Username or email address')
      .fill(account.username);

    await page.getByLabel('Password')
      .fill(account.password);

    await page.getByRole('button', { name: 'Sign in' })
```

```
      .click();

    await page.waitForURL('https://github.com/');

    await expect(
      page.getByRole('button', { name: 'View profile and more' })
    ).toBeVisible();

    await page.context().storageState({ path: fileName });

    await page.close();

    await use(fileName);

  }, { scope: 'worker' }],

});
```

Important concepts in this code

1. `baseTest.extend()`

```
export const test = baseTest.extend(...)
```

Used to create custom fixtures or override existing fixtures.

2. `storageState` fixture

```
storageState: ({ workerStorageState }, use) =>
  use(workerStorageState),
```

Overrides the default storage state fixture.

It tells Playwright to use the authentication state generated for the current worker.

3. `parallelIndex`

```
const id = test.info().parallelIndex;
```

Returns the index of the current parallel worker.

For example:

Worker	parallelIndex
Worker 1	0
Worker 2	1
Worker 3	2

4. `fs.existsSync()`

```
if (fs.existsSync(fileName)) {
  await use(fileName);
  return;
}
```

Checks whether the authentication file already exists.

If it exists, the worker reuses it instead of logging in again.

5. `scope: 'worker'`

```
{ scope: 'worker' }
```

This is very important.

It means the fixture is initialized once per worker, rather than once per test.

All tests executed by that worker can reuse its authentication state.

6. `acquireAccount(id)`

```
const account = await acquireAccount(id);
```

This is a placeholder function from the documentation.

It represents your own account-allocation logic. You must implement it to return a unique username and password for each worker.

Step 2: Import custom fixtures

```
import { test, expect } from '../playwright/fixtures';
```

```
test('verify profile', async ({ page }) => {

  await page.goto('https://github.com/');

});
```

Important: Import `test` from your custom fixture file, not directly from `@playwright/test`, so the overridden authentication fixture is used.

5. Advanced scenario – Authenticate using API request

Sometimes logging in through the UI takes more time.

If the application supports API authentication, you can log in directly through an API request.

UI authentication

```
Open browser
   ↓
Open login page
   ↓
Enter username/password
   ↓
Click login
   ↓
Wait for dashboard
   ↓
Save authentication state
```

API authentication

```
Send API login request
   ↓
Receive authentication cookies
   ↓
Save authentication state
   ↓
Reuse state in browser tests
```

Example

```
import { test as setup } from '@playwright/test';

const authFile = 'playwright/.auth/user.json';

setup('authenticate', async ({ request }) => {

  await request.post('https://example.com/api/login', {
    data: {
      username: 'user',
      password: 'password'
    }
  });

  await request.storageState({ path: authFile });

});
```

Here:

- `request` is Playwright's API request fixture.
- `post()` sends a login request.
- `data` contains the request body.
- `storageState()` saves the API request context's authentication state.

Important: The API must actually establish the authentication state in a way the browser application can reuse, typically through compatible cookies or supported storage. A successful HTTP response alone does not guarantee that browser login will work.

This is especially useful when API login is faster than UI login.

6. Multiple signed-in roles

Real-world applications often have different user roles.

For example:

Role	Permissions
Admin	Manage users, approve requests
Normal User	View profile, create requests
Manager	Review and approve requests

You may need to test each role separately.

Example: Save admin authentication

```
const adminFile = 'playwright/.auth/admin.json';

setup('authenticate as admin', async ({ page }) => {

  await page.goto('https://example.com/login');

  await page.getByLabel('Username').fill('admin');
  await page.getByLabel('Password').fill('password');

  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page.getByText('Welcome Admin')).toBeVisible();

  await page.context().storageState({ path: adminFile });

});
```

Save normal user authentication

```
const userFile = 'playwright/.auth/user.json';

setup('authenticate as user', async ({ page }) => {

  await page.goto('https://example.com/login');

  await page.getByLabel('Username').fill('user');
```

```
  await page.getByLabel('Password').fill('password');

  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page.getByText('Welcome User')).toBeVisible();

  await page.context().storageState({ path: userFile });

});
```

Now we have two authentication files:

```
playwright/.auth/
    admin.json
    user.json
```

Use admin authentication

```
import { test } from '@playwright/test';

test.use({
  storageState: 'playwright/.auth/admin.json'
});

test('admin test', async ({ page }) => {

  await page.goto('https://example.com/dashboard');

});
```

Use normal user authentication

```
test.describe('User tests', () => {

  test.use({
    storageState: 'playwright/.auth/user.json'
  });

  test('user test', async ({ page }) => {

    await page.goto('https://example.com/dashboard');

  });
```

```
});
```

Interview point: `test.use()` can configure authentication state for a particular test file or test group.

7. Testing multiple roles together in one test

This is different from testing admin and user separately.

Suppose you want to test this workflow:

1. Admin creates a request.
2. User logs in.
3. User views the request created by admin.
4. User accepts the request.

You need two separate browser contexts.

Example

```
import { test, expect } from '@playwright/test';

test('admin and user interaction', async ({ browser }) => {

  const adminContext = await browser.newContext({
    storageState: 'playwright/.auth/admin.json'
  });

  const adminPage = await adminContext.newPage();

  const userContext = await browser.newContext({
    storageState: 'playwright/.auth/user.json'
  });

  const userPage = await userContext.newPage();

  // Admin operations
```

```
  await adminPage.goto('https://example.com/admin');

  // User operations
  await userPage.goto('https://example.com/user');

  await adminContext.close();
  await userContext.close();

});
```

Why use two browser contexts?

Because each context has its own isolated cookies, storage and session.

```
Browser
   |
   |---- Admin Context
   |         |
   |         └── Admin Page
   |
   |---- User Context
             |
             └── User Page
```

Both users can interact with the application independently in the same test.

8. Multiple roles using POM fixtures

You can combine authentication with the Page Object Model.

For example, create:

- `AdminPage` class
- `UserPage` class
- `adminPage` fixture
- `userPage` fixture

Example:

```
import { test as base, type Page, type Locator } from
'@playwright/test';

class AdminPage {

  page: Page;
  greeting: Locator;

  constructor(page: Page) {
    this.page = page;
    this.greeting = page.locator('#greeting');
  }

}

class UserPage {

  page: Page;
  greeting: Locator;

  constructor(page: Page) {
    this.page = page;
    this.greeting = page.locator('#greeting');
  }

}

type MyFixtures = {
  adminPage: AdminPage;
  userPage: UserPage;
};

export * from '@playwright/test';

export const test = base.extend<MyFixtures>({

  adminPage: async ({ browser }, use) => {

    const context = await browser.newContext({
      storageState: 'playwright/.auth/admin.json'
    });
```

```
    const adminPage = new AdminPage(await context.newPage());

    await use(adminPage);

    await context.close();

  },

  userPage: async ({ browser }, use) => {

    const context = await browser.newContext({
      storageState: 'playwright/.auth/user.json'
    });

    const userPage = new UserPage(await context.newPage());

    await use(userPage);

    await context.close();

  },

});
```

Test file:

```
import { test, expect } from '../playwright/fixtures';

test('verify admin and user', async ({ adminPage, userPage }) => {

  await expect(adminPage.greeting)
    .toHaveText('Welcome, Admin');

  await expect(userPage.greeting)
    .toHaveText('Welcome, User');

});
```

Why is this useful?

- Reusable page objects.
- Cleaner test code.

- Separate authentication for each role.
- Automatic context cleanup through fixture teardown.

9. Session Storage

This is an important distinction for interviews.

Playwright's `storageState()` saves cookies and supported browser storage, but does not automatically persist session storage.

Difference between Local Storage and Session Storage

Local Storage	Session Storage
Persists across page reloads	Persists across page reloads
Normally remains after browser restart	Cleared when the tab/session ends
Shared by same-origin pages in the same browser context	Specific to a tab and origin
Supported by Playwright storage state	Not automatically included

How to save session storage manually

```
const sessionStorageData = await page.evaluate(() => {
  return JSON.stringify(sessionStorage);
});

fs.writeFileSync(
  'playwright/.auth/session.json',
  sessionStorageData,
  'utf-8'
);
```

Explanation:

- `page.evaluate()` executes JavaScript in the browser page.
- `sessionStorage` reads the current tab's session storage.
- `JSON.stringify()` converts it into a JSON string.
- `writeFileSync()` saves it into a file.

How to restore session storage

```
const sessionStorageData = JSON.parse(
  fs.readFileSync('playwright/.auth/session.json', 'utf-8')
);

await context.addInitScript(storage => {

  if (window.location.hostname === 'example.com') {

    for (const [key, value] of Object.entries(storage)) {
      window.sessionStorage.setItem(key, value);
    }

  }

}, sessionStorageData);
```

`addInitScript()` runs the supplied script when a page is initialized, before the page's own scripts execute.

This lets you restore session storage before the application reads it.

10. Avoid authentication in some tests

Sometimes your entire project uses authenticated state, but you also need to test the login page.

For example:

- Verify login page.
- Verify forgot password.
- Verify registration.

- Verify unauthenticated access restrictions.

These tests should start without authentication.

Use:

```
import { test } from '@playwright/test';

test.use({
  storageState: {
    cookies: [],
    origins: []
  }
});

test('verify login page', async ({ page }) => {

  await page.goto('https://example.com/login');

});
```

This resets the saved cookies and local storage for that test file.

Important: This does not remove authentication from your saved JSON file. It simply tells this test to start with empty storage state.

11. Important methods and properties – Quick revision

Method / Property	Purpose
`storageState()`	Saves or retrieves browser storage state
`storageState: 'user.json'`	Loads authentication state
`browser.newContext()`	Creates an isolated browser context

`context.newPage()`	Creates a new page in a context
`context.close()`	Closes a browser context
`page.context()`	Gets the page's browser context
`test.use()`	Configures options for tests
`test.extend()`	Creates or overrides fixtures
`test.info()`	Gets information about the current test
`parallelIndex`	Identifies the parallel worker
`scope: 'worker'`	Creates a fixture once per worker
`dependencies`	Defines project execution dependencies
`testMatch`	Matches test files
`addInitScript()`	Executes a script when a page is initialized
`fs.existsSync()`	Checks whether a file exists

12. Common interview questions and answers

Q1. What is authentication in Playwright?

Authentication is the process of logging into an application and verifying the user's identity. Playwright allows us to save the authenticated browser state and reuse it in multiple tests without logging in repeatedly.

Q2. Why do we use storageState?

We use `storageState` to save and reuse authentication information such as cookies and local storage.

It reduces repeated login operations and improves test execution speed.

Q3. Where should we store authentication files?

We generally store authentication files inside:

`playwright/.auth/`

We should add this directory to `.gitignore` because it contains sensitive authentication information.

Q4. What is the difference between browser context and browser page?

- Browser: Represents the browser instance.
- Browser Context: Represents an isolated browser session.
- Page: Represents an individual tab inside a browser context.

Q5. What is the difference between `scope: 'test'` and `scope: 'worker'`?

Test scope	Worker scope
Runs once per test	Runs once per worker
Recreated for each test	Shared by tests in that worker
Useful for test-specific data	Useful for worker-level authentication

Q6. What is `parallelIndex`?

`parallelIndex` identifies the current parallel worker.

We use it to assign different accounts and authentication files to different workers.

Q7. How do you handle authentication in parallel execution?

If tests modify shared server-side data, I use one account per worker.

Each worker authenticates once and reuses its own storage state to prevent conflicts between parallel tests.

Q8. How do you test admin and user roles in Playwright?

I create separate authentication state files for admin and user.

Then I use different browser contexts with their respective storage states.

Q9. Can we authenticate using API instead of UI?

Yes. If the application provides an API authentication endpoint, we can send a POST request using Playwright's API request context, save the resulting authentication state, and reuse it in browser tests.

Q10. Does Playwright automatically save session storage?

No. Playwright's normal `storageState()` does not automatically persist session storage. We can manually save and restore it using `page.evaluate()` and `context.addInitScript()`.

Q11. What happens if the authentication state expires?

The tests may fail because the saved cookies or tokens are no longer valid.

We need to authenticate again and generate a fresh storage state file.

Q12. How do you test a login page when the project already uses authentication?

I override the storage state using:

```
test.use({
  storageState: {
    cookies: [],
    origins: []
```

```
  }
});
```

This starts the test without saved authentication.

13. Scenario-based interview questions

Scenario 1: You have 50 test cases. Do you log in separately in all 50 tests?

Answer: No. I create an authentication setup project, log in once, save the browser storage state and reuse it in all 50 tests. This reduces execution time and duplicate login code, provided the tests can safely share the same account.

Scenario 2: Two tests update the same user's profile simultaneously. What will you do?

Answer: I will use separate test accounts for separate parallel workers. This prevents one test from modifying data that another test is verifying.

Scenario 3: You need to verify admin and normal-user permissions in the same test. How will you implement it?

Answer: I will create separate browser contexts using admin and user authentication files. Then I will create separate pages and perform operations using both roles in the same test.

Scenario 4: Login takes 10 seconds, and you have 100 test cases. How can you optimize execution?

Answer: I will reuse saved authentication state rather than performing UI login in every test. If the application supports API authentication, I can authenticate through the API to make setup even faster.

Scenario 5: Your authentication file exists, but the tests still redirect to the login page. What could be the reason?

Answer:

- Authentication cookies or tokens may have expired.
- The wrong storage state file may be configured.
- The application may store authentication in session storage.
- The saved state may belong to another environment or domain.

- The API authentication may not have established browser-compatible authentication.

I would verify the saved state and the application's authentication mechanism.

14. Practical coding exercises

Try these three exercises to strengthen your understanding.

Test your understanding

3 questions

1. You want to log in once and reuse authentication in all independent tests. Which approach should you use?

One account per worker
Shared account with setup project
Create a new context for every login

Incorrect – Shared account with setup project

A setup project authenticates once, saves storage state and lets independent tests reuse it.

2. Two parallel tests modify the same account's settings. What is the better approach?

Use one shared account
Disable all assertions
Use one account per parallel worker

Incorrect – Use one account per parallel worker

Separate worker accounts prevent tests from interfering with each other's server-side data.

3. Which property identifies a parallel worker?

parallelIndex
storageState
testMatch

Correct

test.info().parallelIndex provides the worker's parallel index.

Your score: 1/3

Try again

Final interview revision – Remember these 5 points

1. Authentication reuse: Login once, save storage state and reuse it.
2. Shared account: Use when tests do not interfere with one another.
3. One account per worker: Use when tests modify shared server-side data.
4. Multiple roles: Use separate storage state files and browser contexts.
5. Security: Never commit authentication files or credentials to GitHub.

Most important interview topic: Be prepared to explain the difference between shared authentication and one account per parallel worker, with a real-time example.

Playwright Auto-waiting – Complete Explanation

Auto-waiting is a very important Playwright concept for QA Automation and SDET interviews.

The main purpose of auto-waiting is to make automation scripts more reliable by automatically waiting until an element is ready before performing an action.

Let's understand it with simple examples, code, real-time scenarios and interview questions.

1. What is Auto-waiting?

Auto-waiting is a Playwright feature that automatically waits for an element to become ready before performing an action.

For example, imagine you are testing a login page.

1. Open the login page.

2. Enter username.
3. Enter password.
4. Click the Login button.

Sometimes the Login button takes 3 seconds to become enabled because the application is processing information.

In Selenium, testers commonly use explicit waits:

```
WebDriverWait wait = new WebDriverWait(driver,
Duration.ofSeconds(10));

wait.until(ExpectedConditions.elementToBeClickable(loginButton));

loginButton.click();
```

In Playwright:

```
await page.getByRole('button', { name: 'Login' }).click();
```

Playwright automatically waits for the required actionability checks before clicking.

You generally do not need to write a separate wait before every action.

Key point
Playwright automatically waits for the required conditions, but it does not wait forever. If the conditions are not met within the timeout, it throws a `TimeoutError`.

2. Why do we need Auto-waiting?

Consider this real-time scenario.

You are automating an e-commerce website.

You click the Add to Cart button, but:

- The page is still loading.
- The button is temporarily disabled.
- A loading animation is running.
- A popup is covering the button.

If Playwright clicks before the button is ready, the test may fail.

Auto-waiting helps prevent these timing-related failures.

Without auto-waiting

```
Test starts
    ↓
Find button
    ↓
Immediately click
    ↓
Button not ready
    ↓
Test failure
```

With auto-waiting

```
Test starts
    ↓
Find button
    ↓
Check actionability
    ↓
Wait until conditions pass
    ↓
Click button
    ↓
Test continues
```

3. Five important actionability checks

Before performing actions such as `click()`, Playwright checks whether the element is ready.

You should remember these five terms:

1. Visible
2. Stable
3. Receives Events

4. Enabled
5. Editable

Let's understand each one.

3.1 Visible

An element is considered visible when:

- It has a non-empty bounding box.
- Its computed CSS visibility is not `hidden`.

Example

```
await page.getByRole('button', { name: 'Login' }).click();
```

Before clicking, Playwright checks whether the Login button is visible.

Consider these HTML examples:

```
<button>Login</button>
```

This button is visible.

```
<button style="display:none">Login</button>
```

This button is not visible.

```
<button style="visibility:hidden">Login</button>
```

This button is not visible.

Important interview point: An element with `opacity: 0` is still considered visible by Playwright's visibility definition.

CSS property	Playwright considers it visible?
`display: none`	No
`visibility: hidden`	No

`opacity: 0`	Yes
Zero-size element	No
Normal visible element	Yes

3.2 Stable

An element is considered stable when it maintains the same bounding box for at least two consecutive animation frames.

In simple words:

The element should not be moving or changing its position.

Real-time example

Suppose you are automating a website with an animated Login button.

When the page loads, the button moves from the top to the center.

Playwright waits until the button stops moving before clicking it.

```
await page.getByRole('button', { name: 'Login' }).click();
```

You do not need to manually wait for the animation to finish.

Remember: Stable means the element is not moving, not that the entire website has stopped loading.

3.3 Receives Events

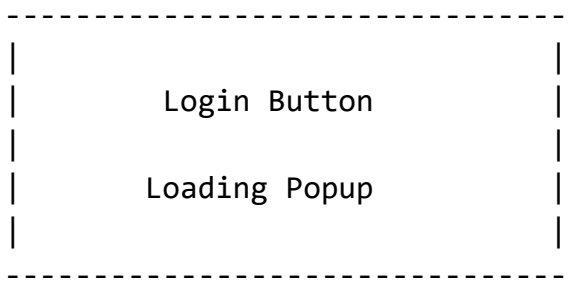
This means the element should receive the actual pointer event.

In simple words:

No other element should block or cover the target element at the action point.

Real-time example

Suppose a popup is displayed over your Login button.



Although the Login button exists and is visible, the popup may cover it.

Playwright checks whether the button actually receives the click event.

If the popup blocks it, Playwright waits for the obstruction to disappear.

Example:

```
await page.getByRole('button', { name: 'Login' }).click();
```

If another element covers the button until the timeout expires, the click fails with a timeout error.

3.4 Enabled

An element is considered enabled when it is not disabled.

Example

```
<button disabled>Sign Up</button>
```

This button is disabled.

Playwright will wait for it to become enabled before performing an action that requires the enabled state.

Example:

```
await page.getByRole('button', { name: 'Sign Up' }).click();
```

Suppose the application checks whether the username is available.

Initially:

```
Username checking...
Sign Up button → Disabled
```

After the server responds:

```
Username available
Sign Up button → Enabled
```

Playwright waits and clicks once the button becomes enabled.

You can also verify the enabled state:

```
await expect(
  page.getByRole('button', { name: 'Sign Up' })
).toBeEnabled();
```

3.5 Editable

An element is considered editable when:

- It is enabled.
- It is not readonly.

Editable checks are important for actions such as `fill()` and `clear()`.

Example

```
<input type="text" placeholder="Enter username">
```

This input is editable.

```
<input type="text" value="Komal" readonly>
```

This input is not editable.

Playwright example:

```
await page.getByPlaceholder('Enter username').fill('Komal');
```

Playwright checks whether the input is visible, enabled and editable before filling it.

4. Actionability checks table

This table is very important for interviews.

Playwright does not perform all five checks for every action.

Action	Visible	Stable	Receives Events	Enabled	Editable
click()	Yes	Yes	Yes	Yes	–
dblclick()	Yes	Yes	Yes	Yes	–
check()	Yes	Yes	Yes	Yes	–
setChecked()	Yes	Yes	Yes	Yes	–
uncheck()	Yes	Yes	Yes	Yes	–
hover()	Yes	Yes	Yes	–	–
dragTo()	Yes	Yes	Yes	–	–
screenshot()	Yes	Yes	–	–	–
fill()	Yes	–	–	Yes	Yes
clear()	Yes	–	–	Yes	Yes
selectOption()	Yes	–	–	Yes	–
selectText()	Yes	–	–	–	–
scrollIntoViewIfNeeded()	–	Yes	–	–	–
focus()	–	–	–	–	–
press()	–	–	–	–	–

`dispatchEvent()`	–	–	–	–	–
`setInputFiles()`	–	–	–	–	–

Interview tip: For `click()`, remember all four checks: Visible, Stable, Receives Events and Enabled.

For `fill()`, remember: Visible, Enabled and Editable.

5. What happens when an element is not ready?

Consider this example:

```
import { test, expect } from '@playwright/test';

test('Login functionality', async ({ page }) => {

  await page.goto('https://example.com/login');

  await page.getByLabel('Username').fill('komal');

  await page.getByLabel('Password').fill('password123');

  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page).toHaveURL(/dashboard/);

});
```

Let's understand the execution.

Step	Playwright action
1	Opens login page

2	Waits for username input to be ready
3	Fills username
4	Waits for password input to be ready
5	Fills password
6	Checks Login button actionability
7	Clicks Login
8	Waits for URL assertion to pass

Notice that we have not added `waitForTimeout()` anywhere.

Playwright automatically waits for the required conditions.

6. Forcing actions

Sometimes you may want Playwright to perform an action without checking certain actionability conditions.

For this, Playwright provides the `force` option.

Normal click

```
await page.getByRole('button', { name: 'Login' }).click();
```

Playwright performs the normal actionability checks.

Force click

```
await page.getByRole('button', { name: 'Login' }).click({
  force: true
});
```

Here, Playwright disables non-essential actionability checks.

For example, it will not check whether the target element receives pointer events.

Difference

Normal click	Force click
Performs normal actionability checks	Skips non-essential checks
Checks whether element receives events	Does not check whether it receives events
More closely represents user interaction	Can interact with elements that are obstructed
Preferred for normal automation	Use only when specifically needed

Real-time scenario

Suppose a transparent overlay is covering a button, but your test specifically needs to trigger the button's click handler.

You might consider:

```
await page.locator('#submit').click({ force: true });
```

However, be careful.

If a real user cannot click the button because it is covered, forcing the click may hide an actual application defect.

Interview answer: I prefer normal clicks and use force only when there is a valid testing reason. I do not use force simply to fix flaky tests.

7. Auto-retrying Assertions

This is another important feature related to auto-waiting.

Playwright provides auto-retrying assertions that wait until a condition becomes true.

Example

```
await expect(
  page.getByText('Login successful')
).toBeVisible();
```

Suppose the success message appears after 3 seconds.

Playwright automatically retries the assertion until:

- The message becomes visible, or
- The assertion timeout expires.

The default timeout for Playwright's `expect` assertions is 5 seconds.

You can configure it in `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  expect: {
    timeout: 10000,
  },

});
```

Now the default assertion timeout is 10 seconds.

You can also configure the timeout for an individual assertion:

```
await expect(
  page.getByText('Login successful')
).toBeVisible({ timeout: 10000 });
```

Important assertions

Assertion	Purpose

`toBeAttached()`	Element is attached to DOM
`toBeChecked()`	Checkbox is checked
`toBeDisabled()`	Element is disabled
`toBeEditable()`	Element is editable
`toBeEmpty()`	Element has no content
`toBeEnabled()`	Element is enabled
`toBeFocused()`	Element has focus
`toBeHidden()`	Element is hidden
`toBeInViewport()`	Element intersects viewport
`toBeVisible()`	Element is visible
`toContainText()`	Element contains specified text
`toHaveAttribute()`	Element has specified attribute
`toHaveClass()`	Element has specified CSS class
`toHaveCount()`	Locator matches expected number of elements
`toHaveCSS()`	Element has expected CSS property
`toHaveId()`	Element has expected ID

`toHaveJSProperty()`	Element has expected JS property
`toHaveText()`	Element has expected text
`toHaveValue()`	Input has expected value
`toHaveValues()`	Select element has expected selected values
`toHaveTitle()`	Page has expected title
`toHaveURL()`	Page has expected URL
`toBeOK()`	HTTP response has an OK status

Real-time example: Verify Add to Cart

```
test('Add product to cart', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByRole('button', { name: 'Add to Cart' }).click();

  await expect(
    page.getByText('Added to cart successfully')
  ).toBeVisible();

});
```

Even if the success message takes some time to appear, Playwright keeps checking the assertion.

Important difference:

- Auto-waiting waits for an action to become possible.
- Auto-retrying assertions wait for a condition to become true.

8. Auto-waiting vs waitForTimeout()

This is a very common interview question.

`waitForTimeout()`

```
await page.waitForTimeout(5000);
```

This pauses execution for exactly 5 seconds.

It does not check whether the element is ready.

Even if the element becomes ready after 1 second, the test still waits for the remaining 4 seconds.

Auto-waiting

```
await page.getByRole('button', { name: 'Login' }).click();
```

Playwright waits only as long as necessary for the required actionability conditions, subject to the timeout.

Auto-waiting	waitForTimeout()
Waits for required conditions	Waits for fixed duration
More efficient	Can waste execution time
Reduces timing-related failures	Can still fail if the fixed delay is insufficient
Preferred approach	Avoid for normal synchronization

Best practice: Prefer locator actions and auto-retrying assertions over hard-coded waits.

9. Auto-waiting vs Selenium explicit wait

Since you have a Selenium background, remember this comparison.

Selenium	Playwright
`WebDriverWait`	Built-in auto-waiting
`ExpectedConditions.elementToBeClickable()`	`locator.click()`
`visibilityOfElementLocated()`	`expect(locator).toBeVisible()`
`elementToBeSelected()`	`expect(locator).toBeChecked()`
`isEnabled()`	`expect(locator).toBeEnabled()`

Selenium example:

```
WebDriverWait wait = new WebDriverWait(
    driver, Duration.ofSeconds(10)
);

WebElement loginButton = wait.until(
    ExpectedConditions.elementToBeClickable(
        By.id("login")
    )
);

loginButton.click();
```

Playwright equivalent:

```
await page.locator('#login').click();
```

Playwright handles the actionability checks automatically.

Note that Playwright also supports explicit waiting methods when a specific condition requires them.

10. Real-time QA automation scenario

Imagine you are testing an e-commerce checkout page.

Requirements:

1. Enter customer name.
2. Enter address.
3. Select payment method.
4. Click Place Order.
5. Verify success message.

```
import { test, expect } from '@playwright/test';

test('Checkout functionality', async ({ page }) => {

  await page.goto('https://example.com/checkout');

  // Enter customer name
  await page.getByLabel('Customer Name').fill('Komal');

  // Enter address
  await page.getByLabel('Address').fill('Pune');

  // Select payment method
  await page.getByLabel('Payment Method')
    .selectOption('credit-card');

  // Click Place Order
  await page.getByRole('button', { name: 'Place Order' }).click();

  // Verify order confirmation
  await expect(
    page.getByText('Order placed successfully')
  ).toBeVisible();

});
```

Where does auto-waiting happen?

- `fill()` waits for the input to be visible, enabled and editable.
- `selectOption()` waits for the select element to be visible and enabled.
- `click()` waits for the button's required actionability checks.
- `toBeVisible()` retries until the confirmation message appears or the timeout expires.

This is a good practical example to explain in interviews.

11. Interview questions and answers

Q1. What is auto-waiting in Playwright?

Auto-waiting is a built-in Playwright feature that waits for the required actionability conditions before performing an action. It helps reduce timing-related test failures and avoids unnecessary hard-coded waits.

Q2. What are the five actionability checks?

The five important checks are:

1. Visible
2. Stable
3. Receives Events
4. Enabled
5. Editable

Different actions require different combinations of checks.

Q3. What is the difference between Visible and Stable?

- Visible means the element has a non-empty bounding box and is not `visibility: hidden`.

- Stable means the element maintains the same bounding box for at least two consecutive animation frames.

Q4. What does Receives Events mean?

It means that the target element receives the pointer event at the action point. Another element, such as a popup or overlay, should not intercept the click.

Q5. Does Playwright check all five conditions for every action?

No. Each action has its own actionability requirements.

For example:

- `click()` checks Visible, Stable, Receives Events and Enabled.
- `fill()` checks Visible, Enabled and Editable.
- `hover()` checks Visible, Stable and Receives Events.

Q6. What is force click?

Force click uses `{ force: true }` to disable non-essential actionability checks.

```
await page.locator('#login').click({ force: true });
```

It is useful in specific scenarios but should not be used as a default solution for flaky tests.

Q7. What happens if an element never becomes ready?

If the required actionability checks do not pass before the timeout expires, Playwright throws a `TimeoutError`.

Q8. What is the difference between auto-waiting and auto-retrying assertions?

Auto-waiting waits for an element to become ready before performing an action.

Auto-retrying assertions repeatedly check whether an expected condition becomes true.

Q9. Is opacity 0 considered visible in Playwright?

Yes. According to Playwright's visibility definition, an element with `opacity: 0` is still considered visible if it has a non-empty bounding box and is not `visibility: hidden`.

Q10. Why should we avoid waitForTimeout()?

Because it waits for a fixed duration regardless of whether the page is ready. It can increase execution time and still cause failures if the chosen delay is insufficient.

12. Scenario-based interview questions

Scenario 1: Your Login button is visible, but Playwright cannot click it. What could be the reason?

Answer:

The button may be covered by another element, such as a popup or overlay. It might also be disabled or moving. I would inspect the actionability checks and resolve the actual issue instead of immediately using force click.

Scenario 2: A success message appears after 4 seconds. How do you verify it?

```
await expect(
  page.getByText('Login successful')
).toBeVisible({ timeout: 10000 });
```

This assertion retries until the message appears or the timeout expires.

Scenario 3: Your test fails because a button is disabled initially but becomes enabled after an API response. What will you do?

Answer:

I will use the normal locator click. Playwright automatically waits for the button to become enabled before clicking it.

```
await page.getByRole('button', { name: 'Submit' }).click();
```

Scenario 4: An element has `opacity: 0`. Will Playwright consider it visible?

Answer:

Yes, if it has a non-empty bounding box and its computed visibility is not hidden. However, whether it receives pointer events is a separate check.

Scenario 5: Your test is failing intermittently because the page takes different amounts of time to load. How will you handle it?

Answer:

I will use Playwright's auto-waiting and auto-retrying assertions instead of fixed waits. I will also check whether the application has a genuine synchronization or performance issue.

13. Quick revision notes

Remember these important points

1. Playwright automatically waits before performing actions.
2. Click requires Visible, Stable, Receives Events and Enabled.
3. Fill requires Visible, Enabled and Editable.
4. Stable means the element's bounding box stays unchanged for two consecutive animation frames.
5. `force: true` skips non-essential actionability checks.
6. Auto-retrying assertions wait for expected conditions.
7. Default assertion timeout is 5 seconds.
8. Prefer auto-waiting over `waitForTimeout()`.

14. Practice questions

Auto-waiting quiz

5 questions

1. Which actionability checks does locator.click() perform?

Visible and Enabled only
Visible, Stable, Receives Events and Enabled
Visible, Editable and Stable

Correct

A click requires all four checks: Visible, Stable, Receives Events and Enabled.

2. An element has opacity: 0. Is it considered visible by Playwright?

Yes, if it has a non-empty bounding box and is not visibility:hidden
No, opacity 0 always means hidden
Only when force:true is used

Correct

Opacity is not part of Playwright's visibility definition.

3. Which action checks whether an element is Editable?

click()
hover()
fill()

Correct

fill() checks visibility, enabled state and editability.

4. What happens when an actionability check does not pass within the timeout?

Playwright clicks anyway
Playwright throws a TimeoutError
Playwright automatically skips the test

Correct

The action fails with a TimeoutError when the required checks do not pass in time.

5. What is the recommended alternative to a fixed 5-second wait?

Use force click
Use auto-waiting and auto-retrying assertions
Add another fixed wait

Correct answer: Use auto-waiting and auto-retrying assertions

Use Playwright's built-in waiting mechanisms to synchronize with the actual UI state.

Score: 4/5

Retry quiz

My interview preparation tip: Pay special attention to the difference between auto-waiting, auto-retrying assertions, explicit waits and `force: true`. Interviewers frequently ask these concepts together.

Playwright Best Practices – Complete Explanation

Best Practices is an important Playwright topic for QA Automation and SDET interviews.

It explains how to write reliable, maintainable, readable and less flaky automation tests.

Let's understand every topic from your documentation with simple explanations, code examples, real-time QA scenarios and interview questions.

1. What are Best Practices in Playwright?

Best practices are recommended methods for writing automation scripts so that:

- Tests are reliable.
- Tests do not fail unnecessarily.
- Code is reusable and maintainable.
- Tests execute efficiently.
- Debugging becomes easier.
- Tests behave consistently across browsers.

For example, instead of writing tests that depend on CSS classes or fixed waits, use user-facing locators and auto-retrying assertions.

2. Testing Philosophy

There are four important principles in the testing philosophy.

2.1 Test user-visible behavior

Meaning: Test what the actual user sees and interacts with, rather than internal application implementation details.

Suppose you are testing a login page.

A real user:

1. Opens the login page.
2. Enters username.
3. Enters password.
4. Clicks Login.
5. Sees the dashboard.

Your automation should verify the same user-facing behavior.

Bad approach

```
await page.locator('.login-btn-blue').click();
```

This depends on a CSS class that may change when the UI design changes.

Good approach

```
await page.getByRole('button', { name: 'Login' }).click();
```

This identifies the button using its accessible role and name.

Another example

Bad:

```
await expect(page.locator('.success-message'))
  .toBeVisible();
```

Better:

```
await expect(page.getByText('Login successful'))
  .toBeVisible();
```

Interview point: Automation should validate the application's behavior from the end user's perspective, not its internal implementation.

2.2 Make tests as isolated as possible

Meaning: Every test should run independently without depending on another test.

Consider two tests:

- Test 1: Create a user.
- Test 2: Verify the user details.

If Test 2 depends on Test 1 creating the user, Test 2 may fail when executed alone.

A better approach is to prepare the required data independently for each test.

Example using beforeEach()

```
import { test } from '@playwright/test';

test.beforeEach(async ({ page }) => {

  await page.goto('https://github.com/login');

  await page.getByLabel('Username or email address')
    .fill('username');

  await page.getByLabel('Password')
    .fill('password');

  await page.getByRole('button', { name: 'Sign in' })
    .click();

});

test('first test', async ({ page }) => {

  // Test 1: Verify profile page

});

test('second test', async ({ page }) => {
```

```
  // Test 2: Verify dashboard

});
```

What is beforeEach()?

`beforeEach()` is a Playwright hook.

It executes before every test in its scope.

For example:

```
beforeEach()
    ↓
Test 1
    ↓
beforeEach()
    ↓
Test 2
    ↓
beforeEach()
    ↓
Test 3
```

Each test gets its own isolated browser context and page through Playwright's fixtures.

Other important hooks

Hook	When does it execute?
`beforeAll()`	Once before all tests in its scope
`beforeEach()`	Before every test
`afterEach()`	After every test
`afterAll()`	Once after all tests in its scope

Important: If you want to avoid repeating UI login in every test, you can use the authentication setup project and saved storage state that we discussed in the Authentication topic.

Test isolation and authentication reuse are compatible: each test can have its own browser context while reusing the same saved login state.

2.3 Avoid testing third-party dependencies

Meaning: Test the functionality that your application controls. Avoid making your test depend on external websites or third-party services that you cannot control.

Real-time example

Suppose your application displays weather information from a third-party weather API.

Your test should not depend on whether the real weather service is available.

Instead, you can mock the API response.

Example

```
import { test, expect } from '@playwright/test';

test('verify weather information', async ({ page }) => {

  await page.route('**/api/weather', async route => {

    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify({
        city: 'Pune',
        temperature: 28
      })
    });

  });

  await page.goto('https://example.com');

  await expect(page.getByText('Pune')).toBeVisible();
```

```
  await expect(page.getByText('28')).toBeVisible();

});
```

Understand these methods

Method	Meaning
`page.route()`	Intercepts matching network requests
`route.fulfill()`	Provides a mock response
`status`	HTTP response status
`contentType`	Response content type
`body`	Response data

For example:

```
await route.fulfill({
  status: 200,
  body: JSON.stringify(testData)
});
```

Instead of calling the actual external API, Playwright provides the test data.

Why is this useful?

- Tests execute faster.
- External server failures do not affect your tests.
- You control the response.
- Tests become more predictable.

Note: You should still test your integration with real third-party services separately where that integration is part of your application's responsibility.

2.4 Testing with a database

If your application uses a database, make sure the test data is controlled.

Example:

Suppose you are testing an employee management application.

Your test expects:

```
Employee ID: 101
Employee Name: Komal
Department: QA
```

If someone changes or deletes this database record, your test may fail.

Best practices

- Use a staging or test environment.
- Prepare predictable test data.
- Clean up test data after execution when appropriate.
- Avoid using production data for destructive tests.
- Keep browser and operating system versions consistent for visual regression testing.

3. Use Locators

Very important for interviews

Locators are used to identify elements on a webpage.

Playwright recommends using built-in locators because they provide auto-waiting and retryability.

Recommended locators

Locator	Purpose	Example
`getByRole()`	Finds by accessible role	Button, link, heading
`getByText()`	Finds by visible text	Welcome
`getByLabel()`	Finds form elements by label	Username
`getByPlaceholder()`	Finds by placeholder	Enter email
`getByTestId()`	Finds by test ID	`data-testid`
`getByAltText()`	Finds images by alt text	Company logo
`getByTitle()`	Finds by title attribute	Tooltip

Example

```
await page.getByRole('button', { name: 'Submit' }).click();
```

This finds a button with the accessible name Submit.

Why use locators?

- Automatic waiting.
- Automatic retrying.
- More readable test code.
- More resilient tests.

4. Use Chaining and Filtering

Sometimes a webpage contains multiple elements with the same text or role.

You need to narrow down the locator to identify the correct element.

This is where chaining and filtering are useful.

Real-time example

Suppose an e-commerce website displays three products.

Product	Action
Product 1	Add to cart
Product 2	Add to cart
Product 3	Add to cart

You want to add only Product 2 to the cart.

Step 1: Find the product

```
const product = page.getByRole('listitem')
  .filter({ hasText: 'Product 2' });
```

Explanation:

- `getByRole('listitem')`: Finds product list items.
- `filter()`: Filters the list items.
- `hasText`: Finds the item containing Product 2.

Step 2: Find the button inside that product

```
await product.getByRole('button', {
  name: 'Add to cart'
}).click();
```

Complete example

```
await page
  .getByRole('listitem')
```

```
  .filter({ hasText: 'Product 2' })
  .getByRole('button', { name: 'Add to cart' })
  .click();
```

Important methods

Method	Purpose
`filter()`	Filters matching locators
`hasText`	Filters using text
`has`	Filters using another locator
`getByRole()`	Finds elements by role
Chaining	Narrows down element selection

Interview scenario: If three Add to Cart buttons exist, how will you click the button for a specific product?

Answer: I will first locate the product container using `filter({ hasText: 'Product 2' })`, then locate and click the Add to Cart button inside that container.

5. Prefer User-facing Attributes over XPath or CSS Selectors

This is one of the most important locator best practices.

Bad approach – CSS selector

```
await page.locator(
  'button.buttonIcon.episode-actions-later'
).click();
```

Why is this not preferred?

Because the designer might change the CSS class.

Your automation script may then fail even though the application functionality is working correctly.

Better approach

```
await page.getByRole('button', {
  name: 'Submit'
}).click();
```

CSS vs XPath vs Playwright locators

Locator	Example	Recommendation
Role	`getByRole('button', { name: 'Login' })`	Highly recommended
Label	`getByLabel('Username')`	Highly recommended
Text	`getByText('Welcome')`	Recommended
Test ID	`getByTestId('login-button')`	Recommended when needed
CSS	`locator('#login')`	Use when appropriate
XPath	`locator('//button[@id="login"]')`	Avoid when a better locator exists

Important: CSS and XPath are not always wrong. Sometimes they are necessary, especially for elements without suitable user-facing attributes. But prefer locators based on the user's experience or an explicit test contract.

6. Generate Locators using Codegen

Playwright provides a test generator that can help you generate automation code and locators.

Command

```
npx playwright codegen https://playwright.dev
```

What happens after executing it?

1. A browser window opens.
2. Playwright Inspector opens.
3. You can interact with the webpage.
4. Playwright records your actions.
5. It generates corresponding automation code and locators.

How to pick a locator

1. Run the Codegen command.
2. Stop recording using the Record button.
3. Click Pick Locator.
4. Hover over an element.
5. Playwright highlights the matching locator.
6. Click the element to add the locator to the Inspector.
7. Copy the generated locator into your test.

Test generator | Playwright
playwright.dev

Example generated locator

```
page.getByRole('button', { name: 'Get started' });
```

Important interview point

Codegen helps generate locators, but you should review the generated code and ensure it is readable, unique and reliable.

VS Code extension

You can also use the Playwright VS Code extension to:

- Generate tests.
- Pick locators.
- Run tests.
- Debug tests.
- View test results.

7. Use Web-First Assertions

Web-first assertions are Playwright assertions that automatically wait and retry until the expected condition is satisfied or the timeout expires.

Bad approach

```
expect(
  await page.getByText('Welcome').isVisible()
).toBe(true);
```

Why is this not recommended?

`isVisible()` returns a boolean immediately. It does not keep retrying until the element becomes visible.

Good approach

```
await expect(
  page.getByText('Welcome')
).toBeVisible();
```

Here, Playwright waits and retries the assertion.

Real-time example

Suppose a success message appears 2 seconds after clicking Login.

```
await page.getByRole('button', { name: 'Login' }).click();

await expect(
  page.getByText('Welcome Komal')
).toBeVisible();
```

Playwright waits for the expected message.

Difference

Manual assertion	Web-first assertion
`isVisible()`	`toBeVisible()`
Returns current state	Retries until expected state
Can fail due to timing	Better synchronization
No automatic retry	Automatic retry

Remember: Use `await expect(locator).toBeVisible()` instead of `expect(await locator.isVisible()).toBe(true)` when you want retry behavior.

8. Configure Debugging

Debugging means identifying and fixing problems in your automation scripts.

Playwright provides different debugging options.

8.1 Local debugging using VS Code

Install the official Playwright VS Code extension.

You can:

- Run individual tests.
- Set breakpoints.

- Pause test execution.
- Inspect locators.
- View matching elements.
- Step through the test.

Debugging Tests | Playwright
playwright.dev

8.2 Debug using Playwright Inspector

Command:

```
npx playwright test --debug
```

This opens the Playwright Inspector and browser.

You can:

- Execute tests step by step.
- Pause execution.
- Inspect locators.
- View actionability logs.
- Edit locators and check their matches.

Debug a specific test

```
npx playwright test example.spec.ts:9 --debug
```

Here:

- `example.spec.ts` is the test file.
- 9 is the line number.
- `--debug` starts the debugger.

8.3 Debugging on CI using Trace Viewer

CI means Continuous Integration.

Examples include:

- GitHub Actions
- Jenkins
- Azure DevOps

When tests fail on CI, it can be difficult to reproduce the failure on your local machine.

Playwright provides Trace Viewer.

It helps you inspect:

- Test execution timeline.
- Actions performed.
- DOM snapshots.
- Network requests.
- Screenshots.
- Errors and logs.

Trace viewer | Playwright Python
playwright.dev

Configure trace in Playwright

```
import { defineConfig } from '@playwright/test';

export default defineConfig({

  use: {
    trace: 'on-first-retry',
  },

});
```

This records a trace on the first retry of a failed test.

Other trace options

Option	Meaning
`off`	No trace
on	Record trace for every test
`on-first-retry`	Record on first retry
`on-all-retries`	Record on retries
`retain-on-failure`	Retain traces for failed tests

For debugging locally:

```
npx playwright test --trace on
```

View the HTML report:

```
npx playwright show-report
```

Interview point: For CI failures, Trace Viewer is especially useful because it provides a detailed execution history, including DOM snapshots and network information.

9. Use Playwright's Tooling

Playwright provides several tools to improve productivity.

Tool	Purpose
VS Code extension	Write, run and debug tests

Codegen	Generate test scripts and locators
Trace Viewer	Debug test execution
UI Mode	Run and debug tests with a time-travel experience
HTML Report	View test execution results
TypeScript support	Catch coding and type errors

What is UI Mode?

UI Mode allows you to explore and run tests interactively.

You can:

- View all test files.
- Run individual tests.
- Watch tests.
- Debug tests.
- Inspect test execution.
- View test history and time-travel through actions.

Command:

```
npx playwright test --ui
```

TypeScript support

Playwright supports TypeScript directly.

Example:

```
let username: string = 'Komal';
```

```
let age: number = 25;

let isLoggedIn: boolean = true;
```

TypeScript helps identify incorrect types and coding errors before test execution.

10. Test Across All Browsers

Your application should work consistently across different browsers.

Playwright supports three browser engines:

1. Chromium
2. Firefox
3. WebKit

Example configuration

playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({

  projects: [

    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },

    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },

    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
```

```
  ],
});
```

Explanation

Project	Browser engine
Chromium	Chromium
Firefox	Firefox
WebKit	WebKit

You can execute the tests across all three projects.

```
npx playwright test
```

Or execute only Chromium:

```
npx playwright test --project=chromium
```

Interview point: Cross-browser testing helps identify browser-specific UI and functionality issues.

11. Keep Playwright Dependency Up to Date

Playwright regularly releases updates.

Updating Playwright helps you use newer browser versions, fixes and features.

Check installed version

```
npx playwright --version
```

Update Playwright

```
npm install -D @playwright/test@latest
```

After updating, install the corresponding browser binaries:

```
npx playwright install
```

Why update?

- Bug fixes.
- Improved browser compatibility.
- New features.
- Better performance.
- Updated browser versions.

12. Run Tests on CI

CI stands for Continuous Integration.

It automatically runs your tests when developers push code or create pull requests.

Example workflow

```
Developer writes code
        ↓
Pushes code to GitHub
        ↓
GitHub Actions starts
        ↓
Playwright tests execute
        ↓
Tests pass or fail
        ↓
Report is generated
```

Why use CI?

- Automatically verifies code changes.
- Detects bugs early.
- Reduces manual testing.

- Helps prevent broken code from reaching production.

Common CI tools

- GitHub Actions
- Jenkins
- Azure DevOps
- GitLab CI

Playwright can generate a GitHub Actions workflow when you create a new project using its project initializer.

13. Optimize Browser Downloads on CI

When executing tests in CI, install only the browsers that you need.

Install all browsers

```
npx playwright install --with-deps
```

Install only Chromium

```
npx playwright install chromium --with-deps
```

Why?

If your project only tests Chromium, installing Firefox and WebKit unnecessarily consumes time and disk space.

Installation	Result
`playwright install`	Installs supported browser binaries
`playwright install chromium`	Installs Chromium

`playwright install firefox`	Installs Firefox
`playwright install webkit`	Installs WebKit

14. Lint Your Tests

Linting helps identify coding mistakes and maintain code quality.

Playwright recommends using TypeScript and ESLint.

What is ESLint?

ESLint is a tool that analyzes JavaScript and TypeScript code to identify potential problems and enforce coding standards.

For example, it can help identify:

- Unused variables.
- Incorrect code patterns.
- Missing `await`.
- Potential coding errors.

Important rule

`@typescript-eslint/no-floating-promises`

This rule helps identify promises that are not properly handled.

Example – Incorrect

```
test('login', async ({ page }) => {

  page.goto('https://example.com');

});
```

Here, the asynchronous operation is not awaited.

Correct

```
test('login', async ({ page }) => {

  await page.goto('https://example.com');

});
```

TypeScript check

```
npx tsc --noEmit
```

This checks TypeScript errors without generating JavaScript files.

Interview point: Missing `await` can cause unpredictable test execution because Playwright actions are asynchronous.

15. Use Parallelism and Sharding

This is an important topic for SDET interviews.

15.1 Parallelism

Parallelism means executing multiple tests simultaneously instead of executing them one by one.

Example:

Without parallelism:

```
Test 1 → Test 2 → Test 3 → Test 4
```

With parallelism:

```
Worker 1 → Test 1 and Test 2
Worker 2 → Test 3 and Test 4
```

Playwright supports parallel execution.

Example

```
import { test } from '@playwright/test';

test.describe.configure({
  mode: 'parallel'
});

test('Test 1', async ({ page }) => {

  // Test 1 actions

});

test('Test 2', async ({ page }) => {

  // Test 2 actions

});
```

Here, tests inside the describe block can run in parallel.

Important execution modes

Mode	Meaning
`default`	Tests in a file run in order
`parallel`	Tests in the describe block can run in parallel
`serial`	Tests run sequentially and can depend on one another

Important: Parallel tests should be independent. They should not modify shared data in a way that causes conflicts.

15.2 Sharding

Sharding means splitting a large test suite into multiple parts and executing those parts on different machines.

Example

Suppose your project has 300 test cases.

You can divide them into three shards.

```
Total 300 tests

Machine 1 → Shard 1 → Part of test suite
Machine 2 → Shard 2 → Part of test suite
Machine 3 → Shard 3 → Part of test suite
```

Command

```
npx playwright test --shard=1/3
```

This runs the first shard out of three.

Other commands:

```
npx playwright test --shard=2/3
npx playwright test --shard=3/3
```

Parallelism vs Sharding

Parallelism	Sharding
Runs tests simultaneously using workers	Splits tests into multiple groups
Usually within one machine	Commonly across multiple machines
Uses worker processes	Uses shard assignments

Speeds up test execution	Speeds up large test suites across machines

16. Use Soft Assertions

This is another important interview topic.

Normally, if an assertion fails, Playwright stops executing the remaining steps in that test.

But sometimes you want to validate multiple things even if one assertion fails.

For that, Playwright provides `expect.soft()`.

Normal assertion

```
await expect(page.getByTestId('status'))
  .toHaveText('Success');
```

If this assertion fails, the test stops at that point.

Soft assertion

```
await expect.soft(page.getByTestId('status'))
  .toHaveText('Success');

await page.getByRole('link', { name: 'Next page' }).click();
```

If the first assertion fails, Playwright records the failure but continues executing the next step.

At the end, the test is marked failed if any soft assertions failed.

Real-time QA example

Suppose you are testing an employee profile page.

You need to verify:

- Employee name.
- Employee ID.

- Department.
- Job title.

You want to check all four fields, even if one is incorrect.

```
import { test, expect } from '@playwright/test';

test('Verify employee details', async ({ page }) => {

  await page.goto('https://example.com/employee');

  await expect.soft(
    page.getByText('Komal')
  ).toBeVisible();

  await expect.soft(
    page.getByText('EMP101')
  ).toBeVisible();

  await expect.soft(
    page.getByText('QA')
  ).toBeVisible();

  await expect.soft(
    page.getByText('Automation Tester')
  ).toBeVisible();

});
```

At the end, Playwright reports all soft assertion failures.

Normal vs Soft assertions

Normal assertion	Soft assertion
Uses `expect()`	Uses `expect.soft()`
Stops the test when it fails	Continues test execution

Suitable for critical conditions	Suitable for independent validations
Reports failure immediately	Collects failures and reports them

Important: Use soft assertions only when continuing after a failure is safe. If a failed check means the next action is unsafe, use a normal assertion.

17. Important Best Practices – Quick Revision

No.	Best practice	Why?
1	Test user-visible behavior	Represents actual user experience
2	Keep tests isolated	Prevents dependency and cascading failures
3	Avoid third-party dependencies	Reduces external failures
4	Control test database data	Makes results predictable
5	Use Playwright locators	Improves reliability
6	Use chaining and filtering	Finds specific elements
7	Prefer role and text locators	Avoids fragile DOM dependencies
8	Use Codegen	Helps generate locators
9	Use web-first assertions	Provides automatic retrying
10	Use debugging tools	Makes failures easier to investigate

11	Test across browsers	Ensures cross-browser compatibility
12	Keep dependencies updated	Provides fixes and new features
13	Run tests on CI	Detects problems early
14	Install only required browsers	Saves CI resources
15	Use ESLint and TypeScript checks	Detects coding mistakes
16	Use parallelism and sharding	Improves execution speed
17	Use soft assertions appropriately	Allows multiple validations

18. Interview Questions and Answers

Q1. What are the best practices in Playwright?

Best practices include using user-facing locators, keeping tests independent, using web-first assertions, avoiding hard-coded waits, mocking third-party APIs when appropriate, using debugging tools, and running tests across browsers.

Q2. Why should we use getByRole() instead of CSS selectors?

`getByRole()` identifies elements using their accessible role and name. It is generally more resilient to changes in CSS classes and DOM structure.

Q3. What is the purpose of beforeEach()?

`beforeEach()` is a hook that executes before every test in its scope. It is useful for common setup tasks such as navigating to a page or logging in.

Q4. What is the difference between beforeAll() and beforeEach()?

`beforeAll()` executes once before all tests in its scope, whereas `beforeEach()` executes before every individual test.

Q5. Why should tests be isolated?

Test isolation ensures that each test can run independently. It prevents one test's data or execution from affecting another test and makes debugging easier.

Q6. What is the difference between CSS and role locators?

CSS locators identify elements using DOM selectors and CSS attributes. Role locators identify elements using their accessible role and name, making them closer to how users interact with the application.

Q7. What is Codegen?

Codegen is Playwright's test generator. It records browser actions and generates automation code and locators.

Q8. What are web-first assertions?

Web-first assertions automatically wait and retry until the expected condition is met or the timeout expires.

Example:

```
await expect(page.getByText('Welcome')).toBeVisible();
```

Q9. What is Trace Viewer?

Trace Viewer is a debugging tool that helps investigate test execution using timelines, DOM snapshots, network requests, screenshots and logs.

Q10. What is the difference between parallelism and sharding?

Parallelism executes tests simultaneously using multiple workers. Sharding divides the test suite into multiple groups that can run on separate machines.

Q11. What is the purpose of soft assertions?

Soft assertions allow a test to continue after an assertion failure, collecting multiple failures before marking the test as failed.

Q12. Why do we use ESLint?

ESLint helps maintain code quality and identify potential issues, such as missing awaits, unused variables and incorrect coding patterns.

19. Scenario-based Interview Questions

Scenario 1: Your test passes locally but fails on CI. What will you do?

Answer:

I will inspect the CI logs, HTML report and Trace Viewer. I will check whether the failure is caused by timing, browser differences, environment configuration, test data or an application issue.

Scenario 2: Your locator fails whenever the developer changes CSS classes. How will you improve it?

Answer:

I will replace fragile CSS class-based locators with user-facing locators such as `getByRole()`, `getByLabel()` or `getByText()`, depending on the element.

Scenario 3: You have 500 test cases and execution takes too long. What will you do?

Answer:

I will use Playwright parallel execution and sharding to distribute the tests across multiple workers and machines. I will also avoid unnecessary browser installations and repeated login operations.

Scenario 4: Your application depends on an external API, but the API is frequently unavailable during testing. What will you do?

Answer:

I will use `page.route()` and `route.fulfill()` to mock predictable API responses for tests that verify the application's UI behavior. I will test the actual integration separately.

Scenario 5: You need to validate 10 independent fields on a profile page, even if some fields are incorrect. What will you use?

Answer:

I will use soft assertions with `expect.soft()` so the test continues and reports multiple failures at the end.

20. Practical coding questions

Here are five tasks to practise.

Best Practices Quiz

5 questions

1. Which locator is generally recommended for a Submit button?

page.locator('.blue-button')
page.getByRole('button', { name: 'Submit' })
page.locator('//div[3]/button')

2. Which hook executes before every test?

beforeAll()
afterEach()
beforeEach()

3. Which assertion automatically retries until the element becomes visible?

expect(locator.isVisible()).toBe(true)
await expect(locator).toBeVisible()
console.log(locator)

4. Which tool is most useful for investigating a failed test on CI?

Trace Viewer
Codegen
npm install

5. Which feature lets a test continue after an assertion failure?

force:true
expect.soft()
waitForTimeout()
Submit answers

Final interview preparation – Most important topics

For this Best Practices chapter, prioritise these topics:

1. `getByRole()` vs CSS vs XPath.
2. Test isolation and `beforeEach()`.
3. Web-first assertions vs manual assertions.
4. Codegen and locator generation.
5. Trace Viewer and debugging on CI.
6. Parallelism vs sharding.
7. Normal assertions vs soft assertions.
8. API mocking using `page.route()` and `route.fulfill()`.

Practical interview task: Be ready to write a small e-commerce test using a role-based locator, chained locator, web-first assertion and soft assertion. These concepts together demonstrate good Playwright coding practices.

Playwright – Browsers

Let's understand this complete section in simple English, with practical examples and interview questions suitable for 1–2 years of QA Automation / SDET experience.

1. Introduction to Playwright Browsers

What are browsers in Playwright?

A browser is an application used to open and access websites.

Examples:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

- Safari

Playwright supports three main browser engines:

1. Chromium – Used by Google Chrome, Microsoft Edge and other Chromium-based browsers.
2. Firefox – Browser engine used by Mozilla Firefox.
3. WebKit – Browser engine used by Safari.

Google Chrome - YouTube

Chromium

Used for testing websites in Chrome-like browsers.

mobilityinfo - Blog

Firefox

Used for testing websites in Firefox.

U.K. Court Blocks Mass Legal Action Over Google's Alleged Tracking of Safari Users - MacRumors

WebKit

Used for testing Safari-like browser behavior.

Important: What is a browser engine?

A browser engine is the internal technology responsible for processing HTML, CSS and JavaScript and displaying a webpage.

For example:

Browser	Engine
Google Chrome	Blink (Chromium project)
Microsoft Edge	Blink (Chromium project)

Mozilla Firefox	Gecko
Apple Safari	WebKit

Why do we test on different browsers?

Because the same website may behave differently in different browsers.

Example:

Suppose you are testing an e-commerce website.

- Login works in Chrome.
- Login works in Firefox.
- Payment button does not work in Safari.

This is called a cross-browser compatibility issue.

2. Install Browsers

Playwright requires specific browser versions to execute tests.

When you install Playwright, you also need to download its supported browser binaries.

Command

```
npx playwright install
```

Explanation:

- `npx` – Executes a package command.
- `playwright` – Playwright command-line tool.
- `install` – Downloads the required browsers.

By default, it installs Chromium, Firefox and WebKit.

Install a specific browser

```
npx playwright install chromium
```

Installs only Chromium.

```
npx playwright install firefox
```

Installs only Firefox.

```
npx playwright install webkit
```

Installs only WebKit.

View all supported installation options

```
npx playwright install --help
```

Real-time example

Suppose your project requires testing only on Chrome.

Instead of installing all three browsers, you can install Chromium only.

Interview tip: Browser installation is separate from installing the Playwright npm package. You need the browser binaries that match your Playwright version.

3. Install System Dependencies

What are system dependencies?

System dependencies are additional operating-system libraries and packages required to run browsers.

For example:

- Linux libraries
- Font packages
- Graphics libraries
- Audio and video-related libraries

Sometimes Playwright browsers cannot launch because some operating-system dependencies are missing.

Command

```
npx playwright install-deps
```

Installs the required system dependencies for supported browsers.

Install dependencies for one browser

```
npx playwright install-deps chromium
```

Installs system dependencies required by Chromium.

Install browser and dependencies together

```
npx playwright install --with-deps chromium
```

This command:

1. Installs Chromium.
2. Installs its required system dependencies.

Where is this useful?

Mostly in CI/CD environments such as Jenkins, GitHub Actions and Linux servers, where browsers may need additional system libraries.

Remember:

Command	Purpose
`npx playwright install`	Installs browsers
`npx playwright install-deps`	Installs OS dependencies
`npx playwright install --with-deps chromium`	Installs Chromium and its OS dependencies

4. Update Playwright Regularly

Playwright frequently releases new versions.

New versions may include:

- New features.
- Bug fixes.
- Updated browser binaries.
- Improved browser compatibility.

Step 1: Update Playwright

```
npm install -D @playwright/test@latest
```

Explanation:

- `npm install` – Installs a package.
- `-D` – Adds it as a development dependency.
- `@playwright/test` – Playwright testing package.
- `@latest` – Installs the latest published version.

Step 2: Install updated browsers

```
npx playwright install
```

Why?

Because the new Playwright version may require different browser binary versions.

Step 3: Check Playwright version

```
npx playwright --version
```

This displays the installed Playwright CLI version.

Interview scenario:

Q: You updated Playwright but your browser tests are failing to launch. What will you do?

Answer:

1. Check the installed Playwright version.
2. Run `npx playwright install`.
3. Make sure the required browser binaries are downloaded.
4. Run the tests again.

5. Configure Browsers

Playwright allows us to execute tests on multiple browsers using the `projects` section in `playwright.config.ts`.

What is a project in Playwright?

A project is a configuration that allows us to run the same test suite with different browser settings, devices or environments.

For example:

- Project 1: Chromium
- Project 2: Firefox
- Project 3: WebKit
- Project 4: Mobile Chrome

Example configuration

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
```

```
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
  ],
});
```

Understand the code line by line

1. Import defineConfig and devices

```
import { defineConfig, devices } from '@playwright/test';
```

- `defineConfig` – Helps define Playwright configuration.
- `devices` – Provides predefined browser and device settings.

2. Define projects

```
projects: [
```

The projects array contains multiple browser configurations.

3. Chromium project

```
{
  name: 'chromium',
  use: { ...devices['Desktop Chrome'] },
}
```

- `name` – Project name.
- `use` – Configuration options used by the project.
- `Desktop Chrome` – Predefined desktop Chrome-like device settings.
- `...` – Spread operator, which copies the properties from the predefined configuration.

The same explanation applies to Firefox and WebKit.

What happens when we run tests?

```
npx playwright test
```

Playwright executes the test suite against all configured projects by default.

Suppose you have one test:

```
import { test, expect } from '@playwright/test';

test('Verify page title', async ({ page }) => {
  await page.goto('https://example.com');

  await expect(page).toHaveTitle('Example Domain');
});
```

If you configure three projects:

- Chromium
- Firefox
- WebKit

The same test executes three times, once in each browser project.

Test execution

```
Verify page title
```

ファイル:Chromium Logo.svg - ウィキニュース

Chromium

Run
File:Firefox logo, 2017.svg - Wikimedia Commons

Firefox

Run
Safari : rapide et privé

WebKit

Run

One test file, executed against three browser projects.

Run tests on a specific browser

```
npx playwright test --project=firefox
```

This executes tests only in the Firefox project.

Other examples:

```
npx playwright test --project=chromium
npx playwright test --project=webkit
```

Important interview point: Projects help us perform cross-browser testing without writing separate test cases for each browser.

6. Mobile Browser Testing

Playwright also supports mobile device emulation.

It can emulate:

- Mobile screen size.
- Device viewport.
- Touch support.
- Device scale factor.
- Mobile browser settings.

Example:

```
{
  name: 'Mobile Chrome',
  use: { ...devices['Pixel 5'] },
},
{
  name: 'Mobile Safari',
  use: { ...devices['iPhone 12'] },
},
```

What does device emulation mean?

Suppose you have an e-commerce website.

You want to check whether:

- Login button is visible on mobile.
- Navigation menu works.
- Product images fit the screen.
- Add to Cart button is clickable.

You can test these scenarios using mobile device emulation.

Important: Mobile emulation is not the same as testing on a real physical mobile device. It emulates device and browser characteristics.

7. Google Chrome and Microsoft Edge

Playwright supports two types of Chromium-based browsers:

1. Playwright's bundled Chromium.
2. Official Google Chrome and Microsoft Edge installations.

What is the difference?

Playwright normally uses its own Chromium browser binary.

But sometimes we need to test the actual installed Google Chrome or Microsoft Edge.

For that, we use the `channel` option.

Google Chrome

```
{
  name: 'Google Chrome',
  use: {
    ...devices['Desktop Chrome'],
    channel: 'chrome',
  },
}
```

Microsoft Edge

```
{
  name: 'Microsoft Edge',
  use: {
    ...devices['Desktop Edge'],
    channel: 'msedge',
  },
}
```

What is channel?

A channel specifies which branded browser installation Playwright should launch.

Available channels include:

Channel	Browser
`chrome`	Google Chrome Stable
`chrome-beta`	Google Chrome Beta
`chrome-dev`	Google Chrome Dev
`chrome-canary`	Google Chrome Canary
`msedge`	Microsoft Edge Stable
`msedge-beta`	Microsoft Edge Beta
`msedge-dev`	Microsoft Edge Dev
`msedge-canary`	Microsoft Edge Canary

Important difference

Chromium	Google Chrome
Open-source browser project/build	Official branded browser
Installed by Playwright by default	Not installed by Playwright by default
Good for general automation	Useful for testing actual Chrome behavior

Install Google Chrome or Edge

```
npx playwright install chrome
npx playwright install msedge
```

Note: Installing these browsers can affect existing system installations because they use the operating system's default global installation locations.

8. When to Use Chromium vs Google Chrome or Edge?

This is an important interview topic.

Case 1: Normal automation testing

Use Playwright's default Chromium.

Why?

- It is easy to configure.
- It works well with Playwright.
- It helps identify upcoming browser compatibility issues.

Case 2: Regression testing

Use stable Chrome or Edge.

Why?

Your application should work correctly in the browser versions currently used by customers.

Case 3: Media codecs testing

Use official Chrome or Edge.

Why?

Official branded browsers may include media codecs that are not included in Playwright's bundled Chromium.

For example, when testing certain video or audio playback formats.

Case 4: Enterprise browser policies

Organizations sometimes apply security policies to Chrome and Edge.

These policies may restrict:

- Extensions.
- Network access.
- Browser automation.
- Proxy configurations.

In such cases, bundled Chromium may be easier for local automation testing.

9. Firefox

Playwright provides its own patched version of Firefox.

Important: Playwright does not support the normal branded Firefox installation in the same way because Playwright relies on specific browser patches.

Why does Playwright use a patched browser?

Because Playwright needs special communication and automation capabilities to control the browser reliably.

For interviews, remember:

- Playwright supports Firefox.
- It uses its own Firefox build.
- It does not use the branded Firefox installation.

10. WebKit

WebKit is the browser engine used by Safari.

Playwright provides its own WebKit build.

Important: Playwright does not directly automate branded Safari using its normal WebKit project.

Instead, it uses Playwright's WebKit build.

Why is WebKit useful?

Suppose your application works properly in Chrome but has a layout issue in Safari.

You can execute the same tests in WebKit to identify potential Safari compatibility issues.

Important difference

- Chromium is useful for Chrome-like browsers.
- Firefox is useful for Firefox.
- WebKit is useful for Safari compatibility testing.

For the closest Safari experience, testing WebKit on macOS is recommended, particularly for platform-dependent features such as media playback.

11. Install Browsers Behind a Firewall or Proxy

What is a proxy?

A proxy server acts as an intermediary between your computer and the internet.

In some companies, direct internet downloads are restricted.

Playwright normally downloads browser binaries from Microsoft's CDN.

If your company blocks direct downloads, you can configure a proxy.

Example

```
HTTPS_PROXY=https://192.0.2.1 npx playwright install
```

This tells Playwright to use the configured HTTPS proxy while downloading browsers.

Custom certificate

Sometimes a company uses its own certificate authority.

You might see this error:

```
self signed certificate in certificate chain
```

To configure the trusted root certificate:

```
export NODE_EXTRA_CA_CERTS="/path/to/cert.pem"
```

Download timeout

If browser downloading is slow or gets stuck:

```
PLAYWRIGHT_DOWNLOAD_CONNECTION_TIMEOUT=120000 npx playwright install
```

This increases the download idle timeout to 120 seconds.

Interview point: Proxy configuration is useful in corporate environments where browser downloads are restricted.

12. Download from Artifact Repository

What is an artifact repository?

An artifact repository is a central location where organizations store software packages and binary files.

For example, a company may maintain an internal repository for Playwright browser binaries.

Instead of downloading browsers directly from Microsoft's CDN, we can configure a custom download location.

Command

```
PLAYWRIGHT_DOWNLOAD_HOST=http://192.0.2.1 npx playwright install
```

Browser-specific download locations

```
PLAYWRIGHT_CHROMIUM_DOWNLOAD_HOST=http://203.0.113.3
PLAYWRIGHT_DOWNLOAD_HOST=http://192.0.2.1
npx playwright install
```

The browser-specific setting takes precedence over the general download host for that browser.

Interview point: This configuration is useful in enterprise environments with internal artifact repositories.

13. Managing Browser Binaries

What are browser binaries?

Browser binaries are the executable files and related resources required to launch a browser.

Playwright downloads browser binaries into operating-system-specific cache folders.

OS	Default browser location

Windows	`%USERPROFILE%\AppData\Local\ms-playwright`
macOS	`~/Library/Caches/ms-playwright`
Linux	`~/.cache/ms-playwright`

Why does Playwright maintain these folders?

To store downloaded browser versions so that tests can launch them without downloading browsers every time.

Change the browser installation location

```
PLAYWRIGHT_BROWSERS_PATH=$HOME/pw-browsers npx playwright install
```

This installs browsers into a custom folder.

Use the same location when running tests

```
PLAYWRIGHT_BROWSERS_PATH=$HOME/pw-browsers npx playwright test
```

This tells Playwright where to find the installed browsers.

Important: The same environment variable should be configured during installation and execution.

14. Hermetic Install

What is a hermetic install?

A hermetic install means installing Playwright browser binaries inside the local Playwright package directory instead of the default shared browser cache.

Command

```
PLAYWRIGHT_BROWSERS_PATH=0 npx playwright install
```

Browser binaries are stored in:

```
node_modules/playwright-core/.local-browsers
```

Difference

Normal installation	Hermetic installation
Uses OS-specific browser cache	Uses local package directory
Can share browser binaries across projects	Keeps binaries within the local package
Default behavior	Must be explicitly enabled

Interview tip: Hermetic installation is useful when you want browser binaries stored locally with the project dependencies.

15. Stale Browser Removal

What does stale mean?

Stale means old or no longer required.

Suppose:

- Playwright version 1 requires Chromium version A.
- You update Playwright.
- The new version requires Chromium version B.
- No installed Playwright package needs version A anymore.

Playwright can remove the unused browser version to save disk space.

This process is called browser garbage collection.

Disable automatic browser removal

```
PLAYWRIGHT_SKIP_BROWSER_GC=1
```

Or:

```
npx playwright install --no-remove
```

This prevents automatic removal of unused browser binaries.

List all installed browsers

```
npx playwright install --list
```

Uninstall browsers

```
npx playwright uninstall
```

Removes browser binaries associated with the current Playwright installation.

Uninstall browsers from all Playwright installations

```
npx playwright uninstall --all
```

16. Important Commands – Quick Revision

Command	Purpose
`npx playwright install`	Install all default browsers
`npx playwright install chromium`	Install Chromium
`npx playwright install firefox`	Install Firefox
`npx playwright install webkit`	Install WebKit
`npx playwright install-deps`	Install system dependencies
`npx playwright install --with-deps chromium`	Install Chromium with dependencies

`npm install -D @playwright/test@latest`	Update Playwright
`npx playwright --version`	Check version
`npx playwright test`	Run all configured projects
`npx playwright test --project=firefox`	Run Firefox project
`npx playwright install --list`	List installed browsers
`npx playwright uninstall`	Uninstall current installation's browsers
`npx playwright uninstall --all`	Uninstall all Playwright browser installations

17. Interview Questions and Answers

Here are the important interview questions based on this complete section.

Basic-level questions

Q1. Which browsers does Playwright support?

Answer:

Playwright supports three main browser engines:

1. Chromium
2. Firefox
3. WebKit

It also supports branded browsers such as Google Chrome and Microsoft Edge.

Q2. What is the command to install Playwright browsers?

Answer:

```
npx playwright install
```

It downloads the default supported browsers: Chromium, Firefox and WebKit.

Q3. What is Chromium?

Answer:

Chromium is an open-source browser project. Google Chrome and Microsoft Edge are based on Chromium.

Playwright uses its own supported Chromium build for browser automation.

Q4. What is WebKit?

Answer:

WebKit is a browser engine used by Apple Safari. Playwright uses its own patched WebKit build to perform browser automation and test Safari compatibility.

Q5. What is the difference between Chromium and Chrome?

Answer:

Chromium is an open-source browser project, whereas Google Chrome is Google's official branded browser based on Chromium.

Playwright downloads Chromium by default but does not automatically install Google Chrome.

Q6. How do you install only Firefox?

Answer:

```
npx playwright install firefox
```

Q7. How do you check the installed Playwright version?

Answer:

```
npx playwright --version
```

Q8. What are system dependencies?

Answer:

System dependencies are operating-system libraries required to launch and run browsers.

For example, Linux may require additional libraries for graphics, fonts and browser execution.

We can install them using:

```
npx playwright install-deps
```

Intermediate-level questions

Q9. How can you execute tests in multiple browsers using Playwright?

Answer:

We can configure multiple projects in `playwright.config.ts`.

Example:

```
projects: [
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'] },
  },
  {
    name: 'firefox',
    use: { ...devices['Desktop Firefox'] },
  },
  {
    name: 'webkit',
    use: { ...devices['Desktop Safari'] },
  },
]
```

When we execute:

```
npx playwright test
```

Playwright runs the tests in all configured projects by default.

Q10. How do you run tests only in Firefox?

Answer:

```
npx playwright test --project=firefox
```

The `--project` option selects a specific configured project.

Q11. What is the use of projects in Playwright?

Answer:

Projects allow us to execute the same test suite with different configurations.

For example:

- Different browsers.
- Different devices.
- Different environments.
- Different test configurations.

They are commonly used for cross-browser testing.

Q12. What is the purpose of the `channel` option?

Answer:

The `channel` option specifies which branded browser to launch.

Example:

```
use: {
  channel: 'chrome'
}
```

This launches the installed Google Chrome browser instead of Playwright's default Chromium build.

Q13. What is cross-browser testing?

Answer:

Cross-browser testing is the process of testing a web application on multiple browsers to verify that it works consistently.

For example, testing login, search, payment and checkout functionality on Chrome, Firefox and Safari-compatible WebKit.

Q14. Does Playwright support real mobile devices?

Answer:

Playwright supports mobile device emulation through predefined device configurations.

It can emulate viewport size, touch support and other device characteristics.

However, emulation is not the same as testing on a physical mobile device.

Q15. Why should we update Playwright regularly?

Answer:

We should update Playwright to get:

- New features.
- Bug fixes.
- Updated browser versions.
- Better browser compatibility.

After updating Playwright, we should run:

```
npx playwright install
```

This ensures the required browser binaries are installed.

Advanced-level questions

Q16. What is the difference between Playwright Chromium and Google Chrome?

Answer:

Playwright Chromium is a browser build maintained and tested for Playwright automation.

Google Chrome is the official branded browser released by Google.

Playwright Chromium is generally sufficient for automation. However, official Chrome is useful when we need to verify behavior in the actual browser used by customers.

Q17. Why doesn't Playwright support branded Firefox and Safari directly?

Answer:

Playwright depends on specific patches and automation capabilities in its browser builds.

Therefore, it uses its own patched Firefox and WebKit builds rather than directly automating the standard branded Firefox and Safari installations.

Q18. What is headless mode?

Answer:

Headless mode means running a browser without displaying its graphical user interface.

It is commonly used in CI/CD pipelines because tests can execute in the background.

Example:

```
npx playwright test
```

Playwright Test runs in headless mode by default unless configured otherwise.

Q19. What is the difference between headless and headed mode?

Answer:

Headless	Headed
Browser UI is not visible	Browser UI is visible
Useful in CI/CD	Useful for debugging
Runs in background	We can observe browser actions

To run tests in headed mode:

```
npx playwright test --headed
```

Q20. What is the use of `PLAYWRIGHT_BROWSERS_PATH`?

Answer:

It is an environment variable used to specify a custom location for downloading and finding Playwright browser binaries.

Example:

```
PLAYWRIGHT_BROWSERS_PATH=$HOME/pw-browsers npx playwright install
```

Q21. What is a hermetic installation?

Answer:

A hermetic installation stores browser binaries inside the local Playwright package directory rather than the default OS cache.

Command:

```
PLAYWRIGHT_BROWSERS_PATH=0 npx playwright install
```

Q22. What is browser garbage collection in Playwright?

Answer:

Playwright automatically removes unused browser versions when they are no longer required by installed Playwright packages.

This helps save disk space.

We can disable this using:

```
PLAYWRIGHT_SKIP_BROWSER_GC=1
```

Q23. How do you execute Playwright browser installation behind a corporate proxy?

Answer:

We can configure the HTTPS_PROXY environment variable.

```
HTTPS_PROXY=https://proxy-address npx playwright install
```

This allows browser downloads through the configured proxy.

18. Real-time Scenario-Based Interview Questions

These are particularly important for QA Automation interviews.

Q24. Your test case passes in Chrome but fails in Firefox. What will you do?

Answer:

First, I will reproduce the issue in Firefox and check the error message.

Then I will investigate:

1. Whether the element locator behaves differently.
2. Whether there is a browser-specific rendering issue.
3. Whether the application has JavaScript compatibility issues.
4. Whether the element is visible and enabled.
5. Whether the issue is related to timing or browser-specific behavior.

After identifying the root cause, I will fix the issue and execute the regression tests again.

Q25. Your Playwright tests are working locally but failing in Jenkins. What could be the reason?

Answer:

Possible reasons include:

- Missing browser binaries.
- Missing Linux system dependencies.
- Different Playwright versions.
- Environment configuration differences.
- Proxy or network restrictions.
- Different browser execution settings.

I will check the Jenkins logs and install the required browsers and dependencies.

```
npx playwright install --with-deps
```

Q26. You updated Playwright and suddenly tests are failing. What will you do?

Answer:

I will check whether the browser binaries are compatible with the new Playwright version.

Then I will execute:

```
npx playwright install
```

I will also check release notes for breaking changes and execute the regression suite.

Q27. Your company requires testing on Chrome, Edge and Firefox. How will you implement this?

Answer:

I will configure three projects in `playwright.config.ts`.

- Chromium or Chrome for Chrome testing.
- Microsoft Edge using `channel: 'msedge'`.
- Firefox using its Playwright-supported browser build.

Then I will execute the tests using:

```
npx playwright test
```

This allows the same test cases to run across all three browsers.

Q28. Why would you use official Chrome instead of default Chromium?

Answer:

I would use official Chrome when the requirement is to validate the application against the actual stable Chrome browser used by customers.

It is also useful for testing certain media codecs and browser-specific behavior.

19. Important Differences to Remember

Topic	Remember
Chromium	Open-source browser project
Chrome	Google's branded browser
Firefox	Uses Playwright's patched Firefox
WebKit	Used for Safari compatibility testing
Projects	Run tests with different configurations
Channel	Selects a branded browser
Headless	Browser UI is hidden
Headed	Browser UI is visible
System dependencies	OS libraries required by browsers
Browser binaries	Downloaded browser executable files
Hermetic install	Browser binaries stored locally with package
Garbage collection	Removes unused browser versions

20. Test Your Interview Knowledge

Try answering these questions without looking at the explanations.

Playwright Browsers Quiz

5/5 answered

1. Which command installs all default Playwright browsers?

npm install playwright
npx playwright install
npx playwright test
npm update

Correct

npx playwright install downloads the default supported browser binaries.

2. Which browser engine is used by Safari?

Chromium
Gecko
WebKit
Blink

Correct

Safari uses WebKit. Playwright uses its own WebKit build.

3. How do you run tests only on Firefox?

npx playwright test firefox
npx playwright --firefox
npx playwright test --project=firefox
npx playwright install firefox

Correct

The --project option selects a configured Playwright project.

4. What is the purpose of the channel option?

Install system dependencies
Select a branded browser
Run tests in parallel
Change test timeout

Correct

For example, channel: 'chrome' launches the installed Google Chrome browser.

5. What does PLAYWRIGHT_BROWSERS_PATH=0 do?

Uninstalls browsers
Enables headed mode
Enables hermetic browser installation
Disables browser testing

Incorrect – Enables hermetic browser installation

It places browser binaries inside the local Playwright package directory.

Score: 4/5

Try again

My interview preparation advice: Focus especially on browser engines, Chromium vs Chrome, projects, channel, cross-browser testing, headless vs headed mode, and browser installation commands. These are the most useful concepts from this section.

Send me the next Playwright documentation section, and we'll continue with the same detailed explanation and interview preparation format.

Playwright – Chrome Extensions

Let's understand this section step by step in simple English, with practical examples and detailed interview questions for QA Automation / SDET roles.

1. Introduction to Chrome Extensions

What is a Chrome Extension?

A Chrome Extension is a small software program that adds extra functionality to a web browser.

Examples:

- Ad blockers
- Password managers
- Grammar checkers

- Screenshot tools
- Translation extensions
- Dark mode extensions

Google Translate Chrome Extension – TechSmart Inc.
Browser Extensions
Build and Launch Your Own Chrome Extension in Just 10 Minutes! | by Sahil Ali | ILLUMINATION | Medium
8

Can Playwright automate Chrome Extensions?

Yes, Playwright supports testing browser extensions.

However, there are some important conditions:

1. Extensions work only with Chromium.
2. Extensions require a persistent browser context.
3. Use Playwright's bundled Chromium rather than branded Google Chrome or Microsoft Edge for loading extensions.
4. You can test extension functionality, popup pages and service workers.

What is a persistent browser context?

A persistent browser context is a browser context that uses a specific user data directory to store browser data.

It can maintain information such as:

- Cookies
- Local storage
- Browser settings
- Extension data
- Login sessions

In Playwright, we use:

`chromium.launchPersistentContext()`

Difference between normal and persistent context:

Normal context	Persistent context
Created using `browser.newContext()`	Created using `chromium.launchPersistentContext()`
Doesn't use a persistent user data directory	Uses a user data directory
Browser data is isolated and generally temporary	Browser data can persist between launches using the same directory
Commonly used for normal web testing	Required for extension testing

Interview point: Why do Chrome extensions require a persistent context in Playwright?

Because Playwright needs to launch Chromium with a user data directory so that the extension can be loaded and initialized properly.

2. Important Note: Why Chromium Instead of Google Chrome?

The documentation mentions that Google Chrome and Microsoft Edge removed the command-line flags needed to side-load extensions.

What does side-loading mean?

Side-loading means installing or loading an extension directly from a local folder instead of installing it through the Chrome Web Store.

For example, suppose you developed your own extension and its files are stored in:

`my-extension/`

You want to load this extension directly into the browser for testing.

Playwright uses bundled Chromium to support this workflow.

Remember:

- Use Playwright's Chromium.
- Do not use `channel: 'chrome'` for this extension-loading workflow.
- Use `channel: 'chromium'`.

3. Launch Chromium with a Chrome Extension

Let's understand the most important code from the documentation.

Complete code

```
const { chromium } = require('playwright');

(async () => {
  const pathToExtension = require('path').join(
    __dirname,
    'my-extension'
  );

  const userDataDir = '/tmp/test-user-data-dir';

  const browserContext =
    await chromium.launchPersistentContext(userDataDir, {
      channel: 'chromium',
      args: [
        `--disable-extensions-except=${pathToExtension}`,
        `--load-extension=${pathToExtension}`
      ]
    });

  let [serviceWorker] = browserContext.serviceWorkers();

  if (!serviceWorker)
```

```
    serviceWorker = await
browserContext.waitForEvent('serviceworker');

  await browserContext.close();
})();
```

Line-by-line explanation

Step 1: Import Chromium

```
const { chromium } = require('playwright');
```

We import Chromium from the Playwright package.

We use Chromium because Playwright supports extension testing through Chromium.

Step 2: Get the extension folder path

```
const pathToExtension = require('path').join(
  __dirname,
  'my-extension'
);
```

Explanation:

- `path` – Node.js module used to work with file paths.
- `join()` – Combines folder names into a valid path.
- `__dirname` – Gives the current JavaScript file's directory.
- `my-extension` – Folder containing the extension files.

Example folder structure:

```
PlaywrightProject/
│
├── tests/
│   └── extension.spec.js
│
└── my-extension/
    ├── manifest.json
    ├── popup.html
    ├── popup.js
    └── background.js
```

Step 3: Define user data directory

```
const userDataDir = '/tmp/test-user-data-dir';
```

This specifies where Chromium stores its persistent browser data.

For Windows, you can use a suitable Windows directory path.

Step 4: Launch persistent context

```
const browserContext =
  await chromium.launchPersistentContext(userDataDir, {
```

This launches Chromium with a persistent browser context.

Notice that we are not using:

```
chromium.launch()
```

We are using:

```
chromium.launchPersistentContext()
```

This is important for extension testing.

Step 5: Specify the browser channel

```
channel: 'chromium',
```

This tells Playwright to use its bundled Chromium build with the new headless mode support for extensions.

Step 6: Load the extension

```
args: [
  `--disable-extensions-except=${pathToExtension}`,
  `--load-extension=${pathToExtension}`
]
```

These are Chromium command-line arguments.

Let's understand both.

Argument	Purpose

`--disable-extensions-except`	Disables other extensions except the specified extension
`--load-extension`	Loads the extension from the specified local folder

Step 7: Get the service worker

```
let [serviceWorker] = browserContext.serviceWorkers();
```

This checks whether a service worker is already available.

Step 8: Wait for service worker if not available

```
if (!serviceWorker)
  serviceWorker = await
browserContext.waitForEvent('serviceworker');
```

Sometimes the service worker is not immediately available when Chromium launches.

Therefore, we wait for the `serviceworker` event.

Step 9: Close the browser context

```
await browserContext.close();
```

This closes the persistent browser context and releases browser resources.

4. What Is a Service Worker?

Important interview topic

A service worker is a JavaScript script that runs in the background of a browser and handles tasks independently of a webpage.

In Chrome extensions, service workers can handle:

- Background events.
- Extension installation events.
- Message handling.
- Notifications.

- Communication between extension components.

Example

Suppose you have a Chrome extension that changes the background color of a website.

The service worker can handle extension-related background operations, while the extension's content script interacts with the webpage.

What is Manifest V3?

Manifest V3 (MV3) is the modern Chrome extension platform version.

It uses service workers for background functionality instead of the older persistent background pages.

Important difference

Manifest V2	Manifest V3
Uses background pages	Uses service workers
Background pages can remain active	Service workers can become inactive
Older extension architecture	Modern extension architecture

For Playwright extension testing, the documentation demonstrates retrieving the service worker of a Manifest V3 extension.

5. Service Worker Idle Suspension (MV3)

This is an important concept because it can cause confusion during automation.

What is idle suspension?

Chrome Manifest V3 service workers are automatically suspended after approximately 30 seconds of inactivity.

When another event requires the service worker, Chrome can restart it.

Example

Suppose:

1. You launch Chromium.
2. Your extension's service worker starts.
3. You do not perform any activity for around 30 seconds.
4. Chrome suspends the service worker.
5. Later, you send a message to the service worker.
6. Chrome restarts it when needed.

How does Playwright handle this?

Playwright keeps the same Worker object alive.

You don't necessarily receive a new `serviceworker` event when the worker restarts.

Example:

```
const sw = await context.waitForEvent('serviceworker');

// Service worker gets suspended and restarted.

await sw.evaluate(() => sendMessage({
  type: 'ping'
}));
```

What does `evaluate()` mean here?

`evaluate()` executes JavaScript inside the service worker's execution environment.

In this example:

```
await sw.evaluate(() => sendMessage({
  type: 'ping'
}));
```

It sends a ping message using the extension's `sendMessage()` function.

Important exception

If an `evaluate()` call is already executing at the exact moment the service worker is suspended, it can fail with:

`Service worker restarted`

But a new `evaluate()` call issued during the restart window can wait for the worker to become ready and resume automatically.

Interview point: Does Playwright generate a new service worker event every time an MV3 service worker restarts?

No. Playwright keeps the same Worker object alive, so a new `serviceworker` event is not necessarily emitted.

6. Testing Chrome Extensions Using Fixtures

Very important for practical interviews

Now let's understand how to integrate Chrome extension testing into Playwright Test.

What is a fixture?

A fixture is a reusable setup mechanism in Playwright.

It provides the required resources to test cases.

For example:

- Browser
- Browser context
- Page
- Login session
- Extension ID

Instead of writing browser setup code in every test, we can create a fixture once and reuse it.

Why use fixtures for extension testing?

Suppose you have 20 test cases for a Chrome extension.

You need to:

1. Launch Chromium.
2. Load the extension.
3. Get the extension ID.
4. Execute tests.
5. Close the browser context.

Instead of repeating this code in all 20 tests, we can create a fixture.

Step 1: Create fixtures.ts

```
import {
  test as base,
  chromium,
  type BrowserContext
} from '@playwright/test';

import path from 'path';

export const test = base.extend<{
  context: BrowserContext;
  extensionId: string;
}>({

  context: async ({ }, use) => {

    const pathToExtension = path.join(
      __dirname,
      'my-extension'
    );

    const context = await chromium.launchPersistentContext('', {
      channel: 'chromium',

      args: [
```

```
        `--disable-extensions-except=${pathToExtension}`,
        `--load-extension=${pathToExtension}`,
      ],
    });

    await use(context);

    await context.close();
  },

  extensionId: async ({ context }, use) => {

    let [serviceWorker] = context.serviceWorkers();

    if (!serviceWorker)
      serviceWorker = await context.waitForEvent('serviceworker');

    const extensionId = serviceWorker.url().split('/')[2];

    await use(extensionId);
  },
});

export const expect = test.expect;
```

Understand the code

1. Import test and Chromium

```
import {
  test as base,
  chromium,
  type BrowserContext
} from '@playwright/test';
```

- `test as base` – Imports the original Playwright test object with another name.
- `chromium` – Used to launch Chromium.
- `BrowserContext` – TypeScript type for browser context.

2. Extend the base test

```
export const test = base.extend<{
  context: BrowserContext;
  extensionId: string;
}>({
```

Here, we extend Playwright's base test with two custom fixtures:

- `context`
- `extensionId`

3. Create the context fixture

```
context: async ({ }, use) => {
```

This creates a custom context fixture.

`use` passes the created resource to the test and allows the fixture to run cleanup after the test finishes.

4. Launch Chromium

```
const context = await chromium.launchPersistentContext('', {
  channel: 'chromium',
```

This launches Chromium with a persistent context.

An empty user data directory argument means Playwright creates a temporary user data directory.

5. Load the extension

```
args: [
  `--disable-extensions-except=${pathToExtension}`,
  `--load-extension=${pathToExtension}`,
],
```

These arguments load our local extension.

6. Provide context to the test

```
await use(context);
```

This makes the browser context available to the test.

7. Close context

```
await context.close();
```

This performs cleanup after the test has finished using the fixture.

8. Create extension ID fixture

```
extensionId: async ({ context }, use) => {
```

This fixture depends on the `context` fixture.

It gets the browser context and uses it to retrieve the extension ID.

9. Get the service worker

```
let [serviceWorker] = context.serviceWorkers();

if (!serviceWorker)
  serviceWorker = await context.waitForEvent('serviceworker');
```

This retrieves the service worker, waiting for it if necessary.

10. Extract the extension ID

```
const extensionId = serviceWorker.url().split('/')[2];
```

This is an important line.

A service worker URL may look like:

```
chrome-extension://abcdefghijklmnop/background.js
```

When we split it using `/`, we get:

Index	Value
0	`chrome-extension:`
1	Empty string
2	`abcdefghijklmnop`

```
3       background.js
```

Therefore:

```
.split('/')[2]
```

Returns the extension ID.

11. Provide the extension ID

```
await use(extensionId);
```

This makes the extension ID available in test cases.

12. Export expect

```
export const expect = test.expect;
```

This exports the matching Playwright assertion function.

7. Write Test Cases for Chrome Extensions

After creating fixtures, we can use them in our test files.

Test Case 1: Verify extension functionality

```
import { test, expect } from './fixtures';

test('example test', async ({ page }) => {

  await page.goto('https://example.com');

  await expect(page.locator('body'))
    .toHaveText('Changed by my-extension');

});
```

Explanation

1. Import custom `test` and `expect`.

2. Use the page fixture.
3. Navigate to the website.
4. Verify whether the extension modified the webpage content.

Here:

```
await page.goto('https://example.com');
```

Opens the website.

```
await expect(page.locator('body'))
  .toHaveText('Changed by my-extension');
```

Checks whether the body contains the expected text.

This is an example of testing an extension's effect on a webpage.

Test Case 2: Test extension popup page

```
test('popup page', async ({ page, extensionId }) => {

  await page.goto(
    `chrome-extension://${extensionId}/popup.html`
  );

  await expect(page.locator('body'))
    .toHaveText('my-extension popup');

});
```

Explanation

What is a popup page?

A popup page is the small interface that appears when a user clicks the extension icon in the browser toolbar.

For example, a password manager extension may show a popup containing saved passwords.

How do we open it in Playwright?

```
await page.goto(
  `chrome-extension://${extensionId}/popup.html`
);
```

Here:

- `chrome-extension://` – Special URL scheme for extension pages.
- `extensionId` – Unique ID of the extension.
- `popup.html` – Extension popup HTML file.

Then we verify the popup content using:

```
await expect(page.locator('body'))
  .toHaveText('my-extension popup');
```

Important: This code navigates directly to the popup page. It does not simulate clicking the extension toolbar icon.

8. Complete Flow of Chrome Extension Testing

Understand this flow for interviews.

1. Create Chrome Extension

manifest.json, popup.html, background.js

2. Configure Playwright Fixture

Create reusable browser setup

3. Launch Persistent Chromium

Load the extension from local folder

4. Retrieve Service Worker

Get the extension ID

5. Execute Test Cases

Test webpage changes and popup page

6. Close Browser Context

Release browser resources

9. Interview Questions and Answers

Basic-level questions

Q1. What is a Chrome Extension?

Answer:

A Chrome Extension is a small software program that adds additional functionality to a browser.

Examples include ad blockers, password managers and grammar checkers.

Playwright can automate and test Chrome extensions using Chromium.

Q2. Can we test Chrome extensions using Playwright?

Answer:

Yes. Playwright supports extension testing using Chromium with a persistent browser context.

We can test:

- Extension functionality.
- Popup pages.
- Service workers.
- Changes made by extensions to webpages.

Q3. Which browser does Playwright use for Chrome extension testing?

Answer:

Playwright uses its bundled Chromium browser.

Chrome extensions are supported only in Chromium when launched with a persistent context.

Q4. What is a persistent browser context?

Answer:

A persistent browser context is a browser context that uses a user data directory to maintain browser-related data.

It is required for extension testing because the extension needs to be loaded into the browser context.

Q5. What is a Chrome Extension service worker?

Answer:

A service worker is a background JavaScript script used by a Chrome extension to handle background operations and events.

In Manifest V3, service workers replace persistent background pages.

Q6. What is Manifest V3?

Answer:

Manifest V3 is the modern Chrome extension platform.

It uses service workers for background functionality and introduces changes to extension architecture and security.

Q7. What is an extension popup?

Answer:

An extension popup is the small user interface displayed when a user interacts with the extension icon.

We can test the popup by navigating to its extension URL using the extension ID.

Intermediate-level questions

Q8. What is the difference between `launch()` and `launchPersistentContext()`?

Answer:

`launch()` launches a browser instance. We can then create normal browser contexts using `browser.newContext()`.

`launchPersistentContext()` launches the browser directly with a persistent context and user data directory.

For Chrome extension testing, we use `launchPersistentContext()`.

Q9. Why do we use `channel: 'chromium'`?

Answer:

We use the Chromium channel to run Playwright's bundled Chromium with the new headless mode, which supports extension testing.

Alternatively, we can launch Chromium in headed mode.

Q10. What is the purpose of `--load-extension`?

Answer:

It is a Chromium command-line argument that loads an extension from a specified local folder.

Example:

`` `--load-extension=${pathToExtension}` ``

Q11. What is the purpose of `--disable-extensions-except`?

Answer:

It disables other extensions except the extension specified in the argument.

It helps us isolate the extension being tested.

Q12. How can you retrieve an extension ID in Playwright?

Answer:

For a Manifest V3 extension, we can retrieve the service worker URL and extract the extension ID.

```
const [serviceWorker] = context.serviceWorkers();

const extensionId = serviceWorker.url().split('/')[2];
```

We should also wait for the service worker if it has not started yet.

Q13. How do you open a Chrome extension popup using Playwright?

Answer:

First, retrieve the extension ID. Then navigate to its popup URL.

```
await page.goto(
  `chrome-extension://${extensionId}/popup.html`
);
```

Q14. What is the purpose of `waitForEvent('serviceworker')`?

Answer:

It waits for the service worker event when the service worker is not immediately available.

Example:

```
const serviceWorker =
  await context.waitForEvent('serviceworker');
```

Q15. Why do we use fixtures for extension testing?

Answer:

Fixtures provide reusable setup and teardown functionality.

We can create a custom fixture to:

- Launch Chromium.
- Load the extension.
- Retrieve its ID.
- Share the context with test cases.
- Close the context after execution.

This avoids repeating browser setup code in every test.

Advanced-level questions

Q16. What happens when an MV3 service worker is inactive for approximately 30 seconds?

Answer:

Chrome automatically suspends the service worker after approximately 30 seconds of inactivity.

When another event requires it, Chrome restarts it.

Playwright keeps the same Worker object alive, so a new service worker event is not necessarily generated.

Q17. Does Playwright create a new Worker object whenever an MV3 service worker restarts?

Answer:

No. Playwright maintains the same Worker object across the service worker restart.

New `evaluate()` calls can wait for the restarted service worker and resume automatically.

Q18. What happens if an evaluate call is in progress when the service worker gets suspended?

Answer:

The operation may fail with a `Service worker restarted` error.

This happens when the service worker is suspended while the evaluation is already in progress.

Q19. Why can't we use Google Chrome directly for loading local extensions in this approach?

Answer:

Google Chrome and Microsoft Edge removed the command-line flags required for this extension side-loading workflow.

Therefore, Playwright recommends using its bundled Chromium for this purpose.

Q20. How can you test both the extension functionality and its popup page?

Answer:

I would create separate test cases.

First, I would open a normal webpage and verify that the extension modifies the expected webpage content.

Second, I would retrieve the extension ID, navigate to the popup URL and verify the popup elements and content.

10. Real-Time Scenario-Based Interview Questions

Q21. Suppose your Chrome extension modifies website content. How will you test it?

Answer:

I will follow these steps:

1. Launch Chromium with a persistent browser context.
2. Load the extension from its local folder.
3. Open the target website.
4. Verify whether the extension modifies the expected webpage content.
5. Validate the expected output using Playwright assertions.

Example:

```
await page.goto('https://example.com');

await expect(page.locator('body'))
  .toHaveText('Changed by my-extension');
```

Q22. Your extension is loaded, but the popup page is not opening. What will you check?

Answer:

I will check:

1. Whether the extension folder path is correct.
2. Whether `manifest.json` exists.
3. Whether the popup file is correctly configured in the manifest.
4. Whether the extension ID is retrieved correctly.
5. Whether the popup HTML filename is correct.

6. Whether the service worker has started successfully.

Q23. Your extension test works in headed mode but fails in headless mode. What will you do?

Answer:

I will verify that I am using the Chromium channel that supports the new headless mode.

I will also check the extension loading arguments, browser logs and whether the extension behaves differently in headless mode.

Q24. How would you avoid repeating extension setup code in multiple test cases?

Answer:

I would create custom Playwright fixtures.

The fixture would handle browser launching, extension loading, service worker initialization and cleanup.

All test cases can reuse the same setup.

Q25. How do you test the extension ID dynamically instead of hardcoding it?

Answer:

I would retrieve the service worker URL and extract the extension ID.

```
const extensionId = serviceWorker.url().split('/')[2];
```

This avoids hardcoding the extension ID because it can vary between extension builds or installation contexts.

11. Important Code and Concepts for Revision

Concept	Important syntax
Launch Chromium	`chromium.launchPersistentContext()`

Load extension	`--load-extension`
Disable other extensions	`--disable-extensions-except`
Get service workers	`context.serviceWorkers()`
Wait for service worker	`context.waitForEvent('serviceworker')`
Execute worker JavaScript	`serviceWorker.evaluate()`
Get extension ID	`serviceWorker.url().split('/')[2]`
Open popup	`chrome-extension://ID/popup.html`
Create fixture	`base.extend()`
Pass fixture resource	`await use(context)`
Close context	`await context.close()`

12. Test Your Interview Knowledge

Try these questions to check whether you understood the section.

Chrome Extensions Quiz

6/6 answered

1. Which browser is used for Chrome extension testing in Playwright?

Firefox
WebKit
Bundled Chromium
Microsoft Edge

Correct!

Playwright uses bundled Chromium for extension testing.

2. Which method is required to launch a persistent context?

chromium.launch()
chromium.launchPersistentContext()
browser.newContext()
page.goto()

Incorrect. Correct answer: chromium.launchPersistentContext()

launchPersistentContext() creates a persistent browser context.

3. What is Manifest V3 using for background functionality?

Persistent background pages
Service workers
HTML pages
CSS files

Correct!

Manifest V3 uses service workers for background functionality.

4. Which argument loads an extension from a local folder?

--load-extension
--install-extension
--start-extension
--open-extension

Correct!

--load-extension specifies the local extension directory.

5. How do you open an extension popup page?

http://extension/popup.html
chrome-extension://ID/popup.html
file://popup.html
browser://popup.html

Incorrect. Correct answer: chrome-extension://ID/popup.html

The extension page uses the chrome-extension:// URL scheme.

6. What happens when an MV3 service worker is inactive for around 30 seconds?

It is permanently deleted
Browser closes
It can be suspended and restarted on demand
The extension is uninstalled

Incorrect. Correct answer: It can be suspended and restarted on demand

Chrome suspends inactive MV3 service workers and restarts them when needed.

Your score: 3/6

Retry quiz

13. What should you remember most for interviews?

If you have limited time, prioritize these seven topics:

1. What is a Chrome Extension?
2. Why do we use Chromium instead of branded Chrome?
3. What is a persistent browser context?
4. What is a service worker and Manifest V3?
5. How do we load an extension using `--load-extension`?
6. How do we retrieve an extension ID?
7. How do we create fixtures to test extensions?

Practical interview tip: Pay special attention to `launchPersistentContext()`, `serviceWorkers()`, `waitForEvent()` and dynamic extension ID extraction. These are the key coding concepts in this documentation section.

Send the next section whenever you're ready. We'll continue with the same format.

Playwright – Clock

This is an important Playwright topic for QA Automation interviews because it helps us test time-dependent functionality without waiting for actual time to pass.

Let's understand each method, its practical usage, code examples and interview questions.

1. Introduction – What is Clock in Playwright?

Simple explanation

Clock is a Playwright feature that allows us to control or manipulate time inside a webpage during automation testing.

Normally, an application uses the system's actual time.

For example:

- OTP expires after 2 minutes.
- User gets logged out after 5 minutes of inactivity.
- A discount coupon expires at midnight.
- A session expires after 30 minutes.
- A notification appears after 10 seconds.

Testing these features using actual time can be slow.

Playwright Clock allows us to simulate time and test these scenarios quickly.

Real-time example

Suppose your application automatically logs out a user after 5 minutes of inactivity.

Without Clock:

1. Login to the application.

2. Wait for 5 minutes.
3. Check whether the user is logged out.

With Playwright Clock:

1. Login to the application.
2. Fast-forward time by 5 minutes.
3. Check whether the user is logged out.

You don't need to wait for 5 actual minutes.

Real-time vs Playwright Clock

Without Clock

5 min

Wait for actual time to pass

With Clock

Fast

Simulate 5 minutes instantly

Illustrative comparison; actual execution time depends on the application and timers.

Where is Clock useful?

Application feature	How Clock helps
OTP expiration	Simulate OTP expiry
Session timeout	Simulate automatic logout
Coupon expiration	Test expired coupons

Countdown timer	Test countdown functionality
Scheduled notifications	Simulate scheduled time
Booking expiration	Test booking expiry
Inactivity monitoring	Simulate user inactivity

2. Clock API – Important Methods

Playwright provides the following methods:

Method	Simple meaning
`setFixedTime()`	Fixes the current date and time
`install()`	Installs a controllable clock
`pauseAt()`	Pauses time at a specified date/time
`fastForward()`	Moves time forward quickly
`runFor()`	Advances time while executing timers
`resume()`	Resumes normal time progression
`setSystemTime()`	Changes the current system time without firing timers

Let's understand each method.

2.1 setFixedTime()

Meaning: It sets a fixed date and time for the webpage.

When we use `setFixedTime()`, calls such as `Date.now()` and `new Date()` return the fixed time.

However, timers such as `setTimeout()` and `setInterval()` continue running normally.

Example

```
await page.clock.setFixedTime(
  new Date('2024-02-02T10:00:00')
);
```

Now the webpage sees the date and time as:

```
2 February 2024, 10:00 AM
```

Even if the actual computer time is different.

When should we use it?

Suppose you want to test an application that displays the current date.

You want to verify that it displays 2 February 2024.

You can use `setFixedTime()` instead of changing your computer's actual date.

Important: Fixed time does not mean that timers are stopped. Timers continue running, but the reported date/time remains fixed until you change it.

2.2 install()

Meaning: It initializes the Playwright clock and gives us control over time.

```
await page.clock.install();
```

After installation, Playwright can control time-related browser APIs.

You can also initialize it with a specific time:

```
await page.clock.install({
  time: new Date('2024-02-02T08:00:00')
});
```

Now you can use methods such as:

- `pauseAt()`
- `fastForward()`
- `runFor()`
- `resume()`
- `setSystemTime()`

Important difference:

`setFixedTime()` is suitable when you only need to control the displayed date/time.

`install()` is useful when you need more control over timers and time progression.

2.3 pauseAt()

Meaning: It pauses the clock at a particular date and time.

Example:

```
await page.clock.pauseAt(
  new Date('2024-02-02T10:00:00')
);
```

The clock will be paused at 10 AM on 2 February 2024.

This is useful when you want to test the application's state at an exact time.

For example:

- Check whether a coupon is still valid at 10 AM.
- Check a countdown at a particular time.
- Check whether a scheduled notification should appear.

2.4 fastForward()

Meaning: It moves time forward quickly.

Example:

```
await page.clock.fastForward('05:00');
```

This moves the clock forward by 5 minutes.

You can also use milliseconds:

```
await page.clock.fastForward(300000);
```

300000 milliseconds = 5 minutes.

Important: `fastForward()` behaves like a user closing a laptop and opening it after some time. Timers that became due fire once immediately rather than executing every missed interval.

2.5 runFor()

Meaning: It advances time for a specified duration while consistently firing timers and animation frames.

Example:

```
await page.clock.runFor(2000);
```

This advances time by 2 seconds.

During this time:

- Timers execute.
- Intervals execute.
- Animation frames are processed.
- Time progresses consistently.

Difference between `fastForward()` and `runFor()`:

fastForward()	runFor()
Moves time forward quickly	Advances time while running timers
Due timers fire once	Timers execute as their scheduled time is reached
Useful for inactivity simulation	Useful for countdowns and interval-based updates

2.6 resume()

Meaning: It resumes normal time progression after the clock has been paused.

Example:

```
await page.clock.resume();
```

Suppose you paused time at 10 AM using `pauseAt()`.

When you call `resume()`, the clock starts progressing again.

2.7 setSystemTime()

Meaning: It changes the current system time seen by the webpage without automatically firing timers.

Example:

```
await page.clock.setSystemTime(
  new Date('2024-02-02T12:00:00')
);
```

Now the webpage sees 12 PM.

However, this method does not automatically execute pending timers.

It is generally recommended for advanced use cases.

2.8 Important: What does Clock control?

When you install the Playwright clock, it overrides several native browser functions related to time.

Browser API	Purpose
`Date`	Gets current date and time
`Temporal.Now`	Gets current time using Temporal API
`setTimeout()`	Executes code after a specified delay
`clearTimeout()`	Cancels a timeout
`setInterval()`	Executes code repeatedly
`clearInterval()`	Stops an interval
`requestAnimationFrame()`	Executes code during browser animation frames
`cancelAnimationFrame()`	Cancels an animation frame
`requestIdleCallback()`	Executes code when the browser is idle
`cancelIdleCallback()`	Cancels an idle callback
`performance`	Provides performance timing information
`Event.timeStamp`	Provides the event timestamp

Important warning from the documentation:

If you use `page.clock.install()`, it must be called before any other clock-related calls in your test.

For example, don't do this:

```
await page.evaluate(() => {
  setInterval(() => console.log('Hello'), 1000);
});

await page.clock.install(); // Wrong order
```

Why?

Because `install()` overrides native time-related functions. Calling it after those functions have already been used can result in undefined behavior.

Correct approach:

```
await page.clock.install();

await page.goto('http://localhost:3333');
```

Install the clock before navigating to the page or executing application code that uses timers.

3. Test with Predefined Time

This is the first practical example from the documentation.

What is predefined time?

Predefined time means setting a specific date and time for testing.

Suppose your application displays the current date and time on the webpage.

You want to test whether the application displays:

2 February 2024, 10:00 AM.

Instead of depending on the actual computer date, you can set a fixed time.

Application HTML and JavaScript

```
<div id="current-time"
     data-testid="current-time">
</div>
```

```
<script>
  const renderTime = () => {
    document.getElementById('current-time').textContent =
      new Date().toLocaleString();
  };

  setInterval(renderTime, 1000);
</script>
```

Understand the application code

1. HTML element

```
<div id="current-time" data-testid="current-time"></div>
```

This creates an empty HTML element where the current time will be displayed.

2. Create renderTime function

```
const renderTime = () => {
  document.getElementById('current-time').textContent =
    new Date().toLocaleString();
};
```

This function:

- Gets the HTML element.
- Gets the current date and time.
- Converts it into a readable string.
- Displays it on the webpage.

3. setInterval

```
setInterval(renderTime, 1000);
```

This executes the `renderTime()` function every 1000 milliseconds (1 second).

Playwright test code

Step 1: Set fixed time

```
await page.clock.setFixedTime(
  new Date('2024-02-02T10:00:00')
);
```

This makes the webpage see 10 AM on 2 February 2024.

Step 2: Open the application

```
await page.goto('http://localhost:3333');
```

Step 3: Verify displayed time

```
await expect(
  page.getByTestId('current-time')
).toHaveText('2/2/2024, 10:00:00 AM');
```

This checks whether the correct date and time are displayed.

Step 4: Change the fixed time

```
await page.clock.setFixedTime(
  new Date('2024-02-02T10:30:00')
);
```

Now the webpage sees 10:30 AM.

Step 5: Verify updated time

```
await expect(
  page.getByTestId('current-time')
).toHaveText('2/2/2024, 10:30:00 AM');
```

Important concept

Even though `setInterval()` runs every second, `Date.now()` and `new Date()` continue returning the fixed time.

How setFixedTime works

First fixed time

10:00 AM

Updated fixed time

10:30 AM

The displayed date changes when setFixedTime is called again, while the interval continues running.

When should we use setFixedTime()?

When we only need to control the date and time displayed by the application, without controlling the execution of timers.

4. Consistent Time and Timers

What is consistent time and timers?

Sometimes your application depends on both:

- `Date.now()`
- JavaScript timers such as `setTimeout()` and `setInterval()`.

In such cases, fixing only the date may not be sufficient.

You need to control both the time and timer execution.

For this, use `install()`.

Real-time example

Suppose an application displays the current time every second.

You want to:

1. Open the application at 8 AM.
2. Let it run normally.
3. Move time forward to 10 AM.
4. Pause the clock.
5. Verify the displayed time.
6. Move forward to 10:30 AM.

Application code

```
<div id="current-time"
     data-testid="current-time">
</div>

<script>
  const renderTime = () => {
    document.getElementById('current-time').textContent =
      new Date().toLocaleString();
  };

  setInterval(renderTime, 1000);
</script>
```

Playwright code

Step 1: Install clock at 8 AM

```
await page.clock.install({
  time: new Date('2024-02-02T08:00:00')
});
```

The clock starts from 8 AM.

Step 2: Navigate to the application

```
await page.goto('http://localhost:3333');
```

Step 3: Pause at 10 AM

```
await page.clock.pauseAt(
  new Date('2024-02-02T10:00:00')
);
```

Now time is paused at 10 AM.

Step 4: Verify the displayed time

```
await expect(
  page.getByTestId('current-time')
).toHaveText('2/2/2024, 10:00:00 AM');
```

Step 5: Fast-forward by 30 minutes

```
await page.clock.fastForward('30:00');
```

The clock moves from 10 AM to 10:30 AM.

Step 6: Verify the updated time

```
await expect(
  page.getByTestId('current-time')
).toHaveText('2/2/2024, 10:30:00 AM');
```

Understand the complete flow

1. 8:00 AM – Install clock

Application starts with the predefined time.

2. 10:00 AM – Pause clock

Time stops at the specified time.

3. 10:30 AM – Fast-forward

Move time forward by 30 minutes.

4. Verify the result

Check that the webpage displays 10:30 AM.

Interview point: Use `install()` when you need control over both date/time and timer execution.

5. Test Inactivity Monitoring

Very important interview scenario

What is inactivity monitoring?

Inactivity monitoring is a feature that checks whether a user is interacting with an application.

If the user does not perform any activity for a specific period, the application automatically logs them out.

Real-time example

Suppose a banking application automatically logs out users after 5 minutes of inactivity.

Expected behavior:

- User logs in.
- A 5-minute inactivity timer starts.
- User performs no activity.
- After 5 minutes, the application logs out automatically.

Without Playwright Clock

You would have to wait for 5 actual minutes to verify the functionality.

With Playwright Clock

You can simulate 5 minutes using `fastForward()`.

Application code

```
const endTime = Date.now() + 5 * 60_000;

const renderTime = () => {

  const diffInSeconds =
    Math.round((endTime - Date.now()) / 1000);

  if (diffInSeconds <= 0) {

    document.getElementById('remaining-time').textContent =
      'You have been logged out due to inactivity.';

  } else {

    document.getElementById('remaining-time').textContent =
      `You will be logged out in ${diffInSeconds} seconds.`;

  }
```

```
  setTimeout(renderTime, 1000);
};

renderTime();
```

Understand the code

1. Calculate the end time

```
const endTime = Date.now() + 5 * 60_000;
```

- `Date.now()` gives the current timestamp in milliseconds.
- `5 * 60_000` means 5 minutes.
- `endTime` stores the time when the session should expire.

2. Calculate remaining time

```
const diffInSeconds =
  Math.round((endTime - Date.now()) / 1000);
```

This calculates how many seconds are left before logout.

3. Check whether the time has expired

```
if (diffInSeconds <= 0)
```

If the remaining time is zero or negative, the application displays the logout message.

4. Otherwise, display remaining time

```
`You will be logged out in ${diffInSeconds} seconds.`
```

This displays a countdown.

5. Schedule the next update

```
setTimeout(renderTime, 1000);
```

This calls the function again after one second.

Playwright test code

Step 1: Install the clock

```
await page.clock.install();
```

This initializes the clock.

Step 2: Open the application

```
await page.goto('http://localhost:3333');
```

Step 3: Interact with the page

```
await page.getByRole('button').click();
```

This simulates user interaction.

Step 4: Fast-forward 5 minutes

```
await page.clock.fastForward('05:00');
```

This simulates 5 minutes passing without waiting for 5 actual minutes.

Step 5: Verify automatic logout

```
await expect(
  page.getByText(
    'You have been logged out due to inactivity.'
  )
).toBeVisible();
```

This verifies whether the logout message is visible.

Why is this useful in real projects?

Imagine you are testing a banking application.

You have multiple test cases:

1. Verify automatic logout after 5 minutes.
2. Verify that user activity resets the inactivity timer.
3. Verify that the session expires correctly.
4. Verify that the login page appears after logout.

Using Clock, you can test these scenarios much faster.

Important interview point: `fastForward()` is useful for testing inactivity because timers that become due fire once immediately, similar to a browser resuming after being inactive.

6. Tick Through Time Manually Using runFor()

What does tick through time mean?

It means advancing the clock step by step while allowing timers to execute at their scheduled times.

We use `runFor()` for this purpose.

Real-time example

Suppose your application displays a countdown:

```
10
9
8
7
6
```

You want to verify that the countdown updates correctly every second.

Instead of waiting for actual time, you can control the clock and advance it manually.

Application code

```
<div id="current-time"
     data-testid="current-time">
</div>

<script>
  const renderTime = () => {
    document.getElementById('current-time').textContent =
      new Date().toLocaleString();
  };
```

```
  setInterval(renderTime, 1000);
</script>
```

Playwright code

Step 1: Install the clock at 8 AM

```
await page.clock.install({
  time: new Date('2024-02-02T08:00:00')
});
```

Step 2: Open the application

```
await page.goto('http://localhost:3333');
```

Step 3: Pause the clock at 10 AM

```
await page.clock.pauseAt(
  new Date('2024-02-02T10:00:00')
);
```

Now time is stopped at 10 AM.

Step 4: Verify the current time

```
await expect(
  page.getByTestId('current-time')
).toHaveText('2/2/2024, 10:00:00 AM');
```

Step 5: Advance time by 2 seconds

```
await page.clock.runFor(2000);
```

This advances time by 2000 milliseconds.

Since the interval is scheduled every second, the application gets two timer executions.

Step 6: Verify the updated time

```
await expect(
  page.getByTestId('current-time')
).toHaveText('2/2/2024, 10:00:02 AM');
```

Difference between fastForward and runFor

This is one of the most important differences to remember.

fastForward	runFor
Moves time forward quickly	Advances time consistently
Due timers fire once	Timers fire at their scheduled times
Useful for inactivity and expiry	Useful for countdowns and repeated updates
Similar to closing and reopening a laptop	Similar to letting time pass while the application runs

Example:

Suppose you have:

```
setInterval(() => {
  console.log('Hello');
}, 1000);
```

You advance time by 5 seconds.

- With `fastForward('00:05')`, the due interval callback fires once.
- With `runFor(5000)`, the interval callback fires at each scheduled second.

7. Difference Between All Clock Methods

This table is very important for technical interviews.

Method	What it does	When to use
`setFixedTime()`	Fixes the date/time but timers continue normally	Testing dates and timestamps
`install()`	Installs a controllable clock	Advanced time-based testing

`pauseAt()`	Pauses time at a specific point	Testing an exact time
`fastForward()`	Quickly moves time forward; due timers fire once	Session expiry and inactivity
`runFor()`	Advances time while executing timers consistently	Countdown and interval testing
`resume()`	Resumes a paused clock	Continuing normal time progression
`setSystemTime()`	Changes the current system time without firing timers	Advanced date/time manipulation

Easy way to remember

- Only change the date? → `setFixedTime()`
- Need complete clock control? → `install()`
- Stop at a particular time? → `pauseAt()`
- Skip 5 minutes? → `fastForward()`
- Execute every timer during 5 seconds? → `runFor()`
- Start time again? → `resume()`
- Change system time without triggering timers? → `setSystemTime()`

8. Important Interview Questions and Answers

Basic-level questions

Q1. What is Clock in Playwright?

Answer:

Clock is a Playwright feature that allows us to control and manipulate time during automation testing.

It helps test time-dependent functionality such as session expiration, countdown timers, OTP expiration and scheduled tasks without waiting for actual time to pass.

Q2. Why do we use Clock in Playwright?

Answer:

We use Clock to simulate different dates and times during testing.

For example, if an application automatically logs out users after 5 minutes of inactivity, we can use Clock to simulate 5 minutes instead of waiting for the actual duration.

Q3. What are the main methods available in Playwright Clock?

Answer:

The main methods are:

1. `setFixedTime()`
2. `install()`
3. `pauseAt()`
4. `fastForward()`
5. `runFor()`
6. `resume()`
7. `setSystemTime()`

Q4. What is the difference between setFixedTime() and install()?

Answer:

`setFixedTime()` fixes the date and time returned by `Date.now()` and `new Date()`, while allowing timers to continue normally.

`install()` provides more control over browser time and timers. We can pause time, fast-forward it, run timers manually and resume the clock.

Q5. What is fastForward() in Playwright?

Answer:

`fastForward()` moves the browser clock forward by a specified duration.

It fires timers that become due once immediately instead of executing every missed interval.

Example:

```
await page.clock.fastForward('05:00');
```

This simulates 5 minutes passing.

Q6. What is runFor() in Playwright?

Answer:

`runFor()` advances the clock for a specified duration while consistently executing scheduled timers and animation frames.

Example:

```
await page.clock.runFor(5000);
```

This advances time by 5 seconds.

Q7. What is pauseAt()?

Answer:

`pauseAt()` pauses the browser clock at a specified date and time.

Example:

```
await page.clock.pauseAt(
  new Date('2024-02-02T10:00:00')
);
```

It is useful when testing application behavior at an exact time.

Q8. What is resume()?

Answer:

`resume()` resumes the clock after it has been paused.

Example:

```
await page.clock.resume();
```

Q9. What is setSystemTime()?

Answer:

`setSystemTime()` changes the system time seen by the webpage without automatically firing timers.

It is mainly used for advanced time manipulation scenarios.

Intermediate-level questions

Q10. What is the difference between fastForward() and runFor()?

Answer:

`fastForward()` moves time forward quickly and fires due timers once.

`runFor()` advances time consistently and executes timers at their scheduled times.

For example, if an interval executes every second and we advance by 5 seconds:

- `fastForward()` executes the overdue interval callback once.
- `runFor()` executes it at each scheduled second.

Q11. Can we control setTimeout() and setInterval() using Playwright Clock?

Answer:

Yes. When we install the Playwright clock, it overrides native browser timer functions such as:

- `setTimeout()`
- `clearTimeout()`
- `setInterval()`
- `clearInterval()`

This allows us to control timer execution during testing.

Q12. What happens if we call install() after using setInterval()?

Answer:

It can result in undefined behavior because `install()` overrides the native clock-related functions.

Therefore, we should call `page.clock.install()` before executing application code that uses these functions.

Q13. How do you test a session timeout of 5 minutes?

Answer:

First, I install the Playwright clock and navigate to the application.

Then I simulate user login or interaction and fast-forward the clock by 5 minutes.

Finally, I verify whether the application automatically logs out the user.

Example:

```
await page.clock.install();

await page.goto('http://localhost:3333');

await page.getByRole('button').click();

await page.clock.fastForward('05:00');

await expect(
  page.getByText('You have been logged out due to inactivity.')
).toBeVisible();
```

Q14. Does setFixedTime() stop JavaScript timers?

Answer:

No. `setFixedTime()` only fixes the date and time returned by date-related APIs.

JavaScript timers such as `setTimeout()` and `setInterval()` continue running normally.

Q15. Which method should be used when we only want to test a specific date?

Answer:

We should use `setFixedTime()` because it allows us to set a specific date and time without controlling the timer execution.

Advanced-level questions

Q16. Which native browser APIs are overridden by page.clock.install()?

Answer:

It overrides browser APIs related to time, including:

- Date
- Temporal.Now
- setTimeout and clearTimeout
- setInterval and clearInterval
- requestAnimationFrame and cancelAnimationFrame
- requestIdleCallback and cancelIdleCallback
- performance
- Event.timeStamp

Q17. Can we test OTP expiration using Playwright Clock?

Answer:

Yes.

Suppose an OTP expires after 2 minutes.

I can install the clock, generate the OTP, fast-forward time by 2 minutes and verify whether the application rejects the expired OTP.

Q18. Can we test coupon expiration using Playwright Clock?

Answer:

Yes.

We can set the clock to a date before the coupon expiration date, verify that the coupon is valid, then change the time to a date after expiration and verify that the coupon is rejected.

Q19. Does fastForward() execute every missed interval callback?

Answer:

No. It fires timers that become due once immediately rather than executing every missed interval callback.

For executing timers consistently throughout the elapsed duration, we should use `runFor()`.

Q20. When should we use setSystemTime() instead of fastForward()?

Answer:

`setSystemTime()` is useful when we need to change the current system time without automatically triggering pending timers.

`fastForward()` is used when we want to advance time and trigger timers that have become due.

9. Real-Time Scenario-Based Interview Questions

These questions are especially useful for QA Automation and SDET interviews.

Q21. Your application automatically logs out a user after 30 minutes. How will you automate this test?

Answer:

I will use Playwright Clock.

1. Install the clock.
2. Login to the application.
3. Verify that the user is logged in.
4. Fast-forward the clock by 30 minutes.

5. Verify that the user is automatically logged out.

Example:

```
await page.clock.install();

await page.goto('https://example.com/login');

// Perform login here.

await page.clock.fastForward('30:00');

await expect(
  page.getByText('Session expired')
).toBeVisible();
```

Q22. Your application has a countdown timer of 60 seconds. How will you test it?

Answer:

I will use `install()`, `pauseAt()` and `runFor()`.

I will initialize the clock, open the application, pause it at the required time and advance it using `runFor()`.

Then I will verify whether the countdown decreases correctly.

Q23. How will you test a coupon that expires at midnight?

Answer:

I will use `setFixedTime()` or `install()` depending on the application's timer behavior.

I will:

1. Set the time before midnight.
2. Apply the coupon and verify that it is valid.
3. Change the time to after midnight.
4. Verify that the coupon is expired.

Q24. Your test is failing because the application uses the actual system date. How will you handle it?

Answer:

I will use Playwright Clock to control the date and time.

If only the current date needs to be controlled, I will use `setFixedTime()`.

If timer execution also needs to be controlled, I will use `install()`.

Q25. What is the difference between real-time testing and simulated-time testing?

Answer:

Real-time testing uses the actual system clock and requires waiting for the specified duration.

Simulated-time testing uses Playwright Clock to control the date and time, allowing us to verify time-dependent functionality quickly and consistently.

10. Practical Code – Complete Example

Let's take a common interview scenario: automatic logout after 5 minutes.

Here is a simplified Playwright test:

```
import { test, expect } from '@playwright/test';

test('Verify automatic logout after 5 minutes', async ({ page }) =>
{

  // Install clock
  await page.clock.install();

  // Open application
  await page.goto('http://localhost:3333');

  // Simulate user activity
  await page.getByRole('button', {
    name: 'Interaction'
  }).click();

  // Fast-forward 5 minutes
  await page.clock.fastForward('05:00');
```

```
  // Verify automatic logout
  await expect(
    page.getByText(
      'You have been logged out due to inactivity.'
    )
  ).toBeVisible();

});
```

How to explain this in an interview:

"I use Playwright Clock to test time-dependent functionality. For example, if an application has a 5-minute inactivity timeout, I install the clock before navigating to the application, simulate user activity, fast-forward the clock by 5 minutes and verify whether the user is automatically logged out. This avoids waiting for the actual 5 minutes."

11. Quick Revision Notes

Question	Answer
Control browser time	Playwright Clock
Fix date/time	`setFixedTime()`
Initialize controllable clock	`install()`
Pause at specific time	`pauseAt()`
Skip time quickly	`fastForward()`
Advance time consistently	`runFor()`
Resume clock	`resume()`
Change system time	`setSystemTime()`

Test session expiry	`fastForward()`
Test countdown	`runFor()`
Test specific date	`setFixedTime()`
Installation order	Install before other clock-related calls

12. Test Your Interview Knowledge

Try these questions to check your understanding.

Playwright Clock – Interview Quiz

7/7 answered

1. Which Playwright method fixes Date.now() and new Date() while timers continue normally?

install()
setFixedTime()
pauseAt()
runFor()

Correct!

setFixedTime() fixes the date/time without stopping normal timers.

2. Which method moves time forward and fires due timers once?

resume()
runFor()
fastForward()
setSystemTime()

Correct!

fastForward() quickly advances time and fires due timers once.

3. Which method executes timers consistently during a time advance?

runFor()
setFixedTime()
setSystemTime()
pauseAt()

Correct!

runFor() advances time while executing scheduled timers and animation frames.

4. When should page.clock.install() be called?

After all tests
After setInterval()
Before other clock-related calls
After browser.close()

Correct!

install() must be called before other clock-related operations.

5. Which method resumes a paused clock?

install()
resume()
fastForward()
setFixedTime()

Correct!

resume() resumes normal time progression.

6. You want to test an automatic logout after 5 minutes. Which method is most suitable?

fastForward('05:00')
setFixedTime() only
clearInterval()
browser.close()

Correct!

fastForward('05:00') simulates five minutes passing and triggers due timers.

7. Which method changes system time without automatically firing timers?

pauseAt()
runFor()
setSystemTime()
resume()

Correct!

setSystemTime() changes the clock's system time without automatically triggering timers.

Score: 7/7

Retry quiz

My recommendation for your interview preparation

For this Clock section, concentrate on these five things:

1. Difference between `setFixedTime()` and `install()`.
2. Difference between `fastForward()` and `runFor()`.
3. How to test session timeout.
4. How to test OTP and coupon expiration.
5. Why `install()` must be called before other clock-related operations.

These concepts are more important than memorizing every API method.

Send the next Playwright section whenever you're ready.

Playwright – Component Testing

Component testing is an important Playwright interview topic, especially for QA Automation and SDET roles.

Let's understand it step by step with simple explanations, practical examples, and detailed interview questions and answers.

1. What is component testing?

Component testing means testing one individual UI component separately, without testing the entire application flow.

A web application contains many components, such as:

- Login button
- Search box
- Navigation bar
- Dropdown
- Shopping cart
- Popup
- Product card

Instead of testing the complete application, we test whether each component works correctly on its own.

Example:

Suppose an application has an Expandable component.

- Initially, details are hidden.
- When the user clicks the button, details should appear.

We can test only this Expandable component without going through the entire application.

Component testing vs end-to-end testing

Component testing	End-to-end testing
Tests an individual component	Tests a complete user journey

Runs components in a gallery page	Runs the application
Focuses on component behavior	Focuses on application workflow
Example: Test a button click	Example: Login → Search → Add to cart → Checkout

Interview point: Playwright component testing runs the component in a real browser, so it can test real clicks, rendering, layout, and screenshots.

2. How does Playwright component testing work?

The documentation describes three important concepts:

1. Story

Defines the component and its test scenario.

Example: Primary button, Disabled button.

2. Gallery

A web page that renders the selected story.

It is served by your development server.

3. mount() fixture

Loads the story and returns a Locator for the component.

4. Test

Performs actions and verifies expected results.

Understand these three concepts

1. Story

A story is a small wrapper around a component that defines a particular scenario.

For example:

- Primary button
- Disabled button
- Button with a long title

2. Gallery

The gallery is a web page that displays the selected component story. It has two important functions:

- `window.mount(params)` – renders a story inside the gallery.
- `window.unmount()` – removes the rendered component.

3. mount() fixture

`mount()` is a built-in Playwright fixture used to render a component story.

Example:

```
const component = await mount('components/Button/Primary');
```

Here:

- `mount()` loads the story.
- `'components/Button/Primary'` is the story ID.
- `component` is the Locator returned for the gallery root.
- We can use locator methods on `component`.

3. Why does Playwright use a framework-agnostic approach?

Earlier, Playwright provided experimental component-testing packages:

- `@playwright/experimental-ct-react`
- `@playwright/experimental-ct-react17`
- `@playwright/experimental-ct-vue`

These packages have now been removed and are no longer published.

The new approach uses the normal `@playwright/test` package and your own development server.

Old approach vs new approach

Old experimental approach	New approach
Used separate experimental packages	Uses `@playwright/test`
JSX was written directly in test files	Components are defined in story files
Playwright managed its own bundling pipeline	Your application manages its own build pipeline
Separate configuration was required	Uses your existing development setup
Framework-specific packages	Framework-agnostic approach

What does framework-agnostic mean?

It means the component-testing approach is not restricted to one framework.

You can use it with:

- React
- Vue
- Svelte
- Solid
- Other frameworks that can render components through a gallery.

Interview question: Why did Playwright change its component-testing approach?

Answer: The old experimental packages required Playwright to manage its own bundler and build configuration. This created compatibility problems with custom

configurations, plugins, CSS, and different frameworks. The new approach allows developers to use their own development server and build pipeline. This makes component testing more flexible and framework-agnostic.

4. Getting started – Setup component testing

The documentation explains five steps.

Step 1: Install Playwright skills

```
npx playwright init-skills
```

This installs Playwright skills that coding agents can use to help configure component testing.

You can ask your coding agent:

```
Set up component testing using the
playwright-component-testing skill.
```

The agent can help:

- Identify your framework.
- Create the gallery.
- Configure the Playwright project.
- Create the first story.
- Create the first component test.

Important: The gallery is application code that your project owns. It is not automatically provided as a separate Playwright testing runtime.

Step 2: Configure Playwright

File: `playwright.config.ts`

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  projects: [
    {
```

```
      name: 'components',
      testDir: './tests/components',
      use: {
        ...devices['Desktop Chrome'],
        baseURL:
'http://localhost:5173/playwright/gallery/index.html',
        serviceWorkers: 'block',
        reuseContext: true,
      },
    },
  ],

  webServer: {
    command: 'npm run dev',
    url: 'http://localhost:5173/playwright/gallery/index.html',
    reuseExistingServer: !process.env.CI,
  },
});
```

Let's understand the important options.

Configuration	Meaning
`name: 'components'`	Name of the Playwright project
`testDir`	Location of component test files
`devices['Desktop Chrome']`	Uses Desktop Chrome settings
`baseURL`	URL of the component gallery
`serviceWorkers: 'block'`	Prevents service workers from interfering with route mocks
`reuseContext: true`	Reuses the browser context between tests in a worker
`webServer`	Starts the development server before tests run

`npm run dev`	Command to start the application development server

Important interview question: Why do we use `serviceWorkers: 'block'`?

Answer: It prevents the application's service worker from serving cached responses that could interfere with network mocking using `page.route()`.

What is `reuseContext: true`?

It allows tests running in the same worker to reuse the browser context. This can improve component-suite execution speed.

Step 3: Create a story

File: `src/components/Button.story.tsx`

```
import { Button } from './Button';

export const Primary = () => (
  <Button title="Submit" />
);

export const Disabled = () => (
  <Button title="Submit" disabled />
);
```

Explanation:

- `Button.story.tsx` contains stories for the Button component.
- `Primary` represents a normal button.
- `Disabled` represents a disabled button.
- Each named export represents one scenario.

Step 4: Write the component test

File: `tests/components/button.spec.ts`

```
import { test, expect } from '@playwright/test';

test('renders primary button', async ({ mount }) => {
```

```
  const component = await mount('components/Button/Primary');

  await expect(component.getByRole('button'))
    .toHaveText('Submit');
});

test('disabled button is disabled', async ({ mount }) => {
  const component = await mount('components/Button/Disabled');

  await expect(component.getByRole('button'))
    .toBeDisabled();
});
```

Let's understand:

First test:

1. `mount()` loads the Primary story.
2. `getByRole('button')` locates the button.
3. `toHaveText('Submit')` verifies its text.

Second test:

1. `mount()` loads the Disabled story.
2. `getByRole('button')` locates the button.
3. `toBeDisabled()` checks whether the button is disabled.

Step 5: Run the component tests

```
npx playwright test --project=components
```

This runs only the tests belonging to the `components` project.

5. Stories as a testing methodology

Stories are not just used to make component testing possible. They also work as documentation for different component states.

For example:

```
export const Primary = () => <Button title="Submit" />;

export const Disabled = () => <Button title="Submit" disabled />;

export const WithLongTitle = () =>
  <Button title="Submit Your Application Now" />;
```

Each story represents a different scenario.

Important rules:

1. Prefer one named export per scenario.
2. Keep story files next to the component.
3. Give each scenario a meaningful name.
4. The story should contain everything the component needs, including providers and mock data.
5. The test should focus on user interactions and assertions.

How is a story ID created?

Suppose the story file is:

`src/components/Button.story.tsx`

And it contains:

```
export const Primary = () => <Button title="Submit" />;
```

Its story ID can be:

`components/Button/Primary`

You use this ID in:

```
const component = await mount('components/Button/Primary');
```

6. Testing patterns

This section is particularly important for interviews.

Pattern 1: Record state for assertions

Suppose we have an Expandable component.

Expected behavior:

- Initially, the component is collapsed.
- When the user clicks the button, it expands.
- We want to verify that the expanded state becomes `true`.

Story:

```
import { useState } from 'react';
import { Expandable } from './Expandable';

export const Stateful = () => {
  const [expanded, setExpanded] = useState(false);

  return <>
    <Expandable
      expanded={expanded}
      setExpanded={setExpanded}
      title="Title"
    >
      Details
    </Expandable>

    <form hidden>
      <input
        data-testid="expanded"
        readOnly
        value={String(expanded)}
      />
    </form>
  </>;
};
```

Test:

```
test('click should expand', async ({ mount }) => {
  const component = await mount(
    'components/Expandable/Stateful'
```

```
  );

  await component.getByRole('button').click();

  await expect(
    component.getByTestId('expanded')
  ).toHaveValue('true');
});
```

Understand the execution:

1. `useState(false)` initializes the expanded state as false.
2. The story renders the Expandable component.
3. The hidden input stores the current state.
4. The test mounts the story.
5. The test clicks the button.
6. The component updates its state to true.
7. The input value changes to `"true"`.
8. `toHaveValue('true')` verifies the result.

Why do we use a hidden input?

Because the test runs in Node.js, while the component runs inside the browser.

We should not try to directly access the component's internal state or pass live callback functions between Node.js and the browser.

Instead, the story records the observable result in the DOM, and the test verifies it using a locator.

Why use `String(expanded)`?

The state is a Boolean (`true` or `false`), but an HTML input's value is a string. So we convert the Boolean into a string.

Interview question: How do you test component state changes in Playwright component testing?

Answer: I maintain the component state inside the story using the framework's state management, such as React's `useState`. I record the updated value in a DOM element with a `data-testid`. Then, from the test, I interact with the component and verify the updated value using a web-first assertion such as `toHaveValue()`.

Pattern 2: Per-test props

Sometimes we want to test the same component with different input values.

Instead of creating multiple stories, we can pass serializable props to `mount()`.

Story:

```
import { Button } from './Button';

export const WithTitle = ({
  title = 'Default'
}: {
  title?: string
}) => <Button title={title} />;
```

Test:

```
const component = await mount(
  'Button/WithTitle',
  { title: 'Hello' }
);

await expect(
  component.getByRole('button')
).toHaveText('Hello');
```

Explanation:

- `WithTitle` accepts a title property.
- The default title is `Default`.
- The test passes `Hello` as the title.
- The component renders the button with `Hello`.

What are serializable props?

They are values that can be safely represented as data and passed between the test and browser, such as:

- Strings
- Numbers
- Booleans
- Arrays
- Plain objects

Callbacks should be defined inside the story instead.

Pattern 3: Prop transitions using `update()`

Suppose a Counter component initially displays 1.

We want to change its prop to 2 without mounting the component again.

```
const component = await mount(
  'components/Counter/Default',
  { value: 1 }
);

await expect(
  component.getByTestId('value')
).toHaveText('1');

await component.update({ value: 2 });

await expect(
  component.getByTestId('value')
).toHaveText('2');
```

Explanation:

- `mount()` renders the component with value 1.
- `update()` changes its props to value 2.
- The existing rendering root is reused.
- Component state is preserved rather than resetting through a fresh mount.

Important difference:

`mount()`	`update()`
Loads a story	Updates props of an already mounted story
Starts a new mount/navigation	Does not remount the component
Used to render a scenario	Used to test prop changes

Pattern 4: Multiple states and visual comparison

You can test the visual appearance of different component states.

```
await expect(
  await mount('Button/Primary')
).toHaveScreenshot('primary.png');

await expect(
  await mount('Button/Disabled')
).toHaveScreenshot('disabled.png');
```

This performs visual regression testing.

It compares the current screenshot against the expected baseline screenshot.

Interview question: Can Playwright component testing perform visual testing?

Answer: Yes. We can use `toHaveScreenshot()` on the returned component Locator to capture and compare the component's appearance. It helps identify unexpected visual changes.

Important: Screenshot the component Locator rather than the entire page, so unrelated gallery content does not affect the screenshot.

Pattern 5: Handling network requests

Suppose the component loads product information from an API.

We want to test how it behaves when the API returns a server error.

```
test('renders the error state', async ({ page, mount }) => {

  await page.route('**/api/items', route =>
    route.fulfill({
      status: 500
    })
  );

  const component = await mount(
    'components/ItemList/Default'
  );

  await expect(
    component.getByRole('alert')
  ).toContainText('Something went wrong');

});
```

Explanation:

1. `page.route()` intercepts requests matching the API URL.
2. `route.fulfill()` returns a mocked response.
3. Status `500` represents a server error.
4. `mount()` renders the component.
5. The test verifies that the error message appears.

Important interview point: Register `page.route()` before calling `mount()` because mounting navigates to the gallery and the component may make its API request during rendering.

Pattern 6: Debugging stories

You can open the gallery URL directly in a browser.

You can also run this command in the browser DevTools console:

```
await window.mount({
  story: 'components/Button/Primary'
});
```

This allows you to inspect the component manually.

If a story ID is incorrect or the component fails to render, `window.mount()` rejects, and the test's `mount()` call throws an error.

7. Migration from experimental packages

The new methodology replaces the older experimental component-testing packages.

Old method	New method
`mount(<Button />)`	`mount('Button/Primary')`
JSX written in test	JSX written in story
`component.update(<Button count={2} />)`	`component.update({ count: 2 })`
`component.unmount()`	`component.unmount()`
`ctViteConfig`	No longer needed
Experimental CT package	`@playwright/test`

Important migration note: The documentation recommends migrating incrementally while pinned to Playwright 1.62, then removing the old package and upgrading after migration.

8. Important interview questions and answers

Here are questions you should prepare for a Playwright QA Automation or SDET interview.

Basic questions

Q1. What is component testing in Playwright?

Answer: Component testing is the process of testing individual UI components independently from the complete application. Playwright renders the component in a browser and allows us to verify its behavior, appearance, and interactions.

Q2. Is component testing the same as end-to-end testing?

Answer: No. Component testing focuses on an individual component, whereas end-to-end testing validates a complete user workflow across the application.

Q3. Which package is used for the new Playwright component testing approach?

Answer: The standard `@playwright/test` package. The built-in `mount()` fixture is used to mount components.

Q4. What is a story in component testing?

Answer: A story is a wrapper around a component that defines a particular scenario. For example, a Button component can have Primary, Disabled, and WithLongTitle stories.

Q5. What is a component gallery?

Answer: A gallery is a web page served by the application's development server. It discovers stories and provides functions to mount and unmount components.

Q6. What is the purpose of the `mount()` fixture?

Answer: It loads a specific story in the gallery and returns a Locator for the gallery root. We can use this Locator to interact with and verify the component.

Q7. What is the difference between `window.mount()` and Playwright's `mount()`?

Answer:

- `window.mount()` is a browser-side function implemented by the gallery to render a story.
- Playwright's `mount()` is a test fixture that navigates to the gallery, calls the browser-side function, and returns a Locator.

Intermediate questions

Q8. Why is Playwright component testing called framework-agnostic?

Answer: Because Playwright does not manage component compilation or rendering through a framework-specific testing runtime. The developer owns the gallery and can implement it using React, Vue, Svelte, Solid, or another framework.

Q9. Why do we use `data-testid` in component testing?

Answer: We use `data-testid` to identify an element that exposes observable state or data needed by the test. For example, a hidden input can record whether a component is expanded.

Q10. Why do we use `toHaveValue()` instead of a normal assertion?

Answer: `toHaveValue()` is a web-first assertion. It automatically retries until the expected value appears or the timeout is reached. This helps handle asynchronous UI updates.

Q11. Can we pass callback functions directly from the test to the component?

Answer: In this approach, callbacks should be defined inside the story because the test runs in Node.js and the component runs in the browser. We record callback effects in the DOM and verify them using Playwright locators.

Q12. What is the purpose of `component.update()`?

Answer: It updates the props of an already mounted component without remounting it. It is useful for testing how a component reacts when its input properties change.

Q13. How do you mock API responses in component testing?

Answer: We use `page.route()` to intercept the API request and `route.fulfill()` to provide a mocked response. We register the route before calling `mount()`.

Q14. Why do we use `serviceWorkers: 'block'`?

Answer: It prevents service workers from serving cached responses that might interfere with API request interception and mocking.

Q15. How do you perform visual regression testing?

Answer: We use `toHaveScreenshot()` on the component Locator. Playwright compares the captured screenshot against the stored baseline to detect visual changes.

Scenario-based questions

Q16. You have a Login button. How will you test whether it is disabled?

Answer: I will create a story for the disabled Login button, mount that story, locate the button using `getByRole('button')`, and verify it using `toBeDisabled()`.

Q17. You have a dropdown. How will you test that clicking it opens the options?

Answer: I will create a story for the dropdown, mount it, click the dropdown using a Locator, and verify that the expected options become visible using `toBeVisible()`.

Q18. How will you test that a component displays an error when an API returns 500?

Answer: I will mock the API using `page.route()` and return a 500 response using `route.fulfill()`. After mounting the component, I will verify the error message using `toContainText()`.

Q19. How will you test a component with different input values?

Answer: I can create a story that accepts props and pass different serializable values through the second argument of `mount()`. This allows me to test multiple input scenarios.

Q20. How will you test a component when its props change dynamically?

Answer: I will mount the component with the initial props, verify the initial output, call `component.update()` with new props, and verify the updated output.

Q21. Can we access a component's internal methods or instance directly?

Answer: It is not recommended or supported by this methodology. We should test components from the user's perspective by performing interactions and checking observable results in the DOM.

Q22. What happens if a story ID is incorrect?

Answer: The gallery cannot resolve the story, so `window.mount()` rejects. The Playwright `mount()` fixture then throws an error.

Q23. Can we use the existing application CSS and plugins?

Answer: Yes. The gallery runs through our own development server, so it can use the application's existing CSS, plugins, aliases, and build configuration.

Q24. Why should stories live next to their components?

Answer: It makes component scenarios easier to find, maintain, review, and update when the component is renamed or refactored.

Q25. What is the main advantage of the new approach over the experimental packages?

Answer: It gives developers control over their own build pipeline, supports different frameworks, avoids callback marshalling between Node.js and the browser, and uses the standard Playwright Test package.

9. Quick revision notes

Topic	Remember
Component testing	Tests individual UI components
Story	Defines a component scenario
Gallery	Page that renders stories
`window.mount()`	Browser-side rendering function
`mount()`	Playwright fixture
`window.unmount()`	Removes the rendered component
`component.update()`	Updates props without remounting

`data-testid`	Identifies recorded state
`toHaveValue()`	Verifies input value
`toHaveScreenshot()`	Visual regression testing
`page.route()`	Intercepts network requests
`route.fulfill()`	Returns mocked API response
`serviceWorkers: 'block'`	Prevents service-worker interference
`reuseContext: true`	Reuses browser context in a worker

10. Test your understanding

Try these interview-style questions. Select an answer to check your understanding.

Component Testing Quiz

5/5 answered

1. What is a story in Playwright component testing?

A complete end-to-end test
A wrapper defining a component scenario
A browser configuration file
An API response

Correct answer: A wrapper defining a component scenario

A story wraps a component in a particular scenario, such as Primary or Disabled.

2. Which fixture is used to render a component story?

page()
browser()
mount()

context()

Correct

The built-in mount fixture loads a story in the component gallery.

3. Which method updates props without remounting?

page.reload()
component.update()
window.unmount()
route.fulfill()

Correct answer: component.update()

component.update() updates props on the existing rendering root.

4. Which method intercepts network requests?

page.route()
page.goto()
page.screenshot()
mount()

Correct

page.route() intercepts matching network requests so you can mock responses.

5. Why do we record state in a hidden input?

To improve CSS
To pass callbacks directly to Node.js
To expose observable state in the DOM for assertions
To stop the browser

Correct

The test can read the DOM value without accessing the component's internal state.

Check answers

3 / 5 correct

Try again

Most important for your interview: Understand the difference between `story`, `gallery`, and `mount()`, and practise the `mount()`, `update()`, `page.route()`, and `toHaveScreenshot()` examples. These are the core concepts of this section.

Playwright – Debugging Tests

Debugging is one of the most important topics for Playwright QA Automation and SDET interviews.

Debugging means finding, understanding, and fixing errors when an automated test fails or does not behave as expected.

For example:

- A button is not getting clicked.
- A locator matches multiple elements.
- An element is not visible.
- An assertion fails.
- An API request returns an unexpected response.
- A test passes locally but fails in CI.

Playwright provides several debugging tools:

1. VS Code debugger
2. Playwright Inspector
3. Trace Viewer
4. Browser Developer Tools
5. Verbose API logs
6. Headed mode and slow motion

Let's understand each one with practical examples.

1. VS Code debugger

Debugging Tests | Playwright
playwright.dev

Playwright recommends using the VS Code extension because it allows you to debug tests directly inside VS Code.

You can:

- Set breakpoints.
- Run tests in debug mode.
- View error messages.
- Step through test execution.
- Inspect locators.
- Highlight matching elements in the browser.
- Pick locators directly from the browser.

A. Error messages

Suppose your test contains:

```
await expect(
  page.getByRole('button', { name: 'Login' })
).toHaveText('Submit');
```

But the actual button text is `Login`.

The assertion fails because:

- Expected: `Submit`
- Received: `Login`

VS Code displays the error directly in the editor, along with the call log.

What is a call log?

A call log shows the sequence of actions Playwright performed before the test failed.

It helps you understand:

- Which action failed.
- Which locator was used.
- What Playwright expected.
- What actually happened.

B. Live debugging

Live debugging allows you to inspect locators while a test is running.

For example:

```
await page.getByRole('button', {
  name: 'Login'
}).click();
```

When the test is running with the browser visible, clicking the locator in VS Code highlights the matching element in the browser.

If multiple elements match, Playwright also shows the matching elements.

You can even edit the locator in VS Code and see the matching elements update live.

Interview question: What is live debugging in Playwright?

Answer: Live debugging allows us to inspect and modify locators while the test is running. Playwright highlights the matching elements in the browser, which helps us identify incorrect or duplicate locators.

C. Pick a locator

Best Practices | Playwright
playwright.dev

This feature helps you generate a locator without manually writing it.

Steps:

1. Open the Testing sidebar in VS Code.
2. Click Pick Locator.

3. Hover over the required element in the browser.
4. Click the element.
5. Playwright generates the locator.
6. Press Enter to copy it to the clipboard.
7. Paste it into your test file.

For example, Playwright may generate:

```
page.getByRole('button', { name: 'Login' });
```

Playwright prioritizes:

- Role locators
- Text locators
- Test ID locators

It tries to generate a locator that uniquely identifies the target element.

Why is Pick Locator useful?

It saves time when writing locators and helps create more reliable locators.

D. Run in Debug Mode

You can set a breakpoint by clicking next to the line number in your test file.

A red dot appears.

Then:

1. Right-click near the test you want to debug.
2. Select Debug Test.
3. Playwright opens the browser.
4. Test execution pauses at the breakpoint.
5. You can execute the test step by step.

What is a breakpoint?

A breakpoint is a point in the code where execution pauses temporarily so that we can inspect the test.

E. Debug using Chrome DevTools

You can also use Chrome DevTools for debugging.

According to the documentation:

1. Select Run Test instead of Debug Test.
2. Enable Show Browser.
3. Run the test.
4. The browser session is reused.
5. Open Chrome DevTools to inspect the application.

This is useful when you want to inspect:

- DOM elements
- Console errors
- Network requests
- JavaScript execution

F. Debug in different browsers

By default, VS Code debugging uses the Chromium profile.

You can select another browser profile from the Testing sidebar.

For example:

- Chromium
- Firefox
- WebKit
- Mobile Safari
- Microsoft Edge

This helps identify browser-specific failures.

2. Playwright Inspector

Running and debugging tests | Playwright
playwright.dev

Playwright Inspector is a GUI tool used to debug Playwright tests.

It allows you to:

- Execute tests step by step.
- Pause and resume execution.
- Set breakpoints.
- Edit locators.
- Pick locators.
- View actionability logs.
- Inspect matching elements.

A. Run tests in debug mode

Use:

```
npx playwright test --debug
```

This command opens Playwright Inspector and runs tests in debug mode.

Important defaults when using `--debug`:

Feature	Behavior
Browser	Headed mode
Default timeout	0 (no timeout)

Test execution	One test at a time
Inspector	Opens automatically

What is headed mode?

The browser opens visibly, so you can watch the test execution.

What does timeout 0 mean?

The default timeout is disabled for debugging, allowing you to inspect the test without it failing just because you paused for too long.

B. Debug all tests

```
npx playwright test --debug
```

This runs tests one by one with the Inspector and browser window.

C. Debug one specific test

```
npx playwright test example.spec.ts:10 --debug
```

Explanation:

- `example.spec.ts` is the test file.
- `10` is the line number containing the test.
- `--debug` opens the Inspector.

This is useful when you don't want to debug the complete test suite.

D. Debug on a specific browser

You can use `--project` to debug tests on a specific browser.

```
npx playwright test --project=chromium --debug
npx playwright test --project="Mobile Safari" --debug
npx playwright test --project="Microsoft Edge" --debug
```

The project name must match the project configured in `playwright.config.ts`.

E. Debug one test on a specific browser

```
npx playwright test example.spec.ts:10 --project=webkit --debug
```

This runs the selected test on the WebKit project.

F. Stepping through tests

The Inspector provides controls to:

- Play – continue test execution.
- Pause – pause test execution.
- Step over – execute the current step and move to the next one.
- Step into – enter a function or method.
- Step out – finish the current function and return to its caller.

The current action is highlighted in the test code, and matching elements are highlighted in the browser.

Interview question: What is Playwright Inspector?

Answer: Playwright Inspector is a GUI debugging tool that allows us to pause tests, execute them step by step, inspect locators, pick elements, and view actionability logs.

G. Use `page.pause()` to stop execution

Suppose your test has 20 steps, but you want to debug only step 15.

Instead of stepping through all previous actions, use:

```
await page.pause();
```

Example:

```
import { test, expect } from '@playwright/test';

test('login test', async ({ page }) => {

  await page.goto('https://example.com/login');

  await page.getByLabel('Username').fill('admin');
```

```
  await page.getByLabel('Password').fill('password123');

  await page.pause();

  await page.getByRole('button', {
    name: 'Login'
  }).click();

  await expect(page).toHaveURL(/dashboard/);

});
```

Run the test in debug mode:

```
npx playwright test --debug
```

Execution pauses at `page.pause()`.

You can inspect the page, check locators, and then click Resume in the Inspector to continue.

Important interview question: What is the purpose of `page.pause()`?

Answer: `page.pause()` pauses test execution at a specific point and opens the Playwright Inspector. It helps us inspect the current browser state and debug a particular action without stepping through all previous actions.

H. Live editing locators

While the test is paused, you can edit a locator in the Inspector's Pick Locator field.

For example, change:

```
page.locator('.login-btn');
```

to:

```
page.getByRole('button', { name: 'Login' });
```

Playwright highlights the matching elements immediately.

This helps you find a better locator before updating your test code.

I. Actionability logs

This is a very important interview topic.

Before performing an action such as `click()`, Playwright checks whether the element is ready for interaction.

For example:

```
await page.getByRole('button', {
  name: 'Submit'
}).click();
```

Playwright checks whether the element:

- Exists and resolves to a unique element.
- Is visible.
- Is stable.
- Is enabled.
- Receives pointer events.

If the checks are not satisfied, Playwright waits and retries until the actionability conditions are met or the timeout is reached.

Example:

Suppose the Submit button is covered by a loading popup.

Playwright may not be able to click it because the button does not receive pointer events.

The actionability log helps identify this problem.

Example actionability log – simplified

Locator resolved to an element

Element is visible

Element is stable

Element is enabled

Element receives pointer events – failed

Click action pending

Interview question: What are actionability checks in Playwright?

Answer: Actionability checks are automatic checks performed before an action. For example, before clicking an element, Playwright checks whether it is visible, stable, enabled, and able to receive pointer events. These checks help make tests more reliable.

3. Trace Viewer

Best Practices | Playwright
playwright.dev

Trace Viewer is a GUI tool that helps us investigate what happened during a Playwright test execution.

It is especially useful when a test fails in CI/CD and you cannot reproduce the failure locally.

Trace Viewer displays:

Section	What it shows
Left side	List of test actions
Middle	DOM snapshot of the selected action
Right side	Action details, parameters, return value, and logs
Console	Browser console messages
Network	Network requests and responses
Source	Test source code

Example

Suppose your test fails while clicking the Login button.

With Trace Viewer, you can:

1. Select the failed click action.
2. Inspect the DOM snapshot.
3. Check which element was located.
4. View the action parameters.
5. Inspect console errors.
6. Check network requests.
7. Understand why the action failed.

Important: Trace Viewer displays recorded test information. You need to configure trace recording to have a trace available.

Interview question: What is Playwright Trace Viewer?

Answer: Trace Viewer is a graphical debugging tool that allows us to inspect recorded test execution. It provides action history, DOM snapshots, screenshots, logs, network activity, console messages, and source code. It is particularly useful for debugging failures in CI/CD environments.

Difference between Inspector and Trace Viewer

Playwright Inspector	Trace Viewer
Mainly used for live debugging	Mainly used for reviewing recorded execution
Can pause and resume tests	Navigate through recorded actions
Can edit and pick locators	Inspect recorded DOM snapshots
Shows live actionability logs	Shows recorded action details and logs

Useful during development	Useful for investigating completed runs

4. Browser Developer Tools

📘 Day 1 - 30 Days Javascript
30dayjavascript.js.org

Browser Developer Tools help us inspect the application directly.

They are useful for:

- Inspecting the DOM.
- Finding element selectors.
- Checking JavaScript console errors.
- Monitoring network requests.
- Inspecting application behavior.

How to debug using Developer Tools

First, add a pause in your test:

```
await page.pause();
```

Then run your test using `PWDEBUG=console`.

Bash:

```
PWDEBUG=console npx playwright test
```

PowerShell:

```
$env:PWDEBUG="console"
npx playwright test
```

Once the browser opens, open Developer Tools.

The `playwright` object becomes available in the browser console.

Important Playwright console methods

1. `playwright.$(selector)`

Finds the first element matching a Playwright selector.

```
playwright.$('.auth-form >> text=Log in');
```

2. `playwright.$$(selector)`

Finds all elements matching the selector.

```
playwright.$$('li >> text=John');
```

Difference:

Method	Meaning
`playwright.$()`	Returns the first matching element
`playwright.$$()`	Returns all matching elements

3. `playwright.inspect(selector)`

Reveals the matching element in the browser's Elements panel.

```
playwright.inspect('text=Log in');
```

4. `playwright.locator(selector)`

Creates a Playwright Locator that you can inspect.

```
playwright.locator('.auth-form', {
  hasText: 'Log in'
});
```

5. `playwright.selector(element)`

Generates a Playwright selector for a DOM element.

Example:

```
playwright.selector($0);
```

Here, `$0` refers to the currently selected element in the browser's Elements panel.

Interview question: What is the purpose of `PWDEBUG=console`?

Answer: It launches Playwright in debugging mode and exposes the Playwright object in the browser Developer Tools console. We can use this object to inspect elements, find selectors, create locators, and debug the page.

5. Verbose API logs

Sometimes a test fails, but you don't know which Playwright action caused the problem.

Playwright supports verbose logging using the `DEBUG` environment variable.

Command:

Bash:

```
DEBUG=pw:api npx playwright test
```

PowerShell:

```
$env:DEBUG="pw:api"
npx playwright test
```

This prints detailed API logs in the terminal.

For example, you can see logs related to:

- Navigation
- Locator resolution
- Click actions
- Waiting
- Assertions

What is verbose logging?

Verbose logging means displaying detailed information about what the program is doing during execution.

Interview question: How do you enable detailed Playwright API logs?

Answer: We can set the DEBUG environment variable to `pw:api` and run the Playwright test. This displays detailed API execution logs in the terminal.

Important note: When using WebKit, launching WebKit Inspector during execution can prevent the Playwright script from continuing and reset preconfigured user-agent and device-emulation settings.

6. Headed mode

By default, Playwright runs browsers in headless mode.

Headless means the browser runs in the background without displaying its user interface.

Sometimes we want to see the browser while the test executes. In that case, we use headed mode.

Headless vs headed

Headless Browser Testing in Playwright | ArtifactGeeks

Headless mode

Browser runs without a visible window.

Playwright Testing Essentials: A Beginner's Guide | Better Stack Community

Headed mode

Browser opens visibly during execution.

To enable headed mode:

```
import { chromium } from '@playwright/test';

const browser = await chromium.launch({
  headless: false
});
```

What is `slowMo`?

`slowMo` slows down browser operations by a specified number of milliseconds.

Example:

```
const browser = await chromium.launch({
  headless: false,
  slowMo: 100
});
```

Here:

- `headless: false` opens the browser visibly.
- `slowMo: 100` adds a 100 ms delay to each browser operation.

This makes it easier to watch the execution.

Interview question: What is the difference between headless and headed mode?

Answer: In headless mode, the browser runs without a visible UI, which is useful for automated CI/CD execution. In headed mode, the browser opens visibly, making it easier to observe and debug test execution.

7. Important interview questions and answers

These are the questions I recommend preparing for QA Automation and SDET interviews.

Basic questions

Q1. What is debugging in Playwright?

Answer: Debugging is the process of identifying and fixing issues in automated tests. Playwright provides tools such as Inspector, Trace Viewer, VS Code debugger, and browser Developer Tools.

Q2. Which tools can we use to debug Playwright tests?

Answer:

1. VS Code debugger
2. Playwright Inspector
3. Trace Viewer

4. Browser Developer Tools
5. Verbose API logs
6. Headed mode

Q3. What is Playwright Inspector?

Answer: It is a GUI debugging tool that allows us to pause tests, execute them step by step, inspect locators, pick locators, and view actionability logs.

Q4. How do you run Playwright tests in debug mode?

Answer:

```
npx playwright test --debug
```

This opens the Playwright Inspector and launches the browser in headed mode.

Q5. What is a breakpoint?

Answer: A breakpoint is a point in the test code where execution pauses so that we can inspect the current state and debug the test.

Q6. What is `page.pause()`?

Answer: It pauses test execution at a specific point and opens the Playwright Inspector. It is useful when we want to debug a particular step.

Q7. What is Trace Viewer?

Answer: Trace Viewer is a GUI tool that helps us investigate recorded test execution using action history, DOM snapshots, logs, screenshots, network activity, and console messages.

Intermediate questions

Q8. What is the difference between `--debug` and `--headed`?

Answer:

- `--debug` opens the Inspector and configures the test for debugging, including headed execution and no default timeout.
- `--headed` only makes the browser visible. It does not automatically provide the Inspector's debugging experience.

Q9. What is the difference between `page.pause()` and a breakpoint?

Answer:

- A breakpoint is generally set using the editor or debugger at a particular code line.
- `page.pause()` is an explicit Playwright statement that pauses execution when reached and opens the Inspector.

Q10. What is the purpose of the Pick Locator feature?

Answer: It allows us to select an element directly from the browser and generate a locator. Playwright prioritizes user-facing locators such as role, text, and test ID.

Q11. What are actionability logs?

Answer: Actionability logs show the checks Playwright performs before interacting with an element, such as checking visibility, stability, enabled state, and whether it receives pointer events.

Q12. What is the purpose of `PWDEBUG=console`?

Answer: It enables debugging support in browser Developer Tools and exposes the Playwright object in the browser console for inspecting elements and creating locators.

Q13. How can you generate a locator for an element selected in Chrome DevTools?

Answer:

```
playwright.selector($0);
```

Here, `$0` represents the currently selected DOM element in the Elements panel.

Q14. How do you enable verbose Playwright logs?

Answer:

```
DEBUG=pw:api npx playwright test
```

It prints detailed Playwright API execution information in the terminal.

Q15. What is the purpose of `slowMo`?

Answer: `slowMo` adds a delay between browser operations so we can visually follow the test execution.

Scenario-based questions

Q16. Your test passes locally but fails in Jenkins. How will you debug it?

Answer:

I would first check the Jenkins execution logs and failure message. Then I would inspect the Playwright trace, if available, to understand which action failed. I would check the DOM snapshot, screenshots, console errors, network requests, and actionability logs. This helps identify whether the issue is related to timing, locators, environment configuration, or application behavior.

Q17. Your locator matches two buttons. How will you debug it?

Answer:

I would use Pick Locator or inspect the locator in Playwright Inspector. I would check the matching elements and improve the locator using a unique role, accessible name, text, or test ID.

Q18. Your click action is pending for a long time. What will you check?

Answer:

I would inspect the actionability logs to determine whether the element is visible, stable, enabled, and receiving pointer events. I would also check whether an overlay or animation is blocking the element.

Q19. Your test fails because an element is not visible. What will you do?

Answer:

I would inspect the DOM and browser state using Inspector or Trace Viewer. I would check whether the element is hidden, outside the viewport, covered by another element, or displayed only after another action.

Q20. How will you debug a test that fails only in Firefox?

Answer:

I would run the test using the Firefox project and inspect the failure using Inspector or Trace Viewer. I would check browser-specific behavior, selectors, timing, and application compatibility.

Q21. How will you debug a test without executing all previous steps manually?

Answer:

I can add `await page.pause()` at the required location and run the test in debug mode. Execution will pause at that point.

Q22. How do you debug a test that has already failed in CI?

Answer:

I would open its recorded trace using Trace Viewer. I can inspect the failed action, DOM snapshot, logs, console messages, and network requests without rerunning the entire test locally.

8. Important commands for interview revision

Purpose	Command
Debug all tests	`npx playwright test --debug`
Debug a particular test	`npx playwright test example.spec.ts:10 --debug`
Debug Chromium	`npx playwright test --project=chromium --debug`
Debug WebKit	`npx playwright test example.spec.ts:10 --project=webkit --debug`
Enable API logs – Bash	`DEBUG=pw:api npx playwright test`
Enable console debugging – Bash	`PWDEBUG=console npx playwright test`

9. Quick revision: Which tool should you use?

Situation	Recommended tool

Want to pause and execute tests step by step	Playwright Inspector
Want to inspect a previous failed CI execution	Trace Viewer
Want to generate a locator from the browser	Pick Locator
Want to inspect DOM and network activity	Browser DevTools
Want detailed execution logs	`DEBUG=pw:api`
Want to watch browser execution	Headed mode
Want to slow down execution	`slowMo`
Want to stop at a particular test step	`page.pause()`

My recommendation: For interviews, give special attention to Inspector vs Trace Viewer, `page.pause()`, actionability logs, Pick Locator, and debugging CI failures. These are particularly useful practical questions.

10. Test your understanding

Debugging Tests Quiz

5/5 answered

1. Which command opens Playwright Inspector?

npx playwright test --headed
npx playwright test --debug
npx playwright test --ui
npx playwright test --list

Correct

The --debug flag opens the Inspector and configures tests for debugging.

2. Which method pauses execution at a specific point?

page.stop()
page.wait()
page.pause()
page.debug()

Correct

page.pause() pauses execution and opens Playwright Inspector.

3. Which tool is best for investigating a previously failed CI test?

Trace Viewer
Pick Locator
slowMo
Headed mode

Correct

Trace Viewer lets you inspect recorded actions, snapshots, logs, network, and console messages.

4. What does DEBUG=pw:api enable?

Visual testing
Verbose Playwright API logs
Mobile emulation
Browser installation

Correct

It prints detailed Playwright API logs in the terminal.

5. What does slowMo: 100 do?

Waits 100 seconds
Adds 100 ms delay between browser operations
Disables timeouts
Runs 100 tests

Correct

slowMo slows down browser operations by the specified milliseconds.

Check answers

5 / 5 correct

Try again

Send me the next Playwright documentation section, and I'll explain it in the same format with practical examples and detailed interview questions and answers.

Playwright – Dialogs

Dialogs are a common topic in Playwright interviews, especially for QA Automation and SDET roles.

Let's understand the different types of browser dialogs, how to handle them, practical examples, and important interview questions.

1. What are dialogs in Playwright?

Dialogs are browser popup messages that appear when a web page performs certain actions.

For example:

- A website displays an alert message.
- A website asks you to confirm deletion.
- A website asks you to enter your name.
- A website asks whether you want to leave a page with unsaved changes.

Playwright supports four main types of dialogs:

1. Alert dialog

How To Write Your First JavaScript Program | DigitalOcean
digitalocean.com

Displays a message with an OK button.

2. Confirm dialog

JavaScript | ダイアログを表示する(window.alert,window.prompt,window.confirm)

javadrive.jp

Asks the user to confirm or cancel an action.

3. Prompt dialog

prompt and confirm | Spaceship 🛸
spaceship.roadtocode.org

Asks the user to enter some text.

4. Beforeunload dialog

React: How To Prompt User of Unsaved Data before Leaving Site | by Lucia | Medium
medium.com

Asks whether the user wants to leave a page, commonly when there are unsaved changes.

Real-world examples:

Dialog	Example
Alert	Registration successful
Confirm	Are you sure you want to delete this account?
Prompt	Please enter your username
Beforeunload	You have unsaved changes. Leave this site?

2. Alert, Confirm and Prompt dialogs

First, understand the JavaScript methods that create these dialogs.

A. Alert dialog – `alert()`

An alert displays a message with an OK button.

Example JavaScript:

```
alert('Login successful!');
```

Playwright test:

```
import { test, expect } from '@playwright/test';

test('handle alert dialog', async ({ page }) => {

  await page.goto('https://example.com');

  page.on('dialog', async dialog => {
    console.log(dialog.message());

    await dialog.accept();
  });

  await page.getByRole('button', {
    name: 'Show Alert'
  }).click();

});
```

Explanation:

1. `page.on('dialog')` listens for browser dialogs.
2. `dialog.message()` retrieves the popup message.
3. `dialog.accept()` clicks the OK button.
4. The test clicks the button that triggers the alert.

Important methods:

- `dialog.message()` – gets the dialog message.
- `dialog.accept()` – accepts the dialog.
- `dialog.dismiss()` – dismisses the dialog.

B. Confirm dialog – `confirm()`

A confirm dialog provides two options:

- OK
- Cancel

Example JavaScript:

```
confirm('Are you sure you want to delete?');
```

Playwright test – accept:

```
test('accept confirm dialog', async ({ page }) => {

  await page.goto('https://example.com');

  page.on('dialog', async dialog => {
    console.log(dialog.message());

    await dialog.accept();
  });

  await page.getByRole('button', {
    name: 'Delete'
  }).click();

});
```

Playwright test – dismiss:

```
test('dismiss confirm dialog', async ({ page }) => {

  await page.goto('https://example.com');

  page.on('dialog', async dialog => {
    await dialog.dismiss();
  });

  await page.getByRole('button', {
    name: 'Delete'
  }).click();

});
```

Difference:

Method	Action
`dialog.accept()`	Clicks OK
`dialog.dismiss()`	Clicks Cancel

C. Prompt dialog – `prompt()`

A prompt dialog asks the user to enter text.

Example JavaScript:

```
prompt('Enter your name:');
```

Playwright test:

```
test('handle prompt dialog', async ({ page }) => {

  await page.goto('https://example.com');

  page.on('dialog', async dialog => {

    console.log(dialog.message());

    await dialog.accept('Komal');

  });

  await page.getByRole('button', {
    name: 'Enter Name'
  }).click();

});
```

Here:

```
await dialog.accept('Komal');
```

The string `Komal` is entered into the prompt and submitted.

To cancel the prompt:

```
await dialog.dismiss();
```

Important: `dialog.accept('text')` is used to submit text in a prompt. For an alert or confirm dialog, simply call `dialog.accept()`.

3. Important concept – How Playwright handles dialogs by default

This is one of the most important points in the documentation and interviews.

By default, Playwright automatically dismisses browser dialogs if you have not registered a dialog event listener.

For example:

```
test('automatic dialog dismissal', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByRole('button', {
    name: 'Show Alert'
  }).click();

});
```

If the button triggers an alert, Playwright automatically dismisses it because there is no dialog listener.

You don't need to handle it manually in this situation.

What happens if you register a dialog listener?

Once you register:

```
page.on('dialog', dialog => {
  console.log(dialog.message());
});
```

Playwright expects your listener to handle the dialog.

But this code only prints the message. It does not accept or dismiss the dialog.

As a result, the dialog remains open and blocks the browser. The click action will hang.

Incorrect approach

```
page.on('dialog', dialog => {
  console.log(dialog.message());
});

await page.getByRole('button').click();
```

Correct approach

```
page.on('dialog', async dialog => {
  console.log(dialog.message());

  await dialog.accept();
});

await page.getByRole('button').click();
```

Remember this rule:

If you register a `page.on('dialog')` listener, you must accept or dismiss the dialog.

Why does the listener need to be registered before clicking?

Because the dialog can appear immediately after clicking the button.

If you register the listener after the click, you may miss the dialog event.

Correct order:

```
// 1. Register listener
page.on('dialog', async dialog => {
  await dialog.accept();
});

// 2. Trigger dialog
await page.getByRole('button').click();
```

Incorrect order:

```
// 1. Trigger dialog
await page.getByRole('button').click();

// 2. Register listener too late
page.on('dialog', async dialog => {
  await dialog.accept();
});
```

4. Beforeunload dialog

What is a beforeunload dialog?

A `beforeunload` dialog appears when the browser is about to leave or close a page that has registered a `beforeunload` event handler.

For example:

Imagine you are filling out a registration form.

You enter:

- Name
- Email
- Password

But you haven't saved the form.

When you try to close the page, the browser may display a confirmation message asking whether you want to leave.

This is called a `beforeunload` dialog.

How to handle it in Playwright

```
test('handle beforeunload dialog', async ({ page }) => {

  page.on('dialog', async dialog => {

    console.log(dialog.type());

    await dialog.dismiss();

  });

  await page.goto('https://example.com');

  await page.close({
    runBeforeUnload: true
  });
```

```
});
```

Let's understand the important code.

1. `page.on('dialog')`

Listens for the dialog event.

2. `dialog.type()`

Returns the dialog type, such as:

- `alert`
- `confirm`
- `prompt`
- `beforeunload`

3. `dialog.dismiss()`

Dismisses the beforeunload confirmation. For this example, it means choosing to stay on the page.

4. `page.close({ runBeforeUnload: true })`

Runs the page's unload handlers before closing.

What is `runBeforeUnload: true`?

Normally:

```
await page.close();
```

Playwright closes the page without running the page's `beforeunload` handlers.

But:

```
await page.close({
  runBeforeUnload: true
});
```

It runs those handlers, which may trigger a beforeunload confirmation dialog.

Important interview point: When `runBeforeUnload: true` is used, `page.close()` does not wait for the page to actually close. The page may remain open if the user dismisses the confirmation.

Accept vs dismiss in beforeunload

Method	Result
`dialog.accept()`	Confirms leaving the page
`dialog.dismiss()`	Cancels leaving and stays on the page

5. Print dialogs

What is a print dialog?

When a website uses:

```
window.print();
```

The browser opens its print interface, allowing users to print or save the page as a PDF.

Example: You are viewing an invoice and click the Print Invoice button.

Playwright documentation provides a way to verify that `window.print()` was triggered.

Example

```
test('verify print dialog was triggered', async ({ page }) => {

  await page.goto('https://example.com');

  await page.evaluate(() => {
    (window as any).waitForPrintDialog =
      new Promise(resolve => {
        window.print = resolve as any;
      });
  });

  await page.getByText('Print it!').click();
```

```
  await page.waitForFunction(
    () => (window as any).waitForPrintDialog
  );

});
```

Step-by-step explanation

Step 1: Open the application

```
await page.goto('https://example.com');
```

Step 2: Override `window.print()`

```
await page.evaluate(() => {
  (window as any).waitForPrintDialog =
    new Promise(resolve => {
      window.print = resolve as any;
    });
});
```

Here:

- `page.evaluate()` executes JavaScript inside the browser page.
- A Promise is created.
- The original `window.print()` function is replaced with a function that resolves the Promise.
- This allows the test to detect when the application calls `window.print()`.

Step 3: Click the Print button

```
await page.getByText('Print it!').click();
```

This triggers the application's print functionality.

Step 4: Wait for the print function to be called

```
await page.waitForFunction(
  () => (window as any).waitForPrintDialog
);
```

This waits until the Promise is resolved, indicating that the overridden `window.print()` function was called.

Important: This technique verifies that the application triggered `window.print()`. It does not open or automate the actual operating-system/browser print interface.

Also, the override must be installed after the page loads but before clicking the Print button.

6. Important methods to remember

Method	Purpose
`page.on('dialog')`	Listens for browser dialog events
`dialog.accept()`	Accepts a dialog
`dialog.dismiss()`	Dismisses a dialog
`dialog.message()`	Gets the dialog message
`dialog.type()`	Gets the dialog type
`dialog.defaultValue()`	Gets the default value of a prompt
`page.close()`	Closes the page
`page.close({ runBeforeUnload: true })`	Runs beforeunload handlers
`page.evaluate()`	Executes JavaScript in the browser
`page.waitForFunction()`	Waits for a browser-side function condition

7. Important interview questions and answers

Basic questions

Q1. What are dialogs in Playwright?

Answer: Dialogs are browser popup messages generated by JavaScript or browser events. Playwright supports alert, confirm, prompt, and beforeunload dialogs.

Q2. What are the different types of dialogs?

Answer:

1. Alert
2. Confirm
3. Prompt
4. Beforeunload

Q3. How do you handle an alert dialog in Playwright?

Answer: We use the `page.on('dialog')` event listener and `dialog.accept()` to accept the alert.

```
page.on('dialog', async dialog => {
  await dialog.accept();
});

await page.getByRole('button').click();
```

Q4. What is the difference between `accept()` and `dismiss()`?

Answer:

- `accept()` confirms the dialog, similar to clicking OK.
- `dismiss()` cancels the dialog, similar to clicking Cancel.

Q5. How do you handle a prompt dialog?

Answer: We use `dialog.accept('text')` to enter text and submit the prompt.

```
page.on('dialog', async dialog => {
  await dialog.accept('Komal');
});
```

Q6. What happens if we don't handle a dialog in Playwright?

Answer: If no dialog listener is registered, Playwright automatically dismisses the dialog. We don't need to handle it manually.

Intermediate questions

Q7. What happens if we register a dialog listener but don't accept or dismiss it?

Answer: The dialog remains open and blocks further page execution. The action that triggered the dialog, such as `locator.click()`, will remain pending.

Q8. Why should we register the dialog listener before clicking the button?

Answer: Because the dialog may appear immediately after the click. Registering the listener beforehand ensures that Playwright can capture and handle the dialog event.

Q9. How do you get the message displayed in an alert?

Answer:

```
page.on('dialog', async dialog => {
  console.log(dialog.message());
  await dialog.accept();
});
```

Q10. How do you identify the type of dialog?

Answer:

```
page.on('dialog', async dialog => {
  console.log(dialog.type());
  await dialog.accept();
});
```

The type can be `alert`, `confirm`, `prompt`, or `beforeunload`.

Q11. What is a beforeunload dialog?

Answer: A beforeunload dialog is a browser confirmation displayed when a user tries to leave a page that has an unload handler, commonly used to warn about unsaved changes.

Q12. What is `runBeforeUnload: true`?

Answer: It is an option passed to `page.close()` that causes the page's beforeunload handlers to run before closing. The page may remain open if the confirmation is dismissed.

Q13. Can Playwright handle print dialogs?

Answer: Playwright documentation provides a technique to verify that `window.print()` was triggered by overriding the function with a Promise and waiting for that Promise to resolve. This does not automate the actual print interface.

Scenario-based questions

Q14. A website displays an alert after successful login. How will you handle it?

Answer: I will register a dialog listener before clicking the Login button, verify the alert message using `dialog.message()`, and accept it using `dialog.accept()`.

Q15. A delete button displays a confirmation popup. How will you test the Cancel functionality?

Answer: I will register a dialog listener and call `dialog.dismiss()`. Then I will verify that the record has not been deleted.

Q16. A prompt asks for the user's name. How will you enter the name?

Answer: I will use `dialog.accept('Komal')` inside the dialog event listener to enter and submit the name.

Q17. Your test hangs after clicking a button. You suspect an alert. What will you check?

Answer: I will check whether the button triggers a browser dialog. I will verify that the dialog listener is registered before the click and that the listener calls either `accept()` or `dismiss()`.

Q18. How will you test unsaved changes when a user closes a page?

Answer: I will trigger the page's beforeunload behavior, register a dialog listener, close the page using `runBeforeUnload: true`, and verify the expected confirmation behavior.

8. Quick revision notes

Remember these important points

- Alert → `dialog.accept()`
- Confirm OK → `dialog.accept()`
- Confirm Cancel → `dialog.dismiss()`
- Prompt with text → `dialog.accept('text')`
- Beforeunload → Handle using `page.on('dialog')`
- No listener → Dialogs are automatically dismissed.
- Listener registered → Must accept or dismiss.
- Register listener before the triggering action.
- Print dialog → Verify `window.print()` was triggered.

9. Test your understanding

Dialogs – Interview Quiz

6/6 answered

1. Which event is used to handle browser dialogs?

page.on('popup')
page.on('dialog')
page.on('alert')
page.on('window')

Correct

The dialog event is captured using page.on('dialog').

2. Which method clicks OK on an alert?

dialog.dismiss()
dialog.close()
dialog.accept()

dialog.message()

Correct

dialog.accept() accepts the dialog.

3. How do you enter text in a prompt dialog?

dialog.fill('Komal')
dialog.accept('Komal')
dialog.type('Komal')
dialog.input('Komal')

Correct

dialog.accept('Komal') enters and submits the prompt value.

4. What happens if there is no dialog listener?

Test always fails
Browser closes
Playwright automatically dismisses the dialog
Test waits forever

Correct answer: Playwright automatically dismisses the dialog

Playwright automatically dismisses dialogs when no listener is registered.

5. What happens if a dialog listener only logs the message?

Dialog is accepted
Dialog is dismissed
The triggering action can hang
The page reloads

Correct

A registered listener must handle the dialog by accepting or dismissing it.

6. Which option runs beforeunload handlers when closing a page?

force: true
runBeforeUnload: true
waitUntil: 'load'
headless: false

Correct

page.close({ runBeforeUnload: true }) runs the page's unload handlers.

Check answers

5 / 6 correct

Try again

Most important for interviews: Understand `page.on('dialog')`, `dialog.accept()`, `dialog.dismiss()`, automatic dismissal, and why an unhandled dialog listener causes a test to hang.

Send the next Playwright topic whenever you're ready!

Playwright – Downloads

Let's understand the Downloads section in simple language, with practical examples and interview questions for QA Automation / SDET interviews.

1. What is file downloading in Playwright?

When we click a download button on a website, the browser downloads a file such as:

- PDF file
- Excel file
- CSV file
- Image
- ZIP file
- Text file

Playwright provides a `download` event to handle these downloaded files.

Important: Whenever a page downloads an attachment, Playwright emits the `download` event.

For example:

Imagine an application has an Export Report button. When you click it, an Excel file gets downloaded. Playwright can detect that download, get its filename, and save it to a location.

2. Important methods and concepts

Method / Concept	Explanation
`page.on('download')`	Listens for download events
`page.waitForEvent('download')`	Waits until a download starts
`download.suggestedFilename()`	Gets the suggested filename
`download.url()`	Gets the URL from which the file was downloaded
`download.path()`	Gets the temporary file path after download completes
`download.saveAs()`	Saves the file to a location you specify
`downloadsPath`	Specifies the browser's download directory

Let's understand the most important methods with examples.

3. Handling a file download

Consider a website with a button called Download file.

UI Buttons & When to Use Them Interactive Lesson

Example:

```
import { test, expect } from '@playwright/test';

test('Download a file', async ({ page }) => {

  await page.goto('https://example.com');

  // Start waiting for the download
  const downloadPromise = page.waitForEvent('download');

  // Click the download button
  await page.getByText('Download file').click();

  // Get the download object
  const download = await downloadPromise;

  // Save the downloaded file
  await download.saveAs(
    './downloads/' + download.suggestedFilename()
  );
});
```

Step-by-step explanation

Step 1:

```
const downloadPromise = page.waitForEvent('download');
```

Playwright starts waiting for the download event.

It does not wait for the download immediately. It creates a Promise that will resolve when the download starts.

Step 2:

```
await page.getByText('Download file').click();
```

Clicks the Download file button.

This action triggers the file download.

Step 3:

```
const download = await downloadPromise;
```

Waits for the download event and receives the Download object.

Step 4:

```
await download.saveAs(
  './downloads/' + download.suggestedFilename()
);
```

Saves the downloaded file to the specified location using its suggested filename.

For example, if the suggested filename is `report.xlsx`, the saved path becomes:

```
./downloads/report.xlsx
```

Interview important
Why do we wait for the download event before clicking the button?

Because the download may start immediately after clicking. If we start waiting after clicking, we might miss the event.

4. Why don't we use await in the first line?

```
const downloadPromise = page.waitForEvent('download');
```

Notice that we haven't used `await`.

This is intentional.

We want to start listening for the download event first, then click the button.

Correct sequence:

```
const downloadPromise = page.waitForEvent('download');

await page.getByText('Download file').click();

const download = await downloadPromise;
```

Incorrect sequence:

```
await page.getByText('Download file').click();

const download = await page.waitForEvent('download');
```

Why is the second approach incorrect?

The download might already have started before Playwright begins waiting for it. This can cause the test to time out.

Remember this interview point:

Always register the download wait before performing the action that triggers the download.

5. Understanding Download object methods

After receiving the Download object, we can access its information.

A. suggestedFilename()

```
const filename = download.suggestedFilename();

console.log(filename);
```

Returns the filename suggested by the website.

Example output:

```
Employee_Report.xlsx
```

B. url()

```
const fileUrl = download.url();

console.log(fileUrl);
```

Returns the URL from which the file was downloaded.

Example:

https://example.com/reports/Employee_Report.xlsx

C. path()

```
const filePath = await download.path();

console.log(filePath);
```

Returns the temporary location

Playwright – Evaluating JavaScript

This is an important Playwright topic for QA Automation and SDET interviews.

In simple words, Evaluating JavaScript means executing JavaScript code directly inside the browser page from your Playwright test.

For example, suppose you want to:

- Get the current page URL.
- Read text from an element.
- Access browser objects like `window` and `document`.
- Execute JavaScript inside the browser.
- Pass test data from your Playwright script to the browser.
- Set up mock data before a page loads.

Playwright provides methods such as `page.evaluate()`, `page.evaluateHandle()` and `page.addInitScript()` for these tasks.

1. Understanding two different environments

This is the most important concept in this section.

Playwright works in two separate JavaScript environments.

1. Playwright / Test Environment

Your test code runs here – Node.js

```
const data = 'Hello';
await page.goto('https://example.com');
```

page.evaluate()

Executes code in the browser and returns the result

2. Browser / Page Environment

Your website's JavaScript runs here

```
window
document
location
```

These environments are separate. They run in different JavaScript contexts.

Example:

- Your Playwright test runs in Node.js.
- Your application runs inside Chrome, Firefox or WebKit.
- A variable created in your Playwright test is not automatically available inside the browser page.

Interview question

Q1. What are the two JavaScript environments in Playwright?

Answer:

1. Playwright environment: Where the test scripts execute, usually in Node.js.
2. Browser environment: Where the website's JavaScript executes.

They are separate environments. To exchange data, we must explicitly pass arguments or return values using evaluation methods.

2. What is page.evaluate()?

`page.evaluate()` executes JavaScript code inside the browser page and returns the result to the Playwright test.

Syntax:

```
const result = await page.evaluate(() => {
  // JavaScript code runs inside the browser
  return document.title;
});
```

Example:

```
import { test, expect } from '@playwright/test';

test('Get page URL', async ({ page }) => {
```

```
  await page.goto('https://example.com');

  const href = await page.evaluate(() => {
    return document.location.href;
  });

  console.log(href);

});
```

Simple explanation:

1. Open the website.
2. `page.evaluate()` executes JavaScript inside the browser.
3. `document.location.href` gets the current page URL.
4. The returned URL comes back to the Playwright environment.
5. We print the URL using `console.log()`.

Example output:

https://example.com/

More practical examples

Example 1: Get page title

```
const title = await page.evaluate(() => {
  return document.title;
});

console.log(title);
```

Example 2: Get page URL

```
const url = await page.evaluate(() => {
  return window.location.href;
});
```

Example 3: Get text from an element

```
const text = await page.evaluate(() => {
  return document.querySelector('h1')?.textContent;
});

console.log(text);
```

Example 4: Get the total number of links

```
const count = await page.evaluate(() => {
  return document.querySelectorAll('a').length;
});

console.log(count);
```

Example 5: Get the background color of the page

```
const color = await page.evaluate(() => {
  return getComputedStyle(document.body).backgroundColor;
});

console.log(color);
```

Important interview point
Inside `page.evaluate()`, you can access browser objects such as `window`, `document` and `location`.

3. Does page.evaluate() support async/await?

Yes!

If the function passed to `page.evaluate()` returns a Promise, Playwright automatically waits for that Promise to resolve.

Example from the documentation:

```
const status = await page.evaluate(async () => {

  const response = await fetch(location.href);

  return response.status;

});
```

Explanation:

1. `async` makes the browser-side function asynchronous.
2. `fetch(location.href)` sends a request to the current page URL.
3. `await` waits for the response.
4. `response.status` returns the HTTP status code.
5. Playwright waits for the returned Promise and receives the result.

For a successful response, the status is usually `200`.

Interview Q2. Does page.evaluate() wait for an asynchronous function to complete?

Answer: Yes. Playwright automatically waits for the Promise returned by the evaluated function to resolve and then returns its result to the test.

4. Passing variables between Playwright and browser

This is one of the most important concepts for interviews.

Suppose you create a variable in your Playwright test:

```
const data = 'some data';
```

Now you want to use this variable inside `page.evaluate()`.

Incorrect approach

```
const data = 'some data';

const result = await page.evaluate(() => {
  window.myApp.use(data);
});
```

Why is this wrong?

Because `data` belongs to the Playwright environment. The browser environment cannot directly access it.

You may get an error like:

```
ReferenceError: data is not defined
```

Correct approach

Pass the variable as an argument to `page.evaluate()`.

```
const data = 'some data';

const result = await page.evaluate((data) => {
  window.myApp.use(data);
}, data);
```

Understand the syntax:

```
page.evaluate((parameter) => {
  // Browser-side code
}, argument);
```

- First `data`: Parameter received inside the browser.
- Second `data`: Value passed from the Playwright test.

The value is transferred explicitly from one environment to another.

Another practical example

Suppose you want to check whether a username is displayed on the webpage.

```
const username = 'Komal';

const result = await page.evaluate((name) => {

  return document.body.innerText.includes(name);

}, username);

console.log(result);
```

Output:

```
true
```

Here, the Playwright variable `username` is passed to the browser as `name`.

Interview Q3. Can we directly access Playwright variables inside page.evaluate()?

Answer: No. Playwright and browser environments are separate. We must pass the required variables explicitly as arguments to `page.evaluate()`.

5. Evaluation Arguments

Playwright evaluation methods accept one optional argument.

You can pass different types of data:

- Primitive values
- Arrays
- Objects
- JSHandle instances
- A combination of serializable values and handles

Let's understand each one.

A. Passing a primitive value

Primitive values include numbers, strings and booleans.

```
const result = await page.evaluate((num) => {
  return num * 2;
}, 42);

console.log(result);
```

Output:

84

Here, 42 is passed into the browser function as `num`.

B. Passing an array

```
const result = await page.evaluate((array) => {
  return array.length;
}, [1, 2, 3]);

console.log(result);
```

Output:

```
3
```

We pass an array containing three elements and get its length.

C. Passing an object

```
const result = await page.evaluate((object) => {
  return object.foo;
}, { foo: 'bar' });

console.log(result);
```

Output:

```
bar
```

Here, we pass an object and access its `foo` property inside the browser.

D. Passing JSHandle

First, understand what a JSHandle is.

A JSHandle is a reference to a JavaScript object inside the browser environment.

For example, you can create a handle for a browser-side object:

```
const button = await page.evaluateHandle(() => {
  return document.querySelector('button');
});
```

Now `button` represents a browser-side button element.

You can use it in another evaluation:

```
const text = await page.evaluate((button) => {
  return button.textContent;
}, button);

console.log(text);
```

The result is the button's text.

Important difference:

- `page.evaluate()` returns the result as a serializable value.
- `page.evaluateHandle()` returns a JSHandle that refers to an object in the browser.

For example:

```
const title = await page.evaluate(() => document.title);
```

This returns a string.

```
const button = await page.evaluateHandle(() => {
  return document.querySelector('button');
});
```

This returns a handle to the browser-side element.

E. Passing multiple handles using an object

You can pass multiple JSHandles together inside an object.

```
const button1 = await page.evaluateHandle(
  () => document.querySelector('#button1')
);

const button2 = await page.evaluateHandle(
  () => document.querySelector('#button2')
);

const result = await page.evaluate((o) => {

  return o.button1.textContent +
         o.button2.textContent;

}, { button1, button2 });

console.log(result);
```

Here, both handles are passed together as object properties.

F. Object destructuring

The same example can be written using destructuring.

```
const result = await page.evaluate(
  ({ button1, button2 }) => {
    return button1.textContent + button2.textContent;
  },
  { button1, button2 }
);
```

Why are parentheses required around `{ button1, button2 }`?

Because JavaScript uses parentheses to make it clear that the curly braces represent an object parameter being destructured, rather than a function body.

G. Array destructuring

You can also pass handles in an array.

```
const result = await page.evaluate(
  ([b1, b2]) => {
    return b1.textContent + b2.textContent;
  },
  [button1, button2]
);
```

Here:

- `b1` receives the first handle.
- `b2` receives the second handle.

H. Mixing serializable values and handles

You can combine normal values and JSHandles.

```
const result = await page.evaluate(
  (x) => {
    return x.button1.textContent +
           x.list[0].textContent +
           String(x.foo);
  },
  {
    button1,
    list: [button2],
    foo: null
```

```
  }
);
```

This example combines:

- A JSHandle for `button1`.
- An array containing another JSHandle.
- A normal value (`null`).

Interview Q4. What types of arguments can we pass to page.evaluate()?

Answer: We can pass serializable values such as strings, numbers, booleans, arrays and objects. We can also pass JSHandle instances or combinations of serializable values and handles.

Interview Q5. What is the difference between evaluate() and evaluateHandle()?

evaluate()	**evaluateHandle()**
Executes JavaScript in the browser	Executes JavaScript in the browser
Returns a serializable result	Returns a JSHandle
Useful for getting values	Useful for keeping a reference to browser objects
Example: page title	Example: button element

6. What are Init Scripts?

Sometimes we need to execute JavaScript before the website's own JavaScript starts running.

For this purpose, Playwright provides:

1. `page.addInitScript()`
2. `browserContext.addInitScript()`

These methods execute JavaScript during page initialization, before the page's scripts run.

Real-world example

Suppose your application uses:

`Math.random()`

This generates a random number.

For testing, random values can make tests unpredictable.

For example:

- One test may generate `0.23`.
- Another test may generate `0.89`.
- Your application may behave differently depending on the value.

We want to replace the random value with a fixed value, such as 42.

This is called mocking.

Method 1: Using a separate JavaScript file

First, create a file called `preload.js`.

```
// preload.js

Math.random = () => 42;
```

This replaces the original `Math.random()` function with one that always returns 42.

Now add the init script to Playwright.

```
import { test } from '@playwright/test';
import path from 'path';

test.beforeEach(async ({ page }) => {

  await page.addInitScript({
    path: path.resolve(
      __dirname,
```

```
      '../mocks/preload.js'
    )
  });

});
```

Explanation:

- `test.beforeEach()` runs before every test.
- `page.addInitScript()` registers a script to run before the page's own scripts.
- `path.resolve()` creates an absolute path to the preload file.
- The preload file changes `Math.random()`.

Method 2: Passing a function directly

For small scripts, we don't need a separate JavaScript file.

We can directly pass a function to `page.addInitScript()`.

```
import { test } from '@playwright/test';

test.beforeEach(async ({ page }) => {

  const value = 42;

  await page.addInitScript((value) => {

    Math.random = () => value;

  }, value);

});
```

Explanation:

1. Create a variable `value = 42` in the Playwright environment.
2. Pass it as an argument to `addInitScript()`.
3. The browser receives the value.
4. `Math.random()` is replaced with a function that returns 42.

Where is this useful in real projects?

Scenario	Use
Random number testing	Replace `Math.random()` with a fixed value
Mock application settings	Set test configuration before app scripts run
Feature flag testing	Set browser-side flags before application initialization
Test data setup	Prepare browser-side data before application code executes

page.addInitScript() vs browserContext.addInitScript()

`page.addInitScript()`	`browserContext.addInitScript()`
Registers the script for a particular page	Registers the script for pages in that browser context
Useful when testing one page	Useful when testing multiple pages or popups in the context

Interview Q6. What is page.addInitScript()?

Answer: `page.addInitScript()` allows us to execute JavaScript in the browser before the page's own scripts run. It is useful for setting up mocks, fixed values and test data before application initialization.

Interview Q7. What is the difference between page.evaluate() and page.addInitScript()?

`page.evaluate()`	`page.addInitScript()`
Executes JavaScript when we call it	Registers JavaScript to run during page initialization

Used to interact with an already-loaded page	Used for setup before application scripts run
Example: Get page title	Example: Mock `Math.random()`

Important points about init scripts:

- They run before the website's own JavaScript.
- They can run again when the page navigates.
- Context-level init scripts are useful when multiple pages need the same setup.
- They are useful for controlling browser-side behavior during testing.

7. Important differences – Quick revision

Feature	Purpose
`page.evaluate()`	Execute JavaScript in the browser and return a value
`page.evaluateHandle()`	Execute JavaScript and return a JSHandle
`page.addInitScript()`	Run setup JavaScript before page scripts
`browserContext.addInitScript()`	Register setup JavaScript for pages in a browser context
Evaluation argument	Pass data from Playwright to browser
JSHandle	Reference to an object in the browser

8. Interview questions and answers

Here are additional questions you should prepare.

Q8. Can we access `window` and `document` inside page.evaluate()?

Answer: Yes. `page.evaluate()` executes JavaScript in the browser context, so we can access browser globals such as `window`, `document` and `location`.

Q9. What happens if page.evaluate() returns a Promise?

Answer: Playwright automatically waits for the Promise to resolve and then returns the resolved value to the Playwright environment.

Q10. Why do we use arguments in page.evaluate()?

Answer: The Playwright environment and browser environment are separate. Arguments allow us to pass data explicitly from the test script to the browser.

Q11. What is a JSHandle?

Answer: A JSHandle is a reference to a JavaScript object in the browser. It allows Playwright to work with browser-side objects without immediately converting them into serializable values.

Q12. When would you use evaluateHandle() instead of evaluate()?

Answer: I would use `evaluateHandle()` when I need to retain a reference to a browser-side object, such as a DOM element or JavaScript object. I would use `evaluate()` when I only need the returned value.

Q13. What is the purpose of mocking Math.random()?

Answer: Mocking `Math.random()` helps us generate predictable values during testing. It avoids unpredictable test results caused by random numbers.

Q14. Can we pass an argument to addInitScript()?

Answer: Yes. We can pass an argument as the second parameter of `page.addInitScript()`. The function receives that value inside the browser context.

Q15. What is the difference between page.addInitScript() and browserContext.addInitScript()?

Answer: `page.addInitScript()` registers an initialization script for a particular page, whereas `browserContext.addInitScript()` registers it for pages in the browser context. The context-level method is useful when the same setup is required across multiple pages.

Q16. Can page.evaluate() directly access a variable declared in the Playwright test?

Answer: No. We must explicitly pass the variable as an argument to the evaluation function.

Q17. Can we use async/await inside page.evaluate()?

Answer: Yes. We can pass an async function to `page.evaluate()`. Playwright waits for the returned Promise to resolve.

Q18. What is the main use of JavaScript evaluation in automation testing?

Answer: It allows us to execute browser-side JavaScript, retrieve page information, access DOM properties, pass test data and configure the browser environment for testing.

9. Practical interview coding examples

Try to understand these examples because interviewers may ask you to write them.

Task 1: Get the page title

```
const title = await page.evaluate(() => document.title);
console.log(title);
```

Task 2: Count all buttons

```
const count = await page.evaluate(() => {
  return document.querySelectorAll('button').length;
});

console.log(count);
```

Task 3: Pass a value from Playwright to browser

```
const price = 500;

const result = await page.evaluate((price) => {
  return price * 2;
}, price);

console.log(result);
```

Output: `1000`

Task 4: Get the text of a heading

```
const heading = await page.evaluate(() => {
  return document.querySelector('h1')?.textContent;
});

console.log(heading);
```

Task 5: Mock Math.random() before application scripts run

```
await page.addInitScript(() => {
  Math.random = () => 0.5;
});

await page.goto('https://example.com');
```

10. Test your knowledge – Interview quiz

Try answering these questions without looking at the explanations above.

Playwright – Evaluating JavaScript

6/6 answered

1. Where does page.evaluate() execute JavaScript?

Playwright environment
Browser page environment
Operating system
Test runner only

Correct

page.evaluate() executes JavaScript inside the browser page.

2. How do you pass a Playwright variable into page.evaluate()?

Use a global variable automatically
Pass it as an argument
Use console.log()
Use page.goto()

Incorrect – Pass it as an argument

Pass the value as the second argument to page.evaluate().

3. What does page.evaluateHandle() return?

String
Boolean
JSHandle
URL

Correct

It returns a JSHandle referencing an object in the browser.

4. What happens when page.evaluate() returns a Promise?

Playwright ignores it
Playwright automatically waits for it
The test always fails
It returns the Promise as text

Correct

Playwright waits for the returned Promise to resolve.

5. Which method runs JavaScript before page scripts?

page.evaluate()
page.locator()
page.addInitScript()
page.title()

Correct

addInitScript registers JavaScript to run during page initialization, before page scripts.

6. Why would you mock Math.random()?

To close the browser
To make random values predictable
To take screenshots
To navigate to another page

Correct

Mocking Math.random() makes test behavior deterministic.

5 / 6 correct

Try again

11. Final revision – Remember these five things

1. `page.evaluate()` executes JavaScript inside the browser.
2. Playwright and browser environments are separate.
3. Pass variables explicitly as arguments.
4. `evaluateHandle()` returns a JSHandle, while `evaluate()` returns a value.
5. `addInitScript()` is used to set up mocks or data before application scripts execute.

Next section: Send me the next Playwright documentation topic, and I'll explain it in the same format with practical examples, interview questions, and a quiz.

Playwright – Events

This is an important topic for QA Automation and SDET interviews because real web applications generate many events while users interact with them.

Let's understand Events in Playwright with simple explanations, practical examples, interview questions and a quiz.

1. What are Events in Playwright?

An event is an action or occurrence that happens in a webpage or browser.

For example:

- A network request is sent.
- A network request finishes.
- A popup window opens.
- A browser dialog appears.

- A new page or tab is created.
- A download starts.
- A dedicated web worker is created.

Playwright allows us to listen to these events and perform actions when they happen.

Real-world example:

Suppose you are testing an e-commerce application.

When you click the Login button:

1. The browser sends a login API request.
2. The server responds.
3. The page displays the user's dashboard.

You can use Playwright events to detect and verify these actions.

2. Three ways to handle events

Playwright provides three common approaches:

Method	Purpose
`waitForEvent()` / `waitForRequest()`	Wait for a specific event or request
`page.on()`	Listen continuously to an event
`page.off()`	Remove an event listener
`page.once()`	Listen for an event only once

Let's understand each one.

3. Waiting for an event

Sometimes we know exactly which event will happen after performing an action.

In this situation, we use event-waiting methods.

Example 1: Wait for a network request

Suppose you open Wikipedia and want to wait until the logo image is requested.

Playwright provides:

```
page.waitForRequest()
```

Example:

```
import { test } from '@playwright/test';

test('Wait for network request', async ({ page }) => {

  // Start waiting for the request
  const requestPromise =
    page.waitForRequest('**/*logo*.png');

  // Navigate to the website
  await page.goto('https://wikipedia.org');

  // Get the request
  const request = await requestPromise;

  console.log(request.url());

});
```

Step-by-step explanation

Step 1:

```
const requestPromise =
  page.waitForRequest('**/*logo*.png');
```

Playwright starts waiting for a network request whose URL matches the specified pattern.

Here:

`**/*logo*.png`

means a URL containing `logo` and ending in `.png`, subject to the wildcard pattern.

Step 2:

```
await page.goto('https://wikipedia.org');
```

Opens Wikipedia. During page loading, the browser may request its logo image.

Step 3:

```
const request = await requestPromise;
```

Waits until the matching request occurs and returns the Request object.

Step 4:

```
console.log(request.url());
```

Prints the URL of the requested resource.

Important: Why no await in the first line?

```
const requestPromise = page.waitForRequest('**/*logo*.png');
```

We intentionally don't use `await` here.

We first register the wait, then perform the action that triggers the request.

Correct sequence:

```
const requestPromise = page.waitForRequest('**/*logo*.png');

await page.goto('https://wikipedia.org');

const request = await requestPromise;
```

If we start waiting only after navigation, the request might already have happened and we could miss it.

Interview important
Always start waiting for an event before performing the action that triggers it.

Example 2: Wait for a popup window

Suppose a website contains a button called Open the popup.

When you click it, a new browser page opens.

Playwright provides:

`page.waitForEvent('popup')`

Example:

```
test('Handle popup window', async ({ page }) => {

  await page.goto('https://example.com');

  // Start waiting for popup
  const popupPromise = page.waitForEvent('popup');

  // Click the button
  await page.getByText('open the popup').click();

  // Get the popup page
  const popup = await popupPromise;

  // Navigate popup to another URL
  await popup.goto('https://wikipedia.org');

});
```

Step-by-step explanation

1. Open the main website.
2. `page.waitForEvent('popup')` starts waiting for a popup created by this page.
3. Click the button that opens the popup.
4. `await popupPromise` gets the newly opened page.
5. Use the popup object just like a normal Playwright `Page` object.

Important: A popup is a new page opened by the current page, commonly through a link or `window.open()`.

Picture-in-Picture any site para Google Chrome - Extension Descargar

Interview Q1. How do you handle a popup in Playwright?

Answer:

I use `page.waitForEvent('popup')`. First, I create the popup promise, then click the element that opens the popup, and finally await the promise to get the new Page object.

```
const popupPromise = page.waitForEvent('popup');

await page.getByText('Open popup').click();

const popup = await popupPromise;
```

Interview Q2. Why should we wait for the popup before clicking?

Answer:

Because the popup may open immediately after clicking. If we start waiting afterward, we may miss the popup event, causing the test to time out.

4. Adding event listeners using page.on()

Sometimes we don't know exactly when an event will happen.

For example, we want to monitor all network requests sent by the browser while a test is running.

In this situation, we use:

`page.on()`

It registers an event listener that keeps listening until we remove it or the page is closed.

Example 1: Listen to network requests

```
test('Listen to requests', async ({ page }) => {

  page.on('request', request => {
    console.log(`Request sent: ${request.url()}`);
  });
```

```
  await page.goto('https://wikipedia.org');

});
```

Explanation:

`page.on('request', request => {`

- `page.on()` registers an event listener.
- `'request'` is the event name.
- `request` is the Request object passed to the callback.
- `request.url()` returns the requested URL.

Whenever the page sends a network request, the callback executes.

For example, you may see requests for:

https://www.wikipedia.org/
https://www.wikipedia.org/portal/wikipedia.org/assets/img/Wikipedia-logo-v2.png
https://www.wikipedia.org/favicon.ico

The exact URLs depend on the website.

Important network events

Event	Meaning
`request`	A request is issued
`response`	Response headers are received
`requestfinished`	Request has completed successfully
`requestfailed`	Request has failed

Understand the difference:

Suppose your application sends a login API request.

1. `request` – Browser sends the login request.
2. `response` – Browser receives the response headers.
3. `requestfinished` – The request and response body have finished downloading.
4. `requestfailed` – The request fails at the network level instead.

Note: An HTTP error such as 404 or 500 is still a completed HTTP request. It normally triggers `requestfinished`, not `requestfailed`.

Example 2: Listen to requestfinished

```
test('Listen to completed requests', async ({ page }) => {

  const listener = request => {
    console.log(`Request finished: ${request.url()}`);
  };

  page.on('requestfinished', listener);

  await page.goto('https://wikipedia.org');

});
```

Here, the listener executes when a network request finishes.

5. Removing event listeners using page.off()

Sometimes we only want to monitor events for a particular part of the test.

After that, we want to stop listening.

Playwright provides:

`page.off()`

Example

```
test('Add and remove event listener', async ({ page }) => {

  const listener = request => {
```

```
    console.log(`Request finished: ${request.url()}`);
  };

  // Add listener
  page.on('requestfinished', listener);

  // First navigation
  await page.goto('https://wikipedia.org');

  // Remove listener
  page.off('requestfinished', listener);

  // Second navigation
  await page.goto('https://www.openstreetmap.org/');

});
```

Step-by-step explanation

Step 1: Create a listener

```
const listener = request => {
  console.log(`Request finished: ${request.url()}`);
};
```

We define a callback function that prints the completed request URL.

Step 2: Register the listener

```
page.on('requestfinished', listener);
```

Now Playwright starts monitoring completed requests.

Step 3: Navigate to Wikipedia

```
await page.goto('https://wikipedia.org');
```

The listener prints the URLs of completed requests.

Step 4: Remove the listener

```
page.off('requestfinished', listener);
```

Now Playwright stops calling this listener.

Step 5: Navigate to another website

```
await page.goto('https://www.openstreetmap.org/');
```

The previously registered listener no longer receives events.

Interview important
When using `page.off()`, pass the same callback function reference that you used with `page.on()`.

For example, this works:

```
const listener = request => {
  console.log(request.url());
};

page.on('request', listener);

page.off('request', listener);
```

This does not remove the original listener:

```
page.on('request', request => {
  console.log(request.url());
});

page.off('request', request => {
  console.log(request.url());
});
```

Why? Because these are two different function objects, even though their code looks identical.

Interview Q3. What is the difference between page.on() and page.off()?

Answer:

- `page.on()` registers an event listener.
- `page.off()` removes a previously registered event listener.

We use `page.off()` when we no longer need to monitor a particular event.

Interview Q4. Can we remove an event listener without passing its callback?

Answer:

To remove a specific listener using `page.off()`, we need to provide the same callback function reference that was registered earlier.

6. Adding one-time event listeners using page.once()

Sometimes we want to handle an event only one time.

For example, a browser dialog appears when we click a button. We want to accept that dialog and don't need to listen for future dialogs.

Playwright provides:

`page.once()`

It registers an event listener that automatically removes itself after the first event.

Example: Handle a prompt dialog

```
test('Handle prompt dialog', async ({ page }) => {

  await page.goto('https://example.com');

  page.once('dialog', async dialog => {

    console.log(dialog.message());

    await dialog.accept('2021');

  });

  await page.evaluate(() => {
    prompt('Enter a number:');
  });

});
```

Step-by-step explanation

Step 1:

```
page.once('dialog', async dialog => {
```

Registers a listener for the next dialog event only.

Step 2:

```
console.log(dialog.message());
```

Gets and prints the dialog message.

Step 3:

```
await dialog.accept('2021');
```

Accepts the prompt and enters `2021` as the input value.

Step 4:

```
await page.evaluate(() => {
  prompt('Enter a number:');
});
```

Executes JavaScript inside the browser to display the prompt.

After the first dialog event, the `page.once()` listener is automatically removed.

Where is page.once() useful?

- Handling a dialog only once.
- Handling the first popup event.
- Capturing a particular request event once.
- Listening for a one-time browser event.

Interview Q5. What is page.once()?

Answer:

`page.once()` registers an event listener that executes only once. After the first matching event, Playwright automatically removes the listener.

7. Difference between page.on(), page.once() and page.off()

This is a very common interview question.

Method	Purpose	Listener behavior
`page.on()`	Add an event listener	Continues listening
`page.once()`	Add a one-time listener	Automatically removed after first event
`page.off()`	Remove an event listener	Stops listening to that callback

Easy way to remember:

- on = Keep listening.
- `once` = Listen one time.
- `off` = Stop listening.

8. Difference between waiting for an event and adding a listener

Another important interview question.

Waiting for event	Adding event listener
`page.waitForEvent()`	`page.on()`
Waits for a particular event	Listens continuously
Returns a Promise	Executes a callback

Useful when an event is expected after an action	Useful when events can happen at different times
Example: Wait for popup	Example: Monitor network requests

Example of waiting:

```
const popupPromise = page.waitForEvent('popup');

await page.getByText('Open popup').click();

const popup = await popupPromise;
```

Example of continuous listening:

```
page.on('request', request => {
  console.log(request.url());
});

await page.goto('https://example.com');
```

Interview Q6. When would you use page.waitForEvent() instead of page.on()?

Answer:

I use `page.waitForEvent()` when I expect a particular event to happen after a specific action, such as a popup opening after clicking a button.

I use `page.on()` when I want to continuously monitor events, such as network requests during the execution of a test.

9. Practical examples for QA automation

These are useful patterns to understand for real projects.

Example 1: Capture an API request

```
const requestPromise = page.waitForRequest(
  request => request.url().includes('/api/login')
);

await page.getByRole('button', { name: 'Login' }).click();
```

```
const request = await requestPromise;

console.log(request.url());
console.log(request.method());
```

This waits for a request whose URL contains `/api/login` and prints its URL and HTTP method.

Example 2: Wait for an API response

```
const responsePromise = page.waitForResponse(
  response => response.url().includes('/api/login')
);

await page.getByRole('button', { name: 'Login' }).click();

const response = await responsePromise;

console.log(response.status());
```

This waits for the matching response and prints its HTTP status.

Example 3: Monitor failed requests

```
page.on('requestfailed', request => {

  console.log('Failed URL:', request.url());

  console.log(
    'Failure reason:',
    request.failure()?.errorText
  );

});
```

Useful for identifying network failures during test execution.

Example 4: Listen to a popup

```
const popupPromise = page.waitForEvent('popup');

await page.getByRole('link', {
```

```
  name: 'Open new window'
}).click();

const popup = await popupPromise;

console.log(await popup.title());
```

Gets the title of the newly opened popup page.

10. Important interview questions and answers

Q7. What are events in Playwright?

Answer: Events are occurrences in a browser page, such as network requests, responses, popup creation, dialogs and downloads. Playwright allows us to wait for or listen to these events.

Q8. What is the difference between waitForRequest() and waitForResponse()?

Answer:

- `waitForRequest()` waits until a matching network request is issued.
- `waitForResponse()` waits until a matching network response is received.

Q9. What is the difference between requestfinished and requestfailed?

Answer:

- `requestfinished` indicates that the request and its response body have finished downloading.
- `requestfailed` indicates that the request failed at the network level.

Q10. How do you monitor all network requests in Playwright?

Answer:

```
page.on('request', request => {
  console.log(request.url());
});
```

This listens to every request issued by the page.

Q11. How do you stop monitoring network requests?

Answer:

I first store the callback function in a variable and then use `page.off()` with the same event name and callback reference.

Q12. Can page.once() be used for dialogs?

Answer: Yes. We can use `page.once('dialog', ...)` to handle the next dialog only once.

Q13. What is a popup event?

Answer: A popup event occurs when a page opens another page, usually through a link or JavaScript such as `window.open()`.

Q14. What does page.waitForEvent() return?

Answer: It returns a Promise that resolves with the event payload when the specified event occurs.

Q15. What happens if the expected event never occurs?

Answer: The wait eventually times out and the test fails with a timeout error, unless a different timeout has been configured.

Q16. Can we use a condition with waitForRequest()?

Answer: Yes. We can pass a URL pattern or a predicate function to identify the request we want.

Example:

```
await page.waitForRequest(
  request => request.url().includes('/api/users')
);
```

Q17. Can page.on() listen to multiple event types?

Answer: Yes. We can register separate listeners for different event types, such as `request`, `response`, `dialog` and `requestfailed`.

Q18. Why should we remove event listeners?

Answer: Removing unnecessary listeners avoids unwanted callbacks, duplicate logs and unnecessary event processing during later test steps.

11. Quick revision notes

Method	Remember this
`page.waitForRequest()`	Wait for a network request
`page.waitForResponse()`	Wait for a network response
`page.waitForEvent()`	Wait for a particular page event
`page.on()`	Continuously listen
`page.off()`	Remove a listener
`page.once()`	Listen only once
`page.waitForEvent('popup')`	Wait for a popup
`page.on('requestfailed')`	Monitor failed requests

12. Interview quiz

Try these six questions to check your understanding.

Playwright Events Quiz

6/6 answered

1. Which method is used to continuously listen to network requests?

page.once()
page.on()
page.off()
page.evaluate()

Correct

page.on() keeps listening until the listener is removed or the page is closed.

2. Which method automatically removes its listener after the first event?

page.on()
page.off()
page.once()
page.waitForRequest()

Correct

page.once() listens for one matching event and then removes itself.

3. Which method waits for a popup?

page.waitForEvent('popup')
page.on('request')
page.waitForRequest()
page.once('response')

Correct

page.waitForEvent('popup') waits for a popup created by the page.

4. What does page.off() do?

Opens a popup
Starts navigation
Removes an event listener
Waits for a request

Correct

page.off() removes the specified listener.

5. Which event indicates that a network request has been issued?

requestfinished
request
requestfailed
dialog

Correct

The request event fires when a request is issued.

6. Why should we create a waitForEvent promise before clicking?

To close the browser
To avoid missing the event
To make the page faster
To remove the listener

Correct

The event may happen immediately after clicking, so we register the wait first.

6/6 correct

Try again

Final interview tip: Be especially confident about the difference between `waitForEvent()`, `page.on()`, `page.once()` and `page.off()`. Interviewers commonly test these differences using popup and network-request scenarios.

Send me the next Playwright section, and we'll continue with the same explanation and interview-preparation format.

Playwright – Extensibility: Custom Selector Engines

Let's understand this topic in simple language with practical examples and interview questions for QA Automation / SDET interviews.

1. What is extensibility?

Extensibility means adding new functionality to Playwright according to our project requirements.

Playwright already provides built-in locators such as:

- `getByRole()`
- `getByText()`
- `getByLabel()`

- `getByPlaceholder()`
- `locator()`

But sometimes our project may require a custom way to identify elements.

For this, Playwright supports Custom Selector Engines.

Simple example

Suppose your application has many buttons, and you want to identify buttons using their HTML tag name.

You can create a custom selector engine called `tag`.

Then you can write:

```
const button = page.locator('tag=button');
await button.click();
```

Here, `tag=button` is a custom selector.

Normally, you could also use:

```
page.locator('button');
```

But this example helps us understand how custom selector engines work.

2. What is a Custom Selector Engine?

A custom selector engine allows us to define our own logic for finding elements on a webpage.

Playwright provides:

```
playwright.selectors.register()
```

We use this method to register a custom selector engine.

Syntax:

```
await playwright.selectors.register(
  'customName',
  selectorEngine
);
```

- `customName`: Name of your custom selector.
- `selectorEngine`: Function that contains the logic for finding elements.

For example:

```
await playwright.selectors.register(
  'tag',
  createTagNameEngine
);
```

After registration, we can use:

```
page.locator('tag=button');
```

Here, `tag` is the custom selector engine name.

3. Important properties of a Custom Selector Engine

A custom selector engine must have two important functions:

1. `query()`
2. `queryAll()`

Let's understand both.

A. query()

The `query()` function returns the first element matching the given selector.

Example:

```
query(root, selector) {
  return root.querySelector(selector);
}
```

Suppose the HTML is:

```
<button>Login</button>
<button>Submit</button>
<button>Cancel</button>
```

If we use:

```
page.locator('tag=button');
```

The `query()` function finds the first matching button.

Remember
`query()` = First matching element.

B. queryAll()

The `queryAll()` function returns all elements matching the given selector.

Example:

```
queryAll(root, selector) {
  return Array.from(root.querySelectorAll(selector));
}
```

Suppose the HTML is:

```
<button>Login</button>
<button>Submit</button>
<button>Cancel</button>
```

The function finds all three buttons.

Remember
`queryAll()` = All matching elements.

Difference between query() and queryAll()

query()	queryAll()
Returns the first matching element	Returns all matching elements
Uses `querySelector()`	Uses `querySelectorAll()`
Returns one element or null	Returns an array of elements

Interview Q1. What are the two required functions in a custom selector engine?

Answer: `query()` and `queryAll()`. The `query()` function returns the first matching element, whereas `queryAll()` returns all matching elements.

4. Understanding root and selector

In this example:

```
query(root, selector) {
  return root.querySelector(selector);
}
```

There are two parameters.

root: The DOM element or search root from which Playwright should search for matching elements.

selector: The selector string used to find an element.

For example:

```
page.locator('tag=button');
```

- `root` represents the search area, usually the page's document.
- `selector` is `button`.

Another example:

```
page.locator('tag=div').getByText('Click me');
```

Here, the custom selector engine identifies the `div` elements, and Playwright's text locator is used to find the text within the selected area.

5. Complete example – Registering a custom selector engine

Let's understand the documentation's code step by step.

Step 1: Create baseTest.ts

This file registers the custom selector engine.

```
import { test as base } from '@playwright/test';

export { expect } from '@playwright/test';

// Create custom selector engine
```

```
const createTagNameEngine = () => ({

  // Find first matching element
  query(root, selector) {
    return root.querySelector(selector);
  },

  // Find all matching elements
  queryAll(root, selector) {
    return Array.from(root.querySelectorAll(selector));
  }

});

// Extend Playwright test
export const test = base.extend<{}, {
  selectorRegistration: void
}>({

  selectorRegistration: [
    async ({ playwright }, use) => {

      // Register custom selector
      await playwright.selectors.register(
        'tag',
        createTagNameEngine
      );

      await use();

    },

    {
      scope: 'worker',
      auto: true
    }
  ]

});
```

Explanation of important code

1. Import the base test

```
import { test as base } from '@playwright/test';
```

We import Playwright's original test object and rename it as `base`.

Why?

Because we want to extend the existing Playwright test with our custom setup.

2. Create the selector engine

```
const createTagNameEngine = () => ({
```

This function creates our custom selector engine.

It contains `query()` and `queryAll()`.

3. Register the selector

```
await playwright.selectors.register(
  'tag',
  createTagNameEngine
);
```

This registers the engine with the name `tag`.

After registration, we can use selectors like:

```
page.locator('tag=button');
```

4. Understand the fixture

```
selectorRegistration: [
  async ({ playwright }, use) => {
    // Register engine
    await playwright.selectors.register(
      'tag',
      createTagNameEngine
    );

    await use();
  },
  {
```

```
    scope: 'worker',
    auto: true
  }
]
```

Here, Playwright uses a fixture to register the custom selector.

- `playwright`: Provides access to Playwright APIs.
- `use()`: Allows the fixture to continue and provide its setup to the test.
- `scope: 'worker'`: Runs the fixture once per worker rather than before every individual test.
- `auto: true`: Automatically runs the fixture without requiring us to mention it in every test.

Why use worker scope?

We only need to register the selector engine once per worker, rather than registering it repeatedly for every test.

Step 2: Use the selector in example.spec.ts

```
import { test, expect } from './baseTest';

test('selector engine test', async ({ page }) => {

  // Find button using custom selector
  const button = page.locator('tag=button');

  await button.click();

  // Combine custom selector with built-in locator
  await page.locator('tag=div')
    .getByText('Click me')
    .click();

  // Verify the number of buttons
  await expect(
    page.locator('tag=button')
  ).toHaveCount(3);

});
```

What happens here?

1. We import `test` and `expect` from our custom `baseTest.ts`.
2. Playwright automatically registers the custom selector through the fixture.
3. We use `tag=button` to find buttons.
4. We click the button.
5. We combine the custom selector with a built-in text locator.
6. We verify that the page contains three buttons.

6. What is contentScript: true?

This is another important concept from the documentation.

By default, a custom selector engine runs directly in the frame's JavaScript context.

This means it shares the page's JavaScript environment.

However, we can register the selector engine with:

```
await playwright.selectors.register(
  'tag',
  createTagNameEngine,
  { contentScript: true }
);
```

Why do we need contentScript: true?

Suppose a website modifies JavaScript's built-in methods.

For example, the application changes:

`Node.prototype.querySelector`

This could interfere with a custom selector engine running in the page's JavaScript context.

Using:

`{ contentScript: true }`

isolates the selector engine from the page's JavaScript environment while still allowing it to access the DOM.

This makes it safer against tampering with global JavaScript objects.

Important points:

- Default custom engines run in the frame's JavaScript context.
- `contentScript: true` provides an isolated environment.
- DOM access is still available.
- All built-in Playwright selector engines run as content scripts.
- Isolation is not guaranteed when the engine is used together with other custom engines.

Interview Q2. What is contentScript in custom selector registration?

Answer: `contentScript: true` runs the custom selector engine in an isolated environment, protecting it from changes made to global JavaScript objects by the application. It still allows the engine to access the DOM.

7. Important rules to remember

1. Register custom selector engines before creating a page.
2. Every custom selector engine should provide `query()` and `queryAll()`.
3. Use `selectors.register()` to register an engine.
4. Use `contentScript: true` when isolation is required.
5. Use worker-scoped fixtures when registration should happen once per worker.
6. Once registered, the custom selector can be used with Playwright locator methods.

8. Real-time project scenario

Imagine you are automating an e-commerce application.

The application contains multiple elements:

```
<button>Login</button>
<button>Search</button>
<button>Add to Cart</button>
```

Your team wants to use a common custom selector convention across its automation framework.

You create a custom selector engine called `tag`.

Now the team can write:

```
await page.locator('tag=button').first().click();
```

Or combine it with built-in locators:

```
await page.locator('tag=div')
  .getByText('Add to Cart')
  .click();
```

Practical advice: Custom selector engines are useful when a project genuinely needs custom element-matching logic. For normal automation, prefer Playwright's built-in locators such as `getByRole()` and `getByTestId()` because they are generally more readable and maintainable.

9. Interview questions and answers

Q3. What is a custom selector engine in Playwright?

Answer: A custom selector engine allows us to define our own logic to identify elements on a webpage. We register it using `playwright.selectors.register()`.

Q4. Which method is used to register a custom selector engine?

Answer: `playwright.selectors.register()`.

Q5. What is the purpose of query()?

Answer: It returns the first element matching the specified selector within the given root.

Q6. What is the purpose of queryAll()?

Answer: It returns all elements matching the specified selector within the given root.

Q7. What is the difference between querySelector() and querySelectorAll()?

Answer: `querySelector()` returns the first matching element, while `querySelectorAll()` returns all matching elements as a NodeList.

Q8. What is the purpose of contentScript: true?

Answer: It isolates the custom selector engine from the webpage's JavaScript environment while preserving DOM access. This helps protect it from application-side changes to global JavaScript objects.

Q9. When should we register custom selector engines?

Answer: We must register them before creating a page.

Q10. Can we combine custom selectors with built-in Playwright locators?

Answer: Yes. For example:

```
await page.locator('tag=div')
  .getByText('Click me')
  .click();
```

Q11. What is the purpose of scope: 'worker' in the fixture?

Answer: It makes the fixture execute once per worker instead of once per test. This avoids unnecessary repeated registration.

Q12. What does auto: true mean in a Playwright fixture?

Answer: It means the fixture runs automatically for tests using the extended test object, even if the fixture is not explicitly requested in the test parameters.

Q13. Can we use custom selectors with expect() assertions?

Answer: Yes. We can use custom selectors with Playwright methods that accept selectors, including locator assertions.

Example:

```
await expect(
  page.locator('tag=button')
).toHaveCount(3);
```

Q14. Are custom selector engines necessary for every Playwright project?

Answer: No. Playwright already provides many built-in locators. Custom selector engines are useful when we need special element-identification logic that built-in locators do not provide conveniently.

10. Quick revision notes

Topic	Remember
Extensibility	Adding custom functionality
Custom selector engine	Custom logic to locate elements
Registration method	`playwright.selectors.register()`
`query()`	First matching element
`queryAll()`	All matching elements
`root`	Search starting point
`selector`	Element-matching expression
`contentScript: true`	Isolated selector execution
`scope: 'worker'`	Once per worker
`auto: true`	Automatic fixture execution

11. Interview quiz

Custom Selector Engines Quiz

6/6 answered

1. Which API registers a custom selector engine?

page.locator()
playwright.selectors.register()
page.evaluate()
test.extend()

Correct

selectors.register() registers the custom selector engine.

2. Which function returns the first matching element?

queryAll()
query()
register()
evaluate()

Correct

query() returns the first matching element.

3. Which function returns all matching elements?

query()
queryAll()
getByRole()
register()

Correct

queryAll() returns all matching elements.

4. What does contentScript: true provide?

Faster screenshots
Isolated selector execution
Automatic retries
File downloads

Correct

It isolates the selector engine from the page's JavaScript environment while allowing DOM access.

5. When must a custom selector be registered?

After creating the page
Before creating the page
After closing the browser
After every click

Correct

Selectors must be registered before creating the page.

6. What does scope: 'worker' mean?

Runs once per worker
Runs after every assertion
Runs once per browser action
Runs only after a failure

Correct

Worker-scoped fixtures run once per worker.

6/6 correct

Retry quiz

Most important interview takeaway: A custom selector engine lets us define our own element-finding logic using `query()` and `queryAll()`, register it with `playwright.selectors.register()`, and use it through Playwright locators.

Playwright – Frames

Let's understand Frames in Playwright in simple language with practical examples and interview questions for QA Automation / SDET interviews.

1. What is a Frame?

A Frame is a separate HTML document embedded inside another webpage.

For example, imagine you open a banking website. The main page contains a login form, but the login form is displayed inside a separate embedded section called an iframe.

Playwright needs to identify that frame before interacting with its elements.

How To Handle Frames In Selenium - Scaler Topics

Real-time examples of iframes

- Payment gateway forms.
- Login forms embedded in another website.
- YouTube videos embedded in websites.
- Google Maps embedded in websites.
- Captcha sections.
- Third-party chat widgets.

2. Understanding Page and Frame

According to Playwright:

- A Page can contain one or more Frame objects.
- Every page has one main frame.
- Additional frames can be attached using the HTML `iframe` tag.
- Page-level interactions such as `click()` normally operate in the main frame.

Let's understand the structure.

Page (Parent webpage)

Main Frame

Main page elements

Header · Navigation · Main content

Contains iframe

Child Frame (iframe)

Separate embedded document

Username: [Enter username]

Password: [Enter password]

Login button

Important: The main page and child frame have separate document contexts. To interact with an element inside an iframe, use a frame-aware locator or get the Frame object.

3. How to interact with elements inside a Frame?

Playwright provides `frameLocator()`.

What is frameLocator()?

`frameLocator()` helps us locate and interact with elements inside an iframe.

Syntax:

```
page.frameLocator('iframe-selector')
```

Example from the documentation:

```
const username = await page
  .frameLocator('.frame-class')
  .getByLabel('User Name');

await username.fill('John');
```

Step-by-step explanation

Step 1:

```
page.frameLocator('.frame-class')
```

Identifies the iframe using its CSS class.

Step 2:

```
.getByLabel('User Name')
```

Finds the username input field inside that iframe using its accessible label.

Step 3:

```
await username.fill('John');
```

Enters John into the username field.

Practical example

Suppose your HTML looks like this:

```
<iframe class="login-frame">
  <!-- Separate document -->
  <input aria-label="User Name">
  <input aria-label="Password">
  <button>Login</button>
</iframe>
```

Playwright code:

```
import { test, expect } from '@playwright/test';

test('Login inside iframe', async ({ page }) => {

  await page.goto('https://example.com');

  const frame = page.frameLocator('.login-frame');

  await frame.getByLabel('User Name').fill('Komal');

  await frame.getByLabel('Password').fill('Password123');

  await frame.getByRole('button', {
    name: 'Login'
  }).click();

});
```

What is happening?

1. Open the website.
2. Identify the iframe.
3. Locate the username field inside the iframe.
4. Enter the username.

5. Enter the password.
6. Click the Login button inside the iframe.

Interview Q1. What is frameLocator() in Playwright?

Answer: `frameLocator()` is used to locate elements inside an iframe. It allows us to perform actions such as click, fill and select inside the frame without manually switching browser contexts.

4. What are Frame Objects?

Playwright also provides the `page.frame()` method to access Frame objects.

A Frame object represents a particular frame attached to a page.

Syntax:

```
const frame = page.frame('frame-name');
```

We can identify frames using:

1. Frame name.
2. Frame URL.

A. Get frame using its name

HTML example:

```
<iframe name="frame-login" src="/login"></iframe>
```

Playwright code:

```
const frame = page.frame('frame-login');
```

Here, Playwright finds the frame using its name attribute.

B. Get frame using its URL

HTML example:

```
<iframe src="https://example.com/login"></iframe>
```

Playwright code:

```
const frame = page.frame({
  url: /.*domain.*/
});
```

Here, Playwright finds a frame whose URL matches the regular expression.

For example, if the frame URL contains `domain`, it can match.

C. Interact with the Frame object

Once we get the Frame object, we can interact with its elements.

```
const frame = page.frame('frame-login');

await frame.fill('#username-input', 'John');
```

Here:

- `page.frame()` gets the Frame object.
- `fill()` enters text into the input field inside that frame.

Important: `page.frame()` returns a Frame object or `null` if no matching frame is found. So in TypeScript, handle the possibility that the frame is not found.

Safer example:

```
const frame = page.frame('frame-login');

if (!frame) {
  throw new Error('Login frame not found');
}

await frame.locator('#username-input').fill('John');
```

5. Difference between frameLocator() and page.frame()

This is a very important interview question.

`frameLocator()`	`page.frame()`
Returns a FrameLocator	Returns a Frame object or null

Used to locate elements inside a frame	Used to access the frame itself
Supports locator methods	Supports Frame methods
Does not require manually getting the Frame object	Requires identifying the frame
Supports auto-waiting through locators	Frame methods have their own waiting behavior

Example using frameLocator()

```
await page
  .frameLocator('#login-frame')
  .locator('#username')
  .fill('Komal');
```

Example using page.frame()

```
const frame = page.frame({
  name: 'login-frame'
});

if (!frame) {
  throw new Error('Frame not found');
}

await frame.locator('#username').fill('Komal');
```

When should we use which?

- Use `frameLocator()` for normal UI automation inside iframes. It is usually simpler.
- Use `page.frame()` when you need to access the Frame object and its properties or methods, such as its URL, name or lifecycle events.

6. Important Frame methods for interviews

Here are additional methods worth knowing.

Method	Purpose
`page.mainFrame()`	Returns the main frame
`page.frames()`	Returns all frames attached to a page
`page.frame()`	Finds a frame by name or URL
`frame.url()`	Returns the frame URL
`frame.name()`	Returns the frame name
`frame.locator()`	Locates elements inside the frame
`frame.frameLocator()`	Locates nested frames
`frame.parentFrame()`	Returns the parent frame
`frame.childFrames()`	Returns child frames
`frame.isDetached()`	Checks whether the frame is detached

Example: Get all frames

```
const allFrames = page.frames();

console.log(allFrames.length);
```

This returns all frames attached to the page, including the main frame.

Example: Get main frame

```
const mainFrame = page.mainFrame();

console.log(mainFrame.url());
```

Example: Get frame URL

```
const frame = page.frame('frame-login');

if (frame) {
  console.log(frame.url());
}
```

7. Real-time interview scenario

Scenario: You are automating a payment application. The payment form is inside an iframe. How will you enter card details?

Example:

```
test('Payment using iframe', async ({ page }) => {

  await page.goto('https://example.com/checkout');

  const paymentFrame = page.frameLocator(
    'iframe[title="Payment"]'
  );

  await paymentFrame
    .getByPlaceholder('Card number')
    .fill('4111111111111111');

  await paymentFrame
    .getByPlaceholder('Expiry date')
    .fill('12/28');

  await paymentFrame
    .getByPlaceholder('CVV')
    .fill('123');

  await paymentFrame
```

```
    .getByRole('button', { name: 'Pay' })
    .click();

});
```

This demonstrates how to automate a payment form embedded inside an iframe. In real projects, use test card details supplied by the payment provider rather than real customer card details.

8. Interview Questions and Answers

Q2. What is an iframe?

Answer: An iframe is an HTML element that embeds another HTML document inside the current webpage. It has its own document context.

Q3. Does every Playwright page have a main frame?

Answer: Yes. Every page has one main frame. It can also contain additional child frames.

Q4. How do you handle an iframe in Playwright?

Answer: We can use `frameLocator()` to locate elements inside an iframe or `page.frame()` to get the Frame object.

Q5. What is the difference between a Page and a Frame?

Answer: A Page represents a browser tab or popup, whereas a Frame represents a document inside that page. Every page has a main frame and may contain additional child frames.

Q6. How can you identify a frame by its name?

Answer:

```
const frame = page.frame('frame-login');
```

Q7. How can you identify a frame by its URL?

Answer:

```
const frame = page.frame({
  url: /.*login.*/
});
```

Q8. What does page.frames() return?

Answer: It returns an array containing all Frame objects attached to the page, including the main frame.

Q9. What does page.mainFrame() return?

Answer: It returns the main Frame object of the page.

Q10. Can we use getByRole() inside an iframe?

Answer: Yes. We can use it through `frameLocator()`.

```
await page
  .frameLocator('#login-frame')
  .getByRole('button', { name: 'Login' })
  .click();
```

Q11. What happens if page.frame() cannot find a matching frame?

Answer: It returns `null`. We should check whether the frame exists before interacting with it.

Q12. Can an iframe contain another iframe?

Answer: Yes. An iframe can contain another iframe. We can handle nested frames using chained `frameLocator()` calls.

Example:

```
await page
  .frameLocator('#parent-frame')
  .frameLocator('#child-frame')
  .locator('#username')
  .fill('Komal');
```

Q13. What is the difference between page.locator() and frameLocator()?

Answer: `page.locator()` locates elements in the page's main frame, whereas `frameLocator()` locates elements inside a specified iframe.

Q14. How do you handle a frame that loads dynamically?

Answer: We can use `frameLocator()` with a locator-based action. Playwright automatically waits for the matching iframe and target element to become available, subject to the action's timeout.

Q15. What is a detached frame?

Answer: A detached frame is a frame that has been removed from the page. For example, an iframe may be removed when a popup or embedded section is closed.

9. Quick revision notes

Topic	Remember
Frame	Separate document inside a page
Main frame	Primary document of a page
iframe	HTML element used to embed another document
`frameLocator()`	Interact with iframe elements
`page.frame()`	Get Frame object
`page.frames()`	Get all frames
`page.mainFrame()`	Get main frame
`frame.url()`	Get frame URL
`frame.name()`	Get frame name
`frame.parentFrame()`	Get parent frame
`frame.childFrames()`	Get child frames

10. Interview quiz

Try these questions to check your understanding.

Frames – Quiz

6/6 answered

1. What is an iframe?

A browser tab
An HTML document embedded inside another page
A Playwright fixture
A browser event

Correct

An iframe embeds another HTML document inside the current webpage.

2. Which method is used to interact with elements inside an iframe using locators?

page.frame()
page.frameLocator()
page.mainFrame()
page.frames()

Correct

frameLocator() creates a locator scope inside the specified iframe.

3. What does page.frame() return if no matching frame is found?

An empty array
False
Null
Undefined

Correct answer: Null

page.frame() returns a Frame object or null.

4. Which method returns all frames attached to a page?

page.frames()
page.frame()
page.mainFrame()
page.locator()

Correct

page.frames() returns an array of all attached Frame objects.

5. Which method returns the main frame?

page.frame()
page.mainFrame()
frame.parentFrame()
frame.childFrames()

Correct

page.mainFrame() returns the page's main frame.

6. Can an iframe contain another iframe?

No
Yes
Only in Chrome
Only in Firefox

Correct

Nested iframes are supported and can be accessed using chained frameLocator() calls.

5/6 correct

Try again

Most important interview point: Use `frameLocator()` for normal iframe automation. Use `page.frame()` when you need to access the actual Frame object.

Send me the next Playwright section, and we'll continue in the same simple explanation and interview Q&A format.

Playwright – Handles (JSHandle and ElementHandle)

1. What are Handles?

In simple words, a Handle is a reference to an object or DOM element inside the browser.

Playwright runs in Node.js, while the actual webpage runs inside the browser. A handle allows Playwright to access and work with objects that exist inside the browser.

Playwright provides two types of handles:

1. JSHandle – Reference to any JavaScript object inside the browser.
2. ElementHandle – Reference specifically to a DOM element inside the browser.

Handle types

JSHandle

References any JavaScript object

Special type of JSHandle

ElementHandle

References a DOM element

Remember: Every ElementHandle is a JSHandle, but every JSHandle is not an ElementHandle.

For example:

- `window` → JSHandle
- JavaScript array → JSHandle
- Login button → ElementHandle
- Username input → ElementHandle

2. JSHandle – Example

The easiest way to create a JSHandle is using `page.evaluateHandle()`.

```
import { test } from '@playwright/test';

test('JSHandle example', async ({ page }) => {
  await page.goto('https://example.com');

  const jsHandle = await page.evaluateHandle(() => window);
```

```
  console.log(jsHandle);
});
```

Explanation:

- `page.evaluateHandle()` executes JavaScript inside the browser.
- `window` represents the browser window object.
- Playwright receives a handle referring to that object.
- We can use this handle for further evaluations.

Interview point: `evaluate()` returns a serializable value, whereas `evaluateHandle()` returns a handle to an object inside the browser.

3. ElementHandle

An ElementHandle refers to one particular HTML element.

For example, suppose the webpage contains:

```
<div id="box" class="highlighted">
  Hello Playwright
</div>
```

We can access it like this:

```
import { test, expect } from '@playwright/test';

test('ElementHandle example', async ({ page }) => {
  await page.goto('https://example.com');

  const elementHandle = await page.waitForSelector('#box');

  const boundingBox = await elementHandle.boundingBox();

  console.log(boundingBox);

  const classNames = await elementHandle.getAttribute('class');

  expect(classNames).toContain('highlighted');
});
```

Explanation:

Method	Meaning
`waitForSelector()`	Waits for the element to appear
`boundingBox()`	Gets element position and dimensions
`getAttribute()`	Gets an HTML attribute
`innerText()`	Gets visible text
`innerHTML()`	Gets HTML inside the element
`textContent()`	Gets text content

For example, `boundingBox()` can return:

```
{
  x: 100,
  y: 150,
  width: 100,
  height: 50
}
```

This tells us the element's position and size on the page.

Important
Playwright discourages using ElementHandle for normal automation. Prefer Locator APIs.

4. Handles as Parameters

This is an important concept for interviews.

We can pass a handle to `page.evaluate()` and perform operations on the actual browser-side object.

Consider this example:

```
test('Handle as parameter', async ({ page }) => {

  // Create an array inside the browser
  const myArrayHandle = await page.evaluateHandle(() => {
    const myArray = [1];
    return myArray;
  });

  // Get array length
  const length = await page.evaluate(
    array => array.length,
    myArrayHandle
  );

  console.log(length); // 1

  // Add another element
  await page.evaluate(
    ({ array, newElement }) => {
      array.push(newElement);
    },
    {
      array: myArrayHandle,
      newElement: 2
    }
  );

  // Release handle
  await myArrayHandle.dispose();
});
```

Understand it step by step:

1. `evaluateHandle()` creates an array `[1]` inside the browser.
2. `myArrayHandle` stores a reference to that array.
3. `page.evaluate()` receives the handle as a parameter.
4. `array.length` returns `1`.
5. `array.push(2)` adds another element.

6. `dispose()` releases the handle when it is no longer needed.

The array is stored in the browser, but Playwright can interact with it using the handle.

5. Handle Lifecycle

Lifecycle means how a handle is created, used and released.

Step 1: Create a handle

You can create handles using:

Method	Purpose
`page.evaluateHandle()`	Gets a handle to a JavaScript object
`page.$()`	Gets an ElementHandle for the first matching element
`page.$$()`	Gets ElementHandles for all matching elements
`frame.evaluateHandle()`	Gets a handle inside a frame
`frame.$()`	Gets an element handle inside a frame

Step 2: Use the handle

You can:

- Read element attributes.
- Get element text.
- Get dimensions.
- Execute JavaScript on the referenced object.

Step 3: Release the handle

```
await jsHandle.dispose();
```

Why is this important?

Handles retain references to browser objects and can prevent those objects from being garbage-collected. Releasing unused handles helps manage memory.

Handles are also released when the page navigates.

6. Locator vs ElementHandle

Very important interview question

This is the most important part of this topic.

ElementHandle: Points to a particular DOM element.

Locator: Stores the logic or selector used to find an element. It finds the current matching element whenever an action is performed.

Example using ElementHandle

```
const handle = await page.$('text=Submit');

await handle.hover();

await handle.click();
```

Here, the handle refers to one particular DOM element.

Suppose React replaces that element after a page update. The handle may still refer to the old, detached element and cause an error.

Example using Locator

```
const locator = page.getByText('Submit');

await locator.hover();

await locator.click();
```

Here, Playwright locates the matching element again when each action is performed.

Comparison table

Feature	Locator	ElementHandle
References	Element-finding logic	Particular DOM element
Re-fetches element	Yes, when used	No
Auto-waiting	Yes	Limited
Handles DOM updates	More reliable	Can become stale
Recommended	Yes	Discouraged
Best for	Normal automation	Rare DOM traversal cases

Real-world example:

Imagine an e-commerce website where React refreshes the Add to Cart button after selecting a product.

- ElementHandle may point to the old button.
- Locator can find the updated button when you click it.

That's why Playwright recommends Locator over ElementHandle.

7. Real-world QA automation example

Suppose you are testing a login page.

```
import { test, expect } from '@playwright/test';

test('Login functionality', async ({ page }) => {

  await page.goto('https://example.com/login');

  // Use locators for user actions
```

```
  await page.getByLabel('Username').fill('testuser');

  await page.getByLabel('Password').fill('password123');

  await page.getByRole('button', { name: 'Login' }).click();

  // Web-first assertion
  await expect(page.getByText('Welcome')).toBeVisible();

});
```

Here, we use Locators instead of ElementHandles because they provide auto-waiting and are more resilient to DOM changes.

Interview Questions and Answers

Basic questions

1. What is a Handle in Playwright?

A handle is a reference to an object inside the browser that allows Playwright to access and interact with that object.

2. What are the two types of handles?

- JSHandle
- ElementHandle

3. What is JSHandle?

JSHandle is used to reference any JavaScript object inside the browser, such as an array, object or window.

4. What is ElementHandle?

ElementHandle is used to reference a particular DOM element and provides methods to interact with or inspect that element.

5. What is the difference between JSHandle and ElementHandle?

JSHandle references any JavaScript object, whereas ElementHandle specifically references a DOM element. ElementHandle is a specialized type of JSHandle.

Intermediate questions

6. How do you create a JSHandle?

Using `page.evaluateHandle()`.

```
const handle = await page.evaluateHandle(() => window);
```

7. What is the difference between `evaluate()` and `evaluateHandle()`?

- `evaluate()` returns a serializable result.
- `evaluateHandle()` returns a handle referencing an object inside the browser.

8. How do you release a handle?

Using:

```
await handle.dispose();
```

9. What happens if we don't dispose of a handle?

The handle can retain a reference to the browser object and prevent it from being garbage-collected while the handle remains active.

10. What is `boundingBox()`?

It returns the position and dimensions of an element, including x, y, width and height.

Advanced questions

11. What is the difference between Locator and ElementHandle?

Locator stores the logic to find an element and resolves it when needed.
ElementHandle refers to a specific DOM element, which may become stale if the DOM changes.

12. Why does Playwright discourage ElementHandle?

Because it can point to outdated or detached elements. Locators provide auto-waiting and handle DOM changes more reliably.

13. Can we pass a JSHandle to `page.evaluate()`?

Yes. A handle can be passed as an argument to access the referenced browser-side object.

14. What happens to handles after page navigation?

Handles associated with the previous page document are disposed when navigation invalidates their execution context.

15. When should we use ElementHandle?

Only in rare cases, such as extensive DOM traversal on a static page, where a specific element reference is required.

Quick Revision Notes

Concept	Remember
JSHandle	Reference to any JS object
ElementHandle	Reference to a DOM element
`evaluateHandle()`	Creates a handle
`evaluate()`	Returns a serializable value
`dispose()`	Releases a handle
`boundingBox()`	Gets element dimensions and position
Locator	Recommended for automation
ElementHandle	Use only in rare cases

Test Your Knowledge

Playwright Handles Quiz

4/4 answered

1. Which method creates a JSHandle?

page.evaluate()
page.evaluateHandle()
page.locator()

Correct! evaluateHandle() returns a handle to an object inside the browser.

2. Which one is recommended for normal Playwright automation?

ElementHandle
JSHandle
Locator

Correct! Locator supports auto-waiting and resolves the current matching element.

3. How do you release a handle?

handle.close()
handle.dispose()
handle.remove()

Correct! dispose() releases the handle and its reference.

4. Is every ElementHandle a JSHandle?

Yes
No

Correct! ElementHandle is a specialized type of JSHandle because DOM elements are JavaScript objects.

Your score: 4/4

Try again

Interview tip: If asked about Handles, focus especially on the difference between `JSHandle`, `ElementHandle`, and `Locator`. The most important point is that Locator is preferred over ElementHandle for regular test automation.

Playwright – Test Isolation

1. What is Test Isolation?

In simple words, Test Isolation means every test runs independently without sharing data or browser state with other tests.

Playwright achieves test isolation using BrowserContext.

Each test gets its own isolated browser context, which is similar to opening a new Incognito window.

For example:

Imagine you are testing a shopping website.

- Test 1: Login as Komal and add a product to the cart.
- Test 2: Open the shopping website without logging in.
- Test 3: Login as another user.

Each test gets a fresh browser context, so cookies, local storage and session storage are not shared between tests.

Modern Web Testing with Playwright | Automation Panda
automationpanda.com

2. What is BrowserContext?

A BrowserContext is an isolated browser session.

Think of it like separate Incognito windows running inside the same browser.

Each BrowserContext has its own:

- Cookies
- Local storage
- Session storage
- Permissions

- Pages and tabs

Important: Multiple BrowserContexts can run inside a single browser instance.

Browser vs BrowserContext vs Page

Term	Meaning
Browser	Actual browser instance, such as Chromium
BrowserContext	An isolated browser session
Page	A browser tab inside a context

Example structure:

Browser – Chromium

Context 1

Isolated

Page 1

User A cookies and storage

Context 2

Isolated

Page 2

User B cookies and storage

3. Why is Test Isolation Important?

There are three main benefits.

1. No failure carry-over

If Test 1 fails, it should not affect Test 2.

Example: If Test 1 adds a product to the cart, Test 2 should not automatically see that product.

2. Easy debugging

You can run one test independently without worrying about the state created by another test.

3. Supports parallel execution

Multiple tests can run at the same time without sharing browser session data.

For example:

- Test 1 → Login
- Test 2 → Search product
- Test 3 → Add to cart

Each test can execute independently.

4. Two Ways of Test Isolation

There are two strategies.

Strategy 1: Start from scratch

Create a completely fresh environment for every test.

Example:

- Test 1 gets a new browser context.
- Test 2 gets another new browser context.
- Test 3 gets another new browser context.

Recommended by Playwright

Strategy 2: Cleanup between tests

Use the same environment but clean up the state after each test.

For example:

- Clear cookies.
- Clear local storage.
- Remove added products.
- Log out the user.

Problem with cleanup:

Sometimes developers forget to clean up everything. Some browser state can be difficult to reset, which can cause one test to affect another.

Comparison

Start from scratch	Cleanup between tests
Creates a fresh context	Reuses an environment
Strong isolation	Depends on cleanup
Less state leakage	Risk of state leakage
Easier debugging	Harder debugging
Recommended	Less reliable

Interview point: Playwright prefers creating a fresh BrowserContext instead of relying on cleanup between tests.

5. How Playwright Achieves Test Isolation

Playwright Test Runner automatically creates a new BrowserContext for each test.

Let's understand the code.

```
import { test, expect } from '@playwright/test';

test('First test', async ({ page, context }) => {
```

```
  // This test has its own isolated context
  await page.goto('https://example.com');

});

test('Second test', async ({ page, context }) => {

  // This test gets another isolated context
  await page.goto('https://example.com');

});
```

Explanation

`test()`

Defines an individual test case.

`{ page, context }`

These are Playwright fixtures.

- `page` → Browser tab.
- `context` → Isolated browser session for that test.

Important: Playwright automatically creates these fixtures when using the Playwright Test Runner.

You don't need to manually create a new context for every normal test.

6. Multiple Contexts in a Single Test

This is a very important concept.

Sometimes we need to test two different users interacting with the same application.

Example: Chat application.

- Admin sends a message.
- User receives the message.

- Both users should have separate login sessions.

For this, we can create multiple BrowserContexts inside one test.

Code example

```
import { test, expect } from '@playwright/test';

test('Admin and user chat', async ({ browser }) => {

  // Create separate browser contexts
  const adminContext = await browser.newContext();

  const userContext = await browser.newContext();

  // Create pages
  const adminPage = await adminContext.newPage();

  const userPage = await userContext.newPage();

  // Admin login
  await adminPage.goto('https://example.com');

  // User login
  await userPage.goto('https://example.com');

  // Both users have separate sessions

  // Close contexts after test
  await adminContext.close();

  await userContext.close();

});
```

Understand step by step

1. `browser.newContext()` creates a new isolated session.
2. `adminContext` belongs to the Admin.
3. `userContext` belongs to the User.

4. `newPage()` opens a new tab inside each context.
5. Both users can log in independently.
6. Closing the contexts cleans up their browser sessions.

Real-world uses:

- Admin and customer login testing.
- Buyer and seller communication.
- Sender and receiver chat testing.
- Two-user collaboration applications.
- Different user permission testing.

7. Practical QA Automation Example – Verify User Isolation

Suppose you are testing an e-commerce website.

You want to verify that logging in as User A does not automatically log in User B.

```
import { test, expect } from '@playwright/test';

test('Verify separate user sessions', async ({ browser }) => {

  const userAContext = await browser.newContext();
  const userBContext = await browser.newContext();

  const userAPage = await userAContext.newPage();
  const userBPage = await userBContext.newPage();

  await userAPage.goto('https://example.com/login');

  await userBPage.goto('https://example.com/login');

  // Login User A
  await userAPage.getByLabel('Username').fill('userA');

  await userAPage.getByLabel('Password').fill('passwordA');
```

```
  await userAPage.getByRole('button', { name: 'Login' }).click();

  // User B should remain logged out
  await expect(
    userBPage.getByRole('button', { name: 'Login' })
  ).toBeVisible();

  await userAContext.close();
  await userBContext.close();

});
```

The important point is that User A's login cookies are not shared with User B.

Interview Questions and Answers

Basic questions

1. What is Test Isolation in Playwright?

Test Isolation means each test runs independently with its own browser context, cookies, local storage and session storage.

2. How does Playwright achieve test isolation?

Playwright uses BrowserContexts. It creates a fresh BrowserContext for each test when using the Playwright Test Runner.

3. What is BrowserContext?

BrowserContext is an isolated browser session similar to an Incognito window. It maintains its own cookies, storage and permissions.

4. What is the difference between Browser and BrowserContext?

Browser represents the actual browser instance, while BrowserContext represents an isolated session inside that browser.

5. What is a Page in Playwright?

A Page represents a browser tab. A BrowserContext can contain multiple pages.

Intermediate questions

6. Why is Test Isolation important?

It prevents one test's data or browser state from affecting another test. It improves reliability, debugging and parallel execution.

7. Does Playwright create a new browser for every test?

No. It generally reuses the browser instance and creates a new BrowserContext for each test.

8. Are cookies shared between two BrowserContexts?

No. Each BrowserContext maintains its own cookies and browser storage.

9. Can multiple BrowserContexts run in one browser?

Yes. We can create multiple isolated contexts using `browser.newContext()`.

10. What is the difference between cleanup and starting from scratch?

Starting from scratch creates a fresh environment for each test. Cleanup reuses an environment and removes previous test data before the next test.

Advanced questions

11. How do you test two users simultaneously in Playwright?

Create two BrowserContexts using `browser.newContext()`, create a Page in each context and log in with different user credentials.

12. Can two pages in the same BrowserContext share cookies?

Yes. Pages within the same BrowserContext share the context's cookies and storage.

13. Can tests run in parallel with isolation?

Yes. Each test gets its own isolated context, allowing tests to run independently in parallel.

14. What is the difference between `page` and `context` fixtures?

- `page` provides a browser tab.
- `context` provides the isolated browser session containing that tab.

15. When would you create multiple contexts manually?

When testing scenarios involving multiple independent users, such as admin and customer, buyer and seller, or sender and receiver.

Quick Revision Notes

Topic	Important point
Test Isolation	Tests run independently
BrowserContext	Isolated browser session
Page	Browser tab
`browser.newContext()`	Creates a new context
`context.newPage()`	Creates a page inside context
Cookies	Not shared between contexts
Default Playwright behavior	Fresh context per test
Multiple contexts	Useful for multi-user testing
Best strategy	Start from scratch

Test Your Knowledge

Isolation Quiz

5/5 answered

1. What does Playwright use to achieve test isolation?

Browser
BrowserContext
Locator

Correct! BrowserContext provides an isolated browser session.

2. Does Playwright create a new browser instance for every test?

Yes
No

Correct! It generally reuses the browser and creates a new context for each test.

3. Are cookies shared between two different BrowserContexts?

Yes
No

Correct! Each context maintains its own cookies.

4. Which method creates a new BrowserContext?

browser.newContext()
page.newPage()
context.newBrowser()

Correct! browser.newContext() creates a fresh isolated context.

5. When do we use multiple contexts in one test?

For CSS testing
For multi-user scenarios
For screenshots only

Correct! Multiple contexts allow independent users to interact with the same application.

Score: 5/5

Try again

Most important interview answer to remember:

"Playwright achieves test isolation using BrowserContexts. Each test gets a fresh BrowserContext with separate cookies, local storage and session storage. This prevents one test from affecting another and makes parallel execution more reliable."

Playwright – Locators

Very important for QA Automation / SDET interviews

Locators are one of the most important concepts in Playwright. You will use them in almost every automation test.

1. What are Locators?

In simple words, a Locator is a way to find an element on a webpage whenever you want to interact with it.

For example, suppose a login page contains:

- Username textbox
- Password textbox
- Sign In button

We need locators to identify these elements and perform actions such as filling, clicking and checking visibility.

```
await page.getByLabel('Username').fill('Komal');

await page.getByLabel('Password').fill('password123');

await page.getByRole('button', { name: 'Sign in' }).click();
```

Important: Locators are the central part of Playwright's auto-waiting and retry mechanism.

What does auto-waiting mean?

Suppose a button takes 3 seconds to appear.

In Selenium, you may need to write an explicit wait.

In Playwright, when you use a Locator action such as `click()`, Playwright automatically waits for the element to become ready for that action.

It checks conditions such as:

- Element is visible.
- Element is stable.
- Element is enabled.
- Element receives pointer events.

If these conditions are not satisfied, Playwright keeps retrying until the action succeeds or the timeout is reached.

What does retry-ability mean?

Suppose you write:

```
await expect(page.getByText('Login successful')).toBeVisible();
```

Playwright repeatedly checks whether the text becomes visible until the assertion passes or times out.

This is called a web-first assertion.

2. Types of Built-in Locators

Playwright provides several built-in locators.

Locator	Purpose	Example
`getByRole()`	Find by accessibility role and name	Button, textbox
`getByText()`	Find by visible text	Welcome
`getByLabel()`	Find form controls by label	Username
`getByPlaceholder()`	Find by placeholder	Enter email
`getByAltText()`	Find by image alternative text	Logo
`getByTitle()`	Find by title attribute	Issues count

`getByTestId()`	Find by test ID	data-testid
`locator()`	Find using CSS or XPath	#username

Let's understand each one with examples.

3. Locate by Role – `getByRole()`

This is one of the most recommended locators in Playwright.

It identifies elements based on their accessibility role and accessible name.

Example HTML:

```
<button>Sign in</button>
```

Playwright code:

```
await page.getByRole('button', { name: 'Sign in' }).click();
```

Explanation:

- `button` → Role of the element.
- `name: 'Sign in'` → Accessible name of the button.
- `click()` → Clicks the button.

More examples:

```
// Locate heading
await expect(
  page.getByRole('heading', { name: 'Sign up' })
).toBeVisible();

// Locate checkbox
await page.getByRole('checkbox', { name: 'Subscribe' }).check();

// Locate button
await page.getByRole('button', { name: /submit/i }).click();
```

Here `/submit/i` is a regular expression. The `i` means case-insensitive matching.

Common roles:

- button
- textbox
- checkbox
- radio
- link
- heading
- list
- listitem
- table
- img

When to use: Prefer `getByRole()` for interactive elements such as buttons, links and checkboxes.

4. Locate by Label – `getByLabel()`

Use this locator to find form fields using their associated label.

HTML:

```
<label>
  Password
  <input type="password" />
</label>
```

Playwright:

```
await page.getByLabel('Password').fill('secret123');
```

Explanation:

- Finds the input associated with the Password label.
- Enters the password.

Another example:

```
await page.getByLabel('Username').fill('Komal');
await page.getByLabel('Email').fill('komal@gmail.com');
```

When to use: When locating form controls that have associated labels.

5. Locate by Placeholder – `getByPlaceholder()`

A placeholder is the hint displayed inside an input field.

HTML:

```
<input
  type="email"
  placeholder="name@example.com"
/>
```

Playwright:

```
await page
  .getByPlaceholder('name@example.com')
  .fill('komal@gmail.com');
```

Explanation:

- Finds the input using its placeholder.
- Enters the email address.

When to use: When an input does not have a label but has a placeholder.

6. Locate by Text – `getByText()`

This locator finds elements based on their text content.

HTML:

```
<span>Welcome, John</span>
```

Playwright:

```
await expect(
  page.getByText('Welcome, John')
).toBeVisible();
```

You can use three types of matching.

1. Partial text matching

```
page.getByText('Welcome');
```

2. Exact text matching

```
page.getByText('Welcome, John', { exact: true });
```

3. Regular expression matching

```
page.getByText(/welcome, [A-Za-z]+$/i);
```

Important: Playwright normalizes whitespace when matching text. Multiple spaces and line breaks are normalized, and leading/trailing whitespace is ignored.

When to use: Best for non-interactive elements such as `div`, `span` and `p`.

For interactive elements, prefer role locators.

7. Locate by Alt Text – `getByAltText()`

Images usually have an `alt` attribute that describes the image.

HTML:

```
<img alt="Playwright logo" src="/logo.svg" />
```

Playwright:

```
await page.getByAltText('Playwright logo').click();
```

When to use: When locating images or other elements that support alternative text.

8. Locate by Title – `getByTitle()`

This locator identifies elements using their HTML `title` attribute.

HTML:

```
<span title="Issues count">25 issues</span>
```

Playwright:

```
await expect(
  page.getByTitle('Issues count')
).toHaveText('25 issues');
```

When to use: When the element has a title attribute.

9. Locate by Test ID – `getByTestId()`

Test ID is an attribute specifically added to HTML elements for automation testing.

HTML:

```
<button data-testid="login-button">
  Login
</button>
```

Playwright:

```
await page.getByTestId('login-button').click();
```

Why is this useful?

Suppose the developer changes the button text from `Login` to `Sign in`.

Your test can still work because the test ID remains unchanged.

When to use:

- When there is no reliable user-facing locator.
- When developers provide dedicated test IDs.
- When you want an explicit testing contract.

Configure a custom test ID

By default, Playwright uses `data-testid`.

You can change it to `data-pw`.

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
```

```
  use: {
    testIdAttribute: 'data-pw'
  }
});
```

HTML:

```
<button data-pw="login-button">Login</button>
```

Playwright:

```
await page.getByTestId('login-button').click();
```

The method remains `getByTestId()` even after changing the attribute.

10. Locate by CSS or XPath – `locator()`

Sometimes we need to identify elements using CSS selectors or XPath expressions.

Playwright supports both using `page.locator()`.

CSS selector

HTML:

```
<input id="username" />
```

Playwright:

```
await page.locator('#username').fill('Komal');
```

Other CSS examples:

```
// By ID
page.locator('#username');

// By class
page.locator('.login-button');

// By tag
page.locator('button');

// By attribute
page.locator('input[type="password"]');
```

XPath selector

```
await page.locator('//button').click();
```

You can also explicitly specify the selector type:

```
await page.locator('css=button').click();
```

```
await page.locator('xpath=//button').click();
```

Playwright can automatically detect CSS and XPath in common selector formats.

Interview point

Why does Playwright not recommend long CSS and XPath selectors?

Because they depend heavily on the DOM structure. If developers change the HTML structure, these selectors may break.

For example, this is not a good practice:

```
await page.locator(
  '#tsf > div:nth-child(2) > div > div > input'
).click();
```

Prefer:

```
await page.getByRole('button', { name: 'Sign in' }).click();
```

11. Locate Elements Inside Shadow DOM

First, understand Shadow DOM.

Shadow DOM is a web technology used to encapsulate HTML, CSS and functionality inside a web component.

For example, a webpage may contain a custom component with its own internal DOM.

Shadow DOM: Was ist das eigentlich & wie funktioniert es? - IONOS AT
ionos.at

Playwright's standard locators can automatically work with open Shadow DOM.

Example:

```
await page.getByText('Details').click();
```

Even if the `Details` element is inside an open shadow root, Playwright can locate it.

You can also locate the parent component:

```
await page
  .locator('x-details', { hasText: 'Details' })
  .click();
```

Important exceptions:

- XPath does not pierce Shadow DOM.
- Closed-mode Shadow DOM is not supported by Playwright locators.

Interview point: Playwright supports open Shadow DOM by default, but XPath cannot locate elements inside shadow roots.

12. Filtering Locators

This is a very important topic for real-world automation.

Suppose an e-commerce website displays two products:

Washing Machine isolated

Product 1

Washing machine

Available

Add to cart

Product Photography - Snap360 Photography

Product 2

Refrigerator

Available

Add to cart

Both products have the same Add to Cart button.

How do we click only the button belonging to Product 2?

We use `filter()`.

A. Filter by text – hasText

```
await page
  .getByRole('listitem')
  .filter({ hasText: 'Product 2' })
  .getByRole('button', { name: 'Add to cart' })
  .click();
```

Explanation:

1. Find all list items.
2. Filter the item containing Product 2.
3. Find the Add to Cart button inside that item.
4. Click it.

You can also use regular expressions:

```
await page
  .getByRole('listitem')
  .filter({ hasText: /Product 2/ })
  .getByRole('button', { name: 'Add to cart' })
  .click();
```

B. Filter by NOT having text – hasNotText

Suppose five products are in stock and we want to exclude products marked Out of Stock.

```
await expect(
  page.getByRole('listitem')
    .filter({ hasNotText: 'Out of stock' })
).toHaveCount(5);
```

This means: Find list items that do not contain the text Out of Stock and verify that there are five.

C. Filter by child or descendant – has

Suppose we want to find a product card containing a particular heading.

```
await page
  .getByRole('listitem')
  .filter({
    has: page.getByRole('heading', { name: 'Product 2' })
  })
  .getByRole('button', { name: 'Add to cart' })
  .click();
```

Here:

- has checks whether the outer element contains a matching inner element.
- The inner locator is evaluated relative to the outer locator.

We can verify that only one product card matches:

```
await expect(
  page.getByRole('listitem')
    .filter({
      has: page.getByRole('heading', { name: 'Product 2' })
    })
).toHaveCount(1);
```

D. Filter by NOT having a child – hasNot

```
await expect(
  page.getByRole('listitem')
    .filter({
      hasNot: page.getByText('Product 2')
    })
).toHaveCount(1);
```

This selects list items that do not contain Product 2.

Important: The inner locator must be relative to the outer locator. It should not try to locate an element outside the current outer element.

13. Locator Operators

Locator operators help us combine or narrow down locators.

A. Chaining locators

We can create a locator and then use it to find another element inside it.

```
const product = page
  .getByRole('listitem')
  .filter({ hasText: 'Product 2' });

await product
  .getByRole('button', { name: 'Add to cart' })
  .click();

await expect(product).toHaveCount(1);
```

This is called locator chaining.

B. and() – Match both conditions

Suppose we want a button that:

- Has the role button.
- Has the title Subscribe.

```
const button = page
  .getByRole('button')
  .and(page.getByTitle('Subscribe'));

await button.click();
```

and() means both locators must match the same element.

C. or() – Match either condition

Suppose sometimes a New Email button appears, but sometimes a security dialog appears.

```
const newEmail = page.getByRole('button', {
  name: 'New'
});

const dialog = page.getByText('Confirm security settings');

await expect(
  newEmail.or(dialog).first()
).toBeVisible();

if (await dialog.isVisible()) {
```

```
  await page.getByRole('button', {
    name: 'Dismiss'
  }).click();
}

await newEmail.click();
```

Explanation:

- `or()` matches either locator.
- `first()` selects the first matching element.
- We check whether the dialog is visible.
- If visible, we dismiss it.
- Then we click New Email.

Important: If both elements match, `or()` can produce multiple matches.
Using `.first()` avoids a strictness error in this example, but you should still ensure the correct element is selected.

D. `first()`, `last()` and `nth()`

These methods select an element based on its position.

```
// First matching element
await page.getByRole('button').first().click();

// Last matching element
await page.getByRole('button').last().click();

// Second matching element
await page.getByRole('button').nth(1).click();
```

Remember: Indexing starts from zero.

Method	Selects
`first()`	First element

`last()`	Last element
`nth(0)`	First element
`nth(1)`	Second element
`nth(2)`	Third element

Use these methods carefully because the order of elements may change.

E. Matching visible elements

Suppose the page contains two buttons:

```
<button style="display:none">Invisible</button>
<button>Visible</button>
```

The following locator matches both buttons:

```
page.locator('button');
```

If your goal is to select only the visible button, you can use the visibility filter:

```
await page.locator('button')
  .filter({ visible: true })
  .click();
```

To select hidden elements:

```
page.locator('button')
  .filter({ visible: false });
```

Note: The documentation snippet uses `.visible()`; the current documented Locator API uses the `visible` option in `filter()`. Prefer the filter syntax above.

14. Working with Lists

Suppose a webpage contains:

```
<ul>
  <li>apple</li>
```

```
  <li>banana</li>
  <li>orange</li>
</ul>
```

Let's understand how to work with lists.

A. Count list items

```
await expect(
  page.getByRole('listitem')
).toHaveCount(3);
```

Checks whether the list contains exactly three items.

B. Verify all list texts

```
await expect(
  page.getByRole('listitem')
).toHaveText([
  'apple',
  'banana',
  'orange'
]);
```

Checks the text of all three list items in order.

C. Get a specific item by text

```
await page.getByText('orange').click();
```

D. Get an item using filter

```
await page
  .getByRole('listitem')
  .filter({ hasText: 'orange' })
  .click();
```

E. Get an item by Test ID

HTML:

```
<li data-testid="orange">orange</li>
```

Playwright:

```
await page.getByTestId('orange').click();
```

F. Get an item using nth()

```
await page.getByRole('listitem').nth(1).click();
```

This clicks banana because the index starts at zero.

Interview tip: Prefer locating an item by its unique text or test ID rather than using `nth()`, whenever possible.

15. Chaining Multiple Filters

This is useful when multiple records have similar information.

Suppose a list contains:

Name	Action
John	Say hello
Mary	Say hello
John	Say goodbye
Mary	Say goodbye

We want the row containing Mary and the Say goodbye button.

```
const rowLocator = page.getByRole('listitem');

await rowLocator
  .filter({ hasText: 'Mary' })
  .filter({
    has: page.getByRole('button', {
      name: 'Say goodbye'
    })
  })
  .screenshot({ path: 'screenshot.png' });
```

Explanation:

1. Find all list items.
2. Filter items containing Mary.
3. From those, select the item containing Say goodbye.
4. Take a screenshot of that row.

You can combine multiple filters to find a very specific element.

16. Iterating Through List Elements

Sometimes we need to read every item in a list.

Method 1: Using `all()`

```
const rows = await page.getByRole('listitem').all();

for (const row of rows) {
  console.log(await row.textContent());
}
```

Method 2: Using a normal for loop

```
const rows = page.getByRole('listitem');

const count = await rows.count();

for (let i = 0; i < count; i++) {
  console.log(await rows.nth(i).textContent());
}
```

Method 3: Using `evaluateAll()`

```
const rows = page.getByRole('listitem');

const texts = await rows.evaluateAll(
  elements => elements.map(element => element.textContent)
);

console.log(texts);
```

What is `evaluateAll()`?

It runs JavaScript in the browser against all elements matched by the locator and returns the result.

Important: `all()` does not wait for elements to appear. If a list is still loading, first wait for a reliable condition or assertion before collecting its elements.

17. Strictness in Playwright

Frequently asked interview question

Locators in Playwright are strict.

Strictness means: If an action expects one element but the locator matches multiple elements, Playwright throws a strict mode violation error.

Example HTML:

```
<button>Login</button>
<button>Submit</button>
<button>Cancel</button>
```

This code will throw an error:

```
await page.getByRole('button').click();
```

Why?

Because three buttons match, and Playwright doesn't know which one you want to click.

How to solve this?

Solution 1: Use a unique role and name

```
await page.getByRole('button', {
  name: 'Login'
}).click();
```

Solution 2: Use first()

```
await page.getByRole('button').first().click();
```

Solution 3: Use last()

```
await page.getByRole('button').last().click();
```

Solution 4: Use nth()

```
await page.getByRole('button').nth(1).click();
```

Remember: `nth(1)` selects the second element.

Does strictness apply to count()?

No. Some operations are designed to work with multiple elements.

```
const count = await page.getByRole('button').count();

console.log(count);
```

This works even if there are multiple buttons.

Other examples of multiple-element operations include `all()` and `evaluateAll()`.

Best practice: Always try to create a unique locator rather than immediately using `first()`, `last()` or `nth()`.

18. Complete Practical Example – E-Commerce Website

Let's combine multiple locator concepts in one test.

Suppose an e-commerce website contains:

- Search textbox
- Search button
- Product list
- Add to Cart buttons
- Cart confirmation message

```
import { test, expect } from '@playwright/test';

test('Search product and add to cart', async ({ page }) => {
```

```
  await page.goto('https://example.com');

  // Search for a product
  await page.getByPlaceholder('Search products')
    .fill('Washing Machine');

  await page.getByRole('button', { name: 'Search' })
    .click();

  // Verify product is visible
  await expect(
    page.getByText('Washing Machine')
  ).toBeVisible();

  // Find the required product card
  const product = page.getByRole('listitem')
    .filter({ hasText: 'Washing Machine' });

  // Click Add to Cart inside that product
  await product.getByRole('button', {
    name: 'Add to cart'
  }).click();

  // Verify confirmation
  await expect(
    page.getByText('Added to cart')
  ).toBeVisible();

});
```

This example uses:

1. `getByPlaceholder()` – Search input.
2. `getByRole()` – Search and Add to Cart buttons.
3. `getByText()` – Verify product and confirmation.
4. `filter()` – Find the correct product card.
5. Locator chaining – Find the button inside the product.
6. `expect()` – Verify the expected result.

This is the kind of locator usage you should practise for QA Automation interviews.

Interview Questions and Answers

Basic questions

1. What is a Locator in Playwright?

A Locator is a way to find elements on a webpage. It supports auto-waiting and retry-ability, making tests more reliable.

2. What are the built-in locators in Playwright?

The commonly used built-in locators are:

- `getByRole()`
- `getByText()`
- `getByLabel()`
- `getByPlaceholder()`
- `getByAltText()`
- `getByTitle()`
- `getByTestId()`

3. Which locator is recommended in Playwright?

`getByRole()` is generally preferred because it identifies elements based on how users and assistive technologies perceive them.

4. What is the difference between `getByText()` and `getByRole()`?

`getByText()` finds elements using their text content. `getByRole()` finds elements using their accessibility role and accessible name.

5. What is `getByLabel()` used for?

It locates form controls using their associated label text, such as Username and Password fields.

6. What is `getByPlaceholder()`?

It locates input elements using their placeholder attribute.

Intermediate questions

7. What is the difference between Locator and ElementHandle?

Locator stores the logic to find an element and resolves it when needed. ElementHandle refers to a particular DOM element. Locator is preferred because it handles DOM updates more reliably.

8. What is auto-waiting in Playwright?

Playwright automatically waits for an element to become actionable before performing actions such as click and fill.

9. What is retry-ability?

Playwright automatically retries web-first assertions until the expected condition becomes true or the timeout is reached.

10. What is the difference between CSS and XPath?

CSS uses CSS selector syntax to locate elements, while XPath uses XML path expressions to navigate the DOM.

11. What is `filter()` in Playwright?

`filter()` narrows down a locator based on conditions such as text or the presence or absence of a child element.

12. What is the difference between `hasText` and `has`?

- `hasText` filters an element based on text content.
- `has` filters an element based on whether it contains a matching descendant element.

13. What is `nth()`?

It selects an element based on its zero-based index.

```
page.getByRole('button').nth(1);
```

This selects the second button.

14. What is strictness in Playwright?

Strictness means an action expecting a single element throws an error if multiple elements match the locator.

Advanced questions

15. What is the difference between `and()` and `or()`?

- `and()` matches an element satisfying both locator conditions.
- `or()` matches elements satisfying either locator condition.

16. Can Playwright locate elements inside Shadow DOM?

Yes. Playwright locators support open Shadow DOM by default. XPath and closed-mode Shadow DOM are exceptions.

17. What is `getByTestId()`?

It locates elements using a dedicated testing attribute, which is `data-testid` by default.

18. How can we configure a custom Test ID attribute?

Using `testIdAttribute` in `playwright.config.ts`.

```
use: {
  testIdAttribute: 'data-pw'
}
```

19. What is `evaluateAll()`?

It executes JavaScript in the browser against all elements matched by a locator and returns the result.

20. How do you locate the Add to Cart button of a specific product when multiple products have the same button?

Use `filter()` to identify the product card and then chain a button locator inside it.

```
await page.getByRole('listitem')
  .filter({ hasText: 'Product 2' })
```

```
  .getByRole('button', { name: 'Add to cart' })
  .click();
```

Quick Revision Cheat Sheet

Method	Remember
`getByRole()`	Role + accessible name
`getByText()`	Text content
`getByLabel()`	Form label
`getByPlaceholder()`	Input placeholder
`getByAltText()`	Image alt text
`getByTitle()`	Title attribute
`getByTestId()`	Testing attribute
`locator()`	CSS / XPath
`filter()`	Narrow down elements
`hasText`	Contains particular text
`hasNotText`	Does not contain text
`has`	Contains matching child
`hasNot`	Does not contain matching child

`and()`	Both conditions
`or()`	Either condition
`first()`	First matching element
`last()`	Last matching element
`nth()`	Element by index
`count()`	Number of matches
`evaluateAll()`	Run JS on all matched elements

Test Your Knowledge – Locator Quiz

Locator Interview Quiz

8/8 answered

1. Which locator is generally preferred for buttons?

getByText()
getByRole()
getByTitle()

Correct! getByRole() identifies a button by its accessibility role and name.

2. Which locator is used for a field with placeholder text?

getByPlaceholder()
getByAltText()
getByTestId()

Correct! getByPlaceholder() locates an input using its placeholder.

3. What does nth(1) select?

First element

Second element
Third element

Correct! Locator indexing starts at zero.

4. Which method filters a product card based on text?

and()
filter({ hasText: 'Product 2' })
dispose()

Correct! hasText filters matching elements based on their text content.

5. What happens if click() matches multiple buttons?

Clicks all buttons
Clicks the first button
Throws a strict mode violation

Correct! An action expecting one element throws when multiple elements match.

6. What does and() do?

Matches either locator
Matches both conditions on the same element
Counts elements

Correct! and() combines two locators so the same element must satisfy both.

7. Does Playwright support open Shadow DOM?

Yes
No

Correct! Playwright's regular locators pierce open Shadow DOM by default.

8. Which method counts matching elements?

count()
nth()
filter()

Correct! count() returns the number of elements matching the locator.

8/8

Your score

Try again

My interview preparation advice: Practise these five topics particularly well:

1. `getByRole()` vs `getByText()`
2. Locator vs ElementHandle
3. `filter()` with `hasText` and `has`
4. `and()` vs `or()`
5. Strictness and auto-waiting

These concepts are especially important when explaining how you build stable Playwright automation frameworks.

Playwright – Mock APIs

Important for QA Automation / SDET interviews

API mocking is a very useful concept in Playwright. It helps us test frontend applications without depending completely on the actual backend APIs.

Let's understand it in simple language with practical examples.

1. What is API Mocking?

Normally, when you open a website, the frontend sends an API request to the backend server to get data.

For example, imagine a shopping website:

1. You open the product page.
2. Frontend sends a request to the backend.
3. Backend returns product details.

4. Frontend displays the products.

But what happens if the backend is unavailable?

Your UI test may fail even if the frontend is working correctly.

API Mocking means intercepting a real API request and providing our own predefined response instead of depending on the actual API response.

Normal API flow

Frontend / Browser

HTTP API request and response

Backend API / Server

Mock API flow

Frontend / Browser

API request

Playwright – Intercepts request

Mock response

Returns predefined data

Mock JSON response

Frontend displays data

The actual backend API is not called in this flow.

Why do we use API mocking?

- Test frontend independently of backend.
- Test when the backend is unavailable.
- Test different API responses.
- Test error scenarios such as 404 and 500.
- Avoid depending on changing production or test data.

- Make tests more predictable and repeatable.

2. Mock API Requests

Playwright provides `page.route()` to intercept network requests.

Let's understand the documentation example.

```
import { test, expect } from '@playwright/test';

test("mocks a fruit and doesn't call api", async ({ page }) => {

  // Intercept API request
  await page.route('*/**/api/v1/fruits', async route => {

    // Custom mock data
    const json = [
      { name: 'Strawberry', id: 21 }
    ];

    // Return mock response
    await route.fulfill({ json });

  });

  // Open webpage
  await page.goto('https://demo.playwright.dev/api-mocking');

  // Verify mock data
  await expect(
    page.getByText('Strawberry')
  ).toBeVisible();

});
```

Explanation line by line

Step 1: `page.route()`

```
await page.route('*/**/api/v1/fruits', async route => {
```

This intercepts requests matching the URL pattern.

Here:

- `page.route()` → Intercepts network requests.
- URL pattern → Identifies which API request to intercept.
- `route` → Gives us control over the intercepted request.

Step 2: Create mock data

```
const json = [
  { name: 'Strawberry', id: 21 }
];
```

We create our own JSON response.

Step 3: `route.fulfill()`

```
await route.fulfill({ json });
```

This sends our custom response back to the webpage.

The original API server is not contacted for this intercepted request.

Step 4: Verify the result

```
await expect(
  page.getByText('Strawberry')
).toBeVisible();
```

We verify that the frontend displays the mocked Strawberry data.

Remember
`page.route()` intercepts the request, and `route.fulfill()` provides the response.

3. Important Methods Used in API Mocking

Method	Meaning
`page.route()`	Intercepts network requests
`route.fulfill()`	Returns a custom response

`route.fetch()`	Sends the original request to the server
`route.continue()`	Allows the original request to continue
`route.abort()`	Blocks the request
`page.unroute()`	Removes a registered route

Let's understand the three most important methods.

route.fulfill()

Returns a custom response without calling the actual API.

```
await route.fulfill({
  status: 200,
  contentType: 'application/json',
  body: JSON.stringify({
    message: 'Success'
  })
});
```

route.continue()

Allows the intercepted request to continue to the actual server.

```
await page.route('**/api/products', async route => {
  await route.continue();
});
```

route.abort()

Blocks the request.

```
await page.route('**/api/products', async route => {
  await route.abort();
});
```

Useful for testing network failure scenarios.

4. Modify API Responses

Sometimes we don't want to completely replace the API response.

Instead, we want to:

1. Call the actual API.
2. Receive its original response.
3. Modify some data.
4. Return the modified response to the frontend.

For this, Playwright provides `route.fetch()` and `route.fulfill()`.

Documentation example

```
import { test, expect } from '@playwright/test';

test('gets json from api and adds a new fruit', async ({ page }) =>
{

  await page.route('*/**/api/v1/fruits', async route => {

    // Call actual API
    const response = await route.fetch();

    // Get original JSON
    const json = await response.json();

    // Add new fruit
    json.push({ name: 'Loquat', id: 100 });

    // Return modified response
    await route.fulfill({
      response,
      json
    });

  });

  await page.goto('https://demo.playwright.dev/api-mocking');
```

```
  // Verify modified response
  await expect(
    page.getByText('Loquat', { exact: true })
  ).toBeVisible();

});
```

Understand the flow

Frontend sends API request

Playwright intercepts request

route.fetch()

Calls actual backend API

Modify JSON

Add Loquat fruit

route.fulfill()

Return modified response

Frontend displays updated data

Important difference

Mock API	Modify API response
`route.fulfill()`	`route.fetch()` + `route.fulfill()`
Does not call original API	Calls original API
Returns completely custom data	Modifies original data
Useful for independent frontend testing	Useful for testing with realistic API data

Interview question: What is the difference between `route.fulfill()` and `route.fetch()`?

Answer: `route.fulfill()` provides a response directly, whereas `route.fetch()` sends the actual request to the server and returns its response so we can inspect or modify it.

5. Mocking with HAR Files

Important advanced concept

First, understand what a HAR file is.

HAR stands for HTTP Archive.

A HAR file stores information about network requests and responses made by a webpage.

It can contain:

- Request URL
- HTTP method (GET, POST, PUT, DELETE)
- Request headers
- Request body
- Response status
- Response headers
- Response body
- Timing information

Think of a HAR file as a recorded history of the API communication between the browser and server.

Why do we use HAR files?

Suppose you are testing an e-commerce application.

The application calls multiple APIs:

- Get products
- Get product details
- Get cart
- Get user profile

Instead of writing separate mock responses for every API, you can record the network traffic in a HAR file and replay it during testing.

Three steps to use HAR files

1. Record a HAR file

Capture actual network requests and responses.

2. Modify the HAR file if required

Update response data to create the scenario you want to test.

3. Replay the HAR file

Playwright uses the saved responses instead of calling the actual server.

Step 1: Record a HAR file

Playwright provides `page.routeFromHAR()` and `browserContext.routeFromHAR()`.

You can use the `update: true` option to record or update network information.

Example:

```
import { test, expect } from '@playwright/test';

test('Record API responses in HAR', async ({ page }) => {

  await page.routeFromHAR('./hars/fruit.har', {
    url: '**/api/v1/fruits',
    update: true
  });

  await page.goto('https://demo.playwright.dev/api-mocking');

});
```

Explanation:

- `routeFromHAR()` configures HAR-based network handling.
- `./hars/fruit.har` is the HAR file path.
- `url` limits recording to matching requests.
- `update: true` allows the actual network traffic to be used and the HAR file to be created or updated.

The updated HAR data is written when the associated browser context closes. Playwright Test normally handles context cleanup at the end of the test.

Step 2: Modify the HAR file

After recording, you can edit the saved HAR data to change the response.

For example, add a fruit:

```
[
  {
    "name": "Playwright",
    "id": 100
  },
  {
    "name": "Strawberry",
    "id": 21
  }
]
```

Now the saved response contains Playwright and Strawberry.

Step 3: Replay API requests from HAR

Once the HAR file is ready, remove the update option or set it to `false`.

```
import { test, expect } from '@playwright/test';

test('Replay API response from HAR', async ({ page }) => {

  await page.routeFromHAR('./hars/fruit.har', {
    url: '**/api/v1/fruits',
    update: false
```

```
  });

  await page.goto('https://demo.playwright.dev/api-mocking');

  await expect(
    page.getByText('Playwright', { exact: true })
  ).toBeVisible();

});
```

What happens here?

1. Playwright intercepts the API request.
2. It searches for a matching request in the HAR file.
3. It returns the recorded response.
4. The actual backend API is not called for that matched request.
5. The webpage displays the saved data.

HAR recording vs HAR replay

Recording	Replaying
`update: true`	`update: false`
Uses actual network	Uses saved HAR responses
Creates or updates HAR	Reads existing HAR
Useful for capturing API traffic	Useful for repeatable tests

Important HAR matching rules

Playwright matches HAR entries using:

- Request URL
- HTTP method

- POST request payload, when applicable
- Matching request headers to select the most appropriate entry when multiple entries match

If a matching HAR entry results in a redirect, Playwright follows the redirect automatically.

Interview tip: HAR mocking is useful when an application makes multiple API requests and we want to replay previously recorded network responses consistently.

6. Recording HAR Using Playwright CLI

We can also record HAR files using the Playwright command line.

Example:

```
npx playwright open --save-har=example.har --save-har-glob="**/api/**" https://example.com
```

Explanation:

- `npx playwright open` → Opens the browser.
- `--save-har=example.har` → Saves network traffic in a HAR archive.
- `--save-har-glob="**/api/**"` → Records only requests matching the API URL pattern.
- `https://example.com` → Opens the website.

You can interact with the website, and Playwright records the matching network traffic.

Remember: The CLI approach is another way to capture HAR data. You can also use `routeFromHAR()` with `update: true` in your tests.

7. Mock WebSockets

First, understand what a WebSocket is.

Normally, HTTP works like this:

- Client sends a request.

- Server sends a response.
- The communication is completed.

But WebSocket provides two-way communication between client and server over a persistent connection.

For example:

- Live chat applications.
- Real-time notifications.
- Stock market updates.
- Live sports scores.
- Online multiplayer games.

Understanding WebSocket Connection (Simple Explanation) - DEV Community
dev.to

Playwright provides:

`page.routeWebSocket()`

This allows us to intercept and mock WebSocket communication.

A. Mock the entire WebSocket connection

Suppose the frontend sends a message:

`request`

We want Playwright to return:

`response`

Code:

```
await page.routeWebSocket('wss://example.com/ws', ws => {

  ws.onMessage(message => {

    if (message === 'request') {
      ws.send('response');
```

```
    }

  });

});
```

Explanation:

- `routeWebSocket()` → Intercepts the WebSocket connection.
- `ws.onMessage()` → Listens for messages received from the page.
- `message` → The message sent by the frontend.
- `ws.send()` → Sends a message back to the frontend.

In this example, the actual WebSocket server is not contacted.

B. Connect to the actual WebSocket server and modify messages

Sometimes we don't want to mock the entire connection.

We want to connect to the real server but modify messages before sending them.

Example from the documentation:

```
await page.routeWebSocket('wss://example.com/ws', ws => {

  // Connect to actual server
  const server = ws.connectToServer();

  ws.onMessage(message => {

    if (message === 'request') {

      // Modify the message
      server.send('request2');

    } else {

      // Forward other messages unchanged
      server.send(message);

    }
```

```
  });

});
```

Understand the difference:

Full WebSocket mock	Modify WebSocket messages
Does not connect to actual server	Connects to actual server
Playwright sends mock responses	Playwright forwards messages
Uses `ws.send()`	Uses `server.send()`
Useful for independent testing	Useful for testing message modification

Important: Register the WebSocket route before the page establishes its WebSocket connection.

8. Real-world QA Automation Examples

Let's understand where you can use API mocking in your automation projects.

Scenario 1: Test product listing without backend dependency

Suppose the product API is unavailable, but you want to test the product listing UI.

```
import { test, expect } from '@playwright/test';

test('Verify mocked products', async ({ page }) => {

  await page.route('**/api/products', async route => {

    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify([
        { id: 1, name: 'Laptop', price: 50000 },
```

```
        { id: 2, name: 'Mobile', price: 20000 }
      ])
    });

  });

  await page.goto('https://example.com/products');

  await expect(page.getByText('Laptop')).toBeVisible();

  await expect(page.getByText('Mobile')).toBeVisible();

});
```

Scenario 2: Test API error – 500 Internal Server Error

Suppose the backend returns an internal server error.

We want to verify whether the frontend displays an appropriate error message.

```
test('Verify server error message', async ({ page }) => {

  await page.route('**/api/products', async route => {

    await route.fulfill({
      status: 500,
      contentType: 'application/json',
      body: JSON.stringify({
        message: 'Internal Server Error'
      })
    });

  });

  await page.goto('https://example.com/products');

  await expect(
    page.getByText('Something went wrong')
  ).toBeVisible();

});
```

Scenario 3: Test empty API response

```
test('Verify empty product list', async ({ page }) => {

  await page.route('**/api/products', async route => {

    await route.fulfill({
      status: 200,
      json: []
    });

  });

  await page.goto('https://example.com/products');

  await expect(
    page.getByText('No products found')
  ).toBeVisible();

});
```

These examples show how API mocking helps test different frontend scenarios without changing the actual backend data.

9. Important Differences – Interview Preparation

Concept	Purpose
`page.route()`	Intercept HTTP requests
`route.fulfill()`	Return custom response
`route.fetch()`	Call actual API
`route.continue()`	Let original request continue

`route.abort()`	Block request
`routeFromHAR()`	Replay recorded API responses
`routeWebSocket()`	Intercept WebSocket communication
`ws.send()`	Send a WebSocket message
`ws.connectToServer()`	Connect to actual WebSocket server

10. Interview Questions and Answers

Basic questions

1. What is API mocking in Playwright?

API mocking is the process of intercepting API requests and returning predefined responses instead of depending on the actual backend API.

2. Why do we use API mocking?

We use API mocking to test frontend functionality independently, simulate different responses, test error scenarios and reduce backend dependencies.

3. Which method is used to intercept API requests in Playwright?

`page.route()`.

4. Which method is used to return a custom API response?

`route.fulfill()`.

5. Does the actual API get called when using `route.fulfill()` with mock data?

No. Playwright returns the custom response directly for that intercepted request.

6. What is a HAR file?

HAR stands for HTTP Archive. It stores recorded network requests and responses, which can be replayed during testing.

Intermediate questions

7. What is the difference between `route.fulfill()` and `route.fetch()`?

`route.fulfill()` returns a custom response, whereas `route.fetch()` sends the original request to the actual server and retrieves its response.

8. How do you modify an API response in Playwright?

Use `route.fetch()` to get the original response, modify its JSON data and use `route.fulfill()` to return the modified response.

9. How do you mock a 500 error response?

```
await page.route('**/api/products', async route => {
  await route.fulfill({
    status: 500,
    body: 'Server Error'
  });
});
```

10. What is the difference between `route.abort()` and `route.continue()`?

- `route.abort()` blocks the request.
- `route.continue()` sends the request to the actual server.

11. How do you replay API requests from a HAR file?

Use `page.routeFromHAR()` with `update: false`.

```
await page.routeFromHAR('./hars/fruit.har', {
  update: false
});
```

12. What is the difference between HAR recording and replaying?

Recording uses `update: true` to create or update the HAR file using actual network traffic. Replaying uses `update: false` to serve saved responses.

Advanced questions

13. Can Playwright mock multiple API requests?

Yes. We can register multiple `page.route()` handlers for different API URL patterns or use a HAR file to replay multiple recorded requests.

14. Can we mock API requests before page navigation?

Yes. It is recommended to register the route before `page.goto()` so that requests made during page loading are intercepted.

15. What is WebSocket mocking?

WebSocket mocking intercepts WebSocket communication and allows us to simulate messages between the frontend and server without depending on the actual server.

16. Which method is used to intercept WebSocket connections?

`page.routeWebSocket()`.

17. What is the difference between HTTP and WebSocket communication?

HTTP generally follows a request-response model, whereas WebSocket supports persistent, two-way communication between client and server.

18. Can we modify the actual API response without completely mocking it?

Yes. Use `route.fetch()` to get the original response, modify the response data and return it using `route.fulfill()`.

19. What is the advantage of HAR mocking over manually mocking every API?

HAR mocking allows us to record and replay multiple network requests and their responses from a saved archive, reducing the need to define each response manually.

20. Can API mocking be used to test negative scenarios?

Yes. We can simulate 400, 401, 403, 404 and 500 responses, empty data, delayed responses and network failures.

Quick Revision Notes

Topic	Remember
API mocking	Return predefined API data
`page.route()`	Intercept requests
`route.fulfill()`	Custom response
`route.fetch()`	Actual API request
`route.continue()`	Continue original request
`route.abort()`	Block request
Response modification	Fetch → Modify → Fulfill
HAR	Recorded network traffic
`update: true`	Record/update HAR
`update: false`	Replay HAR
WebSocket	Two-way communication
`routeWebSocket()`	Intercept WebSocket

Test Your Knowledge – API Mocking Quiz

API Mocking Interview Quiz

8/8 answered

1. Which method intercepts HTTP API requests?

page.route()
page.locator()
page.evaluate()

Correct! page.route() intercepts matching network requests.

2. Which method returns a custom mock response?

route.fetch()
route.fulfill()
route.abort()

Correct! route.fulfill() provides the response to the browser.

3. Does route.fetch() call the actual API?

Yes
No

Correct! route.fetch() sends the request to the actual server.

4. What does HAR stand for?

HTTP API Response
HTTP Archive
HTML Archive Request

Incorrect. HAR stands for HTTP Archive.

5. Which option is used to replay a HAR file?

update: true
update: false
update: null

Incorrect. update:false serves saved responses from the HAR.

6. Which method blocks an API request?

route.continue()
route.fulfill()
route.abort()

Correct! route.abort() cancels the request.

7. Which method intercepts WebSocket connections?

page.routeWebSocket()
page.route()
page.waitForEvent()

Correct! routeWebSocket() intercepts WebSocket connections.

8. What is WebSocket mainly used for?

Only static HTML
Two-way real-time communication
Only database storage

Correct! WebSocket supports persistent two-way communication.

6/8

Your score

Try again

Most important interview answer

If an interviewer asks, "How have you used API mocking in Playwright?", you can answer:

"I use Playwright's `page.route()` method to intercept API requests and `route.fulfill()` to return custom JSON responses. This helps me test frontend functionality independently of the backend. I can also use `route.fetch()` to get the actual API response, modify its data and return it using `route.fulfill()`. For multiple recorded API requests, I can use HAR files to replay responses."

My recommendation: Practise the three most important coding scenarios: mock a successful API response, mock a 500 error, and modify an actual API response. These are particularly useful for SDET interviews.

Playwright – Mock Browser APIs

1. What are Mock Browser APIs?

Sometimes a web application uses browser features that Playwright does not directly support or that are not supported by all browsers.

In such situations, we can mock browser APIs.

Mocking means creating a fake version of a real browser API to control its behavior during testing.

Instead of depending on the actual browser or device, we provide our own test data.

Real-world example

Suppose your application displays the laptop's battery status:

- Battery percentage: 75%
- Charging: Yes
- Remaining charging time: 30 minutes

We don't want to depend on the actual laptop battery during automation testing.

So, we create a fake Battery API that always returns the values we specify.

Why do we use mocking?

- To test browser features that are difficult to control.
- To get predictable test results.
- To test different scenarios without depending on real devices.
- To test edge cases easily.

2. Creating Mocks using `page.addInitScript()`

The most important method in this section is:

```
page.addInitScript()
```

It executes JavaScript in the browser page before the page's own scripts start running.

Why is this important?

Suppose your application calls the Battery API immediately when the page loads. If we create the mock after loading the page, the application may already have called the real API.

Therefore, we must configure the mock before navigation.

Example

```
await page.addInitScript(() => {
  const mockBattery = {
    level: 0.75,
    charging: true,
    chargingTime: 1800,
    dischargingTime: Infinity,
    addEventListener: () => {}
  };

  window.navigator.getBattery = async () => mockBattery;
});
```

Explanation line by line

Code	Meaning
`page.addInitScript()`	Runs code before page scripts.
`mockBattery`	Creates fake battery information.
`level: 0.75`	Battery level is 75%.
`charging: true`	Device is charging.
`chargingTime: 1800`	1800 seconds = 30 minutes.

`addEventListener`	Provides a method for registering event listeners.
`getBattery`	Overrides the browser's Battery API to return our fake data.

Remember: `0.75` means 75%, not 0.75%.

Using it in a Playwright test

```
import { test, expect } from '@playwright/test';

test.beforeEach(async ({ page }) => {
  await page.addInitScript(() => {
    const mockBattery = {
      level: 0.90,
      charging: true,
      chargingTime: 1800,
      dischargingTime: Infinity,
      addEventListener: () => {}
    };

    window.navigator.getBattery = async () => mockBattery;
  });
});

test('show battery status', async ({ page }) => {
  await page.goto('/');

  await expect(page.locator('.battery-percentage'))
    .toHaveText('90%');

  await expect(page.locator('.battery-status'))
    .toHaveText('Adapter');

  await expect(page.locator('.battery-fully'))
    .toHaveText('00:30');
});
```

What is `beforeEach()` doing?

It sets up the mock before every test, so every test starts with the same battery information.

3. Mocking Read-Only APIs

Some browser properties are read-only.

For example:

```
navigator.cookieEnabled = true;
```

This assignment will not change the actual property value because it is read-only.

But sometimes we can override a property using:

```
Object.defineProperty()
```

Example

```
await page.addInitScript(() => {
  Object.defineProperty(
    Object.getPrototypeOf(navigator),
    'cookieEnabled',
    {
      value: false
    }
  );
});
```

Explanation

- `Object.getPrototypeOf(navigator)` gets the prototype of the navigator object.
- `'cookieEnabled'` identifies the property we want to override.
- `value: false` sets the mocked value to false.

Now, when the application checks:

```
navigator.cookieEnabled
```

It receives `false`.

Important: This works when the property can be redefined. Not every browser property is configurable.

4. Verifying API Calls

Sometimes we don't just want to verify the UI. We also want to check whether the application called the expected browser API methods.

For example, we want to verify that the application:

1. Called `getBattery()`.
2. Registered a `chargingchange` event listener.
3. Registered a `levelchange` event listener.

We can use:

`page.exposeFunction()`

What is `page.exposeFunction()`?

It exposes a Node.js function to the browser page.

This allows JavaScript running inside the browser to communicate with our Playwright test code.

Example

```
test('log battery calls', async ({ page }) => {
  const log = [];

  await page.exposeFunction('logCall', msg => log.push(msg));

  await page.addInitScript(() => {
    const mockBattery = {
      level: 0.75,
      charging: true,
      chargingTime: 1800,
      dischargingTime: Infinity,

      addEventListener: (name, cb) => {
        logCall(`addEventListener:${name}`);
```

```
      }
    };

    window.navigator.getBattery = async () => {
      logCall('getBattery');
      return mockBattery;
    };
  });

  await page.goto('/');

  await expect(page.locator('.battery-percentage'))
    .toHaveText('75%');

  expect(log).toEqual([
    'getBattery',
    'addEventListener:chargingchange',
    'addEventListener:levelchange'
  ]);
});
```

Understand the execution

Playwright Test (Node.js)

Creates `log` array and exposes `logCall()`.

Browser Page

Calls `getBattery()` and registers event listeners.

Node.js Test

Receives the messages and stores them in `log`.

Assertion

Compares actual API calls with expected calls using `expect(log).toEqual()`.

Here, `logCall()` is available inside the browser because we exposed it using `page.exposeFunction()`.

What is a golden result?

A golden result is the expected output or expected result that we compare with the actual result.

In this example, the expected API calls are:

```
[
  'getBattery',
  'addEventListener:chargingchange',
  'addEventListener:levelchange'
]
```

5. Updating a Mock

Sometimes we need to test how the UI behaves when browser API values change.

For example:

- Initially, battery is 10%.
- Later, battery becomes 27.5%.
- Initially, device is not charging.
- Later, the adapter is connected.

A simple mock object is not enough. We also need to trigger events so that the application knows the values have changed.

For this, we create a custom `BatteryMock` class.

Example

```
test('update battery status', async ({ page }) => {
  await page.addInitScript(() => {

    class BatteryMock {
      level = 0.10;
      charging = false;
      chargingTime = 1800;
      dischargingTime = Infinity;

      _chargingListeners = [];
      _levelListeners = [];
```

```
    addEventListener(eventName, listener) {
      if (eventName === 'chargingchange')
        this._chargingListeners.push(listener);

      if (eventName === 'levelchange')
        this._levelListeners.push(listener);
    }

    _setLevel(value) {
      this.level = value;
      this._levelListeners.forEach(cb => cb());
    }

    _setCharging(value) {
      this.charging = value;
      this._chargingListeners.forEach(cb => cb());
    }
  }

  const mockBattery = new BatteryMock();

  window.navigator.getBattery = async () => mockBattery;

  window.mockBattery = mockBattery;
});

await page.goto('/');

await expect(page.locator('.battery-percentage'))
  .toHaveText('10%');

await page.evaluate(() => {
  window.mockBattery._setLevel(0.275);
});

await expect(page.locator('.battery-percentage'))
  .toHaveText('27.5%');

await expect(page.locator('.battery-status'))
  .toHaveText('Battery');

await page.evaluate(() => {
```

```
    window.mockBattery._setCharging(true);
  });

  await expect(page.locator('.battery-status'))
    .toHaveText('Adapter');

  await expect(page.locator('.battery-fully'))
    .toHaveText('00:30');
});
```

Explanation of important methods

1. addEventListener()

```
addEventListener(eventName, listener) {
  if (eventName === 'levelchange')
    this._levelListeners.push(listener);
}
```

Stores the event listener so that we can call it later.

2. _setLevel()

```
_setLevel(value) {
  this.level = value;
  this._levelListeners.forEach(cb => cb());
}
```

- Updates the battery level.
- Calls all registered level-change listeners.
- The application receives the update and refreshes its UI.

3. _setCharging()

```
_setCharging(value) {
  this.charging = value;
  this._chargingListeners.forEach(cb => cb());
}
```

Updates charging status and notifies the registered listeners.

4. window.mockBattery

```
window.mockBattery = mockBattery;
```

Stores the mock object on the browser's `window` object so that the test can access it later.

5. `page.evaluate()`

```
await page.evaluate(() => {
  window.mockBattery._setLevel(0.275);
});
```

Executes JavaScript inside the browser page.

Here, Playwright tells the mock battery to change its level to 27.5%.

Understand the complete flow

Battery mock test flow

1. Create mock battery

Initial level = 10%, charging = false.

2. Load application

UI displays 10% and Battery.

3. Update battery level

Call `_setLevel(0.275)`.

4. Trigger levelchange event

UI updates to 27.5%.

5. Connect adapter

Call `_setCharging(true)`.

6. Trigger chargingchange event

UI displays Adapter and 00:30.

6. Real-world QA automation examples

Scenario	What you can mock
Battery status application	Battery API
Website checking internet status	`navigator.onLine`
Location-based application	Geolocation API
Permission-based application	Notifications or permissions
Application using device orientation	Device orientation API
Application using clipboard	Clipboard API

For example, you can mock geolocation to test whether a delivery application displays the correct location without using the actual device location.

7. Interview Questions and Answers

Q1. What is mocking in Playwright?

Mocking means replacing a real API or browser feature with a fake implementation so that we can control its behavior during testing.

Q2. Why do we mock browser APIs?

We mock browser APIs when actual browser functionality is difficult to control, unavailable, or dependent on the device. It helps us create predictable test scenarios.

Q3. What is `page.addInitScript()`?

It executes JavaScript in the browser page before the page's own scripts run. We use it to configure mocks before application code accesses browser APIs.

Q4. Why should we create mocks before `page.goto()`?

Because the application may call browser APIs immediately during page loading. Creating mocks before navigation ensures the application uses the mocked API.

Q5. What is `page.exposeFunction()`?

It exposes a Node.js function to the browser page, allowing browser-side JavaScript to communicate with the Playwright test.

Q6. How can you mock a read-only browser property?

We can use `Object.defineProperty()` to override a property if it is configurable.

Q7. What is a golden result?

A golden result is the expected output used to compare against the actual result during testing.

Q8. How can you update a mock value during a test?

We can expose the mock object on `window` and use `page.evaluate()` to change its value. We should also trigger the corresponding events so the application updates its UI.

Q9. What is the difference between `page.evaluate()` and `page.addInitScript()`?

page.evaluate()	**page.addInitScript()**
Executes JavaScript when called.	Executes before page scripts.
Useful for interacting with an already loaded page.	Useful for setting up mocks before application code runs.
Can update an existing mock.	Can initialize a mock.

Q10. Can we use mocks to test different scenarios?

Yes. For example, we can mock battery levels of 10%, 50%, and 100%, or test charging and non-charging scenarios without changing the actual device battery.

Q11. What is the purpose of `addEventListener()` in the BatteryMock class?

It stores event listeners so that we can notify the application whenever the mocked battery properties change.

Q12. What is the difference between mocking an API and verifying an API call?

- Mocking: Replaces the real API with a fake implementation.
- Verifying: Checks whether the application called the expected API methods.

8. Quick revision notes

Method / Concept	Purpose
`page.addInitScript()`	Set up mock before application scripts.
`Object.defineProperty()`	Override configurable read-only properties.
`page.exposeFunction()`	Connect browser JavaScript with Node.js.
Golden result	Expected result for comparison.
`page.evaluate()`	Execute JavaScript inside the browser.
`addEventListener()`	Register event listeners.
`window.mockBattery`	Make the mock accessible to the test.

Important for interviews

Remember these four concepts:

1. `addInitScript()` – Create mocks before page loading.
2. `exposeFunction()` – Communicate from browser to Node.js.
3. `evaluate()` – Execute code inside the browser.
4. Events – Notify the application when mocked values change.

9. Test your understanding

1. Which method is used to create a mock before application scripts execute?

page.evaluate()
page.addInitScript()
page.locator()

Correct! addInitScript runs before the page's own scripts.

2. Which method exposes a Node.js function to browser JavaScript?

page.exposeFunction()
page.addInitScript()
page.goto()

Correct! exposeFunction makes a Node.js function callable from the page.

3. Which method executes JavaScript inside an already loaded browser page?

page.evaluate()
page.exposeFunction()
test.beforeEach()

Correct! evaluate runs the supplied JavaScript in the page context.

4. How can we override a configurable read-only property?

page.route()
Object.defineProperty()
page.close()

Correct! Object.defineProperty can redefine a configurable property.

Your score: 4/4

Try again

Interview tip: Be ready to explain the difference between `addInitScript()`, `evaluate()` and `exposeFunction()` with a practical example. These are the most important methods in this section.

Playwright – Navigations

1. What is Navigation?

Navigation means moving from one web page to another URL.

In Playwright, navigation can happen in two ways:

1. Direct navigation: Opening a URL using `page.goto()`.
2. User interaction navigation: Clicking a link or button that redirects to another page.

Example:

- Opening Google directly.
- Clicking the Login button and navigating to the login page.
- Clicking a product and navigating to its details page.

2. Basic Navigation – `page.goto()`

The simplest way to navigate to a website is:

```
import { test, expect } from '@playwright/test';

test('basic navigation', async ({ page }) => {
  await page.goto('https://example.com');

  await expect(page).toHaveTitle('Example Domain');
});
```

Explanation:

- `page.goto()` opens the specified URL.
- Playwright waits for the page's `load` event by default.
- `toHaveTitle()` verifies the page title.

What is the Load event?

The `load` event fires when the document and its dependent resources have loaded, such as:

- HTML
- CSS
- JavaScript
- Images
- Iframes

Important: `page.goto()` does not necessarily mean that every activity on the website has finished. Modern applications can continue loading data after the `load` event.

Interview question

Q1. Which method is used for navigation in Playwright?

Answer: `page.goto()` is used to navigate directly to a specified URL.

Q2. What does `page.goto()` wait for by default?

Answer: It waits for the `load` event, which indicates that the document and its dependent resources have loaded.

3. When is the Page Loaded?

Modern websites don't always finish all their work when the `load` event fires.

For example, after opening an e-commerce website:

1. HTML loads.
2. CSS and JavaScript load.
3. Product information is fetched from an API.
4. Product images load.
5. Dynamic content appears.

Some of these activities happen after the `load` event.

So, how do we know when we can interact with an element?

Playwright automatically waits for elements to become actionable before performing actions.

Example

```
test('navigate and click', async ({ page }) => {
  await page.goto('https://example.com');

  await page.getByText('Example Domain').click();
});
```

Here, Playwright automatically waits for the element to satisfy the required actionability checks before clicking it.

What is Actionability?

Actionability means checking whether an element is ready for an action.

For a click, Playwright checks that the element is:

- Visible
- Stable
- Receives events
- Enabled

It also checks that the locator resolves to exactly one element.

Important interview point: You usually don't need to add `waitForTimeout()` after every navigation. Playwright has built-in auto-waiting.

4. Hydration

Important interview topic

What is Hydration?

Hydration is the process of making a static HTML page interactive by attaching JavaScript functionality and event listeners.

Let's understand with an example.

Suppose you open a login page.

Initially, the browser displays:

- Username field
- Password field
- Login button

But the JavaScript responsible for handling the login button has not loaded yet.

The button looks enabled, but clicking it does nothing.

This happens because the page has not completed hydration.

How does hydration work?

Step 1: Static HTML loads

The page displays the login button and input fields.

Visible

Step 2: JavaScript is loading

The button is visible, but its event listeners might not be attached yet.

Not fully interactive

Step 3: Hydration completed

JavaScript attaches event listeners and the page becomes interactive.

Interactive

Why is hydration a problem in Playwright?

Playwright is very fast.

It may see the login button and click it immediately, even before the application's JavaScript has finished attaching the event listeners.

As a result:

- The click may not work.
- Entered text may disappear.
- The expected action may not happen.

How can we identify a hydration issue?

The documentation suggests:

1. Open Chrome DevTools.
2. Go to the Network panel.
3. Select Slow 3G.
4. Reload the application.
5. Try clicking the button as soon as it appears.

If the click is ignored or entered text disappears, the application may have a hydration problem.

What is the correct solution?

The application developers should disable interactive controls until hydration is complete.

For example, the Login button should remain disabled until its JavaScript event handlers are ready.

Interview question

Q3. What is hydration in web applications?

Answer: Hydration is the process of attaching JavaScript functionality and event listeners to static HTML so that the page becomes interactive.

Q4. Why can Playwright clicks fail during hydration?

Answer: Playwright can interact with a visible and enabled element before the application's JavaScript event listeners are attached. The click happens, but the application does not respond.

Q5. How can we solve hydration issues?

Answer: The application should keep interactive controls disabled until hydration is complete. In automation, we should wait for a meaningful application-ready condition rather than adding fixed delays.

5. Waiting for Navigation – `page.waitForURL()`

Sometimes clicking a button or link redirects the application to another URL.

In these situations, we can use:

`page.waitForURL()`

It waits until the page URL matches the expected URL.

Example

```
test('login navigation', async ({ page }) => {
  await page.goto('https://example.com');

  await page.getByText('Click me').click();

  await page.waitForURL('**/login');
});
```

The pattern `**/login` means that the URL can have any prefix but should end with `/login`.

Recommended approach

For more reliable navigation handling, start waiting before clicking:

```
test('wait for login navigation', async ({ page }) => {
  await page.goto('https://example.com');

  const navigationPromise = page.waitForURL('**/login');

  await page.getByText('Click me').click();
```

```
  await navigationPromise;
});
```

Why start waiting before clicking?

Because navigation may happen immediately after the click. Starting the wait first avoids missing the URL change.

Real-world example

Suppose you are testing an e-commerce application:

- Open the home page.
- Click Login.
- Verify that the URL changes to `/login`.
- Verify that the login page is displayed.

```
test('verify login page navigation', async ({ page }) => {
  await page.goto('https://myshop.com');

  const navigationPromise = page.waitForURL('**/login');

  await page.getByRole('link', { name: 'Login' }).click();

  await navigationPromise;

  await expect(page.getByRole('heading', {
    name: 'Login'
  })).toBeVisible();
});
```

Interview question

Q6. What is `page.waitForURL()`?

Answer: It waits until the current page URL matches a specified URL or URL pattern.

Q7. Why should we use `waitForURL()` before clicking a navigation link?

Answer: Starting the wait before clicking ensures that Playwright is already waiting when navigation begins, avoiding timing issues.

6. Back/Forward Cache (BFCache)

What is BFCache?

BFCache stands for Back/Forward Cache.

It is a browser feature that stores a page's DOM and JavaScript state in memory so the browser can restore the page quickly when the user presses Back or Forward.

Example:

1. Open Google.
2. Navigate to YouTube.
3. Press the browser's Back button.
4. Google can be restored instantly from memory instead of being loaded again.

How does Playwright handle BFCache?

According to this documentation:

- Playwright disables BFCache by default across browsers.
- Explicitly enabling BFCache does not mean BFCache restoration testing is supported.
- Restoring a page from BFCache skips normal network navigation lifecycle events.
- This can desynchronize Playwright's internal page state and cause timeouts or failed interactions.

Interview question

Q8. What is BFCache?

Answer: BFCache is a browser feature that stores a page's DOM and JavaScript state in memory so it can be restored quickly when navigating backward or forward.

Q9. Does Playwright support testing BFCache restorations?

Answer: No. Playwright disables BFCache by default, and testing BFCache restorations is not supported because restoration skips standard navigation lifecycle events that Playwright relies on.

7. Navigation Events

This is an important concept for understanding how a browser loads a new page.

Playwright divides the process into two major stages:

1. Navigation
2. Loading

Stage 1: Navigation

Navigation starts when:

- We open a new URL.
- We click a link.
- We perform an action that changes the page URL.

Navigation is committed when the browser receives and parses the response headers and updates the session history.

At this point, the new URL is available through:

`page.url()`

Stage 2: Loading

After navigation is committed, the browser starts loading the document.

The process generally looks like this:

Page navigation lifecycle

1. Navigation starts

User clicks a link or Playwright opens a URL.

2. Navigation committed

Response headers are received and the browser updates the URL.

3. Document loading

HTML response body is downloaded and parsed.

4. DOMContentLoaded event

The initial HTML document has been parsed and deferred/module scripts have executed.

5. Load event

The document and its dependent resources have loaded.

6. Dynamic scripts and resources

Additional scripts and application activities may continue afterward.

Important navigation events

Event	Meaning
`commit`	Response headers received and navigation committed.
`domcontentloaded`	Initial HTML document parsed and deferred/module scripts executed.
`load`	Document and dependent resources loaded.

How to listen to navigation events

```
test('navigation events', async ({ page }) => {

  page.on('domcontentloaded', () => {
    console.log('DOM content loaded');
  });

  page.on('load', () => {
    console.log('Page fully loaded');
  });
```

```
  await page.goto('https://example.com');
});
```

This code listens for two events and prints messages when they occur.

You can also use different `waitUntil` options with navigation methods:

```
await page.goto('https://example.com', {
  waitUntil: 'domcontentloaded'
});
```

Other commonly used options include:

- `commit`
- `domcontentloaded`
- `load`

There is also `networkidle`, but Playwright discourages using it as a general readiness condition for tests. Prefer waiting for the specific UI state you need.

8. Difference between Navigation Methods and Events

Method / Event	Purpose
`page.goto()`	Navigate directly to a URL.
`page.url()`	Get the current URL.
`page.waitForURL()`	Wait for a URL change or match.
`domcontentloaded`	Initial HTML document has been parsed.
`load`	Document and dependent resources have loaded.
`commit`	Navigation has been committed.

9. Important Interview Questions – Navigation

Q10. What is the difference between `load` and `DOMContentLoaded`?

Answer:

- `DOMContentLoaded` fires when the HTML document has been parsed and deferred/module scripts have executed.
- `load` fires after the document and its dependent resources have loaded.

Q11. Does `page.goto()` wait until all API calls are completed?

Answer: No. It waits for the selected navigation lifecycle event, which is `load` by default. API calls and dynamic content loading may continue afterward.

Q12. What is auto-waiting in Playwright?

Answer: Auto-waiting means Playwright automatically waits for an element to satisfy the necessary actionability checks before performing an action.

Q13. What is the difference between `page.goto()` and `page.waitForURL()`?

Answer:

- `page.goto()` directly navigates to a URL.
- `page.waitForURL()` waits for the current page to reach a specified URL, usually after an action triggers navigation.

Q14. What is navigation committed?

Answer: Navigation is committed when the response headers have been parsed and the browser updates its session history to the new URL.

Q15. What happens after navigation is committed?

Answer: The browser starts loading the document, parsing HTML, executing scripts, loading resources and firing lifecycle events.

Q16. Should we use `waitForTimeout()` after every navigation?

Answer: No. Playwright provides auto-waiting. We should wait for a specific URL, element, or application state instead of using unnecessary fixed delays.

10. Quick Revision Notes

Topic	Remember
Basic navigation	`page.goto()`
Current URL	`page.url()`
Wait for URL	`page.waitForURL()`
Auto-waiting	Waits for actionability checks
Hydration	Makes static HTML interactive
BFCache	Stores pages in memory for quick restoration
Navigation committed	Response headers parsed, URL updated
DOMContentLoaded	Initial document parsed
Load	Dependent resources loaded

11. Test Your Understanding

1. Which method is used to open a URL directly?

page.goto()
page.waitForURL()
page.url()

Incorrect. page.goto() navigates directly to the given URL.

2. What does page.goto() wait for by default?

networkidle
load

DOMContentLoaded

Correct! The default waitUntil condition is load.

3. What is hydration?

Loading images
Attaching JavaScript functionality to static HTML
Changing the URL

Correct! Hydration makes the page interactive by attaching event handlers and application behavior.

4. Which method waits for the URL to match a pattern?

page.locator()
page.waitForURL()
page.evaluate()

Correct! waitForURL() waits for the current URL to match the expected URL.

5. What does BFCache stand for?

Browser Fast Cache
Back/Forward Cache
Browser File Cache

Incorrect. BFCache means Back/Forward Cache.

6. Which event fires after the HTML document is parsed and deferred/module scripts have executed?

load
commit
DOMContentLoaded

Incorrect. DOMContentLoaded fires before the load event.

Score: 3/6

Retry quiz

Most important for your QA Automation interview: Focus on `page.goto()`, `page.waitForURL()`, auto-waiting, hydration, and the difference between

`DOMContentLoaded` and `load`. These are particularly useful when explaining navigation synchronization in Playwright.

Playwright – Network

1. What is Network in Playwright?

Network means communication between a browser and a server.

Whenever we open a website, the browser sends requests to the server and receives responses.

For example, when you open an e-commerce website:

1. Browser requests the home page.
2. Server sends HTML, CSS and JavaScript.
3. Browser requests product information through an API.
4. Server sends product details.
5. Browser displays the products.

Playwright allows us to monitor, intercept, modify, mock and block network requests and responses.

It supports:

- HTTP and HTTPS requests.
- API requests such as XHR and Fetch.
- Network request and response monitoring.
- Request and response modification.
- API mocking.
- WebSocket monitoring.

Important terms

Term	Meaning
Request	Browser asks the server for information.
Response	Server sends information back.
HTTP	Protocol used for communication between browser and server.
API	Interface through which applications communicate.
XHR	XMLHttpRequest, used to make asynchronous HTTP requests.
Fetch	Modern JavaScript API for making network requests.

2. Mock APIs

API mocking means creating a fake API response instead of depending on the actual backend server.

Playwright supports three important approaches:

1. Mock an API without calling the actual server.
2. Call the actual API and modify its response.
3. Use HAR files to mock network requests.

Example: Mock product API

Suppose your application normally gets products from:

`https://example.com/api/products`

The actual API returns:

```
[
  { "id": 1, "name": "Laptop" },
  { "id": 2, "name": "Mobile" }
]
```

But during testing, you want to return your own product data.

You can use `page.route()` and `route.fulfill()`.

We will understand these methods in detail below.

3. Network Mocking – `page.route()` and `context.route()`

Playwright does not require any additional configuration to intercept network requests.

We can use:

- `page.route()` – Intercepts requests for a particular page.
- `context.route()` – Intercepts requests for pages in a browser context, including popups.

Example 1: Block CSS requests

```
import { test, expect } from '@playwright/test';

test.beforeEach(async ({ context }) => {
  await context.route(/.css$/, route => route.abort());
});

test('loads page without CSS', async ({ page }) => {
  await page.goto('https://playwright.dev');
});
```

Explanation:

- `context.route()` intercepts matching requests.
- `/.css$/` matches URLs ending in `.css`.
- `route.abort()` blocks the request.
- The browser cannot load the blocked CSS file.

Result: The website loads without its CSS styling.

Example 2: Block images

```
test('loads page without images', async ({ page }) => {

  await page.route(/(png|jpeg)$/, route => route.abort());

  await page.goto('https://playwright.dev');
});
```

Here, PNG and JPEG requests are blocked.

Difference between `page.route()` and `context.route()`

`page.route()`	`context.route()`
Works for a particular page.	Works across the browser context.
Useful for one-page testing.	Useful for multiple pages.
Can intercept requests from that page.	Also applies to popup windows and opened links in that context.

Interview Q1. What is `page.route()` in Playwright?

Answer: `page.route()` intercepts network requests made by a page. We can mock, modify, continue or abort those requests.

Interview Q2. Difference between `page.route()` and `browserContext.route()`?

Answer: `page.route()` applies to a particular page, whereas `browserContext.route()` applies to pages within the browser context.

4. HTTP Authentication

HTTP Authentication is used when a website requires a username and password before allowing access.

Playwright provides `httpCredentials` to configure authentication.

Example

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    httpCredentials: {
      username: 'bill',
      password: 'pa55w0rd',
    }
  }
});
```

Explanation:

- `httpCredentials` configures HTTP authentication.
- `username` provides the authentication username.
- `password` provides the authentication password.

Now Playwright uses these credentials when accessing a site protected by HTTP authentication.

Important: This is for HTTP authentication, such as Basic or Digest authentication, not ordinary application login forms.

Interview Q3. How do you handle HTTP authentication in Playwright?

Answer: We use the `httpCredentials` option in `playwright.config.ts` to provide the username and password.

5. HTTP Proxy

What is a Proxy?

A proxy server acts as an intermediary between your browser and the internet.

Instead of directly accessing a website, the browser sends its request through the proxy server.

Proxy communication

Playwright Browser

Proxy Server

Intermediary server

Website / Server

Configure a global proxy

In `playwright.config.ts`:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    proxy: {
      server: 'http://myproxy.com:3128',
      username: 'usr',
      password: 'pwd'
    }
  }
});
```

Here:

- `server` specifies the proxy address.
- `username` specifies proxy authentication.
- `password` specifies the proxy password.

Configure proxy for a specific context

```
import { test } from '@playwright/test';

test('use custom proxy', async ({ browser }) => {

  const context = await browser.newContext({
    proxy: {
      server: 'http://myproxy.com:3128',
    }
```

```
  });

  const page = await context.newPage();

  await page.goto('https://example.com');

  await context.close();
});
```

Difference:

- Global proxy: Configured in Playwright configuration and used by the configured browser sessions.
- Context proxy: Configured for a particular browser context.

Interview Q4. What is an HTTP proxy?

Answer: A proxy is an intermediary server between the browser and the destination server. Playwright supports configuring HTTP, HTTPS and SOCKSv5 proxies.

6. Network Events

Playwright allows us to monitor requests and responses using event listeners.

Two important events are:

- `request`
- `response`

Example

```
test('monitor network events', async ({ page }) => {

  page.on('request', request => {
    console.log('>>', request.method(), request.url());
  });

  page.on('response', response => {
    console.log('<<', response.status(), response.url());
  });
```

```
  await page.goto('https://example.com');
});
```

Explanation

1. Request event

```
page.on('request', request => {
  console.log(request.method(), request.url());
});
```

It executes whenever the page sends a network request.

You can inspect:

- `request.url()` – Request URL.
- `request.method()` – GET, POST, PUT, DELETE, etc.

2. Response event

```
page.on('response', response => {
  console.log(response.status(), response.url());
});
```

It executes whenever a response is received.

You can inspect:

- `response.status()` – HTTP status code.
- `response.url()` – Response URL.

Example output

```
>> GET https://example.com
<< 200 https://example.com
>> GET https://example.com/style.css
<< 200 https://example.com/style.css
```

The exact requests depend on the website.

Interview Q5. How do you monitor network requests in Playwright?

Answer: We use `page.on('request')` to monitor outgoing requests and `page.on('response')` to monitor incoming responses.

7. Waiting for Network Responses – `page.waitForResponse()`

Sometimes we want to verify whether an API response is received after clicking a button.

For example:

- Open an employee application.
- Click the Update button.
- The application calls an API.
- The API returns employee information.

We can use:

`page.waitForResponse()`

Example

```
test('wait for API response', async ({ page }) => {

  await page.goto('https://example.com');

  const responsePromise =
    page.waitForResponse('**/api/fetch_data');

  await page.getByText('Update').click();

  const response = await responsePromise;

  console.log(response.status());
});
```

Understand the execution

1. `waitForResponse()` starts waiting for the expected API response.
2. We click the Update button.
3. The application sends an API request.
4. The server returns a response.
5. Playwright captures the response.
6. We can verify its status or data.

Important: Notice that we don't write `await` immediately before `page.waitForResponse()`.

We store its Promise first, perform the action, and then await the Promise.

This prevents us from missing a fast response.

Different ways to use `waitForResponse()`

A. Using a glob pattern

```
const responsePromise =
  page.waitForResponse('**/api/fetch_data');
```

B. Using RegExp

```
const responsePromise =
  page.waitForResponse(/\.jpeg$/);
```

This waits for a response URL ending in `.jpeg`.

C. Using a condition (predicate)

```
const responsePromise =
  page.waitForResponse(response =>
    response.url().includes('token')
  );
```

This waits for a response whose URL contains `token`.

Practical interview example

```
test('verify employee API', async ({ page }) => {

  await page.goto('https://example.com');

  const responsePromise = page.waitForResponse(
    response =>
      response.url().includes('/api/employees') &&
      response.request().method() === 'GET'
  );

  await page.getByRole('button', { name: 'Load
Employees' }).click();

  const response = await responsePromise;

  expect(response.status()).toBe(200);
});
```

Interview Q6. What is `page.waitForResponse()`?

Answer: It waits for a network response that matches a specified URL pattern or condition.

Interview Q7. Why should we start `waitForResponse()` before clicking a button?

Answer: Because the API response may arrive immediately after the click. Starting the wait first prevents missing the response.

8. Handle Requests – `route.fulfill()`

We can intercept an API request and return our own response without calling the actual backend.

The important method is:

```
route.fulfill()
```

Example

```
test('mock API response', async ({ page }) => {
```

```
  await page.route('**/api/fetch_data', async route => {

    await route.fulfill({
      status: 200,
      body: JSON.stringify({
        name: 'Komal',
        role: 'QA Engineer'
      }),
      contentType: 'application/json'
    });

  });

  await page.goto('https://example.com');
});
```

Explanation

- `page.route()` intercepts the matching API request.
- `route.fulfill()` sends a fake response.
- `status: 200` means successful response.
- body contains our custom data.
- `contentType` tells the browser that the response is JSON.

Important: The actual backend API is not called for the intercepted request.

Real-world QA example

Suppose your application displays a list of products.

Instead of depending on the actual product API, you can mock:

```
await page.route('**/api/products', async route => {

  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([
      { id: 1, name: 'Laptop', price: 50000 },
      { id: 2, name: 'Mobile', price: 20000 }
```

```
    ])
  });

});
```

Now your application receives the two fake products.

You can test the product listing UI without depending on the backend.

9. Modify Requests

Sometimes we don't want to block an API request. We want to modify the request before sending it to the server.

We use:

`route.continue()`

Example 1: Delete an HTTP header

```
await page.route('**/*', async route => {

  const headers = route.request().headers();

  delete headers['x-secret'];

  await route.continue({ headers });
});
```

Explanation:

1. Intercept the outgoing request.
2. Get the existing request headers.
3. Delete the `x-secret` header.
4. Continue the request with modified headers.

The request reaches the server without that header.

Example 2: Change the request method to POST

```
await page.route('**/*', async route => {

  await route.continue({
    method: 'POST'
  });

});
```

Here, the intercepted request is continued as a POST request.

Interview Q8. What is the difference between `route.fulfill()` and `route.continue()`?

`route.fulfill()`	`route.continue()`
Returns a custom response.	Sends the request to the server.
Does not call the actual server for that request.	Continues network communication.
Used for API mocking.	Used for request modification or normal continuation.

10. Abort Requests – `route.abort()`

Sometimes we want to block certain network requests.

For example:

- Block images.
- Block CSS files.
- Block unwanted third-party requests.
- Simulate a failed network request.

We use:

```
route.abort()
```

Example 1: Block image requests

```
await page.route('**/*.{png,jpg,jpeg}', async route => {
  await route.abort();
});
```

This blocks PNG, JPG and JPEG requests.

Example 2: Abort based on resource type

```
await page.route('**/*', async route => {

  if (route.request().resourceType() === 'image') {
    await route.abort();
  } else {
    await route.continue();
  }

});
```

Here:

- `resourceType()` checks the request type.
- If it is an image, the request is blocked.
- Otherwise, the request continues.

Common resource types

Resource type	Meaning
`document`	HTML document
`stylesheet`	CSS file
`script`	JavaScript file

`image`	Image
`font`	Font file
`xhr`	XMLHttpRequest
`fetch`	Fetch request

Interview Q9. How can you block API requests in Playwright?

Answer: We can use `page.route()` and `route.abort()` to intercept and block matching network requests.

11. Modify Responses – `route.fetch()` and `route.fulfill()`

This is slightly different from normal API mocking.

Sometimes we want to:

1. Call the actual server.
2. Receive its original response.
3. Modify some response data.
4. Return the modified response to the browser.

We use:

- `route.fetch()` – Fetches the original response from the server.
- `route.fulfill()` – Sends the modified response to the browser.

Example

```
await page.route('**/title.html', async route => {

  // Get original response
  const response = await route.fetch();
```

```
  // Read response body
  let body = await response.text();

  // Modify the title
  body = body.replace(
    '<title>',
    '<title>My prefix:'
  );

  // Return modified response
  await route.fulfill({
    response,
    body,
    headers: {
      ...response.headers(),
      'content-type': 'text/html'
    }
  });

});
```

Explanation

Step 1:

```
const response = await route.fetch();
```

Calls the actual server and gets its response.

Step 2:

```
let body = await response.text();
```

Reads the original response as text.

Step 3:

```
body = body.replace('<title>', '<title>My prefix:');
```

Modifies the HTML title.

Step 4:

```
await route.fulfill({
  response,
```

```
  body
});
```

Returns the modified response to the browser.

Difference between mocking and modifying a response

Mocking	Modifying
Creates a completely fake response.	Starts with the actual server response.
Uses `route.fulfill()`.	Uses `route.fetch()` followed by `route.fulfill()`.
Actual backend is not called for the intercepted request.	Actual backend is called.

Interview Q10. What is `route.fetch()`?

Answer: `route.fetch()` sends the intercepted request to the actual server and retrieves its response so that we can inspect or modify it before returning it to the browser.

12. How Request Interception Works

This is an important conceptual topic.

Playwright's route handler sits between the browser page and the network stack.

The request passes through Playwright before it reaches the network.

Request interception flow

Page

Application sends a request.

Playwright Route Handler

Intercepts the request.

continue()

Send to server

fulfill()

Return fake response

abort()

Block request

Three important methods

Method	What happens?
`route.continue()`	Request goes to the network.
`route.fulfill()`	Playwright returns a response directly.
`route.abort()`	Request fails and is not sent normally.

Remember these three methods for interviews.

13. Headers Owned by the Network Stack

The browser automatically adds certain HTTP headers when sending requests.

Examples:

- `Cookie`
- `Host`
- `Accept-Encoding`
- `Content-Length`
- `Sec-Fetch-*`

These headers are managed by the browser's network stack.

The route handler runs before some of these headers are added.

Therefore:

- They may not be available in `request.headers()` or `request.allHeaders()` during interception.
- You cannot reliably override them.
- Passing a custom `Cookie` header to `route.continue()` does not override the browser's cookie store.

What about response headers?

The network stack has already processed the response.

So `response.allHeaders()` can return the response headers sent by the server, including `Set-Cookie` for HttpOnly cookies.

Interview Q11. Can we modify every request header using `route.continue()`?

Answer: No. Certain headers, such as `Cookie`, `Host` and `Content-Length`, are controlled by the browser's network stack and cannot be reliably overridden.

14. Redirects

What is a Redirect?

A redirect means the server or application sends the browser to another URL.

Example:

```
/user
   |
   v
/login
```

Suppose you request `/user`, but the server redirects you to `/login`.

Playwright treats the original request and its redirects as one request chain.

Important points:

- The route handler is called for the original request.

- The browser follows the redirect.
- `response.request()` returns the last request in the redirect chain.
- `request.redirectedFrom()` lets you navigate backward through the redirect chain.

Example

```
page.on('request', request => {

  console.log('URL:', request.url());

  const previousRequest = request.redirectedFrom();

  if (previousRequest) {
    console.log('Redirected from:', previousRequest.url());
  }

});
```

Important note about 3xx responses

If you use `route.fulfill()` with a 3xx redirect status, it does not give you another opportunity to intercept the redirected request.

- Chromium and Firefox follow the redirect without calling your handler again.
- WebKit rejects this call.

If you want to send the request somewhere else, use `route.continue({ url })` or `route.fallback()`.

Interview Q12. How does Playwright handle redirects?

Answer: Playwright treats a request and its redirects as a single chain. The route handler normally intercepts the original request, and the browser follows subsequent redirects.

15. Glob URL Patterns

Glob patterns are simplified patterns used to match URLs.

They are commonly used with:

- `page.route()`
- `context.route()`
- `page.waitForResponse()`

Important wildcard symbols

Symbol	Meaning
*	Matches any characters except /.
**	Matches any characters, including /.
{}	Matches one of several comma-separated options.
\	Escapes special characters.

Examples

Example 1:

```
await page.route('https://example.com/*.js', route =>
route.abort());
```

Matches:

https://example.com/file.js

Does not match:

https://example.com/path/file.js

Because * does not match /.

Example 2:

```
await page.route('**/*.js', route => route.abort());
```

Matches JavaScript files at different folder levels.

Example 3:

```
await page.route('**/*.{png,jpg,jpeg}', route => route.abort());
```

Matches PNG, JPG and JPEG files.

Important interview point: A glob pattern must match the entire URL, not just a portion of it.

For complex matching requirements, use RegExp.

16. WebSockets

What is WebSocket?

WebSocket is a communication protocol that provides a persistent connection between the browser and server.

Unlike normal HTTP communication, where the browser generally sends a request and receives a response, WebSockets allow both sides to exchange messages over an ongoing connection.

Real-world examples

- Live chat applications.
- Real-time stock price updates.
- Live notifications.
- Online multiplayer games.

How does Playwright monitor WebSockets?

Playwright provides:

```
page.on('websocket')
```

Example

```
test('monitor websocket', async ({ page }) => {
```

```
  page.on('websocket', ws => {

    console.log(`WebSocket opened: ${ws.url()}`);

    ws.on('framesent', event => {
      console.log('Sent:', event.payload);
    });

    ws.on('framereceived', event => {
      console.log('Received:', event.payload);
    });

    ws.on('close', () => {
      console.log('WebSocket closed');
    });

  });

  await page.goto('https://example.com');
});
```

Explanation

Event	Meaning
`websocket`	A WebSocket connection is created.
`framesent`	Browser sends a WebSocket message.
`framereceived`	Browser receives a WebSocket message.
`close`	WebSocket connection closes.

Interview Q13. What is the difference between HTTP and WebSocket?

Answer:

- HTTP generally follows a request-response communication pattern.

- WebSocket maintains a persistent connection and supports two-way communication between the client and server.

Q14. How do you monitor WebSocket messages in Playwright?

Answer: We use `page.on('websocket')` and listen to `framesent`, `framereceived` and `close` events.

17. Missing Network Events and Service Workers

What is a Service Worker?

A Service Worker is JavaScript that runs in the background of a browser and can intercept network requests.

It is commonly used for:

- Offline functionality.
- Caching resources.
- Push notifications.
- Handling network requests.

Why can Playwright miss network events?

Suppose your application uses Mock Service Worker (MSW).

MSW creates its own Service Worker, which can intercept network requests before Playwright's routing mechanisms see them.

As a result, requests may appear missing from:

`page.route()`

or:

`context.route()`

Solution

Playwright documentation suggests disabling Service Workers when using its native routing and network interception APIs:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    serviceWorkers: 'block'
  }
});
```

Now Service Workers are blocked, allowing Playwright's native routing mechanisms to handle the requests.

Important: Blocking Service Workers can affect application behavior. Use it when appropriate for the network tests you are running.

Interview Q15. Why might network requests be missing in Playwright?

Answer: Service Workers may intercept requests before Playwright's native routing APIs can observe them. We can configure `serviceWorkers: 'block'` when using Playwright's native network interception.

18. Most Important Network Methods – Quick Revision

Method	Purpose
`page.route()`	Intercept page network requests.
`context.route()`	Intercept requests across a browser context.
`route.fulfill()`	Return a custom response.
`route.continue()`	Send request to the server.
`route.abort()`	Block a request.

`route.fetch()`	Fetch the original server response.
`page.on('request')`	Monitor outgoing requests.
`page.on('response')`	Monitor incoming responses.
`page.waitForResponse()`	Wait for a matching response.
`page.on('websocket')`	Monitor WebSocket connections.
`request.resourceType()`	Identify the type of request.
`request.redirectedFrom()`	Get the previous request in a redirect chain.

19. Important Interview Questions – Basic to Advanced

Q16. What is network interception in Playwright?

Network interception means capturing browser network requests before they reach the server so we can inspect, modify, mock or block them.

Q17. How do you mock an API returning a 500 error?

```
await page.route('**/api/products', async route => {
  await route.fulfill({
    status: 500,
    body: 'Internal Server Error'
  });
});
```

Q18. How do you verify an API returns status code 200?

```
const response = await page.waitForResponse(
  '**/api/products'
);

expect(response.status()).toBe(200);
```

In a real test, start the response wait before the action that triggers the API request.

Q19. What is the difference between `route.fetch()` and `route.fulfill()`?

Answer: `route.fetch()` calls the actual server and retrieves the response. `route.fulfill()` sends a response to the browser, which can be completely mocked or based on a modified original response.

Q20. How do you block images in Playwright?

```
await page.route('**/*.{png,jpg,jpeg}', route => route.abort());
```

Q21. What is the difference between `request` and `response` events?

Answer: The request event occurs when the browser sends a request. The response event occurs when the browser receives a response.

Q22. How do you test an API without depending on the backend?

Answer: We intercept the API using `page.route()` and return predefined test data using `route.fulfill()`.

Q23. What is the purpose of HAR mocking?

Answer: HAR (HTTP Archive) files record HTTP requests and responses. Playwright can replay recorded responses from a HAR file to test applications without depending on live backend responses.

Q24. What is the difference between * and ** in glob patterns?

Answer: * matches characters except /, whereas ** matches characters including /.

Q25. What is MSW?

Answer: Mock Service Worker is a tool used to intercept and mock network requests using Service Workers.

20. Practical QA Automation Scenario

Imagine you are testing an e-commerce application with a Products API.

You need to test three scenarios:

1. Successful API response – 200.
2. Server error – 500.

3. Empty product list – no products available.

You can test all three without depending on the actual backend.

```
import { test, expect } from '@playwright/test';

test('products API success', async ({ page }) => {
  await page.route('**/api/products', route =>
    route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify([
        { id: 1, name: 'Laptop' },
        { id: 2, name: 'Mobile' }
      ])
    })
  );

  await page.goto('https://example.com');

  await expect(page.getByText('Laptop')).toBeVisible();
});

test('products API server error', async ({ page }) => {
  await page.route('**/api/products', route =>
    route.fulfill({
      status: 500,
      body: 'Internal Server Error'
    })
  );

  await page.goto('https://example.com');

  // Verify your application's error message here.
});

test('products API empty response', async ({ page }) => {
  await page.route('**/api/products', route =>
    route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify([])
```

```
    })
  );

  await page.goto('https://example.com');

  // Verify your application's empty-state message here.
});
```

This is a common practical application of API mocking in QA automation.

21. Test Your Understanding

Network – Interview Practice

8 questions

1. Which method is used to intercept network requests?

page.goto()
page.route()
page.evaluate()

Correct! page.route() intercepts matching network requests.

2. Which method returns a custom response without calling the actual backend?

route.fulfill()
route.continue()
route.fetch()

Correct! route.fulfill() sends a response directly to the browser.

3. Which method blocks a network request?

route.fetch()
route.abort()
route.continue()

Correct! route.abort() fails the request.

4. Which method fetches the original server response?

route.fulfill()
route.fetch()

route.abort()

Correct! route.fetch() sends the request to the server and retrieves its response.

5. Which method waits for an API response?

page.waitForResponse()
page.waitForURL()
page.waitForLoadState()

Correct! waitForResponse() waits for a matching network response.

6. What does ** match in a glob URL pattern?

Characters except slash
Characters including slash
Only numbers

Correct! Double asterisk matches characters including slashes.

7. Which event monitors outgoing requests?

response
request
load

Correct! The request event fires when a request is issued.

8. Which setting can block Service Workers?

serviceWorkers: 'allow'
serviceWorkers: 'block'
serviceWorkers: 'disable'

Correct! Use serviceWorkers: 'block'.

8/8

Retry quiz

My interview preparation tip: Give extra attention to these five topics:

1. `page.route()` and `route.fulfill()`.
2. Difference between `route.continue()`, `route.abort()` and `route.fetch()`.

3. `page.waitForResponse()` and API response validation.
4. Request and response events.
5. API mocking scenarios (200, 400, 500 and empty response).

These are especially useful for Selenium-to-Playwright QA Automation and SDET interviews.

Playwright – Other Locators

I'll explain each topic in simple language with practical examples, followed by important interview questions and answers. I'll also highlight which locators are commonly used in real-time automation projects.

1. Introduction – Other Locators

Playwright provides different ways to locate web elements such as buttons, text fields, links, checkboxes, etc.

The recommended locators are:

- `getByRole()` – Locates elements based on their role.
- `getByText()` – Locates elements using visible text.
- `getByLabel()` – Locates form elements using labels.
- `getByTestId()` – Locates elements using test IDs.

Apart from these, Playwright supports CSS, XPath, text selectors, and other locators.

Example:

```
await page.locator('css=button').click();
```

This locates a button using its CSS selector and clicks it.

2. CSS Locator

CSS selectors are used to identify HTML elements using their tag name, ID, class, or attributes.

Example:

```
await page.locator('css=button').click();
```

Other examples:

```
page.locator('#username');        // ID
page.locator('.login-btn');       // Class
page.locator('input');            // Tag name
page.locator('[type="text"]');    // Attribute
```

Important: Playwright recommends using user-visible locators like `getByRole()` and `getByText()` instead of CSS when possible because CSS selectors depend on the HTML structure and may break when the application changes.

CSS matching by text

Playwright provides special pseudo-classes to locate elements using text.

A. `:has-text()`

Finds an element that contains the given text anywhere inside it, including child elements.

```
await page.locator('article:has-text("Playwright")').click();
```

Meaning: Find an article containing the text Playwright and click it.

B. `:text()`

Finds the smallest element containing the specified text.

```
await page.locator('#nav-bar :text("Home")').click();
```

Meaning: Find Home text inside the navigation bar and click it.

C. `:text-is()`

Finds an element with exact text. It is case-sensitive.

```
await page.locator('#nav-bar :text-is("Home")').click();
```

It matches Home, but not home or Homepage.

D. `:text-matches()`

Finds text using a regular expression.

```
await page.locator('#nav-bar :text-matches("Log\\s*in",
"i")').click();
```

It can match:

- Login
- Log in
- log IN

Here, `i` means case-insensitive.

Difference between text pseudo-classes

Selector	Meaning
`:has-text()`	Element contains the text
`:text()`	Smallest element containing the text
`:text-is()`	Exact text match
`:text-matches()`	Text matching a regular expression

Interview Question 1: What is the difference between `:has-text()` and `:text()`?

Answer:

- `:has-text()` matches an element containing the specified text, including text inside child elements.
- `:text()` matches the smallest element containing that text.

3. CSS – Matching Only Visible Elements

Playwright supports `:visible` to locate only visible elements.

Suppose a webpage contains two buttons:

```
<button style="display:none">Login</button>
<button>Login</button>
```

If we write:

```
await page.locator('button').click();
```

Playwright finds both buttons, which can cause a strictness violation.

Instead:

```
await page.locator('button:visible').click();
```

This selects only the visible button.

Interview Question 2: How do you locate only visible elements in Playwright?

Answer: We can use the `:visible` pseudo-class in a CSS selector.

```
await page.locator('button:visible').click();
```

4. CSS – Elements That Contain Other Elements

The `:has()` pseudo-class is used to locate a parent element based on the presence of a child element.

Example HTML:

```
<article>
    <div class="promo">Special Offer</div>
</article>
```

Code:

```
await page.locator('article:has(div.promo)').textContent();
```

Meaning: Find an article that contains a div with class `promo` and get its text content.

Interview Question 3: What is the use of `:has()` in Playwright?

Answer: `:has()` helps locate a parent element based on a matching child element.

5. CSS – Elements Matching One of the Conditions

You can use comma-separated CSS selectors or the `:is()` pseudo-class to match elements satisfying different conditions.

Example:

```
await page.locator(
  'button:has-text("Log in"), button:has-text("Sign in")'
).click();
```

Meaning: Find a button containing either Log in or Sign in and click it.

The `:is()` pseudo-class also allows multiple selector conditions.

Interview Question 4: What is the use of comma-separated CSS selectors?

Answer: They allow us to match elements using multiple CSS selectors. An element matching any one of those selectors will be selected.

6. CSS – Matching Elements Based on Layout

Layout selectors locate elements based on their position relative to another element.

For example, imagine a login page:

[Enable PrestoSQL/Trino Password File Authentication | by Hasan Ahmadkhani | Medium]

You want to locate the input field positioned to the right of the Username text.

Playwright provides:

Selector	Meaning
`:right-of()`	Element to the right
`:left-of()`	Element to the left

`:above()`	Element above
`:below()`	Element below
`:near()`	Element nearby (within 50 CSS pixels by default)

Example 1:

```
await page.locator('input:right-
of(:text("Username"))').fill('Komal');
```

Meaning: Find the input field to the right of Username and enter Komal.

Example 2:

```
await page.locator('button:near(.promo-card)').click();
```

Meaning: Find a button near the promo card and click it.

Important interview point: Layout selectors are deprecated. They may be removed in the future. Prefer stable locators such as `getByRole()`, `getByLabel()`, and `filter()`.

Interview Question 5: What are layout selectors in Playwright?

Answer: Layout selectors locate elements based on their physical position relative to another element, such as above, below, left, right, or near.

7. CSS – Pick nth Match from the Query Result

Sometimes a webpage contains multiple elements with the same text.

Example:

```
<button>Buy</button>
<button>Buy</button>
<button>Buy</button>
```

If you want to select the third Buy button, use `:nth-match()`.

```
await page.locator(':nth-match(:text("Buy"), 3)').click();
```

Meaning: Select the third matching Buy element.

Important: The index starts from 1.

You can also use it to wait until the third matching element appears:

```
await page.locator(':nth-match(:text("Buy"), 3)').waitFor();
```

Interview Question 6: What is `:nth-match()`?

Answer: It is used to select a particular matching element from multiple elements. Its index is one-based, meaning counting starts from 1.

8. N-th Element Locator

The `nth()` locator is used to select a particular element from multiple matching elements.

In the documentation, the equivalent selector syntax is `nth=`.

```
// First button
await page.locator('button').locator('nth=0').click();

// Last button
await page.locator('button').locator('nth=-1').click();
```

Important difference:

Locator	Index
`nth=0`	First element
`nth=1`	Second element
`nth=2`	Third element
`nth=-1`	Last element

Interview Question 7: What is the difference between `:nth-match()` and `nth=`?

Answer:

- `:nth-match()` uses a one-based index.

- `nth=` uses a zero-based index.
- Both help select a particular element from multiple matching elements.

9. Parent Element Locator

Sometimes you locate a child element but need to access its parent element.

Example HTML:

```
<li>
    <label>Hello</label>
</li>

<li>
    <label>World</label>
</li>
```

You want to locate the parent `li` containing Hello.

Recommended approach: `filter()`

```
const child = page.getByText('Hello');

const parent = page.getByRole('listitem').filter({
    has: child
});
```

Meaning:

1. Locate the text Hello.
2. Find the list item containing Hello.
3. Store that parent element in the parent locator.

Alternative: XPath

```
const parent = page.getByText('Hello').locator('xpath=..');
```

Here, `xpath=..` selects the immediate parent.

Interview Question 8: How do you locate a parent element in Playwright?

Answer: We can use `locator.filter({ has: childLocator })` to find a parent containing a specific child. XPath `..` is another option, but `filter()` is generally more reliable.

10. XPath Locator

XPath is another way to identify HTML elements using their DOM structure and attributes.

Example:

```
await page.locator('xpath=//button').click();
```

This locates a button using XPath and clicks it.

Other examples:

```
// By ID
page.locator('//input[@id="username"]');

// By text
page.locator('//button[text()="Login"]');

// By attribute
page.locator('//input[@type="password"]');
```

Important points:

- XPath is useful for complex DOM structures.
- XPath depends on the HTML structure.
- If the DOM changes, XPath may break.
- XPath does not pierce shadow DOM.
- Playwright recommends user-visible locators over XPath.

Interview Question 9: What is XPath in Playwright?

Answer: XPath is a locator strategy used to identify elements based on their DOM structure, attributes, and text.

11. XPath Union

The XPath union operator | allows you to combine multiple XPath expressions.

Example:

```
await page.locator(
  `//span[contains(@class,
'spinner__loading')]|//div[@id='confirmation']`
).waitFor();
```

Meaning: Wait for either:

- Loading spinner, or
- Confirmation dialog.

Interview Question 10: What is the use of the XPath union operator?

Answer: The | operator combines multiple XPath expressions and matches elements selected by either expression.

12. Label to Form Control Retargeting

Playwright can automatically identify the input associated with a label.

Example HTML:

```
<label for="password">Password:</label>
<input id="password" type="password">
```

Code:

```
await page.getByText('Password').fill('secret');
```

Even though we locate the Password label, Playwright fills the associated input field.

Other supported actions include:

- `click()`
- `fill()`
- `inputValue()`

- `selectText()`
- `setInputFiles()`
- `selectOption()`

However, assertions such as `toHaveText()` target the label itself.

```
await expect(page.locator('label')).toHaveText('Password');
```

Interview Question 11: What is label-to-control retargeting?

Answer: It is a Playwright feature that allows us to locate a form label and perform input-related actions on its associated form control automatically.

Best practice: Prefer `getByLabel()` for form controls.

```
await page.getByLabel('Password').fill('secret');
```

13. Legacy Text Locator

The legacy text locator is an older way to locate elements using text.

Example:

```
await page.locator('text=Log in').click();
```

It supports three matching approaches.

A. Partial text matching

```
await page.locator('text=Log').click();
```

Matches `Log` inside `Log in`.

B. Exact text matching

```
await page.locator('text="Log in"').click();
```

Matches exact text with case sensitivity.

C. Regular expression matching

```
await page.locator('text=/Log\\s*in/i').click();
```

Matches Login or Log in, ignoring case.

Important: Playwright recommends modern text locators like `getByText()` instead of legacy `text=`.

Interview Question 12: What is a legacy text locator?

Answer: It is an older Playwright locator syntax used to identify elements using their text content. Modern methods like `getByText()` are recommended.

14. ID, data-testid, data-test-id, data-test Selectors

Playwright supports shorthand selectors for certain attributes.

Supported attributes:

- `id`
- `data-testid`
- `data-test-id`
- `data-test`

Examples:

Using ID:

```
await page.locator('id=username').fill('Komal');
```

Using data-test-id:

```
await page.locator('data-test-id=submit').click();
```

Using data-testid:

```
await page.locator('data-testid=login').click();
```

Important: These shorthand selectors are not CSS selectors. For CSS-specific features, use proper CSS syntax.

Example:

```
await page.locator('css=[data-test="login"]:enabled').click();
```

Interview Question 13: What is data-testid in Playwright?

Answer: `data-testid` is a custom HTML attribute used to identify elements specifically for automation testing. It provides stable locators that are less affected by UI changes.

15. Chaining Selectors

Chaining selectors means combining multiple selectors to locate an element step by step.

Example:

```
await page.locator(
  'css=article >> css=.bar > .baz >> css=span'
).click();
```

Meaning:

1. Find an article.
2. Inside the article, find an element matching `.bar > .baz`.
3. Inside that element, find a span.
4. Click the span.

Modern recommended approach:

```
await page.locator('article')
    .locator('.bar > .baz')
    .locator('span')
    .click();
```

Interview Question 14: What is selector chaining?

Answer: Selector chaining means combining multiple locators where each next locator searches inside the result of the previous locator.

16. Intermediate Matches

By default, chained selectors return the element matched by the last selector.

If you want to capture an intermediate element, use * before that selector.

Example:

```
css=article >> text=Hello
```

This selects the element containing Hello.

But:

```
*css=article >> text=Hello
```

This selects the article element containing the Hello element.

Interview Question 15: What is an intermediate match?

Answer: Intermediate matching allows us to select an element found earlier in a chained selector instead of selecting only the final matching element.

Important Interview Questions – Quick Revision

These are additional questions you should prepare from this section.

Q1. Which locators are recommended in Playwright?

Answer: `getByRole()`, `getByText()`, `getByLabel()`, and `getByTestId()`.

Q2. Does CSS selector pierce Shadow DOM in Playwright?

Answer: Yes, Playwright CSS locators can pierce open Shadow DOM. XPath cannot.

Q3. What is the difference between CSS and XPath?

Answer: CSS locators identify elements using tags, classes, IDs, and attributes. XPath can identify elements using DOM relationships, attributes, and text.

Q4. What is strictness violation?

Answer: It occurs when an action that requires a single element matches multiple elements.

Q5. What is the difference between `filter()` and `locator()`?

Answer:

- `locator()` finds elements using a selector.

- `filter()` narrows down existing locator results based on conditions such as text or child elements.

Q6. Which locator would you use if multiple buttons have the same name?

Answer: I would first try to identify the button using a parent locator or a unique attribute. If necessary, I can use `nth()`.

Q7. Can we use XPath to locate parent elements?

Answer: Yes. We can use `xpath=..` to locate the immediate parent element.

Q8. Why should we avoid layout selectors?

Answer: Because they depend on the physical position of elements. Small UI changes can cause tests to identify the wrong element.

Most Important Topics for Interviews

Priority for QA Automation interviews

High

Your revision progress

8/8 completed

CSS vs XPath

Must know

getByRole() vs getByText()

Must know

nth() vs nth-match()

Must know

filter() for parent locator

Must know

CSS :has-text(), :visible, :has()

Important

data-testid

Important

Selector chaining

Important

Label retargeting

Additional

Interview tip: Don't just memorize the syntax. Be prepared to explain why you choose a particular locator and how you handle multiple matching elements in a real-time automation project.

Playwright – Pages

Let's understand the Pages topic in simple language with practical examples and important interview questions and answers.

1. Introduction to Pages

In Playwright, a Page represents a single browser tab or popup window.

We use a Page to:

- Open a website.
- Navigate from one URL to another.
- Click buttons and links.
- Enter data into text fields.
- Read page content.
- Perform automation testing.

Understand Browser, BrowserContext and Page

Modern Web Testing with Playwright | Automation Panda

Term	Meaning
Browser	The entire browser, such as Chrome
BrowserContext	An isolated browser session
Page	A single tab or popup inside a context

Example:

```
// Create a new page
const page = await context.newPage();

// Open website
await page.goto('https://example.com');

// Enter text
await page.locator('#search').fill('Playwright');

// Click button
await page.locator('#submit').click();

// Get current URL
console.log(page.url());
```

Explanation:

1. `context.newPage()` – Creates a new browser tab.
2. `page.goto()` – Opens the website.
3. `locator().fill()` – Enters text into the input field.
4. `locator().click()` – Clicks the button.
5. `page.url()` – Returns the current URL.

Interview Q1. What is a Page in Playwright?

Answer: A Page represents a single browser tab or popup window. We use it to navigate websites and interact with web elements.

Interview Q2. How do you create a new page in Playwright?

Answer:

```
const page = await context.newPage();
```

It creates a new page inside the existing BrowserContext.

2. Multiple Pages

A single BrowserContext can contain multiple pages (tabs).

For example, suppose you open:

- Tab 1 – Google
- Tab 2 – Amazon
- Tab 3 – Flipkart

All three tabs can belong to the same BrowserContext.

Example:

```
// Create two pages
const pageOne = await context.newPage();
const pageTwo = await context.newPage();

// Open websites
await pageOne.goto('https://www.google.com');
await pageTwo.goto('https://www.amazon.com');

// Get all pages
const allPages = context.pages();

console.log(allPages.length);
```

Output:

2

Important points

1. Each page works independently.

2. You don't need to bring a page to the front to interact with it.
3. All pages inside the same context share context-level settings.
4. These settings include viewport size, browser locale and network routes.

Interview Q3. Can one BrowserContext have multiple pages?

Answer: Yes. A single BrowserContext can contain multiple tabs or pages. We can create them using `context.newPage()`.

Interview Q4. How do you get all open pages in a BrowserContext?

Answer:

```
const allPages = context.pages();
```

It returns an array containing all currently open pages in that context.

Interview Q5. Do we need to bring a page to the front before interacting with it?

Answer: No. Playwright can interact with pages without bringing them to the front.

3. Handling New Pages

Sometimes, when we click a link or button, a website opens a new tab.

For example:

- Open Amazon.
- Click a product link.
- A new tab opens with product details.

We need to handle that newly opened page.

Playwright provides:

```
context.waitForEvent('page')
```

This waits for a new page to open inside the BrowserContext.

Example

```
// Start waiting for new page
const pagePromise = context.waitForEvent('page');

// Click link that opens a new tab
await page.getByText('open new tab').click();

// Get newly opened page
const newPage = await pagePromise;

// Interact with new page
await newPage.getByRole('button').click();

// Get page title
console.log(await newPage.title());
```

Explanation step by step

1. `context.waitForEvent('page')` – Starts waiting for a new tab.
2. Click the link that opens the new tab.
3. `await pagePromise` – Waits until the new page is created and returns it.
4. `newPage` – Stores the newly opened page.
5. We can now perform actions on that new tab.

Important interview concept

Why do we write `context.waitForEvent('page')` before clicking the link?

Because the new page may open immediately after clicking. If we start waiting afterward, we may miss the event.

Handling unknown new pages

Sometimes, we don't know exactly which action will open a new tab.

In that case, we can use an event listener.

```
context.on('page', async page => {
```

```
    await page.waitForLoadState();

    console.log(await page.title());

});
```

Explanation:

- `context.on('page')` – Listens whenever a new page is created.
- `waitForLoadState()` – Waits for the page to reach its load state.
- `page.title()` – Gets the title of the newly opened page.

Interview Q6. How do you handle a new tab in Playwright?

Answer: We can use `context.waitForEvent('page')` to wait for a new page and then interact with it.

Interview Q7. What is the difference between `context.waitForEvent('page')` and `context.on('page')`?

Answer:

waitForEvent('page')	**on('page')**
Waits for a particular event	Continuously listens for events
Returns the newly opened page	Executes a callback when a page opens
Useful for a specific action	Useful for monitoring multiple new pages

4. Handling Popups

A popup is a new browser window or tab opened by a particular page.

For example:

- Click Login with Google.

- Click a payment link.
- Click a link that opens an external website in a new window.

Playwright provides:

`page.waitForEvent('popup')`

This waits for a popup opened by the current page.

Example

```
// Start waiting for popup
const popupPromise = page.waitForEvent('popup');

// Click link that opens popup
await page.getByText('open the popup').click();

// Get popup page
const popup = await popupPromise;

// Click button inside popup
await popup.getByRole('button').click();

// Get popup title
console.log(await popup.title());
```

Explanation

1. `page.waitForEvent('popup')` – Waits for a popup opened by the current page.
2. Click the link that opens the popup.
3. `await popupPromise` – Gets the popup reference.
4. `popup.getByRole()` – Interacts with elements inside the popup.
5. `popup.title()` – Gets the popup's title.

Handling unknown popups

If we don't know when a popup will open, we can use:

```
page.on('popup', async popup => {

    await popup.waitForLoadState();

    console.log(await popup.title());

});
```

This listens for popups opened by the current page.

Interview Q8. How do you handle popups in Playwright?

Answer: We use `page.waitForEvent('popup')` to wait for a popup opened by the current page. After getting the popup reference, we can interact with it like a normal page.

5. Difference Between New Page and Popup

This is one of the most important interview questions from this topic.

BrowserContext page event	Page popup event
`context.waitForEvent('page')`	`page.waitForEvent('popup')`
Detects pages opened anywhere in the same context	Detects popups opened by a particular page
Works at BrowserContext level	Works at Page level
Can detect new tabs and popups	Detects popups associated with that page

Interview Q9. What is the difference between page and popup events?

Answer: Both events help handle newly opened pages.

- The page event is available on BrowserContext and detects new pages created anywhere in that context.

- The popup event is available on Page and detects new pages opened by that particular page.

6. Real-Time Interview Coding Examples

Scenario 1: Open a new tab and get its title

```
const newTabPromise = context.waitForEvent('page');

await page.getByText('Open New Tab').click();

const newTab = await newTabPromise;

await newTab.waitForLoadState();

console.log(await newTab.title());
```

Scenario 2: Open a popup and enter username

```
const popupPromise = page.waitForEvent('popup');

await page.getByText('Login').click();

const popup = await popupPromise;

await popup.locator('#username').fill('Komal');
```

Scenario 3: Open three tabs and count them

```
const page1 = await context.newPage();
const page2 = await context.newPage();
const page3 = await context.newPage();

console.log(context.pages().length);
```

Output:

3

7. Important Interview Questions – Quick Revision

Question	Short answer
What is Page?	A single browser tab or popup
What is BrowserContext?	An isolated browser session
Can one context have multiple pages?	Yes
How to create a page?	`context.newPage()`
How to get all pages?	`context.pages()`
How to get current URL?	`page.url()`
How to get page title?	`page.title()`
How to handle new tabs?	`context.waitForEvent('page')`
How to handle popups?	`page.waitForEvent('popup')`
How to monitor new pages?	`context.on('page')`
How to monitor popups?	`page.on('popup')`

8. Test Your Understanding

Try these interview questions yourself.

Pages – MCQ Practice

4 Questions

Q1. What does context.newPage() do?

Creates a new browser
Creates a new tab inside the context
Closes the current page

Correct!context.newPage() creates a new Page (tab) inside the existing BrowserContext.

Q2. Which event is used to handle a popup opened by a particular page?

context.waitForEvent('page')
page.waitForEvent('popup')
browser.waitForEvent('popup')

Correct!page.waitForEvent('popup') listens for a popup opened by that specific page.

Q3. What does context.pages() return?

All browser contexts
All open pages in the context
Only popup pages

Correct!It returns an array of all currently open pages in that BrowserContext.

Q4. Why should we start waiting for a new page before clicking a link?

To close the page
To avoid missing the page event
To refresh the browser

Correct!The new page can open immediately after the click, so we start waiting first.

4/4 Correct

Retry Quiz

My interview tip: Focus especially on `BrowserContext`, `Page`, `context.waitForEvent('page')`, and `page.waitForEvent('popup')`. These are frequently asked concepts in Playwright QA Automation interviews.

Playwright – Page Object Model (POM)

This is a very important topic for QA Automation interviews, especially when you have experience with Selenium POM and Playwright.

Let's understand it in simple language, with examples, code explanations, and interview questions and answers.

1. Introduction to Page Object Model (POM)

POM stands for Page Object Model.

It is a design pattern used in automation testing to organize test code and make it reusable and maintainable.

In POM, we create a separate class for each page of a web application.

For example, consider an e-commerce application:

The Path to Purchase: eCommerce UX Best Practices | Toptal®

We can create separate page classes:

Web application page	Page class
Home Page	`HomePage.ts`
Login Page	`LoginPage.ts`
Product Page	`ProductPage.ts`
Cart Page	`CartPage.ts`
Checkout Page	`CheckoutPage.ts`

Each class contains:

1. Locators – To identify web elements.

2. Methods – To perform actions on those elements.

Simple example:

Suppose you want to automate login functionality.

Without POM, you might write all locators and actions directly inside the test file.

With POM:

- Create `LoginPage.ts` to store username, password and login button locators.
- Create methods such as `login()`.
- Call the `login()` method from your test file.

This makes your code reusable and easier to maintain.

Advantages of POM

1. Code Reusability

We can reuse the same methods in multiple test cases.

2. Easy Maintenance

If a locator changes, we update it in one page class instead of updating multiple test files.

3. Code Readability

Test files become shorter and easier to understand.

4. Easy Test Management

We can maintain separate classes for different application pages.

Interview Q1. What is Page Object Model in Playwright?

Answer: Page Object Model is a design pattern used in automation testing. It creates separate classes for application pages and stores their locators and actions in those classes. It improves code reusability, readability and maintainability.

Interview Q2. What are the advantages of POM?

Answer:

- Code reusability.
- Easy maintenance.
- Better readability.
- Reduced code duplication.
- Better test organization.

2. Implementation of POM in Playwright

Now let's understand the code given in the documentation.

We are creating a helper class called `PlaywrightDevPage` to automate the Playwright website.

Step 1: Create a Page Object Class

File name: `playwright-dev-page.ts`

```
import { expect, type Locator, type Page } from '@playwright/test';
```

Explanation:

We import three things:

Import	Purpose
`expect`	Used for assertions
`Locator`	TypeScript type for web element locators
`Page`	TypeScript type representing a browser page

Interview Q3. Why do we import Page and Locator in POM?

Answer: `Page` represents the browser page, and `Locator` represents web elements. We use them to define and manage page objects with TypeScript type checking.

Step 2: Create a Class

```
export class PlaywrightDevPage {
```

Explanation:

- `class` creates a class.
- `PlaywrightDevPage` is the class name.
- `export` allows us to use this class in other files.

Step 3: Declare Page and Locators

```
readonly page: Page;

readonly getStartedLink: Locator;

readonly gettingStartedHeader: Locator;

readonly pomLink: Locator;

readonly tocList: Locator;
```

Explanation:

Here we declare one Page object and four locators.

Variable	Purpose
`page`	Stores the browser page
`getStartedLink`	Locates Get Started link
`gettingStartedHeader`	Locates Installation heading
`pomLink`	Locates Page Object Model link
`tocList`	Locates table of contents links

What is `readonly`?

`readonly` means the property cannot be reassigned after initialization.

For example:

```
readonly page: Page;
```

We can assign the page in the constructor, but we cannot later assign a different page to this property.

Step 4: Constructor

```
constructor(page: Page) {

    this.page = page;

    this.getStartedLink = page.locator('a', {
        hasText: 'Get started'
    });

    this.gettingStartedHeader = page.locator('h1', {
        hasText: 'Installation'
    });

    this.pomLink = page.locator('li', {
        hasText: 'Guides',
    }).locator('a', {
        hasText: 'Page Object Model',
    });

    this.tocList = page.locator(
        'article div.markdown ul > li > a'
    );

}
```

What is a constructor?

A constructor is a special method that executes automatically when we create an object of a class.

In this example, the constructor receives the Playwright `page` object and initializes all the locators.

Let's understand each part.

1. `this.page = page;`

Stores the received browser page inside the class.

2. `getStartedLink`

```
this.getStartedLink = page.locator('a', {
    hasText: 'Get started'
});
```

Finds an anchor (a) element containing the text Get started.

3. `gettingStartedHeader`

```
this.gettingStartedHeader = page.locator('h1', {
    hasText: 'Installation'
});
```

Finds an H1 heading containing Installation.

4. `pomLink`

```
this.pomLink = page.locator('li', {
    hasText: 'Guides',
}).locator('a', {
    hasText: 'Page Object Model',
});
```

This uses chained locators.

First, it finds a list item containing Guides.

Then, inside that list item, it finds the anchor containing Page Object Model.

5. `tocList`

```
this.tocList = page.locator(
    'article div.markdown ul > li > a'
);
```

Locates all anchor elements inside the specified article table of contents structure.

Interview Q4. What is a constructor in POM?

Answer: A constructor is a special method that executes automatically when an object is created. In Playwright POM, we generally pass the Page object through the constructor and initialize the locators.

Interview Q5. What is the use of `this` keyword?

Answer: The `this` keyword refers to the current class object. It is used to access the properties and methods of that class.

3. Create Methods for Page Actions

Now we create methods inside the class to perform actions.

Method 1: goto()

```
async goto() {
    await this.page.goto('https://playwright.dev');
}
```

Explanation:

- `async` allows asynchronous operations.
- `await` waits for navigation to complete according to Playwright's navigation behavior.
- `this.page.goto()` opens the website.

Method 2: getStarted()

```
async getStarted() {

    await this.getStartedLink.first().click();

    await expect(this.gettingStartedHeader).toBeVisible();

}
```

Explanation:

1. `this.getStartedLink` accesses the locator declared in the constructor.
2. `.first()` selects the first matching element.

3. `.click()` clicks the link.
4. `expect().toBeVisible()` checks whether the Installation heading is visible.

Why do we use `first()`?

Because the locator might match multiple elements. We use `first()` to select the first matching element.

Method 3: pageObjectModel()

```
async pageObjectModel() {

    await this.getStarted();

    await this.pomLink.click();

}
```

Explanation:

1. Calls the existing `getStarted()` method.
2. Clicks the Page Object Model link.

Notice that we are reusing an existing method instead of writing the same steps again.

This is called code reusability.

Interview Q6. Why do we create methods inside a Page Object class?

Answer: We create methods to encapsulate application actions such as login, search, navigation and checkout. This allows us to reuse the same functionality across multiple test cases.

4. Complete Page Object Class

Here is the complete implementation:

```
import { expect, type Locator, type Page } from '@playwright/test';

export class PlaywrightDevPage {
```

```
readonly page: Page;
readonly getStartedLink: Locator;
readonly gettingStartedHeader: Locator;
readonly pomLink: Locator;
readonly tocList: Locator;

constructor(page: Page) {

    this.page = page;

    this.getStartedLink = page.locator('a', {
        hasText: 'Get started'
    });

    this.gettingStartedHeader = page.locator('h1', {
        hasText: 'Installation'
    });

    this.pomLink = page.locator('li', {
        hasText: 'Guides'
    }).locator('a', {
        hasText: 'Page Object Model'
    });

    this.tocList = page.locator(
        'article div.markdown ul > li > a'
    );
}

async goto() {
    await this.page.goto('https://playwright.dev');
}

async getStarted() {
    await this.getStartedLink.first().click();
    await expect(this.gettingStartedHeader).toBeVisible();
}

async pageObjectModel() {
    await this.getStarted();
    await this.pomLink.click();
}
```

```
}
```

5. Using POM in Test Files

After creating the page object class, we can use it in our test files.

File name: `example.spec.ts`

```
import { test, expect } from '@playwright/test';

import { PlaywrightDevPage } from './playwright-dev-page';
```

Explanation:

We import:

- `test` – To write test cases.
- `expect` – To perform assertions.
- `PlaywrightDevPage` – Our page object class.

Test Case 1: Verify Table of Contents

```
test('getting started should contain table of contents',
async ({ page }) => {

    const playwrightDev = new PlaywrightDevPage(page);

    await playwrightDev.goto();

    await playwrightDev.getStarted();

    await expect(playwrightDev.tocList).toHaveText([
        'How to install Playwright',
        "What's installed",
        'How to run the example test',
        'How to open the HTML test report',
        'Write tests using web-first assertions, fixtures and
locators',
        'Run single or multiple tests; headed mode',
        'Generate tests with Codegen',
```

```
        'View a trace of your tests'
    ]);

});
```

Explanation step by step

1. Create an object

```
const playwrightDev = new PlaywrightDevPage(page);
```

Creates an object of `PlaywrightDevPage` and passes the current browser page to its constructor.

2. Open website

```
await playwrightDev.goto();
```

Calls the `goto()` method from the page object class.

3. Click Get Started

```
await playwrightDev.getStarted();
```

Calls the method that clicks Get Started and verifies the Installation heading.

4. Verify table of contents

```
await expect(playwrightDev.tocList).toHaveText([...]);
```

Checks whether the table of contents contains the expected list of text values.

What is `toHaveText()`?

It is a Playwright assertion used to verify the text content of an element or a collection of elements.

Test Case 2: Verify Page Object Model Article

```
test('should show Page Object Model article',
async ({ page }) => {

    const playwrightDev = new PlaywrightDevPage(page);

    await playwrightDev.goto();
```

```
    await playwrightDev.pageObjectModel();

    await expect(page.locator('article'))
        .toContainText('Page Object Model is a common pattern');

});
```

Explanation:

1. Create an object of `PlaywrightDevPage`.
2. Open the Playwright website.
3. Navigate to the Page Object Model section.
4. Verify that the article contains the expected text.

6. Understand the POM Project Structure

A typical Playwright POM framework can look like this:

PlaywrightProject

pages

LoginPage.ts

HomePage.ts

CartPage.ts

tests

login.spec.ts

cart.spec.ts

playwright.config.ts

package.json

- `pages` – Contains page classes, locators and methods.
- `tests` – Contains test cases and assertions.

- `playwright.config.ts` – Contains test configuration.
- `package.json` – Contains dependencies and execution scripts.

7. Selenium POM vs Playwright POM

Since both Selenium and Playwright use the Page Object Model, this is useful for interviews.

Selenium POM	Playwright POM
Usually uses Java classes	Often uses TypeScript classes
Uses WebDriver	Uses Page
Uses WebElement	Uses Locator
Uses `findElement()`	Uses `page.locator()`
Uses `@FindBy` with PageFactory	Usually initializes locators in constructor
Uses TestNG/JUnit assertions	Uses Playwright `expect()`

Interview Q7. Is POM in Selenium and Playwright the same?

Answer: The basic concept is the same. Both use separate classes for application pages and store locators and actions in those classes. However, their implementation and APIs are different.

8. Most Important Interview Questions and Answers

Q8. What is the difference between POM and a normal test script?

Answer: In a normal test script, locators and actions are written directly inside the test. In POM, we separate locators and page actions into reusable classes, making tests easier to maintain.

Q9. Why do we pass the Page object through the constructor?

Answer: We pass the Page object so that the page class can access the browser tab and perform actions on its elements.

Q10. Can we use one page class in multiple test cases?

Answer: Yes. We can create and reuse the same page class in multiple test cases.

Q11. What is the difference between Locator and Page?

Answer:

- Page represents a browser tab.
- Locator represents a web element or a collection of matching elements on that page.

Q12. What happens if a locator changes in the application?

Answer: We update the locator in the corresponding page class instead of changing it in every test case.

Q13. Should we write assertions inside page classes?

Answer: Generally, test-specific assertions are written in test files, while page classes contain locators and reusable actions. However, page classes can also contain validations when they are part of a reusable page operation, as shown in the documentation.

Q14. Is POM a framework or a design pattern?

Answer: POM is a design pattern. It is used to structure an automation framework.

Q15. What is code reusability in POM?

Answer: Code reusability means creating a method once and using it in multiple test cases instead of writing the same steps repeatedly.

9. Test Your Understanding – Interview MCQs

POM Interview Practice

6 MCQs

Q1. What is the full form of POM?

Page Object Model
Playwright Object Method
Page Operation Model

Correct!POM stands for Page Object Model.

Q2. Where do we generally store locators in POM?

package.json
Page class
playwright.config.ts

Correct!Locators are generally declared and initialized in the corresponding page class.

Q3. What is the purpose of the constructor?

To execute test cases
To initialize the page object and locators
To close the browser

Correct!The constructor receives the Page object and initializes the class properties.

Q4. Which keyword refers to the current class object?

super
static
this

Correct!this refers to the current class instance.

Q5. Is POM a design pattern or a testing tool?

Design pattern
Testing tool
Browser

Correct!POM is a design pattern used to organize automation code.

Q6. What is the main advantage of POM?

Increases code duplication
Improves maintainability and reusability
Removes the need for locators

Correct!POM centralizes locators and reusable actions, making maintenance easier.

6/6

Retry

Final interview tip: Be ready to explain how you implemented POM in your own automation project. A good answer should cover separate page classes, constructor, locators, reusable methods, and how test files call those methods.

For your Selenium-to-Playwright preparation, pay special attention to the difference between Java Selenium POM and TypeScript Playwright POM.

Playwright – Screenshots

The Screenshot feature in Playwright allows us to capture images of a webpage or a specific element during test execution.

In automation testing, screenshots are useful for:

- Debugging test failures.
- Identifying UI issues.
- Verifying the appearance of a webpage.
- Maintaining evidence of failed test cases.
- Comparing actual and expected UI designs.

Let's understand each concept with examples.

1. Basic Screenshot

Code:

```
await page.screenshot({ path: 'screenshot.png' });
```

Explanation:

Code	Meaning
`page`	Represents the browser page or tab.
`screenshot()`	Captures the screenshot of the page.
`path`	Specifies where to save the screenshot.
`'screenshot.png'`	Name of the screenshot file.
`await`	Waits until the screenshot operation completes.

Example:

```
test('Capture homepage screenshot', async ({ page }) => {
  await page.goto('https://example.com');

  await page.screenshot({ path: 'homepage.png' });
});
```

Step-by-step:

1. Open the website.
2. Wait for the navigation to complete.
3. Capture the visible page.
4. Save the image as `homepage.png`.

Interview point

By default, `page.screenshot()` captures the currently visible viewport, not the entire scrollable webpage.

2. Full Page Screenshot

A full-page screenshot captures the entire scrollable webpage, including content below the visible screen.

Code:

```
await page.screenshot({
  path: 'screenshot.png',
  fullPage: true
});
```

Important parameter:

`fullPage: true` tells Playwright to capture the complete page instead of only the visible viewport.

Example:

```
test('Capture full page screenshot', async ({ page }) => {
  await page.goto('https://example.com');

  await page.screenshot({
    path: 'fullpage.png',
    fullPage: true
  });
});
```

Difference:

HTML Course | Building Header of the Website - GeeksforGeeks

Normal screenshot

Captures only the visible viewport.

A complete guide on how to take full page screenshots with Puppeteer, Playwright or Selenium

Full-page screenshot

Captures the entire scrollable page.

Practical QA example:

Suppose an e-commerce website has:

- Header
- Product categories
- Product listings
- Footer

If you want to capture all these sections in one image, use `fullPage: true`.

3. Capture Screenshot into Buffer

Normally, screenshots are saved directly into a file using the `path` parameter.

But sometimes, we don't want to save the screenshot as a file. Instead, we want to store the screenshot image temporarily in memory.

For this, Playwright provides a Buffer.

Code:

```
const buffer = await page.screenshot();

console.log(buffer.toString('base64'));
```

Explanation:

Code	Meaning
`page.screenshot()`	Captures the screenshot.
`buffer`	Stores the screenshot image data in memory.
`toString('base64')`	Converts the image data into a Base64-encoded string.
`console.log()`	Prints the Base64 string in the console.

What is a Buffer?

A Buffer is a temporary memory area used to store binary data.

Images, PDFs and other files can be represented as binary data.

What is Base64?

Base64 is a method of converting binary data into a text format.

For example:

- Screenshot image → Buffer
- Buffer → Base64 string

Why use Buffer?

- To send screenshots to third-party tools.
- To perform image comparison.
- To process images without saving them to disk.
- To attach screenshots to custom reports.

Interview point

When you call `page.screenshot()` without specifying `path`, Playwright returns a Buffer containing the screenshot data.

4. Element Screenshot

Sometimes we don't need a screenshot of the entire webpage. We only need to capture a particular element.

For example:

- Login form
- Header
- Product image
- Shopping cart
- Error message

Playwright allows us to capture an individual element using `locator().screenshot()`.

Code:

```
await page.locator('.header').screenshot({
  path: 'screenshot.png'
});
```

Explanation:

Code	Meaning
`page.locator()`	Locates an element on the webpage.
`'.header'`	CSS selector identifying the header element.
`screenshot()`	Captures only that element.
`path`	Saves the screenshot to the specified file.

Practical example:

```
test('Capture product screenshot', async ({ page }) => {
  await page.goto('https://example.com');

  const product = page.locator('.product-card');

  await product.screenshot({
    path: 'product.png'
  });
});
```

Step-by-step:

1. Open the website.
2. Locate the product card.
3. Capture only the product card.

4. Save it as `product.png`.

Important: The element must be visible for its screenshot to be captured. If the element is outside the viewport, Playwright can scroll it into view before capturing it.

5. Important Screenshot Methods – Quick Revision

Method	Purpose
`page.screenshot()`	Captures the visible page.
`page.screenshot({ fullPage: true })`	Captures the entire scrollable page.
`page.screenshot({ path: 'file.png' })`	Saves screenshot to a file.
`page.screenshot()` without path	Returns screenshot as Buffer.
`locator.screenshot()`	Captures a specific element.

6. Practical QA Automation Example

Scenario: Capture a screenshot when a login test fails.

```
import { test, expect } from '@playwright/test';

test('Login test', async ({ page }) => {
  await page.goto('https://example.com/login');

  try {
    await page.getByRole('button', {
      name: 'Login'
    }).click();

    await expect(page).toHaveURL(/dashboard/);

  } catch (error) {
```

```
    await page.screenshot({
      path: 'login-failure.png',
      fullPage: true
    });

    throw error;
  }
});
```

How does it work?

1. Opens the login page.
2. Clicks the Login button.
3. Checks whether the user reaches the dashboard.
4. If an error occurs, captures a full-page screenshot.
5. Throws the error again so the test is marked as failed.

Real-world interview tip: In Playwright Test, you can also use the built-in screenshot configuration:

```
use: {
  screenshot: 'only-on-failure'
}
```

This automatically captures screenshots when tests fail. Other options include:

- `'off'` – No automatic screenshots.
- `'on'` – Capture screenshots for every test.
- `'only-on-failure'` – Capture screenshots only for failed tests.

7. Selenium vs Playwright – Screenshot Comparison

Selenium	Playwright
`getScreenshotAs()`	`page.screenshot()`

`TakesScreenshot` interface	No separate interface required
Element screenshot supported	`locator.screenshot()`
Full-page capture may require additional handling	`fullPage: true`

Selenium example:

```
File screenshot = ((TakesScreenshot) driver)
    .getScreenshotAs(OutputType.FILE);
```

Playwright example:

```
await page.screenshot({ path: 'screenshot.png' });
```

8. Interview Questions and Answers

Q1. What is a screenshot in Playwright?

A: A screenshot is an image captured from a webpage or a particular element during test execution. It helps in debugging and identifying UI issues.

Q2. How do you take a screenshot in Playwright?

A:

```
await page.screenshot({ path: 'screenshot.png' });
```

Q3. How do you capture a full-page screenshot?

A:

```
await page.screenshot({
  path: 'fullpage.png',
  fullPage: true
});
```

Q4. What is the difference between a normal screenshot and a full-page screenshot?

A: A normal screenshot captures the visible viewport, whereas a full-page screenshot captures the entire scrollable webpage.

Q5. What is a Buffer in Playwright?

A: A Buffer stores screenshot image data in memory instead of directly saving it to a file.

Q6. How do you capture a screenshot of a particular element?

A:

```
await page.locator('.header').screenshot({
  path: 'header.png'
});
```

Q7. Can we capture screenshots without saving them to a file?

A: Yes. Calling `page.screenshot()` without the `path` option returns a Buffer.

Q8. How can you automatically capture screenshots when a test fails?

A: Configure the screenshot option in `playwright.config.ts`:

```
use: {
  screenshot: 'only-on-failure'
}
```

Q9. Can Playwright capture screenshots for image comparison?

A: Yes. We can capture screenshots as Buffers and pass them to image comparison or visual testing tools.

Q10. What is the difference between `page.screenshot()` and `locator.screenshot()`?

A:

- `page.screenshot()` captures the page.
- `locator.screenshot()` captures a specific element.

9. MCQ Practice

Test your understanding before moving to the next Playwright topic.

Playwright Screenshots Quiz

5/5 answered

1. Which method captures a screenshot of the webpage?

page.capture()
page.screenshot()
page.image()
browser.screenshot()

Correct! page.screenshot() is the Playwright API for capturing a page screenshot.

2. Which option captures the entire scrollable webpage?

fullPage: false
entirePage: true
fullPage: true
scroll: true

Incorrect. fullPage: true captures the complete scrollable page.

3. What does page.screenshot() return when path is not provided?

String
File
Buffer
Locator

Incorrect. It returns a Buffer containing the screenshot image data.

4. Which method captures only a particular element?

page.locator().screenshot()
page.elementScreenshot()
browser.screenshot()
page.captureElement()

Correct! The locator screenshot method captures the selected element.

5. Which configuration captures screenshots only when tests fail?

screenshot: 'on'
screenshot: 'off'
screenshot: 'only-on-failure'
screenshot: 'always-fail'

Correct! only-on-failure automatically captures screenshots for failed tests.

3/5 correct

Try again

10. Important Points to Remember

1. `page.screenshot()` captures the visible page.
2. `fullPage: true` captures the entire scrollable page.
3. `path` specifies where to save the screenshot.
4. Without `path`, the screenshot is returned as a Buffer.
5. `locator.screenshot()` captures a specific element.
6. `screenshot: 'only-on-failure'` captures screenshots automatically for failed tests.

Next topic: Share the next Playwright documentation section, and I'll explain it in the same format with practical examples and interview questions.

Playwright – Service Workers

1. Introduction

A Service Worker is a JavaScript file that runs in the background of a browser and acts like a middleman between a web application and the network.

It can intercept network requests and help a website:

- Cache files such as HTML, CSS, JavaScript and images.
- Load previously cached content.
- Provide offline functionality.
- Improve website performance.
- Handle network requests independently of the webpage.

Simple real-life example

Imagine you visit an e-commerce website.

1. You open the website for the first time.
2. The browser downloads images, CSS, JavaScript and other files.
3. A Service Worker can cache some of these files.
4. You visit the website again, but your internet connection is unavailable.
5. The Service Worker may serve cached files so parts of the website can still work.

Técnicas comuns para criar aplicativos off-line | web.dev
web.dev

How does it work?

A Service Worker acts as a proxy between:

- Webpage (browser)
- External network (server)
- Cache storage

Important interview point

Service Workers are supported by Playwright only in Chromium-based browsers, according to this Playwright documentation section.

2. Why are Service Workers used?

Feature	Explanation
Caching	Stores resources for reuse.
Offline support	Can serve cached resources without internet.
Network interception	Can intercept requests using a FetchEvent handler.

Performance	Can reduce repeated network requests.
Background handling	Handles certain operations independently of the webpage.

Important: A Service Worker is not the same as a normal browser page. It runs separately from the page's JavaScript environment.

3. How to Disable Service Workers

Playwright allows us to disable Service Workers during testing.

Why would we disable them?

- To make tests more predictable.
- To avoid unexpected cached responses.
- To simplify network mocking.
- To reduce interference with request routing.

Code – playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    serviceWorkers: 'block'
  },
});
```

Explanation

Code	Meaning
`defineConfig`	Defines Playwright test configuration.
`use`	Contains browser and context options.

`serviceWorkers`	Controls Service Worker behavior.
`'block'`	Prevents Service Workers from being registered or used.

What happens when we set `serviceWorkers: 'block'`?

Playwright blocks Service Workers during testing.

For example, if a website normally uses a Service Worker to return cached product information, blocking it may change how that website behaves.

Interview question: Why would you disable Service Workers in Playwright?

Answer: To make network-related tests more predictable and avoid interference from Service Worker caching or request interception. However, if the application depends on Service Workers, blocking them may change the application's actual behavior.

4. Accessing Service Workers

Playwright provides two important ways to access Service Workers.

A. `browserContext.serviceWorkers()`

This method returns the list of Service Workers currently registered in the browser context.

Example:

```
const workers = context.serviceWorkers();

console.log(workers);
```

Explanation:

- `context` represents the browser context.
- `serviceWorkers()` returns the available Service Worker instances.
- `workers` stores the returned list.

Use this when you want to check which Service Workers are available.

B. `context.waitForEvent('serviceworker')`

Sometimes, you want to wait until a webpage registers a Service Worker.

For this, use the `serviceworker` event.

Code:

```
const serviceWorkerPromise =
  context.waitForEvent('serviceworker');

await page.goto('/example-with-a-service-worker.html');

const serviceworker = await serviceWorkerPromise;
```

Step-by-step explanation:

1. `context.waitForEvent('serviceworker')` starts waiting for a Service Worker event.
2. `page.goto()` opens the webpage.
3. The webpage registers a Service Worker.
4. Playwright detects the registration event.
5. `serviceworker` stores the Service Worker instance.

Why do we create the promise before navigating?

Because the Service Worker might be registered immediately when the page loads. Starting to wait first helps us avoid missing the event.

Interview point

`waitForEvent('serviceworker')` waits for the Service Worker event, while `serviceWorkers()` returns the currently available Service Workers.

5. Waiting for Service Worker Activation

This is an important concept.

When a Service Worker is registered, it does not immediately control the webpage.

It generally goes through these lifecycle states:

Service Worker Lifecycle

Installing

Browser installs the Service Worker.

Installed / Waiting

Waits until it can become active.

Activating

Activation tasks are performed.

Active

Activated

The Service Worker can control pages.

Why do we wait for activation?

Playwright's `browserContext.on('serviceworker')` event fires before the Service Worker has necessarily taken control of the page.

Therefore, before executing JavaScript inside the worker using `worker.evaluate()`, we should wait until activation.

Code from documentation

```
await page.evaluate(async () => {
  const registration =
    await window.navigator.serviceWorker.getRegistration();

  if (registration.active?.state === 'activated')
    return;

  await new Promise(resolve => {
    window.navigator.serviceWorker.addEventListener(
      'controllerchange',
      resolve
    );
  });
});
```

Understand it line by line:

Code	Explanation
`page.evaluate()`	Executes JavaScript inside the webpage.
`navigator.serviceWorker`	Accesses the browser's Service Worker API.
`getRegistration()`	Gets the Service Worker registration for the current page.
`registration.active`	Gets the active Service Worker, if available.
`?.state`	Safely checks the worker's state.
`activated`	Indicates that the worker has reached the activated state.
`controllerchange`	Event fired when the page's controlling Service Worker changes.

What does this condition mean?

```
if (registration.active?.state === 'activated')
  return;
```

If the Service Worker is already activated, there is no need to wait again.

Otherwise, the code waits for the `controllerchange` event.

Interview question: What is the difference between registering and activating a Service Worker?

Answer:

- Registration tells the browser about the Service Worker.
- Activation means the Service Worker has completed its activation lifecycle and can become the page's controller.

Note: The documentation's example is a simplified waiting pattern. In production tests, handle cases where no controller change occurs, such as when registration fails or the page is already controlled.

6. Network Events and Routing

This is one of the most important sections for QA automation interviews.

Service Workers can make their own network requests.

Playwright allows us to monitor those requests through the `BrowserContext` object.

Important events

Event	Purpose
`browserContext.on('request')`	Triggered when a request is made.
`browserContext.on('requestfinished')`	Triggered when a request finishes successfully.
`browserContext.on('response')`	Triggered when a response is received.
`browserContext.on('requestfailed')`	Triggered when a request fails.
`browserContext.route()`	Allows us to intercept and handle routable requests.

Example:

```
context.on('request', request => {
  console.log('URL:', request.url());
});

context.on('response', response => {
  console.log('Status:', response.status());
});
```

Here:

- The first listener logs the requested URL.
- The second listener logs the HTTP response status.

Two important methods

1. `request.serviceWorker()`

This method tells us whether the request was made by a Service Worker.

```
const worker = request.serviceWorker();
```

If the request belongs to a Service Worker, it returns its Service Worker instance. Otherwise, it returns `null`.

2. `response.fromServiceWorker()`

This method tells us whether the response was handled by a Service Worker's fetch handler.

```
const isFromSW = response.fromServiceWorker();

console.log(isFromSW);
```

It returns:

- `true` – Response was handled by a Service Worker.
- `false` – Response was not handled by a Service Worker.

Important difference

`request.serviceWorker()` identifies requests made by a Service Worker.

`response.fromServiceWorker()` identifies responses handled by a Service Worker's fetch handler.

7. Understanding the Transparent Service Worker Example

Let's understand the example from your documentation.

File: transparent-service-worker.js

```
self.addEventListener('fetch', event => {

  // actually make the request
  const responsePromise = fetch(event.request);

  // send it back to the page
  event.respondWith(responsePromise);

});

self.addEventListener('activate', event => {
  event.waitUntil(clients.claim());
});
```

Explanation

Part 1: Fetch event

```
self.addEventListener('fetch', event => {
```

- `self` represents the Service Worker global scope.
- `addEventListener()` registers an event listener.
- `'fetch'` listens for network requests made by the controlled page.
- `event` represents the fetch event.

Part 2: Make the actual network request

```
const responsePromise = fetch(event.request);
```

The Service Worker forwards the original request to the actual network using `fetch()`.

Part 3: Return the response to the page

```
event.respondWith(responsePromise);
```

This tells the browser to use the response from the promise as the response to the original page request.

Part 4: Activation event

```
self.addEventListener('activate', event => {
  event.waitUntil(clients.claim());
});
```

- `'activate'` runs when the Service Worker activates.
- `event.waitUntil()` keeps the activation process pending until the supplied promise completes.
- `clients.claim()` allows the activated Service Worker to take control of eligible pages in its scope without requiring a reload.

What is a transparent Service Worker?

It is a Service Worker that forwards requests to the actual network and returns the response without modifying it.

In simple words, it acts like a proxy.

8. Understanding Network Request Events

Your documentation provides an event table. Let's simplify it.

Suppose:

1. A webpage loads `index.html`.
2. The webpage registers a Service Worker.
3. The webpage requests `data.json`.
4. The Service Worker intercepts that request.
5. The Service Worker forwards it to the network.

The request flow looks like this:

Webpage

Requests data.json

Fetch event

Service Worker

Intercepts the request

fetch(event.request)

External Network / Server

Processes the request and returns a response

Response travels back through the Service Worker

Webpage

Receives the response

Why are there two requests for `data.json`?

This is a frequently confusing concept.

The documentation shows two different requests:

Request owner	URL	Meaning
Frame	`data.json`	Original request made by the webpage.
Service Worker	`data.json`	New request made by the Service Worker using `fetch(event.request)`.

Both requests have the same URL, but they have different owners.

Important routing concept:

- The Service Worker-owned request can be intercepted using `browserContext.route()`.
- The original Frame-owned request is not routable in this example because the Service Worker already handles it through its fetch handler.

This is why you might see multiple request events for the same URL.

9. Important Limitation: `request.frame()` and `response.frame()`

Normally, we can use:

`request.frame();`

to identify which frame initiated a request.

However, when the request is made by a Service Worker, calling `request.frame()` throws an exception.

Why?

Because a Service Worker is not a webpage frame.

The same limitation applies to `response.frame()` for responses associated with requests made by a Service Worker.

Interview question: Why does `request.frame()` throw an exception for Service Worker requests?

Answer: Because a Service Worker operates independently of a webpage frame. A Service Worker-owned request does not have a corresponding frame.

10. Routing Service Worker Requests Only

Suppose you want to intercept only requests made by Service Workers and allow all other requests to continue normally.

Playwright provides this approach.

Code:

```
await context.route('**', async route => {

  if (route.request().serviceWorker()) {

    await route.fulfill({
      contentType: 'text/plain',
      status: 200,
      body: 'from sw',
    });
```

```
  } else {
    await route.continue();
  }
});
```

Explanation

1. Route all matching requests

```
await context.route('**', async route => {
```

- `context.route()` sets up request interception.
- `'**'` matches all URLs.
- `route` represents the intercepted request.

2. Check whether the request belongs to a Service Worker

```
if (route.request().serviceWorker()) {
```

Checks whether the request was made by a Service Worker.

3. Return a mocked response

```
await route.fulfill({
  contentType: 'text/plain',
  status: 200,
  body: 'from sw',
});
```

Parameter	Meaning
`contentType`	Response content type.
`status`	HTTP response status code.
`body`	Response content.

Here, Playwright returns a mocked response instead of allowing the Service Worker-owned request to reach the network.

4. Continue other requests

```
await route.continue();
```

If the request is not from a Service Worker, Playwright allows it to continue normally.

Interview point

Use `route.request().serviceWorker()` to identify Service Worker-owned requests. Do not call `route.request().frame()` for these requests because it throws an exception.

11. Known Limitations

Your documentation mentions one important limitation:

Requests for updated Service Worker main script code cannot currently be routed.

In simple words:

When the browser tries to download an updated version of the Service Worker JavaScript file, Playwright cannot currently intercept that particular update request using routing.

This is a limitation documented in the Playwright guide.

12. Selenium vs Playwright

Feature	Selenium	Playwright
Service Worker handling	Requires browser-specific approaches and APIs	Provides Service Worker APIs
List Service Workers	Browser-specific implementation	`context.serviceWorkers()`

Wait for Service Worker	Browser-specific implementation	`context.waitForEvent('serviceworker')`
Block Service Workers	Browser-specific configuration	`serviceWorkers: 'block'`
Identify SW requests	Browser-specific network tooling	`request.serviceWorker()`
Detect SW responses	Browser-specific network tooling	`response.fromServiceWorker()`

13. Interview Questions and Answers

Q1. What is a Service Worker?

A: A Service Worker is a JavaScript script that runs separately from a webpage and can intercept network requests, manage caching and provide offline functionality.

Q2. Why are Service Workers used?

A: They are used for caching, offline support, network request interception and improving web application performance.

Q3. Does Playwright support Service Workers in all browsers?

A: According to the documented limitation, Service Workers are supported only in Chromium-based browsers.

Q4. How do you disable Service Workers in Playwright?

A:

```
use: {
  serviceWorkers: 'block'
}
```

Q5. How do you get the list of Service Workers?

A:

```
const workers = context.serviceWorkers();
```

Q6. How do you wait for a Service Worker registration event?

A:

```
const workerPromise = context.waitForEvent('serviceworker');

await page.goto('/example.html');

const worker = await workerPromise;
```

Q7. Why should we wait for Service Worker activation?

A: Because the Service Worker registration event can occur before the worker takes control of the page. Waiting for activation helps ensure the worker is ready before interacting with it.

Q8. What is `request.serviceWorker()`?

A: It identifies whether a request was made by a Service Worker. It returns the associated Service Worker instance or `null`.

Q9. What is `response.fromServiceWorker()`?

A: It returns `true` when a response was handled by a Service Worker's fetch handler.

Q10. What is the difference between `request.serviceWorker()` and `response.fromServiceWorker()`?

A:

- `request.serviceWorker()` identifies the owner of the request.
- `response.fromServiceWorker()` identifies whether the response was handled by a Service Worker.

Q11. Why does `request.frame()` throw an exception for Service Worker requests?

A: Because a Service Worker request is not associated with a webpage frame.

Q12. How can you intercept only Service Worker requests?

A: Use `context.route()` and check `route.request().serviceWorker()`.

Q13. What is `clients.claim()`?

A: It allows an activated Service Worker to take control of eligible pages within its scope.

Q14. What is `event.respondWith()`?

A: It tells the browser to use the supplied promise or response as the response to the intercepted fetch request.

Q15. What is the difference between `browserContext.route()` and Service Worker fetch handling?

A: `browserContext.route()` is Playwright's request interception mechanism, whereas a Service Worker's fetch handler is browser-native code that intercepts requests made by controlled pages.

14. Quick Revision – Important Methods

Method / Option	Purpose
`context.serviceWorkers()`	Lists available Service Workers.
`context.waitForEvent('serviceworker')`	Waits for a Service Worker event.
`serviceWorkers: 'block'`	Blocks Service Workers.
`request.serviceWorker()`	Identifies Service Worker-owned requests.
`response.fromServiceWorker()`	Checks whether a response came through a Service Worker fetch handler.
`context.route()`	Intercepts routable network requests.
`route.fulfill()`	Returns a mocked response.
`route.continue()`	Allows a request to continue.

`event.respondWith()`	Supplies a response to a fetch event.
`clients.claim()`	Claims control of eligible pages.

15. MCQ Practice

Service Workers Quiz

7/7 answered

1. Which browser engines support Service Workers in this Playwright documentation?

All browsers
Chromium-based browsers only
Firefox only
WebKit only

Correct! The documentation states that Service Workers are supported only on Chromium-based browsers.

2. How do you disable Service Workers in Playwright?

serviceWorkers: 'off'
serviceWorkers: 'disable'
serviceWorkers: 'block'
serviceWorkers: false

Incorrect. Use serviceWorkers: 'block' in the Playwright configuration.

3. Which method returns the list of available Service Workers?

context.serviceWorkers()
page.serviceWorkers()
browser.workers()
context.getWorkers()

Correct! browserContext.serviceWorkers() returns the available Service Workers.

4. What does request.serviceWorker() identify?

The browser engine
The page URL

Whether the request was made by a Service Worker
The HTTP status

Correct! It returns the associated Service Worker instance or null.

5. What does response.fromServiceWorker() return?

A URL
A Boolean
A Buffer
A Locator

Correct! It returns true or false depending on whether the response was handled by a Service Worker fetch handler.

6. Why can request.frame() throw for Service Worker-owned requests?

The browser is closed
The request has no associated webpage frame
The URL is invalid
The response is empty

Correct! A Service Worker operates independently of webpage frames.

7. Which method provides a mocked response to an intercepted request?

route.continue()
route.fulfill()
route.frame()
route.response()

Correct! route.fulfill() lets you provide a custom response.

6/7 correct

Retry quiz

16. Most Important Interview Takeaways

Remember these five points:

1. Service Workers act as a proxy between the webpage and the network.

2. `serviceWorkers: 'block'` disables Service Workers during testing.
3. `context.serviceWorkers()` lists Service Workers.
4. `request.serviceWorker()` identifies Service Worker-owned requests.
5. `response.fromServiceWorker()` checks whether the response was handled by a Service Worker.

My suggestion for interview preparation: Give extra attention to the difference between `request.serviceWorker()` and `response.fromServiceWorker()`, and understand why `request.frame()` throws an exception. These are useful scenario-based interview topics.

Send the next Playwright documentation section, and we'll continue with the same explanation, practical examples, interview Q&A and MCQ practice.

Playwright – Snapshot Testing

Snapshot testing is an important concept in Playwright, especially for UI testing, accessibility testing, and regression testing.

Let's understand it step by step with simple explanations, practical QA examples, code explanations, interview questions, and MCQs.

1. Introduction to Snapshot Testing

What is Snapshot Testing?

Snapshot testing is a testing technique in which we compare the current output of an application with a previously defined expected structure (snapshot).

If the current output does not match the expected snapshot, the test fails.

In Playwright, the snapshot feature covered in this documentation is specifically ARIA Snapshot Testing.

It checks the accessibility tree of a webpage rather than comparing the visual appearance of the page.

Simple example

Suppose your webpage contains:

- Heading: Welcome
- Button: Login
- Link: Forgot Password

You create a snapshot of this structure.

Later, if someone accidentally removes the Login button or changes the heading, the snapshot test can detect the difference.

Accessibility Tree: Wie ein Screenreader deine Seite liest - Gehirngerecht Digital
gehirngerecht.digital

Why do we use snapshot testing?

- To verify the overall structure of a webpage.
- To detect unintended UI structure changes.
- To perform accessibility regression testing.
- To validate complex components without writing many individual assertions.

2. Basic Snapshot Testing Example

Let's understand the example from your documentation.

```
await page.goto('https://playwright.dev/');

await expect(page).toMatchAriaSnapshot(`
  - banner:
    - heading /Playwright enables reliable end-to-end/ [level=1]
    - link "Get started":
      - /url: /docs/intro
    - link "Star microsoft/playwright on GitHub":
      - /url: https://github.com/microsoft/playwright
    - link /[\\d]+k\\+ stargazers on GitHub/
`);
```

Explanation

Step 1: Open the webpage

```
await page.goto('https://playwright.dev/');
```

Opens the Playwright website.

Step 2: Start the snapshot assertion

```
await expect(page).toMatchAriaSnapshot(`...`);
```

This compares the webpage's current accessibility tree against the expected ARIA snapshot template.

Step 3: Understand the snapshot

```
- banner:
```

Checks for a banner landmark.

```
- heading /Playwright enables reliable end-to-end/ [level=1]
```

Checks for a heading whose accessible name matches the regular expression and whose heading level is 1.

```
- link "Get started":
    - /url: /docs/intro
```

Checks for a link named `Get started` with the specified destination.

```
- link /[\d]+k\+ stargazers on GitHub/
```

Checks for a link whose accessible name matches a regular expression.

What happens if the structure doesn't match?

The test fails and Playwright reports an ARIA snapshot mismatch.

Interview point

`toMatchAriaSnapshot()` checks the accessibility structure of a page. It does not compare screenshots or pixel-level visual appearance.

3. Assertion Testing vs Snapshot Testing

This is a very important interview topic.

Both approaches verify application behavior, but their purposes are different.

A. Assertion Testing

Assertion testing checks a specific value or condition.

Example:

```
await expect(page.locator('h1'))
  .toHaveText('Welcome');

await expect(page.locator('#email'))
  .toHaveValue('test@gmail.com');
```

Explanation:

- `toHaveText()` checks whether the element contains the expected text.
- `toHaveValue()` checks whether the input has the expected value.

You are checking individual properties.

Advantages:

- Easy to understand.
- Specific error information.
- Good for validating individual functionality.
- Easier to identify the exact failing condition.

Disadvantages:

- Many assertions may be required for complex structures.
- Can become lengthy when checking multiple elements.

B. Snapshot Testing

Snapshot testing compares a larger structure against a predefined template.

Example:

```
await expect(page).toMatchAriaSnapshot(`
  - heading "Welcome"
  - textbox "Email"
```

```
  - button "Login"
`);
```

Instead of writing separate assertions for each element, you check their accessible structure together.

Advantages:

- Useful for complex UI structures.
- Reduces the need for many individual assertions.
- Helps identify unexpected structural changes.
- Useful for regression testing.

Disadvantages:

- Large snapshots can be difficult to maintain.
- Dynamic content can cause frequent failures.
- Accepting snapshot updates without reviewing them can hide bugs.

Comparison table

Assertion Testing	Snapshot Testing
Checks specific conditions	Checks a broader structure
Uses methods like `toHaveText()`	Uses `toMatchAriaSnapshot()`
Gives targeted validation	Gives structural validation
Good for individual functionality	Good for complex UI structures
Usually easier to maintain for small checks	Requires snapshot maintenance

Practical QA example:

For a login page:

- Assertion testing: Verify that the Login button is enabled.
- Snapshot testing: Verify that the login form contains the expected heading, email textbox, password textbox and Login button.

Interview question: Which is better, assertion testing or snapshot testing?

Answer: Neither is universally better. Assertion testing is suitable for specific functional checks, while snapshot testing is useful for broad structural checks. Combining both provides better test coverage.

4. What are ARIA Snapshots?

ARIA stands for Accessible Rich Internet Applications.

ARIA helps assistive technologies, such as screen readers, understand webpage elements and their roles.

An ARIA snapshot is a YAML representation of the webpage's accessibility tree.

It describes:

- Element roles.
- Accessible names.
- Attributes and states.
- Parent-child relationships.
- Text content.

Example

HTML:

```
<h1>Welcome</h1>
<button>Login</button>
```

ARIA snapshot:

```
- heading "Welcome" [level=1]
- button "Login"
```

Let's understand the structure.

```
- heading "Welcome" [level=1]
```

Part	Meaning
`heading`	Accessible role of the element
`"Welcome"`	Accessible name
`[level=1]`	Heading level

```
- button "Login"
```

Part	Meaning
`button`	Accessible role
`"Login"`	Accessible name

General syntax

```
- role "name" [attribute=value]
```

- `role`: Type of accessible element.
- `name`: Accessible name of the element.
- `attribute`: Optional accessibility attribute.
- `value`: Value of that attribute.

Important: YAML indentation represents the hierarchy of elements. A nested element is written with additional indentation.

5. Snapshot Matching

Playwright provides two methods for matching ARIA snapshots:

Method	Purpose
`expect(page).toMatchAriaSnapshot()`	Matches the accessibility tree of the entire page.
`expect(locator).toMatchAriaSnapshot()`	Matches the accessibility tree within a particular element.

Example 1: Page-level snapshot

HTML:

```
<h1>title</h1>
```

Test:

```
await expect(page).toMatchAriaSnapshot(`
  - heading "title"
`);
```

This checks whether the page's accessibility tree contains a heading named `title`.

Example 2: Locator-level snapshot

```
await expect(page.getByRole('main'))
  .toMatchAriaSnapshot(`
    - heading "Dashboard"
    - button "Logout"
  `);
```

This checks the accessible structure within the main section, rather than the entire page.

Important matching rules

1. Case-sensitive: `Login` and `login` are different.
2. Whitespace normalization: Whitespace within names and text is normalized.
3. Order-sensitive: Elements must appear in the order specified by the template.

4. Nesting matters: YAML indentation determines the expected parent-child structure.
5. Single-node matching: A single-node template can match at any depth within the selected page or locator.
6. Multiple top-level nodes: They must match under the same parent, in the specified order.

Interview point

ARIA snapshot matching is order-sensitive and case-sensitive. YAML indentation is important because it defines the accessibility hierarchy.

6. Partial Matching

Partial matching means checking only the important parts of an accessibility tree without specifying every detail.

This is useful when some element names, attributes or child elements may change.

Example 1: Match only the button role

HTML:

```
<button>Submit</button>
```

Snapshot:

```
- button
```

Here, we are checking only whether a button exists.

We are not checking its accessible name.

Therefore, the following could also match:

```
<button>Save</button>
```

The role is still `button`.

Example 2: Checkbox

HTML:

```
<input type="checkbox" checked>
```

Snapshot:

```
- checkbox
```

The snapshot checks only the checkbox role.

It does not enforce whether the checkbox is checked or unchecked.

Example 3: List

HTML:

```
<ul>
  <li>Feature A</li>
  <li>Feature B</li>
  <li>Feature C</li>
</ul>
```

Snapshot:

```
- list:
  - listitem: Feature B
```

This checks that the list contains a matching list item with the text `Feature B`.

It does not require Feature A and Feature C to be specified in the template.

Why is partial matching useful?

Imagine your application displays a list of 100 products, but you only want to verify that a particular product appears.

You don't need to specify all 100 products in your snapshot.

7. Strict Matching

This is one of the most important concepts in snapshot testing.

By default, Playwright uses `contain` matching for children.

The `/children` property controls how strictly the children of a node are compared.

There are three matching modes:

Mode	Meaning
`contain`	Expected children must exist in the specified order. Extra children are allowed.
`equal`	Direct children must match exactly in the specified order.
`deep-equal`	Children and all nested children must match exactly.

A. Contain – Default Mode

HTML:

```
<ul>
  <li>Feature A</li>
  <li>Feature B</li>
  <li>Feature C</li>
</ul>
```

Snapshot:

```
- list:
  - listitem: Feature B
```

Result: Passes.

Why? Because Feature B exists in the list. Other children are allowed.

B. Equal – Exact direct children

Snapshot:

```
- list:
  - /children: equal
  - listitem: Feature A
  - listitem: Feature B
```

Result: Fails.

Why? The actual list has three direct children:

- Feature A
- Feature B
- Feature C

But the snapshot expects exactly two direct children.

C. Deep-equal – Exact nested structure

Suppose the HTML is:

```
<ul>
  <li>Feature A</li>
  <li>
    Feature B
    <button>View</button>
  </li>
</ul>
```

Snapshot:

```
- list:
  - /children: deep-equal
  - listitem: Feature A
  - listitem: Feature B
```

Result: Fails because the second list item also contains a nested button, which is not specified in the snapshot.

Quick comparison

Actual list

- Feature A
- Feature B
- Feature C

Expected snapshot

- Feature A
- Feature B

contain

Pass

Extra Feature C is allowed.

equal

Fail

Feature C is an unexpected direct child.

deep-equal

Fail

Unexpected nested children also cause a mismatch.

Interview question: What is the difference between `equal` and `deep-equal`?

Answer:

- `equal` checks whether the direct children match exactly.
- `deep-equal` checks whether the children and their nested descendants match exactly.

8. Setting Children Mode Globally

Instead of specifying `/children` in every snapshot, we can configure a default matching mode in `playwright.config.ts`.

Code:

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  expect: {
    toMatchAriaSnapshot: {
      children: 'equal',
    },
  },
});
```

Explanation

- `expect`: Contains assertion configuration.
- `toMatchAriaSnapshot`: Configures ARIA snapshot assertions.
- `children: 'equal'`: Makes exact direct-child matching the default.

Can we override the global setting?

Yes.

An individual snapshot can specify its own `/children` property.

For example:

```
- list:
  - /children: contain
  - listitem: Feature A
```

Even if the global configuration is `equal`, this particular node uses `contain`.

Interview point: Global configuration sets the default. Explicit `/children` settings in an individual snapshot can override it.

9. Matching with Regular Expressions

Sometimes, webpage content changes dynamically.

For example:

- Issues 10
- Issues 12
- Issues 25

If we write an exact snapshot for every possible number, maintaining it becomes difficult.

Regular expressions solve this problem.

HTML:

```
<h1>Issues 12</h1>
```

ARIA snapshot:

```
- heading /Issues \d+/
```

Explanation

Pattern	Meaning
`/.../`	Defines a regular expression.
`Issues`	Matches the word Issues.
`\d`	Matches a digit.
`+`	Matches one or more occurrences.

Therefore, this pattern matches:

- Issues 1
- Issues 12
- Issues 250

It does not match `Issues ABC`.

Another example:

```
- heading /Welcome, .+/
```

This can match:

- Welcome, Alice
- Welcome, Bob
- Welcome, Komal

Interview question: Why do we use regular expressions in ARIA snapshots?

Answer: To match dynamic or variable accessible names and text without requiring an exact fixed value.

10. Generating Snapshots

Playwright provides different ways to generate ARIA snapshots.

A. Using Playwright Code Generator

The Playwright Code Generator provides two useful options.

1. Assert snapshot

This option automatically generates an ARIA snapshot assertion for the selected element.

2. Aria snapshot tab

This displays the accessibility structure of the selected locator.

It helps us inspect:

- Roles
- Accessible names
- Attributes
- Element hierarchy

This is useful when writing snapshot tests because we don't need to manually guess the accessible structure.

B. Updating snapshots using the command line

Playwright provides the `--update-snapshots` option.

Command:

```
npx playwright test --update-snapshots
```

Short version:

```
npx playwright test -u
```

This updates snapshots that do not match the current accessibility tree.

Important update modes

Mode	Meaning
`missing`	Generates missing snapshots and keeps those tests passing.
`changed`	Updates snapshots that differ from the current output.
`all`	Regenerates all snapshots, including matching ones.
none	Prevents snapshot updates.

Important default behavior:

When running without an update flag:

- Missing snapshots are generated.
- Tests that generated those new snapshots fail, so you can review the generated baseline.

This prevents a test run from silently passing just because a new snapshot was created.

Example:

```
npx playwright test --update-snapshots=all
```

Regenerates every snapshot.

When should you update snapshots?

Suppose a developer intentionally changes the login page by adding a Remember Me checkbox.

The previous snapshot does not contain this checkbox.

After reviewing the intended UI change, update the snapshot so the new structure becomes the expected baseline.

Important QA practice

Never update a snapshot blindly just to make a failed test pass. First verify whether the change is expected or whether it indicates a real application defect.

11. Empty Template for Snapshot Generation

Playwright also allows you to generate an ARIA snapshot using an empty template.

Code:

```
await expect(locator).toMatchAriaSnapshot('');
```

Here:

- `locator` identifies the element or section.
- `''` is an empty snapshot template.
- Playwright generates the snapshot based on the current accessibility structure.

This works with the default snapshot update behavior because missing snapshots are generated by default.

Interview question: Can we generate an ARIA snapshot without writing its YAML structure manually?

Answer: Yes. We can use an empty template with `toMatchAriaSnapshot('')` or use Playwright Code Generator.

12. Snapshot Patch Files

When updating inline snapshot templates, Playwright can create patch files containing the differences.

A patch file records the changes between the old and new snapshot.

Playwright supports three source update methods:

Method	Meaning
`patch`	Creates a unified diff patch that can be applied using Git.

`3way`	Creates merge conflict markers so you can choose which changes to accept.
`overwrite`	Directly overwrites the source code with the new snapshot values.

Example:

```
npx playwright test --update-snapshots --update-source-method=3way
```

This uses the three-way merge approach.

Simple example:

Old snapshot:

```
- button "Login"
```

New snapshot:

```
- button "Sign In"
```

The patch records this difference so you can review and apply the change.

Important distinction:

- Inline snapshot templates are updated through source-code patching.
- Separate `.aria.yml` snapshot files are written directly to their configured snapshot path.

13. Snapshots as Separate Files

Instead of writing the snapshot directly inside your test file, you can save it in a separate `.aria.yml` file.

Example:

```
await expect(page.getByRole('main'))
  .toMatchAriaSnapshot({
    name: 'main.aria.yml'
  });
```

What does name do?

It specifies the name of the external snapshot file.

For example:

```
tests/
  example.spec.ts
  example.spec.ts-snapshots/
    main.aria.yml
```

By default, snapshots for `example.spec.ts` are stored inside:

```
example.spec.ts-snapshots/
```

Custom snapshot path

You can configure a custom snapshot path.

```
export default defineConfig({
  expect: {
    toMatchAriaSnapshot: {
      pathTemplate: '__snapshots__/{testFilePath}/{arg}{ext}',
    },
  },
});
```

Explanation:

Placeholder	Meaning
`{testFilePath}`	Path of the test file.
`{arg}`	Snapshot name or argument.
`{ext}`	Snapshot file extension.

Important: Even when running tests across multiple browsers, Playwright saves one ARIA snapshot because the accessibility structure is expected to be the same across browsers.

14. `page.ariaSnapshot()` and `locator.ariaSnapshot()`

These methods allow us to generate an ARIA snapshot programmatically.

A. `page.ariaSnapshot()`

Returns the YAML representation of the page's accessibility structure.

Example:

```
const snapshot = await page.ariaSnapshot();

console.log(snapshot);
```

Explanation:

1. `page.ariaSnapshot()` obtains the accessibility snapshot.
2. The result is stored in the `snapshot` variable.
3. `console.log()` prints the YAML structure.

B. `locator.ariaSnapshot()`

Generates an accessibility snapshot for a particular locator.

Example:

```
const snapshot = await page.getByRole('main').ariaSnapshot();

console.log(snapshot);
```

This generates the snapshot only for the main section.

Difference between these methods

Method	Purpose
`page.ariaSnapshot()`	Gets the page-level accessibility snapshot.

`locator.ariaSnapshot()`	Gets the snapshot for a particular locator.
`toMatchAriaSnapshot()`	Compares the current accessibility tree against an expected template.

Very important interview question: What is the difference between `ariaSnapshot()` and `toMatchAriaSnapshot()`?

Answer:

- `ariaSnapshot()` generates and returns the accessibility structure as YAML.
- `toMatchAriaSnapshot()` is an assertion that compares the current accessibility structure with an expected template.

15. Accessibility Tree Examples

Let's understand the different HTML elements and how they appear in ARIA snapshots.

A. Headings with levels

HTML:

```
<h1>Title</h1>
<h2>Subtitle</h2>
```

ARIA snapshot:

```
- heading "Title" [level=1]
- heading "Subtitle" [level=2]
```

`level=1` represents H1, and `level=2` represents H2.

B. Text nodes

HTML:

```
<div>Sample accessible name</div>
```

ARIA snapshot:

```
- text: Sample accessible name
```

Standalone text can appear as a text node.

C. Multiline text

HTML:

```
<p>Line 1<br>Line 2</p>
```

ARIA snapshot:

```
- paragraph: Line 1 Line 2
```

Notice that the line break is normalized into a space.

D. Links

HTML:

```
<a href="#more-info">Read more about Accessibility</a>
```

ARIA snapshot:

```
- link "Read more about Accessibility":
    - /url: "#more-info"
```

Here:

- `link` represents the accessible role.
- The quoted text is the accessible name.
- `/url` represents the link destination.

E. Text boxes

HTML:

```
<input type="text" value="Enter your name">
```

ARIA snapshot:

```
- textbox: Enter your name
```

The textbox's current value is represented in the snapshot.

F. Lists with items

HTML:

```
<ul aria-label="Main Features">
  <li>Feature 1</li>
  <li>Feature 2</li>
</ul>
```

ARIA snapshot:

```
- list "Main Features":
  - listitem: Feature 1
  - listitem: Feature 2
```

Here:

- `list` represents the list.
- `Main Features` is its accessible name.
- `listitem` represents individual list items.

G. Grouped elements

HTML:

```
<details>
  <summary>Summary</summary>
  <p>Detail content here</p>
</details>
```

ARIA snapshot:

```
- group: Summary
```

The details element is represented as a group with its summary.

H. Attributes and states

ARIA snapshots can also capture the current states of interactive elements.

Attribute	Meaning

`checked`	Checkbox or radio selection state
`disabled`	Whether an element is disabled
`expanded`	Whether a collapsible element is expanded
`invalid`	Whether an input has an invalid value
`level`	Heading level
`pressed`	Toggle button state
`selected`	Whether an option is selected

Example 1: Checked checkbox

HTML:

```
<input type="checkbox" checked>
```

Snapshot:

```
- checkbox [checked]
```

Example 2: Pressed button

HTML:

```
<button aria-pressed="true">Toggle</button>
```

Snapshot:

```
- button "Toggle" [pressed=true]
```

Example 3: Invalid email

HTML:

```
<input
  type="text"
```

```
  aria-label="Email"
  aria-invalid="true"
  value="not-an-email">
```

Snapshot:

```
- textbox "Email" [invalid]: not-an-email
```

Notice that `aria-invalid="true"` is represented as `[invalid]`.

Example 4: Invalid spelling

HTML:

```
<input
  type="text"
  aria-label="Bio"
  aria-invalid="spelling">
```

Snapshot:

```
- textbox "Bio" [invalid=spelling]
```

Here, the specific invalid value is displayed.

16. Practical QA Automation Example

Scenario: Verify the accessibility structure of a login page.

Suppose your login page contains:

- Login heading
- Email textbox
- Password textbox
- Remember Me checkbox
- Login button
- Forgot Password link

We can validate the structure using an ARIA snapshot.

Test code:

```
import { test, expect } from '@playwright/test';

test('Verify login page accessibility structure', async ({ page })
=> {

  await page.goto('https://example.com/login');

  await expect(page).toMatchAriaSnapshot(`
    - heading "Login" [level=1]
    - textbox "Email"
    - textbox "Password"
    - checkbox "Remember Me"
    - button "Login"
    - link "Forgot Password"
  `);

});
```

Step-by-step execution

1. Open the login page.
2. Playwright reads the accessibility tree.
3. It compares the actual structure with the expected YAML template.
4. If the expected elements and their structure match, the test passes.
5. If an expected element is missing or its accessible name changes, the test fails.

Real-world example:

Suppose a developer accidentally removes the Forgot Password link.

The snapshot assertion detects that the expected link is missing.

This helps catch unintended structural changes during regression testing.

Important: This test checks accessible structure, not whether login works. You should also write functional assertions to verify login behavior.

17. Common Mistakes to Avoid

Mistake	Correct approach
Confusing ARIA snapshot with screenshot	ARIA snapshot checks accessibility structure, not pixels.
Ignoring YAML indentation	Use correct indentation to represent parent-child relationships.
Forgetting element order	Snapshot matching is order-sensitive.
Using exact text for dynamic values	Use regular expressions where appropriate.
Updating snapshots without review	Review every change before accepting it.
Using large snapshots for dynamic content	Prefer focused snapshots and targeted assertions.
Confusing `ariaSnapshot()` with `toMatchAriaSnapshot()`	One generates a snapshot; the other asserts a match.

18. Interview Questions and Answers

Q1. What is snapshot testing in Playwright?

A: Snapshot testing compares the current accessibility structure of a webpage against a predefined ARIA snapshot template to detect unexpected changes.

Q2. What is an ARIA snapshot?

A: An ARIA snapshot is a YAML representation of the accessibility tree containing roles, accessible names, attributes, states and hierarchy.

Q3. Which method is used for ARIA snapshot assertions?

A:

```
await expect(page).toMatchAriaSnapshot(`
  - heading "Welcome"
`);
```

Q4. What is the difference between assertion testing and snapshot testing?

A: Assertion testing checks specific values or conditions, whereas snapshot testing checks a broader structure against an expected template.

Q5. What is the difference between `page.ariaSnapshot()` and `toMatchAriaSnapshot()`?

A: `ariaSnapshot()` generates and returns the accessibility tree as YAML, while `toMatchAriaSnapshot()` compares it against an expected template.

Q6. Can we perform snapshot testing on a particular element?

A: Yes. We can use `expect(locator).toMatchAriaSnapshot()`.

Q7. What is partial matching?

A: Partial matching verifies only specified roles, names, attributes or children without requiring the entire accessibility structure to match.

Q8. What are the three children matching modes?

A:

- `contain`: Expected children must exist in order; extra children are allowed.
- `equal`: Direct children must match exactly.
- `deep-equal`: Children and nested descendants must match exactly.

Q9. What is the default children matching mode?

A: `contain`.

Q10. What is the purpose of regular expressions in ARIA snapshots?

A: They allow us to match dynamic accessible names or text values.

Q11. How do you update snapshots in Playwright?

A:

```
npx playwright test --update-snapshots
```

Or:

```
npx playwright test -u
```

Q12. What happens when a snapshot does not match?

A: The assertion fails and Playwright reports the difference between the expected and actual accessibility structures.

Q13. What is the purpose of `/url` in an ARIA snapshot?

A: It allows us to verify the destination URL of a link.

Q14. What is the purpose of `[level=1]`?

A: It represents the heading level, such as H1.

Q15. What is the difference between `equal` and `deep-equal`?

A: `equal` checks only direct children, while `deep-equal` checks direct and nested children.

Q16. Can ARIA snapshots validate checkbox states?

A: Yes. For example, `[checked]` verifies that the checkbox is checked.

Q17. What are snapshot patch files?

A: They capture differences between old and new inline snapshots so that changes can be reviewed and applied to source code.

Q18. When should you use snapshot testing instead of individual assertions?

A: When you need to validate a broader accessibility structure, such as a complete form, navigation section or complex UI component.

19. Quick Revision – Important Methods and Concepts

Concept	Remember
`toMatchAriaSnapshot()`	Compares accessibility structure
`ariaSnapshot()`	Generates YAML accessibility snapshot

Partial matching	Matches only specified details
`contain`	Allows extra children
`equal`	Exact direct children
`deep-equal`	Exact nested structure
Regular expressions	Match dynamic text
`--update-snapshots`	Updates snapshots
`.aria.yml`	External snapshot file
`/url`	Matches link destination
`[level=1]`	Heading level
`[checked]`	Checkbox state
`[pressed=true]`	Toggle button state

20. MCQ Practice

Snapshot Testing Quiz

10/10 answered

1. Which method is used for ARIA snapshot assertions?

toHaveScreenshot()
toMatchAriaSnapshot()
ariaSnapshotMatch()
toMatchSnapshot()

Correct! toMatchAriaSnapshot() compares the accessibility tree with an expected template.

2. What format is used for ARIA snapshots?

JSON
XML
YAML
HTML

Incorrect. ARIA snapshots use YAML representation.

3. What is the default children matching mode?

equal
deep-equal
contain
strict

Incorrect. contain is the default and allows extra children.

4. Which mode checks only direct children exactly?

contain
equal
deep-equal
partial

Incorrect. equal requires the direct children to match exactly in order.

5. Which mode checks nested descendants exactly?

contain
equal
deep-equal
regex

Correct! deep-equal checks children and their nested children.

6. Which method returns an accessibility snapshot as YAML?

page.ariaSnapshot()
page.screenshot()
page.accessibility()
page.snapshot()

Incorrect. ariaSnapshot() generates and returns the accessibility structure.

7. Which command updates snapshots?

npx playwright test --update-snapshots
npx playwright test --snapshot
npx playwright update
npx playwright test --refresh

Incorrect. --update-snapshots is the snapshot update option.

8. What does [level=1] represent?

Button priority
Heading level
Element position
Accessibility score

Correct! level=1 represents an H1 heading.

9. What does /url represent in an ARIA snapshot?

Current page title
Link destination
Image source
Element ID

Correct! /url can be used to match a link's destination.

10. What is the difference between ariaSnapshot() and toMatchAriaSnapshot()?

Both take screenshots
One generates YAML and the other asserts a match
Both update configuration
One is for Selenium

Correct! ariaSnapshot() generates the structure; toMatchAriaSnapshot() validates it.

5/10 correct

Retry quiz

21. Most Important Interview Takeaways

Focus on these concepts for QA Automation interviews:

1. Difference between assertion testing and snapshot testing.
2. What an ARIA snapshot is and why it is used.
3. Difference between `ariaSnapshot()` and `toMatchAriaSnapshot()`.
4. Difference between `contain`, `equal` and `deep-equal`.
5. How regular expressions handle dynamic text.
6. How to generate and update snapshots.
7. How to validate accessibility roles, names and states.

Practical interview tip: Be prepared to write a snapshot for a login page and explain what happens if an expected button, textbox or link is missing.

Send the next Playwright documentation section whenever you're ready.

Playwright – Test Generator (Codegen)

The Playwright Test Generator, also called Codegen, is a feature that automatically generates Playwright test code while you interact with a website.

Instead of manually writing every action such as clicking buttons, entering text, selecting options and navigating pages, you can perform those actions in a browser and Playwright generates the corresponding code.

It is especially useful for QA Automation Engineers who are getting started with Playwright.

1. Introduction – What is Test Generator?

Suppose you want to automate a login page.

Normally, you write code manually:

```
await page.goto('https://example.com/login');
```

```
await page.getByLabel('Email').fill('test@gmail.com');

await page.getByLabel('Password').fill('password123');

await page.getByRole('button', { name: 'Login' }).click();
```

Using Test Generator:

1. Open the website.
2. Enter your email.
3. Enter your password.
4. Click Login.
5. Playwright automatically generates the test code.

Test generator | Playwright
playwright.dev

Why do we use Test Generator?

- Automatically generates test code.
- Helps create locators.
- Records user interactions.
- Generates assertions.
- Saves time during initial test development.
- Helps beginners understand Playwright syntax.

Interview point

Playwright Codegen prioritizes locators based on role, text and test ID. If a locator matches multiple elements, it tries to improve the locator so that it uniquely identifies the target.

2. Generate Tests in VS Code

Playwright provides a VS Code extension that allows us to generate tests directly inside Visual Studio Code.

Prerequisite:

Install the Playwright VS Code extension from the VS Code Marketplace.

Steps

1. Open VS Code.
2. Open the project containing Playwright.
3. Open the Testing sidebar.
4. Click Record new.
5. A browser window opens.
6. Navigate to the website you want to test.
7. Perform actions such as clicking, filling and selecting.
8. Playwright generates code directly in VS Code.

Getting started - VS Code | Playwright
playwright.dev

What happens after clicking Record new?

Playwright creates a file named:

`test-1.spec.ts`

This file contains the generated test code.

For example, if you perform these actions:

- Open login page.
- Enter username.
- Enter password.
- Click Login.

Playwright may generate code similar to:

```
import { test, expect } from '@playwright/test';

test('test', async ({ page }) => {
  await page.goto('https://example.com/login');

  await page.getByRole('textbox', { name:
'Username' }).fill('komal');

  await page.getByRole('textbox', { name:
'Password' }).fill('password123');

  await page.getByRole('button', { name: 'Login' }).click();
});
```

Important: Generated code is a starting point. Review and improve it before using it in your automation framework.

3. Generating Assertions

The Test Generator can generate assertions in addition to recording user actions.

In VS Code, the recording toolbar provides different assertion options.

There are three important assertions:

Assertion	Purpose
Assert visibility	Checks whether an element is visible.
Assert text	Checks whether an element contains specific text.
Assert value	Checks whether an input has a specific value.

A. Assert Visibility

Suppose you want to verify that the Login button is visible.

Generated assertion:

```
await expect(
  page.getByRole('button', { name: 'Login' })
).toBeVisible();
```

Meaning: Checks whether the Login button is visible on the webpage.

B. Assert Text

Suppose you want to verify the success message.

```
await expect(
  page.getByText('Login successful')
).toBeVisible();
```

The generator's Assert text action creates an assertion that checks the selected element's text.

C. Assert Value

Suppose the email input contains test@gmail.com.

```
await expect(
  page.getByRole('textbox', { name: 'Email' })
).toHaveValue('test@gmail.com');
```

Meaning: Checks whether the input field has the expected value.

Interview point

The three assertion options in the Test Generator are visibility, text and value.

4. Record at Cursor

This is another useful VS Code feature.

What is Record at Cursor?

It allows you to record additional browser actions and insert the generated code at a specific position in an existing test file.

Example

Suppose you already have a test:

```
test('Todo application test', async ({ page }) => {

  await page.goto('https://demo.playwright.dev/todomvc');

  await page.getByPlaceholder('What needs to be done?')
    .fill('Buy groceries');

});
```

Now you want to record another action, such as marking the task as completed.

Instead of recording the entire test again:

1. Place your cursor after the existing code.
2. Open the Testing sidebar.
3. Click Record at cursor.
4. Perform the new action in the browser.
5. Playwright generates the additional code at your cursor position.

Example generated action:

```
await page.getByText('Buy groceries').click();
```

Why is this useful?

- Adds new actions to existing tests.
- Avoids recording the complete flow again.
- Helps extend existing automation scripts.
- Saves development time.

Important: If the browser is not already open, first run the test with Show browser checked, then select Record at cursor.

5. Generating Locators

Playwright Codegen can also generate locators without recording an entire test.

What is a locator?

A locator identifies an element on a webpage so that Playwright can interact with it.

For example:

```
page.getByRole('button', { name: 'Login' });
```

This locator identifies a button with the accessible name Login.

Steps to generate a locator in VS Code

1. Open the Testing sidebar.
2. Click Pick locator.
3. Hover over elements in the browser.
4. Playwright highlights the locator for the selected element.
5. Click the required element.
6. The locator appears in the Pick Locator box.
7. Press Enter to copy it to the clipboard.
8. Paste it into your test file.

Best Practices | Playwright
playwright.dev

Example

Suppose you select the Login button.

Playwright may generate:

```
page.getByRole('button', { name: 'Login' });
```

You can use it directly:

```
await page.getByRole('button', { name: 'Login' }).click();
```

Important: Press Escape to cancel locator selection.

6. Test Generator Using Playwright Inspector

Apart from VS Code, Playwright provides another way to generate tests using the command line.

It uses two windows:

1. Browser window: Where you interact with the application.
2. Playwright Inspector: Where generated code is displayed.

Running Codegen

Use this command:

```
npx playwright codegen demo.playwright.dev/todomvc
```

Test generator | Playwright
playwright.dev

Explanation

Command	Meaning
`npx`	Executes a package command.
`playwright`	Playwright CLI tool.
`codegen`	Starts the test generator.
`demo.playwright.dev/todomvc`	Website URL to open.

Is the URL mandatory?

No.

You can run:

```
npx playwright codegen
```

Then enter the website URL directly into the opened browser.

How to record a test

1. Run the Codegen command.
2. Browser and Inspector windows open.
3. Perform actions in the browser.
4. Generated code appears in the Inspector.
5. Stop recording.
6. Click Copy.
7. Paste the generated code into your editor.

You can also use the Clear button to remove the generated code and start recording again.

7. Generating Locators Using Playwright Inspector

You can also generate locators using Playwright Inspector.

Steps

1. Run the Codegen command.
2. Click the Record button to stop recording.
3. The Pick Locator button becomes available.
4. Click Pick Locator.
5. Hover over elements in the browser.
6. Playwright highlights the locator.
7. Click the element you want.
8. The generated locator appears in the Inspector.
9. Copy the locator and paste it into your test code.

Example:

Suppose you select an email textbox.

Playwright might generate:

```
page.getByRole('textbox', { name: 'Email' });
```

You can then use:

```
await page.getByRole('textbox', { name: 'Email' })
  .fill('test@gmail.com');
```

Difference between Record and Pick Locator

Record	Pick Locator
Records user actions	Generates a locator
Generates test code	Generates a selector
Used to create test flows	Used to identify elements

Interview question: What is the purpose of Pick Locator?

Answer: Pick Locator helps us identify webpage elements and automatically generate their Playwright locator expressions.

8. Emulation in Test Generator

What is Emulation?

Emulation means testing a website under different device and browser conditions without physically using those devices.

Playwright Codegen supports emulation for:

- Viewport size
- Mobile devices
- Color scheme
- Geolocation
- Language

- Timezone
- Authenticated browser state

Why do we need emulation?

Suppose your application works correctly on a desktop but has layout issues on mobile.

Using emulation, we can generate and test scripts under mobile device conditions.

9. Emulate Viewport Size

What is a viewport?

A viewport is the visible area of a webpage inside the browser window.

For example:

- Desktop: 1920 × 1080
- Tablet: 1024 × 768
- Mobile: 390 × 844

Playwright provides the `--viewport-size` option.

Command:

```
npx playwright codegen --viewport-size="800,600" playwright.dev
```

Explanation

Parameter	Meaning
`--viewport-size`	Sets the browser viewport dimensions.
`800`	Viewport width in pixels.
`600`	Viewport height in pixels.

`playwright.dev` Website to open.

This opens the website with a viewport of 800 × 600 pixels.

Practical QA example:

You want to verify that the navigation menu is displayed correctly on a smaller screen.

You can generate the test using a smaller viewport.

Interview point

Viewport emulation helps test responsive layouts under different screen dimensions.

10. Emulate Devices

Playwright can emulate mobile devices using the `--device` option.

Command:

```
npx playwright codegen --device="iPhone 13" playwright.dev
```

What does device emulation do?

It configures device characteristics such as:

- Viewport size
- User agent
- Device scale factor
- Mobile behavior
- Touch support

The exact settings depend on the selected device profile.

Example:

Suppose you want to test a shopping website on an iPhone.

You can run the command above and record:

1. Opening the shopping website.
2. Searching for a product.
3. Opening product details.
4. Adding the product to the cart.

Playwright generates the code under the selected device emulation.

Difference between viewport and device emulation

Viewport emulation	Device emulation
Changes width and height	Applies a device profile
Uses `--viewport-size`	Uses `--device`
Useful for responsive layout testing	Useful for mobile-device testing

11. Emulate Color Scheme

Websites may support two themes:

- Light mode
- Dark mode

Playwright provides the `--color-scheme` option.

Command:

```
npx playwright codegen --color-scheme=dark playwright.dev
```

This opens the website with dark color scheme preferences.

For light mode:

```
npx playwright codegen --color-scheme=light playwright.dev
```

Practical QA example:

Suppose your application supports light and dark themes.

You can record separate tests to verify:

- Text visibility.
- Background colors.
- Button appearance.
- Navigation menu appearance.

Interview question: How do you generate tests in dark mode using Playwright Codegen?

Answer:

```
npx playwright codegen --color-scheme=dark https://example.com
```

12. Emulate Geolocation, Language and Timezone

Playwright allows us to generate tests using different location, language and timezone settings.

This is useful when testing applications that depend on a user's location or regional settings.

A. Timezone

The `--timezone` option changes the browser's timezone.

B. Geolocation

The `--geolocation` option specifies latitude and longitude.

C. Language

The `--lang` option specifies the browser's language preference.

Command from your documentation:

```
npx playwright codegen --timezone="Europe/Rome" --
geolocation="41.890221,12.492348" --lang="it-IT" bing.com/maps
```

Explanation

Option	Meaning
`--timezone="Europe/Rome"`	Sets the browser timezone to Rome.
`--geolocation="41.890221,12.492348"`	Sets the simulated location coordinates.
`--lang="it-IT"`	Sets the language preference to Italian.
`bing.com/maps`	Opens Bing Maps.

Practical QA example:

Suppose you are testing a location-based application.

You want to check whether the application displays the correct nearby places when the user is in Rome.

You can use geolocation emulation instead of physically travelling to Rome.

Important: A website may ask for location permission. You may need to accept it before geolocation works.

13. Preserve Authenticated State

This is a very important topic for QA Automation interviews.

What is authenticated state?

Authenticated state means the browser has already logged into an application and contains the necessary login information.

For example, after logging into GitHub, the browser may store:

- Cookies
- Local storage
- IndexedDB data

Normally, if you run a new test, you may need to log in again.

Playwright allows you to save this authentication information and reuse it.

A. Save authenticated state

Use the `--save-storage` option.

Command:

```
npx playwright codegen github.com/microsoft/playwright --save-storage=auth.json
```

How does it work?

1. Playwright opens GitHub.
2. You log in using your credentials.
3. You close the browser after authentication.
4. Playwright saves the authentication storage information into `auth.json`.

GitHub Personal Access Token – CVEN 5999 - Summer 2026
cven5999-ss26.github.io

What is auth.json?

It is a JSON file containing the saved browser storage state.

It can contain cookies and other authentication-related data.

Example of a simplified storage-state structure:

```
{
  "cookies": [],
  "origins": []
}
```

This is only an illustrative structure, not a complete authentication file.

Security warning

The `auth.json` file may contain sensitive authentication information.

- Do not share it publicly.

- Do not commit it to GitHub.
- Add it to `.gitignore`.
- Delete it when it is no longer needed.

B. Load authenticated state

After saving the authentication information, you can reuse it with `--load-storage`.

Command:

```
npx playwright codegen --load-storage=auth.json
github.com/microsoft/playwright
```

How does it work?

1. Playwright reads `auth.json`.
2. It restores the saved cookies and supported browser storage.
3. The browser opens with the restored authentication state.
4. You can continue recording tests without performing login again, provided the authentication is still valid.

Save vs Load storage

Option	Purpose
`--save-storage=auth.json`	Saves authentication state.
`--load-storage=auth.json`	Loads previously saved authentication state.

Real-world QA example:

Suppose you have 20 test cases for an e-commerce application.

Every test requires a logged-in user.

Instead of recording the login steps repeatedly, you can save the authenticated state and reuse it while recording other tests.

Interview question: What is the difference between `--save-storage` and `--load-storage`?

Answer:

- `--save-storage` saves cookies and supported browser storage into a file.
- `--load-storage` restores the previously saved storage state so you can continue from an authenticated session.

14. Use Existing `userDataDir`

Playwright Codegen also allows you to use an existing browser profile.

What is userDataDir?

A user data directory is a folder that stores browser profile information, such as:

- Cookies
- Browser preferences
- Local storage
- Session-related information
- Other browser profile data

Instead of creating a new browser profile, Codegen can use an existing custom profile.

Command:

```
npx playwright codegen --user-data-dir=/path/to/your/browser/data/ https://github.com/microsoft/playwright
```

Explanation

Parameter	Meaning
`--user-data-dir`	Specifies the browser profile directory.

`/path/to/your/browser/data/`	Location of the custom profile.
Website URL	Website to open.

Why is it useful?

Suppose you have already logged into an application using a custom browser profile.

You can use that profile in Codegen and continue recording tests with its existing authentication state.

Important Chrome limitation

As mentioned in the documentation, from Chrome 136, the default Chrome user data directory cannot be accessed by automated tooling such as Playwright.

You must create a separate user data directory for testing.

Interview question: What is the difference between `--user-data-dir` and `--load-storage`?

Answer:

- `--user-data-dir` uses an existing browser profile directory.
- `--load-storage` restores saved cookies and supported storage data from a storage-state file.

15. Authenticate with HTTP Credentials

Some websites require HTTP Basic Authentication before allowing access.

For example, when you open a protected website, the browser asks for a username and password.

Playwright Codegen provides the `--http-credentials` option.

Command:

```
npx playwright codegen --http-credentials="username:password"
example.com
```

Explanation

Parameter	Meaning
`--http-credentials`	Supplies HTTP authentication credentials.
`username`	Authentication username.
`password`	Authentication password.
`example.com`	Website to access.

What is HTTP Basic Authentication?

It is an authentication mechanism in which the client sends a username and password using the HTTP Authorization header, typically with Base64 encoding.

Important difference:

HTTP Basic Authentication is different from a normal website login form.

HTTP Basic Authentication	Normal website login
Uses HTTP authentication credentials	Uses a webpage login form
Often displays a browser authentication prompt	Displays username/password input fields
Can use `--http-credentials`	Can record normal fill and click actions

Important security point:

According to the documentation, these credentials are sent to any origin that requests them during the recording session and are included in the generated code.

Therefore, do not use sensitive credentials casually.

16. Record Using Custom Setup

Sometimes, we need to configure the browser before recording actions.

For example:

- Configure network routing.
- Add custom browser context options.
- Set up a test environment.
- Configure authentication or other browser settings.

For these situations, Playwright provides `page.pause()`.

Code from documentation

```
const { chromium } = require('@playwright/test');

(async () => {

  // Make sure to run headed.
  const browser = await chromium.launch({
    headless: false
  });

  // Setup context however you like.
  const context = await browser.newContext({});

  await context.route('**/*', route => route.continue());

  // Pause the page, and start recording manually.
  const page = await context.newPage();

  await page.pause();

})();
```

Understand the code step by step

Step 1: Import Chromium

```
const { chromium } = require('@playwright/test');
```

Imports the Chromium browser launcher.

Step 2: Launch browser

```
const browser = await chromium.launch({
  headless: false
});
```

- `chromium.launch()` starts Chromium.
- `headless: false` opens the browser visibly.

Why is this necessary?

Because we need to interact with the browser manually while recording.

Step 3: Create browser context

```
const context = await browser.newContext({});
```

Creates a new isolated browser context.

Step 4: Configure network routing

```
await context.route('**/*', route => route.continue());
```

- `**/*` matches all URLs.
- `route.continue()` allows each intercepted request to continue.

This is an example of custom network configuration.

Step 5: Create a page

```
const page = await context.newPage();
```

Creates a new browser page or tab.

Step 6: Pause the page

```
await page.pause();
```

Pauses execution and opens Playwright Inspector controls so you can inspect the page and interact with the recording tools.

Interview point

`page.pause()` is useful when you want to start recording or debugging from a custom browser setup.

It requires a headed browser for interactive use.

17. Important Codegen Commands – Quick Revision

Command	Purpose
`npx playwright codegen`	Opens the test generator.
`npx playwright codegen https://example.com`	Opens Codegen with a website.
`--viewport-size="800,600"`	Sets viewport dimensions.
`--device="iPhone 13"`	Emulates a device profile.
`--color-scheme=dark`	Emulates dark mode.
`--timezone="Europe/Rome"`	Sets timezone.
`--geolocation="41.890221,12.492348"`	Sets location coordinates.
`--lang="it-IT"`	Sets browser language.
`--save-storage=auth.json`	Saves browser storage state.
`--load-storage=auth.json`	Loads saved browser storage state.

`--user-data-dir=...`	Uses a custom browser profile.
`--http-credentials="username:password"`	Supplies HTTP Basic Authentication credentials.

18. Practical QA Automation Example

Scenario: Record an e-commerce application test using Codegen.

Suppose you want to test an e-commerce website.

Test flow:

1. Open the application.
2. Log in.
3. Search for a mobile phone.
4. Open a product.
5. Add it to the cart.
6. Verify that the cart displays the product.

Step 1: Start Codegen

```
npx playwright codegen https://example.com
```

Step 2: Perform actions

Interact with the website:

- Enter username and password.
- Click Login.
- Search for a mobile phone.
- Select a product.
- Click Add to Cart.

Step 3: Generate assertion

Select the cart element and use Assert text.

The generated code might look like:

```
import { test, expect } from '@playwright/test';

test('Add product to cart', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByRole('textbox', { name: 'Username' })
    .fill('komal');

  await page.getByRole('textbox', { name: 'Password' })
    .fill('password123');

  await page.getByRole('button', { name: 'Login' })
    .click();

  await page.getByPlaceholder('Search products')
    .fill('Mobile Phone');

  await page.getByRole('button', { name: 'Search' })
    .click();

  await page.getByRole('button', { name: 'Add to Cart' })
    .click();

  await expect(page.getByText('Mobile Phone'))
    .toBeVisible();

});
```

Illustrative generated test. Actual locators depend on the website's accessible names and structure.

Step 4: Review generated code

After Codegen generates the test:

- Check whether locators are unique.

- Remove unnecessary actions.
- Add meaningful test names.
- Add assertions for important functionality.
- Replace hardcoded test data where appropriate.
- Organize the code into your POM framework if required.

Important QA point: Codegen helps create the initial test, but it does not replace manual test design, code review or framework development.

19. Important Interview Questions and Answers

Q1. What is Playwright Test Generator?

A: Playwright Test Generator, also called Codegen, automatically generates test code by recording user interactions with a webpage.

Q2. How do you start Playwright Codegen?

A:

```
npx playwright codegen https://example.com
```

Q3. Which locators does Playwright Codegen prioritize?

A: It prioritizes:

- Role locators.
- Text locators.
- Test ID locators.

It also tries to make locators unique when multiple elements match.

Q4. What is the purpose of Record new in VS Code?

A: It creates a new test file and opens a browser to record user actions. Playwright generates the corresponding test code.

Q5. What is Record at Cursor?

A: It records additional browser actions and inserts the generated code at the selected cursor position in an existing test.

Q6. What is Pick Locator?

A: It allows us to select an element in the browser and generate its Playwright locator.

Q7. Which assertions can be generated using Codegen?

A:

1. Assert visibility.
2. Assert text.
3. Assert value.

Q8. What is Playwright Inspector?

A: Playwright Inspector is a tool that allows us to record tests, inspect locators, generate code and debug Playwright tests.

Q9. What is the difference between VS Code Test Generator and Playwright Inspector?

A:

- VS Code Test Generator integrates recording and locator generation directly into VS Code.
- Playwright Inspector provides a separate interface for recording and inspecting generated test code.

Q10. What is the purpose of `--viewport-size`?

A: It sets the browser viewport width and height for responsive UI testing.

Q11. How do you emulate an iPhone using Codegen?

A:

```
npx playwright codegen --device="iPhone 13" https://example.com
```

Q12. How do you generate tests in dark mode?

A:

```
npx playwright codegen --color-scheme=dark https://example.com
```

Q13. How do you emulate geolocation in Playwright?

A: Use the `--geolocation` option with latitude and longitude coordinates.

Q14. What is the purpose of `--save-storage`?

A: It saves cookies and supported browser storage data into a file so that the authenticated state can be reused.

Q15. What is the purpose of `--load-storage`?

A: It loads previously saved browser storage data to restore an authenticated session.

Q16. What is the difference between `--load-storage` and `--user-data-dir`?

A: `--load-storage` restores browser storage from a storage-state file, whereas `--user-data-dir` uses an existing browser profile directory.

Q17. What is HTTP Basic Authentication?

A: It is an HTTP authentication mechanism where a client provides a username and password through HTTP authentication credentials.

Q18. What is the purpose of `page.pause()`?

A: It pauses test execution and opens Playwright Inspector, allowing us to inspect the page, debug and start recording manually.

Q19. Does Codegen generate production-ready automation frameworks?

A: No. It generates initial test code. We should review locators, add meaningful assertions, remove unnecessary steps and organize the code using practices such as Page Object Model.

Q20. Can Codegen generate tests for mobile applications?

A: It can generate browser tests using mobile device emulation through the `--device` option. It does not automate native mobile applications.

20. Common Mistakes to Avoid

Mistake	Correct approach

Assuming generated code is always perfect	Review and optimize generated code.
Using duplicate locators	Check locator uniqueness.
Recording actions without assertions	Add validations for expected results.
Sharing `auth.json`	Keep authentication files private.
Confusing viewport and device emulation	Viewport changes dimensions; device emulation applies a device profile.
Using `page.pause()` in unattended CI execution	Use it for interactive debugging and recording.
Assuming Codegen replaces POM	Organize generated code into page classes when needed.

21. MCQ Practice

Test your understanding of Playwright Test Generator.

Test Generator Quiz

10/10 answered

1. What is Playwright Codegen used for?

Database management
Automatically generating tests by recording browser actions
Creating CSS files
Installing browsers

Correct! Codegen records user interactions and generates Playwright test code.

2. Which command starts the Playwright Test Generator?

npx playwright testgen
npx playwright codegen
npx playwright record

npx playwright generate

Correct! The command is npx playwright codegen.

3. Which locator types does Codegen prioritize?

Only XPath
Only CSS
Role, text and test ID
Only class selectors

Correct! Codegen prioritizes role, text and test ID locators.

4. What does Record at Cursor do?

Creates a new project
Adds recorded actions at the cursor position
Deletes the test
Runs all tests

Correct! It inserts newly generated actions at the selected position in an existing test.

5. Which assertion options are available in the generator?

Visibility, text and value
Only URL
Only screenshot
Only API response

Correct! The generator can create visibility, text and value assertions.

6. Which option is used to generate tests for an iPhone profile?

--viewport-size
--device
--mobile-only
--browser

Correct! The --device option applies a device profile.

7. Which option is used to emulate dark mode?

--theme=dark
--dark
--color-scheme=dark

--mode=dark

Incorrect. Use --color-scheme=dark.

8. Which option saves browser storage state?

--load-storage
--save-storage
--user-data-dir
--save-browser

Correct! --save-storage saves cookies and supported browser storage.

9. Which option restores previously saved storage state?

--save-storage
--load-storage
--restore-user
--auth-mode

Correct! --load-storage loads the saved storage-state file.

10. What does page.pause() do?

Closes the browser
Pauses execution and opens Inspector controls
Clears cookies
Restarts the test

Correct! page.pause() opens the interactive Playwright Inspector.

9/10 correct

Retry quiz

22. Most Important Points for Interviews

Focus especially on these topics:

1. What is Playwright Codegen?
2. How to generate tests using VS Code and Playwright Inspector.

3. Difference between Record new and Record at Cursor.
4. How to generate locators using Pick Locator.
5. Which locators does Codegen prioritize?
6. Difference between `--viewport-size` and `--device`.
7. Difference between `--save-storage` and `--load-storage`.
8. How to use `page.pause()` for custom recording.
9. Why generated code needs manual optimization.

Practical exercise: Try recording a login and product search flow using Codegen, then improve the generated locators and add assertions manually. This will help you understand how Codegen fits into a real Selenium-to-Playwright QA automation workflow.

Playwright – Touch Events (Legacy)

1. Introduction

Touch Events are used in web applications to handle touch-based gestures such as:

- Tap: Touching the screen once.
- Swipe: Moving a finger across the screen.
- Pan: Dragging the screen to move content, such as a map.
- Pinch: Using two fingers to zoom in or zoom out.

Playwright allows us to test these gestures by manually dispatching touch events using:

`locator.dispatchEvent()`

Simple meaning: We can simulate finger movements on a mobile web page through Playwright code instead of physically touching the screen.

Interview point
These examples use legacy Touch Events, not the modern Playwright touchscreen API.

2. Important concepts

Term	Meaning
`touchstart`	Triggered when a finger touches the screen
`touchmove`	Triggered when a finger moves on the screen
`touchend`	Triggered when a finger is lifted
`Touch`	Represents one finger touching the screen
`identifier`	Unique ID for each touch point
`clientX`	Horizontal position relative to the browser viewport
`clientY`	Vertical position relative to the browser viewport
`changedTouches`	Touch points that changed during the event
`targetTouches`	Touch points currently touching the event target
`touches`	All active touch points

Touch event sequence

touchstart

Finger touches screen

touchmove

Finger moves

touchend

Finger lifted

3. Important: `dispatchEvent()`

Syntax:

```
await locator.dispatchEvent('touchstart', eventInit);
```

- `locator`: The element on which the event is dispatched.
- `'touchstart'`: The event name.
- `eventInit`: The event properties, including touch points.

Important limitation: `dispatchEvent()` does not create a trusted browser event.

The event's `isTrusted` property is `false`.

If an application checks `event.isTrusted` and ignores synthetic events, these tests may not work. Prefer testing without weakening real security checks in production.

4. Emulating a Pan Gesture

What is a pan gesture?

A pan gesture means moving one finger across the screen to move content.

Example: On Google Maps, you drag the map left, right, up, or down to view another location.

HungaroMet Meteora
71net300.met.hu

Code explanation

Step 1: Import required packages

```
import { test, expect, devices, type Locator } from
'@playwright/test';
```

- `test`: Defines the test.
- `expect`: Performs assertions.

- `devices`: Provides predefined mobile device settings.
- `Locator`: TypeScript type for a Playwright locator.

Step 2: Configure a mobile device

```
test.use({ ...devices['Pixel 7'] });
```

This configures the test to use Pixel 7 device settings, including mobile viewport and touch-related settings.

Step 3: Create the pan function

```
async function pan(
  locator: Locator,
  deltaX?: number,
  deltaY?: number,
  steps?: number
)
```

This is a reusable function to simulate a pan gesture.

Parameter	Meaning
`locator`	Element to move, such as a map
`deltaX`	Total horizontal movement
`deltaY`	Total vertical movement
`steps`	Number of intermediate movements

The `?` means that the parameter is optional.

For example:

```
await pan(map, 200, 100, 5);
```

Moves the touch point 200 pixels horizontally and 100 pixels vertically in 5 steps.

Step 4: Find the center of the element

```
const { centerX, centerY } = await locator.evaluate(
  (target: HTMLElement) => {
    const bounds = target.getBoundingClientRect();

    const centerX = bounds.left + bounds.width / 2;
    const centerY = bounds.top + bounds.height / 2;

    return { centerX, centerY };
  }
);
```

Here:

- `locator.evaluate()` executes JavaScript in the browser.
- `getBoundingClientRect()` gets the element's position and dimensions.
- `centerX` calculates the horizontal center.
- `centerY` calculates the vertical center.

For example, if the map's left position is 100 and its width is 400:

\[centerX = 100 + \frac{400}{2} = 300 \]

So, the touch starts at the map's horizontal center.

Step 5: Create a touch point

```
const touches = [{
  identifier: 0,
  clientX: centerX,
  clientY: centerY,
}];
```

This creates one finger touch.

- `identifier: 0` identifies the finger.
- `clientX` is its horizontal position.
- `clientY` is its vertical position.

Step 6: Start touching

```
await locator.dispatchEvent('touchstart', {
  touches,
  changedTouches: touches,
  targetTouches: touches
});
```

This tells the application that a finger has touched the map.

Step 7: Move the finger

```
steps = steps ?? 5;
deltaX = deltaX ?? 0;
deltaY = deltaY ?? 0;

for (let i = 1; i <= steps; i++) {
  const touches = [{
    identifier: 0,
    clientX: centerX + deltaX * i / steps,
    clientY: centerY + deltaY * i / steps,
  }];

  await locator.dispatchEvent('touchmove', {
    touches,
    changedTouches: touches,
    targetTouches: touches
  });
}
```

This is the main logic of the pan gesture.

- `??` is the nullish coalescing operator.
- If `steps` is not provided, it defaults to 5.
- The loop moves the finger gradually.
- Each iteration dispatches a `touchmove` event.

For example, if `deltaX = 200` and `steps = 5`:

Step	Horizontal movement

1	40 px
2	80 px
3	120 px
4	160 px
5	200 px

Step 8: End the touch

```
await locator.dispatchEvent('touchend');
```

This simulates lifting the finger from the screen.

Complete pan function

```
async function pan(
  locator: Locator,
  deltaX?: number,
  deltaY?: number,
  steps?: number
) {
  const { centerX, centerY } = await locator.evaluate(
    (target: HTMLElement) => {
      const bounds = target.getBoundingClientRect();

      return {
        centerX: bounds.left + bounds.width / 2,
        centerY: bounds.top + bounds.height / 2
      };
    }
  );

  const touches = [{
    identifier: 0,
    clientX: centerX,
    clientY: centerY
  }];
```

```
  await locator.dispatchEvent('touchstart', {
    touches,
    changedTouches: touches,
    targetTouches: touches
  });

  steps = steps ?? 5;
  deltaX = deltaX ?? 0;
  deltaY = deltaY ?? 0;

  for (let i = 1; i <= steps; i++) {
    const touches = [{
      identifier: 0,
      clientX: centerX + deltaX * i / steps,
      clientY: centerY + deltaY * i / steps
    }];

    await locator.dispatchEvent('touchmove', {
      touches,
      changedTouches: touches,
      targetTouches: touches
    });
  }

  await locator.dispatchEvent('touchend');
}
```

Pan test example

```
test('pan gesture to move the map', async ({ page }) => {
  await page.goto('https://www.google.com/maps/place/@37.4117722,-122.0713234,15z',
    { waitUntil: 'commit' });

  await page.getByRole('button', {
    name: 'Keep using web'
  }).click();

  const map = page.locator('[data-test-id="met"]');

  for (let i = 0; i < 5; i++) {
```

```
    await pan(map, 200, 100);
  }

  await expect(map).toHaveScreenshot();
});
```

What is happening?

1. Opens Google Maps.
2. Clicks the button to continue using the web version.
3. Finds the map element.
4. Performs the pan gesture five times.
5. Takes a screenshot assertion to check the map's visual state.

`toHaveScreenshot()` compares the current screenshot against the stored baseline screenshot. It does not independently prove that the map moved unless the expected screenshot represents the moved state.

5. Emulating a Pinch Gesture

What is a pinch gesture?

A pinch gesture uses two fingers on the screen.

- Pinch in: Fingers move closer together.
- Pinch out: Fingers move farther apart.

Vector multi-touch gesture for tablets and smartphone. Pinch to zoom

Pinch in

Fingers move closer. Typically zooms out.

Mobile phone touch gestures, pinch and unpinch

Pinch out

Fingers move apart. Typically zooms in.

Code explanation

Step 1: Create the pinch function

```
async function pinch(
  locator: Locator,
  arg: {
    deltaX?: number,
    deltaY?: number,
    steps?: number,
    direction?: 'in' | 'out'
  }
)
```

This function accepts:

Parameter	Meaning
`locator`	Element on which pinch is performed
`deltaX`	Horizontal distance between touch points
`deltaY`	Available in the options, but not used in this example
`steps`	Number of movement steps
`direction`	`'in'` or `'out'`

Step 2: Find the element center

```
const { centerX, centerY } = await locator.evaluate(
  (target: HTMLElement) => {
    const bounds = target.getBoundingClientRect();

    return {
      centerX: bounds.left + bounds.width / 2,
      centerY: bounds.top + bounds.height / 2
    };
```

```
  }
);
```

This calculates the center of the map where the pinch will happen.

Step 3: Set default values

```
const deltaX = arg.deltaX ?? 50;
const steps = arg.steps ?? 5;
const stepDeltaX = deltaX / (steps + 1);
```

- Default distance is 50 pixels.
- Default steps are 5.
- `stepDeltaX` calculates the distance between each intermediate position.

Step 4: Create two touch points

```
const touches = [
  {
    identifier: 0,
    clientX: centerX - (arg.direction === 'in' ? deltaX :
stepDeltaX),
    clientY: centerY
  },
  {
    identifier: 1,
    clientX: centerX + (arg.direction === 'in' ? deltaX :
stepDeltaX),
    clientY: centerY
  }
];
```

Here we create two fingers:

- First finger has `identifier: 0`.
- Second finger has `identifier: 1`.
- Both fingers are positioned around the center of the map.

Step 5: Start the pinch

```
await locator.dispatchEvent('touchstart', {
  touches,
  changedTouches: touches,
  targetTouches: touches
});
```

This tells the application that two fingers are touching the map.

Step 6: Move the fingers

```
for (let i = 1; i <= steps; i++) {
  const offset = (
    arg.direction === 'in'
      ? deltaX - i * stepDeltaX
      : stepDeltaX * (i + 1)
  );

  const touches = [
    {
      identifier: 0,
      clientX: centerX - offset,
      clientY: centerY
    },
    {
      identifier: 1,
      clientX: centerX + offset,
      clientY: centerY
    }
  ];

  await locator.dispatchEvent('touchmove', {
    touches,
    changedTouches: touches,
    targetTouches: touches
  });
}
```

This moves both fingers:

- For `in`, the offset decreases, so fingers move closer.
- For `out`, the offset increases, so fingers move farther apart.

Important correction in the documentation snippet: The second finger's identifier in the `touchmove` event should be `1`, not `0`, so it remains consistent with the original touch point.

Step 7: End the pinch

```
await locator.dispatchEvent('touchend', {
  touches: [],
  changedTouches: [],
  targetTouches: []
});
```

This indicates that both fingers have been lifted.

Pinch test example

```
test('pinch in gesture to zoom out the map', async ({ page }) => {
  await page.goto('https://www.google.com/maps/place/@37.4117722,-122.0713234,15z',
    { waitUntil: 'commit' });

  await page.getByRole('button', {
    name: 'Keep using web'
  }).click();

  const map = page.locator('[data-test-id="met"]');

  for (let i = 0; i < 5; i++) {
    await pinch(map, {
      deltaX: 40,
      direction: 'in'
    });
  }

  await expect(map).toHaveScreenshot();
});
```

What is happening?

1. Opens Google Maps.
2. Finds the map.
3. Calls the pinch function five times.

4. Each call moves two fingers closer.
5. Checks the visual result using a screenshot assertion.

6. Pan vs Pinch

Feature	Pan	Pinch
Number of fingers	1	2
Main purpose	Move content	Zoom content
Touch points	One	Two
Common events	Start, move, end	Start, move, end
Example	Move Google Maps	Zoom Google Maps

7. Selenium vs Playwright

Feature	Selenium	Playwright
Mobile touch gestures	Supports touch actions through appropriate APIs	Supports touchscreen tap and legacy event dispatch
Pan	Can use W3C Actions	Can dispatch touch events
Pinch	Requires suitable multi-touch actions or platform support	Can dispatch two-point touch events
Event dispatch	JavaScript execution can dispatch events	`locator.dispatchEvent()`

Device emulation	Depends on browser and driver support	Built-in device descriptors

Interview point: Playwright's `locator.dispatchEvent()` is useful for dispatching custom or legacy events, but it does not reproduce every aspect of a real physical touch interaction.

8. Important interview questions and answers

Q1. What are touch events in Playwright?

Touch events are browser events used to handle touch-based interactions such as tap, swipe, pan and pinch. Playwright can dispatch these events using `locator.dispatchEvent()`.

Q2. What is `dispatchEvent()`?

It is a Playwright method used to dispatch an event on a particular element.

Example:

```
await page.locator('#map').dispatchEvent('touchstart');
```

Q3. What is the difference between `touchstart`, `touchmove` and `touchend`?

- `touchstart`: Finger touches the screen.
- `touchmove`: Finger moves.
- `touchend`: Finger is lifted.

Q4. What is a pan gesture?

A pan gesture involves moving one finger across the screen to move content, such as dragging a map.

Q5. What is a pinch gesture?

A pinch gesture uses two fingers to zoom in or zoom out.

Q6. How many touch points are required for a pinch gesture?

Two touch points.

Q7. What is `identifier` in a Touch object?

It uniquely identifies an individual touch point during a multi-touch interaction.

Q8. What is `clientX` and `clientY`?

They represent the horizontal and vertical coordinates of a touch point relative to the browser viewport.

Q9. What does `getBoundingClientRect()` do?

It returns the element's size and position relative to the viewport.

Q10. What is `Event.isTrusted`?

It indicates whether an event was generated by a real user interaction or programmatically. Events dispatched using Playwright's `dispatchEvent()` are not trusted.

Q11. Why do we use `devices['Pixel 7']`?

To configure the browser context with Pixel 7 device emulation settings.

Q12. Can `dispatchEvent()` simulate a real physical touch?

No. It dispatches a synthetic event. It does not fully reproduce real browser input or all native touch behavior.

Q13. What is the purpose of `changedTouches`?

It identifies the touch points that changed during the current touch event.

Q14. What is the purpose of `targetTouches`?

It contains the active touch points associated with the event target.

Q15. How do you verify that a map has moved after a pan gesture?

We can verify the changed map state using an application-specific assertion, such as updated coordinates or visible location text. We can also use screenshot comparison.

9. Common mistakes to avoid

1. Using the same `identifier` for both fingers in a pinch gesture.

2. Forgetting to dispatch `touchend`.
3. Starting both fingers at the same position.
4. Using incorrect `clientX` and `clientY` coordinates.
5. Assuming `dispatchEvent()` generates trusted events.
6. Assuming a screenshot assertion alone proves that a particular gesture worked.
7. Forgetting that the application must actually support legacy Touch Events.

10. MCQ practice

Test your understanding

5/5 answered

1. Which Playwright method is used to manually dispatch legacy touch events?

locator.click()
locator.dispatchEvent()
page.goto()
locator.fill()

Correct

dispatchEvent() dispatches events such as touchstart and touchmove.

2. Which event occurs when a finger first touches the screen?

touchmove
touchend
touchstart
mouseover

Correct

touchstart is fired when a touch point is placed on the screen.

3. How many touch points are required for a pinch gesture?

One
Two
Three

Four

Correct

A pinch gesture uses two fingers.

4. What does a pan gesture usually do?

Zooms in
Moves content
Refreshes a page
Submits a form

Correct

Pan moves content, such as dragging a map.

5. What is Event.isTrusted for an event dispatched with dispatchEvent()?

true
false
undefined
null

Incorrect — false

Programmatically dispatched events are synthetic and are not trusted.

4 / 5

Try again

Key takeaway for interviews: Legacy touch gestures in Playwright can be simulated by dispatching `touchstart`, `touchmove`, and `touchend` events. A pan uses one touch point, while a pinch uses two. Remember that dispatched events are synthetic, not trusted browser input.

Playwright – Trace Viewer

1. Introduction

Trace Viewer is a GUI (Graphical User Interface) tool provided by Playwright that helps us debug and investigate test execution.

It records what happened during a test, including:

- Which actions were performed.
- Which locators were used.
- How long each action took.
- Screenshots before and after actions.
- DOM snapshots.
- Browser console logs.
- Network requests and responses.
- Errors and failure details.

Simple meaning: Trace Viewer is like a recording of your Playwright test. If a test fails, you can open the recording and understand exactly what happened, step by step.

Trace viewer | Playwright
playwright.dev

Why do we use Trace Viewer?

Imagine you have an automation test:

1. Open an e-commerce website.
2. Login.
3. Search for a mobile phone.
4. Add the phone to the cart.
5. Verify the cart.

Suppose the test fails at step 4 on Jenkins, but it passes on your local machine.

How will you identify the issue?

With Trace Viewer, you can check:

- Was the Add to Cart button visible?
- Did Playwright click the correct button?
- Was the button disabled?
- Did the API request fail?
- Was there a console error?
- What was the page state before and after clicking?

This makes debugging much easier, especially for CI/CD failures.

Important interview topic
Trace Viewer is particularly useful for debugging failed tests in CI environments.

2. Opening Trace Viewer

There are two main ways to open a saved trace.

Method 1: Using Playwright CLI

Command:

```
npx playwright show-trace path/to/trace.zip
```

Example:

```
npx playwright show-trace test-results/login-test/trace.zip
```

Explanation:

- `npx`: Runs a package command.
- `playwright`: Runs the Playwright CLI.
- `show-trace`: Opens the Trace Viewer.
- `path/to/trace.zip`: Location of the recorded trace file.

The trace file is usually named `trace.zip`.

Method 2: Using the browser

Open:

Playwright Trace Viewer

Then:

1. Open the Trace Viewer website.
2. Drag and drop your `trace.zip` file.
3. Alternatively, click Select file.
4. Choose your trace file.
5. Explore the recorded test execution.

How to Use Playwright Trace Viewer for Faster Debugging
qable.io

Important: According to Playwright documentation, the browser-based Trace Viewer loads and processes the trace locally in your browser rather than uploading the trace contents to an external server.

3. Viewing Remote Traces

Sometimes test execution happens on Jenkins, GitHub Actions, or another CI server.

Instead of downloading the trace file manually, you can open it directly using its accessible URL.

Using CLI

```
npx playwright show-trace https://example.com/trace.zip
```

Using Trace Viewer website

You can also provide the remote trace URL as a query parameter:

https://trace.playwright.dev/?trace=https://demo.playwright.dev/reports/todomvc/data/e6099cadf79aa753d5500aa9508f9d1dbd87b5ee.zip

What is CORS?

CORS stands for Cross-Origin Resource Sharing.

It is a browser security mechanism that controls whether a website can access resources from another domain.

For remote traces, the server hosting the ZIP file must allow the browser to fetch it. Otherwise, CORS restrictions may prevent loading.

Interview point: Remote trace viewing helps developers debug CI failures without manually downloading trace files.

4. Recording a Trace

Before opening a trace, we need to record it.

Playwright provides different ways to record traces.

A. Recording traces locally

Command:

```
npx playwright test --trace on
```

This command runs Playwright tests and records a trace for every test.

After execution, open the HTML report:

```
npx playwright show-report
```

Then:

1. Open the HTML report.
2. Find the required test.
3. Click the trace icon.
4. Trace Viewer opens with the recorded execution.

Best Practices | Playwright
playwright.dev

Important: `--trace on` records every test, which can increase execution overhead and generate more trace files. It is useful for local debugging but is generally not recommended for every test in CI.

B. Recording traces using UI Mode

Playwright also provides UI Mode.

Command:

```
npx playwright test --ui
```

UI Mode provides an interactive interface to:

- Run tests.
- Debug tests.
- Watch test execution.
- Inspect test steps.
- Explore traces.

UI Mode automatically traces each test, making it useful during development.

C. Recording traces on CI

In real projects, tests usually run on CI tools such as Jenkins, GitHub Actions, or Azure DevOps.

We generally don't want to record traces for every successful test because it can consume extra time and storage.

Instead, Playwright recommends recording a trace when a test is retried for the first time.

playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  retries: 1,

  use: {
```

```
    trace: 'on-first-retry',
  },
});
```

Let's understand each setting.

Setting	Meaning
`retries: 1`	Retry a failed test once
`trace: 'on-first-retry'`	Record a trace when the test runs for its first retry

Example execution:

Suppose a login test fails on its first attempt.

1. Playwright detects the failure.
2. Playwright retries the test.
3. During the retry, it records the trace.
4. If the retry also fails, you can inspect the trace to understand the problem.

Available trace options

Option	Explanation
`'on'`	Records a trace for every test
`'off'`	Does not record traces
`'on-first-retry'`	Records a trace on the first retry
`'on-all-retries'`	Records traces on all retry attempts

`'retain-on-failure'`	Records tests and retains traces for failed tests; removes traces for successful tests

Which option should we use?

- Local debugging: `on`
- CI debugging with retries: `on-first-retry`
- CI debugging without retries: `retain-on-failure`
- Disable tracing: `off`

Important
`on-first-retry` is a commonly recommended CI configuration because it captures useful debugging information without recording every successful test.

D. Using tracing without Playwright Test Runner

If you are not using `@playwright/test` and are using the Playwright Library directly, you can use the `browserContext.tracing` API.

Example:

```
const context = await browser.newContext();

await context.tracing.start({
  screenshots: true,
  snapshots: true
});

// Perform browser actions here

await context.tracing.stop({
  path: 'trace.zip'
});
```

- `tracing.start()` starts recording.
- `screenshots: true` records screenshots.
- `snapshots: true` records DOM snapshots.

- `tracing.stop()` stops recording and saves the trace.

5. Trace Viewer Features

This is one of the most important parts of the topic.

Trace Viewer provides several tabs to investigate test execution.

1. Actions tab

Trace viewer | Playwright
playwright.dev

The Actions tab shows all the actions performed during test execution.

For example:

```
await page.goto('https://example.com');
await page.getByLabel('Username').fill('komal');
await page.getByLabel('Password').fill('test123');
await page.getByRole('button', { name: 'Login' }).click();
```

In the Actions tab, you can see:

Action	Meaning
`page.goto()`	Navigation to the website
`fill()`	Entering username
`fill()`	Entering password
`click()`	Clicking the Login button

When you select an action, you can inspect:

- Action snapshots.
- Action log.
- Source code location.

- Screenshots.

Before and After tabs

These help us compare the page state before and after an action.

Example:

You click the Login button.

- Before: Login form is visible.
- After: Dashboard is displayed.

This helps identify whether the action worked correctly.

2. Screenshots

Trace viewer | Playwright Python
playwright.dev

When screenshots are enabled, Trace Viewer displays screenshots as a film strip.

A film strip is a sequence of screenshots showing how the page changed during test execution.

You can:

- Hover over the film strip to preview screenshots.
- See a magnified image of the selected point.
- Identify the page state at a particular action.
- Double-click an action to inspect its time range.

Example: If a test fails after clicking Submit, you can inspect the screenshots around that action to see what appeared on the page.

3. Timeline

The timeline shows when actions occurred during test execution.

You can select a particular time range using the timeline slider.

For example:

- Select the login action's time range.
- The Actions tab shows the selected actions.
- Console logs are filtered to that period.
- Network requests are filtered to that period.

This helps narrow down the exact time when a failure happened.

4. Snapshots

Trace viewer | Playwright
playwright.dev

Snapshots are DOM representations of the page captured during test execution.

Playwright generally captures three types of snapshots for actions:

Snapshot	Meaning
Before	Page state when the action is called
Action	Page state at the time of the performed input
After	Page state after the action

Example:

Suppose Playwright clicks the Login button.

- Before: Login button is available.
- Action: Playwright performs the click.
- After: Page displays the dashboard.

The Action snapshot is particularly useful because it can highlight the DOM node and the exact click position.

Interview point: Snapshots help us understand what the DOM looked like at different stages of an action.

5. Source tab

Playwright Tutorial: Your guide to modern web testing
leantest.io

The Source tab shows the test source code.

When you select an action in the sidebar, Playwright highlights the corresponding line of code.

Example:

```
await page.getByRole('button', { name: 'Login' }).click();
```

If the click fails, you can select that action and see the exact source code line responsible for it.

6. Call tab

Vibe testing with Playwright
timdeschryver.dev

The Call tab provides details about a particular action.

It shows information such as:

- Locator used.
- Action performed.
- Execution duration.
- Whether strict mode was used.
- Keyboard key used, where applicable.

Example:

```
await page.getByRole('button', { name: 'Login' }).click();
```

The Call tab helps you understand which locator Playwright used and how long the click action took.

7. Log tab

momentic.ai

The Log tab shows the detailed internal steps performed by Playwright.

For example, when executing:

```
await page.getByRole('button', { name: 'Login' }).click();
```

The log may show:

1. Waiting for the locator.
2. Checking whether the element is visible.
3. Checking whether the element is stable.
4. Checking whether the element is enabled.
5. Scrolling the element into view if needed.
6. Performing the click.

Why is this useful?

Suppose your test is stuck because the button is not enabled.

The Log tab helps you understand what Playwright is waiting for instead of simply showing that the click timed out.

8. Errors tab

Trace viewer | Playwright
playwright.dev

The Errors tab displays test failure messages.

For example:

```
Error: Timed out 5000ms waiting for expect(locator).toBeVisible()
```

It helps you identify:

- What error occurred.
- Which assertion failed.

- Where the error happened.
- Which source code line caused the failure.

The timeline also displays a red line indicating where the error occurred.

9. Console tab

[Faster Debugging with Playwright Trace Viewer: Complete Guide
alphabin.co]

The Console tab displays console messages from:

- Browser application.
- Playwright test code.

For example:

```
console.log('Login test started');
```

Browser application:

```
console.error('Unable to fetch user details');
```

You can inspect console messages to identify JavaScript errors, warnings, and debugging information.

Important: You can double-click an action to filter console messages to the selected action's time range. Select Show all to see all logs again.

10. Network tab

[Mock APIs | Playwright Java
playwright.dev]

The Network tab shows all network requests made during test execution.

You can inspect:

Information	Meaning
Request URL	API or resource URL

Method	GET, POST, PUT, DELETE, etc.
Status code	200, 400, 404, 500, etc.
Request headers	Information sent with the request
Response headers	Information returned with the response
Request body	Data sent to the server
Response body	Data returned by the server
Duration	Time taken by the request
Content type	Type of returned content
Size	Size of the request or response data

Practical QA example:

Suppose your test clicks the Login button, but the dashboard does not open.

You can inspect the Network tab.

Status code	**Possible meaning**
200	Request succeeded
400	Bad request
401	Unauthorized
403	Forbidden

404	Resource not found
500	Internal server error

If the login API returns `401`, you can investigate authentication-related issues.

You can also double-click an action to filter network requests to the selected action's time range.

11. Metadata tab

playwright 1.35.0 on Node.js NPM
newreleases.io

The Metadata tab displays information about the test execution.

For example:

- Browser name.
- Viewport size.
- Test duration.
- Test information.
- Other execution details.

Why is it useful?

Suppose a test fails only in Firefox but passes in Chromium.

You can check Metadata to identify which browser was used during the failed execution.

12. Attachments tab

My Transition to Playwright for Frontend End-to-End Testing | Blog | Nyef.nl
nyef.nl

The Attachments tab allows you to inspect files attached to the test.

It is particularly useful for visual regression testing.

For example, you may see:

- Expected screenshot.
- Actual screenshot.
- Image difference (diff).

You can use the slider to compare the expected and actual images.

Practical example:

Expected screenshot: Login button is blue.

Actual screenshot: Login button is red.

The image diff helps identify the visual difference.

6. All Trace Viewer tabs at a glance

Tab / Feature	Purpose
Actions	Shows executed test actions
Screenshots	Shows visual page changes
Timeline	Filters actions by time
Snapshots	Shows DOM before, during and after actions
Source	Shows corresponding test code
Call	Shows details about an individual action
Log	Shows Playwright's execution steps
Errors	Shows test failure information

Console	Shows browser and test console logs
Network	Shows HTTP requests and responses
Metadata	Shows browser and execution details
Attachments	Shows screenshots and other attachments

7. Practical QA Automation Example

Let's consider an e-commerce login test.

```
import { test, expect } from '@playwright/test';

test('Login functionality', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByLabel('Username').fill('komal');

  await page.getByLabel('Password').fill('test123');

  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page.getByText('Welcome')).toBeVisible();

});
```

Suppose this test fails on Jenkins.

How will you debug it using Trace Viewer?

1. Open the HTML test report using `npx playwright show-report`.
2. Find the failed Login test.
3. Click its trace icon.
4. Open the Actions tab.
5. Select the failed Login action.

6. Check the Before and After snapshots.
7. Check the Log tab to understand why the action failed.
8. Check the Network tab to investigate API failures.
9. Check the Console tab for JavaScript errors.
10. Check the Errors tab for the exact failure message.

This helps us identify whether the issue is related to the locator, application, API, or test script.

8. Important interview questions and answers

Q1. What is Playwright Trace Viewer?

Trace Viewer is a GUI debugging tool that allows us to inspect recorded Playwright test executions, including actions, screenshots, DOM snapshots, logs, network requests and errors.

Q2. Why do we use Trace Viewer?

We use it to debug failed automation tests, especially failures occurring in CI environments where we cannot directly observe the browser execution.

Q3. How do you open a trace file?

Using the Playwright CLI:

```
npx playwright show-trace trace.zip
```

Or upload the ZIP file at https://trace.playwright.dev/.

Q4. What is the default trace file name?

The commonly generated trace archive is `trace.zip`.

Q5. How do you enable tracing in Playwright?

Using the command:

```
npx playwright test --trace on
```

Or through the configuration file:

```
use: {
  trace: 'on'
}
```

Q6. Which trace option is recommended for CI?

`on-first-retry` is commonly recommended because it records a trace when a failed test is retried for the first time.

Q7. What is the difference between on and `on-first-retry`?

- on: Records traces for every test execution.
- `on-first-retry`: Records a trace on the first retry of a failed test.

Q8. What is the difference between `on-first-retry` and `on-all-retries`?

- `on-first-retry`: Records a trace on the first retry only.
- `on-all-retries`: Records a trace on every retry attempt.

Q9. What is the purpose of the Actions tab?

It displays all executed actions, the locators used, execution duration, snapshots and source code locations.

Q10. What are DOM snapshots?

DOM snapshots are captured representations of the page at different stages of an action: Before, Action and After.

Q11. What is the purpose of the Network tab?

It displays network requests and provides information such as request URL, method, status code, headers, request body and response body.

Q12. What is the purpose of the Console tab?

It displays console messages from the browser and Playwright test code, helping us identify JavaScript errors and debugging messages.

Q13. What is the difference between the Log tab and Console tab?

- Log tab: Shows Playwright's internal action execution steps.
- Console tab: Shows browser and test console messages.

Q14. What is the purpose of the Metadata tab?

It displays test execution information such as browser, viewport size and test duration.

Q15. What is the Attachments tab?

It displays files attached to the test, including expected screenshots, actual screenshots and image differences.

Q16. Can we view traces without downloading them from CI?

Yes. If the trace file is available at an accessible URL, we can open it using the CLI or pass its URL to Trace Viewer.

Q17. Does Trace Viewer help identify API failures?

Yes. The Network tab allows us to inspect API requests, response status codes, headers and response bodies.

Q18. Can we use Trace Viewer with Playwright Library instead of Playwright Test?

Yes. We can use the `browserContext.tracing.start()` and `browserContext.tracing.stop()` APIs.

Q19. What is the difference between a screenshot and a trace?

A screenshot captures a single visual state of a page, whereas a trace records a much broader view of test execution, including actions, snapshots, logs and network activity.

Q20. Does Trace Viewer help debug tests that pass locally but fail on Jenkins?

Yes. We can configure tracing on CI and inspect the recorded execution to identify the cause of the failure.

9. Common mistakes to avoid

1. Using `--trace on` for every CI test unnecessarily, which can increase overhead.
2. Forgetting to configure trace recording before test execution.
3. Assuming a trace will exist if tracing was not enabled.
4. Confusing the Log tab with the Console tab.

5. Confusing screenshots with DOM snapshots.
6. Assuming `on-first-retry` records a trace on the initial attempt.
7. Sharing trace files without checking them for sensitive information.

Security reminder
Trace files may contain sensitive information, such as page content, request details, authentication-related data or personal information. Store and share them carefully.

10. Quick revision notes

Remember these commands and configurations for interviews:

Requirement	Command / Configuration
Run tests with tracing	`npx playwright test --trace on`
Open a trace	`npx playwright show-trace trace.zip`
Open HTML report	`npx playwright show-report`
Run UI Mode	`npx playwright test --ui`
Record on first retry	`trace: 'on-first-retry'`
Record only failed tests	`trace: 'retain-on-failure'`
Start tracing with Library	`browserContext.tracing.start()`
Stop tracing with Library	`browserContext.tracing.stop()`

11. MCQ practice

Playwright Trace Viewer Quiz

8/8 answered

1. What is the main purpose of Playwright Trace Viewer?

Writing test cases
Debugging recorded test execution
Installing browsers
Creating locators

Correct

Trace Viewer helps inspect test actions, screenshots, snapshots, logs and network requests.

2. Which command opens a saved trace file?

npx playwright test
npx playwright show-report
npx playwright show-trace trace.zip
npx playwright install

Correct

show-trace opens a saved trace archive.

3. Which trace option is commonly recommended for CI?

on
off
on-first-retry
on-all-retries

Correct

on-first-retry captures a trace when a failed test is retried for the first time.

4. Which tab displays API requests and response details?

Source
Network
Metadata
Attachments

Correct

The Network tab displays requests, methods, status codes, headers and bodies.

5. Which tab displays Playwright's internal execution steps?

Console
Log
Errors
Metadata

Correct

The Log tab shows steps such as waiting, scrolling and checking actionability.

6. What are the three common action snapshot stages?

Start, Run, Stop
Before, Action, After
Open, Click, Close
Request, Response, Complete

Correct

Playwright captures Before, Action and After snapshots.

7. Which command opens the HTML report?

npx playwright show-report
npx playwright show-trace
npx playwright test --ui
npx playwright install

Correct

show-report opens the HTML test report.

8. Which API starts tracing when using Playwright Library?

page.trace.start()
browser.tracing.start()
browserContext.tracing.start()
locator.tracing()

Correct answer: browserContext.tracing.start()

The tracing API belongs to BrowserContext.

7 / 8

Try again

Most important interview answer to remember:

"Playwright Trace Viewer is a debugging tool that helps us investigate test execution, especially when tests fail in CI. It provides action history, screenshots, DOM snapshots, source code, execution logs, console messages, network requests and error details. We commonly configure `trace: 'on-first-retry'` in `playwright.config.ts` to capture traces when failed tests are retried."

Next topic: Send me the next Playwright documentation section, and I'll explain it in the same interview-preparation format.

Playwright – Videos

1. Introduction

Playwright provides a feature to record videos of test execution.

It records what happens on the browser screen while an automated test is running.

For example, suppose you are testing an e-commerce application:

1. Open the application.
2. Login.
3. Search for a product.
4. Add the product to the cart.
5. Verify the cart.

Playwright can record all these actions in a video. If the test fails, you can watch the recording to understand what happened.

Simple meaning: Video recording helps QA engineers visually debug test failures without manually rerunning the test.

[How to Record Video in Playwright with Examples]

2. Record video

Video recording is controlled using the `video` option in `playwright.config.ts`.

By default, video recording is disabled.

There are four important video modes:

Mode	Meaning
`off`	Video recording is disabled.
on	Record video for every test.
`retain-on-failure`	Record every test, but keep videos only for failed tests.
`on-first-retry`	Record video only when a test is retried for the first time.

3. Explanation of each video mode

1. `off`

```
use: {
  video: 'off',
}
```

- Playwright does not record videos.
- This is the default setting.

2. on

```
use: {
  video: 'on',
}
```

- Records video for every test.

- Videos are retained for both passed and failed tests.
- Requires more storage.

3. `retain-on-failure`

```
use: {
  video: 'retain-on-failure',
}
```

- Records video for every test.
- Deletes videos for successful tests.
- Keeps videos for failed tests.

This is useful when you want to investigate failed test cases without storing unnecessary successful recordings.

4. `on-first-retry`

```
use: {
  video: 'on-first-retry',
}
```

- Does not record video during the initial test execution.
- Records video when the test is retried for the first time.
- Useful for debugging intermittent failures.

Interview important
`on-first-retry` is useful in CI pipelines because it records videos when a test fails and is retried, rather than recording every test execution.

4. Configure video recording

File: `playwright.config.ts`

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    video: 'on-first-retry',
```

```
  },
});
```

Explanation

Step 1: Import `defineConfig`

```
import { defineConfig } from '@playwright/test';
```

Used to define the Playwright configuration.

Step 2: Configure the video option

```
use: {
  video: 'on-first-retry',
}
```

This tells Playwright to record videos when a test is retried for the first time.

Step 3: Run the tests

```
npx playwright test
```

Step 4: Find the recorded video

Videos are generally saved inside:

```
test-results/
```

Example folder structure:

```
PlaywrightProject/
│
├── tests/
│   └── login.spec.ts
│
├── test-results/
│   └── login-test-retry1/
│       └── video.webm
│
├── playwright.config.ts
└── package.json
```

The exact folder and filename can vary depending on the test and Playwright configuration.

5. Important: When is the video saved?

Playwright saves the video when the browser page or browser context is closed.

For normal Playwright Test execution, Playwright handles this automatically.

But if you create a browser context manually, you must close it.

Example:

```
const context = await browser.newContext({
  recordVideo: {
    dir: 'videos/',
  },
});

const page = await context.newPage();

await page.goto('https://example.com');

await context.close();
```

Why is `context.close()` important?

Because the video file is finalized and saved when the browser context closes.

Remember
Do not try to access the completed video before closing the page or browser context.

6. Video size and annotations

Playwright also allows you to customize:

1. Video width and height.
2. Action annotations.
3. Test information annotations.

A. Video size

Example:

```
video: {
  mode: 'on-first-retry',
  size: {
    width: 640,
    height: 480,
  },
}
```

Property	Meaning
`width`	Video width in pixels
`height`	Video height in pixels

Here, the recorded video size is 640 × 480 pixels.

By default, the video is based on the viewport size and scaled down to fit within 800 × 800 pixels.

B. Action annotations

When you configure `show: { actions }`, Playwright highlights test actions in the recorded video.

For example:

- Click Login button.
- Fill username.
- Fill password.
- Click Submit.

The video can display an element outline and a subtitle describing the action.

Example:

```
show: {
  actions: {
    duration: 500,
    position: 'top-right',
    fontSize: 14,
```

```
  },
}
```

Property	Meaning
`duration`	How long the annotation appears
`position`	Where the annotation appears
`fontSize`	Annotation text size

Here, the action annotation appears for 500 milliseconds in the top-right corner with a font size of 14.

C. Test annotations

When you configure `show: { test }`, Playwright displays information about the currently running test in the video.

Example:

```
show: {
  test: {
    level: 'step',
    position: 'top-left',
    fontSize: 12,
  },
}
```

Property	Meaning
`level`	Controls the level of test information displayed
`position`	Position of the annotation
`fontSize`	Text size

The `level` can be configured to control the amount of test information shown.

7. Complete configuration example

This combines video recording, custom video size and annotations.

playwright.config.ts

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    video: {
      mode: 'on-first-retry',

      size: {
        width: 640,
        height: 480,
      },

      show: {
        actions: {
          duration: 500,
          position: 'top-right',
          fontSize: 14,
        },

        test: {
          level: 'step',
          position: 'top-left',
          fontSize: 12,
        },
      },
    },
  },
});
```

What does this configuration do?

- Records videos on the first retry.
- Sets video size to 640 × 480.
- Displays action annotations for 500 milliseconds.

- Displays test information in the video.
- Places action information at the top-right and test information at the top-left.

8. Accessing video using `page.video()`

For scenarios involving multiple pages, Playwright provides:

`page.video()`

It returns the video object associated with that page.

Example:

```
const path = await page.video().path();
```

Explanation:

- `page.video()` gets the video object.
- `.path()` gets the path of the recorded video file.
- `await` waits for the operation to complete.

Example:

```
const video = page.video();

await page.goto('https://example.com');

await page.close();

const path = await video.path();

console.log(path);
```

Here, the page is closed before accessing the video path because the video is only available after the page or browser context is closed.

Note: This assumes video recording has been enabled for the browser context.

9. Practical QA automation example

Suppose you are testing an e-commerce login and cart functionality.

```
import { test, expect } from '@playwright/test';

test('Login and add product to cart', async ({ page }) => {

  await page.goto('https://example.com');

  await page.getByLabel('Username').fill('testuser');

  await page.getByLabel('Password').fill('password123');

  await page.getByRole('button', { name: 'Login' }).click();

  await page.getByRole('link', { name: 'Products' }).click();

  await page.getByRole('button', { name: 'Add to cart' }).click();

  await expect(page.getByText('Product added')).toBeVisible();

});
```

Configuration:

```
use: {
  video: 'retain-on-failure',
}
```

How does it help QA?

Suppose the login test fails because the Add to Cart button is not visible.

Instead of manually executing the test again, you can:

1. Open the recorded video.
2. Watch the test execution.
3. Identify where the failure happened.
4. Understand the browser's behavior.
5. Debug the issue.

10. Trace Viewer vs Videos

This is an important interview comparison.

Feature	Trace Viewer	Video Recording
Purpose	Detailed test debugging	Visual test execution recording
Actions	Shows individual actions	Shows actions visually
DOM snapshots	Yes	No interactive DOM snapshots
Source code	Shows source location	Not a detailed source panel
Network requests	Yes	No
Console logs	Yes	No
Screenshots	Yes	Video frames
Best for	Detailed debugging	Understanding visual execution

Remember:

- Video tells you what happened visually.
- Trace Viewer tells you what happened in detail, including actions, DOM snapshots, logs and network requests.

11. Interview questions and answers

Q1. What is video recording in Playwright?

Video recording is a Playwright feature that captures browser test execution so QA engineers can visually debug test failures.

Q2. How do you enable video recording in Playwright?

By setting the `video` option in `playwright.config.ts`.

```
use: {
  video: 'on',
}
```

Q3. What are the four video recording modes?

`off`, `on`, `retain-on-failure`, and `on-first-retry`.

Q4. Which video mode records only during the first retry?

`on-first-retry`.

Q5. Which mode keeps videos only for failed tests?

`retain-on-failure`.

Q6. Where are Playwright videos stored?

Generally inside the `test-results` directory.

Q7. When does Playwright save the video?

When the page or browser context is closed.

Q8. Why do we use `browserContext.close()`?

It closes the browser context and finalizes the recorded video.

Q9. What is `page.video()`?

It returns the video object associated with a particular page.

Q10. How do you get the recorded video file path?

```
const path = await page.video().path();
```

Q11. Can we customize the video dimensions?

Yes. We can configure `size` with `width` and `height`.

Q12. What are action annotations?

They visually highlight test actions in the recorded video with an element outline and action title.

Q13. What is the purpose of `show: { test }`?

It displays information about the current test in the recorded video.

Q14. What is the difference between on and `on-first-retry`?

- on: Records every test execution.
- `on-first-retry`: Records only when a test is retried for the first time.

Q15. What is the difference between Trace Viewer and video recording?

Trace Viewer provides detailed debugging information such as actions, DOM snapshots, console logs and network requests. Video recording captures the visual browser execution.

12. Common mistakes

1. Forgetting to enable the `video` option.
2. Expecting videos to be saved before the browser context closes.
3. Using `page.video().path()` when video recording is disabled.
4. Confusing `on-first-retry` with `retain-on-failure`.
5. Expecting video recordings to contain DOM snapshots or network logs.
6. Recording every test unnecessarily, increasing storage usage.

13. MCQ practice

Test your knowledge

6/6 answered

1. Which option records video for every test?

off
on
on-first-retry
retain-on-failure

Correct

'on' records and retains videos for every test.

2. Which option keeps videos only for failed tests?

on
off
retain-on-failure
on-first-retry

Correct

retain-on-failure removes videos from successful test runs.

3. When is a Playwright video finalized?

When page.goto() runs
When browser context closes
When locator.click() runs
When expect() runs

Correct

The video is saved when its page or browser context is closed.

4. Which method gets the video object for a page?

page.screenshot()
page.video()
page.trace()
page.record()

Correct

page.video() returns the associated video object.

5. Which feature displays action outlines and titles in a video?

show.actions
show.network
show.console
show.snapshot

Correct

show.actions configures action annotations in the recording.

6. Where are test videos generally stored?

playwright.config.ts
node_modules
test-results
tests

Correct

The default test output directory is typically test-results.

6 / 6

Try again

14. Quick revision notes

Topic	Remember
Video configuration	`video`
Default mode	`off`
Record every test	on
Keep failed test videos	`retain-on-failure`
Record first retry	`on-first-retry`
Video output	`test-results`
Access video object	`page.video()`
Get video path	`page.video().path()`

Finalize video	Close page/context
Customize dimensions	`size`
Action annotation	`show.actions`
Test annotation	`show.test`

Most important interview question: What is the difference between `retain-on-failure` and `on-first-retry`?

- `retain-on-failure`: Records every test, then removes successful test videos.
- `on-first-retry`: Records only the first retry of a test.

You can send the next Playwright documentation section, and I'll explain it in the same format.

Playwright – Visual Comparisons

1. Introduction

Playwright provides a feature called Visual Comparisons (also known as Visual Regression Testing).

It helps us check whether the UI of a web application has changed unexpectedly by comparing screenshots.

For example, suppose you are testing an e-commerce application.

Your expected login page looks like this:

Ecommerce Login designs, themes, templates and downloadable graphic elements on Dribbble
dribbble.com

After a developer changes the CSS, the login button moves to another position or its color changes.

Visual comparison helps detect these UI differences automatically.

Simple meaning: Playwright takes a screenshot of the current page and compares it with a previously saved expected screenshot.

Interview important
Visual comparison is mainly used for Visual Regression Testing.

2. How Visual Comparison works

Playwright uses:

```
await expect(page).toHaveScreenshot();
```

There are three main steps:

1. First execution – Generate baseline

Playwright captures a screenshot and saves it as the expected reference image.

2. Subsequent execution – Compare

Playwright captures a new screenshot and compares it with the baseline.

3. Result – Pass or fail

If the difference is within the configured tolerance, the test passes. Otherwise, it fails.

Example

Expected screenshot:

17 Best Login Page Design Examples and Best Practices
agentestudio.com

Actual screenshot:

Belgrade, a Drupal Commerce Theme, Evolves to Match Real-World Requirements | Centarro
centarro.io

If the button color or layout changes unexpectedly, Playwright detects the visual difference.

3. Basic code example

```
import { test, expect } from '@playwright/test';

test('example test', async ({ page }) => {
  await page.goto('https://playwright.dev');

  await expect(page).toHaveScreenshot();
});
```

Step-by-step explanation

Step 1: Import packages

```
import { test, expect } from '@playwright/test';
```

- `test`: Used to write test cases.
- `expect`: Used for assertions.

Step 2: Open the application

```
await page.goto('https://playwright.dev');
```

Navigates to the Playwright website.

Step 3: Take and compare screenshot

```
await expect(page).toHaveScreenshot();
```

- Captures a screenshot of the page.
- On the first run, generates the reference screenshot.
- On subsequent runs, compares the actual screenshot with the reference.

Important: On the first execution, Playwright normally generates the baseline rather than performing a comparison against an existing image. The first run may report a missing snapshot, which is expected.

4. Generating screenshots (Baseline)

What is a baseline screenshot?

A baseline screenshot is the original expected screenshot against which future screenshots are compared.

It is also called a golden screenshot or reference screenshot.

Suppose you run:

```
npx playwright test
```

For the first time, you may see an error similar to:

```
Error: A snapshot doesn't exist at
example.spec.ts-snapshots/example-test-1-chromium-darwin.png,
writing actual.
```

Don't worry! This generally means that the reference screenshot does not exist yet.

Playwright:

1. Runs the test.
2. Captures screenshots until two consecutive screenshots match.
3. Saves the last screenshot as the baseline.
4. Uses that image for future comparisons.

Folder structure

```
PlaywrightProject/
│
├── tests/
│   └── example.spec.ts
│
├── tests/
│   └── example.spec.ts-snapshots/
│       └── example-test-1-chromium-darwin.png
│
└── playwright.config.ts
```

The snapshot folder name starts with the test filename.

Understanding snapshot filename

Example:

`example-test-1-chromium-darwin.png`

Part	Meaning
`example-test`	Test name
`1`	Snapshot sequence number
`chromium`	Browser name
`darwin`	Platform (macOS)
`.png`	Image format

Important: If you run multiple projects in your Playwright configuration, the project name is used instead of the browser/platform naming combination.

5. Providing a custom screenshot name

By default, Playwright generates a screenshot name automatically.

We can also provide our own name.

```
await expect(page).toHaveScreenshot('landing.png');
```

Here, `landing.png` is the name of the reference screenshot.

Another example:

```
await expect(page).toHaveScreenshot('login-page.png');
```

This makes the screenshot easier to identify.

PNG vs WebP

Playwright stores screenshots as PNG by default.

We can also use WebP format.

```
await expect(page).toHaveScreenshot('landing.webp');
```

Format	Meaning
PNG	Default screenshot format
WebP	Alternative image format; supports lossless image storage

Configure WebP globally

File: `playwright.config.ts`

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  expect: {
    toHaveScreenshot: {
      type: 'webp',
    },
  },
});
```

Now unnamed screenshot snapshots use WebP format.

6. Updating screenshots

Why do we update screenshots?

Suppose the developer intentionally changes the login page design:

- Changes the button color.
- Changes the logo.
- Updates the font.

- Changes the page layout.

The existing baseline screenshot still contains the old design.

Therefore, the visual comparison test may fail.

If the UI changes are expected and approved, we need to update the baseline screenshot.

Command

```
npx playwright test --update-snapshots
```

Meaning: This command updates the expected reference screenshots based on the current test execution.

Important
Never blindly update snapshots just to make a failing test pass. First verify whether the UI change is intentional and correct.

7. Hover effects

What is a hover effect?

A hover effect occurs when the mouse pointer moves over an element.

For example:

- Button changes color.
- Menu opens.
- Image becomes highlighted.
- Tooltip appears.

When Playwright captures a screenshot, any active hover effect may also be captured.

Problem

Suppose your expected screenshot shows a normal button, but your actual screenshot captures the button in its hover state.

The screenshots may differ, causing the test to fail.

Solution

Move the mouse outside the page content before capturing the screenshot.

```
await page.mouse.move(-1, -1);

await expect(page).toHaveScreenshot();
```

Explanation:

- `page.mouse.move()` moves the mouse pointer.
- `-1, -1` places it outside the normal viewport.
- This helps avoid hover effects before capturing the screenshot.

8. Options – `maxDiffPixels` and `threshold`

Sometimes two screenshots have small differences because of font rendering, anti-aliasing or other rendering variations.

Playwright allows us to configure acceptable visual differences.

Two important options are:

1. `threshold`
2. `maxDiffPixels`

A. threshold

`threshold` defines how different the color of an individual pixel can be before Playwright considers it a difference.

Default value:

```
threshold: 0.2
```

Range:

\[0 \leq threshold \leq 1 \]

Value	Meaning

`0`	Very strict pixel comparison
`0.2`	Default tolerance
`0.5`	More tolerant
`1`	Very lax

Simple example: If the actual pixel color is slightly different from the expected pixel color, `threshold` controls whether that difference is considered significant.

B. maxDiffPixels

This defines the maximum number of pixels allowed to differ between the screenshots.

Example:

```
await expect(page).toHaveScreenshot({
  maxDiffPixels: 100,
});
```

Meaning: Allow up to 100 differing pixels. If the difference exceeds the configured limit, the assertion fails.

Difference between threshold and maxDiffPixels

Feature	threshold	maxDiffPixels
Checks	Per-pixel color difference	Number of differing pixels
Purpose	Controls pixel sensitivity	Controls allowed difference count
Default	0.2	Unset
Example	0.1	100

Combined example

```
import { test, expect } from '@playwright/test';

test('visual comparison', async ({ page }) => {
  await page.goto('https://playwright.dev');

  await expect(page).toHaveScreenshot({
    maxDiffPixels: 100,
    threshold: 0.1,
  });
});
```

Here:

- `threshold: 0.1` makes pixel color comparison stricter than the default.
- `maxDiffPixels: 100` allows up to 100 differing pixels.

Configure globally

Instead of writing these options in every test, we can configure them globally.

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  expect: {
    toHaveScreenshot: {
      maxDiffPixels: 100,
      threshold: 0.1,
    },
  },
});
```

These settings can also be configured per project.

9. stylePath

What is `stylePath`?

`stylePath` allows us to apply a custom CSS stylesheet before taking a screenshot.

Why do we need it?

Some web elements change frequently and can cause unnecessary screenshot failures.

Examples:

- Advertisements.
- Animations.
- Dynamic timestamps.
- Embedded content.
- Frequently changing banners.

We can hide selected elements using a custom stylesheet.

Step 1: Create a CSS file

File: `screenshot.css`

```
iframe {
  visibility: hidden;
}
```

This hides iframe content visually during screenshot capture.

Step 2: Use `stylePath`

```
import { test, expect } from '@playwright/test';
import path from 'node:path';

test('visual comparison', async ({ page }) => {
  await page.goto('https://playwright.dev');

  await expect(page).toHaveScreenshot({
    stylePath: path.join(__dirname, 'screenshot.css'),
  });
});
```

Explanation

- `stylePath` specifies the location of the CSS file.

- `path.join()` combines directory and filename into a valid path.
- Playwright applies the stylesheet during screenshot capture.

Configure globally

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  expect: {
    toHaveScreenshot: {
      stylePath: './screenshot.css',
    },
  },
});
```

Interview point: `stylePath` helps make visual tests more stable by hiding dynamic or volatile elements.

10. Non-image snapshots

Visual comparisons are not limited to screenshots.

Playwright also supports snapshot comparison for:

- Text.
- Strings.
- Binary data.
- Other supported values.

The method used is:

```
expect(value).toMatchSnapshot();
```

Example

```
import { test, expect } from '@playwright/test';

test('text snapshot comparison', async ({ page }) => {
  await page.goto('https://playwright.dev');
```

```
  expect(
    await page.textContent('.hero__title')
  ).toMatchSnapshot('hero.txt');
});
```

Step-by-step explanation

Step 1: Open the website

```
await page.goto('https://playwright.dev');
```

Step 2: Get text from an element

```
await page.textContent('.hero__title');
```

This retrieves the text content of the element matching `.hero__title`.

Step 3: Compare text with the baseline

```
expect(
  await page.textContent('.hero__title')
).toMatchSnapshot('hero.txt');
```

Playwright saves the expected text in a snapshot file and compares future results against it.

Difference between the two methods

Method	Purpose
`toHaveScreenshot()`	Compares screenshots/images
`toMatchSnapshot()`	Compares text or arbitrary supported data

Example:

```
// Screenshot comparison
await expect(page).toHaveScreenshot('homepage.png');

// Text comparison
expect(pageTitle).toMatchSnapshot('title.txt');
```

11. Important warning: Browser rendering differences

Playwright warns that screenshots can differ depending on:

- Operating system.
- Browser version.
- Browser settings.
- Hardware.
- Power source.
- Headless or headed mode.
- Font rendering.

For example, a screenshot generated on Windows may look slightly different from one generated on macOS.

Best practice: Generate and compare baseline screenshots in the same environment.

For CI/CD, try to use the same operating system, browser version and configuration when generating and validating screenshots.

12. Practical QA automation example

Suppose you are testing the login page of an application.

You want to verify:

- Logo position.
- Login form layout.
- Username field.
- Password field.
- Login button.
- Overall page design.

```
import { test, expect } from '@playwright/test';

test('Login page visual validation', async ({ page }) => {

  await page.goto('https://example.com/login');

  await page.mouse.move(-1, -1);

  await expect(page).toHaveScreenshot('login-page.png');

});
```

How does it work?

1. Opens the login page.
2. Moves the mouse away to avoid hover effects.
3. Captures the login page screenshot.
4. Compares it against the saved baseline.
5. Passes or fails depending on the visual difference.

Real-world QA scenario:

A developer accidentally changes the login button's position while modifying CSS.

Functional testing might still pass because the login button works. But visual comparison can detect that the page layout has changed.

This is why functional testing and visual testing are both important.

13. Common mistakes to avoid

1. Forgetting to generate the baseline screenshot before expecting a comparison.
2. Updating snapshots without checking whether the UI change is intentional.
3. Generating baselines on one OS and comparing on another.
4. Ignoring hover effects during screenshot capture.
5. Setting `maxDiffPixels` too high, which can hide genuine UI defects.

6. Setting `threshold` too high, making comparisons less strict.
7. Forgetting to commit snapshot folders to Git.
8. Including dynamic content without stabilizing or masking it.

14. Interview questions and answers

Q1. What is Visual Regression Testing?

Visual Regression Testing checks whether the UI has changed unexpectedly by comparing the current screenshot with an expected baseline screenshot.

Q2. Which Playwright method is used for screenshot comparison?

```
await expect(page).toHaveScreenshot();
```

Q3. What is a baseline screenshot?

It is the reference screenshot generated during the initial execution and used for future comparisons.

Q4. What happens during the first execution of `toHaveScreenshot()`?

Playwright captures screenshots until two consecutive screenshots match, then saves the final screenshot as the reference image.

Q5. How do you update baseline screenshots?

```
npx playwright test --update-snapshots
```

Q6. Where are screenshot snapshots stored?

Inside a snapshot directory associated with the test file, generally named:

```
example.spec.ts-snapshots/
```

Q7. What is `maxDiffPixels`?

It defines the maximum number of differing pixels allowed between the actual and expected screenshots.

Q8. What is `threshold`?

It controls the acceptable color difference between corresponding pixels. Its default value is `0.2`.

Q9. What is `stylePath`?

It allows us to apply a custom CSS stylesheet during screenshot capture to hide dynamic elements and improve screenshot consistency.

Q10. How do you avoid hover effects during screenshot capture?

```
await page.mouse.move(-1, -1);
await expect(page).toHaveScreenshot();
```

Q11. What is the difference between `toHaveScreenshot()` and `toMatchSnapshot()`?

- `toHaveScreenshot()` compares screenshots.
- `toMatchSnapshot()` compares text or other supported data.

Q12. Why should we use the same environment for visual testing?

Because operating systems, browser versions, fonts and rendering settings can affect screenshots.

Q13. Can we provide our own screenshot name?

Yes.

```
await expect(page).toHaveScreenshot('homepage.png');
```

Q14. What is the default screenshot format?

PNG.

Q15. Can Playwright store screenshots in WebP format?

Yes. We can specify `.webp` in the screenshot name or configure the `type` option.

Q16. Why should we commit snapshot folders to Git?

Because the baseline screenshots must be available to other developers and CI pipelines for consistent visual comparison.

Q17. What is the difference between functional testing and visual regression testing?

Functional testing verifies whether application features work correctly. Visual regression testing verifies whether the UI looks as expected.

15. Selenium vs Playwright – Visual comparison

Feature	Selenium	Playwright
Screenshot capture	`getScreenshotAs()`	`toHaveScreenshot()`
Automatic baseline comparison	Requires additional implementation/tools	Built into Playwright Test
Pixel difference validation	Requires external library/tool	Built-in screenshot assertion
Update baseline	Depends on implementation/tool	`--update-snapshots`
Text snapshot comparison	Custom implementation	`toMatchSnapshot()`

Interview tip: Selenium supports screenshots, but Playwright Test provides built-in visual snapshot assertions and baseline management.

16. MCQ practice

Test your understanding

8/8 answered

1. Which method is used for visual screenshot comparison?

toMatchSnapshot()
toHaveScreenshot()
toBeVisible()
toContainText()

Incorrect — toHaveScreenshot()

toHaveScreenshot() captures and compares screenshots.

2. What is a baseline screenshot?

A failed screenshot
A reference screenshot
A browser screenshot only
A network screenshot

Correct

The baseline is the expected reference image for future comparisons.

3. Which command updates baseline screenshots?

npx playwright test --update-snapshots
npx playwright show-report
npx playwright test --debug
npx playwright install

Correct

The --update-snapshots flag updates reference snapshots.

4. What is the default threshold value?

0
0.1
0.2
1

Correct

The default threshold is 0.2.

5. What does maxDiffPixels control?

Screenshot size
Maximum allowed differing pixels
Browser version
Image quality

Correct

It sets the maximum acceptable number of differing pixels.

6. Which option applies a custom CSS stylesheet during screenshot capture?

type

threshold
stylePath
snapshotPathTemplate

Correct

stylePath specifies a CSS file to apply during screenshot capture.

7. Which method compares text or arbitrary supported data with a snapshot?

toHaveScreenshot()
toMatchSnapshot()
toBeEnabled()
toHaveURL()

Correct

toMatchSnapshot() supports text and other snapshot data.

8. Why should baseline screenshots be generated in the same environment?

To increase test count
To avoid rendering differences
To reduce locators
To enable retries

Correct

OS, fonts, browser versions and rendering can change screenshots.

7 / 8

Try again

17. Quick revision notes

Topic	Remember
Visual testing	Visual Regression Testing

Screenshot assertion	`toHaveScreenshot()`
Baseline	Expected reference image
Update snapshots	`--update-snapshots`
Default format	PNG
Alternative format	WebP
Pixel tolerance	`threshold`
Allowed different pixels	`maxDiffPixels`
Custom CSS	`stylePath`
Text comparison	`toMatchSnapshot()`
Avoid hover	`page.mouse.move(-1, -1)`
Snapshot folder	`test-file-snapshots`

Most important interview concept:

Functional testing checks whether a button works. Visual regression testing checks whether that button looks as expected.

Both are important in a QA Automation Framework.

Send me the next Playwright documentation section whenever you're ready.

Playwright – WebView2

1. Introduction in simple language

WebView2 is a Microsoft technology that allows developers to display web pages inside a desktop application.

For example, imagine a Windows desktop application built using C# and WinForms. Instead of opening a separate browser, the application can display a website inside its own window using Microsoft Edge.

That embedded browser component is called Microsoft Edge WebView2.

Playwright can automate and test the web content displayed inside WebView2.

Get started with WebView2 in WinForms apps - Microsoft Edge Developer documentation | Microsoft Learn
learn.microsoft.com

How does it work?

- WinForms: Used to build the Windows desktop application.
- WebView2: Displays web content inside that application.
- Microsoft Edge: Browser engine used by WebView2.
- Playwright: Automates and tests the web content.
- CDP: Chrome DevTools Protocol, used by Playwright to connect to WebView2.

Important interview point

Playwright connects to WebView2 using:

```
browserType.connectOverCDP()
```

CDP stands for Chrome DevTools Protocol.

2. Overview – Enabling remote debugging

Before Playwright can automate WebView2, the application must allow remote debugging connections.

For this, we enable CDP using a remote debugging port.

Example:

```
--remote-debugging-port=9222
```

Here, 9222 is the port number through which Playwright connects to WebView2.

You can use another unused port too.

C# code explanation

```
await this.webView.EnsureCoreWebView2Async(
    await CoreWebView2Environment.CreateAsync(
        null,
        null,
        new CoreWebView2EnvironmentOptions()
        {
            AdditionalBrowserArguments =
                "--remote-debugging-port=9222",
        }
    )
).ConfigureAwait(false);
```

Step-by-step:

1. `EnsureCoreWebView2Async()` initializes the WebView2 control.
2. `CoreWebView2Environment.CreateAsync()` creates the WebView2 environment.
3. `CoreWebView2EnvironmentOptions()` provides browser configuration.
4. `AdditionalBrowserArguments` passes extra arguments to Edge.
5. `--remote-debugging-port=9222` enables CDP connections.

In simple words: this code starts WebView2 with remote debugging enabled so Playwright can connect to it.

3. Connecting Playwright to WebView2

Once the WebView2 desktop application is running, Playwright can connect to it.

```
const browser = await playwright.chromium.connectOverCDP(
  'http://localhost:9222'
);

const context = browser.contexts()[0];

const page = context.pages()[0];
```

Line-by-line explanation:

Code	Meaning
`connectOverCDP()`	Connects Playwright to the running WebView2 application
`http://localhost:9222`	Address and port where CDP is listening
`browser.contexts()[0]`	Gets the first existing browser context
`context.pages()[0]`	Gets the first open page inside that context

Important methods:

- `browser.contexts()` – Returns the browser contexts.
- `context.pages()` – Returns all pages in a context.
- `page` – Represents the web page that Playwright will automate.

Remember: In this example, Playwright is connecting to an already running WebView2 application. It is not launching a normal Chrome browser.

4. Waiting for WebView2 initialization

Sometimes Playwright tries to connect before WebView2 is completely initialized.

To avoid this, we can listen for the `CoreWebView2InitializationCompleted` event.

```
this.webView.CoreWebView2InitializationCompleted += (_, e) =>
{
    if (e.IsSuccess)
```

```
    {
        Console.WriteLine("WebView2 initialized");
    }
};
```

Explanation:

- `CoreWebView2InitializationCompleted` – Event triggered when initialization finishes.
- `e.IsSuccess` – Checks whether initialization was successful.
- `Console.WriteLine()` – Prints a message when WebView2 is ready.

In simple words: this event tells us when WebView2 is ready to accept automation connections.

5. Writing and running tests

Very important concept

Problem: Multiple tests interfering with each other

By default, WebView2 instances can share the same user data directory.

The user data directory stores browser-related information such as:

- Cookies
- Cache
- Local storage
- Session information
- Browser profile data

Suppose two tests run simultaneously:

- Test 1 logs in as User A.
- Test 2 logs in as User B.
- Both tests share the same browser data directory.

Their data may interfere with each other.

Solution: Use a separate user data directory

Set:

`WEBVIEW2_USER_DATA_FOLDER`

Give each test or worker a separate folder.

This helps isolate browser data and prevents tests from interfering with each other.

6. Understanding the WebView2 test fixture

The documentation provides a custom Playwright fixture named `webView2Test.ts`.

A fixture is a reusable setup that prepares resources required by tests. Here, it starts the WebView2 application, connects Playwright and provides a `page` object.

Let's understand the important parts.

A. Importing required modules

```
import { test as base } from '@playwright/test';
import fs from 'fs';
import os from 'os';
import path from 'path';
import childProcess from 'child_process';
```

Module	Purpose
`@playwright/test`	Playwright test runner
`fs`	File and folder operations
`os`	Access operating-system information
`path`	Create file paths

`childProcess`	Start and stop the desktop application

B. Defining the application path

```
const EXECUTABLE_PATH = path.join(
  __dirname,
  '../../webview2-app/bin/Debug/net8.0-windows/webview2.exe',
);
```

This defines the location of the WebView2 executable file.

The `.exe` file is the Windows desktop application that Playwright needs to test.

C. Extending Playwright's browser fixture

```
export const test = base.extend({
  browser: async ({ playwright }, use, testInfo) => {
```

- `base.extend()` – Creates custom fixtures by extending Playwright's built-in fixtures.
- `playwright` – Provides Playwright browser types and APIs.
- `use` – Supplies the created resource to the test and waits for the test to finish.
- `testInfo` – Provides information about the current test, including its worker index.

D. Creating a unique CDP port

```
const cdpPort = 10000 + testInfo.workerIndex;
```

Suppose:

Worker	CDP port
Worker 0	10000
Worker 1	10001

Worker 2 10002

Each worker gets a different debugging port.

This prevents multiple WebView2 processes from trying to use the same port.

E. Checking whether the executable exists

```
fs.accessSync(EXECUTABLE_PATH, fs.constants.X_OK);
```

Checks whether the application executable exists and is accessible for execution.

F. Creating a separate user data directory

```
const userDataDir = path.join(
  fs.realpathSync.native(os.tmpdir()),
  `playwright-webview2-tests/user-data-dir-${testInfo.workerIndex}`,
);
```

This creates a unique folder path for each worker.

For example:

```
playwright-webview2-tests/
    user-data-dir-0/
    user-data-dir-1/
    user-data-dir-2/
```

Each worker uses its own browser data.

G. Starting the WebView2 application

```
const webView2Process = childProcess.spawn(
  EXECUTABLE_PATH,
  [],
  {
    shell: true,
    env: {
      ...process.env,
      WEBVIEW2_ADDITIONAL_BROWSER_ARGUMENTS:
        `--remote-debugging-port=${cdpPort}`,
      WEBVIEW2_USER_DATA_FOLDER: userDataDir,
    }
```

```
  }
);
```

This is one of the most important parts.

- `childProcess.spawn()` – Starts the desktop application as a separate process.
- `EXECUTABLE_PATH` – Specifies which application to start.
- `process.env` – Keeps the existing environment variables.
- `WEBVIEW2_ADDITIONAL_BROWSER_ARGUMENTS` – Enables CDP using the worker's port.
- `WEBVIEW2_USER_DATA_FOLDER` – Assigns a separate browser data folder.

H. Waiting for the application to initialize

```
await new Promise<void>(resolve =>
  webView2Process.stdout.on('data', data => {
    if (data.toString().includes('WebView2 initialized'))
      resolve();
  })
);
```

The test waits until the application prints:

```
WebView2 initialized
```

Only after receiving this message does the test continue.

This avoids connecting before WebView2 is ready.

I. Connecting to WebView2

```
const browser = await playwright.chromium.connectOverCDP(
  `http://127.0.0.1:${cdpPort}`
);
```

Playwright connects to the running WebView2 instance using the worker-specific CDP port.

`127.0.0.1` means the local computer, just like `localhost`.

J. Providing the browser to the test

```
await use(browser);
```

This makes the connected browser available to the test.

After the test finishes, execution continues to the cleanup code.

K. Closing the browser and stopping the application

```
await browser.close();

childProcess.execSync(
  `taskkill /pid ${webView2Process.pid} /T /F`
);

fs.rmdirSync(userDataDir, { recursive: true });
```

- `browser.close()` – Closes the Playwright CDP connection/browser session.
- `taskkill` – Terminates the WebView2 application process and its child processes.
- `fs.rmdirSync()` – Deletes the temporary user data directory.

This is cleanup: it prevents leftover application processes and temporary test data.

L. Creating context and page fixtures

```
context: async ({ browser }, use) => {
  const context = browser.contexts()[0];
  await use(context);
},

page: async ({ context }, use) => {
  const page = context.pages()[0];
  await use(page);
},
```

These fixtures provide the existing context and page from WebView2.

Unlike a typical Playwright test where Playwright creates a new browser context and page, this example retrieves the ones already created by WebView2.

7. Actual test example

```
import { test, expect } from './webView2Test';

test('test WebView2', async ({ page }) => {
  await page.goto('https://playwright.dev');

  const getStarted = page.getByText('Get Started');

  await expect(getStarted).toBeVisible();
});
```

Step-by-step:

1. Import the custom `test` and `expect`.
2. Start the test using the custom fixture.
3. Navigate WebView2 to the Playwright website.
4. Find the text `Get Started`.
5. Verify that the element is visible.

The important point is that we are using normal Playwright page methods and assertions to test the web content inside a desktop application.

8. Debugging WebView2

There are two ways to open developer tools.

Method 1: Manually

- Right-click inside the WebView2 application.
- Select Inspect from the context menu.

Or press F12.

Method 2: Programmatically

```
this.webView.CoreWebView2.OpenDevToolsWindow();
```

This opens the developer tools through C# code.

You can inspect HTML, CSS, JavaScript, console errors and network requests.

9. Selenium vs Playwright – WebView2

Feature	Selenium	Playwright
WebView2 connection	Requires suitable driver/browser integration	Supports CDP connection
Connection method	WebDriver-based	`connectOverCDP()`
Browser automation	WebDriver APIs	Playwright APIs
Auto-waiting	Explicit/implicit waits	Built-in auto-waiting
Assertions	TestNG/JUnit	Playwright `expect()`

Interview point: Playwright can automate WebView2 by connecting to its Edge-based browser engine through CDP.

10. Important interview questions and answers

Q1. What is WebView2?

WebView2 is a Microsoft control that allows developers to embed Microsoft Edge web content inside Windows desktop applications.

Q2. Can Playwright automate WebView2 applications?

Yes. Playwright can automate the web content inside WebView2 using the Chrome DevTools Protocol.

Q3. Which Playwright method is used to connect to WebView2?

`playwright.chromium.connectOverCDP()`

Q4. What is CDP?

CDP stands for Chrome DevTools Protocol. It allows automation tools to communicate with Chromium-based browsers and inspect or control browser functionality.

Q5. Why do we use `--remote-debugging-port=9222`?

It enables remote debugging and allows Playwright to connect to WebView2 through CDP.

Q6. What is `WEBVIEW2_USER_DATA_FOLDER`?

It specifies the folder where WebView2 stores browser data. Using a separate folder for each test worker helps prevent interference between parallel tests.

Q7. Why do we use `childProcess.spawn()`?

To launch the WebView2 desktop application as a separate process from the Playwright test.

Q8. Why do we wait for `CoreWebView2InitializationCompleted`?

To ensure that WebView2 has initialized successfully before attempting to connect and automate it.

Q9. What is `testInfo.workerIndex`?

It identifies the worker executing the test. We can use it to assign unique CDP ports and user data directories to parallel workers.

Q10. Can we use normal Playwright methods after connecting to WebView2?

Yes. We can use methods such as `page.goto()`, `getByText()`, `click()`, `fill()` and assertions such as `toBeVisible()`.

Q11. How can you open developer tools in WebView2?

By right-clicking and selecting Inspect, pressing F12, or calling `OpenDevToolsWindow()`.

Q12. What is the main challenge when running WebView2 tests in parallel?

Multiple instances can share the same user data directory or debugging port. We should assign unique ports and separate user data directories to avoid conflicts.

11. Common mistakes to avoid

1. Forgetting to enable the remote debugging port.
2. Connecting before WebView2 initialization completes.
3. Using the same CDP port for multiple workers.
4. Sharing the same user data directory across parallel tests.
5. Forgetting to close the application after test execution.
6. Assuming WebView2 is a normal browser launched by Playwright. It is an embedded browser inside a desktop application.

12. Quick revision notes

Remember these important terms

Term	Meaning
WebView2	Embedded Edge browser control
CDP	Chrome DevTools Protocol
`connectOverCDP()`	Connects Playwright to WebView2
`9222`	Example remote debugging port
`WEBVIEW2_USER_DATA_FOLDER`	Browser data directory
`childProcess.spawn()`	Starts desktop application
`testInfo.workerIndex`	Identifies parallel worker
`CoreWebView2InitializationCompleted`	Initialization event

`OpenDevToolsWindow()`	Opens developer tools

13. MCQ practice

Test your understanding. Select an answer for each question.

WebView2 knowledge check

5/5 answered

1. Which method connects Playwright to WebView2?

browser.launch()
connectOverCDP()
page.goto()
browser.newContext()

Correct! connectOverCDP() connects to WebView2 through CDP.

2. What does CDP stand for?

Chrome Development Process
Core Debugging Protocol
Chrome DevTools Protocol
Control Development Protocol

Incorrect. CDP means Chrome DevTools Protocol.

3. Why do we use WEBVIEW2_USER_DATA_FOLDER?

To change the browser engine
To isolate browser data
To open DevTools
To take screenshots

Incorrect. Separate folders prevent browser data from interfering across parallel tests.

4. Which method starts the desktop application as a separate process?

fs.accessSync()
childProcess.spawn()
page.context()
browser.close()

Incorrect. childProcess.spawn() launches the WebView2 executable.

5. What does testInfo.workerIndex help us identify?

Browser version
Test URL
Parallel worker
Page title

Incorrect. It identifies the worker and can be used to create unique ports and directories.

Score: 1/5

Try again

Final interview takeaway: WebView2 is an embedded Microsoft Edge browser control. Playwright automates its web content using `connectOverCDP()`. For reliable parallel execution, use unique debugging ports and separate user data directories.

Send me the next Playwright documentation section, and I'll explain it in the same interview-preparation format.